



OpenShift Container Platform 4.21

Security and compliance

Learning about and managing security for OpenShift Container Platform

OpenShift Container Platform 4.21 Security and compliance

Learning about and managing security for OpenShift Container Platform

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document discusses container security, configuring certificates, and enabling encryption to help secure the cluster.

Table of Contents

CHAPTER 1. OPENSIFT CONTAINER PLATFORM SECURITY AND COMPLIANCE	22
1.1. SECURITY OVERVIEW	22
1.1.1. Container security	22
1.1.2. Auditing	22
1.1.3. Certificates	22
1.1.4. Encrypting data	23
1.1.5. Vulnerability scanning	23
1.2. COMPLIANCE OVERVIEW	23
1.2.1. Compliance checking	23
1.2.2. File integrity checking	23
1.3. ADDITIONAL RESOURCES	23
CHAPTER 2. CONTAINER SECURITY	25
2.1. UNDERSTANDING CONTAINER SECURITY	25
2.1.1. What are containers?	26
2.1.2. What is OpenShift Container Platform?	26
2.2. UNDERSTANDING HOST AND VM SECURITY	27
2.2.1. Securing containers on Red Hat Enterprise Linux CoreOS (RHCOS)	27
2.2.2. Comparing virtualization and containers	29
2.2.3. Securing OpenShift Container Platform	29
2.3. HARDENING RHCOS	30
2.3.1. Choosing what to harden in RHCOS	30
2.3.2. Choosing how to harden RHCOS	31
2.3.2.1. Hardening before installation	31
2.3.2.2. Hardening during installation	31
2.3.2.3. Hardening after the cluster is running	31
2.4. CONTAINER IMAGE SIGNATURES	32
2.4.1. Enabling signature verification for Red Hat Container Registries	32
2.4.2. Verifying the signature verification configuration	35
2.4.3. Understanding the verification of container images lacking verifiable signatures	40
2.4.3.1. Automated verification during updates	40
2.4.3.2. Using skopeo to verify signatures of Red Hat container images	40
2.4.4. Additional resources	42
2.5. UNDERSTANDING COMPLIANCE	42
2.5.1. Understanding compliance and risk management	42
2.6. SECURING CONTAINER CONTENT	42
2.6.1. Securing inside the container	42
2.6.2. Creating redistributable images with UBI	43
2.6.3. Security scanning in RHEL	44
2.6.3.1. Scanning OpenShift images	44
2.6.4. Integrating external scanning	44
2.6.4.1. Image metadata	44
2.6.4.1.1. Example annotation keys	45
2.6.4.1.2. Example annotation values	46
2.6.4.2. Annotating image objects	46
2.6.4.2.1. Example annotate CLI command	46
2.6.4.3. Controlling pod execution	47
2.6.4.3.1. Example annotation	47
2.6.4.4. Integration reference	47
2.6.4.4.1. Example REST API call	47
2.7. USING CONTAINER REGISTRIES SECURELY	48

2.7.1. Knowing where containers come from	48
2.7.2. Immutable and certified containers	48
2.7.3. Getting containers from Red Hat Registry and Ecosystem Catalog	49
2.7.4. OpenShift Container Registry	49
2.7.5. Storing containers using Red Hat Quay	49
2.8. SECURING THE BUILD PROCESS	50
2.8.1. Building once, deploying everywhere	50
2.8.2. Managing builds	51
2.8.3. Securing inputs during builds	52
2.8.4. Designing your build process	53
2.8.5. Building Knative serverless applications	54
2.8.6. Additional resources	54
2.9. DEPLOYING CONTAINERS	54
2.9.1. Controlling container deployments with triggers	54
2.9.2. Controlling what image sources can be deployed	55
2.9.3. Using signature transports	57
2.9.4. Creating secrets and config maps	57
2.9.5. Automating continuous deployment	58
2.10. SECURING THE CONTAINER PLATFORM	58
2.10.1. Isolating containers with multitenancy	58
2.10.2. Protecting control plane with admission plugins	59
2.10.2.1. Security context constraints (SCCs)	59
2.10.2.2. Granting roles to service accounts	60
2.10.3. Authentication and authorization	60
2.10.3.1. Controlling access using OAuth	60
2.10.3.2. API access control and management	60
2.10.3.3. Red Hat Single Sign-On	60
2.10.3.4. Secure self-service web console	61
2.10.4. Managing certificates for the platform	61
2.10.4.1. Configuring custom certificates	61
2.11. SECURING NETWORKS	62
2.11.1. Using network namespaces	62
2.11.2. Isolating pods with network policies	62
2.11.3. Using multiple pod networks	62
2.11.4. Isolating applications	62
2.11.5. Securing ingress traffic	62
2.11.6. Securing egress traffic	63
2.12. SECURING ATTACHED STORAGE	63
2.12.1. Persistent volume plugins	63
2.12.2. Shared storage	64
2.12.3. Block storage	64
2.13. MONITORING CLUSTER EVENTS AND LOGS	64
2.13.1. Watching cluster events	65
2.13.2. Logging	66
2.13.3. Audit logs	66
CHAPTER 3. CONFIGURING CERTIFICATES	67
3.1. REPLACING THE DEFAULT INGRESS CERTIFICATE	67
3.1.1. Understanding the default ingress certificate	67
3.1.2. Replacing the default ingress certificate	67
3.1.3. Additional resources	69
3.2. ADDING API SERVER CERTIFICATES	69
3.2.1. Adding an API server named certificate for the first time	69

3.2.2. Updating or renewing an existing API server named certificate	71
3.3. SECURING SERVICE TRAFFIC USING SERVICE SERVING CERTIFICATE SECRETS	73
3.3.1. Understanding service serving certificates	73
3.3.2. Add a service certificate	73
3.3.3. Add the service CA bundle to a config map	75
3.3.4. Add the service CA bundle to an API service	76
3.3.5. Add the service CA bundle to a custom resource definition	77
3.3.6. Add the service CA bundle to a mutating webhook configuration	78
3.3.7. Add the service CA bundle to a validating webhook configuration	79
3.3.8. Manually rotate the generated service certificate	80
3.3.9. Manually rotate the service CA certificate	80
3.4. UPDATING THE CA BUNDLE	81
3.4.1. Understanding the CA Bundle certificate	82
3.4.2. Replacing the CA Bundle certificate	82
3.4.3. Additional resources	82
CHAPTER 4. CERTIFICATE TYPES AND DESCRIPTIONS	84
4.1. USER-PROVIDED CERTIFICATES FOR THE API SERVER	84
4.1.1. Purpose	84
4.1.2. Location	84
4.1.3. Management	84
4.1.4. Expiration	84
4.1.5. Customization	84
4.1.6. Additional resources	84
4.2. PROXY CERTIFICATES	84
4.2.1. Proxy certificate purpose	84
4.2.2. Managing proxy certificates during installation	85
4.2.3. Proxy certificate location	86
4.2.4. Proxy certificate expiration	86
4.2.5. Services using proxy certificates	86
4.2.6. Proxy certificate customization	86
4.2.7. Proxy certificate renewal	87
4.3. SERVICE CA CERTIFICATES	87
4.3.1. Purpose	87
4.3.2. Expiration	87
4.3.3. Management	88
4.3.4. Services	88
4.3.5. Additional resources	89
4.4. NODE CERTIFICATES	89
4.4.1. Renewing node certificates	89
4.4.2. Additional resources	90
4.5. BOOTSTRAP CERTIFICATES	90
4.5.1. Purpose	90
4.5.2. Management	90
4.5.3. Expiration	90
4.5.4. Customization	90
4.6. ETCD CERTIFICATES	91
4.6.1. etcd certificate types	91
4.6.2. Rotating the etcd certificate	91
4.6.3. Removing an unused certificate authority from the bundle	92
4.6.4. etcd certificate rotation alerts and metrics signer certificates	92
4.6.5. Additional resources	93
4.7. OLM CERTIFICATES	93

4.7.1. Management	93
4.7.2. Additional resources	93
4.8. AGGREGATED API CLIENT CERTIFICATES	93
4.8.1. Purpose	93
4.8.2. Management	93
4.8.3. Expiration	93
4.8.4. Customization	94
4.9. MACHINE CONFIG OPERATOR CERTIFICATES	94
4.9.1. Machine Config Operator certificates overview	94
4.9.1.1. Provisioning details	94
4.9.1.2. Provisioning chain of trust	95
4.9.2. Machine Config Operator certificates reference	95
4.9.2.1. Key material inside a cluster	95
4.9.2.2. Management	95
4.9.2.3. Expiration	95
4.9.2.4. Customization	96
4.10. USER-PROVIDED CERTIFICATES FOR DEFAULT INGRESS	96
4.10.1. User-provided certificates for default ingress reference	96
4.10.1.1. Purpose	96
4.10.1.2. Location	97
4.10.1.3. Management	97
4.10.1.4. Expiration	97
4.10.1.5. Services	97
4.10.1.6. Customization	97
4.10.2. Additional resources	97
4.11. INGRESS CERTIFICATES	97
4.11.1. Purpose	97
4.11.2. Ingress certificate location	97
4.11.3. Ingress certificate workflow	98
4.11.3.1. Custom certificate workflow	98
4.11.3.2. Default certificate workflow	98
4.11.4. Ingress certificate expiration	100
4.11.5. Ingress certificate services	100
4.11.6. Ingress certificate management and renewal	100
4.11.7. Additional resources	100
4.12. MONITORING AND OPENSIFT LOGGING OPERATOR COMPONENT CERTIFICATES	100
4.12.1. Expiration	101
4.12.2. Management	101
4.13. CONTROL PLANE CERTIFICATES	101
4.13.1. Location	101
4.13.2. Management	101
4.13.3. Additional resources	101
CHAPTER 5. COMPLIANCE OPERATOR	102
5.1. COMPLIANCE OPERATOR OVERVIEW	102
5.1.1. Compliance Operator concepts	102
5.1.2. Compliance Operator management	102
5.1.3. Compliance Operator scan management	102
5.1.4. Additional resources	103
5.2. RELEASE NOTES FOR THE COMPLIANCE OPERATOR	103
5.2.1. Release notes for OpenShift Compliance Operator 1.9.2	103
5.2.1.1. Fixed issues	103
5.2.2. Release notes for OpenShift Compliance Operator 1.9.1	103

5.2.2.1. Bug fixes	103
5.2.3. Release notes for OpenShift Compliance Operator 1.9.0	104
5.2.3.1. New features and enhancements	104
5.2.3.2. Bug fixes	104
5.2.4. Release notes for OpenShift Compliance Operator 1.8.2	105
5.2.4.1. Bug fixes	105
5.2.5. Release notes for OpenShift Compliance Operator 1.8.1	105
5.2.5.1. Bug fixes	105
5.2.6. Release notes for OpenShift Compliance Operator 1.8.0	105
5.2.6.1. New features and enhancements	106
5.2.6.2. Bug fixes	106
5.2.7. Release notes for OpenShift Compliance Operator 1.7.1	108
5.2.7.1. Bug fixes	108
5.2.8. Release notes for OpenShift Compliance Operator 1.7.0	108
5.2.8.1. New features and enhancements	108
5.2.8.2. Bug fixes	109
5.2.9. Release notes for OpenShift Compliance Operator 1.6.2	110
5.2.10. Release notes for OpenShift Compliance Operator 1.6.1	110
5.2.11. Release notes for OpenShift Compliance Operator 1.6.0	110
5.2.11.1. New features and enhancements	110
5.2.11.2. Bug fixes	111
5.2.12. Release notes for OpenShift Compliance Operator 1.5.1	111
5.2.13. Release notes for OpenShift Compliance Operator 1.5.0	112
5.2.13.1. New features and enhancements	112
5.2.13.2. Bug fixes	112
5.2.14. Release notes for OpenShift Compliance Operator 1.4.1	112
5.2.14.1. New features and enhancements	112
5.2.14.2. Bug fixes	112
5.2.15. Release notes for OpenShift Compliance Operator 1.4.0	113
5.2.15.1. New features and enhancements	113
5.2.15.2. Bug fixes	114
5.2.16. Release notes for OpenShift Compliance Operator 1.3.1	114
5.2.16.1. New features and enhancements	114
5.2.16.2. Known issue	115
5.2.17. Release notes for OpenShift Compliance Operator 1.3.0	115
5.2.17.1. New features and enhancements	115
5.2.18. Release notes for OpenShift Compliance Operator 1.2.0	115
5.2.18.1. New features and enhancements	115
5.2.19. Release notes for OpenShift Compliance Operator 1.1.0	116
5.2.19.1. New features and enhancements	116
5.2.19.2. Bug fixes	116
5.2.20. Release notes for OpenShift Compliance Operator 1.0.0	117
5.2.20.1. New features and enhancements	117
5.2.20.2. Bug fixes	117
5.2.21. Release notes for OpenShift Compliance Operator 0.1.61	118
5.2.21.1. New features and enhancements	118
5.2.21.2. Bug fixes	118
5.2.22. Release notes for OpenShift Compliance Operator 0.1.59	119
5.2.22.1. New features and enhancements	119
5.2.22.2. Bug fixes	119
5.2.23. Release notes for OpenShift Compliance Operator 0.1.57	119
5.2.23.1. New features and enhancements	119
5.2.23.2. Bug fixes	120

5.2.23.3. Deprecations	121
5.2.24. Release notes for OpenShift Compliance Operator 0.1.53	121
5.2.24.1. Bug fixes	121
5.2.24.2. Known issue	122
5.2.25. Release notes for OpenShift Compliance Operator 0.1.52	122
5.2.25.1. New features and enhancements	122
5.2.25.2. Bug fixes	122
5.2.25.3. Known issue	123
5.2.26. Release notes for OpenShift Compliance Operator 0.1.49	123
5.2.26.1. New features and enhancements	123
5.2.26.2. Bug fixes	123
5.2.27. Release notes for OpenShift Compliance Operator 0.1.48	124
5.2.27.1. Bug fixes	124
5.2.28. Release notes for OpenShift Compliance Operator 0.1.47	125
5.2.28.1. New features and enhancements	125
5.2.28.2. Bug fixes	125
5.2.29. Release notes for OpenShift Compliance Operator 0.1.44	125
5.2.29.1. New features and enhancements	125
5.2.29.2. Templating and variable use	126
5.2.29.3. Bug fixes	127
5.2.30. Release notes for OpenShift Compliance Operator 0.1.39	127
5.2.30.1. New features and enhancements	127
5.2.31. Additional resources	127
5.3. COMPLIANCE OPERATOR SUPPORT	128
5.3.1. Compliance Operator lifecycle	128
5.3.2. Getting support	128
5.3.3. Using the must-gather tool for the Compliance Operator	128
5.3.4. Additional resources	129
5.4. COMPLIANCE OPERATOR CONCEPTS	129
5.4.1. Understanding the Compliance Operator	129
5.4.1.1. Compliance Operator profiles	129
5.4.1.2. Compliance Operator profile types	133
5.4.1.3. Additional resources	133
5.4.2. Understanding the Custom Resource Definitions	133
5.4.2.1. Compliance Operator custom resource definition workflow	134
5.4.2.2. ProfileBundle object	134
5.4.2.3. Profile object	135
5.4.2.4. Rule object	136
5.4.2.5. TailoredProfile object	137
5.4.2.6. ScanSetting object	138
5.4.2.6.1. ScanSettingBinding object	140
5.4.2.6.2. ComplianceSuite object	141
5.4.2.6.3. Advanced ComplianceScan Object	142
5.4.2.6.4. ComplianceCheckResult object	143
5.4.2.6.5. ComplianceRemediation object	144
5.5. COMPLIANCE OPERATOR MANAGEMENT	146
5.5.1. Installing the Compliance Operator	146
5.5.1.1. Installing the Compliance Operator through the web console	146
5.5.1.2. Installing the Compliance Operator using the CLI	147
5.5.1.3. Installing the Compliance Operator on ROSA hosted control planes (HCP)	149
5.5.1.4. Installing the Compliance Operator on hosted control planes	151
5.5.1.5. Additional resources	152
5.5.2. Updating the Compliance Operator	153

5.5.2.1. About preparing for an Operator update	153
5.5.2.2. Changing the update channel for an Operator	153
5.5.2.3. Approving a pending Operator update manually	154
5.5.3. Managing the Compliance Operator	154
5.5.3.1. ProfileBundle CR example	154
5.5.3.2. Updating security content	155
5.5.3.3. Additional resources	156
5.5.4. Uninstalling the Compliance Operator	156
5.5.4.1. Uninstalling the OpenShift Compliance Operator from OpenShift Container Platform using the web console	156
5.5.4.2. Uninstalling the OpenShift Compliance Operator from OpenShift Container Platform using the CLI	157
5.5.4.3. Additional resources	158
5.6. COMPLIANCE OPERATOR SCAN MANAGEMENT	158
5.6.1. Supported compliance profiles	158
5.6.1.1. Compliance profiles	159
5.6.1.1.1. CIS compliance profiles	159
5.6.1.1.2. BSI Profile Support	160
5.6.1.1.3. Essential Eight compliance profiles	162
5.6.1.1.4. FedRAMP High compliance profiles	162
5.6.1.1.5. FedRAMP Moderate compliance profiles	163
5.6.1.1.6. NERC-CIP compliance profiles	165
5.6.1.1.7. PCI-DSS compliance profiles	166
5.6.1.1.8. STIG compliance profiles	168
5.6.1.1.9. About extended compliance profiles	169
5.6.1.1.10. Compliance Operator profile types	170
5.6.1.2. Additional resources	171
5.6.2. Compliance Operator scans	171
5.6.2.1. Running compliance scans	171
5.6.2.2. Setting custom storage size for results	175
5.6.2.3. Scheduling the result server pod on a worker node	176
5.6.2.4. ScanSetting Custom Resource	177
5.6.2.5. Configuring the hosted control planes management cluster	178
5.6.2.6. Applying resource requests and limits	179
5.6.2.7. Scheduling Pods with container resource requests	180
5.6.2.8. Additional resources	181
5.6.3. Propagating custom metadata to ComplianceCheckResult objects	181
5.6.3.1. Understanding custom metadata on rules and check results	181
5.6.3.2. Adding custom labels and annotations to a Rule object	182
5.6.3.3. Adding custom labels and annotations to a CustomRule object	183
5.6.4. Tailoring the Compliance Operator	185
5.6.4.1. Creating a new tailored profile	185
5.6.4.2. Using tailored profiles to extend existing ProfileBundles	186
5.6.5. Retrieving Compliance Operator raw results	189
5.6.5.1. Obtaining Compliance Operator raw results from a persistent volume	189
5.6.6. Managing Compliance Operator result and remediation	190
5.6.6.1. Filters for compliance check results	191
5.6.6.2. Reviewing a remediation	192
5.6.6.3. Applying remediation when using customized machine config pools	194
5.6.6.4. Evaluating KubeletConfig rules against default configuration values	195
5.6.6.5. Scanning custom node pools	195
5.6.6.6. Remediating KubeletConfig sub pools	196
5.6.6.7. Applying a remediation	197

5.6.6.8. Remediating a platform check manually	197
5.6.6.9. Updating remediations	198
5.6.6.10. Unapplying a remediation	200
5.6.6.11. Removing a KubeletConfig remediation	200
5.6.6.12. Inconsistent ComplianceScan	202
5.6.6.13. Additional resources	203
5.6.7. Performing advanced Compliance Operator tasks	203
5.6.7.1. Using the ComplianceSuite and ComplianceScan objects directly	203
5.6.7.2. Setting PriorityClass for ScanSetting scans	204
5.6.7.3. Using raw tailored profiles	205
5.6.7.4. Performing a rescan	205
5.6.7.5. Setting custom storage size for results	206
5.6.7.6. Applying remediations generated by suite scans	207
5.6.7.7. Automatically update remediations	207
5.6.7.8. Creating a custom SCC for the Compliance Operator	208
5.6.7.9. Additional resources	209
5.6.8. Troubleshooting Compliance Operator scans	209
5.6.8.1. Anatomy of a scan	210
5.6.8.1.1. Compliance sources	210
5.6.8.1.2. The ScanSetting and ScanSettingBinding objects lifecycle and debugging	211
5.6.8.1.3. ComplianceSuite custom resource lifecycle and debugging	211
5.6.8.1.4. ComplianceScan custom resource lifecycle and debugging	212
5.6.8.1.5. Pending phase	212
5.6.8.1.6. Launching phase	212
5.6.8.1.7. Running phase	213
5.6.8.1.8. Aggregating phase	214
5.6.8.1.9. Done phase	214
5.6.8.1.10. ComplianceRemediation controller lifecycle and debugging	215
5.6.8.1.11. Useful labels	216
5.6.8.2. Increasing Compliance Operator resource limits	216
5.6.8.3. Configuring Operator resource constraints	217
5.6.8.4. Configuring ScanSetting resources	217
5.6.8.5. Configuring ScanSetting timeout	219
5.6.8.6. Getting support	220
5.6.9. Using the oc-compliance plugin	220
5.6.9.1. Installing the oc-compliance plugin	220
5.6.9.2. Fetching raw results	220
5.6.9.3. Re-running scans	222
5.6.9.4. Using ScanSettingBinding custom resources	222
5.6.9.5. Printing controls	224
5.6.9.6. Fetching compliance remediation details	224
5.6.9.7. Viewing ComplianceCheckResult object details	226
5.6.9.8. Additional resources	226
CHAPTER 6. FILE INTEGRITY OPERATOR	227
6.1. FILE INTEGRITY OPERATOR OVERVIEW	227
6.1.1. Additional resources	227
6.2. RELEASE NOTES FOR THE FILE INTEGRITY OPERATOR	227
6.2.1. Release notes for OpenShift File Integrity Operator 1.4.1	227
6.2.1.1. New features and enhancements	227
6.2.2. Release notes for OpenShift File Integrity Operator 1.4.0	228
6.2.2.1. New features and enhancements	228
6.2.2.2. Bug fixes	228

6.2.3. Release notes for OpenShift File Integrity Operator 1.3.8	228
6.2.3.1. Bug fixes	228
6.2.4. Release notes for OpenShift File Integrity Operator 1.3.7	228
6.2.5. Release notes for OpenShift File Integrity Operator 1.3.6	229
6.2.5.1. Bug fixes	229
6.2.6. Release notes for OpenShift File Integrity Operator 1.3.5	229
6.2.7. Release notes for OpenShift File Integrity Operator 1.3.4	229
6.2.7.1. Bug fixes	230
6.2.8. Release notes for OpenShift File Integrity Operator 1.3.3	230
6.2.8.1. New features and enhancements	230
6.2.8.2. Bug fixes	230
6.2.9. Release notes for OpenShift File Integrity Operator 1.3.2	230
6.2.10. Release notes for OpenShift File Integrity Operator 1.3.1	231
6.2.10.1. New features and enhancements	231
6.2.10.2. Bug fixes	231
6.2.10.3. Known Issues	231
6.2.11. Release notes for OpenShift File Integrity Operator 1.2.1	231
6.2.12. Release notes for OpenShift File Integrity Operator 1.2.0	231
6.2.12.1. New features and enhancements	232
6.2.13. Release notes for OpenShift File Integrity Operator 1.0.0	232
6.2.14. Release notes for OpenShift File Integrity Operator 0.1.32	232
6.2.14.1. Bug fixes	232
6.2.15. Release notes for OpenShift File Integrity Operator 0.1.30	232
6.2.15.1. New features and enhancements	232
6.2.15.2. Bug fixes	233
6.2.16. Release notes for OpenShift File Integrity Operator 0.1.24	233
6.2.16.1. New features and enhancements	233
6.2.16.2. Bug fixes	233
6.2.17. Release notes for OpenShift File Integrity Operator 0.1.22	233
6.2.17.1. Bug fixes	233
6.2.18. Release notes for OpenShift File Integrity Operator 0.1.21	234
6.2.18.1. New features and enhancements	234
6.2.18.2. Bug fixes	234
6.2.19. Additional resources	234
6.3. FILE INTEGRITY OPERATOR SUPPORT	234
6.3.1. File Integrity Operator lifecycle	234
6.3.2. Getting support	235
6.3.3. Additional resources	235
6.4. INSTALLING THE FILE INTEGRITY OPERATOR	235
6.4.1. Installing the File Integrity Operator using the web console	235
6.4.2. Installing the File Integrity Operator using the CLI	236
6.4.3. Additional resources	237
6.5. UPDATING THE FILE INTEGRITY OPERATOR	237
6.5.1. About preparing for an Operator update	237
6.5.2. Changing the update channel for an Operator	238
6.5.3. Approving a pending Operator update manually	238
6.6. UNDERSTANDING THE FILE INTEGRITY OPERATOR	239
6.6.1. Creating the FileIntegrity custom resource	239
6.6.2. Checking the FileIntegrity custom resource status	241
6.6.3. FileIntegrity custom resource phases	241
6.6.4. Understanding the FileIntegrityNodeStatuses object	241
6.6.5. FileIntegrityNodeStatus CR status types	242
6.6.5.1. FileIntegrityNodeStatus CR success example	243

6.6.5.2. FileIntegrityNodeStatus CR failure status example	243
6.6.6. Understanding events	245
6.7. CONFIGURING THE CUSTOM FILE INTEGRITY OPERATOR	246
6.7.1. Viewing FileIntegrity object attributes	246
6.7.2. Important attributes	247
6.7.3. Examine the default configuration	248
6.7.4. Understanding the default File Integrity Operator configuration	248
6.7.5. Supplying a custom AIDE configuration	249
6.7.6. Defining a custom File Integrity Operator configuration	249
6.7.7. Changing the custom File Integrity configuration	250
6.8. PERFORMING ADVANCED CUSTOM FILE INTEGRITY OPERATOR TASKS	251
6.8.1. Reinitializing the database	251
6.8.2. Machine config integration	251
6.8.3. Exploring the daemon sets	252
6.9. TROUBLESHOOTING THE FILE INTEGRITY OPERATOR	252
6.9.1. General troubleshooting	252
6.9.2. Checking the AIDE configuration	252
6.9.3. Determining the FileIntegrity object phase	253
6.9.4. Determining that the daemon set pods are running on the expected nodes	253
6.10. UNINSTALLING THE FILE INTEGRITY OPERATOR	253
6.10.1. Uninstall the File Integrity Operator using the web console	253
CHAPTER 7. SECURITY PROFILES OPERATOR	255
7.1. SECURITY PROFILES OPERATOR OVERVIEW	255
7.1.1. Additional resources	255
7.2. RELEASE NOTES FOR THE SECURITY PROFILES OPERATOR	255
7.2.1. Release notes for Security Profiles Operator 0.10.0	255
7.2.1.1. Bug fixes	255
7.2.1.2. New features and enhancements	256
7.2.2. Release notes for Security Profiles Operator 0.9.0	256
7.2.2.1. Bug fixes	256
7.2.3. Release notes for Security Profiles Operator 0.8.6	256
7.2.4. Release notes for Security Profiles Operator 0.8.5	257
7.2.4.1. Bug fixes	257
7.2.5. Release notes for Security Profiles Operator 0.8.4	257
7.2.5.1. New features and enhancements	257
7.2.6. Release notes for Security Profiles Operator 0.8.2	257
7.2.6.1. Bug fixes	257
7.2.7. Release notes for Security Profiles Operator 0.8.0	258
7.2.7.1. Bug fixes	258
7.2.8. Release notes for Security Profiles Operator 0.7.1	258
7.2.8.1. New features and enhancements	258
7.2.8.2. Deprecated and removed features	259
7.2.8.3. Bug fixes	259
7.2.8.4. Known issue	259
7.2.9. Release notes for Security Profiles Operator 0.5.2	259
7.2.9.1. Known issue	259
7.2.10. Release notes for Security Profiles Operator 0.5.0	259
7.2.10.1. Known issue	260
7.2.11. Additional resources	260
7.3. SECURITY PROFILES OPERATOR SUPPORT	260
7.3.1. Getting support	260
7.3.2. Additional resources	260

7.4. UNDERSTANDING THE SECURITY PROFILES OPERATOR	260
7.4.1. About security profiles	261
7.5. ENABLING THE SECURITY PROFILES OPERATOR	261
7.5.1. Installing the Security Profiles Operator	261
7.5.2. Installing the Security Profiles Operator using the CLI	262
7.5.3. Configuring logging verbosity	263
7.6. MANAGING SECCOMP PROFILES	264
7.6.1. Creating seccomp profiles	264
7.6.2. Apply seccomp profiles to a pod	264
7.6.2.1. Binding workloads to profiles with ProfileBindings	266
7.6.3. Record profiles from workloads	267
7.6.3.1. Merge per-container profile instances	269
7.6.4. Additional resources	271
7.7. MANAGING SELINUX PROFILES	271
7.7.1. Creating SELinux profiles	271
7.7.2. Apply SELinux profiles to a pod	273
7.7.2.1. Apply SELinux log policies	274
7.7.2.2. Binding workloads to profiles with ProfileBindings	274
7.7.2.3. Replicate controllers and SecurityContextConstraints	276
7.7.3. Record profiles from workloads	277
7.7.3.1. Merge per-container profile instances	279
7.7.3.2. About seLinuxContext: RunAsAny	281
7.7.4. Additional resources	281
7.8. ADVANCED SECURITY PROFILES OPERATOR TASKS	281
7.8.1. Restrict the allowed syscalls in seccomp profiles	282
7.8.2. Base syscalls for a container runtime	282
7.8.3. Enabling memory optimization in the spod daemon	282
7.8.4. Customizing daemon resource requirements	283
7.8.5. Setting a custom priority class name for the spod daemon pod	283
7.8.6. Using metrics	284
7.8.6.1. controller-runtime metrics	285
7.8.7. Use the log enricher	286
7.8.7.1. Using the log enricher to trace an application	287
7.8.8. Configure webhooks	288
7.9. ADVANCED AUDIT LOGGING FRAMEWORK	289
7.9.1. Benefits of the Advanced Audit Logging Framework	289
7.9.2. Performance considerations	290
7.9.3. Prerequisites for the Advanced Audit Logging Framework	290
7.9.4. The Audit JSON log enricher	291
7.9.5. Kubernetes API log compared to Advanced Audit Logging output	291
7.9.6. Enabling Advanced Audit Logging	292
7.9.7. Audit JSON Log Enricher configuration	293
7.9.8. Audit log file fine-tuning and rotation	294
7.9.9. Advanced audit logs for a specific pod	295
7.9.10. Monitor the audit logs	298
7.9.11. Audit node debugging sessions	299
7.9.12. Correlate with Kubernetes audit logs	301
7.9.13. Audit JSON Log Enricher output	301
7.9.14. Kubernetes API audit log output	302
7.9.15. Correlation key	303
7.9.16. Correlating with API Server Audit Log	303
7.9.17. Use the mutating webhook	304
7.9.17.1. The debug pod	304

7.9.18. Disabling Advanced Audit Logging	304
7.9.19. Additional resources	305
7.10. TROUBLESHOOTING THE SECURITY PROFILES OPERATOR	306
7.10.1. Inspecting seccomp profiles	306
7.11. UNINSTALLING THE SECURITY PROFILES OPERATOR	307
7.11.1. Uninstall the Security Profiles Operator by using the web console	307
CHAPTER 8. NBDE TANG SERVER OPERATOR	308
8.1. NBDE TANG SERVER OPERATOR OVERVIEW	308
8.2. NBDE TANG SERVER OPERATOR RELEASE NOTES	308
8.3. UNDERSTANDING THE NBDE TANG SERVER OPERATOR	308
8.3.1. Additional resources	309
8.4. INSTALLING THE NBDE TANG SERVER OPERATOR	309
8.4.1. Installing the NBDE Tang Server Operator using the web console	309
8.4.2. Installing the NBDE Tang Server Operator using CLI	310
8.5. CONFIGURING AND MANAGING TANG SERVERS USING THE NBDE TANG SERVER OPERATOR	311
8.5.1. Deploying a Tang server using the NBDE Tang Server Operator	311
8.5.2. Rotating keys using the NBDE Tang Server Operator	317
8.5.3. Deleting hidden keys with the NBDE Tang Server Operator	318
8.6. IDENTIFYING URL OF A TANG SERVER DEPLOYED WITH THE NBDE TANG SERVER OPERATOR	320
8.6.1. Identifying URL of the NBDE Tang Server Operator using the web console	320
8.6.2. Identifying URL of the NBDE Tang Server Operator using CLI	322
8.6.3. Additional resources	323
CHAPTER 9. UNDERSTANDING SECRETS MANAGEMENT IN OPENSIFT CONTAINER PLATFORM	324
9.1. SECRETS MANAGEMENT OPERATORS IN OPENSIFT CONTAINER PLATFORM	324
9.2. SECRETS MANAGEMENT USE CASES	324
9.2.1. External Secrets Operator for Red Hat OpenShift use cases	324
CHAPTER 10. CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	326
10.1. CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT OVERVIEW	326
10.1.1. About the cert-manager Operator for Red Hat OpenShift	326
10.1.2. cert-manager Operator for Red Hat OpenShift issuer providers	326
10.1.3. Certificate request methods	327
10.1.4. Supported cert-manager Operator for Red Hat OpenShift versions	327
10.1.5. About FIPS compliance for cert-manager Operator for Red Hat OpenShift	327
10.1.6. Additional resources	327
10.2. CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT RELEASE NOTES	328
10.2.1. cert-manager Operator for Red Hat OpenShift 1.20.0	328
10.2.1.1. New features and enhancements	328
10.2.1.2. Fixed issues	331
10.2.2. cert-manager Operator for Red Hat OpenShift 1.19.1	331
10.2.2.1. Fixed issues	331
10.2.2.2. CVEs	331
10.2.3. cert-manager Operator for Red Hat OpenShift 1.19.0	332
10.2.3.1. New features and enhancements	332
10.2.3.2. Fixed issues	332
10.3. INSTALLING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	333
10.3.1. Installing the cert-manager Operator for Red Hat OpenShift by using the web console	333
10.3.2. Installing the cert-manager Operator for Red Hat OpenShift by using the CLI	334
10.3.3. Understanding update channels of the cert-manager Operator for Red Hat OpenShift	337
10.3.3.1. stable-v1 channel	337
10.3.3.2. stable-v1.y channel	337
10.3.4. Additional resources	338

10.4. CONFIGURING THE EGRESS PROXY FOR THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	338
10.4.1. Injecting a custom CA certificate for the cert-manager Operator for Red Hat OpenShift	338
10.4.2. Additional resources	339
10.5. CUSTOMIZING THE CERT-MANAGER OPERATOR BY USING THE CERTMANAGER CUSTOM RESOURCE	339
10.5.1. Explanation of fields in the CertManager custom resource	341
10.5.1.1. Common configurable fields in the CertManager CR for the cert-manager components	341
10.5.1.2. Overridable arguments for the cert-manager components	342
10.5.1.3. Overridable environment variables for the cert-manager controller	345
10.5.1.4. Overridable resource parameters for the cert-manager components	345
10.5.1.5. Overridable scheduling parameters for the cert-manager components	345
10.5.2. Customizing cert-manager by overriding environment variables from the cert-manager Operator API	346
10.5.3. Customizing cert-manager by overriding arguments from the cert-manager Operator API	348
10.5.4. Deleting a TLS secret automatically upon Certificate removal	349
10.5.5. Overriding CPU and memory limits for the cert-manager components	351
10.5.6. Configuring scheduling overrides for cert-manager components	354
10.5.7. Configuring cluster TLS security profile adherence for cert-manager components	356
10.5.8. Verifying TLS security profile adherence for cert-manager components	357
10.6. AUTHENTICATING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	359
10.6.1. Authenticating on AWS	359
10.6.2. Authenticating with AWS Security Token Service	361
10.6.3. Authenticating on Google Cloud	363
10.6.4. Authenticating with Google Cloud Workload Identity	365
10.7. CONFIGURING AN ACME ISSUER	367
10.7.1. About ACME issuers	368
10.7.1.1. Supported ACME challenges types	368
10.7.1.2. Supported DNS-01 providers	369
10.7.2. Configuring an ACME issuer to solve HTTP-01 challenges	369
10.7.3. Configuring an ACME issuer by using explicit credentials for AWS Route53	372
10.7.4. Configuring an ACME issuer by using ambient credentials on AWS	374
10.7.5. Configuring an ACME issuer by using explicit credentials for Google Cloud DNS	376
10.7.6. Configuring an ACME issuer by using ambient credentials on Google Cloud	378
10.7.7. Configuring an ACME issuer by using explicit credentials for Microsoft Azure DNS	380
10.7.8. Additional resources	382
10.8. CONFIGURING CERTIFICATES WITH AN ISSUER	383
10.8.1. Creating certificates for user workloads	383
10.8.2. Creating certificates for the API server	384
10.8.3. Creating certificates for the Ingress Controller	386
10.8.4. Additional resources	387
10.9. SECURING ROUTES WITH THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	388
10.9.1. Configuring certificates to secure routes in your cluster	388
10.9.2. Additional resources	391
10.10. INTEGRATING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT WITH ISTIO-CSR	391
10.10.1. Creating a root CA issuer for the Istio-CSR agent	392
10.10.2. Creating the IstioCSR custom resource	393
10.10.3. Setting the log level for the istio-csr component	395
10.10.4. Configuring the namespace selector for CA bundle distribution	396
10.10.5. Configuring the CA certificate for the Istio server	397
10.10.6. Uninstalling the Istio-CSR agent managed by cert-manager Operator for Red Hat OpenShift	398
10.11. NETWORK POLICY CONFIGURATION FOR CERT-MANAGER OPERATOR	399
10.11.1. Default ingress and egress rules	400

10.11.2. Network policy configuration parameters	400
10.11.3. Network policy configuration examples	401
10.11.4. Verifying the network policy creation	402
10.12. DISTRIBUTING CERTIFICATES BY USING TRUST-MANAGER OPERAND	403
10.12.1. Installing the trust-manager operand	403
10.12.2. Configuring trust bundle	406
10.12.3. Uninstalling the trust-manager operand	408
10.12.4. Trust manager custom resource fields	409
10.13. MONITORING CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	411
10.13.1. Enabling user workload monitoring	411
10.13.2. Configuring metrics collection for cert-manager Operator for Red Hat OpenShift operands by using a ServiceMonitor	412
10.13.3. Querying metrics for the cert-manager Operator for Red Hat OpenShift operands	414
10.13.4. Configuring metrics collection for the istio-csr operand	415
10.13.5. Querying metrics for the istio-csr operand	416
10.14. CONFIGURING LOG LEVELS FOR CERT-MANAGER AND THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	417
10.14.1. Setting a log level for cert-manager	417
10.14.2. Setting a log level for the cert-manager Operator for Red Hat OpenShift	418
10.14.3. Additional resources	419
10.15. UNINSTALLING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT	419
10.15.1. Uninstalling the cert-manager Operator for Red Hat OpenShift	419
10.15.2. Removing cert-manager Operator for Red Hat OpenShift resources	419
CHAPTER 11. ZERO TRUST WORKLOAD IDENTITY MANAGER	422
11.1. ZERO TRUST WORKLOAD IDENTITY MANAGER OVERVIEW	422
11.1.1. SPIFFE	422
11.1.2. SPIRE Server	422
11.1.3. SPIRE Agent	422
11.1.4. Attestation	422
11.2. ZERO TRUST WORKLOAD IDENTITY MANAGER COMPONENTS	423
11.2.1. SPIFFE CSI Driver	423
11.2.2. SPIRE OpenID Connect Discovery Provider	423
11.2.3. SPIRE Controller Manager	423
11.2.4. SPIRE Server and Agent telemetry	423
11.2.5. About the Zero Trust Workload Identity Manager workflow	423
11.3. ZERO TRUST WORKLOAD IDENTITY MANAGER RELEASE NOTES	425
11.3.1. Zero Trust Workload Identity Manager 1.1.0	425
11.3.1.1. New features and enhancements	426
11.3.1.2. Deprecated features	426
11.3.1.3. Fixed issues	427
11.3.2. Zero Trust Workload Identity Manager 1.0.1	428
11.3.2.1. CVEs	429
11.3.3. Zero Trust Workload Identity Manager 1.0.0 (General Availability)	429
11.3.3.1. New features and enhancements	430
11.3.3.2. Status and observability improvements	432
11.3.3.3. Fixed issues	432
11.3.4. Zero Trust Workload Identity Manager 0.2.0 (General Availability)	434
11.3.4.1. New features and enhancements	434
11.3.4.2. Fixed issues	435
11.3.5. Zero Trust Workload Identity Manager 0.1.0 (General Availability)	435
11.4. INSTALLING THE ZERO TRUST WORKLOAD IDENTITY MANAGER	435
11.4.1. Installing the Zero Trust Workload Identity Manager by using the web console	436

11.4.2. Installing the Zero Trust Workload Identity Manager by using the CLI	437
11.5. DEPLOYING ZERO TRUST WORKLOAD IDENTITY MANAGER OPERANDS	439
11.5.1. About the ZeroTrustWorkloadIdentityManager custom resource	439
11.5.2. Deploying the SPIRE Server	440
11.5.3. Deploying the SPIRE Agent	443
11.5.4. Deploying the SPIFFE Container Storage Interface driver	445
11.5.5. Deploying the SPIRE OpenID Connect Discovery Provider	447
11.5.6. Verify the health of the operands	448
11.5.7. Manually delete the custom security context constraints	449
11.6. CONFIGURING THE EGRESS PROXY FOR THE ZERO TRUST WORKLOAD IDENTITY MANAGER	450
11.6.1. Injecting a custom CA certificate for the Zero Trust Workload Identity Manager	450
11.6.2. Additional resources	453
11.7. ZERO TRUST WORKLOAD IDENTITY MANAGER OIDC FEDERATION	453
11.7.1. About the Entra ID OpenID Connect	453
11.7.1.1. Configuring the external certificate for the managed OIDC discovery provider route	453
11.7.1.2. Disabling a managed route	454
11.7.1.3. Using Entra ID with Microsoft Azure	455
11.7.1.4. Configuring Azure blob storage	456
11.7.1.5. Configuring an Azure user managed identity	457
11.7.1.6. Creating the demonstration application	457
11.7.1.7. Deploying the workload application	458
11.7.1.8. Configuring Azure with the SPIFFE identity federation	460
11.7.1.9. Verifying that the application workload can access the content in the Azure Blob Storage	460
11.7.2. About Vault OpenID Connect	461
11.7.2.1. Installing Vault	461
11.7.2.2. Initializing and unsealing Vault	463
11.7.2.3. Enabling the key-value secrets engine and store a test secret	464
11.7.2.4. Configuring JSON Web Token authentication with SPIRE	465
11.7.2.5. Deploying a demonstration application	467
11.7.2.6. Authenticating and retrieving the secret	468
11.8. ZERO TRUST WORKLOAD IDENTITY MANAGER SPIRE FEDERATION	468
11.8.1. Understanding bundle endpoint profiles	469
11.8.2. Federation configuration examples	470
11.8.3. Configuring SPIRE federation with the https_spiffe profile	472
11.8.4. Using SPIRE federation with Automatic Certificate Management Environment protocol	476
11.8.5. Using SPIRE federation with manual certificate management	481
11.8.6. Federation configuration field reference	489
11.9. USING THE SPIFFE HELPER CONTAINER IMAGE	493
11.9.1. SPIFFE Helper container image	493
11.9.2. SPIFFE Helper modes	493
11.9.3. SPIFFE Helper credential types	495
11.9.4. Deploying PostgreSQL with SPIFFE Helper	495
11.9.5. SPIFFE Helper image reference	504
11.9.5.1. Image location	504
11.9.5.2. Command-line flags	504
11.9.5.3. Configuration file options	504
11.9.5.4. Volume and mount requirements	505
11.9.5.5. Quick reference	507
11.9.6. Troubleshooting SPIFFE Helper deployments	507
11.9.6.1. Common issues	508
11.9.6.2. Verify Zero Trust Workload Identity Manager readiness	509
11.9.6.3. Inspect SPIFFE Helper pods	509
11.9.6.4. Review SPIFFE Helper logs	509

11.9.6.5. Verify on-disk SVID certificates	509
11.10. INTEGRATING RED HAT OPENSIFT SERVICE MESH WITH ZERO TRUST WORKLOAD IDENTITY MANAGER IN A SINGLE-CLUSTER	509
11.10.1. SPIRE integration with Red Hat OpenShift Service Mesh	509
11.10.1.1. Component overview	510
11.10.2. SPIRE integration architecture components	510
11.10.3. Deploying SPIRE operands for Red Hat OpenShift Service Mesh integration	511
11.10.4. Deploying Red Hat OpenShift Service Mesh for SPIRE integration	515
11.10.5. Additional resources	519
11.11. INTEGRATING SPIRE FEDERATION WITH MULTI-CLUSTER RED HAT OPENSIFT SERVICE MESH	519
11.11.1. Multi-cluster SPIFFE Runtime Environment integration with Red Hat OpenShift Service Mesh	519
11.11.1.1. What gets federated	520
11.11.2. Preparing the environment for multi-cluster SPIFFE Runtime Environment federation	520
11.11.3. Deploying SPIFFE Runtime Environment with federation on both clusters	521
11.11.4. Additional resources	527
11.11.5. Configuring Red Hat OpenShift Service Mesh for multi-cluster SPIFFE Runtime Environment integration	527
11.11.6. Deploying the Red Hat OpenShift Service Mesh CNI on both clusters	530
11.11.7. Deploying the Istio custom resource with the federation configuration	533
11.11.8. Verifying SPIRE integration with Istio on each cluster	537
11.11.9. Verifying workload mTLS with SPIRE-issued identities on each cluster	541
11.11.10. Deploying east-west gateways	549
11.11.11. Exchanging remote secrets	552
11.11.12. Verifying cross-cluster service communication	553
11.11.13. Additional resources	557
11.12. ENABLING CREATE-ONLY MODE FOR THE ZERO TRUST WORKLOAD IDENTITY MANAGER	557
11.12.1. Pausing Operator reconciliation	557
11.12.2. Resuming Operator reconciliation	559
11.13. SPIRE UPSTREAMAUTHORITY PLUGINS FOR ZERO TRUST WORKLOAD IDENTITY MANAGER	559
11.13.1. About the cert-manager upstream authority plugin	560
11.13.1.1. How the cert-manager plugin works	560
11.13.1.2. Requirements	560
11.13.2. Preparing cert-manager for SPIRE Server	560
11.13.3. Configuring the cert-manager upstream authority plugin	562
11.13.4. cert-manager upstream authority plugin reference	564
11.13.4.1. SpireServer CR fields	564
11.13.5. Troubleshooting the cert-manager Operator for Red Hat OpenShift upstream authority plugin	565
11.13.5.1. Quick reference	565
11.13.5.2. Permission and issuer errors	565
11.13.5.3. SPIRE Server errors	566
11.13.6. About the SPIRE Vault UpstreamAuthority plugin	566
11.13.6.1. Supported authentication method	566
11.13.6.2. Prerequisites and requirements	567
11.13.7. Configuring the SPIRE Vault UpstreamAuthority plugin	567
11.13.8. SPIRE Vault UpstreamAuthority plugin reference	570
11.13.8.1. SpireServer CR fields	570
11.13.8.2. Required Vault policy	571
11.13.9. Troubleshooting SPIRE Vault UpstreamAuthority plugin	571
11.13.9.1. Quick reference	571
11.13.9.2. Connection and TLS errors	572
11.13.9.3. Authentication and signing errors	572
11.13.9.4. TTL mismatch errors	572
11.14. MONITORING ZERO TRUST WORKLOAD IDENTITY MANAGERGIT	573

11.14.1. Enabling user workload monitoring	573
11.14.2. Configuring metrics collection for SPIRE Server by using a ServiceMonitor	574
11.14.3. Configuring metrics collection for SPIRE Agent by using a Service Monitor	575
11.14.4. Configuring metrics collection for the Operator by using a ServiceMonitor	576
11.14.5. Querying metrics for the Zero Trust Workload Identity Manager	580
11.14.6. Zero Trust Workload Identity Manager monitoring available metrics	580
11.15. UNINSTALLING THE ZERO TRUST WORKLOAD IDENTITY MANAGER	581
11.15.1. Uninstalling the Zero Trust Workload Identity Manager	581
11.15.2. Uninstalling Zero Trust Workload Identity Manager resources by using the CLI	582
CHAPTER 12. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	585
12.1. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT OVERVIEW	585
12.1.1. About the External Secrets Operator for Red Hat OpenShift	585
12.1.2. External secrets providers for the External Secrets Operator for Red Hat OpenShift	585
12.1.3. About FIPS compliance for External Secrets Operator for Red Hat OpenShift	586
12.1.4. Security considerations	586
12.1.5. Additional resources	586
12.2. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT RELEASE NOTES	587
12.2.1. Release notes for External Secrets Operator for Red Hat OpenShift 1.2.0	587
12.2.1.1. New features and enhancements	587
12.2.1.2. Fixed issues	589
12.2.2. Release notes for External Secrets Operator for Red Hat OpenShift 1.1.1	589
12.2.2.1. CVEs	590
12.2.2.2. New features and enhancements	590
12.2.3. Release notes for External Secrets Operator for Red Hat OpenShift 1.1.0	590
12.2.3.1. New features and enhancements	590
12.2.4. Release notes for External Secrets Operator for Red Hat OpenShift 1.0.1	591
12.2.4.1. CVEs	591
12.2.5. Release notes for External Secrets Operator for Red Hat OpenShift 1.0.0 (General Availability)	591
12.2.5.1. Fixed issues	591
12.2.5.2. New features and enhancements	591
12.2.6. Release notes for External Secrets Operator for Red Hat OpenShift 0.1.0 (Technology Preview)	592
12.2.6.1. New features and enhancements	593
12.3. INSTALLING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	593
12.3.1. Limitations of External Secrets Operator for Red Hat OpenShift	593
12.3.2. Installing the External Secrets Operator for Red Hat OpenShift by using the web console	593
12.3.3. Installing the External Secrets Operator for Red Hat OpenShift by using the CLI	594
12.3.4. Additional resources	596
12.3.5. Installing the External Secrets operand by using the CLI	596
12.3.6. Understanding update channels of the External Secrets Operator for Red Hat OpenShift	598
12.3.6.1. About the External Secrets Operator for Red Hat OpenShift stable-v1 channel	598
12.3.6.2. About the External Secrets Operator for Red Hat OpenShift stable-v1.y channel	598
12.3.7. Additional resources	599
12.4. CONFIGURING NETWORK POLICY FOR THE OPERAND	599
12.4.1. Adding a custom network policy to allow egress to all external providers	599
12.4.2. Adding a custom network policy to allow egress to a specific provider	600
12.4.3. Default ingress and egress rules	600
12.5. ABOUT THE EGRESS PROXY FOR THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	601
12.5.1. Configuring the egress proxy for the External Secrets Operator for Red Hat OpenShift	601
12.5.2. Additional resources	604
12.6. MONITORING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	604
12.6.1. Enabling user workload monitoring	604

12.6.2. Configuring metrics collection for External Secrets Operator for Red Hat OpenShift by using a ServiceMonitor	605
12.6.3. Querying metrics for the External Secrets Operator for Red Hat OpenShift	608
12.6.4. Configuring metrics collection for External Secrets Operator for Red Hat OpenShift operands by using a ServiceMonitor	609
12.6.5. Querying metrics for the external-secrets operand	611
12.7. CUSTOMIZING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	611
12.7.1. Setting a log level for the External Secrets Operator for Red Hat OpenShift	612
12.7.2. Setting a log level for the External Secrets Operator for Red Hat OpenShift operand	613
12.7.3. Configuring cert-manager for the external-secrets certificate requirements	614
12.7.4. Configuring the bitwardenSecretManagerProvider plugin	615
12.7.5. Adding custom annotations to external-secrets resources	616
12.7.6. Configuring the revisionHistoryLimit for external-secrets components	618
12.7.7. Setting custom environment variables for external-secrets components	619
12.7.8. Enabling optional features for External Secrets Operator for Red Hat OpenShift	620
12.7.9. Mounting a custom trusted certificate authority bundle for external-secrets	622
12.7.10. Overriding operand container arguments for the External Secrets Operator for Red Hat OpenShift	624
12.8. UNINSTALLING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	626
12.8.1. Uninstalling the External Secrets Operator for Red Hat OpenShift using the web console	626
12.8.2. Removing External Secrets Operator for Red Hat OpenShift resources by using the web console	627
12.8.3. Removing External Secrets Operator for Red Hat OpenShift resources by using the CLI	628
12.9. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT APIS	629
12.9.1. applicationConfig	629
12.9.2. bitwardenSecretManagerProvider	631
12.9.3. certManagerConfig	632
12.9.4. certProvidersConfig	635
12.9.5. commonConfigs	635
12.9.6. componentConfig	636
12.9.7. componentName	637
12.9.8. condition	638
12.9.9. conditionalStatus	638
12.9.10. configMapKeyReference	639
12.9.11. controllerConfig	639
12.9.12. controllerStatus	642
12.9.13. deploymentConfig	643
12.9.14. externalSecretsConfig	644
12.9.15. externalSecretsConfigList	644
12.9.16. externalSecretsConfigSpec	645
12.9.17. externalSecretsConfigStatus	645
12.9.18. externalSecretsManager	646
12.9.19. externalSecretsManagerList	646
12.9.20. externalSecretsManagerSpec	647
12.9.21. externalSecretsManagerStatus	647
12.9.22. Feature	648
12.9.23. featureName	651
12.9.24. globalConfig	651
12.9.25. managementState	653
12.9.26. mode	653
12.9.27. networkPolicy	654
12.9.28. objectReference	655
12.9.29. pluginsConfig	656
12.9.30. proxyConfig	656

12.9.31. secretReference	658
12.9.32. webhookConfig	658
12.10. MIGRATING FROM THE COMMUNITY EXTERNAL SECRETS OPERATOR TO THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT	659
12.10.1. Deleting the community External Secrets Operator	659
12.10.2. Uninstalling the community External Secrets Operator	660
12.10.2.1. Uninstalling a helm installed community External Secrets Operator	660
12.10.2.2. Uninstalling an Operator Lifecycle Manager installed community External Secrets Operator	661
12.10.2.3. Uninstalling a raw manifest installed community External Secrets Operator	661
12.10.3. Installing the External Secrets Operator for Red Hat OpenShift	661
12.10.4. Creating the ExternalSecretsConfig Operator	661
CHAPTER 13. VIEWING AUDIT LOGS	665
13.1. ABOUT THE API AUDIT LOG	665
13.2. VIEWING THE AUDIT LOGS	666
13.3. FILTERING AUDIT LOGS	670
13.4. GATHERING AUDIT LOGS	671
13.5. ADDITIONAL RESOURCES	671
CHAPTER 14. CONFIGURING THE AUDIT LOG POLICY	672
14.1. ABOUT AUDIT LOG POLICY PROFILES	672
14.2. CONFIGURING THE AUDIT LOG POLICY	673
14.3. CONFIGURING THE AUDIT LOG POLICY WITH CUSTOM RULES	674
14.4. DISABLING AUDIT LOGGING	675
CHAPTER 15. CONFIGURING TLS SECURITY PROFILES	678
15.1. UNDERSTANDING TLS SECURITY PROFILES	678
15.2. VIEWING TLS SECURITY PROFILE DETAILS	679
15.3. CONFIGURING THE TLS SECURITY PROFILE FOR THE INGRESS CONTROLLER	681
15.4. CONFIGURING THE TLS SECURITY PROFILE FOR THE CONTROL PLANE	683
15.5. CONFIGURING THE TLS SECURITY PROFILE FOR THE KUBELET	686
CHAPTER 16. CONFIGURING SECCOMP PROFILES	689
16.1. VERIFYING THE DEFAULT SECCOMP PROFILE APPLIED TO A POD	689
16.1.1. Upgraded cluster	690
16.1.2. Newly installed cluster	690
16.2. CONFIGURING A CUSTOM SECCOMP PROFILE	691
16.2.1. Creating seccomp profiles	691
16.2.2. Setting up the custom seccomp profile	692
16.2.3. Applying the custom seccomp profile to the workload	692
16.3. ADDITIONAL RESOURCES	693
CHAPTER 17. ALLOWING JAVASCRIPT-BASED ACCESS TO THE API SERVER FROM ADDITIONAL HOSTS	694
17.1. ALLOWING JAVASCRIPT-BASED ACCESS TO THE API SERVER FROM ADDITIONAL HOSTS	694
CHAPTER 18. SCANNING PODS FOR VULNERABILITIES	696
18.1. INSTALLING THE RED HAT QUAY CONTAINER SECURITY OPERATOR	696
18.2. USING THE RED HAT QUAY CONTAINER SECURITY OPERATOR	698
18.3. QUERYING IMAGE VULNERABILITIES FROM THE CLI	698
18.4. UNINSTALLING THE RED HAT QUAY CONTAINER SECURITY OPERATOR	699
18.5. ADDITIONAL RESOURCES	700
CHAPTER 19. NETWORK-BOUND DISK ENCRYPTION (NBDE)	701
19.1. ABOUT DISK ENCRYPTION TECHNOLOGY	701

19.1.1. Disk encryption technology comparison	701
19.1.1.1. Key escrow	701
19.1.1.2. TPM encryption	701
19.1.1.3. Network-Bound Disk Encryption (NBDE)	702
19.1.1.4. Secret sharing encryption	703
19.1.2. Tang server disk encryption	703
19.1.3. Tang server location planning	704
19.1.4. Tang server sizing requirements	705
19.1.5. Logging considerations	706
19.2. TANG SERVER INSTALLATION CONSIDERATIONS	706
19.2.1. Installation scenarios	706
19.2.2. Installing a Tang server	707
19.2.2.1. Compute requirements	707
19.2.2.2. Automatic start at boot	707
19.2.2.3. HTTP versus HTTPS	707
19.3. TANG SERVER ENCRYPTION KEY MANAGEMENT	708
19.3.1. Backing up keys for a Tang server	708
19.3.2. Recovering keys for a Tang server	708
19.3.3. Rekeying Tang servers	708
19.3.3.1. Generating a new Tang server key	709
19.3.3.2. Rekeying all NBDE nodes	711
19.3.3.3. Troubleshooting temporary rekeying errors for Tang servers	714
19.3.3.4. Troubleshooting permanent rekeying errors for Tang servers	714
19.3.4. Deleting old Tang server keys	716
19.4. DISASTER RECOVERY CONSIDERATIONS	717
19.4.1. Loss of a client machine	717
19.4.2. Planning for a loss of client network connectivity	717
19.4.3. Unexpected loss of network connectivity	718
19.4.4. Recovering network connectivity manually	718
19.4.5. Emergency recovery of network connectivity	719
19.4.6. Loss of a network segment	719
19.4.7. Loss of a Tang server	719
19.4.8. Rekeying compromised key material	720

CHAPTER 1. OPENSIFT CONTAINER PLATFORM SECURITY AND COMPLIANCE

Review the security and compliance capabilities available in OpenShift Container Platform, and learn how to secure your cluster.

1.1. SECURITY OVERVIEW

It is important to understand how to properly secure various aspects of your OpenShift Container Platform cluster.

1.1.1. Container security

A good starting point to understanding OpenShift Container Platform security is to review the concepts in [Understanding container security](#). This and subsequent sections provide a high-level walkthrough of the container security measures available in OpenShift Container Platform, including solutions for the host layer, the container and orchestration layer, and the build and application layer. These sections also include information on the following topics:

- Why container security is important and how it compares with existing security standards.
- Which container security measures are provided by the host (RHCOS and RHEL) layer and which are provided by OpenShift Container Platform.
- How to evaluate your container content and sources for vulnerabilities.
- How to design your build and deployment process to proactively check container content.
- How to control access to containers through authentication and authorization.
- How networking and attached storage are secured in OpenShift Container Platform.
- Containerized solutions for API management and SSO.

1.1.2. Auditing

OpenShift Container Platform auditing provides a security-relevant chronological set of records documenting the sequence of activities that have affected the system by individual users, administrators, or other components of the system. Administrators can [configure the audit log policy](#) and [view audit logs](#).

1.1.3. Certificates

Certificates are used by various components to validate access to the cluster. Administrators can [replace the default ingress certificate](#), [add API server certificates](#), or [add a service certificate](#).

You can also review more details about the types of certificates used by the cluster:

- [User-provided certificates for the API server](#)
- [Proxy certificates](#)
- [Service CA certificates](#)

- [Node certificates](#)
- [Bootstrap certificates](#)
- [etcd certificates](#)
- [OLM certificates](#)
- [Aggregated API client certificates](#)
- [Machine Config Operator certificates](#)
- [User-provided certificates for default ingress](#)
- [Ingress certificates](#)
- [Monitoring and cluster logging Operator component certificates](#)
- [Control plane certificates](#)

1.1.4. Encrypting data

You can [enable etcd encryption](#) for your cluster to provide an additional layer of data security. For example, it can help protect the loss of sensitive data if an etcd backup is exposed to the incorrect parties.

1.1.5. Vulnerability scanning

Administrators can use the Red Hat Quay Container Security Operator to run [vulnerability scans](#) and review information about detected vulnerabilities.

1.2. COMPLIANCE OVERVIEW

For many OpenShift Container Platform customers, regulatory readiness, or compliance, on some level is required before any systems can be put into production. That regulatory readiness can be imposed by national standards, industry standards, or the organization's corporate governance framework.

1.2.1. Compliance checking

Administrators can use the [Compliance Operator](#) to run compliance scans and recommend remediations for any issues found. The [oc-compliance plugin](#) is an OpenShift CLI (**oc**) plugin that provides a set of utilities to easily interact with the Compliance Operator.

1.2.2. File integrity checking

Administrators can use the [File Integrity Operator](#) to continually run file integrity checks on cluster nodes and provide a log of files that have been modified.

1.3. ADDITIONAL RESOURCES

- [Understanding authentication](#)
- [Configuring the internal OAuth server](#)

- [Understanding identity provider configuration](#)
- [Using RBAC to define and apply permissions](#)
- [Managing security context constraints](#)

CHAPTER 2. CONTAINER SECURITY

2.1. UNDERSTANDING CONTAINER SECURITY

You should understand how OpenShift Container Platform secures containerized workloads across multiple layers host, container orchestration, build, and application to help you meet your organization's security requirements and compliance standards.

Securing a containerized application relies on multiple levels of security:

- Container security begins with a trusted base container image and continues through the container build process as it moves through your CI/CD pipeline.



IMPORTANT

Image streams by default do not automatically update. This default behavior might create a security issue because security updates to images referenced by an image stream do not automatically occur. You can configure periodic importing to ensure your image streams receive security updates.

- When a container is deployed, its security depends on it running on secure operating systems and networks, and establishing firm boundaries between the container itself and the users and hosts that interact with it.
- Continued security relies on being able to scan container images for vulnerabilities and having an efficient way to correct and replace vulnerable images.

Beyond what a platform such as OpenShift Container Platform offers out of the box, your organization will likely have its own security demands. Some level of compliance verification might be needed before you can even bring OpenShift Container Platform into your data center.

Likewise, you might need to add your own agents, specialized hardware drivers, or encryption features to OpenShift Container Platform, before it can meet your organization's security standards.

This guide provides a high-level walkthrough of the container security measures available in OpenShift Container Platform, including solutions for the host layer, the container and orchestration layer, and the build and application layer. It then points you to specific OpenShift Container Platform documentation to help you achieve those security measures.

This guide contains the following information:

- Why container security is important and how it compares with existing security standards.
- Which container security measures are provided by the host (RHCOS and RHEL) layer and which are provided by OpenShift Container Platform.
- How to evaluate your container content and sources for vulnerabilities.
- How to design your build and deployment process to proactively check container content.
- How to control access to containers through authentication and authorization.
- How networking and attached storage are secured in OpenShift Container Platform.
- Containerized solutions for API management and SSO.

The goal of this guide is to understand the incredible security benefits of using OpenShift Container Platform for your containerized workloads and how the entire Red Hat ecosystem plays a part in making and keeping containers secure. It will also help you understand how you can engage with the OpenShift Container Platform to achieve your organization's security goals.

2.1.1. What are containers?

Containers package applications and their dependencies into single, portable images that you can use for consistent deployment across development, test, and production environments.

The image can be promoted from development, to test, to production, without change. A container might be part of a larger application that works closely with other containers.

Containers provide consistency across environments and multiple deployment targets: physical servers, virtual machines (VMs), and private or public cloud.

Some of the benefits of using containers include:

Infrastructure	Applications
Sandboxed application processes on a shared Linux operating system kernel	Package my application and all of its dependencies
Simpler, lighter, and denser than virtual machines	Deploy to any environment in seconds and enable CI/CD
Portable across different environments	Easily access and share containerized components

Additional resources

- [Understanding Linux containers](#)
- [Building, running, and managing containers](#)

2.1.2. What is OpenShift Container Platform?

You can use OpenShift Container Platform to automate the deployment, operation, and management of containerized applications. OpenShift Container Platform uses Kubernetes as its orchestration engine, enhanced with Red Hat security, support, and enterprise features.

Automating how containerized applications are deployed, run, and managed is the job of a platform such as OpenShift Container Platform. At its core, OpenShift Container Platform relies on the Kubernetes project to provide the engine for orchestrating containers across many nodes in scalable data centers.

Kubernetes is a project, which can run using different operating systems and add-on components that offer no guarantees of supportability from the project. As a result, the security of different Kubernetes platforms can vary.

OpenShift Container Platform is designed to lock down Kubernetes security and integrate the platform with a variety of extended components. To do this, OpenShift Container Platform draws on the extensive Red Hat ecosystem of open source technologies that include the operating systems, authentication, storage, networking, development tools, base container images, and many other components.

OpenShift Container Platform can use Red Hat's experience in uncovering and rapidly deploying fixes for vulnerabilities in the platform itself and the containerized applications running on the platform. Red Hat's experience also extends to efficiently integrating new components with OpenShift Container Platform as they become available and adapting technologies to individual customer needs.

Additional resources

- [Configuring periodic importing of imagestreamtags](#)
- [OpenShift Container Platform architecture](#)
- [OpenShift Security Guide](#)

2.2. UNDERSTANDING HOST AND VM SECURITY

Containers and virtual machines provide ways of separating applications running on a host from the operating system itself. You should understand RHCOS, which is the operating system used by OpenShift Container Platform, to see how the host systems protect containers and hosts from each other.

2.2.1. Securing containers on Red Hat Enterprise Linux CoreOS (RHCOS)

You should understand the security enhancements you can make to the containers in your OpenShift Container Platform clusters.

Containers simplify the act of deploying many applications to run on the same host, using the same kernel and container runtime to spin up each container. The applications can be owned by many users and, because they are kept separate, can run different, and even incompatible, versions of those applications at the same time without issue.

In Linux, containers are just a special type of process, so securing containers is similar in many ways to securing any other running process. An environment for running containers starts with an operating system that can secure the host kernel from containers and other processes running on the host, and secure containers from each other.

Because OpenShift Container Platform 4.21 runs on RHCOS hosts, with the option of using Red Hat Enterprise Linux (RHEL) as worker nodes, the following concepts apply by default to any deployed OpenShift Container Platform cluster. These RHEL security features are at the core of what makes running containers in OpenShift Container Platform more secure:

- *Linux namespaces* enable creating an abstraction of a particular global system resource to make it appear as a separate instance to processes within a namespace. Consequently, several containers can use the same computing resource simultaneously without creating a conflict. Container namespaces that are separate from the host by default include mount table, process table, network interface, user, control group, UTS, and IPC namespaces. Those containers that need direct access to host namespaces need to have elevated permissions to request that access. See *Building, running, and managing containers from the RHEL 9 container documentation* for details on the types of namespaces.
- *SELinux* provides an additional layer of security to keep containers isolated from each other and from the host. SELinux allows administrators to enforce mandatory access controls (MAC) for every user, application, process, and file.

**WARNING**

Disabling SELinux on RHCOS is not supported.

- *CGroups* (control groups) limit, account for, and isolate the resource usage (CPU, memory, disk I/O, network, and so on.) of a collection of processes. CGroups are used to ensure that containers on the same host are not impacted by each other.
- *Secure computing mode* (*seccomp*) profiles can be associated with a container to restrict available system calls.
- Deploying containers using *RHCOS* reduces the attack surface by minimizing the host environment and tuning it for containers. The CRI-O container engine further reduces that attack surface by implementing only those features required by Kubernetes and OpenShift Container Platform to run and manage containers, as opposed to other container engines that implement desktop-oriented standalone features.

RHCOS is a version of Red Hat Enterprise Linux (RHEL) that is specially configured to work as control plane (master) and worker nodes on OpenShift Container Platform clusters. So RHCOS is tuned to efficiently run container workloads, along with Kubernetes and OpenShift Container Platform services.

**NOTE**

To further protect RHCOS systems in OpenShift Container Platform clusters, most containers, except those managing or monitoring the host system itself, should run as a non-root user. Dropping the privilege level or creating containers with the least amount of privileges possible is recommended best practice for protecting your own OpenShift Container Platform clusters.

Additional resources

- [Building, running, and managing containers](#)
- [How nodes enforce resource constraints](#)
- [Managing security context constraints](#)
- [Supported platforms for OpenShift clusters](#)
- [Choosing how to configure RHCOS](#)
- [Ignition](#)
- [Kernel arguments](#)
- [Kernel modules](#)
- [Disk encryption](#)
- [Chrony time service](#)
- [About the OpenShift Update Service](#)

- [Red Hat OpenShift security guide](#)
- [FIPS cryptography](#)

2.2.2. Comparing virtualization and containers

You should understand the differences between containers and VMs to learn the advantages and drawbacks that influence the use cases in which these technologies are typically applied.

Traditional virtualization provides another way to keep application environments separate on the same physical host. However, virtual machines work in a different way than containers. Virtualization relies on a hypervisor spinning up guest virtual machines (VMs), each of which has its own operating system (OS), represented by a running kernel, and the running application and its dependencies.

With VMs, the hypervisor isolates the guests from each other and from the host kernel. Fewer individuals and processes have access to the hypervisor, reducing the attack surface on the physical server. That said, security must still be monitored: one guest VM might be able to use hypervisor bugs to gain access to another VM or the host kernel. And, when the operating system needs to be patched, it must be patched on all guest VMs by using that operating system.

Containers can be run inside guest VMs, and there might be use cases where this is desirable. For example, you might be deploying a traditional application in a container, perhaps to lift-and-shift an application to the cloud.

Container separation on a single host, however, provides a more lightweight, flexible, and easier-to-scale deployment solution. This deployment model is particularly appropriate for cloud-native applications. Containers are generally much smaller than VMs and consume less memory and CPU.

Additional resources

- [Linux Containers Compared to KVM Virtualization](#)

2.2.3. Securing OpenShift Container Platform

To make your OpenShift Container Platform cluster more secure, you should understand the security enhancements you can make to your cluster.

When you deploy OpenShift Container Platform, you have the choice of an installer-provisioned infrastructure (there are several available platforms) or your own user-provisioned infrastructure. Some low-level security-related configuration, such as enabling FIPS mode or adding kernel modules required at first boot, might benefit from a user-provisioned infrastructure. Likewise, user-provisioned infrastructure is appropriate for disconnected OpenShift Container Platform deployments.

Remember when it comes to making security enhancements and other configuration changes to OpenShift Container Platform, the goals should include:

- Keeping the underlying nodes as generic as possible. You want to be able to easily throw away and spin up similar nodes quickly and in prescriptive ways.
- Managing modifications to nodes through OpenShift Container Platform as much as possible, rather than making direct, one-off changes to the nodes.

In pursuit of those goals, most node changes should be done during installation through Ignition or later using MachineConfigs that are applied to sets of nodes by the Machine Config Operator. Examples of security-related configuration changes you can do in this way include:

- Adding kernel arguments
- Adding kernel modules
- Enabling support for FIPS cryptography
- Configuring disk encryption
- Configuring the chrony time service

Besides the Machine Config Operator, there are several other Operators available to configure OpenShift Container Platform infrastructure that are managed by the Cluster Version Operator (CVO). The CVO is able to automate many aspects of OpenShift Container Platform cluster updates.

Additional resources

- [FIPS cryptography](#)

2.3. HARDENING RHCOS

If you are planning to harden RHCOS nodes in OpenShift Container Platform to meet your security needs, you should consider both what to harden and how to go about doing that hardening.

RHCOS was created and tuned to be deployed in OpenShift Container Platform with few if any changes needed to RHCOS nodes. Every organization adopting OpenShift Container Platform has its own requirements for system hardening. As a RHEL system with OpenShift-specific modifications and features added (such as Ignition, ostree, and a read-only `/usr` to provide limited immutability), RHCOS can be hardened just as you would any RHEL system. Differences lie in the ways you manage the hardening.

A key feature of OpenShift Container Platform and its Kubernetes engine is to be able to quickly scale applications and infrastructure up and down as needed. Unless it is unavoidable, you do not want to make direct changes to RHCOS by logging into a host and adding software or changing settings. You want to have the OpenShift Container Platform installer and control plane manage changes to RHCOS so new nodes can be spun up without manual intervention.

So, if you are setting out to harden RHCOS nodes in OpenShift Container Platform to meet your security needs, you should consider both what to harden and how to go about doing that hardening.

2.3.1. Choosing what to harden in RHCOS

You can review the information on how to approach security for any RHEL system in the Red Hat Enterprise Linux 9 Security Hardening guide.

Use this guide to learn how to approach cryptography, evaluate vulnerabilities, and assess threats to various services. Likewise, you can learn how to scan for compliance standards, check file integrity, perform auditing, and encrypt storage devices.

With the knowledge of what features you want to harden, you can then decide how to harden them in RHCOS.

Additional resources

- [Red Hat Enterprise Linux 9 Security Hardening guide](#)

2.3.2. Choosing how to harden RHCOS

Direct modification of RHCOS systems in OpenShift Container Platform is discouraged. Instead, you should think of modifying systems in pools of nodes, such as worker nodes and control plane nodes.

When a new node is needed, in non-bare metal installs, you can request a new node of the type you want and it will be created from an RHCOS image plus the modifications you created earlier.

There are opportunities for modifying RHCOS before installation, during installation, and after the cluster is up and running.

2.3.2.1. Hardening before installation

For bare-metal installations, you can add hardening features to RHCOS before beginning the OpenShift Container Platform installation. For example, you can add kernel options when you boot the RHCOS installer to turn security features on or off, such as various SELinux Boolean values or low-level settings, such as symmetric multithreading.



WARNING

Disabling SELinux on RHCOS nodes is not supported.

Although bare metal RHCOS installations are more difficult, they offer the opportunity of getting operating system changes in place before starting the OpenShift Container Platform installation. This can be important when you need to ensure that certain features, such as disk encryption or special networking settings, be set up at the earliest possible moment.

2.3.2.2. Hardening during installation

You can interrupt the OpenShift Container Platform installation process and change Ignition configs. Through Ignition configs, you can add your own files and systemd services to the RHCOS nodes. You can also make some basic security-related changes to the **install-config.yaml** file used for installation. Contents added in this way are available at each node's first boot.

2.3.2.3. Hardening after the cluster is running

After the OpenShift Container Platform cluster is up and running, there are several ways to apply hardening features to RHCOS:

- **Daemon set:** If you need a service to run on every node, you can add that service with a Kubernetes **DaemonSet** object.
- **Machine config:** **MachineConfig** objects contain a subset of Ignition configs in the same format. By applying machine configs to all worker or control plane nodes, you can ensure that the next node of the same type that is added to the cluster has the same changes applied.

All of the features noted here are described in the OpenShift Container Platform product documentation.

Additional resources

- [Kubernetes DaemonSet documentation](#)
- [OpenShift Security Guide](#)
- [Choosing how to configure RHCOS](#)
- [Modifying Nodes](#)
- [Manually creating the installation configuration file](#)
- [Creating the Kubernetes manifest and Ignition config files](#)
- [Installing RHCOS by using an ISO image](#)
- [Customizing nodes](#)
- [Adding kernel arguments to nodes](#)
- [Optional configuration parameters](#)
- [Support for FIPS cryptography](#)
- [RHEL core crypto components](#)

2.4. CONTAINER IMAGE SIGNATURES

To verify the integrity of the images in the Red Hat Container Registries between Red Hat registries and your infrastructure, you can enable signature verification.

Red Hat delivers signatures for the images in the Red Hat Container Registries. Those signatures can be automatically verified when being pulled to OpenShift Container Platform 4 clusters by using the Machine Config Operator (MCO).

To verify the integrity of those images between Red Hat registries and your infrastructure, enable signature verification.

2.4.1. Enabling signature verification for Red Hat Container Registries

To verify the integrity of the images in the Red Hat Container Registries, you can enable container signature validation for Red Hat Container Registries by writing a signature verification policy file specifying the keys to verify images from these registries.

For RHEL8 nodes, the registries are already defined in **/etc/containers/registries.d** by default.

Procedure

1. Create a Butane config file, **51-worker-rh-registry-trust.bu**, containing the necessary configuration for the worker nodes.



NOTE

The [Butane version](#) you specify in the config file should match the OpenShift Container Platform version and always ends in **0**. For example, **4.21.0**. See "Creating machine configs with Butane" for information about Butane.

```

variant: openshift
version: 4.21.0
metadata:
  name: 51-worker-rh-registry-trust
  labels:
    machineconfiguration.openshift.io/role: worker
storage:
  files:
  - path: /etc/containers/policy.json
    mode: 0644
    overwrite: true
  contents:
    inline: |
      {
        "default": [
          {
            "type": "insecureAcceptAnything"
          }
        ],
        "transports": {
          "docker": {
            "registry.access.redhat.com": [
              {
                "type": "signedBy",
                "keyType": "GPGKeys",
                "keyPath": "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release"
              }
            ],
            "registry.redhat.io": [
              {
                "type": "signedBy",
                "keyType": "GPGKeys",
                "keyPath": "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release"
              }
            ]
          },
          "docker-daemon": {
            "": [
              {
                "type": "insecureAcceptAnything"
              }
            ]
          }
        }
      }

```

2. Use Butane to generate a machine config YAML file, **51-worker-rh-registry-trust.yaml**, containing the file to be written to disk on the worker nodes:

```
$ butane 51-worker-rh-registry-trust.bu -o 51-worker-rh-registry-trust.yaml
```

3. Apply the created machine config:

```
$ oc apply -f 51-worker-rh-registry-trust.yaml
```

4. Check that the worker machine config pool has rolled out with the new machine config:

- a. Check that the new machine config was created:

```
$ oc get mc
```

Sample output

NAME	GENERATEDBYCONTROLLER
IGNITIONVERSION AGE	
00-master 3.5.0 25m	a2178ad522c49ee330b0033bb5cb5ea132060b0a
00-worker 3.5.0 25m	a2178ad522c49ee330b0033bb5cb5ea132060b0a
01-master-container-runtime	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 25m	
01-master-kubelet	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 25m	
01-worker-container-runtime	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 25m	
01-worker-kubelet	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 25m	
51-master-rh-registry-trust	3.5.0 13s
51-worker-rh-registry-trust	3.5.0 53s
99-master-generated-crio-seccomp-use-default	3.5.0
25m	
99-master-generated-registries	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 25m	
99-master-ssh	3.2.0 28m
99-worker-generated-crio-seccomp-use-default	3.5.0
25m	
99-worker-generated-registries	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 25m	
99-worker-ssh	3.2.0 28m
rendered-master-af1e7ff78da0a9c851bab4be2777773b	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 8s	
rendered-master-cd51fd0c47e91812bfef2765c52ec7e6	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 24m	
rendered-worker-2b52f75684fbc711bd1652dd86fd0b82	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 24m	
rendered-worker-be3b3bce4f4aa52a62902304bac9da3c	
a2178ad522c49ee330b0033bb5cb5ea132060b0a 3.5.0 48s	

where:

51-worker-rh-registry-trust

Specifies the new machine config.

rendered-worker-be3b3bce4f4aa52a62902304bac9da3c

Specifies the new rendered machine config.

- b. Check that the worker machine config pool is updating with the new machine config:

```
$ oc get mcp
```

Sample output

```

NAME      CONFIG                                UPDATED  UPDATING  DEGRADED
MACHINECOUNT READYMACHINECOUNT UPDATEDMACHINECOUNT
DEGRADEDMACHINECOUNT AGE
master rendered-master-af1e7ff78da0a9c851bab4be2777773b True    False
False    3      3      3      0      30m
worker rendered-worker-be3b3bce4f4aa52a62902304bac9da3c False   True
False    3      0      0      0      30m

```

When the **UPDATING** field is **True**, the machine config pool is updating with the new machine config. When the field becomes **False**, the worker machine config pool has rolled out to the new machine config.

5. If your cluster uses any RHEL7 worker nodes, when the worker machine config pool is updated, create YAML files on those nodes in the **/etc/containers/registries.d** directory, which specify the location of the detached signatures for a given registry server. The following example works only for images hosted in **registry.access.redhat.com** and **registry.redhat.io**.

- a. Start a debug session to each RHEL7 worker node:

```
$ oc debug node/<node_name>
```

- b. Change your root directory to **/host**:

```
sh-4.2# chroot /host
```

- c. Create a **/etc/containers/registries.d/registry.redhat.io.yaml** file that contains the following:

```

docker:
  registry.redhat.io:
    sigstore: https://registry.redhat.io/containers/sigstore

```

- d. Create a **/etc/containers/registries.d/registry.access.redhat.com.yaml** file that contains the following:

```

docker:
  registry.access.redhat.com:
    sigstore: https://access.redhat.com/webassets/docker/content/sigstore

```

- e. Exit the debug session.

2.4.2. Verifying the signature verification configuration

After you apply the machine configs to the cluster, you can verify that the Machine Config Controller detected the new **MachineConfig** object and generated a new **rendered-worker-*<hash>*** version.

Prerequisites

- You enabled signature verification by using a machine config file.

Procedure

1. On the command line, run the following command to display information about a required worker:

```
$ oc describe machineconfigpool/worker
```

Example output of initial worker monitoring

```
Name:      worker
Namespace:
Labels:    machineconfiguration.openshift.io/mco-built-in=
Annotations: <none>
API Version: machineconfiguration.openshift.io/v1
Kind:      MachineConfigPool
Metadata:
  Creation Timestamp: 2019-12-19T02:02:12Z
  Generation:        3
  Resource Version:   16229
  Self Link:          /apis/machineconfiguration.openshift.io/v1/machineconfigpools/worker
  UID:                92697796-2203-11ea-b48c-fa163e3940e5
Spec:
  Configuration:
    Name: rendered-worker-f6819366eb455a401c42f8d96ab25c02
    Source:
      API Version: machineconfiguration.openshift.io/v1
      Kind:      MachineConfig
      Name:      00-worker
      API Version: machineconfiguration.openshift.io/v1
      Kind:      MachineConfig
      Name:      01-worker-container-runtime
      API Version: machineconfiguration.openshift.io/v1
      Kind:      MachineConfig
      Name:      01-worker-kubelet
      API Version: machineconfiguration.openshift.io/v1
      Kind:      MachineConfig
      Name:      51-worker-rh-registry-trust
      API Version: machineconfiguration.openshift.io/v1
      Kind:      MachineConfig
      Name:      99-worker-92697796-2203-11ea-b48c-fa163e3940e5-registries
      API Version: machineconfiguration.openshift.io/v1
      Kind:      MachineConfig
      Name:      99-worker-ssh
  Machine Config Selector:
    Match Labels:
      machineconfiguration.openshift.io/role: worker
  Node Selector:
    Match Labels:
      node-role.kubernetes.io/worker:
  Paused:      false
Status:
  Conditions:
    Last Transition Time: 2019-12-19T02:03:27Z
    Message:
    Reason:
    Status:      False
```



```

Type:          RenderDegraded
Last Transition Time: 2019-12-19T02:03:43Z
Message:
Reason:
Status:        False
Type:          NodeDegraded
Last Transition Time: 2019-12-19T02:03:43Z
Message:
Reason:
Status:        False
Type:          Degraded
Last Transition Time: 2019-12-19T02:28:23Z
Message:
Reason:
Status:        False
Type:          Updated
Last Transition Time: 2019-12-19T02:28:23Z
Message:       All nodes are updating to rendered-worker-
f6819366eb455a401c42f8d96ab25c02
Reason:
Status:        True
Type:          Updating
Configuration:
Name: rendered-worker-d9b3f4ffcfcd65c30dcf591a0e8cf9b2e
Source:
  API Version:  machineconfiguration.openshift.io/v1
  Kind:         MachineConfig
  Name:         00-worker
  API Version:  machineconfiguration.openshift.io/v1
  Kind:         MachineConfig
  Name:         01-worker-container-runtime
  API Version:  machineconfiguration.openshift.io/v1
  Kind:         MachineConfig
  Name:         01-worker-kubelet
  API Version:  machineconfiguration.openshift.io/v1
  Kind:         MachineConfig
  Name:         99-worker-92697796-2203-11ea-b48c-fa163e3940e5-registries
  API Version:  machineconfiguration.openshift.io/v1
  Kind:         MachineConfig
  Name:         99-worker-ssh
Degraded Machine Count: 0
Machine Count:         1
Observed Generation:   3
Ready Machine Count:   0
Unavailable Machine Count: 1
Updated Machine Count: 0
Events:                <none>

```

2. Run the **oc describe** command again:

```
$ oc describe machineconfigpool/worker
```

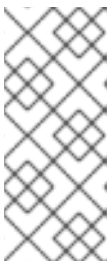
Example output after the worker is updated

```
...
```

```

Last Transition Time: 2019-12-19T04:53:09Z
Message:           All nodes are updated with rendered-worker-
f6819366eb455a401c42f8d96ab25c02
Reason:
Status:           True
Type:             Updated
Last Transition Time: 2019-12-19T04:53:09Z
Message:
Reason:
Status:           False
Type:             Updating
Configuration:
Name: rendered-worker-f6819366eb455a401c42f8d96ab25c02
Source:
  API Version:     machineconfiguration.openshift.io/v1
  Kind:            MachineConfig
  Name:            00-worker
  API Version:     machineconfiguration.openshift.io/v1
  Kind:            MachineConfig
  Name:            01-worker-container-runtime
  API Version:     machineconfiguration.openshift.io/v1
  Kind:            MachineConfig
  Name:            01-worker-kubelet
  API Version:     machineconfiguration.openshift.io/v1
  Kind:            MachineConfig
  Name:            51-worker-rh-registry-trust
  API Version:     machineconfiguration.openshift.io/v1
  Kind:            MachineConfig
  Name:            99-worker-92697796-2203-11ea-b48c-fa163e3940e5-registries
  API Version:     machineconfiguration.openshift.io/v1
  Kind:            MachineConfig
  Name:            99-worker-ssh
Degraded Machine Count: 0
Machine Count:         3
Observed Generation:   4
Ready Machine Count:   3
Unavailable Machine Count: 0
Updated Machine Count: 3
...

```



NOTE

The **Observed Generation** parameter shows an increased count based on the generation of the controller-produced configuration. This controller updates this value even if it fails to process the specification and generate a revision. The **Configuration Source** value points to the **51-worker-rh-registry-trust** configuration.

- Confirm that the **policy.json** file exists with the following command:

```
$ oc debug node/<node> -- chroot /host cat /etc/containers/policy.json
```

Example output

```

Starting pod/<node>-debug ...
To use host binaries, run `chroot /host`
{
  "default": [
    {
      "type": "insecureAcceptAnything"
    }
  ],
  "transports": {
    "docker": {
      "registry.access.redhat.com": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPath": "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release"
        }
      ],
      "registry.redhat.io": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPath": "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release"
        }
      ]
    },
    "docker-daemon": {
      "": [
        {
          "type": "insecureAcceptAnything"
        }
      ]
    }
  }
}

```

4. Confirm that the **registry.redhat.io.yaml** file exists with the following command:

```

$ oc debug node/<node> -- chroot /host cat
/etc/containers/registries.d/registry.redhat.io.yaml

```

Example output

```

Starting pod/<node>-debug ...
To use host binaries, run `chroot /host`
docker:
  registry.redhat.io:
    sigstore: https://registry.redhat.io/containers/sigstore

```

5. Confirm that the **registry.access.redhat.com.yaml** file exists with the following command:

```

$ oc debug node/<node> -- chroot /host cat
/etc/containers/registries.d/registry.access.redhat.com.yaml

```

Example output

```
Starting pod/<node>-debug ...
To use host binaries, run `chroot /host`
docker:
  registry.access.redhat.com:
    sigstore: https://access.redhat.com/webassets/docker/content/sigstore
```

2.4.3. Understanding the verification of container images lacking verifiable signatures

Each OpenShift Container Platform release image is immutable and signed with a Red Hat production key. During cluster update or installation, a release image might deploy container images without a verifiable signature. The signature on the release image validates all release contents transitively.

For example, the image references lacking a verifiable signature are contained in the signed OpenShift Container Platform release image:

Example release info output

```
$ oc adm release info quay.io/openshift-release-dev/ocp-
release@sha256:2309578b68c5666dad62aed696f1f9d778ae1a089ee461060ba7b9514b7ca417 -o
pullspec
quay.io/openshift-release-dev/ocp-v4.0-art-
dev@sha256:9aafb914d5d7d0dec4edd800d02f811d7383a7d49e500af548eab5d00c1bffdb
```

The first line specifies the signed release image SHA. The second line specifies a container image lacking a verifiable signature that is included in the release.

2.4.3.1. Automated verification during updates

Verification of signatures is automatic. The OpenShift Cluster Version Operator (CVO) verifies signatures on the release images during an OpenShift Container Platform update. This is an internal process. An OpenShift Container Platform installation or update fails if the automated verification fails.

Verification of signatures can also be done manually using the **skopeo** command-line utility.

2.4.3.2. Using skopeo to verify signatures of Red Hat container images

You can verify the signatures for container images included in an OpenShift Container Platform release image by pulling those signatures from the OpenShift Container Platform release mirror site.

Because the signatures on the mirror site are not in a format readily understood by Podman or CRI-O, you can use the **skopeo standalone-verify** command to verify that your release images are signed by Red Hat.

Prerequisites

- You have installed the **skopeo** command-line utility.

Procedure

1. Get the full SHA for your release by running the following command:

```
$ oc adm release info <release_version>
```

- Substitute `<release_version>` with your release number, for example, **4.14.3**.

Example output snippet

```
---
Pull From: quay.io/openshift-release-dev/ocp-
release@sha256:e73ab4b33a9c3ff00c9f800a38d69853ca0c4dfa5a88e3df331f66df8f18ec5
5
---
```

2. Pull down the Red Hat release key by running the following command:

```
$ curl -o pub.key https://access.redhat.com/security/data/fd431d51.txt
```

3. Get the signature file for the specific release that you want to verify by running the following command:

```
$ curl -o signature-1 https://mirror.openshift.com/pub/openshift-v4/signatures/openshift-
release-dev/ocp-release/sha256=<sha_from_version>/signature-1
```

Replace **<sha_from_version>** with SHA value from the full link to the mirror site that matches the SHA of your release. For example, the link to the signature for the 4.12.23 release is <https://mirror.openshift.com/pub/openshift-v4/signatures/openshift-release-dev/ocp-release/sha256=e73ab4b33a9c3ff00c9f800a38d69853ca0c4dfa5a88e3df331f66df8f18ec55/signature-1>, and the SHA value is **e73ab4b33a9c3ff00c9f800a38d69853ca0c4dfa5a88e3df331f66df8f18ec55**.

4. Get the manifest for the release image by running the following command:

```
$ skopeo inspect --raw docker://<quay_link_to_release> > manifest.json
```

Replace **<quay_link_to_release>** with the output of the **oc adm release info** command. For example, **quay.io/openshift-release-dev/ocp-release@sha256:e73ab4b33a9c3ff00c9f800a38d69853ca0c4dfa5a88e3df331f66df8f18ec55**.

5. Use skopeo to verify the signature:

```
$ skopeo standalone-verify manifest.json quay.io/openshift-release-dev/ocp-release:
<release_number>-<arch> any signature-1 --public-key-file pub.key
```

where:

<release_number>

Specifies the release number, for example **4.14.3**.

<arch>

Specifies the architecture, for example **x86_64**.

Example output

```
Signature verified using fingerprint 567E347AD0044ADE55BA8A5F199E2F91FD431D51,
digest sha256:e73ab4b33a9c3ff00c9f800a38d69853ca0c4dfa5a88e3df331f66df8f18ec55
```

2.4.4. Additional resources

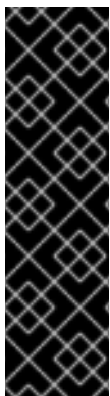
- [Quay.io](#)
- [Red Hat Ecosystem Catalog Container images](#)
- [Introduction to OpenShift Updates](#)
- [Machine Config Overview](#)
- [OpenShift release signatures mirror site](#)
- [Red Hat GPG release key](#)

2.5. UNDERSTANDING COMPLIANCE

You should understand the regulatory readiness, or compliance, that is required before any systems can be put into production. That regulatory readiness can be imposed by national standards, industry standards or the organization's corporate governance framework.

2.5.1. Understanding compliance and risk management

FIPS compliance is one of the most critical components required in highly secure environments to ensure that only supported cryptographic technologies are allowed on nodes.



IMPORTANT

To enable FIPS mode for your cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Switching RHEL to FIPS mode](#).

When running Red Hat Enterprise Linux (RHEL) or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures.

To understand Red Hat's view of OpenShift Container Platform compliance frameworks, refer to the Risk Management and Regulatory Readiness chapter of the OpenShift Security Guide Book.

Additional resources

- [Installing a cluster in FIPS mode](#)

2.6. SECURING CONTAINER CONTENT

To ensure the security of the content inside your containers you need to start with trusted base images, such as Red Hat Universal Base Images, and add trusted software. To check the ongoing security of your container images, there are both Red Hat and third-party tools for scanning images.

2.6.1. Securing inside the container

For the security of your containers, you need to know where any source packages originally came from, what versions are used, who built them, and whether there is any malicious code inside them.

However, when using these packages, you should answer the following questions about the packages:

Containerized versions of these packages are also available. However, you need to know where the packages originally came from, what versions are used, who built them, and whether there is any malicious code inside them.

Some questions to answer include:

- Will what is inside the containers compromise your infrastructure?
- Are there known vulnerabilities in the application layer?
- Are the runtime and operating system layers current?

By building your containers from Red Hat Universal Base Images (UBI) you are assured of a foundation for your container images that consists of the same RPM-packaged software that is included in Red Hat Enterprise Linux. No subscriptions are required to either use or redistribute UBI images.

To assure ongoing security of the containers themselves, security scanning features, used directly from RHEL or added to OpenShift Container Platform, can alert you when an image you are using has vulnerabilities. OpenSCAP image scanning is available in RHEL and the Red Hat Quay Container Security Operator can be added to check container images used in OpenShift Container Platform.

2.6.2. Creating redistributable images with UBI

You can typically start with a trusted base image that offers the components that are usually provided by the operating system to create containerized applications. These include the libraries, utilities, and other features the application expects to see in the operating system's file system.

Red Hat Universal Base Images (UBI) were created to encourage anyone building their own containers to start with one that is made entirely from Red Hat Enterprise Linux RPM packages and other content. These UBI images are updated regularly to keep up with security patches and free to use and redistribute with container images built to include your own software.

Search the Red Hat Ecosystem Catalog to both find and check the health of different UBI images. As someone creating secure container images, you might be interested in these two general types of UBI images:

- **UBI:** There are standard UBI images for RHEL 7, 8, and 9 (**ubi7/ubi**, **ubi8/ubi**, and **ubi9/ubi**), and minimal images based on those systems (**ubi7/ubi-minimal**, **ubi8/ubi-minimal**, and **ubi9/ubi-minimal**). All of these images are preconfigured to point to free repositories of RHEL software that you can add to the container images you build, using standard **yum** and **dnf** commands.



NOTE

Red Hat encourages people to use these images on other distributions, such as Fedora and Ubuntu.

- **Red Hat Software Collections:** Search the Red Hat Ecosystem Catalog for **rhsc/** to find images created to use as base images for specific types of applications. For example, there are Apache httpd (**rhsc/httpd-***), Python (**rhsc/python-***), Ruby (**rhsc/ruby-***), Node.js (**rhsc/nodejs-***) and Perl (**rhsc/perl-***) rhsc images.

Remember that while UBI images are freely available and redistributable, Red Hat support for these images is only available through Red Hat product subscriptions.

2.6.3. Security scanning in RHEL

For Red Hat Enterprise Linux (RHEL) systems, OpenSCAP scanning is available from the **openscap-utils** package. In RHEL, you can use the **openscap-podman** command to scan images for vulnerabilities.

OpenShift Container Platform enables you to use RHEL scanners with your Continuous Integration and Continuous Delivery (CI/CD) process. For example, you can integrate static code analysis tools that test for security flaws in your source code and software composition analysis tools that identify open source libraries to provide metadata on those libraries such as known vulnerabilities.

2.6.3.1. Scanning OpenShift images

For the container images that are running in OpenShift Container Platform and are pulled from Red Hat Quay registries, you can use an Operator to list the vulnerabilities of those images. The Red Hat Quay Container Security Operator can be added to OpenShift Container Platform to provide vulnerability reporting for images added to selected namespaces.

Container image scanning for Red Hat Quay is performed by Clair. In Red Hat Quay, Clair can search for and report vulnerabilities in images built from RHEL, CentOS, Oracle, Alpine, Debian, and Ubuntu operating system software.

2.6.4. Integrating external scanning

OpenShift Container Platform makes use of object annotations to extend functionality. You can use external tools, such as vulnerability scanners, to annotate image objects with metadata to summarize results and control pod execution.

This section describes the recognized format of this annotation so it can be reliably used in consoles to display useful data to users.

2.6.4.1. Image metadata

There are different types of image quality data, including package vulnerabilities and open source software (OSS) license compliance. Additionally, there might be more than one provider of this metadata. To that end, the following annotation format has been reserved:

```
quality.images.openshift.io/<qualityType>.<providerId>: {}
```

Table 2.1. Annotation key format

Component	Description	Acceptable values
qualityType	Metadata type	vulnerability license operations policy
providerId	Provider ID string	openscap redhatcatalog redhatinsights blackduck jfrog

2.6.4.1.1. Example annotation keys

```
quality.images.openshift.io/vulnerability.blackduck: {}
quality.images.openshift.io/vulnerability.jfrog: {}
quality.images.openshift.io/license.blackduck: {}
quality.images.openshift.io/vulnerability.openscap: {}
```

The value of the image quality annotation is structured data that must adhere to the following format:

Table 2.2. Annotation value format

Field	Required?	Description	Type
name	Yes	Provider display name	String
timestamp	Yes	Scan timestamp	String
description	No	Short description	String
reference	Yes	URL of information source or more details. Required so user might validate the data.	String
scannerVersion	No	Scanner version	String
compliant	No	Compliance pass or fail	Boolean
summary	No	Summary of issues found	List (see table below)

The **summary** field must adhere to the following format:

Table 2.3. Summary field value format

Field	Description	Type
label	Display label for component (for example, "critical," "important," "moderate," "low," or "health")	String
data	Data for this component (for example, count of vulnerabilities found or score)	String
severityIndex	Component index allowing for ordering and assigning graphical representation. The value is range 0..3 where 0 = low.	Integer

Field	Description	Type
reference	URL of information source or more details. Optional.	String

2.6.4.1.2. Example annotation values

This example shows an OpenSCAP annotation for an image with vulnerability summary data and a compliance boolean:

OpenSCAP annotation

```
{
  "name": "OpenSCAP",
  "description": "OpenSCAP vulnerability score",
  "timestamp": "2016-09-08T05:04:46Z",
  "reference": "https://www.open-scap.org/930492",
  "compliant": true,
  "scannerVersion": "1.2",
  "summary": [
    { "label": "critical", "data": "4", "severityIndex": 3, "reference": null },
    { "label": "important", "data": "12", "severityIndex": 2, "reference": null },
    { "label": "moderate", "data": "8", "severityIndex": 1, "reference": null },
    { "label": "low", "data": "26", "severityIndex": 0, "reference": null }
  ]
}
```

This example shows the [Container images section of the Red Hat Ecosystem Catalog](#) annotation for an image with health index data with an external URL for additional details:

Red Hat Ecosystem Catalog annotation

```
{
  "name": "Red Hat Ecosystem Catalog",
  "description": "Container health index",
  "timestamp": "2016-09-08T05:04:46Z",
  "reference": "https://access.redhat.com/errata/RHBA-2016:1566",
  "compliant": null,
  "scannerVersion": "1.2",
  "summary": [
    { "label": "Health index", "data": "B", "severityIndex": 1, "reference": null }
  ]
}
```

2.6.4.2. Annotating image objects

While image stream objects are what a user of OpenShift Container Platform operates against, image objects are annotated with security metadata. Image objects are cluster-scoped, pointing to a single image that might be referenced by many image streams and tags.

2.6.4.2.1. Example annotate CLI command

Replace **<image>** with an image digest, for example

sha256:401e359e0f45bfdcf004e258b72e253fd07fba8cc5c6f2ed4f4608fb119ecc2:

```
$ oc annotate image <image> \
  quality.images.openshift.io/vulnerability.redhatcatalog='{ \
    "name": "Red Hat Ecosystem Catalog", \
    "description": "Container health index", \
    "timestamp": "2020-06-01T05:04:46Z", \
    "compliant": null, \
    "scannerVersion": "1.2", \
    "reference": "https://access.redhat.com/errata/RHBA-2020:2347", \
    "summary": "[ \
      { "label": "Health index", "data": "B", "severityIndex": 1, "reference": null } ]" }
```

2.6.4.3. Controlling pod execution

Use the **images.openshift.io/deny-execution** image policy to programmatically control if an image can be run.

2.6.4.3.1. Example annotation

```
annotations:
  images.openshift.io/deny-execution: true
```

2.6.4.4. Integration reference

In most cases, external tools such as vulnerability scanners develop a script or plugin that watches for image updates, performs scanning, and annotates the associated image object with the results. Typically this automation calls the OpenShift Container Platform 4.21 REST APIs to write the annotation. See OpenShift Container Platform REST APIs for general information about the REST APIs.

2.6.4.4.1. Example REST API call

The following example call by using **curl** overrides the value of the annotation. Be sure to replace the values for **<token>**, **<openshift_server>**, **<image_id>**, and **<image_annotation>**.

Patch API call

```
$ curl -X PATCH \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/merge-patch+json" \
  https://<openshift_server>:6443/apis/image.openshift.io/v1/images/<image_id> \
  --data '{ <image_annotation> }'
```

The following is an example of **PATCH** payload data:

Patch call data

```
{
  "metadata": {
    "annotations": {
      "quality.images.openshift.io/vulnerability.redhatcatalog":
        "{ 'name': 'Red Hat Ecosystem Catalog', 'description': 'Container health index', 'timestamp': '2020-
```

```
06-01T05:04:46Z', 'compliant': null, 'reference': 'https://access.redhat.com/errata/RHBA-2020:2347',
'summary': [{ 'label': 'Health index', 'data': '4', 'severityIndex': 1, 'reference': null}] }"
  }
}
```

Additional resources

- [Image stream objects](#)

2.7. USING CONTAINER REGISTRIES SECURELY

You can use container registries to store container images, making the images accessible to others either publicly or privately.

By using a registry, you can include multiple versions of an image, optionally limit access to images based on different authentication methods, or make them publicly available.

There are public container registries, such as Quay.io and Docker Hub where many people and organizations share their images. The Red Hat Registry offers supported Red Hat and partner images, while the Red Hat Ecosystem Catalog offers detailed descriptions and health checks for those images. To manage your own registry, you could purchase a container registry such as Red Hat Quay.

From a security standpoint, some registries provide special features to check and improve the health of your containers. For example, Red Hat Quay offers container vulnerability scanning with Clair security scanner, build triggers to automatically rebuild images when source code changes in GitHub and other locations, and the ability to use role-based access control (RBAC) to secure access to images.

2.7.1. Knowing where containers come from

You can use tools to scan and track the contents of your downloaded and deployed container images. However, there are many public sources of container images. When using public container registries, you can add a layer of protection by using trusted sources.

2.7.2. Immutable and certified containers

Immutable containers are containers that will never be changed while running. You do not step into the running immutable container to replace one or more binaries. From an operational standpoint, you rebuild and redeploy an updated container image to replace a container instead of changing it.

Consuming security updates is particularly important when managing immutable containers.

Red Hat certified images are:

- Free of known vulnerabilities in the platform components or layers
- Compatible across the RHEL platforms, from bare metal to cloud
- Supported by Red Hat

The list of known vulnerabilities is constantly evolving, so you must track the contents of your deployed container images, and newly downloaded images, over time. You can use Red Hat Security Advisories (RHSA) to alert you to any newly discovered issues in Red Hat certified container images, and direct you to the updated image. Alternatively, you can go to the Red Hat Ecosystem Catalog to look up that and other security-related issues for each Red Hat image.

2.7.3. Getting containers from Red Hat Registry and Ecosystem Catalog

Red Hat lists certified container images for Red Hat products and partner offerings from the Container Images section of the Red Hat Ecosystem Catalog. From that catalog, you can see details of each image, including CVE, software packages listings, and health scores.

Red Hat images are actually stored in what is referred to as the *Red Hat Registry*, which is represented by a public container registry (**registry.access.redhat.com**) and an authenticated registry (**registry.redhat.io**). Both include the same set of container images, with **registry.redhat.io** including some additional images that require authentication with Red Hat subscription credentials.

Container content is monitored for vulnerabilities by Red Hat and updated regularly. When Red Hat releases security updates, such as fixes to *glibc*, *DROWN*, or *Dirty Cow*, any affected container images are also rebuilt and pushed to the Red Hat Registry.

Red Hat uses a **health index** to reflect the security risk for each container provided through the Red Hat Ecosystem Catalog. Because containers consume software provided by Red Hat and the errata process, old, stale containers are insecure whereas new, fresh containers are more secure.

To illustrate the age of containers, the Red Hat Ecosystem Catalog uses a grading system. A freshness grade is a measure of the oldest and most severe security errata available for an image. "A" is more up to date than "F". See "Container Health Index grades as used inside the Red Hat Ecosystem Catalog" for more details on this grading system.

See the Red Hat Product Security Center for details on security updates and vulnerabilities related to Red Hat software. Check out Red Hat Security Advisories to search for specific advisories and CVEs.

Additional resources

- [Red Hat Product Security Center](#)
- [Red Hat Security Advisories](#)

2.7.4. OpenShift Container Registry

To manage your container images, you can use the *OpenShift Container Registry*, a private registry in OpenShift Container Platform that runs as an integrated component of the platform. The registry provides role-based access controls that allow you to manage who can pull and push which container images.

OpenShift Container Platform also supports integration with other private registries that you might already be using, such as Red Hat Quay.

Additional resources

- [Integrated OpenShift image registry](#)

2.7.5. Storing containers using Red Hat Quay

Red Hat Quay is an enterprise-quality container registry product from Red Hat. Development for Red Hat Quay is done through the upstream Project Quay. Red Hat Quay is available to deploy on-premise or through the hosted version of Red Hat Quay at Quay.io.

Security-related features of Red Hat Quay include:

- **Time machine:** Allows images with older tags to expire after a set period of time or based on a user-selected expiration time.
- **Repository mirroring:** Lets you mirror other registries for security reasons, such as hosting a public repository on Red Hat Quay behind a company firewall, or for performance reasons, to keep registries closer to where they are used.
- **Action log storage:** Save Red Hat Quay logging output to Elasticsearch storage or Splunk to allow for later search and analysis.
- **Clair:** Scan images against a variety of Linux vulnerability databases, based on the origins of each container image.
- **Internal authentication:** Use the default local database to handle RBAC authentication to Red Hat Quay or choose from LDAP, Keystone (OpenStack), JWT Custom Authentication, or External Application Token authentication.
- **External authorization (OAuth):** Allow authorization to Red Hat Quay from GitHub, GitHub Enterprise, or Google Authentication.
- **Access settings:** Generate tokens to allow access to Red Hat Quay from docker, rkt, anonymous access, user-created accounts, encrypted client passwords, or prefix username autocompletion.

Ongoing integration of Red Hat Quay with OpenShift Container Platform continues, with several OpenShift Container Platform Operators of particular interest. The Quay Bridge Operator lets you replace the internal OpenShift image registry with Red Hat Quay. The Red Hat Quay Container Security Operator lets you check vulnerabilities of images running in OpenShift Container Platform that were pulled from Red Hat Quay registries.

2.8. SECURING THE BUILD PROCESS

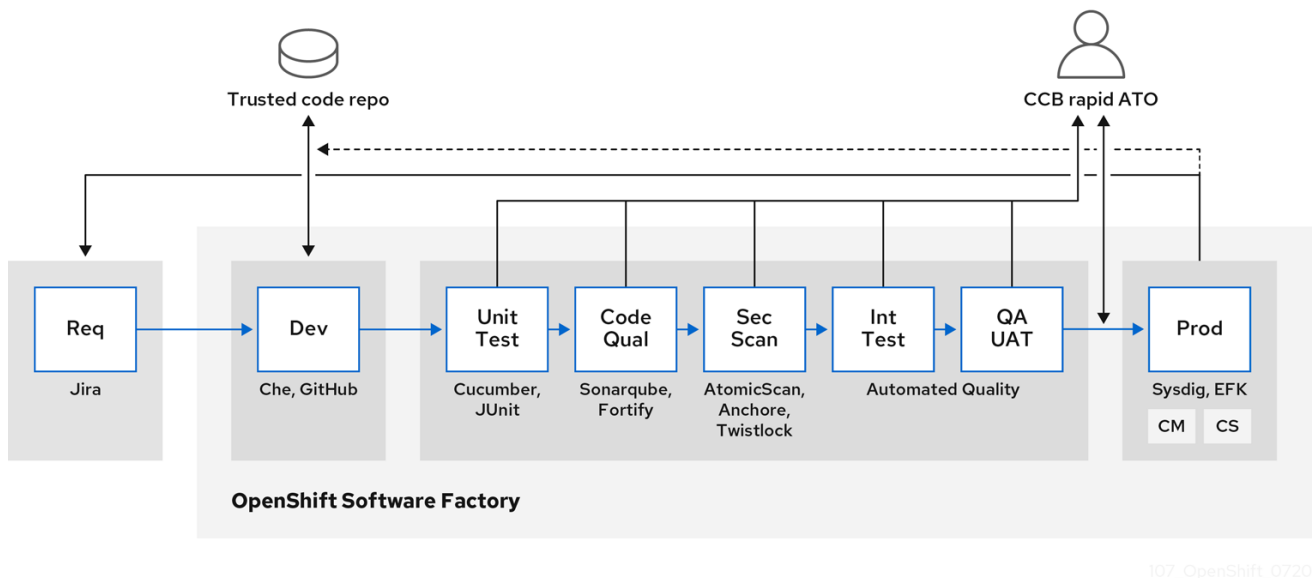
You can secure your software supply chain by using trusted base images, integrating security testing, and building once to deploy everywhere. Managing this build process ensures production deployments match verified builds and protects the software stack where code and libraries integrate.

2.8.1. Building once, deploying everywhere

You can build container images once in a secure environment and deploy them unchanged across all stages. Using OpenShift Container Platform as your build standard guarantees this security, ensuring production deployments match verified builds and preventing runtime vulnerabilities.

It is also important to maintain the immutability of your containers. You should not patch running containers, but rebuild and redeploy them.

As your software moves through the stages of building, testing, and production, it is important that the tools making up your software supply chain be trusted. The following figure illustrates the process and tools that could be incorporated into a trusted software supply chain for containerized software:



OpenShift Container Platform can be integrated with trusted code repositories (such as GitHub) and development platforms (such as Che) for creating and managing secure code. Unit testing frameworks can validate code quality before builds.

You can inspect your containers for vulnerabilities and configuration issues at build, deploy, or runtime with Red Hat Advanced Cluster Security for Kubernetes. For images stored in Quay, you can use the Clair scanner to inspect images at rest. In addition, certified vulnerability scanners are available in the Red Hat ecosystem catalog.

Monitoring tools can provide ongoing visibility of your containerized applications.

Additional resources

- [Cucumber](#)
- [JUnit](#)
- [Red Hat Advanced Cluster Security for Kubernetes](#)
- [Clair scanner](#)
- [Certified vulnerability scanners](#)
- [Sysdig](#)

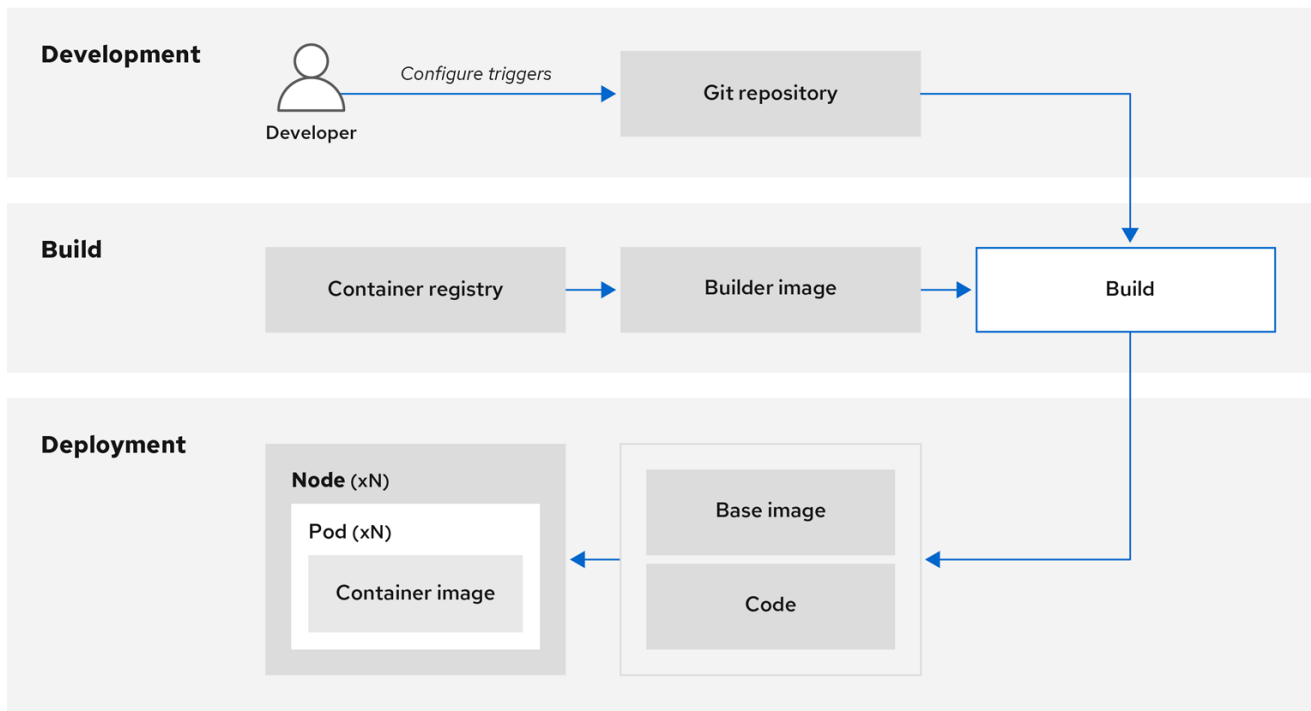
2.8.2. Managing builds

You can use Source-to-Image (S2I) builder images that enable development and operations teams to collaborate on reproducible builds, using Red Hat Universal Base Images that you can freely redistribute with your applications.

You can use Source-to-Image (S2I) to combine source code and base images. *Builder images* make use of S2I to enable your development and operations teams to collaborate on a reproducible build environment. With Red Hat S2I images available as Universal Base Image (UBI) images, you can now freely redistribute your software with base images built from real RHEL RPM packages. Red Hat has removed subscription restrictions to allow this.

When developers commit code with Git for an application by using build images, OpenShift Container Platform can perform the following functions:

- Trigger, either by using webhooks on the code repository or other automated continuous integration (CI) process, to automatically assemble a new image from available artifacts, the S2I builder image, and the newly committed code.
- Automatically deploy the newly built image for testing.
- Promote the tested image to production where it can be automatically deployed using a CI process.



107_OpenShift_0720

You can use the integrated OpenShift Container Registry to manage access to final images. Both S2I and native build images are automatically pushed to your OpenShift Container Registry.

In addition to the included Jenkins for CI, you can also integrate your own build and CI environment with OpenShift Container Platform using RESTful APIs, and use any API-compliant image registry.

2.8.3. Securing inputs during builds

You can protect sensitive credentials required during builds by defining input secrets that give access to dependent resources without exposing those credentials in the final application image.

In some scenarios, build operations require credentials to access dependent resources, but it is undesirable for those credentials to be available in the final application image produced by the build. You can define input secrets for this purpose.

For example, when building a Node.js application, you can set up your private mirror for Node.js modules. To download modules from that private mirror, you must supply a custom **.npmrc** file for the build that has a URL, user name, and password. For security reasons, you do not want to expose your credentials in the application image.

Using this example scenario, you can add an input secret to a new **BuildConfig** object.

Procedure

1. Create the secret, if it does not exist:

```
$ oc create secret generic secret-npmrc --from-file=.npmrc=~/.npmrc
```

This creates a new secret named **secret-npmrc**, which has the base64 encoded content of the `~/.npmrc` file.

2. Add the secret to the **source** section in the existing **BuildConfig** object:

```
source:
  git:
    uri: https://github.com/sclorg/nodejs-ex.git
  secrets:
    - destinationDir: .
      secret:
        name: secret-npmrc
```

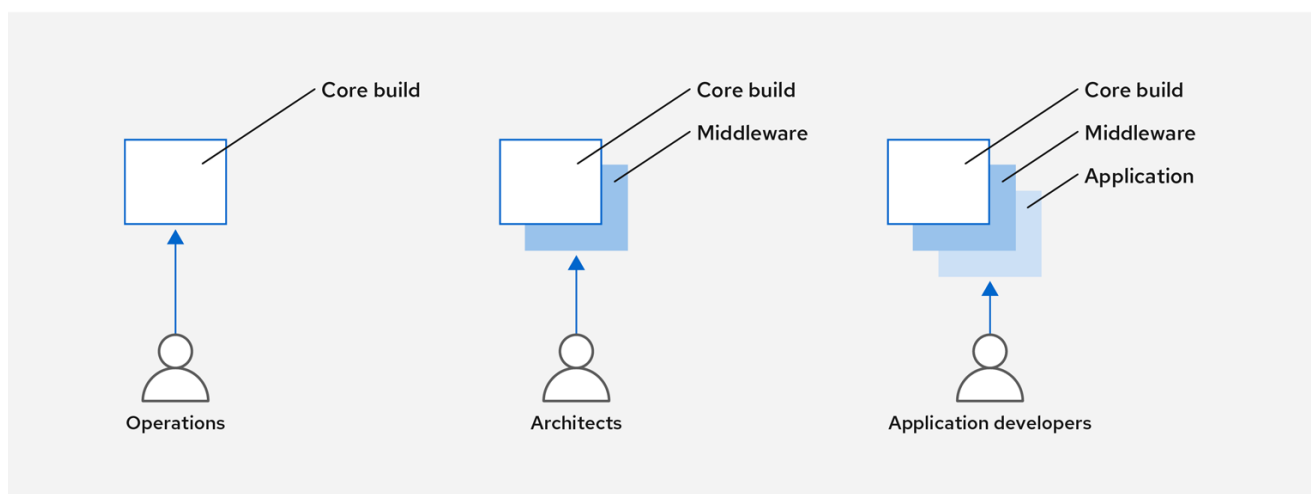
3. To include the secret in a new **BuildConfig** object, run the following command:

```
$ oc new-build \
  openshift/nodejs-010-centos7~https://github.com/sclorg/nodejs-ex.git \
  --build-secret secret-npmrc
```

2.8.4. Designing your build process

You can design your container image management to separate control across teams by using layered images, integrate automated security testing into your CI process, and sign custom containers to ensure integrity between build and deployment.

You can design your container image management and build process to use container layers so that you can separate control.



107_OpenShift_0720

For example, an operations team manages base images, while architects manage middleware, runtimes, databases, and other solutions. Developers can then focus on application layers and focus on writing code.

Because new vulnerabilities are identified daily, you need to proactively check container content over time. To do this, you should integrate automated security testing into your build or CI process. For example:

- SAST / DAST - Static and Dynamic security testing tools.
- Scanners for real-time checking against known vulnerabilities. Tools such as these catalog the open source packages in your container, notify you of any known vulnerabilities, and update you when new vulnerabilities are discovered in previously scanned packages.

Your CI process should include policies that flag builds with issues discovered by security scans so that your team can take appropriate action to address those issues. You should sign your custom built containers to ensure that nothing is tampered with between build and deployment.

Using GitOps methodology, you can use the same CI/CD mechanisms to manage not only your application configurations, but also your OpenShift Container Platform infrastructure.

2.8.5. Building Knative serverless applications

You can build, deploy, and manage serverless applications by using OpenShift Serverless in OpenShift Container Platform, relying on Kubernetes and Kourier, leveraging S2I builder images and Knative services for scalable, event-driven workloads.

As with other builds, you can use S2I images to build your containers, then serve them using Knative services. View Knative application builds through the **Topology** view of the OpenShift Container Platform web console.

2.8.6. Additional resources

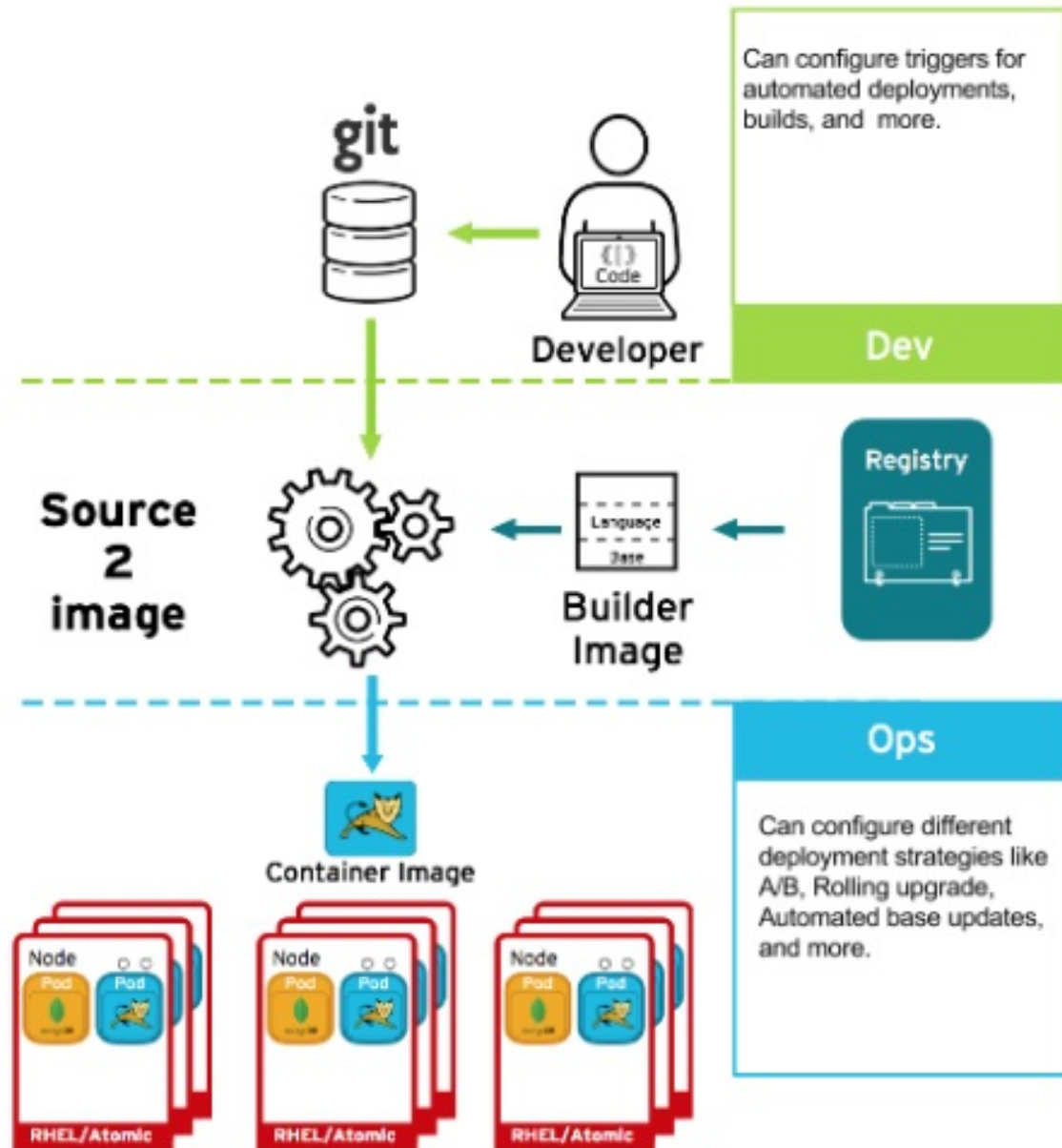
- [Understanding image builds](#)
- [Triggering and modifying builds](#)
- [Creating build inputs](#)
- [Input secrets and config maps](#)
- [OpenShift Serverless overview](#)
- [Viewing application composition using the Topology view](#)

2.9. DEPLOYING CONTAINERS

You can use a variety of techniques to make sure that the containers you deploy hold the latest production-quality content and that they have not been tampered with, such as setting up build triggers and using signatures.

2.9.1. Controlling container deployments with triggers

If something happens during the build process, or if a vulnerability is discovered after an image has been deployed, you can use tool for automated, policy-based deployment to remediate. You can use triggers to rebuild and replace images, ensuring the immutable containers process, instead of patching running containers, which is not recommended.



For example, you build an application by using three container image layers: core, middleware, and applications. An issue is discovered in the core image and that image is rebuilt. After the build is complete, the image is pushed to your OpenShift Container Registry. OpenShift Container Platform detects that the image has changed and automatically rebuilds and deploys the application image, based on the defined triggers. This change incorporates the fixed libraries and ensures that the production code is identical to the most current image.

You can use the **oc set triggers** command to set a deployment trigger. For example, to set a trigger for a deployment called deployment-example:

```
$ oc set triggers deploy/deployment-example \
  --from-image=example:latest \
  --containers=web
```

2.9.2. Controlling what image sources can be deployed

OpenShift Container Platform enables cluster administrators to apply security policy that is broad or narrow, reflecting deployment environment and security requirements.

It is important that the intended images are actually being deployed, that the images including the contained content are from trusted sources, and they have not been altered. Cryptographic signing provides this assurance. OpenShift Container Platform enables cluster administrators to apply security policy that is broad or narrow, reflecting deployment environment and security requirements. Two parameters define this policy:

- one or more registries, with optional project namespace
- trust type, such as accept, reject, or require public key(s)

You can use these policy parameters to allow, deny, or require a trust relationship for entire registries, parts of registries, or individual images. Using trusted public keys, you can ensure that the source is cryptographically verified. The policy rules apply to nodes. Policy might be applied uniformly across all nodes or targeted for different node workloads (for example, build, zone, or environment).

Example image signature policy file

```
{
  "default": [{"type": "reject"}],
  "transports": {
    "docker": {
      "access.redhat.com": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPath": "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release"
        }
      ]
    },
    "atomic": {
      "172.30.1.1:5000/openshift": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPath": "/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release"
        }
      ],
      "172.30.1.1:5000/production": [
        {
          "type": "signedBy",
          "keyType": "GPGKeys",
          "keyPath": "/etc/pki/example.com/pubkey"
        }
      ],
      "172.30.1.1:5000": [{"type": "reject"}]
    }
  }
}
```

The policy can be saved onto a node as **/etc/containers/policy.json**. Saving this file to a node is best accomplished using a new **MachineConfig** object. This example enforces the following rules:

- Require images from the Red Hat Registry (**registry.access.redhat.com**) to be signed by the Red Hat public key.

- Require images from your OpenShift Container Registry in the **openshift** namespace to be signed by the Red Hat public key.
- Require images from your OpenShift Container Registry in the **production** namespace to be signed by the public key for **example.com**.
- Reject all other registries not specified by the global **default** definition.

2.9.3. Using signature transports

You can use a signature transport as a way to store and retrieve the binary signature blob.

Signature transports can be either of the following types:

- **atomic**: Managed by the OpenShift Container Platform API.
- **docker**: Served as a local file or by a web server.

The OpenShift Container Platform API manages signatures that use the **atomic** transport type. You must store the images that use this signature type in your OpenShift Container Registry. Because the docker or distribution **extensions** API auto-discovers the image signature endpoint, no additional configuration is required.

Signatures that use the **docker** transport type are served by local file or web server. These signatures are more flexible; you can serve images from any container image registry and use an independent server to deliver binary signatures.

However, the **docker** transport type requires additional configuration. You must configure the nodes with the Uniform Resource Identifier (URI) of the signature server by placing arbitrarily-named YAML files into a directory on the host system, **/etc/containers/registries.d** by default. The YAML configuration files contain a registry URI and a signature server URI, or *sigstore*:

Example registries.d file

```
docker:
  access.redhat.com:
    sigstore: https://access.redhat.com/webassets/docker/content/sigstore
```

In this example, the Red Hat Registry, **access.redhat.com**, is the signature server that provides signatures for the **docker** transport type. Its URI is defined in the **sigstore** parameter. You might name this file **/etc/containers/registries.d/redhat.com.yaml** and use the Machine Config Operator to automatically place the file on each node in your cluster. No service restart is required since policy and **registries.d** files are dynamically loaded by the container runtime.

2.9.4. Creating secrets and config maps

You can use the **Secret** object type to provide a mechanism to hold sensitive information such as passwords, OpenShift Container Platform client configuration files, **dockercfg** files, and private source repository credentials. Secrets decouple sensitive content from pods.

You can mount secrets into containers by using a volume plugin or the system can use secrets to perform actions on behalf of a pod.

Config maps are similar to secrets, but are designed to support working with strings that do not contain sensitive information. The **ConfigMap** object holds key-value pairs of configuration data that can be consumed in pods or used to store configuration data for system components such as controllers.

For example, to add a secret to your deployment so that it can access a private image repository, use the following procedure.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Create a new project.
3. Navigate to **Resources** → **Secrets** and create a new secret. Set **Secret Type** to **Image Secret** and **Authentication Type** to **Image Registry Credentials** to enter credentials for accessing a private image repository.
4. When creating a deployment (for example, from the **Add to Project** → **Deploy Image** page), set the **Pull Secret** to your new secret.

2.9.5. Automating continuous deployment

You can integrate your own continuous deployment (CD) tooling with OpenShift Container Platform.

By leveraging continuous integration and continuous deployment (CI/CD) and OpenShift Container Platform, you can automate the process of rebuilding the application to incorporate the latest fixes, testing, and ensuring that it is deployed everywhere within the environment.

Additional resources

- [Input secrets and config maps](#)

2.10. SECURING THE CONTAINER PLATFORM

To make your OpenShift Container Platform cluster more secure, you should understand the security enhancements you can make to APIs used by OpenShift Container Platform. OpenShift Container Platform and Kubernetes APIs are key to automating container management at scale. APIs are used to:

- Validate and configure the data for pods, services, and replication controllers.
- Perform project validation on incoming requests and start triggers on other major system components.

Security-related features in OpenShift Container Platform that are based on Kubernetes include:

- Multitenancy, which combines Role-Based Access Controls and network policies to isolate containers at multiple levels.
- Admission plugins, which form boundaries between an API and those making requests to the API.

OpenShift Container Platform uses Operators to automate and simplify the management of Kubernetes-level security features.

2.10.1. Isolating containers with multitenancy

You can configure multitenancy to allow applications on an OpenShift Container Platform cluster that are owned by multiple users, and run across multiple hosts and namespaces, to remain isolated from each other and from outside attacks.

You obtain multitenancy by applying role-based access control (RBAC) to Kubernetes namespaces.

In Kubernetes, *namespaces* are areas where applications can run in ways that are separate from other applications. OpenShift Container Platform uses and extends namespaces by adding extra annotations, including MCS labeling in SELinux, and identifying these extended namespaces as *projects*. Within the scope of a project, users can maintain their own cluster resources, including service accounts, policies, constraints, and various other objects.

RBAC objects are assigned to projects to authorize selected users to have access to those projects. That authorization takes the form of rules, roles, and bindings:

- Rules define what a user can create or access in a project.
- Roles are collections of rules that you can bind to selected users or groups.
- Bindings define the association between users or groups and roles.

Local RBAC roles and bindings attach a user or group to a particular project. Cluster RBAC can attach cluster-wide roles and bindings to all projects in a cluster. There are default cluster roles that can be assigned to provide **admin**, **basic-user**, **cluster-admin**, and **cluster-status** access.

2.10.2. Protecting control plane with admission plugins

Where RBAC controls access rules between users and groups and available projects, you can define access to the OpenShift Container Platform master API by using *admission plugins*.

API requests go through a chain of rules that consist of the following admission plugins:

- Default admissions plugins: These implement a default set of policies and resources limits that are applied to components of the OpenShift Container Platform control plane.
- Mutating admission plugins: These plugins dynamically extend the admission chain. They call out to a webhook server and can both authenticate a request and modify the selected resource.
- Validating admission plugins: These validate requests for a selected resource and can both validate the request and ensure that the resource does not change again.

API requests go through admissions plugins in a chain, with any failure along the way causing the request to be rejected. Each admission plugin is associated with particular resources and only responds to requests for those resources.

2.10.2.1. Security context constraints (SCCs)

You can use *security context constraints* (SCCs) to define a set of conditions that a pod must run with to be accepted into the system.

Some aspects that can be managed by SCCs include:

- Running of privileged containers
- Capabilities a container can request to be added
- Use of host directories as volumes

- SELinux context of the container
- Container user ID

If you have the required permissions, you can adjust the default SCC policies to be more permissive, if required.

2.10.2.2. Granting roles to service accounts

You can assign roles to service accounts, in the same way that users are assigned role-based access. There are three default service accounts created for each project. A service account:

- is limited in scope to a particular project
- derives its name from its project
- is automatically assigned an API token and credentials to access the OpenShift Container Registry

Service accounts associated with platform components automatically have their keys rotated.

2.10.3. Authentication and authorization

2.10.3.1. Controlling access using OAuth

You can use API access control through authentication and authorization for securing your container platform. The OpenShift Container Platform master includes a built-in OAuth server. Users can obtain OAuth access tokens to authenticate themselves to the API.

As an administrator, you can configure OAuth to authenticate using an *identity provider*, such as LDAP, GitHub, or Google. The identity provider is used by default for new OpenShift Container Platform deployments, but you can configure this at initial installation time or postinstallation.

2.10.3.2. API access control and management

Applications can have multiple, independent API services which have different endpoints that require management. OpenShift Container Platform includes a containerized version of the 3scale API gateway so that you can manage your APIs and control access.

3scale gives you a variety of standard options for API authentication and security, which can be used alone or in combination to issue credentials and control access: standard API keys, application ID and key pair, and OAuth 2.0.

You can restrict access to specific endpoints, methods, and services and apply access policy for groups of users. Application plans allow you to set rate limits for API usage and control traffic flow for groups of developers.

2.10.3.3. Red Hat Single Sign-On

The Red Hat Single Sign-On server enables you to secure your applications by providing web single sign-on capabilities based on standards, including SAML 2.0, OpenID Connect, and OAuth 2.0. The server can act as a SAML or OpenID Connect-based identity provider (IdP), mediating with your enterprise user directory or third-party identity provider for identity information and your applications using standards-based tokens. You can integrate Red Hat Single Sign-On with LDAP-based directory services including Microsoft Active Directory and Red Hat Enterprise Linux Identity Management.

2.10.3.4. Secure self-service web console

OpenShift Container Platform provides a self-service web console to ensure that teams do not access other environments without authorization. OpenShift Container Platform ensures a secure multitenant master by providing the following:

- Access to the master uses Transport Layer Security (TLS)
- Access to the API Server uses X.509 certificates or OAuth access tokens
- Project quota limits the damage that a rogue token could do
- The etcd service is not exposed directly to the cluster

2.10.4. Managing certificates for the platform

OpenShift Container Platform has multiple components within its framework that use REST-based HTTPS communication leveraging encryption via TLS certificates. You can configure these certificates during installation.

There are some primary components that generate this traffic:

- masters (API server and controllers)
- etcd
- nodes
- registry
- router

2.10.4.1. Configuring custom certificates

You can configure custom serving certificates for the public hostnames of the API server and web console during initial installation or when redeploying certificates. You can also use a custom CA.

Additional resources

- [Introduction to OpenShift Container Platform](#)
- [Using RBAC to define and apply permissions](#)
- [About admission plugins](#)
- [Managing security context constraints](#)
- [SCC reference commands](#)
- [Examples of granting roles to service accounts](#)
- [Configuring the internal OAuth server](#)
- [Understanding identity provider configuration](#)
- [Certificate types and descriptions](#)

- [Proxy certificates](#)
- [Running APIcast on Red Hat OpenShift](#)

2.11. SECURING NETWORKS

You can manage network security at several levels, such as by using network namespaces and network policies.

At the pod level, network namespaces can prevent containers from seeing other pods or the host system by restricting network access. Network policies give you control over allowing and rejecting connections. You can manage ingress and egress traffic to and from your containerized applications.

2.11.1. Using network namespaces

You can use software-defined networking (SDN) in OpenShift Container Platform to give a unified cluster network that enables communication between containers across the cluster.

Network policy mode, by default, makes all pods in a project accessible from other pods and network endpoints. To isolate one or more pods in a project, you can create **NetworkPolicy** objects in that project to indicate the allowed incoming connections. Using multitenant mode, you can provide project-level isolation for pods and services.

2.11.2. Isolating pods with network policies

Using *network policies*, you can isolate pods from each other in the same project. Network policies can deny all network access to a pod, only allow connections for the Ingress Controller, reject connections from pods in other projects, or set similar rules for how networks behave.

Additional resources

- [About network policy](#)

2.11.3. Using multiple pod networks

Each running container has only one network interface by default. You can use the Multus CNI plugin to create multiple CNI networks, and then attach any of those networks to a pod. In that way, you can do things such as separate private data onto a more restricted network and have multiple network interfaces on each node.

Additional resources

- [Using multiple networks](#)

2.11.4. Isolating applications

You can segment network traffic on a single cluster to make multitenant clusters that isolate users, teams, applications, and environments from non-global resources.

2.11.5. Securing ingress traffic

There are many security implications related to how you configure access to your Kubernetes services from outside of your OpenShift Container Platform cluster.

In addition to exposing HTTP and HTTPS routes, ingress routing allows you to set up **NodePort** or **LoadBalancer** ingress types. NodePort exposes an application's service API object from each cluster worker. LoadBalancer lets you assign an external load balancer to an associated service API object in your OpenShift Container Platform cluster.

Additional resources

- [Configuring ingress cluster traffic](#)

2.11.6. Securing egress traffic

A cluster administrator can control egress traffic by using either a router or firewall method. For example, you can use the IP allow list to control database access. A cluster administrator can assign one or more egress IP addresses to a project by configuring an egress IP address.

Likewise, a cluster administrator can prevent egress traffic from going outside of an OpenShift Container Platform cluster by using an egress firewall.

By assigning a fixed egress IP address, you can have all outgoing traffic assigned to that IP address for a particular project. With the egress firewall, you can prevent a pod from connecting to an external network, prevent a pod from connecting to an internal network, or limit a pod's access to specific internal subnets.

Additional resources

- [Configuring an egress firewall for a project](#)
- [Configuring IPsec encryption](#)

2.12. SECURING ATTACHED STORAGE

You should understand how OpenShift Container Platform secures attached storage to protect persistent data in containerized workloads. OpenShift Container Platform uses Security-Enhanced Linux (SELinux) capabilities, group ID (GID) annotations, and Container Storage Interface (CSI)-compliant storage providers to isolate storage access and prevent unauthorized data exposure.

2.12.1. Persistent volume plugins

Containers are useful for both stateless and stateful applications. Protecting attached storage is a key element of securing stateful services. Using the Container Storage Interface (CSI), OpenShift Container Platform can incorporate storage from any storage back end that supports the CSI interface.

OpenShift Container Platform provides plugins for multiple types of storage, including:

- Red Hat OpenShift Data Foundation *
- AWS Elastic Block Stores (EBS) *
- AWS Elastic File System (EFS) *
- Azure Disk *
- Azure File *
- OpenStack Cinder *

- Google Compute Engine (GCE) Persistent Disks *
- VMware vSphere *
- Network File System (NFS)
- FlexVolume
- Fibre Channel
- Internet Small Computer Systems Interface (iSCSI)

Plugins for those storage types with dynamic provisioning are marked with an asterisk (*). Data in transit is encrypted via HTTPS for all OpenShift Container Platform components communicating with each other.

You can mount a persistent volume (PV) on a host in any way supported by your storage type. Different types of storage have different capabilities and each PV's access modes are set to the specific modes supported by that particular volume.

For example, NFS can support multiple read/write clients, but a specific NFS PV might be exported on the server as read-only. Each PV has its own set of access modes describing that specific PV's capabilities, such as **ReadWriteOnce**, **ReadOnlyMany**, and **ReadWriteMany**.

2.12.2. Shared storage

For shared storage providers such as Network File System (NFS), the persistent volume (PV) registers its group ID (GID) as an annotation on the PV resource.

Then, when the PV is claimed by the pod, the annotated GID is added to the supplemental groups of the pod, giving that pod access to the contents of the shared storage.

2.12.3. Block storage

For block storage providers such as AWS Elastic Block Store (EBS), Google Compute Engine (GCE) Persistent Disks, and Internet Small Computer Systems Interface (iSCSI), OpenShift Container Platform uses Security-Enhanced Linux (SELinux) capabilities to secure the root of the mounted volume for non-privileged pods, making the mounted volume owned by and only visible to the container with which it is associated.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Dynamic provisioning](#)
- [Persistent storage using NFS](#)
- [Persistent storage using AWS Elastic Block Store](#)
- [Persistent storage using GCE Persistent Disk](#)

2.13. MONITORING CLUSTER EVENTS AND LOGS

Monitoring and auditing an OpenShift Container Platform cluster is an important part of safeguarding the cluster and its users against inappropriate usage. There are two main sources of cluster-level information that are useful for this purpose: events and logging.

2.13.1. Watching cluster events

Cluster administrators are encouraged to familiarize themselves with the **Event** resource type and review the list of system events to determine which events are of interest.

Events are associated with a namespace, either the namespace of the resource they are related to or, for cluster events, the **default** namespace. The default namespace holds relevant events for monitoring or auditing a cluster, such as node events and resource events related to infrastructure components.

The master API and **oc** command do not provide parameters to scope a listing of events to only those related to nodes. A simple approach would be to use **grep**:

```
$ oc get event -n default | grep Node
```

Example output

```
1h      20h      3      origin-node-1.example.local Node      Normal      NodeHasDiskPressure ...
```

A more flexible approach is to output the events in a form that other tools can process. For example, the following example uses the **jq** tool against JSON output to extract only **NodeHasDiskPressure** events:

```
$ oc get events -n default -o json \
  | jq '.items[] | select(.involvedObject.kind == "Node" and .reason == "NodeHasDiskPressure")'
```

Example output

```
{
  "apiVersion": "v1",
  "count": 3,
  "involvedObject": {
    "kind": "Node",
    "name": "origin-node-1.example.local",
    "uid": "origin-node-1.example.local"
  },
  "kind": "Event",
  "reason": "NodeHasDiskPressure",
  ...
}
```

Events related to resource creation, modification, or deletion can also be good candidates for detecting misuse of the cluster. The following query, for example, can be used to look for excessive pulling of images:

```
$ oc get events --all-namespaces -o json \
  | jq '[.items[] | select(.involvedObject.kind == "Pod" and .reason == "Pulling")] | length'
```

Example output

**NOTE**

When a namespace is deleted, its events are deleted as well. Events can also expire and are deleted to prevent filling up etcd storage. Events are not stored as a permanent record and frequent polling is necessary to capture statistics over time.

2.13.2. Logging

Using the **oc log** command, you can view container logs, build configs and deployments in real time. Different users can have access different access to logs:

- Users who have access to a project are able to see the logs for that project by default.
- Users with admin roles can access all container logs.

To save your logs for further audit and analysis, you can enable the **cluster-logging** add-on feature to collect, manage, and view system, container, and audit logs. You can deploy, manage, and upgrade OpenShift Logging through the OpenShift Elasticsearch Operator and Red Hat OpenShift Logging Operator.

2.13.3. Audit logs

With *audit logs*, you can follow a sequence of activities associated with how a user, administrator, or other OpenShift Container Platform component is behaving. API audit logging is done on each server.

Additional resources

- [List of system events](#)
- [Viewing audit logs](#)

CHAPTER 3. CONFIGURING CERTIFICATES

3.1. REPLACING THE DEFAULT INGRESS CERTIFICATE

To allow external clients to connect securely to applications under the `.apps` subdomain in OpenShift Container Platform, you can replace the default wildcard ingress certificate with one issued by a trusted public CA.

3.1.1. Understanding the default ingress certificate

You can replace the default ingress certificate with a certificate from a public CA so that external clients connect securely to your applications.

The default ingress certificate in OpenShift Container Platform is a wildcard certificate that the Ingress Operator issues from an internal CA for the web console, CLI, and applications under the `.apps` subdomain.

3.1.2. Replacing the default ingress certificate

To secure the web console, CLI, and all applications under the `.apps` subdomain in OpenShift Container Platform, you can replace the default ingress certificate by creating a TLS secret with your wildcard certificate and updating the Ingress Controller and cluster proxy configuration.



NOTE

Before using the procedure, ensure you understand the following Ingress Controller behaviors:

- When certificates are renewed or rotated by using external certificate management tools, only the contents of the secret, such as the certificate and key, are updated. The secret name remains unchanged. Kubelet automatically propagates these updates to the mounted volume, allowing the router to detect the file changes and hot-reload the new certificate and key. As a result, no rolling update of the router deployment is triggered or required.
- For secret renewal or rotation, the cert-manager Operator changes the secret content, such as a cert/key pair, but does not change the secret name. This happens because kubelet automatically propagates changes to the secret in the volume mount. The router pod detects the file change and then hot reloads the new cert/key pair. Updating the secret content does not trigger rolling update.

Prerequisites

- You must have a wildcard certificate for the fully qualified `.apps` subdomain and its corresponding private key. Each should be in a separate PEM format file.
- The private key must be unencrypted. If your key is encrypted, decrypt it before importing it into OpenShift Container Platform.
- The certificate must include the `subjectAltName` extension showing `*.apps.<clustername>.<domain>`.

- The certificate file can contain one or more certificates in a chain. The file must list the wildcard certificate as the first certificate, followed by other intermediate certificates, and then ending with the root CA certificate.
- Copy the root CA certificate into an additional PEM format file.
- Verify that all certificates which include **-----END CERTIFICATE-----** also end with one carriage return after that line.

Procedure

1. Create a config map that includes only the root CA certificate that is used to sign the wildcard certificate:

```
$ oc create configmap custom-ca \
  --from-file=ca-bundle.crt=</path/to/example-ca.crt> \
  -n openshift-config
```

where

</path/to/example-ca.crt>

The path to the root CA certificate file on your local file system. For example, **/etc/pki/ca-trust/source/anchors**.

2. Update the cluster-wide proxy configuration with the newly created config map:

```
$ oc patch proxy/cluster \
  --type=merge \
  --patch='{"spec":{"trustedCA":{"name":"custom-ca"}}}'
```



NOTE

If you update only the trusted CA for your cluster, the MCO updates the **/etc/pki/ca-trust/source/anchors/openshift-config-user-ca-bundle.crt** file and the Machine Config Controller (MCC) applies the trusted CA update to each node so that a node reboot is not required. However, with these changes, the Machine Config Daemon (MCD) restarts critical services on each node, such as kubelet and CRI-O. These service restarts cause each node to briefly enter the **NotReady** state until the service is fully restarted.

If you change any other parameter in the **openshift-config-user-ca-bundle.crt** file, such as **noproxy**, the MCO reboots each node in your cluster.

3. Create a secret that contains the wildcard certificate chain and key:

```
$ oc create secret tls <secret> \
  --cert=</path/to/cert.crt> \
  --key=</path/to/cert.key> \
  -n openshift-ingress
```

where:

<secret>

Specifies the name of the secret that will contain the certificate chain and private key.

</path/to/cert.crt>

Specifies the path to the certificate chain on your local file system.

</path/to/cert.key>

Specifies the path to the private key associated with this certificate.

4. Update the Ingress Controller configuration with the newly created secret:

```
$ oc patch ingresscontroller.operator default \
  --type=merge -p \
  '{"spec":{"defaultCertificate": {"name": "<secret>"}}} \
  -n openshift-ingress-operator
```

- **<secret>::** Specifies the name used for the secret. Replace **<secret>** with the name used for the secret.

3.1.3. Additional resources

- [Replacing the CA Bundle certificate](#)
- [Proxy certificate customization](#)

3.2. ADDING API SERVER CERTIFICATES

To allow clients outside of the cluster to verify the API server's certificate, you can replace the default API server certificate with one that is issued by a CA that clients trust.

By default, the API server certificate is issued by an internal OpenShift Container Platform cluster CA. As a result, clients outside of the cluster cannot verify the API server's certificate.



NOTE

In hosted control plane clusters, you can add as many custom certificates to your Kubernetes API Server as you need. However, do not add a certificate for the endpoint that worker nodes use to communicate with the control plane. For more information, see [Configuring a custom API server certificate in a hosted cluster](#).

3.2.1. Adding an API server named certificate for the first time

The default API server certificate is issued by an internal OpenShift Container Platform cluster Certificate Authority (CA). You can add alternative certificates that the API server will return based on the fully qualified domain name (FQDN) requested by the client, for example when a reverse proxy or load balancer is used.



NOTE

Adding a custom API server named certificate for the first time triggers the **kube-apiserver-operator** to roll out a new revision of the API server pods. Node reboots are not required.

Prerequisites

- You must have a certificate for the FQDN and its corresponding private key. Each should be in a separate PEM format file.
- The private key must be unencrypted.
- The certificate must include the **subjectAltName** extension showing the FQDN.
- The certificate file can contain one or more certificates in a chain. The certificate for the API server FQDN must be the first certificate in the file, followed by intermediate certificates, and ending with the root CA certificate.



WARNING

Do not provide a named certificate for the internal load balancer (host name **api-int.<cluster_name>.<base_domain>**). Doing so will leave your cluster in a degraded state.

Procedure

1. Log in to the CLI as the **kubeadmin** user:

```
$ oc login -u kubeadmin -p <password> https://<fqdn>:6443
```

where:

<password>

Specifies your cluster administrative password.

<fqdn>

Specifies the fully qualified domain name of the internal cluster API endpoint.

2. Create a secret that contains the certificate chain and private key in the **openshift-config** namespace:

```
$ oc create secret tls <secret_name> \
  --cert=<path_to_certificate_file> \
  --key=<path_to_private_key_file> \
  -n openshift-config
```

where:

<secret_name>

Specifies the name of the new secret resource that will contain the cryptographic key pair.

<path_to_certificate_file>

Specifies the absolute local path to your custom certificate chain file.

<path_to_private_key_file>

Specifies the absolute local path to the unencrypted private key file associated with the certificate.

3. Update the API server to reference the created secret resource:

```
$ oc patch apiserver cluster --type=merge -p '
{
  "spec": {
    "servingCerts": {
      "namedCertificates": [
        {
          "names": ["<fqdn>"],
          "servingCertificate": {
            "name": "<secret_name>"
          }
        }
      ]
    }
  }
}
```

where:

<fqdn>

Specifies the fully qualified domain name for which the API server serves this custom certificate. Do not include a port number.

<secret_name>

Specifies the name of the secret you created in the previous step.

4. Verify that a new revision of the Kubernetes API server rolls out by checking the operator status:

```
$ oc get clusteroperators kube-apiserver
```



NOTE

The **PROGRESSING** status column will change to **True** while the API server operator deploys the new pod revision configured with your custom certificate. Do not interrupt the process or apply additional configuration updates while the rollout is underway. Continue only after the status returns to **False** and **AVAILABLE** reads **True**.

3.2.2. Updating or renewing an existing API server named certificate

Update or renew an expired or expiring named certificate that has already been configured in your cluster to avoid API availability issues. The API server pods dynamically detect and reload the updated certificate asset without disruption.



NOTE

When an existing API server named certificate is renewed by updating its corresponding secret, a new revision of the Kubernetes API server pods does not roll out. Node reboots are not required.

**WARNING**

If the renewed certificate is signed by a different root CA than the previous certificate, internal applications or custom pods that communicate with the API server might encounter X509 certificate validation errors. If these client workloads do not automatically hot-reload their truststores, you must manually restart them to force them to pick up the new certificate chain.

Prerequisites

- You have the renewed certificate chain and private key files in PEM format.
- The secret containing the old certificate already exists in the **openshift-config** namespace and is actively referenced by the **apiserver/cluster** configuration.

Procedure

1. Log in to the CLI as the **kubeadmin** user.
2. Update the existing secret resource in the **openshift-config** namespace with the newly issued certificate and private key:

```
$ oc create secret tls <existing_secret_name> \
  --cert=<path_to_new_cert>.cert \
  --key=<path_to_new_key>.key \
  -n openshift-config \
  --dry-run=client -o yaml | oc replace -f -
```

where:

<existing_secret_name>

Specifies the target name of the existing active secret that you are replacing.

<path_to_new_cert>.cert

Specifies the absolute local file system path to the renewed certificate chain file.

<path_to_new_key>.key

Specifies the absolute local file system path to the corresponding unencrypted private key file.

3. Verify that the **kube-apiserver** pods successfully hot-reload the updated assets without initiating a new cluster deployment revision:

```
$ oc get clusteroperators kube-apiserver
```

Confirm that the **PROGRESSING** status column remains **False**. If the status changes to **True**, verify that your underlying **apiserver/cluster** resource parameters were not modified structural layout changes during the substitution.

3.3. SECURING SERVICE TRAFFIC USING SERVICE SERVING CERTIFICATE SECRETS

Service serving certificates provide automatic TLS encryption for service-to-service communication. Configure certificates for services, ConfigMaps, APIServices, CRDs, and webhooks to secure internal cluster traffic.

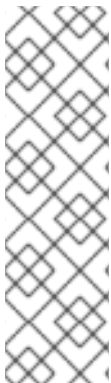
3.3.1. Understanding service serving certificates

Service serving certificates are TLS web server certificates that OpenShift Container Platform issues for middleware applications that require encryption. The **service-ca** controller stores the certificate and key in a secret and automatically replaces them near expiration.

The **service-ca** controller uses the **x509.SHA256WithRSA** signature algorithm to generate service certificates.

The generated certificate and key are in PEM format, stored in **tls.crt** and **tls.key** respectively, within a created secret. The certificate and key are automatically replaced when they get close to expiration.

The service Certificate Authority (CA) certificate, which issues the service certificates, is valid for 26 months and is automatically rotated when there is less than 13 months validity left. After rotation, the previous service CA configuration is still trusted until its expiration. This allows a grace period for all affected services to refresh their key material before the expiration. If you do not upgrade your cluster during this grace period, which restarts services and refreshes their key material, you might need to manually restart services to avoid failures after the previous service CA expires.



NOTE

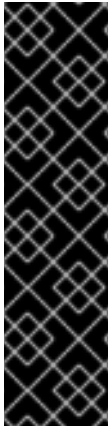
You can use the following command to manually restart all pods in the cluster. Be aware that running this command causes a service interruption, because it deletes every running pod in every namespace. These pods will automatically restart after they are deleted.

```
$ for I in $(oc get ns -o jsonpath='{range .items[*]} {.metadata.name}{"\n"} {end}'); \
do oc delete pods --all -n $I; \
sleep 1; \
done
```

3.3.2. Add a service certificate

To secure internal communication to a service in OpenShift Container Platform, you can annotate the service to generate a signed serving certificate and key pair into a secret in the same namespace.

The generated certificate is only valid for the internal service DNS name **<service.name>**. **<service.namespace>.svc**, and is only valid for internal communications. If your service is a headless service (no **clusterIP** value set), the generated certificate also contains a wildcard subject in the format of ***.<service.name>.<service.namespace>.svc**.



IMPORTANT

Because the generated certificates contain wildcard subjects for headless services, you must not use the service Certificate Authority (CA) if your client must differentiate between individual pods. In this case:

- Generate individual TLS certificates by using a different CA.
- Do not accept the service CA as a trusted CA for connections that are directed to individual pods and must not be impersonated by other pods. These connections must be configured to trust the CA that was used to generate the individual TLS certificates.

Prerequisites

- You must have a service defined.

Procedure

1. Annotate the service with **service.beta.openshift.io/serving-cert-secret-name**:

```
$ oc annotate service <service_name> \
  service.beta.openshift.io/serving-cert-secret-name=<secret_name>
```

- Replace **<service_name>** with the name of the service to secure.
- **<secret_name>** will be the name of the generated secret containing the certificate and key pair.



NOTE

For convenience, it is recommended that this value be the same as **<service_name>**.

For example, use the following command to annotate the service **test1**:

```
$ oc annotate service test1 service.beta.openshift.io/serving-cert-secret-name=test1
```

2. Examine the service to confirm that the annotations are present:

```
$ oc describe service <service_name>
```

Example output

```
...
Annotations:      service.beta.openshift.io/serving-cert-secret-name: <service_name>
                  service.beta.openshift.io/serving-cert-signed-by: openshift-service-serving-
                  signer@1556850837
...
```

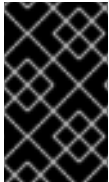
3. After the cluster generates a secret for your service, your **Pod** spec can mount it, and the pod will run after it becomes available.

Additional resources

- [Creating a re-encrypt route with a custom certificate](#)

3.3.3. Add the service CA bundle to a config map

To verify TLS connections to services that use serving certificates in OpenShift Container Platform, you can inject the service Certificate Authority (CA) certificate into a config map. Annotate the config map so that pods can mount the CA bundle from the **service-ca.crt** key.



IMPORTANT

After adding this annotation to a config map, the OpenShift Service CA Operator deletes all the data in the config map. Consider using a separate config map to contain the **service-ca.crt**, instead of using the same config map that stores your pod configuration.

Procedure

1. Annotate the config map with the **service.beta.openshift.io/inject-cabundle=true** annotation by entering the following command:

```
$ oc annotate configmap <config_map_name> \
  service.beta.openshift.io/inject-cabundle=true
```

- Replace **<config_map_name>** with the name of the config map to annotate.



NOTE

Explicitly referencing the **service-ca.crt** key in a volume mount prevents a pod from starting until the config map has been injected with the CA bundle. You can override this behavior by setting the **optional** parameter to **true** in the serving certificate configuration of the volume.

2. View the config map to ensure that the service CA bundle has been injected:

```
$ oc get configmap <config_map_name> -o yaml
```

The CA bundle is displayed as the value of the **service-ca.crt** key in the YAML output:

```
apiVersion: v1
data:
  service-ca.crt: |
    -----BEGIN CERTIFICATE-----
    ...
```

3. Mount the config map as a volume to each container that exists in a pod by configuring your **Deployment** object.

Example Deployment object that defines the volume for the mounted config map

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: my-example-custom-ca-deployment
namespace: my-example-custom-ca-ns
spec:
  ...
  spec:
    ...
    containers:
      - name: my-container-that-needs-custom-ca
        volumeMounts:
          - name: trusted-ca
            mountPath: /etc/pki/ca-trust/extracted/pem
            readOnly: true
        volumes:
          - name: trusted-ca
            configMap:
              name: <config_map_name>
              items:
                - key: ca-bundle.crt
                  path: tls-ca-bundle.pem
# ...

```

where:

<config_map_name>

Specifies the name of the config map that you annotated in an earlier step of the procedure.

ca-bundle.crt

Specifies the **ConfigMap** key. This is required.

tls-ca-bundle.pem

Specifies the **ConfigMap** path. This is required.

3.3.4. Add the service CA bundle to an API service

To allow the Kubernetes API server in OpenShift Container Platform to validate the service Certificate Authority (CA) certificate that secures an API service endpoint, you can annotate an **APIService** object to inject the service CA bundle into the **spec.caBundle** field.

Procedure

1. Annotate the API service with **service.beta.openshift.io/inject-cabundle=true**:

```
$ oc annotate apiservice <api_service_name> \
  service.beta.openshift.io/inject-cabundle=true
```

- Replace **<api_service_name>** with the name of the API service to annotate. For example, use the following command to annotate the API service **test1**:

```
$ oc annotate apiservice test1 service.beta.openshift.io/inject-cabundle=true
```

2. View the API service to ensure that the service CA bundle has been injected:

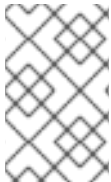
```
$ oc get apiservice <api_service_name> -o yaml
```


The CA bundle is displayed in the **spec.caBundle** field in the YAML output:

```
apiVersion: apiregistration.k8s.io/v1
kind: APIService
metadata:
  annotations:
    service.beta.openshift.io/inject-cabundle: "true"
  ...
spec:
  caBundle: <CA_BUNDLE>
  ...
```

3.3.5. Add the service CA bundle to a custom resource definition

You can annotate a **CustomResourceDefinition** (CRD) object with **service.beta.openshift.io/inject-cabundle=true** to have its **spec.conversion.webhook.clientConfig.caBundle** field populated with the service Certificate Authority (CA) bundle. This allows the Kubernetes API server to validate the service CA certificate used to secure the targeted endpoint.



NOTE

The service CA bundle will only be injected into the CRD if the CRD is configured to use a webhook for conversion. It is only useful to inject the service CA bundle if a CRD's webhook is secured with a service CA certificate.

Procedure

1. Annotate the CRD with **service.beta.openshift.io/inject-cabundle=true**:

```
$ oc annotate crd <crd_name> \
  service.beta.openshift.io/inject-cabundle=true
```

- Replace **<crd_name>** with the name of the CRD to annotate.
For example, use the following command to annotate the CRD **test1**:

```
$ oc annotate crd test1 service.beta.openshift.io/inject-cabundle=true
```

2. View the CRD to ensure that the service CA bundle has been injected:

```
$ oc get crd <crd_name> -o yaml
```

The CA bundle is displayed in the **spec.conversion.webhook.clientConfig.caBundle** field in the YAML output:

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  annotations:
    service.beta.openshift.io/inject-cabundle: "true"
  ...
spec:
  conversion:
    strategy: Webhook
```

```
webhook:
  clientConfig:
    caBundle: <CA_BUNDLE>
...
```

3.3.6. Add the service CA bundle to a mutating webhook configuration

To allow the Kubernetes API server in OpenShift Container Platform to validate the service Certificate Authority (CA) certificate that secures a mutating webhook endpoint, you can annotate a **MutatingWebhookConfiguration** object to inject the service CA bundle into each webhook **clientConfig.caBundle** field.



NOTE

Do not set this annotation for admission webhook configurations that need to specify different CA bundles for different webhooks. If you do, then the service CA bundle will be injected for all webhooks.

Procedure

1. Annotate the mutating webhook configuration with **service.beta.openshift.io/inject-cabundle=true**:

```
$ oc annotate mutatingwebhookconfigurations <mutating_webhook_name> \
  service.beta.openshift.io/inject-cabundle=true
```

- Replace **<mutating_webhook_name>** with the name of the mutating webhook configuration to annotate.
For example, use the following command to annotate the mutating webhook configuration **test1**:

```
$ oc annotate mutatingwebhookconfigurations test1 service.beta.openshift.io/inject-
cabundle=true
```

2. View the mutating webhook configuration to ensure that the service CA bundle has been injected:

```
$ oc get mutatingwebhookconfigurations <mutating_webhook_name> -o yaml
```

The CA bundle is displayed in the **clientConfig.caBundle** field of all webhooks in the YAML output:

```
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  annotations:
    service.beta.openshift.io/inject-cabundle: "true"
  ...
webhooks:
- myWebhook:
  - v1beta1
```

```
clientConfig:
  caBundle: <CA_BUNDLE>
...
```

3.3.7. Add the service CA bundle to a validating webhook configuration

To allow the Kubernetes API server in OpenShift Container Platform to validate the service Certificate Authority (CA) certificate that secures a validating webhook endpoint, you can annotate a **ValidatingWebhookConfiguration** object to inject the service CA bundle into each webhook **clientConfig.caBundle** field.



NOTE

Do not set this annotation for admission webhook configurations that need to specify different CA bundles for different webhooks. If you do, then the service CA bundle will be injected for all webhooks.

Procedure

1. Annotate the validating webhook configuration with **service.beta.openshift.io/inject-cabundle=true**:

```
$ oc annotate validatingwebhookconfigurations <validating_webhook_name> \
  service.beta.openshift.io/inject-cabundle=true
```

- Replace **<validating_webhook_name>** with the name of the validating webhook configuration to annotate.
For example, use the following command to annotate the validating webhook configuration **test1**:

```
$ oc annotate validatingwebhookconfigurations test1 service.beta.openshift.io/inject-
cabundle=true
```

2. View the validating webhook configuration to ensure that the service CA bundle has been injected:

```
$ oc get validatingwebhookconfigurations <validating_webhook_name> -o yaml
```

The CA bundle is displayed in the **clientConfig.caBundle** field of all webhooks in the YAML output:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  annotations:
    service.beta.openshift.io/inject-cabundle: "true"
  ...
webhooks:
- myWebhook:
  - v1beta1
  clientConfig:
    caBundle: <CA_BUNDLE>
  ...
```

3.3.8. Manually rotate the generated service certificate

To replace a generated service serving certificate in OpenShift Container Platform, you can delete the TLS secret named in the service **serving-cert-secret-name** annotation. A new secret and certificate pair are created automatically.

Prerequisites

- A secret containing the certificate and key pair must have been generated for the service.

Procedure

1. Examine the service to determine the secret containing the certificate. This is found in the **serving-cert-secret-name** annotation, as seen below.

```
$ oc describe service <service_name>
```

Example output

```
...
service.beta.openshift.io/serving-cert-secret-name: <secret>
...
```

2. Delete the generated secret for the service. This process will automatically recreate the secret.

```
$ oc delete secret <secret>
```

- Replace **<secret>** with the name of the secret from the previous step.

3. Confirm that the certificate has been recreated by obtaining the new secret and examining the **AGE**.

```
$ oc get secret <service_name>
```

Example output

```
NAME          TYPE          DATA  AGE
<service.name>  kubernetes.io/tls  2      1s
```

3.3.9. Manually rotate the service CA certificate

To refresh the service Certificate Authority (CA) certificate in OpenShift Container Platform outside the automatic renewal cycle, you can delete the **signing-key** secret in the **openshift-service-ca** namespace. Restart pods so that services use certificates signed by the new CA.

The service CA is valid for 26 months and is automatically refreshed when less than 13 months of validity remain.

**WARNING**

A manually-rotated service CA does not maintain trust with the previous service CA. You might experience a temporary service disruption until the pods in the cluster are restarted, which ensures that pods are using service serving certificates issued by the new service CA.

Prerequisites

- You must be logged in as a cluster admin.

Procedure

1. View the expiration date of the current service CA certificate by using the following command.

```
$ oc get secrets/signing-key -n openshift-service-ca \
  -o template={{index .data "tls.crt"}} \
  | base64 --decode \
  | openssl x509 -noout -enddate
```

2. Manually rotate the service CA. This process generates a new service CA which will be used to sign the new service certificates.

```
$ oc delete secret/signing-key -n openshift-service-ca
```

3. To apply the new certificates to all services, restart all the pods in your cluster. This command ensures that all services use the updated certificates.

```
$ for I in $(oc get ns -o jsonpath='{range .items[*]} {.metadata.name}{"\n"} {end}'); \
do oc delete pods --all -n $I; \
sleep 1; \
done
```

**WARNING**

This command will cause a service interruption, as it goes through and deletes every running pod in every namespace. These pods will automatically restart after they are deleted.

3.4. UPDATING THE CA BUNDLE

To trust custom certificate authorities for egress connections in OpenShift Container Platform, you can update the CA bundle by specifying custom CA certificates in the cluster-wide proxy configuration.

3.4.1. Understanding the CA Bundle certificate

Proxy certificates allow users to specify one or more custom certificate authority (CA) used by platform components when making egress connections.

The **trustedCA** field of the Proxy object is a reference to a config map that contains a user-provided trusted certificate authority (CA) bundle. This bundle is merged with the Red Hat Enterprise Linux CoreOS (RHCOS) trust bundle and injected into the trust store of platform components that make egress HTTPS calls. For example, **image-registry-operator** calls an external image registry to download images. If **trustedCA** is not specified, only the RHCOS trust bundle is used for proxied HTTPS connections. Provide custom CA certificates to the RHCOS trust bundle if you want to use your own certificate infrastructure.

The **trustedCA** field should only be consumed by a proxy validator. The validator is responsible for reading the certificate bundle from required key **ca-bundle.crt** and copying it to a config map named **trusted-ca-bundle** in the **openshift-config-managed** namespace. The namespace for the config map referenced by **trustedCA** is **openshift-config**:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-ca-bundle
  namespace: openshift-config
data:
  ca-bundle.crt: |
    -----BEGIN CERTIFICATE-----
    Custom CA certificate bundle.
    -----END CERTIFICATE-----
```

3.4.2. Replacing the CA Bundle certificate

To trust a custom certificate authority for egress connections in OpenShift Container Platform, you can replace the CA bundle by creating a config map with your root CA certificate and updating the cluster proxy configuration.

Procedure

1. Create a config map that includes the root CA certificate used to sign the wildcard certificate:

```
$ oc create configmap custom-ca \
  --from-file=ca-bundle.crt=</path/to/example-ca.crt> \
  -n openshift-config
```

</path/to/example-ca.crt> is the path to the CA certificate bundle on your local file system.

2. Update the cluster-wide proxy configuration with the newly created config map:

```
$ oc patch proxy/cluster \
  --type=merge \
  --patch='{"spec":{"trustedCA":{"name":"custom-ca"}}}'
```

3.4.3. Additional resources

- [Replacing the default ingress certificate](#)
- [Enabling the cluster-wide proxy](#)
- [Proxy certificate customization](#)

CHAPTER 4. CERTIFICATE TYPES AND DESCRIPTIONS

4.1. USER-PROVIDED CERTIFICATES FOR THE API SERVER

Review user-provided TLS certificates for the API server in OpenShift Container Platform to understand their purpose, location, management, and expiration for external client access.

4.1.1. Purpose

The API server is accessible by clients external to the cluster at **api.<cluster_name>.<base_domain>**. You might want clients to access the API server at a different hostname or without the need to distribute the cluster-managed certificate authority (CA) certificates to the clients. The administrator must set a custom default certificate to be used by the API server when serving content.

4.1.2. Location

The user-provided certificates must be provided in a **kubernetes.io/tls** type **Secret** in the **openshift-config** namespace. Update the API server cluster configuration, the **apiserver/cluster** resource, to enable the use of the user-provided certificate.

4.1.3. Management

User-provided certificates are managed by the user.

4.1.4. Expiration

API server client certificate expiration is less than five minutes.

4.1.5. Customization

Update the secret containing the user-managed certificate as needed.

4.1.6. Additional resources

- [Adding API server certificates](#)

4.2. PROXY CERTIFICATES

Proxy certificates allow platform components to trust custom certificate authorities when making egress connections. Understanding proxy certificates helps you configure secure external access for services that require custom certificate authority (CA) trust bundles.

4.2.1. Proxy certificate purpose

Proxy certificates allow platform components to trust custom certificate authorities when making egress connections. Proxy certificates allow users to specify one or more custom certificate authority (CA) certificates used by platform components when making egress connections.

The **trustedCA** field of the Proxy object is a reference to a config map that contains a user-provided trusted certificate authority (CA) bundle. This bundle is merged with the Red Hat Enterprise Linux CoreOS (RHCOS) trust bundle and injected into the **truststore** of platform components that make egress HTTPS calls. For example, **image-registry-operator** calls an external image registry to download

images. If **trustedCA** is not specified, only the RHCOS trust bundle is used for proxied HTTPS connections. Provide custom CA certificates to the RHCOS trust bundle if you want to use your own certificate infrastructure.

The **trustedCA** field should only be consumed by a proxy validator. The validator reads the certificate bundle from the required key **ca-bundle.crt**. The validator copies the bundle to a config map named **user-ca-bundle** in the **openshift-config-managed** namespace.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-ca-bundle
  namespace: openshift-config
data:
  ca-bundle.crt: |
    -----BEGIN CERTIFICATE-----
    Custom CA certificate bundle.
    -----END CERTIFICATE-----
```

Additional resources

- [Configuring the cluster-wide proxy](#)

4.2.2. Managing proxy certificates during installation

Configure proxy-trusted CA certificates during OpenShift Container Platform installation using the **additionalTrustBundle** value in the installation program configuration.

The **additionalTrustBundle** value of the installation program configuration is used to specify any proxy-trusted CA certificates during installation.

Procedure

1. View the installation program configuration file by running the following command:

```
$ cat install-config.yaml
```

Example output

```
...
proxy:
  httpProxy: http://<username:password@proxy.example.com:123/>
  httpsProxy: http://<username:password@proxy.example.com:123/>
  noProxy: <123.example.com,10.88.0.0/16>
additionalTrustBundle: |
  -----BEGIN CERTIFICATE-----
  <MY_HTTPS_PROXY_TRUSTED_CA_CERT>
  -----END CERTIFICATE-----
...
```



NOTE

Proxy certificates are managed by the system and not by users.

4.2.3. Proxy certificate location

The user-provided trust bundle is mounted into the file system of platform components that make egress HTTPS calls.

The user-provided trust bundle is represented as a config map. The config map is mounted into the file system of platform components that make egress HTTPS calls. Typically, Operators mount the config map to `/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem`, but mounting the config map is not required by the proxy. A proxy can modify or inspect the HTTPS connection. In either case, the proxy must generate and sign a new certificate for the connection.

Complete proxy support means connecting to the specified proxy and trusting any signatures the trust bundle has generated. Therefore, it is necessary to let the user specify a trusted root, such that any certificate chain connected to that trusted root is also trusted.

If you use the RHCOS trust bundle, place CA certificates in `/etc/pki/ca-trust/source/anchors`.

Additional resources

- [Using shared system certificates](#)

4.2.4. Proxy certificate expiration

The CA administrator configures the expiration term for proxy certificates before they can be used by OpenShift Container Platform or RHCOS.

The user sets the expiration term of the user-provided trust bundle.

The default expiration term is defined by the CA certificate itself. The CA administrator must configure the default expiration term for the certificate before the certificate can be used by OpenShift Container Platform or RHCOS.



NOTE

Red Hat does not monitor when CAs expire. Due to the long life of the CAs, this is generally not an issue. However, you might need to periodically update the trust bundle.

4.2.5. Services using proxy certificates

Platform components and services running on RHCOS nodes can use proxy certificates to establish trusted egress HTTPS connections.

By default, all platform components that make egress HTTPS calls use the RHCOS trust bundle. If **trustedCA** is defined, the trust certificate is also used.

Any service that is running on the RHCOS node is able to use the trust bundle of the node.

4.2.6. Proxy certificate customization

Update proxy certificates by modifying the config map referenced by **trustedCA** or by using machine configs to write CA certificates to the RHCOS trust bundle.

Updating the user-provided trust bundle consists of completing one of the following tasks:

- Updating the PEM-encoded certificates in the config map referenced by **trustedCA**

- Creating a config map in the namespace **openshift-config** that contains the new trust bundle and updating **trustedCA** to reference the name of the new config map.

The mechanism for writing CA certificates to the RHCOS trust bundle is exactly the same as writing any other file to RHCOS, which is done through the use of machine configs. When the Machine Config Operator (MCO) applies the new machine config that contains the new CA certificates, the MCO runs the **update-ca-trust** program and restarts the CRI-O service on the RHCOS nodes. This update does not require a node reboot. Restarting the CRI-O service automatically updates the trust bundle with the new CA certificates. For example:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 50-examplecorp-ca-cert
spec:
  config:
    ignition:
      version: 3.1.0
    storage:
      files:
      - contents:
          source: data:text/plain;charset=utf-8;base64,<base64_encoded_ca_certificate>
          mode: 0644
          overwrite: true
          path: /etc/pki/ca-trust/source/anchors/examplecorp-ca.crt
```

The **truststore** of machines must also support updating the **truststore** of nodes.

4.2.7. Proxy certificate renewal

No Operators can auto-renew proxy certificates on RHCOS nodes. You might need to periodically update the trust bundle manually.

There are no Operators that can auto-renew certificates on the RHCOS nodes.



NOTE

Red Hat does not monitor when CAs expire. Due to the long life of CAs, this is generally not an issue. However, you might need to periodically update the trust bundle.

4.3. SERVICE CA CERTIFICATES

Review service certificate authority (CA) certificate rotation, expiration, and Operator-managed signing in OpenShift Container Platform to plan maintenance for internal service serving certificates.

4.3.1. Purpose

service-ca is an Operator that creates a self-signed CA when an OpenShift Container Platform cluster is deployed.

4.3.2. Expiration

A custom expiration term is not supported. The self-signed CA is stored in the **service-ca/signing-key** secret. The certificate is in **tls.crt**, the private key is in **tls.key**, and the CA bundle is in **ca-bundle.crt**.

Other services can request a service serving certificate by annotating a service resource with **service.beta.openshift.io/serving-cert-secret-name: <secret name>**. In response, the Operator generates a new certificate as **tls.crt**, and a private key as **tls.key**, in the named secret. The certificate is valid for two years.

Other services can request that the CA bundle for the service CA be injected into API service or config map resources by annotating with **service.beta.openshift.io/inject-cabundle: true** to support validating certificates generated from the service CA. In response, the Operator writes the current CA bundle to the **CABundle** field of an API service or as **service-ca.crt** to a config map.

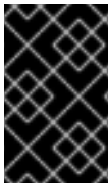
As of OpenShift Container Platform 4.3.5, automated rotation is supported and is backported to some 4.2.z and 4.3.z releases. For any release supporting automated rotation, the service CA is valid for 26 months and is automatically refreshed when there is less than 13 months validity left. If necessary, you can manually refresh the service CA.

The 26-month service CA validity period is longer than the expected upgrade interval for a supported OpenShift Container Platform cluster. Non-control plane consumers of service CA certificates are refreshed after CA rotation and before the expiration of the pre-rotation CA.



WARNING

A manually-rotated service CA does not maintain trust with the previous service CA. You might experience a temporary service disruption until the pods in the cluster are restarted, which ensures that pods are using service serving certificates issued by the new service CA.



IMPORTANT

Applications using the **service-ca** certificate must be capable of dynamically reloading CA certificates. When automated rotation occurs, pods that cannot reload CA certificates dynamically might require a restart to rebuild certificate trust.

4.3.3. Management

These certificates are managed by the system and not the user.

4.3.4. Services

Services that use service CA certificates include:

- **cluster-autoscaler-operator**
- **cluster-monitoring-operator**
- **cluster-authentication-operator**
- **cluster-image-registry-operator**

- **cluster-ingress-operator**
- **cluster-kube-apiserver-operator**
- **cluster-kube-controller-manager-operator**
- **cluster-kube-scheduler-operator**
- **cluster-networking-operator**
- **cluster-openshift-apiserver-operator**
- **cluster-openshift-controller-manager-operator**
- **cluster-samples-operator**
- **cluster-storage-operator**
- **machine-config-operator**
- **console-operator**
- **insights-operator**
- **machine-api-operator**
- **operator-lifecycle-manager**
- CSI driver operators

This is not a comprehensive list.

4.3.5. Additional resources

- [Manually rotate the service CA certificate](#)
- [Securing service traffic using service serving certificate secrets](#)

4.4. NODE CERTIFICATES

Manage node certificates in OpenShift Container Platform, including understanding their purpose for kubelet-API server communication, automatic rotation schedule, and how to manually renew the kubelet CA certificate.

Node certificates are signed by the cluster and allow the kubelet to communicate with the Kubernetes API server. They come from the kubelet CA certificate, which is generated by the bootstrap process.

The kubelet CA certificate is located in the **kube-apiserver-to-kubelet-signer** secret in the **openshift-kube-apiserver-operator** namespace.

These certificates are managed by the system and not the user and are automatically rotated after 30 days.

4.4.1. Renewing node certificates

Although the kubelet CA certificate automatically renews at 292 days, you can manually trigger renewal earlier by annotating the **kube-apiserver-to-kubelet-signer** secret.

The old CA certificate is removed after 365 days. Nodes are not rebooted when a kubelet CA certificate is renewed or removed.

Procedure

- Annotate the secret to trigger manual renewal by running the following command:

```
$ oc annotate -n openshift-kube-apiserver-operator secret kube-apiserver-to-kubelet-signer
auth.openshift.io/certificate-not-after-
```

4.4.2. Additional resources

- [Working with nodes](#)

4.5. BOOTSTRAP CERTIFICATES

You should understand how bootstrap certificates enable kubelet transport layer security (TLS) bootstrapping when nodes join a cluster, including how the certificates are issued and rotated and how the certificates are managed.

4.5.1. Purpose

The kubelet, in OpenShift Container Platform 4 and later, uses the bootstrap certificate located in **/etc/kubernetes/kubeconfig** to initially bootstrap. This is followed by the bootstrap initialization process and the authorization of the kubelet to create a certificate signing request (CSR).

In that process, the kubelet generates a CSR while communicating over the bootstrap channel. The controller manager signs the CSR, resulting in a certificate that the kubelet manages. For more information, see "Bootstrap initialization" and "Authorize kubelet to create a CSR" in the *Additional resources* section.

4.5.2. Management

These certificates are managed by the system and not the user.

4.5.3. Expiration

This bootstrap certificate is valid for 10 years.

The kubelet-managed certificate is valid for one year and rotates automatically at around the 80 percent mark of that one year.



NOTE

OpenShift Lifecycle Manager (OLM) does not update the bootstrap certificate.

4.5.4. Customization

You cannot customize the bootstrap certificates.

Additional resources

- [Bootstrap initialization](#)
- [Authorize kubelet to create a CSR](#)

4.6. ETCD CERTIFICATES

Manage etcd certificates in OpenShift Container Platform, including rotating certificates, removing unused certificate authorities, and understanding certificate types.

etcd certificates are signed by the etcd-signer. The certificates come from a certificate authority (CA) that is generated by the bootstrap process.

The CA certificates are valid for 10 years. The peer, client, and server certificates are valid for three years.

These certificates are managed only by the system and are automatically rotated.

4.6.1. etcd certificate types

Review the etcd peer, client, server, and metric certificate types and related secrets, so you know which certificate applies when configuring or troubleshooting etcd security.

etcd certificates are used for encrypted communication between etcd member peers and encrypted client traffic. The following certificates are generated and used by etcd and other processes that communicate with etcd:

- Peer certificates: Used for communication between etcd members.
- Client certificates: Used for encrypted server-client communication. Client certificates are currently only used by the API server. Except for the proxy, ensure that no other service connects to etcd directly except for the proxy. Client secrets such as **etcd-client**, **etcd-metric-client**, **etcd-metric-signer**, and **etcd-signer** are added to the **openshift-config**, **openshift-etcd**, **openshift-etcd-operator**, and **openshift-kube-apiserver** namespaces.
- Server certificates: Used by the etcd server for authenticating client requests.
- Metric certificates: All metric consumers connect to the proxy with metric-client certificates.

4.6.2. Rotating the etcd certificate

You can manually rotate the etcd certificate before its automatic, scheduled rotation by backing up and deleting the current signer certificate.

Procedure

1. Make a backup copy of the current signer certificate by running the following command:

```
$ oc get secret -n openshift-etcd etcd-signer -oyaml > signer_backup_secret.yaml
```

2. Delete the existing signer certificate by running the following command:

```
$ oc delete secret -n openshift-etcd etcd-signer
```

Verification

- Wait for the static pod roll out by running the following command. The static pod roll out can take a few minutes to complete.

```
$ oc wait --for=condition=Progressing=False --timeout=15m clusteroperator/etcd
```

4.6.3. Removing an unused certificate authority from the bundle

After a manual etcd or metrics signer rotation, delete the **etcd-ca-bundle** or **etcd-metrics-ca-bundle** as appropriate. When the cluster reconciles, unused certificate authority (CA) keys are removed. This ensures that components only trust the current signer.

Procedure

- Delete the key by running the following command:

```
$ oc delete configmap -n openshift-etcd etcd-ca-bundle
```

Verification

- Wait for the static pod rollout by running the following command. The bundle regenerates with the current signer certificate and all unknown or unused keys are deleted.

```
$ oc adm wait-for-stable-cluster --minimum-stable-period 2m
```

4.6.4. etcd certificate rotation alerts and metrics signer certificates

Monitor etcd signer expiration alerts and rotate the metrics signer using the **etcd-metric-signer** parameter and the **etcd-metrics-ca-bundle** when required, so you avoid certificate expiration and maintain secure etcd metrics traffic.

etcdSignerCAExpirationWarning

Occurs 730 days until the signer expires.

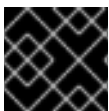
etcdSignerCAExpirationCritical

Occurs 365 days until the signer expires.

These alerts track the expiration date of the signer certificate authorities in the **openshift-etcd** namespace.

You can rotate the certificate for the following reasons:

- You receive an expiration alert.
- The private key is leaked.



IMPORTANT

When a private key is leaked, you must rotate all of the certificates.

There is a separate etcd signer for the OpenShift Container Platform metrics system. To rotate the separate etcd signer, follow the steps in "Rotating the etcd certificate" using the following parameters:

- Use the **etcd-metric-signer** parameter instead of the **etcd-signer**
- Use **etcd-metrics-ca-bundle** bundle instead of the **etcd-ca-bundle**

4.6.5. Additional resources

- [Restoring to an earlier cluster state](#)

4.7. OLM CERTIFICATES

Understand how Operator Lifecycle Manager (OLM) manages certificates for OLM components and creates and rotates certificates when installing Operators that include webhooks or API services. In proxy environments, you must manage Operator certificates yourself because OLM does not update them.

4.7.1. Management

All certificates for Operator Lifecycle Manager (OLM) components, such as **olm-operator**, **catalog-operator**, **packageserver**, and **marketplace-operator**, are managed by the system.

When installing Operators that include webhooks or API services in their **ClusterServiceVersion** (CSV) object, OLM creates and rotates the certificates for these resources. Certificates for resources in the **openshift-operator-lifecycle-manager** namespace are managed by OLM.

OLM does not update the certificates of Operators that it manages in proxy environments. These certificates must be managed by the user using the subscription config.

4.7.2. Additional resources

- [Configuring proxy support in Operator Lifecycle Manager](#)
- [Proxy certificates](#)
- [Replacing the default ingress certificate](#)
- [Updating the CA bundle](#)

4.8. AGGREGATED API CLIENT CERTIFICATES

Review aggregated API client certificate validity and automatic rotation in OpenShift Container Platform to plan maintenance for extension API server authentication.

4.8.1. Purpose

Aggregated API client certificates are used to authenticate the **KubeAPIServer** when connecting to the aggregated API servers.

4.8.2. Management

These certificates are managed by the system and not the user.

4.8.3. Expiration

This certificate authority (CA) is valid for 30 days.

The managed client certificates are valid for 30 days.

CA and client certificates are rotated automatically through the use of controllers.

4.8.4. Customization

You cannot customize the aggregated API server certificates.

4.9. MACHINE CONFIG OPERATOR CERTIFICATES

Understand Machine Config Operator (MCO) certificates used to secure node connections to the Machine Config Server (MCS) during cluster provisioning, including their lifecycle, rotation, and support boundaries.

4.9.1. Machine Config Operator certificates overview

Learn how Machine Config Operator (MCO) certificates secure node connections to the Machine Config Server (MCS) during cluster provisioning, so you can plan for certificate maintenance and for troubleshooting node provisioning issues.

This certificate authority (CA) is used to secure connections from nodes to the MCS during initial provisioning.

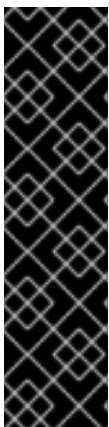
There are two certificates:

- A self-signed CA, the **machine-config-server-ca** config map (MCS CA).
- A derived certificate, the **machine-config-server-tls** secret (MCS certificate).

4.9.1.1. Provisioning details

OpenShift Container Platform installations that use Red Hat Enterprise Linux CoreOS (RHCOS) are installed by using Ignition. This process is split into two parts:

- An Ignition config is created that references a URL for the full configuration served by the MCS.
- For user-provisioned infrastructure installation methods, the Ignition config manifests as a **worker.ign** file created by the **openshift-install** command. For installer-provisioned infrastructure installation methods that use the Machine API Operator, this configuration appears as the **worker-user-data** secret.



IMPORTANT

Currently, there is no supported way to block or restrict the machine config server endpoint. The machine config server must be exposed to the network so that newly-provisioned machines, which have no existing configuration or state, are able to fetch their configuration. In this model, the root of trust is the certificate signing requests (CSR) endpoint, which is where the kubelet sends its certificate signing request for approval to join the cluster. Because of this, machine configs should not be used to distribute sensitive information, such as secrets and certificates.

To ensure that the machine config server endpoints, ports 22623 and 22624, are secured in bare metal scenarios, customers must configure proper network policies.

4.9.1.2. Provisioning chain of trust

The MCS CA is injected into the Ignition configuration under the **security.tls.certificateAuthorities** configuration field. The MCS then provides the complete configuration using the MCS certificate presented by the web server.

The client validates that the MCS certificate presented by the server has a chain of trust to an authority it recognizes. In this case, the MCS CA is that authority, and it signs the MCS certificate. This ensures that the client is accessing the correct server. The client in this case is Ignition running on a machine in the initial RAM filesystem (initramfs).

4.9.2. Machine Config Operator certificates reference

Use this reference to locate Machine Config Operator (MCO) certificate key material, rotation requirements, and support boundaries, so you can plan for certificate maintenance and for scheduling rotation before certificates expire.

4.9.2.1. Key material inside a cluster

The following objects are stored in the **openshift-machine-config-operator** namespace:

- The Machine Config Server (MCS) certificate authority (CA) bundle is stored as the **machine-config-server-ca** config map. The MCS CA bundle stores all valid CAs for the **MachineConfigServer** TLS certificate.
- The MCS CA signing key is stored as the **machine-config-server-ca** secret. The MCS CA signing key is used to sign the **MachineConfigServer** TLS certificate.
- The MCS certificate is stored as the **machine-config-server-tls** secret, which contains the **MachineConfigServer** TLS certificate and key.

The **machine-config-server-ca** config map is used in the following ways:

- The certificate controller updates the ***-user-data** secrets in the **openshift-machine-api** namespace any time the **machine-config-server-ca** configmap is updated.
- The Machine Config Operator renders the **master-user-data-managed** and **worker-user-data-managed** secrets from the **machine-config-server-ca** configmap.

4.9.2.2. Management

At this time, directly modifying either of these certificates is not supported.

4.9.2.3. Expiration

The MCS CA and MCS certificate are valid for 10 years and are automatically rotated by the MCO at 8 years.

The issued serving certificates are valid for 10 years.



NOTE

This automatic certificate rotation applies only to clusters that use machine sets. For clusters that do not use machine sets, such as vSphere user-provisioned infrastructure clusters, you are required to manually rotate these certificates. For more information on manual certificate rotation, see the Red Hat Knowledgebase article *Regenerating CA certificates for the Machine Config Server*.

4.9.2.4. Customization

You cannot customize the MCO certificates.

Additional resources

- [About the Machine Config Operator](#)
- [About the OVN-Kubernetes network plugin](#)
- [Regenerating CA certificates for the Machine Config Server](#)

4.10. USER-PROVIDED CERTIFICATES FOR DEFAULT INGRESS

Review user-provided ingress certificates in OpenShift Container Platform, including transport layer security (TLS) secret storage, **IngressController** references, and replacing Operator-generated defaults.

Use user-provided certificates for the default **IngressController** CR to complete the following tasks:

- Replace Operator-generated default certificates before production use.
- Store TLS secrets in the correct namespace.
- Reference the secret in the **IngressController** CR.

4.10.1. User-provided certificates for default ingress reference

Use user-provided default ingress certificates to allow applications on the default apps domain to present a custom TLS certificate to clients, so clients do not need to have cluster-managed certificate authority (CA) certificates installed.

4.10.1.1. Purpose

Applications are usually exposed at `<route_name>.apps.<cluster_name>.<base_domain>`. The `<cluster_name>` and `<base_domain>` come from the installation config file. `<route_name>` is the host field of the route, if specified, or the route name. For example, **hello-openshift-default.apps.username.devcluster.openshift.com**. **hello-openshift** is the name of the route and the route is in the default namespace. You might want clients to access the applications without distributing cluster-managed CA certificates to the clients. Cluster administrators must set a custom default certificate when serving application content.

**WARNING**

The Ingress Operator generates a default certificate for an **IngressController** CR to serve as a placeholder until you configure a custom default certificate. Do not use Operator-generated default certificates in production clusters.

4.10.1.2. Location

Store user-provided certificates in a **tls** type **Secret** resource in the **openshift-ingress** namespace. Update the **IngressController** CR in the **openshift-ingress-operator** namespace to enable the use of the user-provided certificate. For more information, see "Setting a custom default certificate".

4.10.1.3. Management

User-provided certificates are managed by the user.

4.10.1.4. Expiration

Expiration and renewal are managed by the user.

4.10.1.5. Services

Applications deployed on the cluster use user-provided certificates for default ingress.

4.10.1.6. Customization

Update the secret containing the user-managed certificate as needed.

4.10.2. Additional resources

- [Replacing the default ingress certificate](#)
- [Setting a custom default certificate](#)

4.11. INGRESS CERTIFICATES

Manage ingress certificates in OpenShift Container Platform, including Prometheus metrics and secured routes, secret locations, default and custom workflows, expiration, and Operator renewal.

4.11.1. Purpose

The Ingress Operator uses certificates for:

- Securing access to metrics for Prometheus.
- Securing access to routes.

4.11.2. Ingress certificate location

Locate ingress certificate secrets in OpenShift Container Platform to replace Operator defaults and verify certificates for Prometheus metrics and secured routes.

To secure access to Ingress Operator and Ingress Controller metrics, the Ingress Operator uses service serving certificates. The Operator requests a certificate from the **service-ca** controller for Operator metrics, and the **service-ca** controller puts the certificate in a secret named **metrics-tls** in the **openshift-ingress-operator** namespace. Additionally, the Ingress Operator requests a certificate for each Ingress Controller, and the **service-ca** controller puts the certificate in a secret named **router-metrics-certs-<name>**, where **<name>** is the name of the Ingress Controller, in the **openshift-ingress** namespace.

Each Ingress Controller has a default certificate that it uses for secured routes that do not specify route-specific certificates. Unless you specify a custom certificate, the Operator uses a self-signed certificate by default. The Operator uses a self-signed Operator signing certificate to sign any default certificate that it generates. The Operator generates this signing certificate and puts it in a secret named **router-ca** in the **openshift-ingress-operator** namespace. When the Operator generates a default certificate, it puts the default certificate in a secret named **router-certs-<name>**, where **<name>** is the name of the Ingress Controller, in the **openshift-ingress** namespace.



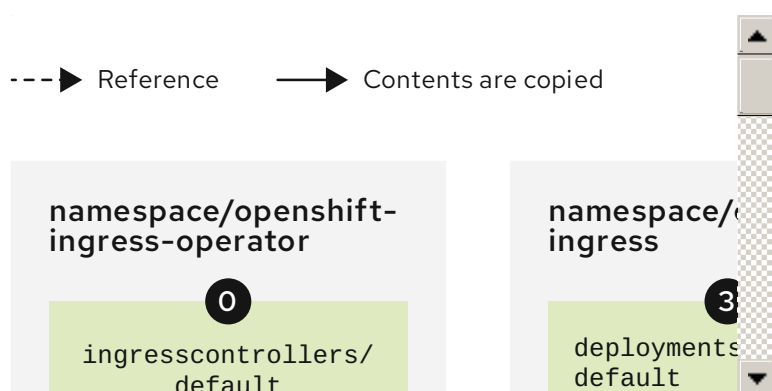
WARNING

The Ingress Operator generates a default certificate for an Ingress Controller to serve as a placeholder until you configure a custom default certificate. Do not use Operator-generated default certificates in production clusters.

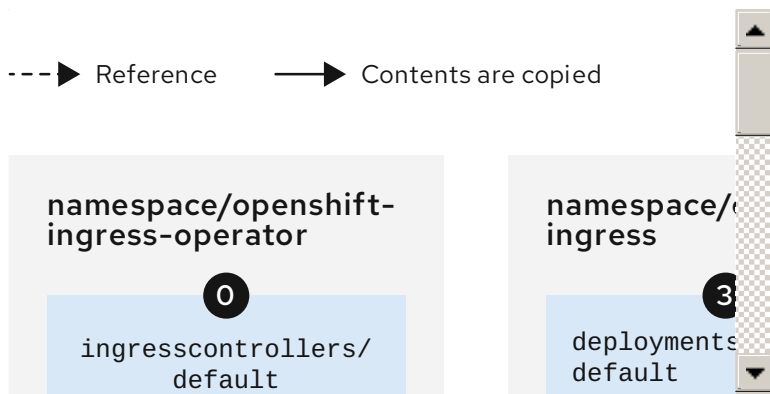
4.11.3. Ingress certificate workflow

Compare default and custom ingress certificate workflows in OpenShift Container Platform and how cluster components trust Operator serving certificates.

4.11.3.1. Custom certificate workflow



4.11.3.2. Default certificate workflow



- 0 An empty **defaultCertificate** field causes the Ingress Operator to use a self-signed certificate authority (CA) to generate a serving certificate for the specified domain.
- 1 The default CA certificate and key generated by the Ingress Operator. Used to sign Operator-generated default serving certificates.
- 2 In the default workflow, the wildcard default serving certificate, created by the Ingress Operator and signed using the generated default CA certificate. In the custom workflow, this is the user-provided certificate.
- 3 The router deployment. Uses the certificate in **secrets/router-certs-default** as its default front-end server certificate.
- 4 In the default workflow, the contents of the wildcard default serving certificate (public and private parts) are copied here to enable OAuth integration. In the custom workflow, this is the user-provided certificate.
- 5 The public (certificate) part of the default serving certificate. Replaces the **configmaps/router-ca** resource.
- 6 The user updates the cluster proxy configuration with the CA certificate that signed the **ingresscontroller** serving certificate. This enables components like **auth**, **console**, and the registry to trust the serving certificate.
- 7 The cluster-wide trusted CA bundle containing the combined Red Hat Enterprise Linux CoreOS (RHCOS) and user-provided CA bundles or an RHCOS-only bundle if a user bundle is not provided.
- 8 The custom CA certificate bundle, which instructs other components (for example, **auth** and **console**) to trust an **ingresscontroller** configured with a custom certificate.
- 9 The **trustedCA** field is used to reference the user-provided CA bundle.
- 10 The Cluster Network Operator injects the trusted CA bundle into the **proxy-ca** config map.
- 11 OpenShift Container Platform 4.21 and newer use **default-ingress-cert**.

4.11.4. Ingress certificate expiration

Review fixed two-year expiration for Ingress Operator and **service-ca** certificates in OpenShift Container Platform to plan maintenance before certificates expire.

The expiration terms for the Ingress Operator certificates are as follows:

- The expiration date for metrics certificates that the **service-ca** controller creates is two years after the date of creation.
- The expiration date for the Operator signing certificate is two years after the date of creation.
- The expiration date for default certificates that the Operator generates is two years after the date of creation.

You cannot specify custom expiration terms on certificates that the Ingress Operator or **service-ca** controller creates.

4.11.5. Ingress certificate services

Review how Prometheus, secured routes, and the Ingress Operator depend on ingress metrics and default serving certificates in OpenShift Container Platform.

Prometheus uses the certificates that secure metrics.

The Ingress Operator specifies a dedicated signing certificate to sign default certificates that it generates for Ingress Controllers for which you do not set custom default certificates.

Cluster components that use secured routes may use the default Ingress Controller default certificate.

Ingress to the cluster via a secured route uses the default certificate of the Ingress Controller by which the route is accessed unless the route specifies a route certificate.

4.11.6. Ingress certificate management and renewal

Review ingress certificate renewal in OpenShift Container Platform to learn which certificates rotate automatically and which Operator defaults you must replace.

Ingress certificates are managed by the user. For more information, see "Replacing the default ingress certificate".

The **service-ca** controller automatically rotates the certificates that it issues. However, it is possible to use **oc delete secret <secret>** to manually rotate service serving certificates.

The Ingress Operator does not rotate its own signing certificate or the default certificates that it generates. Operator-generated default certificates are intended as placeholders for custom default certificates that you configure.

4.11.7. Additional resources

- [Replacing the default ingress certificate](#)

4.12. MONITORING AND OPENSIFT LOGGING OPERATOR COMPONENT CERTIFICATES

Review service certificate authority (CA) certificates for monitoring and Red Hat OpenShift Logging Operator components in OpenShift Container Platform, including validity, automatic rotation, and system-managed namespaces.

4.12.1. Expiration

Monitoring components secure their traffic with service CA certificates. These certificates are valid for 2 years and are replaced automatically on rotation of the service CA, which is every 13 months.

If the certificate is present in the **openshift-monitoring** or **openshift-logging** namespace, it is system managed and rotated automatically.

4.12.2. Management

These certificates are managed by the system and not the user.

4.13. CONTROL PLANE CERTIFICATES

Review control plane certificate namespaces and automatic rotation in OpenShift Container Platform to plan maintenance and recover from expiration.

4.13.1. Location

Control plane certificates are included in these namespaces:

- **openshift-config-managed**
- **openshift-kube-apiserver**
- **openshift-kube-apiserver-operator**
- **openshift-kube-controller-manager**
- **openshift-kube-controller-manager-operator**
- **openshift-kube-scheduler**

4.13.2. Management

Control plane certificates are managed by the system and rotated automatically.

If control plane certificates expire, see "Recovering from expired control plane certificates".

4.13.3. Additional resources

- [Recovering from expired control plane certificates](#)

CHAPTER 5. COMPLIANCE OPERATOR

5.1. COMPLIANCE OPERATOR OVERVIEW

The OpenShift Container Platform Compliance Operator assists users by automating the inspection of numerous technical implementations and compares those against certain aspects of industry standards, benchmarks, and baselines.

The Compliance Operator is not an auditor. To be compliant or certified under these various standards, you need to engage an authorized auditor such as a Qualified Security Assessor (QSA), Joint Authorization Board (JAB), or other industry recognized regulatory authority to assess your environment.

The Compliance Operator makes recommendations based on generally available information and practices regarding such standards and may assist with remediations, but actual compliance is your responsibility. You are required to work with an authorized auditor to achieve compliance with a standard. For more information on compliance support for all Red Hat products, see "Product Compliance".

5.1.1. Compliance Operator concepts

Learn about the Compliance Operator and the custom resource definitions it uses.

[Understanding the Compliance Operator](#)

[Understanding the Custom Resource Definitions](#)

5.1.2. Compliance Operator management

You can install, update, manage, and uninstall the Compliance Operator on your cluster.

[Installing the Compliance Operator](#)

[Updating the Compliance Operator](#)

[Managing the Compliance Operator](#)

[Uninstalling the Compliance Operator](#)

5.1.3. Compliance Operator scan management

You can configure, run, tailor, and troubleshoot Compliance Operator scans, and retrieve and manage their results.

[Supported compliance profiles](#)

[Compliance Operator scans](#)

[Tailoring the Compliance Operator](#)

[Retrieving Compliance Operator raw results](#)

[Managing Compliance Operator remediation](#)

[Performing advanced Compliance Operator tasks](#)

Troubleshooting the Compliance Operator

Using the oc-compliance plugin

5.1.4. Additional resources

- [Compliance Operator release notes](#)
- [Product Compliance](#)

5.2. RELEASE NOTES FOR THE COMPLIANCE OPERATOR

The Compliance Operator lets OpenShift Container Platform administrators describe the required compliance state of a cluster and provides them with an overview of gaps and ways to remediate them.

These release notes track the development of the Compliance Operator in the OpenShift Container Platform.

5.2.1. Release notes for OpenShift Compliance Operator 1.9.2

OpenShift Compliance Operator 1.9.2 is now available. The **stable** update channel tracks and receives updates for the Compliance Operator. For more information, see [Updating the Compliance Operator](#). The following Red Hat Security Advisory (RHSA) is available:

- [RHSA-2026:54500 - OpenShift Compliance Operator 1.9.2 bug fix and enhancement update](#)

5.2.1.1. Fixed issues

- Before this release, the **compliance_operator_compliance_state** metric could report **NON-COMPLIANT** even when related **ComplianceSuite** and **ComplianceScan** results were **COMPLIANT**, which could trigger false alerts. With this release, the Compliance Operator keeps the metric in synchronization with the relevant suite and removes it when you delete that suite. For more information, see ([CMP-4373](#)).
- CVE-2026-33811 is resolved in the Compliance Operator 1.9.2 release. ([CVE-2026-33811](#))
- CVE-2026-27145 is resolved in the Compliance Operator 1.9.2 release. ([CVE-2026-27145](#))
- CVE-2026-42504 is resolved in the Compliance Operator 1.9.2 release. ([CVE-2026-42504](#))

5.2.2. Release notes for OpenShift Compliance Operator 1.9.1

Release notes for OpenShift Compliance Operator 1.9.1.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift Compliance Operator 1.9.1:

- [RHSA-2026:26571 - OpenShift Compliance Operator 1.9.1 bug fix and enhancement update](#)

5.2.2.1. Bug fixes

- Before this release, when you created a **TailoredProfile** object that extended a base profile, you could only enable rules that were available in the XCCDF groups in the base profile. With this release, **TailoredProfile** objects can enable any rule available. For more information, see ([CMP-](#)

[4283](#)).

- Before this release, the Compliance Operator rule, **ocp4-cis-file-permissions-cni-conf**, checked file permissions for every file under the **/etc/cni/net.d/** directory, including the runtime **cni.lock** file, which could cause incorrect fail results. With this release, the rule checks permissions only for CNI configuration files (**.conf**, **.conflist**, **.json**). For more information, see ([CMP-4323](#)).
- Before this release, the Compliance Operator defaulted to an **imagePullPolicy** of **Always**, which could cause unnecessary container image pulls from the registry on every pod start. With this release, the default is **IfNotPresent**. For more information, see ([CMP-4313](#)).
- Before this release, updated DISA STIG reference URLs caused the profile parser to omit STIG reference annotations on **Rule** custom resources. With this release, the parser recognizes the updated URLs and restores those annotations. For more information, see ([CMP-4333](#)).
- Before this release, updated NERC-CIP reference URLs caused the profile parser to omit NERC-CIP annotations on **Rule** custom resources. With this release, the parser recognizes the updated URLs and restores those annotations. For more information, see ([CMP-4349](#)).

5.2.3. Release notes for OpenShift Compliance Operator 1.9.0

Release notes for OpenShift Compliance Operator 1.9.0.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift Compliance Operator 1.9.0:

- [RHSA-2026:8433 - OpenShift Compliance Operator 1.9.0 bug fix and enhancement update](#)

5.2.3.1. New features and enhancements

- With this update, the Compliance Operator has extended the Common Expression Language (CEL) scanner to **Rules**, in Technology Preview status. CEL does not replace the existing Extensible Configuration Checklist Description Format (XCCDF) profiles but extends the ability to comply with custom security policies. For more information, see ([CMP-4134](#)).
- With this release, the Compliance Operator now allows custom attributes defined in **ComplianceRule** objects to be automatically propagated to the corresponding **ComplianceCheckResult** objects. For more information, see ([CMP-3974](#)).
- With this release, the Compliance Operator now supports Center for Internet Security (CIS) OpenShift benchmark version 1.9.0. Version 1.7.0 is now deprecated. For more information, see ([CMP-3520](#)).

5.2.3.2. Bug fixes

- Before this release, Compliance Operator would create an unbound **ServiceAccount** token used for metrics access. This could raise a security concern over an unused token. With this release, the unbound **ServiceAccount** token is not created. For more information, see ([CMP-3743](#)).
- Before this release, File Integrity Operator (FIO) would fail when you installed FIO before Compliance Operator (CO) and CO ran the first scan. With this release, FIO and CO run correctly when both are installed. For more information, see ([CMP-4112](#)).

- Before this release, if you configured **ScanSettings** to disable result storage and then ran a platform scan, the scan would hang and time out. Now, if you configure **ScanSettings** to disable result storage, the platform scan continues to completion. For more information, see ([CMP-4116](#)).
- Before this release, **ProfileBundle** objects could become stuck in a PENDING state indefinitely during Operator upgrades or content image changes. This would require manual intervention to resolve, such as deleting the **profileparser** deployment or restarting the Operator. With this release, the **ProfileBundle** controller now detects and automatically recovers from this condition with no user action required. This improvement is transparent and does not affect any APIs, custom resources, or configuration. For more information, see ([CMP-4117](#)).
- Before this release, Compliance Operator rules did not add the proper annotation for rules selected in CIS profiles, which resulted in absence of the annotation and results not appearing in Red Hat Advanced Cluster Security (ACS). Now, when annotations are added, the checks appear in the final ACS report and the compliance dashboard with the correct control tag. For more information, see ([CMP-4120](#)).

5.2.4. Release notes for OpenShift Compliance Operator 1.8.2

Release notes for OpenShift Compliance Operator 1.8.2.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift Compliance Operator 1.8.2:

- [RHSA-2026:1859 - OpenShift Compliance Operator 1.8.2 bug fix and enhancement update](#)

5.2.4.1. Bug fixes

- Red Hat recommends that customers upgrade to version 1.8.2 of Compliance Operator. For more information, see ([CVE-2025-68973](#)).

5.2.5. Release notes for OpenShift Compliance Operator 1.8.1

Release notes for OpenShift Compliance Operator 1.8.1.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift Compliance Operator 1.8.1:

- [RHSA-2026:0737 - OpenShift Compliance Operator 1.8.1 bug fix and enhancement update](#)

5.2.5.1. Bug fixes

- Before this release, Compliance Operator could cause a privilege escalation due to incorrect permissions on **/etc/passwd**. With this release, the permissions have been corrected. For more information, see ([CVE-2025-7195](#)).
- Previously, Compliance Operator scans using rhcos4 profile would incorrectly return **NOT-APPLICABLE** scan results when using Red Hat Enterprise Linux CoreOS (RHCOS) 10 systems. With this release, scans using rhcos4 profiles return **COMPLIANT** and **NON-COMPLIANT** results. For more information, see ([CMP-4034](#)).

5.2.6. Release notes for OpenShift Compliance Operator 1.8.0

Release notes for OpenShift Compliance Operator 1.8.0.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift Compliance Operator 1.8.0:

- [RHSA-2025:21885 - OpenShift Compliance Operator 1.8.0 bug fix and enhancement update](#)

5.2.6.1. New features and enhancements

- With this update, the Compliance Operator provides the Common Expression Language (CEL) scanner in TECH PREVIEW status. The CEL scanner implements a new **CustomRule** Custom Resource Definition (CRD) that allows administrators to define and enforce custom security policies using CEL expressions. This new content format does not replace the existing XCCDF (Extensible Configuration Checklist Description Format) profiles but extends the ability to comply with custom security policies. For more information, see ([CMP-3118](#)).
- Previously, Compliance Operator required persistent storage to save raw scan results, which presented challenges for edge deployments and environments without storage infrastructure. With this release, Compliance Operator supports running scans without persistent storage. Administrators can set **rawResultStorage.enabled: false** in **ScanSetting** resources to disable storage of scan result files, allowing compliance scans to run in storage-constrained environments such as edge deployments and single-node OpenShift. Compliance check results remain fully available through **ComplianceCheckResult** resources. Raw result storage remains enabled by default for backward compatibility. For more information, see ([CMP-1225](#)).
- Previously, Compliance Operator provided **ocp4-bsi** and **ocp4-bsi-node** profiles for BSI compliance scanning. With this release, the **rhcos4-bsi** profile is now available, extending BSI standard coverage to RHCOS systems. For more information, see ([CMP-3720](#)).
- This release removes the deprecated CIS 1.4.0, CIS 1.5.0, DISA STIG VIR1 and DISA STIG V2R1 profiles. The newer versions have replaced these obsolete profiles for customer use. For more information, see ([CMP-3712](#)).
- With this release, PCI-DSS profiles 3.2.1 and 4.0.0 are now supported on ARM architecture systems. For more information, see ([CMP-3723](#)).

5.2.6.2. Bug fixes

- With this release, automatic remediation for API server encryption now applies the appropriate encryption mode based on OpenShift version: AES-GCM for OpenShift 4.13.0 and higher versions, AES-CBC for earlier versions. Both encryption modes remain compliant across all OpenShift versions. For more information, see ([CMP-3248](#)).
- Before this release, Compliance Operator would remediate SSH settings on RHCOS hosts by deploying a fixed `sshd_config` file containing all SSH hardening settings. If the scan for corresponding rules failed, this could result in unintended configuration changes to SSH. With this release, Compliance Operator applies very specific remediations to SSH according to the ComplianceAsCode shared Kubernetes macros. For more information, see ([CMP-3553](#)).
- For prior versions of Compliance Operator, the log rotation function depended on finding the **logrotate** file in the `/etc/cron.daily` folder. With this release, Compliance Operator works with the **logrotate.timer** service. This provides reliable log rotation behavior from Compliance Operator.
- For previous versions of Compliance Operator, it is possible for the **STIG ID** to be omitted from the compliance report. These omissions were caused by missing **stigref** and **stigid** values. With

this release, the omissions have been corrected and now **STIG ID** reliably shows up in the compliance report.

- Before this release, Compliance Operator STIG control **CNTR-OS-000720** selected rule **rhcos4-audit-rules-suid-privilege-function**, but since the rule was not available in Compliance Operator, no output was generated. With this release, the rule, **rhcos4-audit-rules-suid-privilege-function** is now available in Compliance Operator and listed in the scan output. For more information, see ([CMP-3558](#)).
- In previous versions of Compliance Operator, scanning with the **ocp4-stig** profile would fail for the rule **ocp4-stig-modified-audit-log-forwarding-uses-tls** even if TLS is enabled correctly. This would occur because the **tls://** field is no longer required by the **ClusterLogForwarder** resource, causing the scan output to show an incorrect **FAIL** result. With this release, the protocol prefix is not required and the scan output produces correct results. For more information, see ([routes-protected-by-tls compliance check failing when Red Hat OpenShift Data Foundation 4.11 is installed](#)).
- Previously, there was no automated method to check if API servers were using unsupported configuration overrides as recommended by CIS Benchmark control 1.2.31 or 1.2.33. This release provides dedicated rules for checking for unsupported configuration overrides.
- For prior releases of Compliance Operator, some rules were missing a variable reference in the annotation, such as rule **resource-requests-limits**. With this release, the variable reference is available for rules and the erroneous output is eliminated. For more information, see ([CMP-3582](#)).
- Previously, the **ocp4-routes-rate-limit** rule required setting rate limits for all routes outside the **openshift** and **kube** namespaces. However, using the feature and scanning for it presented problems because other namespaces managed by critical Operators should not be modified and not be scanned for the modification by Compliance Operator. With this release, routes managed by critical Operators are not flagged as errors by the Compliance Operator.
- In prior versions of Compliance Operator, a **ComplianceScan** reported the warning **SDN not found** when the **openshift-sdn** networking provider was not found. In this release, Compliance Operator suppresses the warning when OpenShift-SDN is not the active networking provider. For more information, see ([CMP-3591](#)).
- Previously, duplicate variables could be accidentally created in **TailoredProfile** and were not correctly detected by Compliance Operator. With this release, duplicate **setValues** in **TailoredProfile** are identified and trigger a warning event from a compliance scan.
- In previous releases of Compliance Operator, the rule **ocp4-audit-log-forwarding-uses-tls** failed when the **clusterlogforwarder** output configuration contained maps without a URL key. With this release, the rule correctly filters for outputs that have a URL field, showing **PASS** when TLS is properly enabled for **clusterlogforwarder**. For more information, see ([CMP-3597](#)).
- In prior versions of Compliance Operator, for the rule **rhcos4-service-systemd-coredump-disabled**, no remediation was generated after scanning the cluster. In this release, remediation is provided for **rhcos4-service-systemd-coredump-disabled**.
- In prior versions of Compliance Operator, the rule to check the setting of **imagestream.spec.tags.importPolicy.scheduled** would return **FAIL** even when the configuration was correct. With this release, the rule now correctly excludes imagestreams managed by the samples operator and those owned by ClusterVersion, resulting in accurate compliance status reporting.

- In prior releases, Compliance Operator included outdated TLS cipher suite rules which used unsupported configuration overrides with defective remediations. With this release, these outdated rules have been removed from the default profile. Also, the **ocp4-kubelet-configure-tls-cipher-suites-ingresscontroller** rule has been renamed to **ocp4-ingress-controller-tls-cipher-suites** for better organization. For more information, see ([CMP-3606](#)).
- In prior versions of Compliance Operator, creating **ComplianceScans** directly with custom content images failed during the profile deprecation check. With this release, Compliance Operator gracefully handles cases where the **ProfileBundle** cannot be determined, logging an informational message instead of failing the scan. For more information, see ([CMP-3613](#)).
- Previously, Compliance Operator scanned incorrectly flagged passthrough routes as noncompliant with the **ocp4-routes-protected-by-tls** rule. With this release, passthrough routes are properly excluded from this rule because they delegate TLS termination to the backend application.

5.2.7. Release notes for OpenShift Compliance Operator 1.7.1

Release notes for OpenShift Compliance Operator 1.7.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.7.1:

- [RHBA-2025:14639 - OpenShift Compliance Operator 1.7.1 bug fix update](#)



NOTE

The OpenShift Compliance Operator 1.7.1 supports PCI-DSS versions 3.2.1 and 4.0.0 on IBM Z® (**s390x**) architecture.

5.2.7.1. Bug fixes

- Previously, the Compliance Operator's **pauser** container could be terminated due to running out of memory, showing the status **OOMKilled**. With this update, the memory limit for the **pauser** container is increased to prevent the error and improve overall stability. ([OCPBUGS-50924](#))

5.2.8. Release notes for OpenShift Compliance Operator 1.7.0

Release notes for OpenShift Compliance Operator 1.7.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.7.0:

- [RHBA-2025:3728 - OpenShift Compliance Operator 1.7.0 bug fix and enhancement update](#)

5.2.8.1. New features and enhancements

- A **must-gather** extension is now available for the Compliance Operator installed on **aarch64**, **x86**, **ppc64le**, and **s390x** architectures. The **must-gather** tool provides crucial configuration details to Red Hat Customer Support and engineering. For more information, see [Using the must-gather tool for the Compliance Operator](#).

- CIS Benchmark Support has been added to Compliance Operator 1.7.0. The profile supported is CIS OpenShift Benchmark 1.7.0. For more information, see ([CMP-3081](#))
- Compliance Operator is now supported on **aarch64** architecture for CIS OpenShift Benchmark 1.7.0 and FedRAMP Moderate Revision 4. For more information, see ([CMP-2960](#))
- Compliance Operator 1.7.0 now supports OpenShift DISA STIG V2R2 profiles for OpenShift and RHCOS. For more information, see ([CMP-3142](#))
- Compliance Operator 1.7.0 now supports deprecation of old, unsupported profile versions, such as deprecation of CIS 1.4 profiles, CIS 1.5 profiles, DISA STIG V1R1 profiles and DISA STIG V2R1 profiles. For more information, see ([CMP-3149](#))
- With this release of Compliance Operator 1.7.0, the deprecation of older CIS and DISA STIG profiles mean that these older profiles will no longer be supported with the appearance of Compliance Operator 1.8.0. For more information, see ([CMP-3284](#))
- With this release of Compliance Operator 1.7.0, BSI profile support is added for OpenShift. For more information, refer to the KCS article [BSI Quick Check](#) and [BSI Compliance Summary](#).

5.2.8.2. Bug fixes

- Before this release, Compliance Operator would provide an unneeded remediation recommendation due to differences in filesystem structure for the **s390x** architecture. With this release, the Compliance Operator now recognizes the differences in filesystem structure and does not provide the misleading remediation. With this update, the rule is now more clearly defined. ([OCPBUGS-33194](#))
- Previously, the instructions for rule **ocp4-etcd-unique-ca** did not work for OpenShift 4.17 and later. With this update, the instructions and actionable steps are corrected. ([OCPBUGS-42350](#))
- When using the Compliance Operator with Cluster Logging Operator (CLO) version 6.0, various rules would fail. This is due to backwards incompatible changes to the CRDs that CLO uses. The Compliance Operator relies on those CRDs to verify logging functionality. The CRDs have been corrected to support the PCI-DSS profiles with CLO. ([OCPBUGS-43229](#))
- After installing Cluster Logging Operator (CLO) 6.0, users found that the ComplianceCheckResult **ocp4-cis-audit-log-forwarding-enabled** was failing because there was a change in the API version of the **clusterlogforwarder** resource. Log collection and forwarding configurations are now specified under the new API, part of the observability.openshift.io API group. ([OCPBUGS-43585](#))
- For previous releases of Compliance Operator, the scans would generate an error log for the reconcile loop on the Operator pod. With this release, the Compliance Operator controller logic is more stable. ([OCPBUGS-51267](#))
- Previously, the rules **file-integrity-exists** or **file-integrity-notification-enabled** would fail on **aarch64** OpenShift clusters. With this update, these rules evaluate as **NOT-APPLICABLE** on **aarch64** systems. ([OCPBUGS-52884](#))
- Before this release of the Compliance Operator, the rule **kubelet-configure-tls-cipher-suites** failed for the API server ciphers, resulting in **E2E-FAILURE** status. The rule has been updated to check new ciphers from RFC 8446, which are included with OpenShift 4.18. The rule is now being evaluated correctly. ([OCPBUGS-54212](#))

- Previously, the Compliance Operator platform scan would fail and produce the message **failed to parse Ignition config**. With this release, the Compliance Operator is safe to run on 4.19 clusters, when that version of OpenShift is available to customers. ([OCPBUGS-54403](#))
- Before this release of Compliance Operator, several rules were not platform aware, creating unneeded errors. Now that the rules have been properly ported to other architectures, those rules run correctly and users can observe some Compliance Check Results reporting **NOT-APPLICABLE** appropriately, depending on the architecture they are using. ([OCPBUGS-53041](#))
- Previously, the rule **file-groupowner-ovs-conf-db-hugetlb** would fail unexpectedly. With this release, the rule fails only when this is the needed result. ([OCPBUGS-55190](#))

5.2.9. Release notes for OpenShift Compliance Operator 1.6.2

Release notes for OpenShift Compliance Operator 1.6.2.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.6.2:

- [RHBA-2025:2659 - OpenShift Compliance Operator 1.6.2 update](#)

CVE-2024-45338 is resolved in the Compliance Operator 1.6.2 release. ([CVE-2024-45338](#))

5.2.10. Release notes for OpenShift Compliance Operator 1.6.1

Release notes for OpenShift Compliance Operator 1.6.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.6.1:

- [RHBA-2024:10367 - OpenShift Compliance Operator 1.6.1 update](#)

This update includes upgraded dependencies in underlying base images.

5.2.11. Release notes for OpenShift Compliance Operator 1.6.0

Release notes for OpenShift Compliance Operator 1.6.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.6.0:

- [RHBA-2024:6761 - OpenShift Compliance Operator 1.6.0 bug fix and enhancement update](#)

5.2.11.1. New features and enhancements

- The Compliance Operator now contains supported profiles for Payment Card Industry Data Security Standard (PCI-DSS) version 4. For more information, see [Supported compliance profiles](#).
- The Compliance Operator now contains supported profiles for Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) V2R1. For more information, see [Supported compliance profiles](#).
- A **must-gather** extension is now available for the Compliance Operator installed on **x86**, **ppc64le**, and **s390x** architectures. The **must-gather** tool provides crucial configuration details

to Red Hat Customer Support and engineering. For more information, see [Using the must-gather tool for the Compliance Operator](#).

5.2.11.2. Bug fixes

- Before this release, a misleading description in the **ocp4-route-ip-whitelist** rule resulted in misunderstanding, causing potential for misconfigurations. With this update, the rule is now more clearly defined. ([CMP-2485](#))
- Previously, the reporting of all of the **ComplianceCheckResults** for a **DONE** status **ComplianceScan** was incomplete. With this update, annotation has been added to report the number of total **ComplianceCheckResults** for a **ComplianceScan** with a **DONE** status. ([CMP-2615](#))
- Previously, the **ocp4-cis-scc-limit-container-allowed-capabilities** rule description contained ambiguous guidelines, leading to confusion among users. With this update, the rule description and actionable steps are clarified. ([OCBUGS-17828](#))
- Before this update, sysctl configurations caused certain auto remediations for RHCOS4 rules to fail scans in affected clusters. With this update, the correct sysctl settings are applied and RHCOS4 rules for FedRAMP High profiles pass scans correctly. ([OCBUGS-19690](#))
- Before this update, an issue with a **jq** filter caused errors with the **rhacs-operator-controller-manager** deployment during compliance checks. With this update, the **jq** filter expression is updated and the **rhacs-operator-controller-manager** deployment is exempt from compliance checks pertaining to container resource limits, eliminating false positive results. ([OCBUGS-19690](#))
- Before this update, **rhcos4-high** and **rhcos4-moderate** profiles checked values of an incorrectly titled configuration file. As a result, some scan checks could fail. With this update, the **rhcos4** profiles now check the correct configuration file and scans pass correctly. ([OCBUGS-31674](#))
- Previously, the **accessTokenInactivityTimeoutSeconds** variable used in the **oauthclient-inactivity-timeout** rule was immutable, leading to a **FAIL** status when performing DISA STIG scans. With this update, proper enforcement of the **accessTokenInactivityTimeoutSeconds** variable operates correctly and a **PASS** status is now possible. ([OCBUGS-32551](#))
- Before this update, some annotations for rules were not updated, displaying the incorrect control standards. With this update, annotations for rules are updated correctly, ensuring the correct control standards are displayed. ([OCBUGS-34982](#))
- Previously, when upgrading to Compliance Operator 1.5.1, an incorrectly referenced secret in a **ServiceMonitor** configuration caused integration issues with the Prometheus Operator. With this update, the Compliance Operator will accurately reference the secret containing the token for **ServiceMonitor** metrics. ([OCBUGS-39417](#))

5.2.12. Release notes for OpenShift Compliance Operator 1.5.1

Release notes for OpenShift Compliance Operator 1.5.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.5.1:

- [RHBA-2024:5956 - OpenShift Compliance Operator 1.5.1 bug fix and enhancement update](#)

5.2.13. Release notes for OpenShift Compliance Operator 1.5.0

Release notes for OpenShift Compliance Operator 1.5.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.5.0:

- [RHBA-2024:3533 - OpenShift Compliance Operator 1.5.0 bug fix and enhancement update](#)

5.2.13.1. New features and enhancements

- With this update, the Compliance Operator provides a unique profile ID for easier programmatic use. ([CMP-2450](#))
- With this release, the Compliance Operator is now tested and supported on the ROSA HCP environment. The Compliance Operator loads only Node profiles when running on ROSA HCP. This is because a Red Hat managed platform restricts access to the control plane, which makes Platform profiles irrelevant to the operator's function. ([CMP-2581](#))

5.2.13.2. Bug fixes

- CVE-2024-2961 is resolved in the Compliance Operator 1.5.0 release. ([CVE-2024-2961](#))
- Previously, for ROSA HCP systems, profile listings were incorrect. This update allows the Compliance Operator to provide correct profile output. ([OCBUGS-34535](#))
- With this release, namespaces can be excluded from the **ocp4-configure-network-policies-namespaces** check by setting the **ocp4-var-network-policies-namespaces-exempt-regex** variable in the tailored profile. ([CMP-2543](#))

5.2.14. Release notes for OpenShift Compliance Operator 1.4.1

Release notes for OpenShift Compliance Operator 1.4.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.4.1:

- [RHBA-2024:1830 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.14.1. New features and enhancements

- As of this release, the Compliance Operator now provides the CIS OpenShift 1.5.0 profile rules. ([CMP-2447](#))
- With this update, the Compliance Operator now provides **OCP4 STIG ID** and **SRG** with the profile rules. ([CMP-2401](#))
- With this update, obsolete rules being applied to **s390x** have been removed. ([CMP-2471](#))

5.2.14.2. Bug fixes

- Previously, for Red Hat Enterprise Linux CoreOS (RHCOS) systems using Red Hat Enterprise Linux (RHEL) 9, application of the **ocp4-kubelet-enable-protect-kernel-sysctl-file-exist** rule failed. This update replaces the rule with **ocp4-kubelet-enable-protect-kernel-sysctl**. Now,

after auto remediation is applied, RHEL 9-based RHCOS systems will show **PASS** upon the application of this rule. ([OCBUGS-13589](#))

- Previously, after applying compliance remediations using profile **rhcos4-e8**, the nodes were no longer accessible using SSH to the core user account. With this update, nodes remain accessible through SSH using the ``sshkey1` option. ([OCBUGS-18331](#))
- Previously, the **STIG** profile was missing rules from CaC that fulfill requirements on the published **STIG** for OpenShift Container Platform. With this update, upon remediation, the cluster satisfies **STIG** requirements that can be remediated using Compliance Operator. ([OCBUGS-26193](#))
- Previously, creating a **ScanSettingBinding** object with profiles of different types for multiple products bypassed a restriction against multiple products types in a binding. With this update, the product validation now allows multiple products regardless of the of profile types in the **ScanSettingBinding** object. ([OCBUGS-26229](#))
- Previously, running the **rhcos4-service-debug-shell-disabled** rule showed as **FAIL** even after auto-remediation was applied. With this update, running the **rhcos4-service-debug-shell-disabled** rule now shows **PASS** after auto-remediation is applied. ([OCBUGS-28242](#))
- With this update, instructions for the use of the **rhcos4-banner-etc-issue** rule are enhanced to provide more detail. ([OCBUGS-28797](#))
- Previously the **api_server_api_priority_flowschema_catch_all** rule provided **FAIL** status on OpenShift Container Platform 4.16 clusters. With this update, the **api_server_api_priority_flowschema_catch_all** rule provides **PASS** status on OpenShift Container Platform 4.16 clusters. ([OCBUGS-28918](#))
- Previously, when a profile was removed from a completed scan shown in a **ScanSettingBinding** (SSB) object, the Compliance Operator did not remove the old scan. Afterward, when launching a new SSB using the deleted profile, the Compliance Operator failed to update the result. With this release of the Compliance Operator, the new SSB now shows the new compliance check result. ([OCBUGS-29272](#))
- Previously, on **ppc64le** architecture, the metrics service was not created. With this update, when deploying the Compliance Operator v1.4.1 on **ppc64le** architecture, the metrics service is now created correctly. ([OCBUGS-32797](#))
- Previously, on a HyperShift hosted cluster, a scan with the **ocp4-pci-dss profile** will run into an unrecoverable error due to a **filter cannot iterate** issue. With this release, the scan for the **ocp4-pci-dss** profile will reach **done** status and return either a **Compliance** or **Non-Compliance** test result. ([OCBUGS-33067](#))

5.2.15. Release notes for OpenShift Compliance Operator 1.4.0

Release notes for OpenShift Compliance Operator 1.4.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.4.0:

- [RHBA-2023:7658 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.15.1. New features and enhancements

- With this update, clusters which use custom node pools outside the default **worker** and **master** node pools no longer need to supply additional variables to ensure Compliance Operator aggregates the configuration file for that node pool.
- Users can now pause scan schedules by setting the **ScanSetting.suspend** attribute to **True**. This allows users to suspend a scan schedule and reactivate it without the need to delete and re-create the **ScanSettingBinding**. This simplifies pausing scan schedules during maintenance periods. ([CMP-2123](#))
- Compliance Operator now supports an optional **version** attribute on **Profile** custom resources. ([CMP-2125](#))
- Compliance Operator now supports profile names in **ComplianceRules**. ([CMP-2126](#))
- Compliance Operator compatibility with improved **cronjob** API improvements is available in this release. ([CMP-2310](#))

5.2.15.2. Bug fixes

- Previously, on a cluster with Windows nodes, some rules will FAIL after auto remediation is applied because the Windows nodes were not skipped by the compliance scan. With this release, Windows nodes are correctly skipped when scanning. ([OCPBUGS-7355](#))
- With this update, **rprivate** default mount propagation is now handled correctly for root volume mounts of pods that rely on multipathing. ([OCPBUGS-17494](#))
- Previously, the Compliance Operator would generate a remediation for **coreos_vsyscall_kernel_argument** without reconciling the rule even while applying the remediation. With release 1.4.0, the **coreos_vsyscall_kernel_argument** rule properly evaluates kernel arguments and generates an appropriate remediation. ([OCPBUGS-8041](#))
- Before this update, rule **rhcos4-audit-rules-login-events-faillock** would fail even after auto-remediation has been applied. With this update, **rhcos4-audit-rules-login-events-faillock** failure locks are now applied correctly after auto-remediation. ([OCPBUGS-24594](#))
- Previously, upgrades from Compliance Operator 1.3.1 to Compliance Operator 1.4.0 would cause OVS rules scan results to go from **PASS** to **NOT-APPLICABLE**. With this update, OVS rules scan results now show **PASS** ([OCPBUGS-25323](#))

5.2.16. Release notes for OpenShift Compliance Operator 1.3.1

Release notes for OpenShift Compliance Operator 1.3.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.3.1:

- [RHBA-2023:5669 - OpenShift Compliance Operator bug fix and enhancement update](#)

This update addresses a CVE in an underlying dependency.

5.2.16.1. New features and enhancements

- You can install and use the Compliance Operator in an OpenShift Container Platform cluster running in FIPS mode.



IMPORTANT

To enable FIPS mode for your cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Switching RHEL to FIPS mode](#).

When running Red Hat Enterprise Linux (RHEL) or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures.

5.2.16.2. Known issue

- On a cluster with Windows nodes, some rules will FAIL after auto remediation is applied because the Windows nodes are not skipped by the compliance scan. This differs from the expected results because the Windows nodes must be skipped when scanning. ([OCPBUGS-7355](#))

5.2.17. Release notes for OpenShift Compliance Operator 1.3.0

Release notes for OpenShift Compliance Operator 1.3.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.3.0:

- [RHBA-2023:5102 - OpenShift Compliance Operator enhancement update](#)

5.2.17.1. New features and enhancements

- The Defense Information Systems Agency Security Technical Implementation Guide (DISA-STIG) for OpenShift Container Platform is now available from Compliance Operator 1.3.0. See [Supported compliance profiles](#) for additional information.
- Compliance Operator 1.3.0 now supports IBM Power® and IBM Z® for NIST 800-53 Moderate-Impact Baseline for OpenShift Container Platform platform and node profiles.

5.2.18. Release notes for OpenShift Compliance Operator 1.2.0

Release notes for OpenShift Compliance Operator 1.2.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.2.0:

- [RHBA-2023:4245 - OpenShift Compliance Operator enhancement update](#)

5.2.18.1. New features and enhancements

- The CIS OpenShift Container Platform 4 Benchmark v1.4.0 profile is now available for platform and node applications. To locate the CIS OpenShift Container Platform v4 Benchmark, go to [CIS Benchmarks](#) and click **Download Latest CIS Benchmark** where you can then register to download the benchmark.



IMPORTANT

Upgrading to Compliance Operator 1.2.0 will overwrite the CIS OpenShift Container Platform 4 Benchmark 1.1.0 profiles.

If your OpenShift Container Platform environment contains existing **cis** and **cis-node** remediations, there might be some differences in scan results after upgrading to Compliance Operator 1.2.0.

- Additional clarity for auditing security context constraints (SCCs) is now available for the **scc-limit-container-allowed-capabilities** rule.

5.2.19. Release notes for OpenShift Compliance Operator 1.1.0

Release notes for OpenShift Compliance Operator 1.1.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.1.0:

- [RHBA-2023:3630 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.19.1. New features and enhancements

- A start and end timestamp is now available in the **ComplianceScan** custom resource definition (CRD) status.
- The Compliance Operator can now be deployed on hosted control planes using the software catalog by creating a **Subscription** file. For more information, see [Installing the Compliance Operator on hosted control planes](#).

5.2.19.2. Bug fixes

- Before this update, some Compliance Operator rule instructions were not present. After this update, instructions are improved for the following rules:
 - **classification_banner**
 - **oauth_login_template_set**
 - **oauth_logout_url_set**
 - **oauth_provider_selection_set**
 - **ocp_allowed_registries**
 - **ocp_allowed_registries_for_import** ([OCPBUGS-10473](#))
- Before this update, check accuracy and rule instructions were unclear. After this update, the check accuracy and instructions are improved for the following **sysctl** rules:
 - **kubelet-enable-protect-kernel-sysctl**
 - **kubelet-enable-protect-kernel-sysctl-kernel-keys-root-maxbytes**

- **kubelet-enable-protect-kernel-sysctl-kernel-keys-root-maxkeys**
- **kubelet-enable-protect-kernel-sysctl-kernel-panic**
- **kubelet-enable-protect-kernel-sysctl-kernel-panic-on-oops**
- **kubelet-enable-protect-kernel-sysctl-vm-overcommit-memory**
- **kubelet-enable-protect-kernel-sysctl-vm-panic-on-oom**
([OCPBUGS-11334](#))
- Before this update, the **ocp4-alert-receiver-configured** rule did not include instructions. With this update, the **ocp4-alert-receiver-configured** rule now includes improved instructions. ([OCPBUGS-7307](#))
- Before this update, the **rhcos4-sshd-set-loglevel-info** rule would fail for the **rhcos4-e8** profile. With this update, the remediation for the **sshd-set-loglevel-info** rule was updated to apply the correct configuration changes, allowing subsequent scans to pass after the remediation is applied. ([OCPBUGS-7816](#))
- Before this update, a new installation of OpenShift Container Platform with the latest Compliance Operator install failed on the **scheduler-no-bind-address** rule. With this update, the **scheduler-no-bind-address** rule has been disabled on newer versions of OpenShift Container Platform since the parameter was removed. ([OCPBUGS-8347](#))

5.2.20. Release notes for OpenShift Compliance Operator 1.0.0

Release notes for OpenShift Compliance Operator 1.0.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 1.0.0:

- [RHBA-2023:1682 - OpenShift Compliance Operator bug fix update](#)

5.2.20.1. New features and enhancements

- The Compliance Operator is now stable and the release channel is upgraded to **stable**. Future releases will follow [Semantic Versioning](#). To access the latest release, see [Updating the Compliance Operator](#).

5.2.20.2. Bug fixes

- Before this update, the `compliance_operator_compliance_scan_error_total` metric had an ERROR label with a different value for each error message. With this update, the `compliance_operator_compliance_scan_error_total` metric does not increase in values. ([OCPBUGS-1803](#))
- Before this update, the **ocp4-api-server-audit-log-maxsize** rule would result in a **FAIL** state. With this update, the error message has been removed from the metric, decreasing the cardinality of the metric consistent with best practices. ([OCPBUGS-7520](#))
- Before this update, the **rhcos4-enable-fips-mode** rule description was misleading that FIPS could be enabled after installation. With this update, the **rhcos4-enable-fips-mode** rule description clarifies that FIPS must be enabled at install time. ([OCPBUGS-8358](#))

5.2.21. Release notes for OpenShift Compliance Operator 0.1.61

Release notes for OpenShift Compliance Operator 0.1.61.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.61:

- [RHBA-2023:0557 - OpenShift Compliance Operator bug fix update](#)

5.2.21.1. New features and enhancements

- The Compliance Operator now supports timeout configuration for Scanner Pods. The timeout is specified in the **ScanSetting** object. If the scan is not completed within the timeout, the scan retries until the maximum number of retries is reached. See [Configuring ScanSetting timeout](#) for more information.

5.2.21.2. Bug fixes

- Before this update, Compliance Operator remediations required variables as inputs. Remediations without variables set were applied cluster-wide and resulted in stuck nodes, even though it appeared the remediation applied correctly. With this update, the Compliance Operator validates if a variable needs to be supplied using a **TailoredProfile** for a remediation. ([OCBUGS-3864](#))
- Before this update, the instructions for **ocp4-kubelet-configure-tls-cipher-suites** were incomplete, requiring users to refine the query manually. With this update, the query provided in **ocp4-kubelet-configure-tls-cipher-suites** returns the actual results to perform the audit steps. ([OCBUGS-3017](#))
- Before this update, system reserved parameters were not generated in kubelet configuration files, causing the Compliance Operator to fail to unpause the machine config pool. With this update, the Compliance Operator omits system reserved parameters during machine configuration pool evaluation. ([OCBUGS-4445](#))
- Before this update, **ComplianceCheckResult** objects did not have correct descriptions. With this update, the Compliance Operator sources the **ComplianceCheckResult** information from the rule description. ([OCBUGS-4615](#))
- Before this update, the Compliance Operator did not check for empty kubelet configuration files when parsing machine configurations. As a result, the Compliance Operator would panic and crash. With this update, the Compliance Operator implements improved checking of the kubelet configuration data structure and only continues if it is fully rendered. ([OCBUGS-4621](#))
- Before this update, the Compliance Operator generated remediations for kubelet evictions based on machine config pool name and a grace period, resulting in multiple remediations for a single eviction rule. With this update, the Compliance Operator applies all remediations for a single rule. ([OCBUGS-4338](#))
- Before this update, a regression occurred when attempting to create a **ScanSettingBinding** that was using a **TailoredProfile** with a non-default **MachineConfigPool** marked the **ScanSettingBinding** as **Failed**. With this update, functionality is restored and custom **ScanSettingBinding** using a **TailoredProfile** performs correctly. ([OCBUGS-6827](#))
- Before this update, some kubelet configuration parameters did not have default values. With this update, the following parameters contain default values ([OCBUGS-6708](#)):

•

- **ocp4-cis-kubelet-enable-streaming-connections**
- **ocp4-cis-kubelet-eviction-thresholds-set-hard-imagefs-available**
- **ocp4-cis-kubelet-eviction-thresholds-set-hard-imagefs-inodesfree**
- **ocp4-cis-kubelet-eviction-thresholds-set-hard-memory-available**
- **ocp4-cis-kubelet-eviction-thresholds-set-hard-nodefs-available**
- Before this update, the **selinux_confinement_of_daemons** rule failed running on the kubelet because of the permissions necessary for the kubelet to run. With this update, the **selinux_confinement_of_daemons** rule is disabled. ([OCBUGS-6968](#))

5.2.22. Release notes for OpenShift Compliance Operator 0.1.59

Release notes for OpenShift Compliance Operator 0.1.59.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.59:

- [RHBA-2022:8538 - OpenShift Compliance Operator bug fix update](#)

5.2.22.1. New features and enhancements

- The Compliance Operator now supports Payment Card Industry Data Security Standard (PCI-DSS) **ocp4-pci-dss** and **ocp4-pci-dss-node** profiles on the **ppc64le** architecture.

5.2.22.2. Bug fixes

- Previously, the Compliance Operator did not support the Payment Card Industry Data Security Standard (PCI DSS) **ocp4-pci-dss** and **ocp4-pci-dss-node** profiles on different architectures such as **ppc64le**. Now, the Compliance Operator supports **ocp4-pci-dss** and **ocp4-pci-dss-node** profiles on the **ppc64le** architecture. ([OCBUGS-3252](#))
- Previously, after the recent update to version 0.1.57, the **rerunner** service account (SA) was no longer owned by the cluster service version (CSV), which caused the SA to be removed during the Operator upgrade. Now, the CSV owns the **rerunner** SA in 0.1.59, and upgrades from any previous version will not result in a missing SA. ([OCBUGS-3452](#))

5.2.23. Release notes for OpenShift Compliance Operator 0.1.57

Release notes for OpenShift Compliance Operator 0.1.57.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.57:

- [RHBA-2022:6657 - OpenShift Compliance Operator bug fix update](#)

5.2.23.1. New features and enhancements

- **KubeletConfig** checks changed from **Node** to **Platform** type. **KubeletConfig** checks the default configuration of the **KubeletConfig**. The configuration files are aggregated from all nodes into a single location per node pool. See [Evaluating KubeletConfig rules against default](#)

[configuration values](#).

- The **ScanSetting** Custom Resource now allows users to override the default CPU and memory limits of scanner pods through the **scanLimits** attribute. For more information, see [Increasing Compliance Operator resource limits](#).
- A **PriorityClass** object can now be set through **ScanSetting**. This ensures the Compliance Operator is prioritized and minimizes the chance that the cluster falls out of compliance. For more information, see [Setting PriorityClass for ScanSetting scans](#).

5.2.23.2. Bug fixes

- Previously, the Compliance Operator hard-coded notifications to the default **openshift-compliance** namespace. If the Operator were installed in a non-default namespace, the notifications would not work as expected. Now, notifications work in non-default **openshift-compliance** namespaces. ([BZ#2060726](#))
- Previously, the Compliance Operator was unable to evaluate default configurations used by kubelet objects, resulting in inaccurate results and false positives. [This new feature](#) evaluates the kubelet configuration and now reports accurately. ([BZ#2075041](#))
- Previously, the Compliance Operator reported the **ocp4-kubelet-configure-event-creation** rule in a **FAIL** state after applying an automatic remediation because the **eventRecordQPS** value was set higher than the default value. Now, the **ocp4-kubelet-configure-event-creation** rule remediation sets the default value, and the rule applies correctly. ([BZ#2082416](#))
- The **ocp4-configure-network-policies** rule requires manual intervention to perform effectively. New descriptive instructions and rule updates increase applicability of the **ocp4-configure-network-policies** rule for clusters using Calico CNIs. ([BZ#2091794](#))
- Previously, the Compliance Operator would not clean up pods used to scan infrastructure when using the **debug=true** option in the scan settings. This caused pods to be left on the cluster even after deleting the **ScanSettingBinding**. Now, pods are always deleted when a **ScanSettingBinding** is deleted. ([BZ#2092913](#))
- Previously, the Compliance Operator used an older version of the **operator-sdk** command that caused alerts about deprecated functionality. Now, an updated version of the **operator-sdk** command is included and there are no more alerts for deprecated functionality. ([BZ#2098581](#))
- Previously, the Compliance Operator would fail to apply remediations if it could not determine the relationship between kubelet and machine configurations. Now, the Compliance Operator has improved handling of the machine configurations and is able to determine if a kubelet configuration is a subset of a machine configuration. ([BZ#2102511](#))
- Previously, the rule for **ocp4-cis-node-master-kubelet-enable-cert-rotation** did not properly describe success criteria. As a result, the requirements for **RotateKubeletClientCertificate** were unclear. Now, the rule for **ocp4-cis-node-master-kubelet-enable-cert-rotation** reports accurately regardless of the configuration present in the kubelet configuration file. ([BZ#2105153](#))
- Previously, the rule for checking idle streaming timeouts did not consider default values, resulting in inaccurate rule reporting. Now, more robust checks ensure increased accuracy in results based on default configuration values. ([BZ#2105878](#))
- Previously, the Compliance Operator would fail to fetch API resources when parsing machine

configurations without Ignition specifications, which caused the **api-check-pods** processes to crash loop. Now, the Compliance Operator handles Machine Config Pools that do not have Ignition specifications correctly. ([BZ#2117268](#))

- Previously, rules evaluating the **modprobe** configuration would fail even after applying remediations due to a mismatch in values for the **modprobe** configuration. Now, the same values are used for the **modprobe** configuration in checks and remediations, ensuring consistent results. ([BZ#2117747](#))

5.2.23.3. Deprecations

- Specifying **Install into all namespaces in the cluster** or setting the **WATCH_NAMESPACES** environment variable to "" no longer affects all namespaces. Any API resources installed in namespaces not specified at the time of Compliance Operator installation is no longer be operational. API resources might require creation in the selected namespace, or the **openshift-compliance** namespace by default. This change improves the Compliance Operator's memory usage.

5.2.24. Release notes for OpenShift Compliance Operator 0.1.53

Release notes for OpenShift Compliance Operator 0.1.53.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.53:

- [RHBA-2022:5537 - OpenShift Compliance Operator bug fix update](#)

5.2.24.1. Bug fixes

- Previously, the **ocp4-kubelet-enable-streaming-connections** rule contained an incorrect variable comparison, resulting in false positive scan results. Now, the Compliance Operator provides accurate scan results when setting **streamingConnectionIdleTimeout**. ([BZ#2069891](#))
- Previously, group ownership for **/etc/openvswitch/conf.db** was incorrect on IBM Z® architectures, resulting in **ocp4-cis-node-worker-file-groupowner-ovs-conf-db** check failures. Now, the check is marked **NOT-APPLICABLE** on IBM Z® architecture systems. ([BZ#2072597](#))
- Previously, the **ocp4-cis-scc-limit-container-allowed-capabilities** rule reported in a **FAIL** state due to incomplete data regarding the security context constraints (SCC) rules in the deployment. Now, the result is **MANUAL**, which is consistent with other checks that require human intervention. ([BZ#2077916](#))
- Previously, the following rules failed to account for additional configuration paths for API servers and TLS certificates and keys, resulting in reported failures even if the certificates and keys were set properly:
 - **ocp4-cis-api-server-kubelet-client-cert**
 - **ocp4-cis-api-server-kubelet-client-key**
 - **ocp4-cis-kubelet-configure-tls-cert**
 - **ocp4-cis-kubelet-configure-tls-key**

Now, the rules report accurately and observe legacy file paths specified in the kubelet configuration file. ([BZ#2079813](#))

- Previously, the **content_rule_oauth_or_oauthclient_inactivity_timeout** rule did not account for a configurable timeout set by the deployment when assessing compliance for timeouts. This resulted in the rule failing even if the timeout was valid. Now, the Compliance Operator uses the **var_oauth_inactivity_timeout** variable to set valid timeout length. ([BZ#2081952](#))
- Previously, the Compliance Operator used administrative permissions on namespaces not labeled appropriately for privileged use, resulting in warning messages regarding pod security-level violations. Now, the Compliance Operator has appropriate namespace labels and permission adjustments to access results without violating permissions. ([BZ#2088202](#))
- Previously, applying auto remediations for **rhcos4-high-master-sysctl-kernel-yama-pttrace-scope** and **rhcos4-sysctl-kernel-core-pattern** resulted in subsequent failures of those rules in scan results, even though they were remediated. Now, the rules report **PASS** accurately, even after remediations are applied. ([BZ#2094382](#))
- Previously, the Compliance Operator would fail in a **CrashLoopBackoff** state because of out-of-memory exceptions. Now, the Compliance Operator is improved to handle large machine configuration data sets in memory and function correctly. ([BZ#2094854](#))

5.2.24.2. Known issue

- When **"debug":true** is set within the **ScanSettingBinding** object, the pods generated by the **ScanSettingBinding** object are not removed when that binding is deleted. As a workaround, run the following command to delete the remaining pods:

```
$ oc delete pods -l compliance.openshift.io/scan-name=ocp4-cis
```

([BZ#2092913](#))

5.2.25. Release notes for OpenShift Compliance Operator 0.1.52

Release notes for OpenShift Compliance Operator 0.1.52.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.52:

- [RHBA-2022:4657 - OpenShift Compliance Operator bug fix update](#)

5.2.25.1. New features and enhancements

- The FedRAMP high SCAP profile is now available for use in OpenShift Container Platform environments. For more information, See [Supported compliance profiles](#).

5.2.25.2. Bug fixes

- Previously, the **OpenScap** container would crash due to a mount permission issue in a security environment where **DAC_OVERRIDE** capability is dropped. Now, executable mount permissions are applied to all users. ([BZ#2082151](#))

- Previously, the compliance rule **ocp4-configure-network-policies** could be configured as **MANUAL**. Now, compliance rule **ocp4-configure-network-policies** is set to **AUTOMATIC**. ([BZ#2072431](#))
- Previously, the Cluster Autoscaler would fail to scale down because the Compliance Operator scan pods were never removed after a scan. Now, the pods are removed from each node by default unless explicitly saved for debugging purposes. ([BZ#2075029](#))
- Previously, applying the Compliance Operator to the **KubeletConfig** would result in the node going into a **NotReady** state due to unpausing the Machine Config Pools too early. Now, the Machine Config Pools are unpaused appropriately and the node operates correctly. ([BZ#2071854](#))
- Previously, the Machine Config Operator used **base64** instead of **url-encoded** code in the latest release, causing Compliance Operator remediation to fail. Now, the Compliance Operator checks encoding to handle both **base64** and **url-encoded** Machine Config code and the remediation applies correctly. ([BZ#2082431](#))

5.2.25.3. Known issue

- When **"debug":true** is set within the **ScanSettingBinding** object, the pods generated by the **ScanSettingBinding** object are not removed when that binding is deleted. As a workaround, run the following command to delete the remaining pods:

```
$ oc delete pods -l compliance.openshift.io/scan-name=ocp4-cis
```

([BZ#2092913](#))

5.2.26. Release notes for OpenShift Compliance Operator 0.1.49

Release notes for OpenShift Compliance Operator 0.1.49.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.49:

- [RHBA-2022:1148 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.26.1. New features and enhancements

- The Compliance Operator is now supported on the following architectures:
 - IBM Power®
 - IBM Z®
 - IBM® LinuxONE

5.2.26.2. Bug fixes

- Previously, the **openshift-compliance** content did not include platform-specific checks for network types. As a result, OVN- and SDN-specific checks would show as **failed** instead of **not-applicable** based on the network configuration. Now, new rules contain platform checks for networking rules, resulting in a more accurate assessment of network-specific checks. ([BZ#1994609](#))

- Previously, the **ocp4-moderate-routes-protected-by-tls** rule incorrectly checked TLS settings that results in the rule failing the check, even if the connection secure SSL/TLS protocol. Now, the check properly evaluates TLS settings that are consistent with the networking guidance and profile recommendations. ([BZ#2002695](#))
- Previously, **ocp-cis-configure-network-policies-namespace** used pagination when requesting namespaces. This caused the rule to fail because the deployments truncated lists of more than 500 namespaces. Now, the entire namespace list is requested, and the rule for checking configured network policies works for deployments with more than 500 namespaces. ([BZ#2038909](#))
- Previously, remediations using the **sshd jinja** macros were hard-coded to specific sshd configurations. As a result, the configurations were inconsistent with the content the rules were checking for and the check would fail. Now, the sshd configuration is parameterized and the rules apply successfully. ([BZ#2049141](#))
- Previously, the **ocp4-cluster-version-operator-verify-integrity** always checked the first entry in the Cluster Version Operator (CVO) history. As a result, the upgrade would fail in situations where subsequent versions of OpenShift Container Platform would be verified. Now, the compliance check result for **ocp4-cluster-version-operator-verify-integrity** is able to detect verified versions and is accurate with the CVO history. ([BZ#2053602](#))
- Previously, the **ocp4-api-server-no-adm-ctrl-plugins-disabled** rule did not check for a list of empty admission controller plugins. As a result, the rule would always fail, even if all admission plugins were enabled. Now, more robust checking of the **ocp4-api-server-no-adm-ctrl-plugins-disabled** rule accurately passes with all admission controller plugins enabled. ([BZ#2058631](#))
- Previously, scans did not contain platform checks for running against Linux worker nodes. As a result, running scans against worker nodes that were not Linux-based resulted in a never ending scan loop. Now, the scan schedules appropriately based on platform type and labels complete successfully. ([BZ#2056911](#))

5.2.27. Release notes for OpenShift Compliance Operator 0.1.48

Release notes for OpenShift Compliance Operator 0.1.48.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.48:

- [RHBA-2022:0416 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.27.1. Bug fixes

- Previously, some rules associated with extended Open Vulnerability and Assessment Language (OVAL) definitions had a **checkType** of **None**. This was because the Compliance Operator was not processing extended OVAL definitions when parsing rules. With this update, content from extended OVAL definitions is parsed so that these rules now have a **checkType** of either **Node** or **Platform**. ([BZ#2040282](#))
- Previously, a manually created **MachineConfig** object for **KubeletConfig** prevented a **KubeletConfig** object from being generated for remediation, leaving the remediation in the **Pending** state. With this release, a **KubeletConfig** object is created by the remediation, regardless if there is a manually created **MachineConfig** object for **KubeletConfig**. As a result, **KubeletConfig** remediations now work as expected. ([BZ#2040401](#))

5.2.28. Release notes for OpenShift Compliance Operator 0.1.47

Release notes for OpenShift Compliance Operator 0.1.47.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.47:

- [RHBA-2022:0014 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.28.1. New features and enhancements

- The Compliance Operator now supports the following compliance benchmarks for the Payment Card Industry Data Security Standard (PCI DSS):
 - ocp4-pci-dss
 - ocp4-pci-dss-node
- Additional rules and remediations for FedRAMP moderate impact level are added to the OCP4-moderate, OCP4-moderate-node, and rhcos4-moderate profiles.
- Remediations for KubeletConfig are now available in node-level profiles.

5.2.28.2. Bug fixes

- Previously, if your cluster was running OpenShift Container Platform 4.6 or earlier, remediations for USBGuard-related rules would fail for the moderate profile. This is because the remediations created by the Compliance Operator were based on an older version of USBGuard that did not support drop-in directories. Now, invalid remediations for USBGuard-related rules are not created for clusters running OpenShift Container Platform 4.6. If your cluster is using OpenShift Container Platform 4.6, you must manually create remediations for USBGuard-related rules.
Additionally, remediations are created only for rules that satisfy minimum version requirements. ([BZ#1965511](#))
- Previously, when rendering remediations, the compliance operator would check that the remediation was well-formed by using a regular expression that was too strict. As a result, some remediations, such as those that render **sshd_config**, would not pass the regular expression check and therefore, were not created. The regular expression was found to be unnecessary and removed. Remediations now render correctly. ([BZ#2033009](#))

5.2.29. Release notes for OpenShift Compliance Operator 0.1.44

Release notes for OpenShift Compliance Operator 0.1.44.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.44:

- [RHBA-2021:4530 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.29.1. New features and enhancements

- In this release, the **strictNodeScan** option is now added to the **ComplianceScan**, **ComplianceSuite** and **ScanSetting** CRs. This option defaults to **true** which matches the previous behavior, where an error occurred if a scan was not able to be scheduled on a node.

Setting the option to **false** allows the Compliance Operator to be more permissive about scheduling scans. Environments with ephemeral nodes can set the **strictNodeScan** value to false, which allows a compliance scan to proceed, even if some of the nodes in the cluster are not available for scheduling.

- You can now customize the node that is used to schedule the result server workload by configuring the **nodeSelector** and **tolerations** attributes of the **ScanSetting** object. These attributes are used to place the **ResultServer** pod, the pod that is used to mount a PV storage volume and store the raw Asset Reporting Format (ARF) results. Previously, the **nodeSelector** and the **tolerations** parameters defaulted to selecting one of the control plane nodes and tolerating the **node-role.kubernetes.io/master** taint. This did not work in environments where control plane nodes are not permitted to mount PVs. This feature provides a way for you to select the node and tolerate a different taint in those environments.
- The Compliance Operator can now remediate **KubeletConfig** objects.
- A comment containing an error message is now added to help content developers differentiate between objects that do not exist in the cluster compared to objects that cannot be fetched.
- Rule objects now contain two new attributes, **checkType** and **description**. These attributes allow you to determine if the rule pertains to a node check or platform check, and also allow you to review what the rule does.
- This enhancement removes the requirement that you have to extend an existing profile to create a tailored profile. This means the **extends** field in the **TailoredProfile** CRD is no longer mandatory. You can now select a list of rule objects to create a tailored profile. Note that you must select whether your profile applies to nodes or the platform by setting the **compliance.openshift.io/product-type:** annotation or by setting the **-node** suffix for the **TailoredProfile** CR.
- In this release, the Compliance Operator is now able to schedule scans on all nodes irrespective of their taints. Previously, the scan pods would only tolerated the **node-role.kubernetes.io/master** taint, meaning that they would either ran on nodes with no taints or only on nodes with the **node-role.kubernetes.io/master** taint. In deployments that use custom taints for their nodes, this resulted in the scans not being scheduled on those nodes. Now, the scan pods tolerate all node taints.
- In this release, the Compliance Operator supports the following North American Electric Reliability Corporation (NERC) security profiles:
 - ocp4-nerc-cip
 - ocp4-nerc-cip-node
 - rhcos4-nerc-cip
- In this release, the Compliance Operator supports the NIST 800-53 Moderate-Impact Baseline for the Red Hat OpenShift – Node level, ocp4-moderate-node, security profile.

5.2.29.2. Templating and variable use

- In this release, the remediation template now allows multi-value variables.
- With this update, the Compliance Operator can change remediations based on variables that are set in the compliance profile. This is useful for remediations that include deployment-specific values such as time outs, NTP server host names, or similar. Additionally, the

ComplianceCheckResult objects now use the label **compliance.openshift.io/check-has-value** that lists the variables a check has used.

5.2.29.3. Bug fixes

- Previously, while performing a scan, an unexpected termination occurred in one of the scanner containers of the pods. In this release, the Compliance Operator uses the latest OpenSCAP version 1.3.5 to avoid a crash.
- Previously, using **autoReplyRemediations** to apply remediations triggered an update of the cluster nodes. This was disruptive if some of the remediations did not include all of the required input variables. Now, if a remediation is missing one or more required input variables, it is assigned a state of **NeedsReview**. If one or more remediations are in a **NeedsReview** state, the machine config pool remains paused, and the remediations are not applied until all of the required variables are set. This helps minimize disruption to the nodes.
- The RBAC Role and Role Binding used for Prometheus metrics are changed to 'ClusterRole' and 'ClusterRoleBinding' to ensure that monitoring works without customization.
- Previously, if an error occurred while parsing a profile, rules or variables objects were removed and deleted from the profile. Now, if an error occurs during parsing, the **profileparser** annotates the object with a temporary annotation that prevents the object from being deleted until after parsing completes. ([BZ#1988259](#))
- Previously, an error occurred if titles or descriptions were missing from a tailored profile. Because the XCCDF standard requires titles and descriptions for tailored profiles, titles and descriptions are now required to be set in **TailoredProfile** CRs.
- Previously, when using tailored profiles, **TailoredProfile** variable values were allowed to be set using only a specific selection set. This restriction is now removed, and **TailoredProfile** variables can be set to any value.

5.2.30. Release notes for OpenShift Compliance Operator 0.1.39

Release Notes for Compliance Operator 0.1.39.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift Compliance Operator 0.1.39:

- [RHBA-2021:3214 - OpenShift Compliance Operator bug fix and enhancement update](#)

5.2.30.1. New features and enhancements

- Previously, the Compliance Operator was unable to parse Payment Card Industry Data Security Standard (PCI DSS) references. Now, the Operator can parse compliance content that is provided with PCI DSS profiles.
- Previously, the Compliance Operator was unable to run rules for AU-5 control in the moderate profile. Now, permission is added to the Operator so that it can read **Prometheusrules.monitoring.coreos.com** objects and run the rules that cover AU-5 control in the moderate profile.

5.2.31. Additional resources

- [Understanding the Compliance Operator](#)

- [Updating the Compliance Operator](#)
- [Product Compliance](#)

5.3. COMPLIANCE OPERATOR SUPPORT

You can find support resources for the Compliance Operator, including lifecycle information, general support procedures, and troubleshooting tools.

5.3.1. Compliance Operator lifecycle

The Compliance Operator is a "Rolling Stream" Operator, meaning updates are available asynchronously of OpenShift Container Platform releases. For more information, see "OpenShift Operator Life Cycles" on the Red Hat Customer Portal.

5.3.2. Getting support

Red Hat offers several support channels to help you troubleshoot issues and get the most from OpenShift Container Platform.

From the Red Hat Customer Portal, you can:

- Search or browse through the Red Hat Knowledgebase of articles and solutions about Red Hat products.
- Submit a support case to Red Hat Support.
- Access other product documentation.

To identify issues with your cluster, you can use Red Hat Lightspeed in [OpenShift Cluster Manager](#). Red Hat Lightspeed provides details about issues and, if available, information about how to solve a problem.

To suggest improvements or report errors, give specific details such as the section name and OpenShift Container Platform version.

5.3.3. Using the must-gather tool for the Compliance Operator

You can collect detailed Compliance Operator configuration and logs by using the must-gather tool to aid in troubleshooting issues and support case resolution.

Starting in Compliance Operator v1.6.0, you can collect data about the Compliance Operator resources by running the **must-gather** command with the Compliance Operator image.



NOTE

Consider using the **must-gather** tool when opening support cases or filing bug reports, as it provides additional details about the Operator configuration and logs.

Procedure

- Run the following command to collect data about the Compliance Operator:

```
$ oc adm must-gather --image=$(oc get csv compliance-operator.v1.6.0 -
o=jsonpath='{.spec.relatedImages[?(@.name=="must-gather")].image}')
```

5.3.4. Additional resources

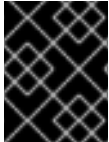
- [About the must-gather tool](#)
- [OpenShift Operator Life Cycles](#)
- [Product Compliance](#)

5.4. COMPLIANCE OPERATOR CONCEPTS

5.4.1. Understanding the Compliance Operator

The Compliance Operator evaluates your OpenShift Container Platform cluster against compliance benchmarks and identifies gaps so you can remediate them. The Operator uses profiles that target platform components, node configurations, or both depending on the compliance standard you need to meet.

The Compliance Operator lets OpenShift Container Platform administrators describe the required compliance state of a cluster and provides them with an overview of gaps and ways to remediate them. The Compliance Operator assesses compliance of both the Kubernetes API resources of OpenShift Container Platform, and the nodes running the cluster. The Compliance Operator uses OpenSCAP, a NIST-certified tool, to scan and enforce security policies provided by the content.



IMPORTANT

The Compliance Operator is available for Red Hat Enterprise Linux CoreOS (RHCOS) deployments only.

5.4.1.1. Compliance Operator profiles

There are several profiles available as part of the Compliance Operator installation. You can use the **oc get** command to view available profiles, profile details, and specific rules.

- View the available profiles:

```
$ oc get profile.compliance -n openshift-compliance
```

Example output

NAME	AGE	VERSION
ocp4-cis	3h49m	1.9.0
ocp4-cis-1-9	3h49m	1.9.0
ocp4-cis-node	3h49m	1.9.0
ocp4-cis-node-1-9	3h49m	1.9.0
ocp4-e8	3h49m	
ocp4-high	3h49m	Revision 4
ocp4-high-node	3h49m	Revision 4
ocp4-high-node-rev-4	3h49m	Revision 4
ocp4-high-rev-4	3h49m	Revision 4
ocp4-moderate	3h49m	Revision 4

```

ocp4-moderate-node      3h49m  Revision 4
ocp4-moderate-node-rev-4 3h49m  Revision 4
ocp4-moderate-rev-4     3h49m  Revision 4
ocp4-nerc-cip           3h49m
ocp4-nerc-cip-node      3h49m
ocp4-pci-dss            3h49m  4.0.0
ocp4-pci-dss-3-2        3h49m  3.2.1
ocp4-pci-dss-4-0        3h49m  4.0.0
ocp4-pci-dss-node       3h49m  4.0.0
ocp4-pci-dss-node-3-2   3h49m  3.2.1
ocp4-pci-dss-node-4-0   3h49m  4.0.0
ocp4-stig               3h49m  V2R3
ocp4-stig-node          3h49m  V2R3
ocp4-stig-node-v2r3     3h49m  V2R3
ocp4-stig-v2r3          3h49m  V2R3
rhcos4-e8               3h49m
rhcos4-high             3h49m  Revision 4
rhcos4-high-rev-4       3h49m  Revision 4
rhcos4-moderate         3h49m  Revision 4
rhcos4-moderate-rev-4   3h49m  Revision 4
rhcos4-nerc-cip         3h49m
rhcos4-stig             3h49m  V2R3
rhcos4-stig-v2r3        3h49m  V2R3

```

These profiles represent different compliance benchmarks. Each profile has the product name that it applies to added as a prefix to the name of the profile. **ocp4-e8** applies the Essential 8 benchmark to the OpenShift Container Platform product, while **rhcos4-e8** applies the Essential 8 benchmark to the Red Hat Enterprise Linux CoreOS (RHCOS) product.

- Run the following command to view the details of the **rhcos4-e8** profile:

```
$ oc get -n openshift-compliance -oyaml profiles.compliance rhcos4-e8
```

Example output

```

apiVersion: compliance.openshift.io/v1alpha1
description: 'This profile contains configuration checks for Red Hat Enterprise Linux
  CoreOS that align to the Australian Cyber Security Centre (ACSC) Essential Eight.
  A copy of the Essential Eight in Linux Environments guide can be found at the ACSC
  website: https://www.cyber.gov.au/acsc/view-all-content/publications/hardening-linux-
  workstations-and-servers'
id: xccdf_org.ssgproject.content_profile_e8
kind: Profile
metadata:
  annotations:
    compliance.openshift.io/image-digest: pb-rhcos4hrdkm
    compliance.openshift.io/product: redhat_enterprise_linux_coreos_4
    compliance.openshift.io/product-type: Node
  creationTimestamp: "2022-10-19T12:06:49Z"
  generation: 1
  labels:
    compliance.openshift.io/profile-bundle: rhcos4
  name: rhcos4-e8
  namespace: openshift-compliance
  ownerReferences:

```

```

- apiVersion: compliance.openshift.io/v1alpha1
  blockOwnerDeletion: true
  controller: true
  kind: ProfileBundle
  name: rhcos4
  uid: 22350850-af4a-4f5c-9a42-5e7b68b82d7d
  resourceVersion: "43699"
  uid: 86353f70-28f7-40b4-bf0e-6289ec33675b
rules:
- rhcos4-accounts-no-uid-except-zero
- rhcos4-audit-rules-dac-modification-chmod
- rhcos4-audit-rules-dac-modification-chown
- rhcos4-audit-rules-execution-chcon
- rhcos4-audit-rules-execution-restorecon
- rhcos4-audit-rules-execution-semanage
- rhcos4-audit-rules-execution-setfiles
- rhcos4-audit-rules-execution-setsebool
- rhcos4-audit-rules-execution-seunshare
- rhcos4-audit-rules-kernel-module-loading-delete
- rhcos4-audit-rules-kernel-module-loading-finit
- rhcos4-audit-rules-kernel-module-loading-init
- rhcos4-audit-rules-login-events
- rhcos4-audit-rules-login-events-faillock
- rhcos4-audit-rules-login-events-lastlog
- rhcos4-audit-rules-login-events-tallylog
- rhcos4-audit-rules-networkconfig-modification
- rhcos4-audit-rules-sysadmin-actions
- rhcos4-audit-rules-time-adjtimex
- rhcos4-audit-rules-time-clock-settime
- rhcos4-audit-rules-time-settimeofday
- rhcos4-audit-rules-time-stime
- rhcos4-audit-rules-time-watch-localtime
- rhcos4-audit-rules-usergroup-modification
- rhcos4-auditd-data-retention-flush
- rhcos4-auditd-freq
- rhcos4-auditd-local-events
- rhcos4-auditd-log-format
- rhcos4-auditd-name-format
- rhcos4-auditd-write-logs
- rhcos4-configure-crypto-policy
- rhcos4-configure-ssh-crypto-policy
- rhcos4-no-empty-passwords
- rhcos4-selinux-policytype
- rhcos4-selinux-state
- rhcos4-service-auditd-enabled
- rhcos4-sshd-disable-empty-passwords
- rhcos4-sshd-disable-gssapi-auth
- rhcos4-sshd-disable-rhosts
- rhcos4-sshd-disable-root-login
- rhcos4-sshd-disable-user-known-hosts
- rhcos4-sshd-do-not-permit-user-env
- rhcos4-sshd-enable-strictmodes
- rhcos4-sshd-print-last-log
- rhcos4-sshd-set-loglevel-info
- rhcos4-sysctl-kernel-dmesg-restrict
- rhcos4-sysctl-kernel-kptr-restrict

```

- rhcos4-sysctl-kernel-randomize-va-space
- rhcos4-sysctl-kernel-unprivileged-bpf-disabled
- rhcos4-sysctl-kernel-yama-pttrace-scope
- rhcos4-sysctl-net-core-bpf-jit-harden

title: Australian Cyber Security Centre (ACSC) Essential Eight

- Run the following command to view the details of the **rhcos4-audit-rules-login-events** rule:

```
$ oc get -n openshift-compliance -oyaml rules rhcos4-audit-rules-login-events
```

Example output

```
apiVersion: compliance.openshift.io/v1alpha1
checkType: Node
description: |-
  The audit system already collects login information for all users and root. If the auditd
  daemon is configured to use the augenrules program to read audit rules during daemon
  startup (the default), add the following lines to a file with suffix.rules in the directory
  /etc/audit/rules.d in order to watch for attempted manual edits of files involved in storing
  logon events:

  -w /var/log/tallylog -p wa -k logins
  -w /var/run/faillock -p wa -k logins
  -w /var/log/lastlog -p wa -k logins

  If the auditd daemon is configured to use the auditctl utility to read audit rules during
  daemon startup, add the following lines to /etc/audit/audit.rules file in order to watch for
  unattempted manual edits of files involved in storing logon events:

  -w /var/log/tallylog -p wa -k logins
  -w /var/run/faillock -p wa -k logins
  -w /var/log/lastlog -p wa -k logins
id: xccdf_org.ssgproject.content_rule_audit_rules_login_events
kind: Rule
metadata:
  annotations:
    compliance.openshift.io/image-digest: pb-rhcos4hrdkm
    compliance.openshift.io/rule: audit-rules-login-events
    control.compliance.openshift.io/NIST-800-53: AU-2(d);AU-12(c);AC-6(9);CM-6(a)
    control.compliance.openshift.io/PCI-DSS: Req-10.2.3
    policies.open-cluster-management.io/controls: AU-2(d),AU-12(c),AC-6(9),CM-6(a),Req-
10.2.3
    policies.open-cluster-management.io/standards: NIST-800-53,PCI-DSS
  creationTimestamp: "2022-10-19T12:07:08Z"
  generation: 1
  labels:
    compliance.openshift.io/profile-bundle: rhcos4
  name: rhcos4-audit-rules-login-events
  namespace: openshift-compliance
  ownerReferences:
    - apiVersion: compliance.openshift.io/v1alpha1
      blockOwnerDeletion: true
      controller: true
      kind: ProfileBundle
      name: rhcos4
```



```

uid: 22350850-af4a-4f5c-9a42-5e7b68b82d7d
resourceVersion: "44819"
uid: 75872f1f-3c93-40ca-a69d-44e5438824a4
rationale: Manual editing of these files may indicate nefarious activity, such as
an attacker attempting to remove evidence of an intrusion.
severity: medium
title: Record Attempts to Alter Logon and Logout Events
warning: Manual editing of these files may indicate nefarious activity, such as an
attacker attempting to remove evidence of an intrusion.

```

5.4.1.2. Compliance Operator profile types

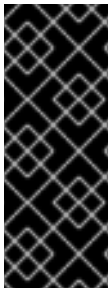
To assess both platform and node compliance for your required benchmarks, you can select from different Compliance Operator profile types.

Platform

Platform profiles evaluate your OpenShift Container Platform cluster components. For example, a Platform-level rule can confirm whether API Server configurations are using strong encryption cyphers.

Node

Node profiles evaluate the OpenShift or RHCOS configuration of each host. You can use two node profiles: **ocp4** node profiles and **rhcos4** node profiles. The **ocp4** node profiles evaluate the OpenShift configuration of each host. For example, they can confirm whether **kubeconfig** files have the correct permissions to meet a compliance standard. The **rhcos4** node profiles evaluate the Red Hat Enterprise Linux CoreOS (RHCOS) configuration of each host. For example, they can confirm whether the SSHD service is configured to disable password logins.



IMPORTANT

For benchmarks that have Node and Platform profiles, such as PCI-DSS, you must run both profiles in your OpenShift Container Platform environment.

For benchmarks that have **ocp4** Platform, **ocp4** Node, and **rhcos4** node profiles, such as FedRAMP High, you must run all three profiles in your OpenShift Container Platform environment.



NOTE

In a cluster with many Nodes, both **ocp4** Node and **rhcos4** Node scans might take a long time to complete.

5.4.1.3. Additional resources

- [ACSC Essential Eight - Hardening Linux Workstations and Servers](#)
- [OpenSCAP project](#)
- [NIST Security Content Automation Protocol \(SCAP\)](#)

5.4.2. Understanding the Custom Resource Definitions

You can use the Custom Resource Definitions (CRDs) provided by the Compliance Operator to run compliance scans and get remediation for the issues found.

The Compliance Operator in the OpenShift Container Platform provides you with several Custom Resource Definitions (CRDs) to run the compliance scans. The Compliance Operator converts security policies into CRDs, which you can use.

The CRD workflow uses these objects:

- **ProfileBundle**, **Profile**, and **TailoredProfile** to define scan requirements
- **ScanSetting** to configure the scan type, occurrence, and location
- **ScanSettingBinding** to process requirements with those settings
- **ComplianceSuite** to monitor deployed scans
- Scan results and remediation after the suite reaches the **DONE** phase

5.4.2.1. Compliance Operator custom resource definition workflow

You can use the Compliance Operator Custom Resource Definition (CRD) workflow to define requirements, configure settings, process scans, monitor compliance checks, and review results.

The CRD workflow includes the following steps:

1. Define your compliance scan requirements.
2. Configure the compliance scan settings.
3. Process compliance requirements with compliance scans settings.
4. Monitor the compliance scans.
5. Check the compliance scan results.

5.4.2.2. ProfileBundle object

When you install the Compliance Operator, it includes ready-to-run **ProfileBundle** objects. The Compliance Operator parses the **ProfileBundle** object and creates a **Profile** object for each profile in the bundle. It also parses **Rule** and **Variable** objects, which are used by the **Profile** object.

Example ProfileBundle object

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ProfileBundle
  name: <profile bundle name>
  namespace: openshift-compliance
status:
  dataStreamStatus: VALID
```

where:

status.dataStreamStatus

Specifies whether the Compliance Operator was able to parse the content files. Value is **VALID** when parsing succeeds.

**NOTE**

When the **contentFile** fails, an **errorMessage** attribute is displayed, which provides details of the error that occurred.

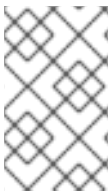
**NOTE**

When you roll back to a known content image from an invalid image, the **ProfileBundle** object stops responding and displays **PENDING** state. As a workaround, you can move to a different image than the earlier one or you can delete and re-create the **ProfileBundle** object to return to the working state.

5.4.2.3. Profile object

You can use the **Profile** object to review parsed out details about an OpenSCAP profile, such as its XCCDF identifier and profile checks for a **Node** or **Platform** type.

The **Profile** object defines the rules and variables that can be evaluated for a certain compliance standard. You can either directly use the **Profile** object or further customize it using a **TailorProfile** object.

**NOTE**

You cannot create or modify the **Profile** object manually because it is derived from a single **ProfileBundle** object. Typically, a single **ProfileBundle** object can include several **Profile** objects.

Example Profile object

```
apiVersion: compliance.openshift.io/v1alpha1
description: <description of the profile>
id: xccdf_org.ssgproject.content_profile_moderate
kind: Profile
metadata:
  annotations:
    compliance.openshift.io/product: <product name>
    compliance.openshift.io/product-type: Node
  creationTimestamp: "YYYY-MM-DDTMM:HH:SSZ"
  generation: 1
  labels:
    compliance.openshift.io/profile-bundle: <profile bundle name>
name: rhcos4-moderate
namespace: openshift-compliance
ownerReferences:
- apiVersion: compliance.openshift.io/v1alpha1
  blockOwnerDeletion: true
  controller: true
  kind: ProfileBundle
  name: <profile bundle name>
  uid: <uid string>
resourceVersion: "<version number>"
selfLink: /apis/compliance.openshift.io/v1alpha1/namespaces/openshift-compliance/profiles/rhcos4-moderate
uid: <uid string>
```

```
rules:
- rhcos4-account-disable-post-pw-expiration
- rhcos4-accounts-no-uid-except-zero
- rhcos4-audit-rules-dac-modification-chmod
- rhcos4-audit-rules-dac-modification-chown
title: <title of the profile>
```

where:

id

Specifies the XCCDF name of the profile. Use this identifier when you define a **ComplianceScan** object as the value of the profile attribute of the scan.

metadata.annotations.compliance.openshift.io/product-type

Specifies either a **Node** or **Platform**. Node profiles scan the cluster nodes and platform profiles scan the Kubernetes platform.

rules

Specifies the list of rules for the profile. Each rule corresponds to a single check.

5.4.2.4. Rule object

You can use the **Rule** object, which represents an individual compliance check, to view check details and understand why a scan result passed or failed.

The **Rule** objects, which form the profiles, are also exposed as objects. You can use the **Rule** object to define your compliance check requirements and specify how a failed compliance check can be remediated.

Example Rule object

```
apiVersion: compliance.openshift.io/v1alpha1
checkType: Platform
description: <description of the rule>
id: xccdf_org.ssgproject.content_rule_configure_network_policies_namespaces
instructions: <manual instructions for the scan>
kind: Rule
metadata:
  annotations:
    compliance.openshift.io/rule: configure-network-policies-namespaces
    control.compliance.openshift.io/CIS-OCP: 5.3.2
    control.compliance.openshift.io/NERC-CIP: CIP-003-3 R4;CIP-003-3 R4.2;CIP-003-3
      R5;CIP-003-3 R6;CIP-004-3 R2.2.4;CIP-004-3 R3;CIP-007-3 R2;CIP-007-3 R2.1;CIP-007-3
      R2.2;CIP-007-3 R2.3;CIP-007-3 R5.1;CIP-007-3 R6.1
    control.compliance.openshift.io/NIST-800-53: AC-4;AC-4(21);CA-3(5);CM-6;CM-6(1);CM-7;CM-
      7(1);SC-7;SC-7(3);SC-7(5);SC-7(8);SC-7(12);SC-7(13);SC-7(18)
  labels:
    compliance.openshift.io/profile-bundle: ocp4
    name: ocp4-configure-network-policies-namespaces
    namespace: openshift-compliance
    rationale: <description of why this rule is checked>
    severity: high
    title: <summary of the rule>
```

where:

checkType

Specifies the type of check this rule executes. **Node** profiles scan the cluster nodes and **Platform** profiles scan the Kubernetes platform. An empty value indicates there is no automated check.

id

Specifies the XCCDF name of the rule, which is parsed directly from the datastream.

severity

Specifies the severity of the rule when it fails.

**NOTE**

The **Rule** object gets an appropriate label for an easy identification of the associated **ProfileBundle** object. The **ProfileBundle** also gets specified in the **OwnerReferences** of this object.

5.4.2.5. TailoredProfile object

You can use the **TailoredProfile** object to modify the default **Profile** object based on your organization requirements. You can enable or disable rules, set variable values, and provide justification for the customization.

After validation, the **TailoredProfile** object creates a **ConfigMap**, which can be referenced by a **ComplianceScan** object.

TIP

You can use the **TailoredProfile** object by referencing it in a **ScanSettingBinding** object. For more information about **ScanSettingBinding**, see [ScanSettingBinding](#) object.

Example TailoredProfile object

```
apiVersion: compliance.openshift.io/v1alpha1
kind: TailoredProfile
metadata:
  name: rhcos4-with-usb
spec:
  extends: rhcos4-moderate
  title: <title of the tailored profile>
  disableRules:
    - name: <name of a rule object to be disabled>
      rationale: <description of why this rule is checked>
  status:
    id: xccdf_compliance.openshift.io_profile_rhcos4-with-usb
    outputRef:
      name: rhcos4-with-usb-tp
      namespace: openshift-compliance
    state: READY
```

where:

spec.extends

Optional parameter. Specifies the name of the **Profile** object upon which the **TailoredProfile** is built. If no value is set, a new profile is created from the **enableRules** list.

status.id

Specifies the XCCDF name of the tailored profile.

status.outputRef.name

Specifies the **ConfigMap** name, which can be used as the value of the **tailoringConfigMap.name** attribute of a **ComplianceScan**.

status.state

Specifies the state of the object such as **READY**, **PENDING**, and **FAILURE**. If the state of the object is **ERROR**, then the attribute **status.errorMessage** provides the reason for the failure.

With the **TailoredProfile** object, you can create a new **Profile** object by using the **TailoredProfile** construct. To create a new **Profile**, set the following configuration parameters:

- an appropriate title
- **extends** value must be empty
- scan type annotation on the **TailoredProfile** object:

```
compliance.openshift.io/product-type: Platform/Node
```



NOTE

If you have not set the **product-type** annotation, the Compliance Operator defaults to **Platform** scan type. Adding the **-node** suffix to the name of the **TailoredProfile** object results in **node** scan type.

5.4.2.6. ScanSetting object

You can use the **ScanSetting** object to define and reuse the operational policies to run your scans, reducing configuration repetition across many scan bindings.

By default, the Compliance Operator creates the following **ScanSetting** objects:

- **default** - Runs a scan every day at 1 AM on both control plane and worker nodes by using a 1Gi Persistent Volume (PV) and keeps the last three results. Remediation is neither applied nor updated automatically.
- **default-auto-apply** - Runs a scan every day at 1 AM on both control plane and worker nodes by using a 1Gi Persistent Volume (PV) and keeps the last three results. Both **autoApplyRemediations** and **autoUpdateRemediations** are set to **true**.

Example ScanSetting object

```
apiVersion: compliance.openshift.io/v1alpha1
autoApplyRemediations: true
autoUpdateRemediations: true
kind: ScanSetting
maxRetryOnTimeout: 3
metadata:
  creationTimestamp: "2022-10-18T20:21:00Z"
```

```

generation: 1
name: default-auto-apply
namespace: openshift-compliance
resourceVersion: "38840"
uid: 8cb0967d-05e0-4d7a-ac1c-08a7f7e89e84
rawResultStorage:
  nodeSelector:
    node-role.kubernetes.io/master: ""
  pvAccessModes:
  - ReadWriteOnce
  rotation: 3
  size: 1Gi
  tolerations:
  - effect: NoSchedule
    key: node-role.kubernetes.io/master
    operator: Exists
  - effect: NoExecute
    key: node.kubernetes.io/not-ready
    operator: Exists
    tolerationSeconds: 300
  - effect: NoExecute
    key: node.kubernetes.io/unreachable
    operator: Exists
    tolerationSeconds: 300
  - effect: NoSchedule
    key: node.kubernetes.io/memory-pressure
    operator: Exists
  roles:
  - master
  - worker
  scanTolerations:
  - operator: Exists
  schedule: 0 1 * * *
  showNotApplicable: false
  strictNodeScan: true
  timeout: 30m

```

where:

autoApplyRemediations

Set to **true** to enable auto remediations. Set to **false** to disable auto remediations.

autoUpdateRemediations

Set to **true** to enable auto remediations for content updates. Set to **false** to disable auto remediations for content updates.

rawResultStorage.rotation

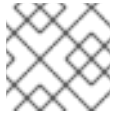
Specifies the number of stored scans in the raw result format. The default value is **3**. As the older results get rotated, the administrator must store the results elsewhere before the rotation happens. To disable the rotation policy, set the value to **0**.

rawResultStorage.size

Specifies the storage size that must be created for the scan to store the raw results. The default value is **1Gi**.

schedule

Specifies how often the scan must be run in cron format.



NOTE

To disable the rotation policy, set the value to **0**.

roles

Specifies the **node-role.kubernetes.io** label value to schedule the scan for **Node** type. This value must match the name of a **MachineConfigPool**.

5.4.2.6.1. ScanSettingBinding object

You can use the **ScanSettingBinding** object to specify your compliance requirements with reference to the **Profile** or **TailoredProfile** object.

The **ScanSettingBinding** object is linked to a **ScanSetting** object, which provides the operational constraints for the scan. Then the Compliance Operator generates the **ComplianceSuite** object based on the **ScanSetting** and **ScanSettingBinding** objects.

Example ScanSettingBinding object

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSettingBinding
metadata:
  name: <name of the scan>
profiles:
  # Node checks
  - name: rhcos4-with-usb
    kind: TailoredProfile
    apiGroup: compliance.openshift.io/v1alpha1
  # Cluster checks
  - name: ocp4-moderate
    kind: Profile
    apiGroup: compliance.openshift.io/v1alpha1
settingsRef:
  name: my-companys-constraints
  kind: ScanSetting
  apiGroup: compliance.openshift.io/v1alpha1
```

where:

profiles

Specifies the details of **Profile** or **TailoredProfile** object to scan your environment.

settingsRef

Specifies the operational constraints, such as schedule and storage size.

The creation of **ScanSetting** and **ScanSettingBinding** objects results in the compliance suite. To get the list of compliance suite, run the following command:

```
$ oc get compliancesuites
```




IMPORTANT

If you delete **ScanSettingBinding**, then compliance suite also is deleted.

5.4.2.6.2. ComplianceSuite object

You can review a **ComplianceSuite** object to keep track of the state of the scans. The object has the raw settings to create scans and an overall result.

For **Node** type scans, map the scan to the **MachineConfigPool**, because the scan has the remediation for any issues. If you specify a label, ensure the label directly applies to a pool.

Example ComplianceSuite object

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ComplianceSuite
metadata:
  name: <name_of_the_suite>
spec:
  autoApplyRemediations: false
  schedule: "0 1 * * *"
  scans:
    - name: workers-scan
      scanType: Node
      profile: xccdf_org.ssgproject.content_profile_moderate
      content: ssg-rhcos4-ds.xml
      contentImage: registry.redhat.io/compliance/openshift-compliance-content-rhel8@sha256:45dc...
      rule: "xccdf_org.ssgproject.content_rule_no_netrc_files"
      nodeSelector:
        node-role.kubernetes.io/worker: ""
  status:
    Phase: DONE
    Result: NON-COMPLIANT
    scanStatuses:
      - name: workers-scan
        phase: DONE
        result: NON-COMPLIANT
```

where:

spec.autoApplyRemediations

Set to **true** to enable auto remediations. Set to **false** to disable auto remediations.

spec.schedule

Specifies how often the scan should be run in cron format.

spec.scans

Specifies a list of scan specifications to run in the cluster.

status.Phase

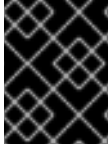
Specifies the progress of the scans.

status.Result

Specifies the overall verdict of the suite.

The suite in the background creates the **ComplianceScan** object based on the **scans** parameter. You can programmatically fetch the **ComplianceSuites** events. To get the events for the suite, run the following command:

```
$ oc get events --field-selector involvedObject.kind=ComplianceSuite,involvedObject.name=<name of the suite>
```



IMPORTANT

You might create errors when you manually define the **ComplianceSuite**, since it contains the XCCDF attributes.

5.4.2.6.3. Advanced ComplianceScan Object

You can use the **ComplianceScan** object to configure advanced options such as custom result storage, debug pods, and scan suspension for troubleshooting and tool integration.

The Compliance Operator includes options for advanced users for debugging or integrating with existing tool. While Red Hat recommends that you should not create a **ComplianceScan** object directly, you can instead manage the object by using a **ComplianceSuite** object.

Example Advanced ComplianceScan object

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ComplianceScan
metadata:
  name: <name_of_the_compliance_scan>
spec:
  scanType: Node
  profile: xccdf_org.ssgproject.content_profile_moderate
  content: ssg-ocp4-ds.xml
  contentImage: registry.redhat.io/compliance/openshift-compliance-content-rhel8@sha256:45dc...
  rule: "xccdf_org.ssgproject.content_rule_no_netrc_files"
  nodeSelector:
    node-role.kubernetes.io/worker: ""
status:
  phase: DONE
  result: NON-COMPLIANT
```

where:

spec.scanType

Specifies either **Node** or **Platform**. Node profiles scan the cluster nodes and platform profiles scan the Kubernetes platform.

spec.profile

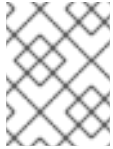
Specifies the XCCDF (Extensible Configuration Checklist Description Format) identifier of the profile that you want to run.

spec.contentImage

Specifies the container image that encapsulates the profile files.

spec.rule

Specifies the scan to run a single rule. This rule must be identified with the XCCDF ID, and must belong to the specified profile.

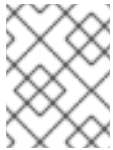


NOTE

If you skip the **rule** parameter, then scan runs for all the available rules of the specified profile.

spec.nodeSelector

If you are on the OpenShift Container Platform and wants to generate a remediation, the **nodeSelector** label must match the **MachineConfigPool** label.



NOTE

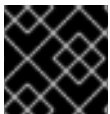
If you do not specify **nodeSelector** parameter or match the **MachineConfig** label, scan will still run, but it will not create remediation.

status.phase

Specifies the current phase of the scan.

status.result

Specifies the verdict of the scan.



IMPORTANT

If you delete a **ComplianceSuite** object, then all the associated scans get deleted.

When the scan is complete, it generates the result as Custom Resources of the **ComplianceCheckResult** object. However, the raw results are available in ARF format. These results are stored in a Persistent Volume (PV), which has a Persistent Volume Claim (PVC) associated with the name of the scan. You can programmatically fetch the **ComplianceScans** events. To generate events for the suite, run the following command:

```
$ oc get events --field-selector involvedObject.kind=ComplianceScan,involvedObject.name=
<name_of_the_compliance_scan>
```

5.4.2.6.4. ComplianceCheckResult object

When you run a scan with a specific profile, several rules in the profiles are verified. For each of these rules, a **ComplianceCheckResult** object is created, which provides the state of the cluster for a specific rule.

Example ComplianceCheckResult object

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ComplianceCheckResult
metadata:
  labels:
    compliance.openshift.io/check-severity: medium
    compliance.openshift.io/check-status: FAIL
    compliance.openshift.io/suite: example-compliancesuite
```

```

  compliance.openshift.io/scan-name: workers-scan
  name: workers-scan-no-direct-root-logins
  namespace: openshift-compliance
  ownerReferences:
  - apiVersion: compliance.openshift.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ComplianceScan
    name: workers-scan
  description: <description of scan check>
  instructions: <manual instructions for the scan>
  id: xccdf_org.ssgproject.content_rule_no_direct_root_logins
  severity: medium
  status: FAIL

```

where:

severity

Specifies the severity of the scan check.

status

Specifies the result of the check. The possible values are:

- **PASS**: Specifies if the check was successful.
- **FAIL**: Specifies if the check was unsuccessful.
- **INFO**: Specifies if the check was successful and found something not severe enough to be considered an error.
- **MANUAL**: Specifies if the check cannot automatically assess the status and manual check is required.
- **INCONSISTENT**: Specifies that the different nodes are reporting different results.
- **ERROR**: Specifies if the check ran successfully, but could not complete.
- **NOTAPPLICABLE**: Specifies if the check did not run as it is not applicable.

To get all the check results from a suite, run the following command:

```

$ oc get compliancecheckresults \
-l compliance.openshift.io/suite=workers-compliancesuite

```

5.4.2.6.5. ComplianceRemediation object

If a Kubernetes fix is available, the Compliance Operator creates a **ComplianceRemediation** object, which you can use to determine a way to fix a problem described in a **ComplianceCheckResult** object.

For a specific check you can have a datastream specified fix. However, if a Kubernetes fix is available, then the Compliance Operator creates a **ComplianceRemediation** object.

Example ComplianceRemediation object

```

  apiVersion: compliance.openshift.io/v1alpha1

```

```

kind: ComplianceRemediation
metadata:
  labels:
    compliance.openshift.io/suite: example-compliancesuite
    compliance.openshift.io/scan-name: workers-scan
    machineconfiguration.openshift.io/role: worker
  name: workers-scan-disable-users-coredumps
  namespace: openshift-compliance
  ownerReferences:
  - apiVersion: compliance.openshift.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ComplianceCheckResult
    name: workers-scan-disable-users-coredumps
    uid: <UID>
spec:
  apply: false
  object:
    current:
      apiVersion: machineconfiguration.openshift.io/v1
      kind: MachineConfig
      spec:
        config:
          ignition:
            version: 2.2.0
          storage:
            files:
            - contents:
                source: data:,%2A%20%20%20%20%20hard%20%20%20core%20%20%20%200
                filesystem: root
                mode: 420
                path: /etc/security/limits.d/75-disable_users_coredumps.conf
            outdated: {}

```

where:

spec.apply

A **true** value specifies the remediation was applied. A **false** value indicates the remediation was not applied.

spec.object.current

Specifies the definition of the remediation.

spec.object.outdated

Specifies remediation that was before parsed from an earlier version of the content. The Compliance Operator still retains the outdated objects to give the administrator a chance to review the new remediations before applying them.

To get all the remediations from a suite, run the following command:

```

$ oc get complianceremediations \
-l compliance.openshift.io/suite=workers-compliancesuite

```

To list all failing checks that can be remediated automatically, run the following command:

```
$ oc get compliancecheckresults \
-l 'compliance.openshift.io/check-status in (FAIL),compliance.openshift.io/automated-remediation'
```

To list all failing checks that can be remediated manually, run the following command:

```
$ oc get compliancecheckresults \
-l 'compliance.openshift.io/check-status in (FAIL),!compliance.openshift.io/automated-remediation'
```

5.5. COMPLIANCE OPERATOR MANAGEMENT

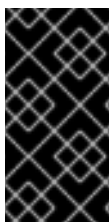
5.5.1. Installing the Compliance Operator

Before you can use the Compliance Operator, you must ensure it is deployed in the cluster.



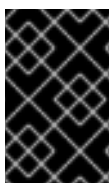
IMPORTANT

All cluster nodes must have the same release version in order for this Operator to function properly. As an example, for nodes running RHCOS, all nodes must have the same RHCOS version.



IMPORTANT

The Compliance Operator might report incorrect results on managed platforms, such as OpenShift Dedicated, Red Hat OpenShift Service on AWS Classic, and Microsoft Azure Red Hat OpenShift. For more information, see the Knowledgebase article on Compliance Operator reports on Managed Services.



IMPORTANT

Before deploying the Compliance Operator, you are required to define persistent storage in your cluster to store the raw results output. For more information, see "Persistent storage overview" and "Managing the default storage class".



IMPORTANT

If the **restricted** Security Context Constraints (SCC) have been modified to contain the **system:authenticated** group or has added **requiredDropCapabilities**, the Compliance Operator might not function properly due to permissions issues. You can create a custom SCC for the Compliance Operator scanner pod service account. For more information, see Additional resources.

5.5.1.1. Installing the Compliance Operator through the web console

You can install the Compliance Operator through the OpenShift Container Platform web console by using the OperatorHub interface.

Prerequisites

- You must have **admin** privileges.
- You must have a **StorageClass** resource configured.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Software Catalog**.
2. Search for the Compliance Operator, then click **Install**.
3. Keep the default selection of **Installation mode** and **namespace** to ensure that the Operator will be installed to the **openshift-compliance** namespace.
4. Click **Install**.

Verification

To confirm that the installation is successful:

1. Navigate to the **Ecosystem → Installed Operators** page.
2. Check that the Compliance Operator is installed in the **openshift-compliance** namespace and its status is **Succeeded**.

If the Operator is not installed successfully:

1. Navigate to the **Ecosystem → Installed Operators** page and inspect the **Status** column for any errors or failures.
2. Navigate to the **Workloads → Pods** page and check the logs in any pods in the **openshift-compliance** project that are reporting issues.

5.5.1.2. Installing the Compliance Operator using the CLI

You can install the Compliance Operator by using the OpenShift CLI by creating the required namespace, Operator group, and subscription objects.

Prerequisites

- You must have **admin** privileges.
- You must have a **StorageClass** resource configured.

Procedure

1. Define a **Namespace** object:

Example namespace-object.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
    pod-security.kubernetes.io/enforce: privileged
  name: openshift-compliance
```

where:

metadata.labels.pod-security.kubernetes.io/enforce

Specifies the pod security label that must be set to **privileged** at the namespace level in OpenShift Container Platform 4.21.

2. Create the **Namespace** object:

```
$ oc create -f namespace-object.yaml
```

3. Define an **OperatorGroup** object:

Example operator-group-object.yaml

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: compliance-operator
  namespace: openshift-compliance
spec:
  targetNamespaces:
    - openshift-compliance
```

4. Create the **OperatorGroup** object:

```
$ oc create -f operator-group-object.yaml
```

5. Define a **Subscription** object:

Example subscription-object.yaml

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: compliance-operator-sub
  namespace: openshift-compliance
spec:
  channel: "stable"
  installPlanApproval: Automatic
  name: compliance-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

6. Create the **Subscription** object:

```
$ oc create -f subscription-object.yaml
```

**NOTE**

If you are setting the global scheduler feature and enable **defaultNodeSelector**, you must create the namespace manually and update the annotations of the **openshift-compliance** namespace, or the namespace where the Compliance Operator was installed, with **openshift.io/node-selector: ""**. This removes the default node selector and prevents deployment failures.

Verification

1. Verify the installation succeeded by inspecting the CSV file:

```
$ oc get csv -n openshift-compliance
```

2. Verify that the Compliance Operator is up and running:

```
$ oc get deploy -n openshift-compliance
```

5.5.1.3. Installing the Compliance Operator on ROSA hosted control planes (HCP)

You can install the Compliance Operator on Red Hat OpenShift Service on AWS by using the OpenShift CLI by creating the required namespace, Operator group, and subscription objects.

As of the Compliance Operator 1.5.0 release, the Operator is tested against Red Hat OpenShift Service on AWS using Hosted control planes.

Red Hat OpenShift Service on AWS Hosted control planes clusters have restricted access to the control plane, which is managed by Red Hat. By default, the Compliance Operator will schedule to nodes within the **master** node pool, which is not available in Red Hat OpenShift Service on AWS Hosted control planes installations. This requires you to configure the **Subscription** object in a way that allows the Operator to schedule on available node pools. This step is necessary for a successful installation on Red Hat OpenShift Service on AWS Hosted control planes clusters.

Prerequisites

- You must have **admin** privileges.
- You must have a **StorageClass** resource configured.

Procedure

1. Define a **Namespace** object:

Example namespace-object.yaml file

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
    pod-security.kubernetes.io/enforce: privileged
  name: openshift-compliance
```

where:

metadata.labels.pod-security.kubernetes.io/enforce

Specifies the pod security label that must be set to **privileged** at the namespace level in OpenShift Container Platform 4.21.

2. Create the **Namespace** object by running the following command:

```
$ oc create -f namespace-object.yaml
```

3. Define an **OperatorGroup** object:

Example operator-group-object.yaml file

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: compliance-operator
  namespace: openshift-compliance
spec:
  targetNamespaces:
    - openshift-compliance
```

4. Create the **OperatorGroup** object by running the following command:

```
$ oc create -f operator-group-object.yaml
```

5. Define a **Subscription** object:

Example subscription-object.yaml file

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: compliance-operator-sub
  namespace: openshift-compliance
spec:
  channel: "stable"
  installPlanApproval: Automatic
  name: compliance-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  config:
    nodeSelector:
      node-role.kubernetes.io/worker: ""
```

- Update the Operator deployment to deploy on **worker** nodes.

6. Create the **Subscription** object by running the following command:

```
$ oc create -f subscription-object.yaml
```

Verification

1. Verify that the installation succeeded by running the following command to inspect the cluster service version (CSV) file:

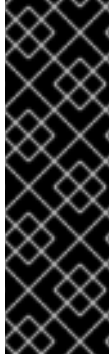
```
$ oc get csv -n openshift-compliance
```

2. Verify that the Compliance Operator is up and running by using the following command:

```
$ oc get deploy -n openshift-compliance
```

5.5.1.4. Installing the Compliance Operator on hosted control planes

Install the Compliance Operator on hosted control planes by creating a **Subscription** file in the software catalog so you can run compliance scans in a hosted control plane environment.



IMPORTANT

Hosted control planes is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You must have **admin** privileges.

Procedure

1. Define a **Namespace** object similar to the following:

Example namespace-object.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
    pod-security.kubernetes.io/enforce: privileged
name: openshift-compliance
```

- In OpenShift Container Platform 4.21, the pod security label must be set to **privileged** at the namespace level.
2. Create the **Namespace** object by running the following command:

```
$ oc create -f namespace-object.yaml
```

3. Define an **OperatorGroup** object:

Example operator-group-object.yaml

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: compliance-operator
  namespace: openshift-compliance
```

```
spec:
  targetNamespaces:
  - openshift-compliance
```

4. Create the **OperatorGroup** object by running the following command:

```
$ oc create -f operator-group-object.yaml
```

5. Define a **Subscription** object:

Example subscription-object.yaml

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: compliance-operator-sub
  namespace: openshift-compliance
spec:
  channel: "stable"
  installPlanApproval: Automatic
  name: compliance-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  config:
    nodeSelector:
      node-role.kubernetes.io/worker: ""
  env:
  - name: PLATFORM
    value: "HyperShift"
```

6. Create the **Subscription** object by running the following command:

```
$ oc create -f subscription-object.yaml
```

Verification

1. Verify the installation succeeded by inspecting the CSV file by running the following command:

```
$ oc get csv -n openshift-compliance
```

2. Verify that the Compliance Operator is up and running by running the following command:

```
$ oc get deploy -n openshift-compliance
```

5.5.1.5. Additional resources

- [Compliance Operator reports incorrect results on Managed Services](#)
- [Persistent storage overview](#)
- [Managing the default storage class](#)
- [Creating a custom SCC for the Compliance Operator](#)

- [Using Operator Lifecycle Manager in disconnected environments](#)

5.5.2. Updating the Compliance Operator

As a cluster administrator, you can update the Compliance Operator on your OpenShift Container Platform cluster.



IMPORTANT

Updating your OpenShift Container Platform cluster to version 4.14 might cause the Compliance Operator to not work as expected. This is due to an ongoing known issue. For more information, see [OCPBUGS-18025](#).

5.5.2.1. About preparing for an Operator update

You can change the update channel to start tracking and receiving updates from a newer channel to access new features and bug fixes. The subscription of an installed Operator specifies an update channel that tracks and receives updates for the Operator.

The names of update channels in a subscription can differ between Operators, but the naming scheme typically follows a common convention within a given Operator. For example, channel names might follow a minor release update stream for the application provided by the Operator (**1.2**, **1.3**) or a release frequency (**stable**, **fast**).



NOTE

You cannot change installed Operators to a channel that is older than the current channel.

Red Hat Customer Portal Labs include an application that helps administrators prepare to update their Operators.

You can use these tools to search for Operators and verify the available Operator versions per update channel across different releases of OpenShift Container Platform. Operators managed by Cluster Version Operator (CVO) are not included.

5.5.2.2. Changing the update channel for an Operator

To change the update channel for an installed Operator, you can use the OpenShift Container Platform web console. The update channel determines which Operator versions your subscription tracks and receives.

TIP

If the approval strategy in the subscription is set to **Automatic**, the update process initiates as soon as a new Operator version is available in the selected channel. If the approval strategy is set to **Manual**, you must manually approve pending updates.

Prerequisites

- An Operator previously installed using Operator Lifecycle Manager (OLM).

Procedure

1. In the web console, navigate to **Ecosystem → Installed Operators**.
2. Click the name of the Operator you want to change the update channel for.
3. Click the **Subscription** tab.
4. Click the name of the update channel under **Update channel**.
5. Click the newer update channel that you want to change to, then click **Save**.
6. For subscriptions with an **Automatic** approval strategy, the update begins automatically. Navigate back to the **Ecosystem → Installed Operators** page to monitor the progress of the update. When complete, the status changes to **Succeeded** and **Up to date**.
For subscriptions with a **Manual** approval strategy, you can manually approve the update from the **Subscription** tab.

5.5.2.3. Approving a pending Operator update manually

If an installed Operator has the approval strategy in its subscription set to **Manual**, you must manually approve the update before installation can begin. Manual approval reviews the changes and control when updates are applied to prevent unexpected downtime.

Prerequisites

- An Operator previously installed using Operator Lifecycle Manager (OLM).

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Installed Operators**.
2. Operators that have a pending update display a status with **Upgrade available**. Click the name of the Operator you want to update.
3. Click the **Subscription** tab. Any updates requiring approval are displayed next to **Upgrade status**. For example, it might display **1 requires approval**.
4. Click **1 requires approval**, then click **Preview Install Plan**.
5. Review the resources that are listed as available for update. When satisfied, click **Approve**.
6. Navigate back to the **Ecosystem → Installed Operators** page to monitor the progress of the update. When complete, the status changes to **Succeeded** and **Up to date**.

5.5.3. Managing the Compliance Operator

You can manage the Compliance Operator security content lifecycle to keep compliance profiles current and create custom **ProfileBundle** objects tailored to your organization security requirements.

5.5.3.1. ProfileBundle CR example

You can configure a **ProfileBundle** to provide the Compliance Operator with the security profiles it needs to scan your cluster.

A **ProfileBundle** custom resource (CR) defines the container image URL in **contentImage** and the compliance content file path in **contentFile**, relative to the root of the file system.

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ProfileBundle
metadata:
  creationTimestamp: "2022-10-19T12:06:30Z"
  finalizers:
    - profilebundle.finalizers.compliance.openshift.io
  generation: 1
  name: rhcos4
  namespace: openshift-compliance
  resourceVersion: "46741"
  uid: 22350850-af4a-4f5c-9a42-5e7b68b82d7d
spec:
  contentFile: ssg-rhcos4-ds.xml
  contentImage: registry.redhat.io/compliance/openshift-compliance-content-rhel8@sha256:900e...
status:
  conditions:
    - lastTransitionTime: "2022-10-19T12:07:51Z"
      message: Profile bundle successfully parsed
      reason: Valid
      status: "True"
      type: Ready
  dataStreamStatus: VALID
```

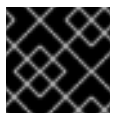
where:

spec.contentFile

Specifies the location of the file containing the compliance content.

spec.contentImage

Specifies the content image location.



IMPORTANT

The base image used for the content images must include **coreutils**.

5.5.3.2. Updating security content

Track **ProfileBundle** updates accurately and ensure predictable compliance profile versions across cluster deployments, by using container image digests instead of tags.

Security content is included as container images that the **ProfileBundle** objects refer to. To accurately track updates to **ProfileBundles** and the custom resources parsed from the bundles, such as rules or profiles, you can view the container image digest in the **ProfileBundle** status.

```
$ oc -n openshift-compliance get profilebundles rhcos4 -oyaml
```

Example output

```
apiVersion: compliance.openshift.io/v1alpha1
```

```
kind: ProfileBundle
metadata:
  creationTimestamp: "2022-10-19T12:06:30Z"
  finalizers:
    - profilebundle.finalizers.compliance.openshift.io
  generation: 1
  name: rhcos4
  namespace: openshift-compliance
  resourceVersion: "46741"
  uid: 22350850-af4a-4f5c-9a42-5e7b68b82d7d
spec:
  contentFile: ssg-rhcos4-ds.xml
  contentImage: registry.redhat.io/compliance/openshift-compliance-content-rhel8@sha256:900e...
status:
  conditions:
    - lastTransitionTime: "2022-10-19T12:07:51Z"
      message: Profile bundle successfully parsed
      reason: Valid
      status: "True"
      type: Ready
  dataStreamStatus: VALID
```

where:

spec.contentImage

Specifies the security container image.

Each **ProfileBundle** is backed by a deployment. When the Compliance Operator detects that the container image digest has changed, the deployment is updated to reflect the change and parse the content again. Using the digest instead of a tag ensures that you use a stable and predictable set of profiles.

5.5.3.3. Additional resources

- [Using Operator Lifecycle Manager in disconnected environments](#)

5.5.4. Uninstalling the Compliance Operator

You can remove the OpenShift Compliance Operator from your cluster by using the OpenShift Container Platform web console or the OpenShift CLI (**oc**).

5.5.4.1. Uninstalling the OpenShift Compliance Operator from OpenShift Container Platform using the web console

To remove the Compliance Operator, you must first delete the objects in the namespace. After the objects are removed, you can remove the Operator and its namespace by deleting the **openshift-compliance** project.

Prerequisites

- Access to an OpenShift Container Platform cluster by using an account with **cluster-admin** permissions.
- The OpenShift Compliance Operator is installed.

Procedure

1. Go to the **Ecosystem → Installed Operators → Compliance Operator** page.
 - a. Click **All instances**.
 - b. In **All namespaces**, click the Options menu  and delete all **ScanSettingBinding**, **ComplianceSuite**, **ComplianceScan**, and **ProfileBundle** objects.
2. Switch to the **Administration → Ecosystem → Installed Operators** page.
3. Click the Options menu  on the **Compliance Operator** entry and select **Uninstall Operator**.
4. Switch to the **Home → Projects** page.
5. Search for 'compliance'.
6. Click the Options menu  next to the **openshift-compliance** project, and select **Delete Project**.
 - a. Confirm the deletion by typing **openshift-compliance** in the dialog box, and click **Delete**.

5.5.4.2. Uninstalling the OpenShift Compliance Operator from OpenShift Container Platform using the CLI

To remove the Compliance Operator, you must first delete the objects in the namespace. After the objects are removed, you can remove the Operator and its namespace by deleting the **openshift-compliance** project.

Prerequisites

- Access to an OpenShift Container Platform cluster by using an account with **cluster-admin** permissions.
- The OpenShift Compliance Operator is installed.

Procedure

1. Delete all objects in the namespace.
 - a. Delete the **ScanSettingBinding** objects:


```
$ oc delete ssb --all -n openshift-compliance
```
 - b. Delete the **ScanSetting** objects:


```
$ oc delete ss --all -n openshift-compliance
```
 - c. Delete the **ComplianceSuite** objects:


```
-
```

```
$ oc delete suite --all -n openshift-compliance
```

- d. Delete the **ComplianceScan** objects:

```
$ oc delete scan --all -n openshift-compliance
```

- e. Delete the **ProfileBundle** objects:

```
$ oc delete profilebundle.compliance --all -n openshift-compliance
```

2. Delete the Subscription object:

```
$ oc delete sub --all -n openshift-compliance
```

3. Delete the CSV object:

```
$ oc delete csv --all -n openshift-compliance
```

4. Delete the project:

```
$ oc delete project openshift-compliance
```

Example output

```
project.project.openshift.io "openshift-compliance" deleted
```

Verification

1. Confirm the namespace is deleted:

```
$ oc get project/openshift-compliance
```

Example output

```
Error from server (NotFound): namespaces "openshift-compliance" not found
```

5.5.4.3. Additional resources

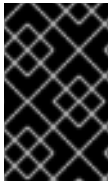
- [Installing the Compliance Operator](#)
- [Managing the Compliance Operator](#)

5.6. COMPLIANCE OPERATOR SCAN MANAGEMENT

5.6.1. Supported compliance profiles

There are several profiles available as part of the Compliance Operator (CO) installation. Although you can use these profiles to assess gaps in a cluster, usage alone does not infer or guarantee compliance with a particular profile and is not an auditor.

To be compliant or certified under these various standards, you need to engage an authorized auditor such as a Qualified Security Assessor (QSA), Joint Authorization Board (JAB), or other industry recognized regulatory authority to assess your environment. You are required to work with an authorized auditor to achieve compliance with a standard.



IMPORTANT

The Compliance Operator might report incorrect results on some managed platforms, such as OpenShift Dedicated and Azure Red Hat OpenShift. For more information, see Red Hat Knowledgebase Solution #6983418.

5.6.1.1. Compliance profiles

When working with the Compliance Operator (CO), you can use the profiles provided by the Operator to meet industry standard benchmarks.



NOTE

The following tables reflect the latest available profiles in the Compliance Operator. The only supported versions of CIS and DISA STIG profiles will be the latest. Our recommendation to customers is to use **ocp4-cis** and **ocp4-cis-node**, **ocp4-stig**, and **ocp4-stig-node**, which always point to the latest version.

5.6.1.1.1. CIS compliance profiles

Table 5.1. Supported CIS compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-cis ^[1]	CIS Red Hat OpenShift Container Platform Benchmark v1.9.0	Platform	CIS Benchmarks™ ^[4]	x86_64 ppc64le s390x aarch64	
ocp4-cis-1-9 ^[3]	CIS Red Hat OpenShift Container Platform Benchmark v1.9.0	Platform	CIS Benchmarks™ ^[4]	x86_64 ppc64le s390x aarch64	

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-cis-node ^[1]	CIS Red Hat OpenShift Container Platform Benchmark v1.9.0	Node [2]	CIS Benchmarks™ [4]	x86_64 ppc64le s390x aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-cis-node-1-9 ^[3]	CIS Red Hat OpenShift Container Platform Benchmark v1.9.0	Node [2]	CIS Benchmarks™ [4]	x86_64 ppc64le s390x aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

1. The **ocp4-cis** and **ocp4-cis-node** profiles maintain the most up-to-date version of the CIS benchmark as it becomes available in the Compliance Operator. If you want to adhere to a specific version, such as CIS v1.9.0, use the **ocp4-cis-1-9** and **ocp4-cis-node-1-9** profiles.
2. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.
3. All earlier CIS profiles are superceded by CIS v1.9.0. It is recommended to apply the latest profile to your environment.
4. To locate the CIS OpenShift Container Platform v4 Benchmark, go to [CIS Benchmarks](#) and click **Download Latest CIS Benchmark**, where you can then register to download the benchmark.

5.6.1.1.2. BSI Profile Support

Table 5.2. Supported BSI compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-bsi ^[1]	BSI IT-Grundschutz (Basic Protection) Building Block SYS.1.6 and APP.4.4	Platform	BSI Basic Protection Compendium	x86_64	

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-bsi-node ^[1]	BSI IT- Grundschutz (Basic Protection) Building Block SYS.1.6 and APP.4.4	Node [2]	BSI Basic Protection Compendium	x86_64	
rhcos4-bsi ^[1]	BSI IT- Grundschutz (Basic Protection) Building Block SYS.1.6 and APP.4.4	Node [2]	BSI Basic Protection Compendium	x86_64	
ocp4-bsi-2022 ^[3]	BSI IT- Grundschutz (Basic Protection) Building Block SYS.1.6 and APP.4.4	Platform	BSI Basic Protection Compendium	x86_64	
ocp4-bsi-node-2022 ^[3]	BSI IT- Grundschutz (Basic Protection) Building Block SYS.1.6 and APP.4.4	Node [2]	BSI Basic Protection Compendium	x86_64	
rhcos4-bsi-2022 ^[3]	BSI IT- Grundschutz (Basic Protection) Building Block SYS.1.6 and APP.4.4	Node [2]	BSI Basic Protection Compendium	x86_64	

1. The **ocp4-bsi**, **ocp4-bsi-node**, and **rhcos4-bsi** profiles maintain the most up-to-date version of the BSI Basic Protection Profile as it becomes available in the Compliance Operator. If you want to adhere to a specific version, such as BSI 2022, use the **ocp4-bsi-2022**, **ocp4-bsi-node-2022** or **rhcos4-bsi-2022** profiles.
2. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.
3. Edition 2022 is the latest available English edition of the BSI IT-Grundschutz (Basic Protection) compendium. There were no changes for Building Blocks SYS.1.6 and APP.4.4, SYS.1.1, and SYS.1.3 in the latest published German compendium (edition 2023).

For more information, see [BSI Quick Check](#).

5.6.1.1.3. Essential Eight compliance profiles

Table 5.3. Supported Essential Eight compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-e8	Australian Cyber Security Centre (ACSC) Essential Eight	Platform	ACSC Hardening Linux Workstations and Servers	x86_64	
rhcos4-e8	Australian Cyber Security Centre (ACSC) Essential Eight	Node	ACSC Hardening Linux Workstations and Servers	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

5.6.1.1.4. FedRAMP High compliance profiles

**IMPORTANT**

Applying automatic remediations to any profile, such as **rhcos4-stig**, that uses the **service-sshd-disabled** rule, automatically disables the **sshd** service. This situation blocks SSH access to control plane nodes and compute nodes. To keep the SSH access enabled, create a **TailoredProfile** object and set the **rhcos4-service-sshd-disabled** rule value for the **disableRules** parameter.

Table 5.4. Supported FedRAMP High compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-high ^[1]	NIST 800-53 High-Impact Baseline for Red Hat OpenShift - Platform level	Platform	NIST SP-800-53 Release Search	x86_64	
ocp4-high-node ^[1]	NIST 800-53 High-Impact Baseline for Red Hat OpenShift - Node level	Node [2]	NIST SP-800-53 Release Search	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-high-node- rev-4	NIST 800-53 High-Impact Baseline for Red Hat OpenShift - Node level	Node [2]	NIST SP-800-53 Release Search	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-high-rev-4	NIST 800-53 High-Impact Baseline for Red Hat OpenShift - Platform level	Platform	NIST SP-800-53 Release Search	x86_64	
rhcos4-high ^[1]	NIST 800-53 High-Impact Baseline for Red Hat Enterprise Linux CoreOS	Node	NIST SP-800-53 Release Search	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
rhcos4-high-rev-4	NIST 800-53 High-Impact Baseline for Red Hat Enterprise Linux CoreOS	Node	NIST SP-800-53 Release Search	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

1. The **ocp4-high**, **ocp4-high-node** and **rhcos4-high** profiles maintain the most up-to-date version of the FedRAMP High standard as it becomes available in the Compliance Operator. If you want to adhere to a specific version, such as FedRAMP high R4, use the **ocp4-high-rev-4** and **ocp4-high-node-rev-4** profiles.
2. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.

5.6.1.1.5. FedRAMP Moderate compliance profiles

Table 5.5. Supported FedRAMP Moderate compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
---------	---------------	-----------------	-------------------------------------	---------------------------------	------------------------

Profile	Profile title	Application	Industry compliance benchmark	Supported architectures	Supported platforms
ocp4-moderate ^[1]	NIST 800-53 Moderate-Impact Baseline for Red Hat OpenShift - Platform level	Platform	NIST SP-800-53 Release Search	x86_64 ppc64le s390x aarch64	
ocp4-moderate-node ^[1]	NIST 800-53 Moderate-Impact Baseline for Red Hat OpenShift - Node level	Node [2]	NIST SP-800-53 Release Search	x86_64 ppc64le s390x aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-moderate-node-rev-4	NIST 800-53 Moderate-Impact Baseline for Red Hat OpenShift - Node level	Node [2]	NIST SP-800-53 Release Search	x86_64 ppc64le s390x aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-moderate-rev-4	NIST 800-53 Moderate-Impact Baseline for Red Hat OpenShift - Platform level	Platform	NIST SP-800-53 Release Search	x86_64 ppc64le s390x aarch64	
rhcos4-moderate ^[1]	NIST 800-53 Moderate-Impact Baseline for Red Hat Enterprise Linux CoreOS	Node	NIST SP-800-53 Release Search	x86_64 aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
rhcos4-moderate- rev-4	NIST 800-53 Moderate-Impact Baseline for Red Hat Enterprise Linux CoreOS	Node	NIST SP-800-53 Release Search	x86_64 aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

1. The **ocp4-moderate**, **ocp4-moderate-node** and **rhcos4-moderate** profiles maintain the most up-to-date version of the FedRAMP Moderate standard as it becomes available in the Compliance Operator. If you want to adhere to a specific version, such as FedRAMP Moderate R4, use the **ocp4-moderate-rev-4** and **ocp4-moderate-node-rev-4** profiles.
2. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.

5.6.1.1.6. NERC-CIP compliance profiles

Table 5.6. Supported NERC-CIP compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-nerc-cip	North American Electric Reliability Corporation (NERC) Critical Infrastructure Protection (CIP) cybersecurity standards profile for the OpenShift Container Platform - Platform level	Platform	NERC CIP Standards	x86_64	

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-nerc-cip-node	North American Electric Reliability Corporation (NERC) Critical Infrastructure Protection (CIP) cybersecurity standards profile for the OpenShift Container Platform - Node level	Node [1]	NERC CIP Standards	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
rhcos4-nerc-cip	North American Electric Reliability Corporation (NERC) Critical Infrastructure Protection (CIP) cybersecurity standards profile for Red Hat Enterprise Linux CoreOS	Node	NERC CIP Standards	x86_64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

1. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.

5.6.1.1.7. PCI-DSS compliance profiles

Table 5.7. Supported PCI-DSS compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-pci-dss [1]	PCI-DSS v4 Control Baseline for OpenShift Container Platform 4	Platform	PCI Security Standards® Council Document Library	x86_64 ppc64le aarch64	

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-pci-dss-3-2 [3]	PCI-DSS v3.2.1 Control Baseline for OpenShift Container Platform 4	Platform	PCI Security Standards® Council Document Library	x86_64 ppc64le s390x aarch64	
ocp4-pci-dss-4-0	PCI-DSS v4 Control Baseline for OpenShift Container Platform 4	Platform	PCI Security Standards® Council Document Library	x86_64 ppc64le aarch64	
ocp4-pci-dss-node ^[1]	PCI-DSS v4 Control Baseline for OpenShift Container Platform 4	Node [2]	PCI Security Standards® Council Document Library	x86_64 ppc64le aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-pci-dss-node-3-2 ^[3]	PCI-DSS v3.2.1 Control Baseline for OpenShift Container Platform 4	Node [2]	PCI Security Standards® Council Document Library	x86_64 ppc64le s390x aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-pci-dss-node-4-0	PCI-DSS v4 Control Baseline for OpenShift Container Platform 4	Node [2]	PCI Security Standards® Council Document Library	x86_64 ppc64le aarch64	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

1. The **ocp4-pci-dss** and **ocp4-pci-dss-node** profiles maintain the most up-to-date version of the PCI-DSS standard as it becomes available in the Compliance Operator. If you want to adhere to a specific version, such as PCI-DSS v3.2.1, use the **ocp4-pci-dss-3-2** and **ocp4-pci-dss-node-3-2** profiles.
2. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.

3. PCI-DSS v3.2.1 is superseded by PCI-DSS v4. It is recommended to apply the latest profile to your environment.

5.6.1.1.8. STIG compliance profiles



IMPORTANT

Applying automatic remediations to any profile, such as **rhcos4-stig**, that uses the **service-sshd-disabled** rule, automatically disables the **sshd** service. This situation blocks SSH access to control plane nodes and compute nodes. To keep the SSH access enabled, create a **TailoredProfile** object and set the **rhcos4-service-sshd-disabled** rule value for the **disableRules** parameter.

Table 5.8. Supported STIG compliance profiles

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-stig ^[1]	Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) for Red Hat OpenShift ^[3]	Platform	DISA-STIG	x86_64 ppc64le	
ocp4-stig-node ^[1]	Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) for Red Hat OpenShift ^[3]	Node ^[2]	DISA-STIG	x86_64 ppc64le	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
ocp4-stig-v2r3	Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) for Red Hat OpenShift V2R3	Platform	DISA-STIG	x86_64 ppc64le	

Profile	Profile title	Applica tion	Industry compliance benchmark	Support ed archite ctures	Supported platforms
ocp4-stig-node-v2r3 ^[1]	Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) for Red Hat OpenShift V2R3	Node	DISA-STIG	x86_64 ppc64le	
rhcos4-stig ^[1]	Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) for Red Hat OpenShift ^[3]	Node	DISA-STIG	x86_64 ppc64le	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)
rhcos4-stig-v2r3	Defense Information Systems Agency Security Technical Implementation Guide (DISA STIG) for Red Hat OpenShift V2R3	Node	DISA-STIG	x86_64 ppc64le	Red Hat OpenShift Service on AWS with hosted control planes (ROSA HCP)

1. The **ocp4-stig**, **ocp4-stig-node** and **rhcos4-stig** profiles maintain the most up-to-date version of the DISA-STIG benchmark as it becomes available in the Compliance Operator. If you want to adhere to a specific version, such as DISA-STIG V2R3, use the **ocp4-stig-v2r3** and **ocp4-stig-node-v2r3** profiles.
2. Node profiles must be used with the relevant Platform profile. For more information, see *Compliance Operator profile types*.
3. DISA-STIG V1R2 is superseded by DISA-STIG V2R3. It is recommended to apply the latest profile to your environment.

5.6.1.1.9. About extended compliance profiles

Some compliance profiles have controls that require following industry best practices, resulting in some profiles extending others. Combining the Center for Internet Security (CIS) best practices with National Institute of Standards and Technology (NIST) security frameworks establishes a path to a secure and compliant environment.

^[1] [https://www.redhat.com/en/topics/security/understanding-compliance-requirements-for-red-hat-openshift](#)

For example, the NIST High-Impact and Moderate-Impact profiles extend the CIS profile to achieve compliance. As a result, extended compliance profiles eliminate the need to run both profiles in a single cluster.

Table 5.9. Profile extensions

Profile	Extends
ocp4-pci-dss	ocp4-cis
ocp4-pci-dss-node	ocp4-cis-node
ocp4-high	ocp4-cis
ocp4-high-node	ocp4-cis-node
ocp4-moderate	ocp4-cis
ocp4-moderate-node	ocp4-cis-node
ocp4-nerc-cip	ocp4-moderate
ocp4-nerc-cip-node	ocp4-moderate-node

5.6.1.10. Compliance Operator profile types

To assess both platform and node compliance for your required benchmarks, you can select from different Compliance Operator profile types.

Platform

Platform profiles evaluate your OpenShift Container Platform cluster components. For example, a Platform-level rule can confirm whether APIServer configurations are using strong encryption cyphers.

Node

Node profiles evaluate the OpenShift or RHCOS configuration of each host. You can use two node profiles: **ocp4** node profiles and **rhcos4** node profiles. The **ocp4** node profiles evaluate the OpenShift configuration of each host. For example, they can confirm whether **kubeconfig** files have the correct permissions to meet a compliance standard. The **rhcos4** node profiles evaluate the Red Hat Enterprise Linux CoreOS (RHCOS) configuration of each host. For example, they can confirm whether the SSHD service is configured to disable password logins.



IMPORTANT

For benchmarks that have Node and Platform profiles, such as PCI-DSS, you must run both profiles in your OpenShift Container Platform environment.

For benchmarks that have **ocp4** Platform, **ocp4** Node, and **rhcos4** node profiles, such as FedRAMP High, you must run all three profiles in your OpenShift Container Platform environment.

**NOTE**

In a cluster with many Nodes, both **ocp4** Node and **rhcos4** Node scans might take a long time to complete.

5.6.1.2. Additional resources

- [Red Hat Knowledgebase Solution #6983418](#)
- [Product Compliance](#)

5.6.2. Compliance Operator scans

You can use the **ScanSetting** and **ScanSettingBinding** APIs to run compliance scans with the Compliance Operator.

For more information on these API objects, run the following command:

```
$ oc explain scansettings
```

or

```
$ oc explain scansettingbindings
```

5.6.2.1. Running compliance scans

You can run a scan using the Center for Internet Security (CIS) profiles to evaluate cluster compliance against CIS benchmarks. For convenience, the Compliance Operator creates a **ScanSetting** object with reasonable defaults on startup. This **ScanSetting** object is named **default**.

**NOTE**

For all-in-one control plane and worker nodes, the compliance scan runs twice on the worker and control plane nodes. The compliance scan might generate inconsistent scan results. You can avoid inconsistent results by defining only a single role in the **ScanSetting** object.

**IMPORTANT**

Compliance Operator scans report **INCONSISTENT** on clusters with multi-architecture compute machines whether the control plane uses **aarch64** or **x86** CPUs. This is due to the same rule behaving differently on different architectures. This applies only to node scans, where the Compliance Operator aggregates results from multiple nodes into a single result.

For more information about inconsistent scan results, see [Compliance Operator shows INCONSISTENT scan result with worker node](#).

Procedure

1. Inspect the **ScanSetting** object by running the following command:

```
$ oc describe scansettings default -n openshift-compliance
```

Example output

```
Name:          default
Namespace:     openshift-compliance
Labels:        <none>
Annotations:   <none>
API Version:   compliance.openshift.io/v1alpha1
Kind:          ScanSetting
Max Retry On Timeout: 3
Metadata:
  Creation Timestamp: 2024-07-16T14:56:42Z
  Generation:        2
  Resource Version:   91655682
  UID:               50358cf1-57a8-4f69-ac50-5c7a5938e402
Raw Result Storage:
  Node Selector:
    node-role.kubernetes.io/master:
  Pv Access Modes:
    ReadWriteOnce
  Rotation:        3
  Size:            1Gi
  Storage Class Name: standard
Tolerations:
  Effect:          NoSchedule
  Key:             node-role.kubernetes.io/master
  Operator:        Exists
  Effect:          NoExecute
  Key:             node.kubernetes.io/not-ready
  Operator:        Exists
  Toleration Seconds: 300
  Effect:          NoExecute
  Key:             node.kubernetes.io/unreachable
  Operator:        Exists
  Toleration Seconds: 300
  Effect:          NoSchedule
  Key:             node.kubernetes.io/memory-pressure
  Operator:        Exists
Roles:
  master
  worker
Scan Tolerations:
  Operator:        Exists
  Schedule:        0 1 * * *
Show Not Applicable: false
Strict Node Scan: true
Suspend:          false
Timeout:          30m
Events:           <none>
```

where:

Raw Result Storage.pvAccessModes.ReadWriteOnce

Specifies access mode for the PV created by the Compliance Operator with the results of

the scans. By default, the PV will use access mode **ReadWriteOnce** because the Compliance Operator cannot make any assumptions about the storage classes configured on the cluster. Additionally, **ReadWriteOnce** access mode is available on most clusters. If you need to fetch the scan results, you can do so by using a helper pod, which also binds the volume. Volumes that use the **ReadWriteOnce** access mode can be mounted by only one pod at time, so it is important to remember to delete the helper pods. Otherwise, the Compliance Operator will not be able to reuse the volume for subsequent scans.

Raw Result Storage.Rotation

Specifies that the Compliance Operator keeps the results of three subsequent scans in the volume; older scans are rotated.

Raw Result Storage.size

Specifies that the Compliance Operator will allocate one GB of storage for the scan results.

Raw Result Storage.Storage Class Name

Specifies the **storageClassName** value to use when creating the **PersistentVolumeClaim** object to store the raw results. The default value is null, which will attempt to use the default storage class configured in the cluster. If there is no default class specified, then you must set a default class.

Roles

If the scan setting uses any profiles that scan cluster nodes, scan these node roles.

Scan Tolerations

The default scan setting object scans all the nodes.

Schedule

The default scan setting object runs scans at 01:00 each day.

As an alternative to the default scan setting, you can use **default-auto-apply**, which has the following settings:

```
Name:                default-auto-apply
Namespace:           openshift-compliance
Labels:              <none>
Annotations:         <none>
API Version:         compliance.openshift.io/v1alpha1
Auto Apply Remediations: true
Auto Update Remediations: true
Kind:                ScanSetting
Metadata:
  Creation Timestamp: 2022-10-18T20:21:00Z
  Generation:        1
  Managed Fields:
    API Version: compliance.openshift.io/v1alpha1
    Fields Type: FieldsV1
    fieldsV1:
      f:autoApplyRemediations:
      f:autoUpdateRemediations:
      f:rawResultStorage:
      ..
      f:nodeSelector:
      ..
      f:node-role.kubernetes.io/master:
      f:pvAccessModes:
      f:rotation:
      f:size:
```

```

    f:tolerations:
    f:roles:
    f:scanTolerations:
    f:schedule:
    f:showNotApplicable:
    f:strictNodeScan:
  Manager:      compliance-operator
  Operation:    Update
  Time:         2022-10-18T20:21:00Z
  Resource Version: 38840
  UID:          8cb0967d-05e0-4d7a-ac1c-08a7f7e89e84
  Raw Result Storage:
  Node Selector:
    node-role.kubernetes.io/master:
  Pv Access Modes:
    ReadWriteOnce
  Rotation: 3
  Size: 1Gi
  Tolerations:
    Effect:      NoSchedule
    Key:         node-role.kubernetes.io/master
    Operator:    Exists
    Effect:      NoExecute
    Key:         node.kubernetes.io/not-ready
    Operator:    Exists
    Toleration Seconds: 300
    Effect:      NoExecute
    Key:         node.kubernetes.io/unreachable
    Operator:    Exists
    Toleration Seconds: 300
    Effect:      NoSchedule
    Key:         node.kubernetes.io/memory-pressure
    Operator:    Exists
  Roles:
    master
    worker
  Scan Tolerations:
    Operator:    Exists
    Schedule:    0 1 * * *
  Show Not Applicable: false
  Strict Node Scan: true
  Events:       <none>

```

- Setting **autoUpdateRemediations** and **autoApplyRemediations** flags to **true** allows you to easily create **ScanSetting** objects that auto-remediate without extra steps.

2. Create a **ScanSettingBinding** object that binds to the default **ScanSetting** object and scans the cluster using the **cis** and **cis-node** profiles. For example:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSettingBinding
metadata:
  name: cis-compliance
  namespace: openshift-compliance
profiles:

```

```

- name: ocp4-cis-node
  kind: Profile
  apiGroup: compliance.openshift.io/v1alpha1
- name: ocp4-cis
  kind: Profile
  apiGroup: compliance.openshift.io/v1alpha1
settingsRef:
  name: default
  kind: ScanSetting
  apiGroup: compliance.openshift.io/v1alpha1

```

3. Create the **ScanSettingBinding** object by running:

```
$ oc create -f <file-name>.yaml -n openshift-compliance
```

At this point in the process, the **ScanSettingBinding** object is reconciled and based on the **Binding** and the **Bound** settings. The Compliance Operator creates a **ComplianceSuite** object and the associated **ComplianceScan** objects.

4. Follow the compliance scan progress by running:

```
$ oc get compliancescan -w -n openshift-compliance
```

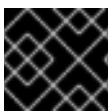
The scans progress through the scanning phases and eventually reach the **DONE** phase when complete. In most cases, the result of the scan is **NON-COMPLIANT**. You can review the scan results and start applying remediations to make the cluster compliant.

5.6.2.2. Setting custom storage size for results

Although **ComplianceCheckResult** custom resources summarize one check across all scanned nodes, raw scanner results in ARF format are too large to store in etcd-backed Kubernetes resources. You can store them on a per-scan persistent volume and increase the default 1 GiB size by setting the **rawResultStorage.size** value in a **ScanSetting** or **ComplianceScan** resource.

A related parameter is **rawResultStorage.rotation** which controls how many scans are retained in the PV before the older scans are rotated. The default value is 3, setting the rotation policy to 0 disables the rotation. Given the default rotation policy and an estimate of 100MB per a raw ARF scan report, you can calculate the right PV size for your environment.

Because OpenShift Container Platform can be deployed in a variety of public clouds or bare metal, the Compliance Operator cannot determine available storage configurations. By default, the Compliance Operator will try to create the PV for storing results by using the default storage class of the cluster, but a custom storage class can be configured using the **rawResultStorage.StorageClassName** attribute.



IMPORTANT

If your cluster does not specify a default storage class, this attribute must be set.

- Configure the **ScanSetting** custom resource to use a standard storage class and create persistent volumes that are 10GB in size and keep the last 10 results:

Example ScanSetting CR

```

apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  name: default
  namespace: openshift-compliance
rawResultStorage:
  storageClassName: standard
  rotation: 10
  size: 10Gi
roles:
- worker
- master
scanTolerations:
- effect: NoSchedule
  key: node-role.kubernetes.io/master
  operator: Exists
schedule: '0 1 * * *'

```

5.6.2.3. Scheduling the result server pod on a worker node

The result server pod mounts the persistent volume (PV) that stores the raw Asset Reporting Format (ARF) scan results. You can use the **nodeSelector** and **tolerations** attributes to configure the location of the result server pod to meet your organization's requirements.

This is helpful for those environments where control plane nodes are not permitted to mount persistent volumes.

Procedure

- Create a **ScanSetting** custom resource (CR) for the Compliance Operator:
 - a. Define the **ScanSetting** CR, and save the YAML file, for example, **rs-workers.yaml**:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  name: rs-on-workers
  namespace: openshift-compliance
rawResultStorage:
  nodeSelector:
    node-role.kubernetes.io/worker: ""
  pvAccessModes:
  - ReadWriteOnce
  rotation: 3
  size: 1Gi
  tolerations:
  - operator: Exists
roles:
- worker
- master
scanTolerations:
- operator: Exists
schedule: 0 1 * * *

```

where:

rawResultStorage.nodeSelector.node-role.kubernetes.io/worker

Specifies the Compliance Operator uses this node to store scan results in ARF format.

rawResultStorage.tolerations.operator

Specifies the result server pod tolerates all taints.

- b. To create the **ScanSetting** CR, run the following command:

```
$ oc create -f rs-workers.yaml
```

Verification

- To verify that the **ScanSetting** object is created, run the following command:

```
$ oc get scansettings rs-on-workers -n openshift-compliance -o yaml
```

Example output

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  creationTimestamp: "2021-11-19T19:36:36Z"
  generation: 1
  name: rs-on-workers
  namespace: openshift-compliance
  resourceVersion: "48305"
  uid: 43fdcf5f-15a7-445a-8bbc-0e4a160cd46e
rawResultStorage:
  nodeSelector:
    node-role.kubernetes.io/worker: ""
  pvAccessModes:
  - ReadWriteOnce
  rotation: 3
  size: 1Gi
  tolerations:
  - operator: Exists
roles:
- worker
- master
scanTolerations:
- operator: Exists
schedule: 0 1 * * *
strictNodeScan: true
```

5.6.2.4. ScanSetting Custom Resource

You can configure the scan limits attribute of the **ScanSetting** custom resource to override the default CPU and memory limits of scanner pods to meet your environment's resource requirements.

The Compliance Operator uses defaults of 500Mi memory and 100m CPU for the scanner container, and 200Mi memory and 100m CPU for the **api-resource-collector** container. To set the memory limits of the Operator, modify the **Subscription** object if installed through OLM or the Operator deployment itself.

**IMPORTANT**

Increasing the memory limit for the Compliance Operator or the scanner pods is needed if the default limits are not sufficient and the Operator or scanner pods are ended by the Out Of Memory (OOM) process. For more information, see [Increasing Compliance Operator resource limits](#).

5.6.2.5. Configuring the hosted control planes management cluster

If you are hosting your own Hosted control planes or Hypershift environment and want to scan a Hosted Cluster from the management cluster, you will need to set the name and prefix namespace for the target Hosted Cluster. You can achieve this by creating a **TailoredProfile**.

**IMPORTANT**

This procedure only applies to users managing their own hosted control planes environment.

**NOTE**

Only **ocp4-cis** and **ocp4-pci-dss** profiles are supported in hosted control planes management clusters.

Prerequisites

- The Compliance Operator is installed in the management cluster.

Procedure

1. Obtain the **name** and **namespace** of the hosted cluster to be scanned by running the following command:

```
$ oc get hostedcluster -A
```

Example output

```

NAMESPACE   NAME                               VERSION  KUBECONFIG
PROGRESS    AVAILABLE  PROGRESSING  MESSAGE
local-cluster 79136a1bdb84b3c13217 4.13.5  79136a1bdb84b3c13217-admin-kubeconfig
Completed   True       False       The hosted control plane is available
```

2. In the management cluster, create a **TailoredProfile** extending the scan Profile and define the name and namespace of the Hosted Cluster to be scanned:

Example management-tailoredprofile.yaml

```

apiVersion: compliance.openshift.io/v1alpha1
kind: TailoredProfile
metadata:
  name: hypershift-cisk57aw88gry
  namespace: openshift-compliance
spec:
  description: This profile test required rules
```

```

extends: ocp4-cis
title: Management namespace profile
setValues:
- name: ocp4-hypershift-cluster
  rationale: This value is used for HyperShift version detection
  value: 79136a1bdb84b3c13217
- name: ocp4-hypershift-namespace-prefix
  rationale: This value is used for HyperShift control plane namespace detection
  value: local-cluster

```

where:

spec.extends

Specifies the name of the **Profile** object upon which the **TailoredProfile** is built. Only **ocp4-cis** and **ocp4-pci-dss** profiles are supported in hosted control planes management clusters.

spec.setValues.value

Specifies the output in the previous step.

spec.setValues.value

Specifies the **NAMESPACE** from the output in the previous step.

3. Create the **TailoredProfile**:

```
$ oc create -n openshift-compliance -f mgmt-tp.yaml
```

5.6.2.6. Applying resource requests and limits

You can configure a container's requests and limits for memory and CPU to define how much CPU time and memory that the container can use.

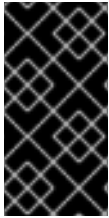
When the kubelet starts a container as part of a Pod, the kubelet passes that container's requests and limits for memory and CPU to the container runtime. In Linux, the container runtime configures the kernel cgroups that apply and enforce the limits you defined.

The CPU limit defines how much CPU time the container can use. During each scheduling interval, the Linux kernel checks to see if this limit is exceeded. If so, the kernel waits before allowing the cgroup to resume execution.

If several different containers (cgroups) want to run on a contended system, workloads with larger CPU requests are allocated more CPU time than workloads with small requests. The memory request is used during Pod scheduling. On a node that uses cgroups v2, the container runtime might use the memory request as a hint to set **memory.min** and **memory.low** values.

If a container attempts to allocate more memory than this limit, the Linux kernel out-of-memory subsystem activates and intervenes by stopping one of the processes in the container that tried to allocate memory. The memory limit for the Pod or container can also apply to pages in memory-backed volumes, such as an **emptyDir**.

The kubelet tracks **tmpfs emptyDir** volumes as container memory is used, rather than as local ephemeral storage. If a container exceeds its memory request and the node that it runs on becomes short of memory overall, the Pod's container might be evicted.



IMPORTANT

A container might not exceed its CPU limit for extended periods. Container run times do not stop Pods or containers for excessive CPU usage. To determine whether a container cannot be scheduled or is being killed due to resource limits, see *Troubleshooting the Compliance Operator*.

5.6.2.7. Scheduling Pods with container resource requests

You can specify CPU and memory resource requests and limits for containers to ensure that pods are placed on nodes with sufficient capacity, preventing resource shortages.

Although memory or CPU resource usage on nodes is very low, the scheduler might still refuse to place a Pod on a node if the capacity check fails to protect against a resource shortage on a node.

For each container, you can specify the following resource limits and request:

```
spec.containers[].resources.limits.cpu
spec.containers[].resources.limits.memory
spec.containers[].resources.limits.hugepages-<size>
spec.containers[].resources.requests.cpu
spec.containers[].resources.requests.memory
spec.containers[].resources.requests.hugepages-<size>
```

Although you can specify requests and limits for only individual containers, it is also useful to consider the overall resource requests and limits for a pod. For a particular resource, a container resource request or limit is the sum of the resource requests or limits of that type for each container in the pod.

Example container resource requests and limits

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
  - name: log-aggregator
    image: images.my-company.example/log-aggregator:v6
    resources:
```



```

requests:
  memory: "64Mi"
  cpu: "250m"
limits:
  memory: "128Mi"
  cpu: "500m"
securityContext:
  allowPrivilegeEscalation: false
capabilities:
  drop: [ALL]

```

where:

spec.containers.resources.requests

Specifies that the container is requesting 64 Mi of memory and 250 m CPU.

spec.containers.resources.limits

Specifies the container's limits are 128 Mi of memory and 500 m CPU.

5.6.2.8. Additional resources

- [Increasing Compliance Operator resource limits](#)
- [Compliance Operator shows INCONSISTENT scan result with worker node](#)
- [Managing Compliance Operator result and remediation](#)

5.6.3. Propagating custom metadata to ComplianceCheckResult objects

Starting in Compliance Operator 1.9.0, add labels and annotations to **Rule** and **CustomRule** objects so matching metadata is displayed on **ComplianceCheckResult** objects after a scan. Downstream tools, dashboards, and ticketing workflows can use this metadata without maintaining a separate mapping.

5.6.3.1. Understanding custom metadata on rules and check results

When the Compliance Operator creates or updates a **ComplianceCheckResult** object for a check that Open Security Content Automation Protocol (OpenSCAP) evaluates, it copies custom labels and annotations from the corresponding **Rule** object in the same namespace.

For checks that the Common Expression Language (CEL) scanner evaluates from a **CustomRule**, the Operator copies custom metadata from that **CustomRule** instead.

Custom metadata is any label or annotation whose name is not managed by the Compliance Operator or core Kubernetes namespaces. Some keys are reserved, so user-supplied values never replace the Operator values for those keys. Examples include the following:

- Keys starting with **compliance.openshift.io/**
- Keys starting with **complianceoperator.openshift.io/**
- Keys starting with **complianceascode.io/**
- Keys where the domain contains **kubernetes.io/** (such as **kubernetes.io/name** or **app.kubernetes.io/**)

- Keys where the domain contains **k8s.io/** (such as **k8s.io/component** or **node.k8s.io/instance-type**)



NOTE

Metadata that you set on **TailoredProfile**, **ScanSettingBinding**, or other orchestration resources is not propagated to individual **ComplianceCheckResult** objects. Attach metadata on the **Rule** or **CustomRule** that owns the check.



NOTE

Custom metadata on **ComplianceRemediation** objects is not covered by this feature.

Profile bundle updates

When a **ProfileBundle** refreshes profile content, the Operator merges user-defined annotations on **Rule** objects with parser-managed annotations so your keys remain while Operator and content fields stay current.

Failure behavior

If the Operator cannot build its in-memory index of **Rule** objects during aggregation, it logs a warning and still creates scan results without your custom metadata. If a label value violates Kubernetes length or syntax rules, the Operator can fail to create that specific **ComplianceCheckResult** while other results continue.

For more background on the enhancement, see [Additional resources](#).

5.6.3.2. Adding custom labels and annotations to a Rule object

You can attach custom labels and annotations to a **Rule** that Open Security Content Automation Protocol (OpenSCAP) evaluates, and verify that they are displayed on the aggregated **ComplianceCheckResult** after the next scan.

Prerequisites

- You have installed Compliance Operator 1.9.0 or later.
- You have access to the **openshift-compliance** namespace (or the namespace where your rules and scans run).
- You identified the **Rule** object name and the **ComplianceScan** that evaluates it.

Procedure

1. Add custom labels to the rule. Replace **<rule_name>** with your **Rule** object name, and adjust labels as needed with the following command:

```
$ oc label rule.compliance/<rule_name> \
  business-unit=payments \
  risk-tier=critical \
  -n openshift-compliance
```

2. Add custom annotations with the following command:

```
$ oc annotate rule.compliance/<rule_name> \
  internal-id=SEC-4021 \
  exception-ticket=JIRA-123 \
  -n openshift-compliance
```

Long or free-form values are better suited to annotations than labels because of Kubernetes label length limits.

3. Trigger a new scan or wait for the next scheduled run. For example, to request a rescan of an existing **ComplianceScan** named **ocp4-cis** use the following command:

```
$ oc annotate compliancescan ocp4-cis \
  compliance.openshift.io/rescan= \
  -n openshift-compliance
```

4. After the scan finishes, list **ComplianceCheckResult** objects that carry your label using the following command:

```
$ oc get compliancecheckresults \
  -l business-unit=payments \
  -n openshift-compliance
```

5. Confirm an annotation on a specific result with the following command:

```
$ oc get compliancecheckresult <result_name> \
  -o jsonpath='{.metadata.annotations.internal-id}' \
  -n openshift-compliance
```

Replace **<result_name>** with the **ComplianceCheckResult** name for your rule.

6. Optional: Verify that a user annotation survives a **ProfileBundle** content image update using the following command.

```
$ oc get rule.compliance/<rule_name> \
  -o jsonpath='{.metadata.annotations.exception-ticket}' \
  -n openshift-compliance
```

After the profile parser reconciles the **Rule**, your key should still be present.

5.6.3.3. Adding custom labels and annotations to a CustomRule object

For platform checks implemented as a **CustomRule** object and evaluated by the Common Expression Language (CEL) scanner, define labels and annotations on the **CustomRule** metadata. The Compliance Operator copies non-reserved entries onto each generated **ComplianceCheckResult**.

Prerequisites

- You have installed Compliance Operator 1.9.0 or later.
- You use a **TailoredProfile** (or equivalent workflow) that enables your **CustomRule** for a CEL profile scan.

Procedure

1. Create a **CustomRule** object that includes your metadata in **metadata.labels** and **metadata.annotations**. The following example defines a platform CEL check; replace names, expressions, and metadata with values appropriate for your environment:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: CustomRule
metadata:
  name: check-pod-security-standard
  namespace: openshift-compliance
  labels:
    break_severity: critical
    weakness_score: "9.5"
  annotations:
    internal-id: SEC-5500
    audit-contact: platform-security-team
spec:
  id: check-pod-security-standard
  title: "Ensure Pod Security Standards are enforced"
  severity: high
  checkType: Platform
  scannerType: CEL
  expression: |
    namespaces.items.all(ns,
      has(ns.metadata.labels) &&
      "pod-security.kubernetes.io/enforce" in ns.metadata.labels
    )
  failureReason: "One or more namespaces do not enforce Pod Security Standards"
  inputs:
    - name: namespaces
      kubernetesInputSpec:
        apiVersion: v1
        resource: namespaces

```

2. Apply the manifest by running the following command:

```
$ oc apply -f custom-rule.yaml
```

3. Reference the **CustomRule** object from a **TailoredProfile**, for example:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: TailoredProfile
metadata:
  name: custom-cel-profile
  namespace: openshift-compliance
spec:
  title: "Custom CEL profile"
  description: "Profile with custom CEL rules"
  enableRules:
    - name: check-pod-security-standard
      rationale: "Enforce pod security standards"
      kind: CustomRule

```

4. Bind the tailored profile with a **ScanSettingBinding** and run the scan using your normal workflow.

5. After the scan completes, query results by a custom label by running the following command:

```
$ oc get compliancecheckresults \
  -l break_severity=critical \
  -n openshift-compliance
```

6. Identify the **ComplianceCheckResult** name for your **CustomRule** object in the scan output, for example by listing results in the namespace and filtering by labels or name.
7. Read a propagated annotation from that object by running the following command:

```
$ oc get compliancecheckresult <result_name> \
  -o jsonpath='{.metadata.annotations.internal-id}' \
  -n openshift-compliance
```

5.6.4. Tailoring the Compliance Operator

Although the Compliance Operator includes ready-to-use profiles, you must modify the profiles to fit your organization's requirements. The process of modifying a profile is called *tailoring*.

The Compliance Operator provides the **TailoredProfile** object to help tailor profiles.

5.6.4.1. Creating a new tailored profile

You can write a tailored profile from scratch by using the **TailoredProfile** object. Set an appropriate **title** and **description** and leave the **extends** field empty.

Indicate to the Compliance Operator what type of scan this custom profile will generate:

- Node scan: Scans the operating system.
- Platform scan: Scans the OpenShift Container Platform configuration.

Procedure

- Set the following annotation on the **TailoredProfile** object:

Example new-profile.yaml

```
apiVersion: compliance.openshift.io/v1alpha1
kind: TailoredProfile
metadata:
  name: new-profile
  annotations:
    compliance.openshift.io/product-type: Node
spec:
  extends: ocp4-cis-node
  description: My custom profile
  title: Custom profile
  enableRules:
    - name: ocp4-etcd-unique-ca
      rationale: We really need to enable this
```

```
disableRules:
- name: ocp4-file-groupowner-cni-conf
  rationale: This does not apply to the cluster
```

where:

metadata.annotations.compliance.openshift.io/product-type

Sets **Node** or **Platform** accordingly.

spec.extends

Optional field to specify the base profile.

spec.description

Specifies the function of the new **TailoredProfile** object.

spec.title

Specifies a title for the **TailoredProfile** object.



NOTE

Adding the **-node** suffix to the **name** field of the **TailoredProfile** object is similar to adding the **Node** product type annotation and generates an operating system scan.

5.6.4.2. Using tailored profiles to extend existing ProfileBundles

Although the **TailoredProfile** CR enables the most common tailoring operations, you can use the XCCDF (Extensible Configuration Checklist Description Format) standard for even more flexibility in tailoring OpenSCAP profiles.

In addition, if your organization has been using OpenScap previously, you might have an existing XCCDF tailoring file and can reuse it.

The **ComplianceSuite** object has an optional **TailoringConfigMap** attribute that you can point to a custom tailoring file. The value of the **TailoringConfigMap** attribute is a name of a config map, which must contain a key called **tailoring.xml** and the value of this key is the tailoring contents.

Procedure

1. Browse the available rules for the Red Hat Enterprise Linux CoreOS (RHCOS) **ProfileBundle**:

```
$ oc get rules.compliance -n openshift-compliance -l compliance.openshift.io/profile-bundle=rhcos4
```

2. Browse the available variables in the same **ProfileBundle**:

```
$ oc get variables.compliance -n openshift-compliance -l compliance.openshift.io/profile-bundle=rhcos4
```

3. Create a tailored profile named **nist-moderate-modified**:

- a. Choose which rules you want to add to the **nist-moderate-modified** tailored profile. This example extends the **rhcos4-moderate** profile by disabling two rules and changing one value. Use the **rationale** value to describe why these changes were made:

Example new-profile-node.yaml

```
apiVersion: compliance.openshift.io/v1alpha1
kind: TailoredProfile
metadata:
  name: nist-moderate-modified
spec:
  extends: rhcos4-moderate
  description: NIST moderate profile
  title: My modified NIST moderate profile
  disableRules:
    - name: rhcos4-file-permissions-var-log-messages
      rationale: The file contains logs of error messages in the system
    - name: rhcos4-account-disable-post-pw-expiration
      rationale: No need to check this as it comes from the IdP
  setValues:
    - name: rhcos4-var-selinux-state
      rationale: Organizational requirements
      value: permissive
```

Table 5.10. Attributes for spec variables

Attribute	Description
extends	Name of the Profile object upon which this TailoredProfile is built.
title	Human-readable title of the TailoredProfile .
disableRules	A list of name and rationale pairs. Each name refers to a name of a rule object that is to be disabled. The rationale value is human-readable text describing why the rule is disabled.
manualRules	A list of name and rationale pairs. When a manual rule is added, the check result status will always be manual and remediation will not be generated. This attribute is automatic and by default has no values when set as a manual rule.
enableRules	A list of name and rationale pairs. Each name refers to a name of a rule object that is to be enabled. The rationale value is human-readable text describing why the rule is enabled.
description	Human-readable text describing the TailoredProfile .
setValues	A list of name, rationale, and value groupings. Each name refers to a name of the value set. The rationale is human-readable text describing the set. The value is the actual setting.

- b. Add the **tailoredProfile.spec.manualRules** attribute:

Example tailoredProfile.spec.manualRules.yaml

```
apiVersion: compliance.openshift.io/v1alpha1
kind: TailoredProfile
metadata:
  name: ocp4-manual-scc-check
spec:
  extends: ocp4-cis
  description: This profile extends ocp4-cis by forcing the SCC check to always return
  MANUAL
  title: OCP4 CIS profile with manual SCC check
  manualRules:
    - name: ocp4-scc-limit-container-allowed-capabilities
      rationale: We use third party software that installs its own SCC with extra privileges
```

- c. Create the **TailoredProfile** object:

```
$ oc create -n openshift-compliance -f new-profile-node.yaml
```

- The **TailoredProfile** object is created in the default **openshift-compliance** namespace.

Example output

```
tailoredprofile.compliance.openshift.io/nist-moderate-modified created
```

4. Define the **ScanSettingBinding** object to bind the new **nist-moderate-modified** tailored profile to the default **ScanSetting** object.

Example new-scansettingbinding.yaml

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSettingBinding
metadata:
  name: nist-moderate-modified
profiles:
  - apiGroup: compliance.openshift.io/v1alpha1
    kind: Profile
    name: ocp4-moderate
  - apiGroup: compliance.openshift.io/v1alpha1
    kind: TailoredProfile
    name: nist-moderate-modified
settingsRef:
  apiGroup: compliance.openshift.io/v1alpha1
  kind: ScanSetting
  name: default
```

5. Create the **ScanSettingBinding** object:

```
$ oc create -n openshift-compliance -f new-scansettingbinding.yaml
```

Example output


```
scansettingbinding.compliance.openshift.io/nist-moderate-modified created
```

5.6.5. Retrieving Compliance Operator raw results

When proving compliance for your OpenShift Container Platform cluster, you might need to provide the scan results for auditing purposes.

5.6.5.1. Obtaining Compliance Operator raw results from a persistent volume

You can view the results of Compliance Operator scans for auditing purposes. The Operator stores the raw results in a persistent volume in Asset Reporting Format (ARF).

Procedure

1. Explore the **ComplianceSuite** object:

```
$ oc get compliancesuites nist-moderate-modified \
-o json -n openshift-compliance | jq '.status.scanStatuses[].resultsStorage'
```

Example output

```
{
  "name": "ocp4-moderate",
  "namespace": "openshift-compliance"
}
{
  "name": "nist-moderate-modified-master",
  "namespace": "openshift-compliance"
}
{
  "name": "nist-moderate-modified-worker",
  "namespace": "openshift-compliance"
}
```

This shows the persistent volume claims where the raw results are accessible.

2. Verify the raw data location by using the name and namespace of one of the results:

```
$ oc get pvc -n openshift-compliance rhcos4-moderate-worker
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
rhcos4-moderate-worker	Bound	pvc-548f6cfe-164b-42fe-ba13-a07cfbc77f3a	1Gi	RWO
gp2	92m			

3. Fetch the raw results by spawning a pod that mounts the volume and copying the results:

```
$ oc create -n openshift-compliance -f pod.yaml
```

Example pod.yaml

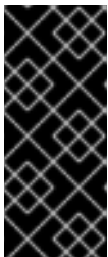
```

apiVersion: "v1"
kind: Pod
metadata:
  name: pv-extract
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: pv-extract-pod
      image: registry.access.redhat.com/ubi9/ubi
      command: ["sleep", "3000"]
      volumeMounts:
        - mountPath: "/workers-scan-results"
          name: workers-scan-vol
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop: [ALL]
  volumes:
    - name: workers-scan-vol
      persistentVolumeClaim:
        claimName: rhcos4-moderate-worker

```

4. After the pod is running, download the results:

```
$ oc cp pv-extract:/workers-scan-results -n openshift-compliance .
```



IMPORTANT

Spawning a pod that mounts the persistent volume will keep the claim as **Bound**. If the volume's storage class in use has permissions set to **ReadWriteOnce**, the volume is only mountable by one pod at a time. You must delete the pod upon completion, or it will not be possible for the Operator to schedule a pod and continue storing results in this location.

5. After the extraction is complete, the pod can be deleted:

```
$ oc delete pod pv-extract -n openshift-compliance
```

5.6.6. Managing Compliance Operator result and remediation

You can review compliance scan results and apply remediations to resolve failing rules. Remediations are not applied automatically, so you can verify each change before applying it to your cluster.

IMPORTANT

Full remediation for Federal Information Processing Standards (FIPS) compliance requires enabling FIPS mode for the cluster. To enable FIPS mode, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Installing the system in FIPS mode](#).

FIPS mode is supported on the following architectures:

- **x86_64**
- **ppc64le**
- **s390x**

5.6.6.1. Filters for compliance check results

You can use the labels in the **ComplianceCheckResult** objects to query the checks and decide on the next steps after the results are generated.

List checks that belong to a specific suite:

```
$ oc get -n openshift-compliance compliancecheckresults \
-l compliance.openshift.io/suite=workers-compliancesuite
```

List checks that belong to a specific scan:

```
$ oc get -n openshift-compliance compliancecheckresults \
-l compliance.openshift.io/scan-name=workers-scan
```

Not all **ComplianceCheckResult** objects create **ComplianceRemediation** objects. Only **ComplianceCheckResult** objects that can be remediated automatically do. A **ComplianceCheckResult** object has a related remediation if it is labeled with the **compliance.openshift.io/automated-remediation** label. The name of the remediation is the same as the name of the check.

List all failing checks that can be remediated automatically:

```
$ oc get -n openshift-compliance compliancecheckresults \
-l 'compliance.openshift.io/check-status=FAIL,compliance.openshift.io/automated-remediation'
```

List all failing checks sorted by severity:

```
$ oc get compliancecheckresults -n openshift-compliance \
-l 'compliance.openshift.io/check-status=FAIL,compliance.openshift.io/check-severity=high'
```

Example output

NAME	STATUS	SEVERITY
nist-moderate-modified-master-configure-crypto-policy	FAIL	high
nist-moderate-modified-master-coreos-pti-kernel-argument	FAIL	high
nist-moderate-modified-master-disable-ctrlaltdel-burstaction	FAIL	high
nist-moderate-modified-master-disable-ctrlaltdel-reboot	FAIL	high

nist-moderate-modified-master-enable-fips-mode	FAIL	high
nist-moderate-modified-master-no-empty-passwords	FAIL	high
nist-moderate-modified-master-selinux-state	FAIL	high
nist-moderate-modified-worker-configure-crypto-policy	FAIL	high
nist-moderate-modified-worker-coreos-pti-kernel-argument	FAIL	high
nist-moderate-modified-worker-disable-ctrlaltdel-burstaction	FAIL	high
nist-moderate-modified-worker-disable-ctrlaltdel-reboot	FAIL	high
nist-moderate-modified-worker-enable-fips-mode	FAIL	high
nist-moderate-modified-worker-no-empty-passwords	FAIL	high
nist-moderate-modified-worker-selinux-state	FAIL	high
ocp4-moderate-configure-network-policies-namespaces	FAIL	high
ocp4-moderate-fips-mode-enabled-on-all-nodes	FAIL	high

List all failing checks that must be remediated manually:

```
$ oc get -n openshift-compliance compliancecheckresults \
-l 'compliance.openshift.io/check-status=FAIL,!compliance.openshift.io/automated-remediation'
```

The manual remediation steps are typically stored in the **description** attribute in the **ComplianceCheckResult** object.

Table 5.11. ComplianceCheckResult Status

ComplianceCheckResult Status	Description
PASS	Compliance check ran to completion and passed.
FAIL	Compliance check ran to completion and failed.
INFO	Compliance check ran to completion and found something not severe enough to be considered an error.
MANUAL	Compliance check does not have a way to automatically assess the success or failure and must be checked manually.
INCONSISTENT	Compliance check reports different results from different sources, typically cluster nodes.
ERROR	Compliance check ran, but could not complete properly.
NOT-APPLICABLE	Compliance check did not run because it is not applicable or not selected.

5.6.6.2. Reviewing a remediation

You can review a **ComplianceRemediation** object and the **ComplianceCheckResult** object to understand what a check verifies, its severity and security controls, and how the remediation fixes the issue. After the first scan, check for remediations with the state **MissingDependencies**.

The **ComplianceCheckResult** object includes human-readable descriptions of what the check does and what security hardening it enforces.

The remediation payload is stored in the **spec.current** attribute. The payload can be any Kubernetes object, but because this remediation was produced by a node scan, the remediation payload in the following example is a **MachineConfig** object. For Platform scans, the remediation payload is often a different kind of an object (for example, a **ConfigMap** or **Secret** object). Typically, applying that remediation is up to the administrator. Otherwise, the Compliance Operator would have required a very broad set of permissions to manipulate any generic Kubernetes object. An example of remediating a Platform check is provided later in the text.

To see exactly what the remediation does when applied, the **MachineConfig** object contents use the Ignition objects for the configuration. See the link to "Ignition specification" in Additional resources for further information about the format. In the following example, the **spec.config.storage.files[0].path** attribute specifies the file that is being created by this remediation (**/etc/sysctl.d/75-sysctl_net_ipv4_conf_all_accept_redirects.conf**) and the **spec.config.storage.files[0].contents.source** attribute specifies the contents of that file.

Procedure

1. Review the example of a check and a remediation called **sysctl-net-ipv4-conf-all-accept-redirects**. This example is redacted to only show **spec** and **status** and omits **metadata**:

```
spec:
  apply: false
  current:
    object:
      apiVersion: machineconfiguration.openshift.io/v1
      kind: MachineConfig
      spec:
        config:
          ignition:
            version: 3.2.0
          storage:
            files:
              - path: /etc/sysctl.d/75-sysctl_net_ipv4_conf_all_accept_redirects.conf
                mode: 0644
                contents:
                  source: data:;net.ipv4.conf.all.accept_redirects%3D0
      outdated: {}
    status:
      applicationState: NotApplied
```

2. Use the following Python script to view the contents:



NOTE

The contents of the files are URL-encoded.

```
$ echo "net.ipv4.conf.all.accept_redirects%3D0" | python3 -c "import sys, urllib.parse;
print(urllib.parse.unquote(''.join(sys.stdin.readlines())))"
```

Example output

```
net.ipv4.conf.all.accept_redirects=0
```



IMPORTANT

The Compliance Operator does not automatically resolve dependency issues that can occur between remediations. Users should perform a rescan after remediations are applied to ensure accurate results.

5.6.6.3. Applying remediation when using customized machine config pools

When you create a custom **MachineConfigPool**, add a label to the **MachineConfigPool** so that **machineConfigPoolSelector** present in the **KubeletConfig** can match the label with **MachineConfigPool**.



IMPORTANT

Do not set **protectKernelDefaults: false** in the **KubeletConfig** file, because the **MachineConfigPool** object might fail to unpause unexpectedly after the Compliance Operator finishes applying remediation.

Procedure

1. List the nodes.

```
$ oc get nodes -n openshift-compliance
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-128-92.us-east-2.compute.internal	Ready	master	5h21m	v1.34.2
ip-10-0-158-32.us-east-2.compute.internal	Ready	worker	5h17m	v1.34.2
ip-10-0-166-81.us-east-2.compute.internal	Ready	worker	5h17m	v1.34.2
ip-10-0-171-170.us-east-2.compute.internal	Ready	master	5h21m	v1.34.2
ip-10-0-197-35.us-east-2.compute.internal	Ready	master	5h22m	v1.34.2

2. Add a label to nodes.

```
$ oc -n openshift-compliance \
label node ip-10-0-166-81.us-east-2.compute.internal \
node-role.kubernetes.io/<machine_config_pool_name>=
```

Example output

```
node/ip-10-0-166-81.us-east-2.compute.internal labeled
```

3. Create custom **MachineConfigPool** CR.

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: <machine_config_pool_name>
  labels:
```

```

    pools.operator.machineconfiguration.openshift.io/<machine_config_pool_name>: "
spec:
  machineConfigSelector:
    matchExpressions:
      - {key: machineconfiguration.openshift.io/role, operator: In, values: [worker,
<machine_config_pool_name>]}
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/<machine_config_pool_name>: ""

```

where:

metadata.labels.pools.operator.machineconfiguration.openshift.io/<machine_config_pool_name>

The **labels** field defines the label name to add for the machine config pool (MCP).

4. Verify MCP created successfully.

```
$ oc get mcp -w
```

5.6.6.4. Evaluating KubeletConfig rules against default configuration values

The Compliance Operator uses the Node/Proxy API to evaluate **KubeletConfig** object rules against actual node configurations, preventing inaccurate results caused by incomplete configuration files and default values for missing options.

OpenShift Container Platform infrastructure might contain incomplete configuration files at run time, and nodes assume default configuration values for missing configuration options. Some configuration options can be passed as command-line arguments. As a result, the Compliance Operator cannot verify if the configuration file on the node is complete because it might be missing options used in the rule checks.

To prevent false negative results where the default configuration value passes a check, the Compliance Operator uses the Node/Proxy API to fetch the configuration for each node in a node pool, then all configuration options that are consistent across nodes in the node pool are stored in a file that represents the configuration for all nodes within that node pool. This increases the accuracy of the scan results.

No additional configuration changes are required to use this feature with default **master** and **worker** node pools configurations.

5.6.6.5. Scanning custom node pools

The Compliance Operator does not maintain a copy of each node pool configuration.

The Compliance Operator aggregates consistent configuration options for all nodes within a single node pool into one copy of the configuration file. The Compliance Operator then uses the configuration file for a particular node pool to evaluate rules against nodes within that pool.

Procedure

1. Add the **example** role to the **ScanSetting** object that will be stored in the **ScanSettingBinding** CR:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  name: default
  namespace: openshift-compliance
rawResultStorage:
  rotation: 3
  size: 1Gi
roles:
- worker
- master
- example
scanTolerations:
- effect: NoSchedule
  key: node-role.kubernetes.io/master
  operator: Exists
schedule: '0 1 * * *'

```

2. Create a scan that uses the **ScanSettingBinding** CR:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSettingBinding
metadata:
  name: cis
  namespace: openshift-compliance
profiles:
- apiGroup: compliance.openshift.io/v1alpha1
  kind: Profile
  name: ocp4-cis
- apiGroup: compliance.openshift.io/v1alpha1
  kind: Profile
  name: ocp4-cis-node
settingsRef:
  apiGroup: compliance.openshift.io/v1alpha1
  kind: ScanSetting
  name: default

```

Verification

- The Platform KubeletConfig rules are checked through the **Node/Proxy** object. You can find those rules by running the following command:

```
$ oc get rules -o json | jq '.items[] | select(.checkType == "Platform") | select(.metadata.name | contains("ocp4-kubelet-")) | .metadata.name'
```

5.6.6.6. Remediating KubeletConfig sub pools

You can apply **KubeletConfig** remediation labels to **MachineConfigPool** sub-pools.

Procedure

- Add a label to the sub-pool **MachineConfigPool** CR:


```
$ oc label mcp <sub-pool-name> pools.operator.machineconfiguration.openshift.io/<sub-pool-name>=
```

5.6.6.7. Applying a remediation

The boolean attribute **spec.apply** controls whether the remediation should be applied by the Compliance Operator. You can apply the remediation by setting the attribute to **true**.

Procedure

1. Apply the remediation by setting the attribute to **true**:

```
$ oc -n openshift-compliance \
  patch complianceremediations/<scan-name>-sysctl-net-ipv4-conf-all-accept-redirects \
  --patch '{"spec":{"apply":true}}' --type=merge
```

After the Compliance Operator processes the applied remediation, the **status.ApplicationState** attribute would change to **Applied** or to **Error** if incorrect. When a machine config remediation is applied, that remediation along with all other applied remediations are rendered into a **MachineConfig** object named **75-\$scan-name-\$suite-name**. That **MachineConfig** object is subsequently rendered by the Machine Config Operator and finally applied to all the nodes in a machine config pool by an instance of the machine control daemon running on each node.

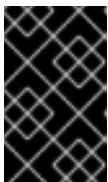
Note that when the Machine Config Operator applies a new **MachineConfig** object to nodes in a pool, all the nodes belonging to the pool are rebooted. This might be inconvenient when applying multiple remediations, each of which re-renders the composite **75-\$scan-name-\$suite-name MachineConfig** object. To prevent applying the remediation immediately, you can pause the machine config pool by setting the **.spec.paused** attribute of a **MachineConfigPool** object to **true**.

2. Optionally, the Compliance Operator can apply remediations automatically. Set **autoApplyRemediations: true** in the **ScanSetting** top-level object.



WARNING

Applying remediations automatically should only be done with careful consideration.



IMPORTANT

The Compliance Operator does not automatically resolve dependency issues that can occur between remediations. Users should perform a rescan after remediations are applied to ensure accurate results.

5.6.6.8. Remediating a platform check manually

You must manually remediate checks from Platform scans so you can fix findings that the Compliance Operator cannot apply automatically.

Manual remediations are necessary for the following reasons:

- It is not always possible to automatically determine the value that must be set. One of the checks requires that a list of allowed registries is provided, but the scanner has no way of knowing which registries the organization wants to allow.
- Different checks modify different API objects, requiring automated remediation to possess **root** or superuser access to modify objects in the cluster, which is not advised.

Procedure

1. The example below uses the **ocp4-ocp-allowed-registries-for-import** rule, which would fail on a default OpenShift Container Platform installation. Inspect the rule **oc get rule.compliance/ocp4-ocp-allowed-registries-for-import -oyaml**, the rule is to limit the registries the users are allowed to import images from by setting the **allowedRegistriesForImport** attribute, The *warning* attribute of the rule also shows the API object checked, so it can be modified and remediate the issue:

```
$ oc edit image.config.openshift.io/cluster
```

Example output

```
apiVersion: config.openshift.io/v1
kind: Image
metadata:
  annotations:
    release.openshift.io/create-only: "true"
    creationTimestamp: "2020-09-10T10:12:54Z"
    generation: 2
    name: cluster
    resourceVersion: "363096"
    selfLink: /apis/config.openshift.io/v1/images/cluster
    uid: 2dcb614e-2f8a-4a23-ba9a-8e33cd0ff77e
spec:
  allowedRegistriesForImport:
    - domainName: registry.redhat.io
status:
  externalRegistryHostnames:
    - default-route-openshift-image-registry.apps.user-cluster-09-10-12-07.devcluster.openshift.com
    internalRegistryHostname: image-registry.openshift-image-registry.svc:5000
```

2. Re-run the scan:

```
$ oc -n openshift-compliance \
  annotate compliancescans/rhcos4-e8-worker compliance.openshift.io/rescan=
```

5.6.6.9. Updating remediations

When you update compliance content to a newer version, the Compliance Operator marks previously applied remediations as **Outdated**. Review these remediations and apply the updated versions to ensure your nodes use the latest configuration.

The previously applied remediation contents would then be stored in the **spec.outdated** attribute of a **ComplianceRemediation** object and the new updated contents would be stored in the **spec.current** attribute. After updating the content to a newer version, the administrator then needs to review the remediation. If the **spec.outdated** attribute exists, it would be used to render the resulting **MachineConfig** object. After the **spec.outdated** attribute is removed, the Compliance Operator re-renders the resulting **MachineConfig** object, which causes the Operator to push the configuration to the nodes.



IMPORTANT

The Compliance Operator does not automatically resolve dependency issues that can occur between remediations. Users should perform a rescan after remediations are applied to ensure accurate results.

Procedure

1. Search for any outdated remediations:

```
$ oc -n openshift-compliance get complianceremediations \
-l complianceoperator.openshift.io/outdated-remediation=
```

Example output

NAME	STATE
workers-scan-no-empty-passwords	Outdated



NOTE

The currently applied remediation is stored in the **Outdated** attribute and the new, unapplied remediation is stored in the **Current** attribute. If you are satisfied with the new version, remove the **Outdated** field. If you want to keep the updated content, remove the **Current** and **Outdated** attributes.

2. Apply the newer version of the remediation:

```
$ oc -n openshift-compliance patch complianceremediations workers-scan-no-empty-
passwords \
--type json -p '[{"op": "remove", "path": "/spec/outdated"}]'
```

3. The remediation state will switch from **Outdated** to **Applied**:

```
$ oc get -n openshift-compliance complianceremediations workers-scan-no-empty-
passwords
```

Example output

NAME	STATE
workers-scan-no-empty-passwords	Applied

4. Verify that the nodes apply the newer remediation version and reboot.

5.6.6.10. Unapplying a remediation

You can unapply a remediation that was previously applied to roll back a change when you need to revert it.



IMPORTANT

The Compliance Operator does not automatically resolve dependency issues that can occur between remediations. Users should perform a rescan after remediations are applied to ensure accurate results.

Procedure

1. Set the **apply** flag to **false**:

```
$ oc -n openshift-compliance \
  patch complianceremediations/rhcos4-moderate-worker-sysctl-net-ipv4-conf-all-accept-
  redirects \
  --patch '{"spec":{"apply":false}}' --type=merge
```

2. Verify that the remediation status has changed to **NotApplied** and the composite **MachineConfig** object is re-rendered to not include the remediation.



IMPORTANT

All affected nodes with the remediation will be rebooted.

5.6.6.11. Removing a KubeletConfig remediation

KubeletConfig remediations are included in node-level profiles. To remove a **KubeletConfig** remediation, you must manually remove it from the **KubeletConfig** objects.

Procedure

1. Locate the **scan-name** and compliance check for the **one-rule-tp-node-master-kubelet-eviction-thresholds-set-hard-imagefs-available** remediation:

```
$ oc -n openshift-compliance get remediation \ one-rule-tp-node-master-kubelet-eviction-
  thresholds-set-hard-imagefs-available -o yaml
```

Example output

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ComplianceRemediation
metadata:
  annotations:
    compliance.openshift.io/xccdf-value-used: var-kubelet-evictionhard-imagefs-available
  creationTimestamp: "2022-01-05T19:52:27Z"
  generation: 1
  labels:
    compliance.openshift.io/scan-name: one-rule-tp-node-master
```

```

  compliance.openshift.io/suite: one-rule-ssb-node
  name: one-rule-tp-node-master-kubelet-eviction-thresholds-set-hard-imagefs-available
  namespace: openshift-compliance
  ownerReferences:
  - apiVersion: compliance.openshift.io/v1alpha1
    blockOwnerDeletion: true
    controller: true
    kind: ComplianceCheckResult
    name: one-rule-tp-node-master-kubelet-eviction-thresholds-set-hard-imagefs-available
    uid: fe8e1577-9060-4c59-95b2-3e2c51709adc
  resourceVersion: "84820"
  uid: 5339d21a-24d7-40cb-84d2-7a2ebb015355
spec:
  apply: true
  current:
    object:
      apiVersion: machineconfiguration.openshift.io/v1
      kind: KubeletConfig
      spec:
        kubeletConfig:
          evictionHard:
            imagefs.available: 10%
    outdated: {}
  type: Configuration
status:
  applicationState: Applied

```

where:

- **metadata.labels.compliance.openshift.io/scan-name** specifies the scan name of the remediation.
- **spec.current.object.spec.kubeletConfig.evictionHard.imagefs.available** specifies the remediation that was added to the **KubeletConfig** objects.



NOTE

If the remediation invokes an **evictionHard** kubelet configuration, you must specify all of the **evictionHard** parameters: **memory.available**, **nodefs.available**, **nodefs.inodesFree**, **imagefs.available**, and **imagefs.inodesFree**. If you do not specify all parameters, only the specified parameters are applied and the remediation will not function properly.

2. Remove the remediation:

- Set **apply** to false for the remediation object:

```

$ oc -n openshift-compliance patch \
  complianceremediations/one-rule-tp-node-master-kubelet-eviction-thresholds-set-hard-
  imagefs-available \
  -p '{"spec":{"apply":false}}' --type=merge

```

- Using the **scan-name**, find the **KubeletConfig** object that the remediation was applied to:

```
$ oc -n openshift-compliance get kubeletconfig \
--selector compliance.openshift.io/scan-name=one-rule-tp-node-master
```

Example output

NAME	AGE
compliance-operator-kubelet-master	2m34s

- c. Manually remove the remediation, **imagefs.available: 10%**, from the **KubeletConfig** object:

```
$ oc edit -n openshift-compliance KubeletConfig compliance-operator-kubelet-master
```



IMPORTANT

All affected nodes with the remediation will be rebooted.



NOTE

You must also exclude the rule from any scheduled scans in your tailored profiles that auto-applies the remediation, otherwise, the remediation will be re-applied during the next scheduled scan.

5.6.6.12. Inconsistent ComplianceScan

The **ScanSetting** object lists the node roles that the compliance scans generated from the **ScanSetting** or **ScanSettingBinding** objects would scan. Each node role usually maps to a machine config pool.



IMPORTANT

All machines in a machine config pool are expected to be identical and all scan results from the nodes in a pool should be identical.

If a compliance scan results in an **INCONSISTENT** result, re-run the compliance scan to get a consistent result by annotating the scan with the **compliance.openshift.io/rescan=** option.

The **ScanSetting** object lists the node roles that the compliance scans generated from the **ScanSetting** or **ScanSettingBinding** objects would scan. Each node role usually maps to a machine config pool.

Because the number of machines in a pool might be quite large, the Compliance Operator attempts to find the most common state and list the nodes that differ from the common state. The most common state is stored in the **compliance.openshift.io/most-common-status** annotation and the annotation **compliance.openshift.io/inconsistent-source** contains pairs of **hostname:status** of check statuses that differ from the most common status. If no common state can be found, all the **hostname:status** pairs are listed in the **compliance.openshift.io/inconsistent-source** annotation.

If possible, a remediation is still created so that the cluster can converge to a compliant status. However, this might not always be possible and correcting the difference between nodes must be done manually.

Procedure

- Re-run the compliance scan to get a consistent result by annotating the scan with the **compliance.openshift.io/rescan=** option:

```
$ oc -n openshift-compliance \
  annotate compliancescans/rhcos4-e8-worker compliance.openshift.io/rescan=
```

5.6.6.13. Additional resources

- [Modifying nodes](#)
- [Ignition specification](#)
- [Installing the system in FIPS mode](#)

5.6.7. Performing advanced Compliance Operator tasks

As an advanced user, you can use options in the Compliance Operator for the purpose of debugging or integrating with existing tooling.

5.6.7.1. Using the ComplianceSuite and ComplianceScan objects directly

You can define a **ComplianceSuite** object directly rather than using the **ScanSetting** and **ScanSettingBinding** objects to define the suites and scans.

The following use cases are valid reasons to define a **ComplianceSuite** object:

- Specifying only a single rule to scan. This can be useful for debugging together with the **debug: true** attribute which increases the OpenSCAP scanner verbosity, as the debug mode tends to get quite verbose otherwise. Limiting the test to one rule helps to lower the amount of debug information.
- Providing a custom nodeSelector. In order for a remediation to be applicable, the nodeSelector must match a pool.
- Pointing the Scan to a bespoke config map with a tailoring file.
- For testing or development when the overhead of parsing profiles from bundles is not required.

The following example shows a **ComplianceSuite** that scans the worker machines with only a single rule:

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ComplianceSuite
metadata:
  name: workers-compliancesuite
spec:
  scans:
    - name: workers-scan
      profile: xccdf_org.ssgproject.content_profile_moderate
      content: ssg-rhcos4-ds.xml
      contentImage: registry.redhat.io/compliance/openshift-compliance-content-rhel8@sha256:45dc...
      debug: true
      rule: xccdf_org.ssgproject.content_rule_no_direct_root_logins
      nodeSelector:
        node-role.kubernetes.io/worker: ""
```

The **ComplianceSuite** object and the **ComplianceScan** objects referred to above specify several attributes in a format that OpenSCAP expects.

To discover the profile, content, or rule values, you can start by creating a similar Suite from **ScanSetting** and **ScanSettingBinding** or inspect the objects parsed from the **ProfileBundle** objects such as rules or profiles. Those objects contain the **xccdf_org** identifiers you can use to refer to them from a **ComplianceSuite**.

5.6.7.2. Setting **PriorityClass** for **ScanSetting** scans

In some clusters, the default **PriorityClass** object can be too low to guarantee pods execute scans on time. To maintain compliance or guarantee automated scanning, you can set the **PriorityClass** variable to ensure the Compliance Operator is always given priority in resource constrained situations.

Prerequisites

- Optional: You have created a **PriorityClass** object. For more information, see "Configuring priority and preemption" in the *Additional resources*.

Procedure

- Set the **PriorityClass** variable:

```
apiVersion: compliance.openshift.io/v1alpha1
strictNodeScan: true
metadata:
  name: default
  namespace: openshift-compliance
priorityClass: compliance-high-priority
kind: ScanSetting
showNotApplicable: false
rawResultStorage:
  nodeSelector:
    node-role.kubernetes.io/master: ""
pvAccessModes:
  - ReadWriteOnce
rotation: 3
size: 1Gi
tolerations:
  - effect: NoSchedule
    key: node-role.kubernetes.io/master
    operator: Exists
  - effect: NoExecute
    key: node.kubernetes.io/not-ready
    operator: Exists
    tolerationSeconds: 300
  - effect: NoExecute
    key: node.kubernetes.io/unreachable
    operator: Exists
    tolerationSeconds: 300
  - effect: NoSchedule
    key: node.kubernetes.io/memory-pressure
    operator: Exists
schedule: 0 1 * * *
roles:
```



```
- master
- worker
scanTolerations:
- operator: Exists
```

where:

PriorityClass

If the **PriorityClass** referenced in the **ScanSetting** cannot be found, the Operator will leave the **PriorityClass** empty, issue a warning, and continue scheduling scans without a **PriorityClass**.

5.6.7.3. Using raw tailored profiles

Although the **TailoredProfile** CR enables the most common tailoring operations, you can use the XCCDF standard for more flexibility in tailoring OpenSCAP profiles.

In addition, if your organization has been using OpenScap previously, you might have an existing XCCDF tailoring file and can reuse it.

The **ComplianceSuite** object contains an optional **TailoringConfigMap** attribute that you can point to a custom tailoring file. The value of the **TailoringConfigMap** attribute is a name of a config map which must contain a key called **tailoring.xml** and the value of this key is the tailoring contents.

Procedure

1. Create the **ConfigMap** object from a file:

```
$ oc -n openshift-compliance \
create configmap nist-moderate-modified \
--from-file=tailoring.xml=/path/to/the/tailoringFile.xml
```

2. Reference the tailoring file in a scan that belongs to a suite:

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ComplianceSuite
metadata:
  name: workers-compliancesuite
spec:
  debug: true
  scans:
    - name: workers-scan
      profile: xccdf_org.ssgproject.content_profile_moderate
      content: ssg-rhcos4-ds.xml
      contentImage: registry.redhat.io/compliance/openshift-compliance-content-
rhel8@sha256:45dc...
      debug: true
      tailoringConfigMap:
        name: nist-moderate-modified
      nodeSelector:
        node-role.kubernetes.io/worker: ""
```

5.6.7.4. Performing a rescan

You can re-run a scan on a defined schedule, such as every Monday or daily. It can also be useful to re-run a scan once after fixing a problem on a node.

To perform a single scan, annotate the scan with the **compliance.openshift.io/rescan=** option:

Procedure

1. Annotate the scan to trigger a rescan:

```
$ oc -n openshift-compliance \
  annotate compliancescans/rhcos4-e8-worker compliance.openshift.io/rescan=
```

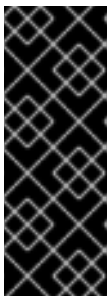
A rescan generates four additional **mc** for **rhcos-moderate** profile:

2. Verify the rescan generated machine configs:

```
$ oc get mc
```

Example output

```
75-worker-scan-chronyd-or-ntpd-specify-remote-server
75-worker-scan-configure-usbguard-auditbackend
75-worker-scan-service-usbguard-enabled
75-worker-scan-usbguard-allow-hid-and-hub
```



IMPORTANT

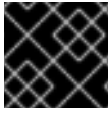
When the scan setting **default-auto-apply** label is applied, remediations are applied automatically and outdated remediations automatically update. If there are remediations that were not applied due to dependencies, or remediations that had been outdated, rescanning applies the remediations and might trigger a reboot. Only remediations that use **MachineConfig** objects trigger reboots. If there are no updates or dependencies to be applied, no reboot occurs.

5.6.7.5. Setting custom storage size for results

Although **ComplianceCheckResult** custom resources summarize one check across all scanned nodes, raw scanner results in ARF format are too large to store in etcd-backed Kubernetes resources. You can store them on a per-scan persistent volume and increase the default 1 GiB size by setting the **rawResultStorage.size** value in a **ScanSetting** or **ComplianceScan** resource.

A related parameter is **rawResultStorage.rotation** which controls how many scans are retained in the PV before the older scans are rotated. The default value is 3, setting the rotation policy to 0 disables the rotation. Given the default rotation policy and an estimate of 100MB per a raw ARF scan report, you can calculate the right PV size for your environment.

Because OpenShift Container Platform can be deployed in a variety of public clouds or bare metal, the Compliance Operator cannot determine available storage configurations. By default, the Compliance Operator will try to create the PV for storing results by using the default storage class of the cluster, but a custom storage class can be configured using the **rawResultStorage.StorageClassName** attribute.



IMPORTANT

If your cluster does not specify a default storage class, this attribute must be set.

- Configure the **ScanSetting** custom resource to use a standard storage class and create persistent volumes that are 10GB in size and keep the last 10 results:

Example ScanSetting CR

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  name: default
  namespace: openshift-compliance
rawResultStorage:
  storageClassName: standard
  rotation: 10
  size: 10Gi
roles:
- worker
- master
scanTolerations:
- effect: NoSchedule
  key: node-role.kubernetes.io/master
  operator: Exists
schedule: '0 1 * * *'
```

5.6.7.6. Applying remediations generated by suite scans

Although you can use the **autoApplyRemediations** boolean parameter in a **ComplianceSuite** object, you can alternatively annotate the object with **compliance.openshift.io/apply-remediations**. This allows the Operator to apply all of the created remediations.

Procedure

- Apply the **compliance.openshift.io/apply-remediations** annotation by running the following command:

```
$ oc -n openshift-compliance \
  annotate compliancesuites/workers-compliancesuite compliance.openshift.io/apply-
  remediations=
```

5.6.7.7. Automatically update remediations

In some cases, a scan with newer content might mark remediations as **OUTDATED**. As an administrator, you can apply the **compliance.openshift.io/remove-outdated** annotation to apply new remediations and remove the outdated ones.

Alternatively, set the **autoUpdateRemediations** flag in a **ScanSetting** or **ComplianceSuite** object to update the remediations automatically.

Procedure

- Apply the **compliance.openshift.io/remove-outdated** annotation:

```
$ oc -n openshift-compliance \
  annotate compliancesuites/workers-compliancesuite compliance.openshift.io/remove-
  outdated=
```

5.6.7.8. Creating a custom SCC for the Compliance Operator

In some environments, you must create a custom Security Context Constraints (SCC) file to ensure the correct permissions are available to the Compliance Operator **api-resource-collector**.

Prerequisites

- You must have **admin** privileges.

Procedure

1. Define the SCC in a YAML file named **restricted-adjusted-compliance.yaml**:

SecurityContextConstraints object definition

```
allowHostDirVolumePlugin: false
allowHostIPC: false
allowHostNetwork: false
allowHostPID: false
allowHostPorts: false
allowPrivilegeEscalation: true
allowPrivilegedContainer: false
allowedCapabilities: null
apiVersion: security.openshift.io/v1
defaultAddCapabilities: null
fsGroup:
  type: MustRunAs
kind: SecurityContextConstraints
metadata:
  name: restricted-adjusted-compliance
priority: 30
readOnlyRootFilesystem: false
requiredDropCapabilities:
- KILL
- SETUID
- SETGID
- MKNOD
runAsUser:
  type: MustRunAsRange
seLinuxContext:
  type: MustRunAs
supplementalGroups:
  type: RunAsAny
users:
- system:serviceaccount:openshift-compliance:api-resource-collector
volumes:
- configMap
- downwardAPI
```

- emptyDir
- persistentVolumeClaim
- projected
- secret

where:

priority

Specifies the priority of this SCC. This value must be higher than any other SCC that applies to the **system:authenticated** group.

system:serviceaccount:openshift-compliance:api-resource-collector

Specifies the Service Account used by Compliance Operator Scanner pod.

2. Create the SCC:

```
$ oc create -n openshift-compliance -f restricted-adjusted-compliance.yaml
```

Example output

```
securitycontextconstraints.security.openshift.io/restricted-adjusted-compliance created
```

Verification

1. Verify the SCC was created:

```
$ oc get -n openshift-compliance scc restricted-adjusted-compliance
```

Example output

```
NAME                PRIV CAPS      SELINUX     RUNASUSER   FSGROUP
SUPGROUP PRIORITY READONLYROOTFS VOLUMES
restricted-adjusted-compliance false <no value> MustRunAs MustRunAsRange
MustRunAs RunAsAny 30 false
["configMap","downwardAPI","emptyDir","persistentVolumeClaim","projected","secret"]
```

5.6.7.9. Additional resources

- [Configuring priority and preemption](#)
- [Managing security context constraints](#)

5.6.8. Troubleshooting Compliance Operator scans

You can use the information on how to troubleshoot the Compliance Operator to learn how to diagnose a problem or provide information in a bug report.

When troubleshooting, review the following general tips:

- The Compliance Operator emits Kubernetes events when something important happens. You can either view all events in the cluster using the command:

```
$ oc get events -n openshift-compliance
```

-

Or view events for an object such as a scan using the command:

```
$ oc describe -n openshift-compliance compliancescan/cis-compliance
```

- The Compliance Operator consists of several controllers, approximately one per API object. It could be useful to filter only those controllers that correspond to the API object having issues. If a **ComplianceRemediation** cannot be applied, view the messages from the **remediationctrl** controller. You can filter the messages from a single controller by parsing with **jq**:

```
$ oc -n openshift-compliance logs compliance-operator-775d7bddbd-gj58f \
| jq -c 'select(.logger == "profilebundlectrl")'
```

- The timestamps are logged as seconds since UNIX epoch in UTC. To convert them to a human-readable date, use **date -d @timestamp --utc**, for example:

```
$ date -d @1596184628.955853 --utc
```

- Many custom resources, most importantly **ComplianceSuite** and **ScanSetting**, allow the **debug** option to be set. Enabling this option increases verbosity of the OpenSCAP scanner pods, and some other helper pods.
- If a single rule is passing or failing unexpectedly, it could be helpful to run a single scan or a suite with only that rule to find the rule ID from the corresponding **ComplianceCheckResult** object and use it as the **rule** attribute value in a **Scan** CR. Then, together with the **debug** option enabled, the **scanner** container logs in the scanner pod would show the raw OpenSCAP logs.

5.6.8.1. Anatomy of a scan

Before troubleshooting Compliance Operator scans, familiarize yourself with the components and stages of Compliance Operator scans.

5.6.8.1.1. Compliance sources

The compliance content is stored in **Profile** objects that are generated from a **ProfileBundle** object. The Compliance Operator creates a **ProfileBundle** object for the cluster and another for the cluster nodes.

```
$ oc get -n openshift-compliance profilebundle.compliance
```

```
$ oc get -n openshift-compliance profile.compliance
```

The **ProfileBundle** objects are processed by deployments labeled with the **Bundle** name. To troubleshoot an issue with the **Bundle**, you can find the deployment and view logs of the pods in a deployment:

```
$ oc logs -n openshift-compliance -lprofile-bundle=ocp4 -c profileparser
```

```
$ oc get -n openshift-compliance deployments,pods -lprofile-bundle=ocp4
```

```
$ oc logs -n openshift-compliance pods/<pod-name>
```

```
$ oc describe -n openshift-compliance pod/<pod-name> -c profileparser
```

5.6.8.1.2. The ScanSetting and ScanSettingBinding objects lifecycle and debugging

With valid compliance content sources, the high-level **ScanSetting** and **ScanSettingBinding** objects can be used to generate **ComplianceSuite** and **ComplianceScan** objects:

```
apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  name: my-companys-constraints
debug: true
# For each role, a separate scan will be created pointing
# to a node-role specified in roles
roles:
  - worker
---
apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSettingBinding
metadata:
  name: my-companys-compliance-requirements
profiles:
  # Node checks
  - name: rhcos4-e8
    kind: Profile
    apiGroup: compliance.openshift.io/v1alpha1
  # Cluster checks
  - name: ocp4-e8
    kind: Profile
    apiGroup: compliance.openshift.io/v1alpha1
settingsRef:
  name: my-companys-constraints
  kind: ScanSetting
  apiGroup: compliance.openshift.io/v1alpha1
```

Both **ScanSetting** and **ScanSettingBinding** objects are handled by the same controller tagged with **logger=scansettingbindingctrl**. These objects have no status. Any issues are communicated in form of events:

```
Events:
  Type    Reason      Age   From              Message
  ----    -
  Normal  SuiteCreated 9m52s scansettingbindingctrl ComplianceSuite openshift-compliance/my-companys-compliance-requirements created
```

Now a **ComplianceSuite** object is created. The flow continues to reconcile the newly created **ComplianceSuite**.

5.6.8.1.3. ComplianceSuite custom resource lifecycle and debugging

The **ComplianceSuite** CR is a wrapper around **ComplianceScan** CRs. The **ComplianceSuite** CR is handled by controller tagged with **logger=suitectrl**. This controller handles creating scans from a suite, reconciling and aggregating individual Scan statuses into a single Suite status. If a suite is set to run

periodically, the **suitectrl** also handles creating a **CronJob** CR that re-runs the scans in the suite after the initial run is done:

```
$ oc get cronjobs
```

Example output

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE	AGE
<cron_name>	0 1 * * *	False	0	<none>	151m

For the most important issues, events are emitted. View them with **oc describe compliancesuites/<name>**. The **Suite** objects also have a **Status** subresource that is updated when any of **Scan** objects that belong to this suite update their **Status** subresource. After all expected scans are created, control is passed to the scan controller.

5.6.8.1.4. ComplianceScan custom resource lifecycle and debugging

The **ComplianceScan** CRs are handled by the **scanctrl** controller. This is also where the actual scans happen and the scan results are created. Each scan goes through several phases:

5.6.8.1.5. Pending phase

The scan is validated for correctness in this phase. If some parameters such as storage size are invalid, the scan transitions to DONE with ERROR result, otherwise proceeds to the Launching phase.

5.6.8.1.6. Launching phase

In this phase, several config maps that contain either environment for the scanner pods or directly the script that the scanner pods will be evaluating. List the config maps:

```
$ oc -n openshift-compliance get cm \
-l compliance.openshift.io/scan-name=rhcos4-e8-worker,complianceoperator.openshift.io/scan-script=
```

These config maps will be used by the scanner pods. If you ever needed to modify the scanner behavior, change the scanner debug level or print the raw results, modifying the config maps is the way to go. Afterwards, a persistent volume claim is created per scan to store the raw ARF results:

```
$ oc get pvc -n openshift-compliance -lcompliance.openshift.io/scan-name=rhcos4-e8-worker
```

The PVCs are mounted by a per-scan **ResultServer** deployment. A **ResultServer** is a simple HTTP server where the individual scanner pods upload the full ARF results to. Each server can run on a different node. The full ARF results might be very large and you cannot presume that it would be possible to create a volume that could be mounted from multiple nodes at the same time. After the scan is finished, the **ResultServer** deployment is scaled down. The PVC with the raw results can be mounted from another custom pod and the results can be fetched or inspected. The traffic between the scanner pods and the **ResultServer** is protected by mutual TLS protocols.

Finally, the scanner pods are launched in this phase; one scanner pod for a **Platform** scan instance and one scanner pod per matching node for a **node** scan instance. The per-node pods are labeled with the node name. Each pod is always labeled with the **ComplianceScan** name:

```
$ oc get pods -lcompliance.openshift.io/scan-name=rhcos4-e8-worker,workload=scanner --show-
```


labels

Example output

```
NAME                                READY STATUS  RESTARTS AGE  LABELS
rhcos4-e8-worker-ip-10-0-169-90.eu-north-1.compute.internal-pod 0/2   Completed 0    39m
compliance.openshift.io/scan-name=rhcos4-e8-worker,targetNode=ip-10-0-169-90.eu-north-1.compute.internal,workload=scanner
```

The scan then proceeds to the Running phase.

5.6.8.1.7. Running phase

The running phase waits until the scanner pods finish. The following terms and processes are in use in the running phase:

- **init container**: There is one init container called **content-container**. It runs the **contentImage** container and executes a single command that copies the **contentFile** to the **/content** directory shared with the other containers in this pod.
- **scanner**: This container runs the scan. For node scans, the container mounts the node filesystem as **/host** and mounts the content delivered by the init container. The container also mounts the **entrypoint ConfigMap** created in the Launching phase and executes it. The default script in the entrypoint **ConfigMap** executes OpenSCAP and stores the result files in the **/results** directory shared between the containers in the pod. Logs from this pod can be viewed to determine what the OpenSCAP scanner checked. More verbose output can be viewed with the **debug** flag.
- **logcollector**: The logcollector container waits until the scanner container finishes. Then, it uploads the full ARF results to the **ResultServer** and separately uploads the XCCDF results along with scan result and OpenSCAP result code as a **ConfigMap**. These result config maps are labeled with the scan name (**compliance.openshift.io/scan-name=rhcos4-e8-worker**):

```
$ oc describe cm/rhcos4-e8-worker-ip-10-0-169-90.eu-north-1.compute.internal-pod
```

Example output

```
Name:      rhcos4-e8-worker-ip-10-0-169-90.eu-north-1.compute.internal-pod
Namespace: openshift-compliance
Labels:    compliance.openshift.io/scan-name-scan=rhcos4-e8-worker
           complianceoperator.openshift.io/scan-result=
Annotations: compliance-remediations/processed:
              compliance.openshift.io/scan-error-msg:
              compliance.openshift.io/scan-result: NON-COMPLIANT
              OpenSCAP-scan-result/node: ip-10-0-169-90.eu-north-1.compute.internal
```

```
Data
====
exit-code:
-----
2
results:
```

```

----
<?xml version="1.0" encoding="UTF-8"?>
...

```

Scanner pods for **Platform** scans are similar, except:

- There is one extra init container called **api-resource-collector** that reads the OpenSCAP content provided by the content-container init, container, figures out which API resources the content needs to examine and stores those API resources to a shared directory where the **scanner** container would read them from.
- The **scanner** container does not need to mount the host file system.

When the scanner pods are done, the scans move on to the Aggregating phase.

5.6.8.1.8. Aggregating phase

In the aggregating phase, the scan controller spawns yet another pod called the aggregator pod. Its purpose is to take the result **ConfigMap** objects, read the results and for each check result create the corresponding Kubernetes object. If the check failure can be automatically remediated, a **ComplianceRemediation** object is created. To provide human-readable metadata for the checks and remediations, the aggregator pod also mounts the OpenSCAP content by using an init container.

When a config map is processed by an aggregator pod, it is labeled the **compliance-remediations/processed** label. The result of this phase are **ComplianceCheckResult** objects:

```
$ oc get compliancecheckresults -lcompliance.openshift.io/scan-name=rhcos4-e8-worker
```

Example output

NAME	STATUS	SEVERITY
rhcos4-e8-worker-accounts-no-uid-except-zero	PASS	high
rhcos4-e8-worker-audit-rules-dac-modification-chmod	FAIL	medium

and **ComplianceRemediation** objects:

```
$ oc get complianceremediations -lcompliance.openshift.io/scan-name=rhcos4-e8-worker
```

Example output

NAME	STATE
rhcos4-e8-worker-audit-rules-dac-modification-chmod	NotApplied
rhcos4-e8-worker-audit-rules-dac-modification-chown	NotApplied
rhcos4-e8-worker-audit-rules-execution-chcon	NotApplied
rhcos4-e8-worker-audit-rules-execution-restorecon	NotApplied
rhcos4-e8-worker-audit-rules-execution-semanage	NotApplied
rhcos4-e8-worker-audit-rules-execution-setfiles	NotApplied

After these CRs are created, the aggregator pod exits and the scan moves on to the Done phase.

5.6.8.1.9. Done phase

In the final scan phase, the scan resources are cleaned up if needed and the **ResultServer** deployment is either scaled down (if the scan was one-time) or deleted if the scan is continuous; the next scan instance would then re-create the deployment again.

It is also possible to trigger a re-run of a scan in the Done phase by annotating it:

```
$ oc -n openshift-compliance \
  annotate compliancescans/rhcos4-e8-worker compliance.openshift.io/rescan=
```

After the scan reaches the Done phase, nothing else happens on its own unless the remediations are set to be applied automatically with **autoApplyRemediations: true**. The OpenShift Container Platform administrator would now review the remediations and apply them as needed. If the remediations are set to be applied automatically, the **ComplianceSuite** controller takes over in the Done phase, pauses the machine config pool to which the scan maps to and applies all the remediations in one go. If a remediation is applied, the **ComplianceRemediation** controller takes over.

5.6.8.1.10. ComplianceRemediation controller lifecycle and debugging

The example scan has reported some findings. One of the remediations can be enabled by toggling its **apply** attribute to **true**:

```
$ oc patch complianceremediations/rhcos4-e8-worker-audit-rules-dac-modification-chmod --patch
'{"spec":{"apply":true}}' --type=merge
```

The **ComplianceRemediation** controller (**logger=remediationctrl**) reconciles the modified object. The result of the reconciliation is change of status of the remediation object that is reconciled, but also a change of the rendered per-suite **MachineConfig** object that contains all the applied remediations.

The **MachineConfig** object always begins with **75-** and is named after the scan and the suite:

```
$ oc get mc | grep 75-
```

Example output

```
75-rhcos4-e8-worker-my-companys-compliance-requirements      3.2.0
2m46s
```

The remediations the **mc** currently consists of are listed in the annotations of the machine config:

```
$ oc describe mc/75-rhcos4-e8-worker-my-companys-compliance-requirements
```

Example output

```
Name:      75-rhcos4-e8-worker-my-companys-compliance-requirements
Labels:    machineconfiguration.openshift.io/role=worker
Annotations: remediation/rhcos4-e8-worker-audit-rules-dac-modification-chmod:
```

The **ComplianceRemediation** controller algorithm works like this:

- All currently applied remediations are read into an initial remediation set.
- If the reconciled remediation is supposed to be applied, it is added to the set.

- A **MachineConfig** object is rendered from the set and annotated with names of remediations in the set. If the set is empty (the last remediation was unapplied), the rendered **MachineConfig** object is removed.
- If and only if the rendered machine config is different from the one already applied in the cluster, the applied MC is updated (or created, or deleted).
- Creating or modifying a **MachineConfig** object triggers a reboot of nodes that match the **machineconfiguration.openshift.io/role** label - see the Machine Config Operator documentation for more details.

The remediation loop ends once the rendered machine config is updated, if needed, and the reconciled remediation object status is updated. In our case, applying the remediation would trigger a reboot. After the reboot, annotate the scan to re-run it:

```
$ oc -n openshift-compliance \
  annotate compliancescans/rhcos4-e8-worker compliance.openshift.io/rescan=
```

The scan will run and finish. Check for the remediation to pass:

```
$ oc -n openshift-compliance \
  get compliancecheckresults/rhcos4-e8-worker-audit-rules-dac-modification-chmod
```

Example output

NAME	STATUS	SEVERITY
rhcos4-e8-worker-audit-rules-dac-modification-chmod	PASS	medium

5.6.8.11. Useful labels

Each pod that is spawned by the Compliance Operator is labeled specifically with the scan it belongs to and the work it does. The scan identifier is labeled with the **compliance.openshift.io/scan-name** label. The workload identifier is labeled with the **workload** label.

The Compliance Operator schedules the following workloads:

- **scanner**: Performs the compliance scan.
- **resultserver**: Stores the raw results for the compliance scan.
- **aggregator**: Aggregates the results, detects inconsistencies and outputs result objects (checkresults and remediations).
- **suitererunner**: Will tag a suite to be re-run (when a schedule is set).
- **profileparser**: Parses a datastream and creates the appropriate profiles, rules and variables.

When debugging and logs are required for a certain workload, run:

```
$ oc logs -l workload=<workload_name> -c <container_name>
```

5.6.8.2. Increasing Compliance Operator resource limits

In some cases, the Compliance Operator might require more memory than the default limits allow. You can mitigate this issue by setting custom resource limits.

To increase the default memory and CPU limits of scanner pods, see *`ScanSetting` Custom resource*.

Procedure

1. To increase the Operator memory limits to 500 Mi, create the following patch file named **co-memlimit-patch.yaml**:

```
spec:
  config:
    resources:
      limits:
        memory: 500Mi
```

2. Apply the patch file:

```
$ oc patch sub compliance-operator -n openshift-compliance --patch-file co-memlimit-patch.yaml --type=merge
```

5.6.8.3. Configuring Operator resource constraints

You can configure the **resources** field in the **compliance-operator** subscription object to define resource constraints for all the containers in the pod created by the Operator Lifecycle Manager (OLM), so the Operator pods have enough CPU and memory.



NOTE

Resource Constraints applied in this process overwrites the existing resource constraints.

Procedure

- Inject a request of 0.25 cpu and 64 Mi of memory, and a limit of 0.5 cpu and 128 Mi of memory in each container by editing the **Subscription** object:

```
kind: Subscription
metadata:
  name: compliance-operator
  namespace: openshift-compliance
spec:
  package: package-name
  channel: stable
  config:
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

5.6.8.4. Configuring ScanSetting resources

When using the Compliance Operator in a cluster that contains more than 500 MachineConfigs, the **ocp4-pci-dss-api-checks-pod** pod might pause in the **init** phase when performing a **Platform** scan.



NOTE

Resource constraints applied in this process overwrites the existing resource constraints.

Procedure

1. Confirm the **ocp4-pci-dss-api-checks-pod** pod is stuck in the **Init:OOMKilled** status:

```
$ oc get pod ocp4-pci-dss-api-checks-pod -w
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
ocp4-pci-dss-api-checks-pod	0/2	Init:1/2	8 (5m56s ago)	25m
ocp4-pci-dss-api-checks-pod	0/2	Init:OOMKilled	8 (6m19s ago)	26m

2. Edit the **scanLimits** attribute in the **ScanSetting** CR to increase the available memory for the **ocp4-pci-dss-api-checks-pod** pod:

```
timeout: 30m
strictNodeScan: true
metadata:
  name: default
  namespace: openshift-compliance
kind: ScanSetting
showNotApplicable: false
rawResultStorage:
  nodeSelector:
    node-role.kubernetes.io/master: "
pvAccessModes:
  - ReadWriteOnce
rotation: 3
size: 1Gi
tolerations:
  - effect: NoSchedule
    key: node-role.kubernetes.io/master
    operator: Exists
  - effect: NoExecute
    key: node.kubernetes.io/not-ready
    operator: Exists
    tolerationSeconds: 300
  - effect: NoExecute
    key: node.kubernetes.io/unreachable
    operator: Exists
    tolerationSeconds: 300
  - effect: NoSchedule
    key: node.kubernetes.io/memory-pressure
    operator: Exists
schedule: 0 1 * * *
roles:
  - master
```

```

- worker
apiVersion: compliance.openshift.io/v1alpha1
maxRetryOnTimeout: 3
scanTolerations:
  - operator: Exists
scanLimits:
  memory: 1024Mi

```

where:

scanLimits.memory

Specifies the default setting is **500Mi**.

3. Apply the **ScanSetting** CR to your cluster:

```
$ oc apply -f scansetting.yaml
```

5.6.8.5. Configuring ScanSetting timeout

The **ScanSetting** object has a timeout option that you can specify in the **ComplianceScanSetting** object as a duration string, such as **1h30m**. If the scan does not finish within the specified timeout, the scan reattempts until the **maxRetryOnTimeout** limit is reached.

Procedure

- To set a **timeout** and **maxRetryOnTimeout** in ScanSetting, modify an existing **ScanSetting** object:

```

apiVersion: compliance.openshift.io/v1alpha1
kind: ScanSetting
metadata:
  name: default
  namespace: openshift-compliance
rawResultStorage:
  rotation: 3
  size: 1Gi
roles:
  - worker
  - master
scanTolerations:
  - effect: NoSchedule
    key: node-role.kubernetes.io/master
    operator: Exists
schedule: '0 1 * * *'
timeout: '10m0s'
maxRetryOnTimeout: 3

```

where:

timeout

Specifies a duration string, such as **1h30m**. The default value is **30m**. To disable the timeout, set the value to **0s**.

maxRetryOnTimeout

Specifies the **maxRetryOnTimeout** variable defines how many times a retry is attempted. The default value is **3**.

5.6.8.6. Getting support

Red Hat offers several support channels to help you troubleshoot issues and get the most from OpenShift Container Platform.

From the Red Hat Customer Portal, you can:

- Search or browse through the Red Hat Knowledgebase of articles and solutions about Red Hat products.
- Submit a support case to Red Hat Support.
- Access other product documentation.

To identify issues with your cluster, you can use Red Hat Lightspeed in [OpenShift Cluster Manager](#). Red Hat Lightspeed provides details about issues and, if available, information about how to solve a problem.

To suggest improvements or report errors, give specific details such as the section name and OpenShift Container Platform version.

5.6.9. Using the oc-compliance plugin

Although the Compliance Operator automates many of the checks and remediations for the cluster, an administrator can use the **oc-compliance** plugin to perform the full process of bringing a cluster into compliance by interacting with the Compliance Operator API and other components.

5.6.9.1. Installing the oc-compliance plugin

You can install the **oc-compliance** plugin to simplify compliance operations from the command line.

Procedure

- Extract the **oc-compliance** image to get the **oc-compliance** binary:

```
$ podman run --rm -v ~/.local/bin:/mnt/out:Z registry.redhat.io/compliance/oc-compliance-rhel8:stable /bin/cp /usr/bin/oc-compliance /mnt/out/
```

Example output

```
W0611 20:35:46.486903 11354 manifest.go:440] Chose linux/amd64 manifest from the manifest list.
```

You can now run **oc-compliance**.

5.6.9.2. Fetching raw results

An administrator or auditor can review the complete detailed results of a scan as created by the OpenSCAP tool. These results contain more details than what is contained in the **ComplianceCheckResult** custom resource (CR).

When a compliance scan finishes, the results of the individual checks are listed in the resulting **ComplianceCheckResult** custom resource (CR). However, an administrator or auditor might require the complete details of the scan. The OpenSCAP tool creates an Advanced Recording Format (ARF) formatted file with the detailed results. This ARF file is too large to store in a config map or other standard Kubernetes resource, so a persistent volume (PV) is created to contain it.

Procedure

1. Fetch the results from the PV by running the following command:

```
$ oc compliance fetch-raw <object-type> <object-name> -o <output-path>
```

where:

- **<object-type>** can be either **scansettingbinding**, **compliancescan** or **compliancesuite**, depending on which of these objects the scans were launched with.
- **<object-name>** is the name of the binding, suite, or scan object to gather the ARF file for, and **<output-path>** is the local directory to place the results.

For example:

```
$ oc compliance fetch-raw scansettingbindings my-binding -o /tmp/
```

Example output

```
Fetching results for my-binding scans: ocp4-cis, ocp4-cis-node-worker, ocp4-cis-node-master
Fetching raw compliance results for scan 'ocp4-cis'.....
The raw compliance results are available in the following directory: /tmp/ocp4-cis
Fetching raw compliance results for scan 'ocp4-cis-node-worker'.....
The raw compliance results are available in the following directory: /tmp/ocp4-cis-node-worker
Fetching raw compliance results for scan 'ocp4-cis-node-master'.....
The raw compliance results are available in the following directory: /tmp/ocp4-cis-node-master
```

2. View the list of files in the directory:

```
$ ls /tmp/ocp4-cis-node-master/
```

Example output

```
ocp4-cis-node-master-ip-10-0-128-89.ec2.internal-pod.xml.bzip2 ocp4-cis-node-master-ip-10-0-150-5.ec2.internal-pod.xml.bzip2 ocp4-cis-node-master-ip-10-0-163-32.ec2.internal-pod.xml.bzip2
```

3. Extract the results:

```
$ bunzip2 -c resultsdir/worker-scan/worker-scan-stage-459-tqkg7-compute-0-pod.xml.bzip2
> resultsdir/worker-scan/worker-scan-ip-10-0-170-231.us-east-2.compute.internal-pod.xml
```

4. View the extracted results:

```
$ ls resultsdir/worker-scan/
```

Example output

```
worker-scan-ip-10-0-170-231.us-east-2.compute.internal-pod.xml
worker-scan-stage-459-tqkg7-compute-0-pod.xml.bzip2
worker-scan-stage-459-tqkg7-compute-1-pod.xml.bzip2
```

5.6.9.3. Re-running scans

Although it is possible to run scans as scheduled jobs, you must often re-run a scan on demand, particularly after remediations are applied or when other changes to the cluster are made.

Rerunning a scan with the Compliance Operator requires the use of an annotation on the scan object. However, with the **oc-compliance** plugin you can rerun a scan with a single command.

Procedure

- Rerun the scans for the **ScanSettingBinding** object named **my-binding** by running the following command:

```
$ oc compliance rerun-now scansettingbindings my-binding
```

Example output

```
Rerunning scans from 'my-binding': ocp4-cis
Re-running scan 'openshift-compliance/ocp4-cis'
```

5.6.9.4. Using ScanSettingBinding custom resources

When using the **ScanSetting** and **ScanSettingBinding** custom resources (CRs) that the Compliance Operator provides, it is possible to run scans for multiple profiles while using a common set of scan options, such as **schedule**, **machine roles**, **tolerations**, and so on.

Although that is easier than working with multiple **ComplianceSuite** or **ComplianceScan** objects, it can confuse new users.

The **oc compliance bind** subcommand helps you create a **ScanSettingBinding** CR.

Procedure

1. Run:

```
$ oc compliance bind [--dry-run] -N <binding name> [-S <scansetting name>]
<objtype/objname> [..<objtype/objname>]
```

- If you omit the **-S** flag, the **default** scan setting provided by the Compliance Operator is used.
- The object type is the Kubernetes object type, which can be **profile** or **tailoredprofile**. More than one object can be provided.

- The object name is the name of the Kubernetes resource, such as **.metadata.name**.
- Add the **--dry-run** option to display the YAML file of the objects that are created. For example, given the following profiles and scan settings:

```
$ oc get profile.compliance -n openshift-compliance
```

Example output

```
NAME                AGE    VERSION
ocp4-cis             3h49m  1.9.0
ocp4-cis-1-9         3h49m  1.9.0
ocp4-cis-node        3h49m  1.9.0
ocp4-cis-node-1-9    3h49m  1.9.0
ocp4-e8              3h49m
ocp4-high            3h49m  Revision 4
ocp4-high-node       3h49m  Revision 4
ocp4-high-node-rev-4 3h49m  Revision 4
ocp4-high-rev-4      3h49m  Revision 4
ocp4-moderate        3h49m  Revision 4
ocp4-moderate-node   3h49m  Revision 4
ocp4-moderate-node-rev-4 3h49m  Revision 4
ocp4-moderate-rev-4  3h49m  Revision 4
ocp4-nerc-cip        3h49m
ocp4-nerc-cip-node   3h49m
ocp4-pci-dss          3h49m  4.0.0
ocp4-pci-dss-3-2      3h49m  3.2.1
ocp4-pci-dss-4-0      3h49m  4.0.0
ocp4-pci-dss-node     3h49m  4.0.0
ocp4-pci-dss-node-3-2 3h49m  3.2.1
ocp4-pci-dss-node-4-0 3h49m  4.0.0
ocp4-stig            3h49m  V2R3
ocp4-stig-node       3h49m  V2R3
ocp4-stig-node-v2r3   3h49m  V2R3
ocp4-stig-v2r3        3h49m  V2R3
rhcos4-e8            3h49m
rhcos4-high          3h49m  Revision 4
rhcos4-high-rev-4     3h49m  Revision 4
rhcos4-moderate       3h49m  Revision 4
rhcos4-moderate-rev-4 3h49m  Revision 4
rhcos4-nerc-cip       3h49m
rhcos4-stig           3h49m  V2R3
rhcos4-stig-v2r3      3h49m  V2R3
```

```
$ oc get scansettings -n openshift-compliance
```

Example output

```
NAME                AGE
default             10m
default-auto-apply  10m
```

2. To apply the **default** settings to the **ocp4-cis** and **ocp4-cis-node** profiles, run:

—

```
$ oc compliance bind -N my-binding profile/ocp4-cis profile/ocp4-cis-node
```

Example output

```
Creating ScanSettingBinding my-binding
```

After the **ScanSettingBinding** CR is created, the bound profile begins scanning for both profiles with the related settings. Overall, this is the fastest way to begin scanning with the Compliance Operator.

5.6.9.5. Printing controls

You can view a report of the compliance standards and controls that a given profile satisfies.

Compliance standards are generally organized into a the following hierarchy:

- A benchmark is the top-level definition of a set of controls for a particular standard. For example, FedRAMP Moderate or Center for Internet Security (CIS) v.1.6.0.
- A control describes a family of requirements that must be met to be in compliance with the benchmark. For example, FedRAMP AC-01 (access control policy and procedures).
- A rule is a single check that is specific for the system being brought into compliance, and one or more of these rules map to a control.
- The Compliance Operator handles the grouping of rules into a profile for a single benchmark. It can be difficult to determine which controls that the set of rules in a profile satisfy.

Procedure

- The **oc compliance controls** subcommand provides a report of the standards and controls that a given profile satisfies:

```
$ oc compliance controls profile ocp4-cis-node
```

Example output

```
+-----+-----+
| FRAMEWORK | CONTROLS |
+-----+-----+
| CIS-OCF  | 1.1.1  |
+      +-----+
|      | 1.1.10 |
+      +-----+
|      | 1.1.11 |
+      +-----+
...
```

5.6.9.6. Fetching compliance remediation details

The Compliance Operator provides remediation objects that are used to automate the changes required to make the cluster compliant. You can use the **fetch-fixes** subcommand to help you understand exactly which configuration remediations are used.

The **fetch-fixes** extracts the remediation objects from a profile, rule, or **ComplianceRemediation** object into a directory for you to inspect.



WARNING

Use caution before applying remediations directly. Some remediations might not be applicable in bulk, such as the usbguard rules in the moderate profile. In these cases, allow the Compliance Operator to apply the rules because it addresses the dependencies and ensures that the cluster remains in a good state.

Procedure

1. View the remediations for a profile:

```
$ oc compliance fetch-fixes profile ocp4-cis -o /tmp
```

Example output

```
No fixes to persist for rule 'ocp4-api-server-api-priority-flowschema-catch-all'
No fixes to persist for rule 'ocp4-api-server-api-priority-gate-enabled'
No fixes to persist for rule 'ocp4-api-server-audit-log-maxbackup'
Persisted rule fix to /tmp/ocp4-api-server-audit-log-maxsize.yaml
No fixes to persist for rule 'ocp4-api-server-audit-log-path'
No fixes to persist for rule 'ocp4-api-server-auth-mode-no-aa'
No fixes to persist for rule 'ocp4-api-server-auth-mode-node'
No fixes to persist for rule 'ocp4-api-server-auth-mode-rbac'
No fixes to persist for rule 'ocp4-api-server-basic-auth'
No fixes to persist for rule 'ocp4-api-server-bind-address'
No fixes to persist for rule 'ocp4-api-server-client-ca'
Persisted rule fix to /tmp/ocp4-api-server-encryption-provider-cipher.yaml
Persisted rule fix to /tmp/ocp4-api-server-encryption-provider-config.yaml
```



NOTE

The **No fixes to persist** warning is expected whenever there are rules in a profile that do not have a corresponding remediation, because either the rule cannot be remediated automatically or a remediation was not provided.

2. You can view a sample of the YAML file. The **head** command will show you the first 10 lines:

```
$ head /tmp/ocp4-api-server-audit-log-maxsize.yaml
```

Example output

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
```

```
name: cluster
spec:
  maximumFileSizeMegabytes: 100
```

3. View the remediation from a **ComplianceRemediation** object created after a scan:

```
$ oc get complianceremediations -n openshift-compliance
```

Example output

```
NAME                                STATE
ocp4-cis-api-server-encryption-provider-cipher  NotApplied
ocp4-cis-api-server-encryption-provider-config  NotApplied
```

```
$ oc compliance fetch-fixes complianceremediations ocp4-cis-api-server-encryption-provider-cipher -o /tmp
```

Example output

```
Persisted compliance remediation fix to /tmp/ocp4-cis-api-server-encryption-provider-cipher.yaml
```

4. You can view a sample of the YAML file. The **head** command will show you the first 10 lines:

```
$ head /tmp/ocp4-cis-api-server-encryption-provider-cipher.yaml
```

Example output

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  name: cluster
spec:
  encryption:
    type: aescbc
```

5.6.9.7. Viewing ComplianceCheckResult object details

When scans are finished running, **ComplianceCheckResult** objects are created for the individual scan rules. You can use the **view-result** subcommand to provide a human-readable output of the **ComplianceCheckResult** object details.

Procedure

- Run:

```
$ oc compliance view-result ocp4-cis-scheduler-no-bind-address
```

5.6.9.8. Additional resources

- [Understanding the Compliance Operator](#)

CHAPTER 6. FILE INTEGRITY OPERATOR

6.1. FILE INTEGRITY OPERATOR OVERVIEW

The File Integrity Operator continually runs file integrity checks on the cluster nodes. It deploys a DaemonSet that initializes and runs privileged Advanced Intrusion Detection Environment (AIDE) containers on each node, providing a log of files that have been modified since the initial run of the DaemonSet pods.



NOTE

File Integrity Operator is not supported on HCP clusters.

6.1.1. Additional resources

- [File Integrity Operator release notes](#)
- [File Integrity Operator support](#)
- [Installing the File Integrity Operator](#)
- [Updating the File Integrity Operator](#)
- [Understanding the File Integrity Operator](#)
- [Configuring the Custom File Integrity Operator](#)
- [Performing advanced Custom File Integrity Operator tasks](#)
- [Troubleshooting the File Integrity Operator](#)
- [Uninstalling the File Integrity Operator](#)

6.2. RELEASE NOTES FOR THE FILE INTEGRITY OPERATOR

The File Integrity Operator for OpenShift Container Platform continually runs file integrity checks on RHCOS nodes.

These release notes track the development of the File Integrity Operator in the OpenShift Container Platform.

6.2.1. Release notes for OpenShift File Integrity Operator 1.4.1

OpenShift File Integrity Operator 1.4.1 is now available. The **stable** update channel tracks and receives updates for the File Integrity Operator. For more information, see [Updating the File Integrity Operator](#). The following Red Hat Security Advisory (RHSA) is available:

- [RHSA-2026:54288 - OpenShift File Integrity Operator 1.4.1 bug fix and enhancement update](#)

This update includes upgraded golang dependencies in the underlying base images.

6.2.1.1. New features and enhancements

- With this release, the File Integrity Operator can manage **NetworkPolicy** resources with **create**, **delete**, **get**, and **update** commands. The **file-integrity-operator** service account now has matching namespace-scoped permissions. ([CMP-4497](#))

6.2.2. Release notes for OpenShift File Integrity Operator 1.4.0

Release notes for OpenShift File Integrity Operator 1.4.0.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift File Integrity Operator 1.4.0:

- [RHSA-2026:22627 OpenShift File Integrity Operator Update](#)

6.2.2.1. New features and enhancements

- With this release, you can optionally set **priorityClassName** in the **FileIntegrity** custom resource (CR) to assign a **PriorityClass** to file integrity daemon pods. On nodes under resource pressure, the scheduler can preempt lower-priority workloads to make room for those pods, helping ensure nodes continue to receive integrity checks. ([RFE-9047](#))

6.2.2.2. Bug fixes

- Before this update, **aide-worker-fileintegrity** pods could use increasing CPU and memory during hourly Advanced Intrusion Detection Environment (AIDE) scan cycles, often approaching DaemonSet resource limits and disrupting integrity checks on affected nodes. With this release, AIDE worker pods use CPU and memory more consistently during scans. ([CMP-4006](#))

This update includes upgraded dependencies in the underlying base images.

6.2.3. Release notes for OpenShift File Integrity Operator 1.3.8

Release notes for OpenShift File Integrity Operator 1.3.8.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift File Integrity Operator 1.3.8:

- [RHSA-2025:23542 OpenShift File Integrity Operator Update](#)

6.2.3.1. Bug fixes

- Before this update, the file-integrity-operator pods and the aide pods running the database used by a recently installed File Integrity Operator (FIO) would go into a terminating state, adding error log entries that were not useful. With this release, the pods needed by FIO do not go into terminating state unless a relevant error occurred or they completed their work. ([CMP-3757](#))
- This update includes upgraded dependencies in the underlying base images.

6.2.4. Release notes for OpenShift File Integrity Operator 1.3.7

Release notes for OpenShift File Integrity Operator 1.3.7.

The following Red Hat Security Advisory (RHSA) is available for the OpenShift File Integrity Operator 1.3.7:

- [RHSA-2025:21913 OpenShift File Integrity Operator Update](#)

This update includes upgraded dependencies in underlying base images.

6.2.5. Release notes for OpenShift File Integrity Operator 1.3.6

Release notes for OpenShift File Integrity Operator 1.3.6.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.3.6:

- [RHBA-2025:11535 OpenShift File Integrity Operator Bug Fix Update](#)

6.2.5.1. Bug fixes

- Before this update, running the **oc annotate fileintegrities/<fileintegrity-name> file-integrity.openshift.io/re-init-on-failed=** command would trigger a reinitialization on all nodes. Now, it only reinitializes the nodes where failures occurred. ([OCPBUGS-18933](#))
- Before this update, resetting FIO cleared the **NodeHasIntegrityFailure** alert. This occurred because the **metric file_integrity_operator_node_failed** setting was also reset. With this release, restarting FIO does not affect the **NodeHasIntegrityFailure** alert. ([OCPBUGS-42807](#))
- Before this update, when a new node was added to a cluster by scaling up the **machineset** object, FIO marked the new node as **Failed** before the node was ready. With this release FIO waits until the new node is ready. ([OCPBUGS-36483](#))
- Before this update, the Advanced Intrusion Detection Environment (AIDE) daemonset pods would constantly force-initialize the AIDE database. With this release, FIO initializes the AIDE database only once. ([OCPBUGS-37300](#))
- Before this update, some link paths in the Machine Config Operator (MCO) configuration, such as **/hostroot/etc/ipsec.d/openshift.conf** and **hostroot/etc/mco/internal-registry-pull-secret.json**, changed during an MCO update. This led to failed file integrity checks on nodes after the update, which disrupted user experience. With this update, the File Integrity Operator (FIO) uses the updated file link paths in the MCO configuration. File integrity checks now pass after an update, helping to ensure a stable cluster. ([OCPBUGS-41628](#))

6.2.6. Release notes for OpenShift File Integrity Operator 1.3.5

Release notes for OpenShift File Integrity Operator 1.3.5.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.3.5:

- [RHBA-2024:10366 OpenShift File Integrity Operator Update](#)

This update includes upgraded dependencies in underlying base images.

6.2.7. Release notes for OpenShift File Integrity Operator 1.3.4

Release notes for OpenShift File Integrity Operator 1.3.4.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.3.4:

- [RHBA-2024:2946 OpenShift File Integrity Operator Bug Fix and Enhancement Update](#)

6.2.7.1. Bug fixes

- Before this update, File Integrity Operator would issue a **NodeHasIntegrityFailure** alert due to multus certificate rotation. With this release, the alert and failing status are now correctly triggered. ([OCPBUGS-31257](#))

6.2.8. Release notes for OpenShift File Integrity Operator 1.3.3

Release notes for OpenShift File Integrity Operator 1.3.3.

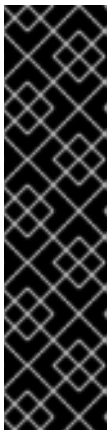
The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.3.3:

- [RHBA-2023:5652 OpenShift File Integrity Operator Bug Fix and Enhancement Update](#)

This update addresses a CVE in an underlying dependency.

6.2.8.1. New features and enhancements

- You can install and use the File Integrity Operator in an OpenShift Container Platform cluster running in FIPS mode.



IMPORTANT

To enable FIPS mode for your cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Switching RHEL to FIPS mode](#).

When running Red Hat Enterprise Linux (RHEL) or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures.

6.2.8.2. Bug fixes

- Before this update, some FIO pods with private default mount propagation in combination with **hostPath: path: /** volume mounts would break the CSI driver relying on multipath. This problem has been fixed and the CSI driver works correctly. ([Some OpenShift Operator pods blocking unmounting of CSI volumes when multipath is in use](#))
- This update resolves CVE-2023-39325. ([CVE-2023-39325](#))

6.2.9. Release notes for OpenShift File Integrity Operator 1.3.2

Release notes for OpenShift File Integrity Operator 1.3.2.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.3.2:

- [RHBA-2023:5107 OpenShift File Integrity Operator Bug Fix Update](#)

This update addresses a CVE in an underlying dependency.

6.2.10. Release notes for OpenShift File Integrity Operator 1.3.1

Release notes for OpenShift File Integrity Operator 1.3.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.3.1:

- [RHBA-2023:3600 OpenShift File Integrity Operator Bug Fix Update](#)

6.2.10.1. New features and enhancements

- FIO now includes kubelet certificates as default files, excluding them from issuing warnings when they're managed by OpenShift Container Platform. ([OCPBUGS-14348](#))
- FIO now correctly directs email to the address for Red Hat Technical Support. ([OCPBUGS-5023](#))

6.2.10.2. Bug fixes

- Before this update, the File Integrity Operator (FIO) would not clean up **FileIntegrityNodeStatus** CRDs when nodes are removed from the cluster. FIO now correctly cleans up node status CRDs on node removal. ([OCPBUGS-4321](#))
- Before this update, FIO would also erroneously indicate that new nodes failed integrity checks. FIO now correctly shows node status CRDs when adding new nodes to the cluster. This provides correct node status notifications. ([OCPBUGS-8502](#))
- Before this update, when FIO was reconciling **FileIntegrity** CRDs, it would pause scanning until the reconciliation was done. This caused an overly aggressive re-initiatization process on nodes not impacted by the reconciliation. This problem also resulted in unnecessary daemonsets for machine config pools which are unrelated to the **FileIntegrity** being changed. FIO correctly handles these cases and only pauses AIDE scanning for nodes that are affected by file integrity changes. ([CMP-1097](#))

6.2.10.3. Known Issues

In FIO 1.3.1, increasing nodes in IBM Z® clusters might result in **Failed** File Integrity node status. For more information, see [Adding nodes in IBM Power® clusters can result in failed File Integrity node status](#) .

6.2.11. Release notes for OpenShift File Integrity Operator 1.2.1

Release notes for OpenShift File Integrity Operator 1.2.1.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.2.1:

- [RHBA-2023:1684 OpenShift File Integrity Operator Bug Fix Update](#)
- This release includes updated container dependencies.

6.2.12. Release notes for OpenShift File Integrity Operator 1.2.0

Release notes for OpenShift File Integrity Operator 1.2.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.2.0:

- [RHBA-2023:1273 OpenShift File Integrity Operator Enhancement Update](#)

6.2.12.1. New features and enhancements

- The File Integrity Operator Custom Resource (CR) now contains an **initialDelay** feature that specifies the number of seconds to wait before starting the first AIDE integrity check. For more information, see [Creating the FileIntegrity custom resource](#).
- The File Integrity Operator is now stable and the release channel is upgraded to **stable**. Future releases will follow [Semantic Versioning](#). To access the latest release, see [Updating the File Integrity Operator](#).

6.2.13. Release notes for OpenShift File Integrity Operator 1.0.0

Release notes for OpenShift File Integrity Operator 1.0.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 1.0.0:

- [RHBA-2023:0037 OpenShift File Integrity Operator Bug Fix Update](#)

6.2.14. Release notes for OpenShift File Integrity Operator 0.1.32

Release notes for OpenShift File Integrity Operator 0.1.32.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 0.1.32:

- [RHBA-2022:7095 OpenShift File Integrity Operator Bug Fix Update](#)

6.2.14.1. Bug fixes

- Before this update, alerts issued by the File Integrity Operator did not set a namespace, making it difficult to understand from which namespace the alert originated. Now, the Operator sets the appropriate namespace, providing more information about the alert. ([BZ#2112394](#))
- Before this update, The File Integrity Operator did not update the metrics service on Operator startup, causing the metrics targets to be unreachable. With this release, the File Integrity Operator now ensures the metrics service is updated on Operator startup. ([BZ#2115821](#))

6.2.15. Release notes for OpenShift File Integrity Operator 0.1.30

Release notes for OpenShift File Integrity Operator 0.1.30.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 0.1.30:

- [RHBA-2022:5538 OpenShift File Integrity Operator Bug Fix and Enhancement Update](#)

6.2.15.1. New features and enhancements

- The File Integrity Operator is now supported on the following architectures:
 - IBM Power®
 - IBM Z® and IBM® LinuxONE

6.2.15.2. Bug fixes

- Before this update, alerts issued by the File Integrity Operator did not set a namespace, making it difficult to understand where the alert originated. Now, the Operator sets the appropriate namespace, increasing understanding of the alert. ([BZ#2101393](#))

6.2.16. Release notes for OpenShift File Integrity Operator 0.1.24

Release notes for OpenShift File Integrity Operator 0.1.24.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 0.1.24:

- [RHBA-2022:1331 OpenShift File Integrity Operator Bug Fix](#)

6.2.16.1. New features and enhancements

- You can now configure the maximum number of backups stored in the **FileIntegrity** Custom Resource (CR) with the **config.maxBackups** attribute. This attribute specifies the number of AIDE database and log backups left over from the **re-init** process to keep on the node. Older backups beyond the configured number are automatically pruned. The default is set to five backups.

6.2.16.2. Bug fixes

- Before this update, upgrading the Operator from versions older than 0.1.21 to 0.1.22 could cause the **re-init** feature to fail. This was a result of the Operator failing to update **configMap** resource labels. Now, upgrading to the latest version fixes the resource labels. ([BZ#2049206](#))
- Before this update, when enforcing the default **configMap** script contents, the wrong data keys were compared. This resulted in the **aide-reinit** script not being updated properly after an Operator upgrade, and caused the **re-init** process to fail. Now, **daemonSets** run to completion and the AIDE database **re-init** process executes successfully. ([BZ#2072058](#))

6.2.17. Release notes for OpenShift File Integrity Operator 0.1.22

Release notes for OpenShift File Integrity Operator 0.1.22.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 0.1.22:

- [RHBA-2022:0142 OpenShift File Integrity Operator Bug Fix](#)

6.2.17.1. Bug fixes

- Before this update, a system with a File Integrity Operator installed might interrupt the OpenShift Container Platform update, due to the **/etc/kubernetes/aide.reinit** file. This occurred if the **/etc/kubernetes/aide.reinit** file was present, but later removed before the **ostree**

validation. With this update, `/etc/kubernetes/aide.reinit` is moved to the `/run` directory so that it does not conflict with the OpenShift Container Platform update. ([BZ#2033311](#))

6.2.18. Release notes for OpenShift File Integrity Operator 0.1.21

Release notes for OpenShift File Integrity Operator 0.1.21.

The following Red Hat Bug Fix Advisory (RHBA) is available for the OpenShift File Integrity Operator 0.1.21:

- [RHBA-2021:4631 OpenShift File Integrity Operator Bug Fix and Enhancement Update](#)

6.2.18.1. New features and enhancements

- The metrics related to **FileIntegrity** scan results and processing metrics are displayed on the monitoring dashboard on the web console. The results are labeled with the prefix of **file_integrity_operator_**.
- If a node has an integrity failure for more than 1 second, the default **PrometheusRule** provided in the operator namespace alerts with a warning.
- The following dynamic Machine Config Operator and Cluster Version Operator related filepaths are excluded from the default AIDE policy to help prevent false positives during node updates:
 - `/etc/machine-config-daemon/currentconfig`
 - `/etc/pki/ca-trust/extracted/java/cacerts`
 - `/etc/cvo/updatepayloads`
 - `/root/.kube`
- The AIDE daemon process has stability improvements over v0.1.16, and is more resilient to errors that might occur when the AIDE database is initialized.

6.2.18.2. Bug fixes

- Before this update, when the Operator automatically upgraded, outdated daemon sets were not removed. With this release, outdated daemon sets are removed during the automatic upgrade.

6.2.19. Additional resources

- [Understanding the File Integrity Operator](#)
- [Updating the File Integrity Operator](#)

6.3. FILE INTEGRITY OPERATOR SUPPORT

The File Integrity Operator is a "Rolling Stream" Operator, meaning that updates are available asynchronously of OpenShift Container Platform releases.

6.3.1. File Integrity Operator lifecycle

For more information about Operator lifecycle policies, see Additional resources.

6.3.2. Getting support

Red Hat offers several support channels to help you troubleshoot issues and get the most from OpenShift Container Platform.

From the Red Hat Customer Portal, you can:

- Search or browse through the Red Hat Knowledgebase of articles and solutions about Red Hat products.
- Submit a support case to Red Hat Support.
- Access other product documentation.

To identify issues with your cluster, you can use Red Hat Lightspeed in [OpenShift Cluster Manager](#). Red Hat Lightspeed provides details about issues and, if available, information about how to solve a problem.

To suggest improvements or report errors, give specific details such as the section name and OpenShift Container Platform version.

6.3.3. Additional resources

- [OpenShift Container Platform Operator Life Cycles on the Red Hat Customer Portal](#)

6.4. INSTALLING THE FILE INTEGRITY OPERATOR

Install the File Integrity Operator on your cluster by using the OpenShift Container Platform web console or the OpenShift CLI (**oc**).



IMPORTANT

All cluster nodes must have the same release version in order for this Operator to function properly. As an example, for nodes running RHCOS, all nodes must have the same RHCOS version.

6.4.1. Installing the File Integrity Operator using the web console

Install the File Integrity Operator from the OpenShift Container Platform web console by using the Software Catalog.

Prerequisites

- You must have **admin** privileges.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Software Catalog**.
2. Search for the File Integrity Operator, then click **Install**.
3. Keep the default selection of **Installation mode** and **namespace** to ensure that the Operator will be installed to the **openshift-file-integrity** namespace.
4. Click **Install**.

Verification

To confirm that the installation is successful:

1. Navigate to the **Ecosystem → Installed Operators** page.
2. Check that the Operator is installed in the **openshift-file-integrity** namespace and its status is **Succeeded**.

If the Operator is not installed successfully:

1. Navigate to the **Ecosystem → Installed Operators** page and inspect the **Status** column for any errors or failures.
2. Navigate to the **Workloads → Pods** page and check the logs in any pods in the **openshift-file-integrity** project that are reporting issues.

6.4.2. Installing the File Integrity Operator using the CLI

Install the File Integrity Operator from the OpenShift CLI (**oc**) by creating **Namespace**, **OperatorGroup**, and **Subscription** objects.

Prerequisites

- You must have **admin** privileges.

Procedure

1. Create a **Namespace** object YAML file by running:

```
$ oc create -f <file_name>.yaml
```

Example output

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
    pod-security.kubernetes.io/enforce: privileged
  name: openshift-file-integrity
```



NOTE

In OpenShift Container Platform 4.21, the pod security label must be set to **privileged** at the namespace level.

2. Create the **OperatorGroup** object YAML file:

```
$ oc create -f <file-name>.yaml
```

Example output

```
■
```



```

apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: file-integrity-operator
  namespace: openshift-file-integrity
spec:
  targetNamespaces:
    - openshift-file-integrity

```

3. Create the **Subscription** object YAML file:

```
$ oc create -f <file-name>.yaml
```

Example output

```

apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: file-integrity-operator
  namespace: openshift-file-integrity
spec:
  channel: "stable"
  installPlanApproval: Automatic
  name: file-integrity-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace

```

Verification

1. Verify the installation succeeded by inspecting the CSV file:

```
$ oc get csv -n openshift-file-integrity
```

2. Verify that the File Integrity Operator is up and running:

```
$ oc get deploy -n openshift-file-integrity
```

6.4.3. Additional resources

- [Using Operator Lifecycle Manager in disconnected environments](#)

6.5. UPDATING THE FILE INTEGRITY OPERATOR

As a cluster administrator, you can update the File Integrity Operator on your OpenShift Container Platform cluster.

6.5.1. About preparing for an Operator update

You can change the update channel to start tracking and receiving updates from a newer channel to access new features and bug fixes. The subscription of an installed Operator specifies an update channel that tracks and receives updates for the Operator.

The names of update channels in a subscription can differ between Operators, but the naming scheme typically follows a common convention within a given Operator. For example, channel names might follow a minor release update stream for the application provided by the Operator (**1.2**, **1.3**) or a release frequency (**stable**, **fast**).

**NOTE**

You cannot change installed Operators to a channel that is older than the current channel.

Red Hat Customer Portal Labs include an application that helps administrators prepare to update their Operators.

You can use these tools to search for Operators and verify the available Operator versions per update channel across different releases of OpenShift Container Platform. Operators managed by Cluster Version Operator (CVO) are not included.

6.5.2. Changing the update channel for an Operator

To change the update channel for an installed Operator, you can use the OpenShift Container Platform web console. The update channel determines which Operator versions your subscription tracks and receives.

TIP

If the approval strategy in the subscription is set to **Automatic**, the update process initiates as soon as a new Operator version is available in the selected channel. If the approval strategy is set to **Manual**, you must manually approve pending updates.

Prerequisites

- An Operator previously installed using Operator Lifecycle Manager (OLM).

Procedure

1. In the web console, navigate to **Ecosystem → Installed Operators**.
2. Click the name of the Operator you want to change the update channel for.
3. Click the **Subscription** tab.
4. Click the name of the update channel under **Update channel**.
5. Click the newer update channel that you want to change to, then click **Save**.
6. For subscriptions with an **Automatic** approval strategy, the update begins automatically. Navigate back to the **Ecosystem → Installed Operators** page to monitor the progress of the update. When complete, the status changes to **Succeeded** and **Up to date**.
For subscriptions with a **Manual** approval strategy, you can manually approve the update from the **Subscription** tab.

6.5.3. Approving a pending Operator update manually

If an installed Operator has the approval strategy in its subscription set to **Manual**, you must manually approve the update before installation can begin. Manual approval reviews the changes and control when updates are applied to prevent unexpected downtime.

Prerequisites

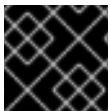
- An Operator previously installed using Operator Lifecycle Manager (OLM).

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Installed Operators**.
2. Operators that have a pending update display a status with **Upgrade available**. Click the name of the Operator you want to update.
3. Click the **Subscription** tab. Any updates requiring approval are displayed next to **Upgrade status**. For example, it might display **1 requires approval**.
4. Click **1 requires approval**, then click **Preview Install Plan**.
5. Review the resources that are listed as available for update. When satisfied, click **Approve**.
6. Navigate back to the **Ecosystem → Installed Operators** page to monitor the progress of the update. When complete, the status changes to **Succeeded** and **Up to date**.

6.6. UNDERSTANDING THE FILE INTEGRITY OPERATOR

The File Integrity Operator is an OpenShift Container Platform Operator that continually runs file integrity checks on the cluster nodes. It deploys a daemon set that initializes and runs privileged advanced intrusion detection environment (AIDE) containers on each node, providing a status object with a log of files that are modified during the initial run of the daemon set pods.



IMPORTANT

Currently, only Red Hat Enterprise Linux CoreOS (RHCOS) nodes are supported.

6.6.1. Creating the FileIntegrity custom resource

An instance of a **FileIntegrity** custom resource (CR) represents a set of continuous file integrity scans for one or more nodes.

Each **FileIntegrity** CR is backed by a daemon set running AIDE on the nodes matching the **FileIntegrity** CR specification.



NOTE

For all-in-one control plane and worker nodes, separate **FileIntegrity** CRs that use **node-role.kubernetes.io/master** and **node-role.kubernetes.io/worker** selectors can schedule many daemon sets that run Advanced Intrusion Detection Environment (AIDE) on the same nodes, because schedulable control plane nodes often have both labels. Redundant scans waste resources and can complicate file integrity monitoring. You can avoid this by using a single **FileIntegrity** CR whose **nodeSelector** targets each node only once for your cluster layout.

Procedure

1. Create the following example **FileIntegrity** CR named **worker-fileintegrity.yaml** to enable scans on worker nodes:

```
apiVersion: fileintegrity.openshift.io/v1alpha1
kind: FileIntegrity
metadata:
  name: worker-fileintegrity
  namespace: openshift-file-integrity
spec:
  nodeSelector:
    node-role.kubernetes.io/worker: ""
  tolerations:
    key: "myNode"
    operator: "Exists"
    effect: "NoSchedule"
  config:
    name: "myconfig"
    namespace: "openshift-file-integrity"
    key: "config"
    gracePeriod: 20
    maxBackups: 5
    initialDelay: 60
  debug: false
status:
  phase: Active
```

spec.nodeSelector	Specifies the selector for scheduling node scans.
spec.tolerations	Specify tolerations to schedule on nodes with custom taints. When not specified, a default toleration allowing running on main and infra nodes is applied.
spec.config	Specify a ConfigMap containing an AIDE configuration to use.
spec.config.gracePeriod	The number of seconds to pause in between AIDE integrity checks. Frequent AIDE checks on a node might be resource intensive, so it can be useful to specify a longer interval. Default is 900 seconds (15 minutes).
spec.config.maxBackups	The maximum number of AIDE database and log backups (leftover from the re-init process) to keep on a node. Older backups beyond this number are automatically pruned by the daemon. Default is set to 5.
spec.config.initialDelay	The number of seconds to wait before starting the first AIDE integrity check. Default is set to 0.
status.phase	The running status of the FileIntegrity instance. Statuses are Initializing , Pending , or Active .

Initializing	The FileIntegrity object is currently initializing or re-initializing the AIDE database.
Pending	The FileIntegrity deployment is still being created.
Active	The scans are active and ongoing.

2. Apply the YAML file to the **openshift-file-integrity** namespace:

```
$ oc apply -f worker-fileintegrity.yaml -n openshift-file-integrity
```

Verification

- Confirm the **FileIntegrity** object was created successfully by running the following command:

```
$ oc get fileintegrities -n openshift-file-integrity
```

Example output

```
NAME          AGE
worker-fileintegrity 14s
```

6.6.2. Checking the FileIntegrity custom resource status

The **FileIntegrity** custom resource (CR) reports its status through the **.status.phase** subresource.

Procedure

- To query the **FileIntegrity** CR status, run:

```
$ oc get fileintegrities/worker-fileintegrity -o jsonpath="{.status.phase}"
```

Example output

```
Active
```

6.6.3. FileIntegrity custom resource phases

The **FileIntegrity** CR reports one of the following phases during its lifecycle.

- **Pending** - The phase after the custom resource (CR) is created.
- **Active** - The phase when the backing daemon set is up and running.
- **Initializing** - The phase when the AIDE database is being reinitialized.

6.6.4. Understanding the FileIntegrityNodeStatuses object

The scan results of the **FileIntegrity** CR are reported in another object called **FileIntegrityNodeStatuses**.

```
$ oc get fileintegritynodestatuses
```

Example output

NAME	AGE
worker-fileintegrity-ip-10-0-130-192.ec2.internal	101s
worker-fileintegrity-ip-10-0-147-133.ec2.internal	109s
worker-fileintegrity-ip-10-0-165-160.ec2.internal	102s



NOTE

It might take some time for the **FileIntegrityNodeStatus** object results to be available.

There is one result object per node. The **nodeName** attribute of each **FileIntegrityNodeStatus** object corresponds to the node being scanned. The status of the file integrity scan is represented in the **results** array, which holds scan conditions.

```
$ oc get fileintegritynodestatuses.fileintegrity.openshift.io -ojsonpath='{.items[*].results}' | jq
```

The **fileintegritynodestatus** object reports the latest status of an AIDE run and exposes the status as **Failed**, **Succeeded**, or **Errored** in a **status** field.

```
$ oc get fileintegritynodestatuses -w
```

Example output

NAME	NODE	STATUS
example-fileintegrity-ip-10-0-134-186.us-east-2.compute.internal	ip-10-0-134-186.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-150-230.us-east-2.compute.internal	ip-10-0-150-230.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-169-137.us-east-2.compute.internal	ip-10-0-169-137.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-180-200.us-east-2.compute.internal	ip-10-0-180-200.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-194-66.us-east-2.compute.internal	ip-10-0-194-66.us-east-2.compute.internal	Failed
example-fileintegrity-ip-10-0-222-188.us-east-2.compute.internal	ip-10-0-222-188.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-134-186.us-east-2.compute.internal	ip-10-0-134-186.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-222-188.us-east-2.compute.internal	ip-10-0-222-188.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-194-66.us-east-2.compute.internal	ip-10-0-194-66.us-east-2.compute.internal	Failed
example-fileintegrity-ip-10-0-150-230.us-east-2.compute.internal	ip-10-0-150-230.us-east-2.compute.internal	Succeeded
example-fileintegrity-ip-10-0-180-200.us-east-2.compute.internal	ip-10-0-180-200.us-east-2.compute.internal	Succeeded

6.6.5. FileIntegrityNodeStatus CR status types

These conditions are reported in the results array of the corresponding **FileIntegrityNodeStatus** CR status.

- **Succeeded** - The integrity check passed; the files and directories covered by the AIDE check have not been modified since the database was last initialized.
- **Failed** - The integrity check failed; some files or directories covered by the AIDE check have been modified since the database was last initialized.
- **Errored** - The AIDE scanner encountered an internal error.

6.6.5.1. FileIntegrityNodeStatus CR success example

The following example shows a **FileIntegrityNodeStatus** CR with successful scan conditions.

Example output of a condition with a success status

```
[
  {
    "condition": "Succeeded",
    "lastProbeTime": "2020-09-15T12:45:57Z"
  }
]
[
  {
    "condition": "Succeeded",
    "lastProbeTime": "2020-09-15T12:46:03Z"
  }
]
[
  {
    "condition": "Succeeded",
    "lastProbeTime": "2020-09-15T12:45:48Z"
  }
]
```

In this case, all three scans succeeded and so far there are no other conditions.

6.6.5.2. FileIntegrityNodeStatus CR failure status example

To simulate a failure condition, modify one of the files AIDE tracks. For example, modify **/etc/resolv.conf** on one of the worker nodes:

```
$ oc debug node/ip-10-0-130-192.ec2.internal
```

Example output

```
Creating debug namespace/openshift-debug-node-ldfbj ...
Starting pod/ip-10-0-130-192ec2internal-debug ...
To use host binaries, run `chroot /host`
Pod IP: 10.0.130.192
If you don't see a command prompt, try pressing enter.
sh-4.2# echo "# integrity test" >> /host/etc/resolv.conf
sh-4.2# exit
```

```
Removing debug pod ...
Removing debug namespace/openshift-debug-node-ldfbj ...
```

After some time, the **Failed** condition is reported in the results array of the corresponding **FileIntegrityNodeStatus** object. The previous **Succeeded** condition is retained, which allows you to pinpoint the time the check failed.

```
$ oc get fileintegritynodestatuses.fileintegrity.openshift.io/worker-fileintegrity-ip-10-0-130-192.ec2.internal -ojsonpath='{.results}' | jq -r
```

Alternatively, if you are not mentioning the object name, run:

```
$ oc get fileintegritynodestatuses.fileintegrity.openshift.io -ojsonpath='{.items[*].results}' | jq
```

Example output

```
[
  {
    "condition": "Succeeded",
    "lastProbeTime": "2020-09-15T12:54:14Z"
  },
  {
    "condition": "Failed",
    "filesChanged": 1,
    "lastProbeTime": "2020-09-15T12:57:20Z",
    "resultConfigMapName": "aide-ds-worker-fileintegrity-ip-10-0-130-192.ec2.internal-failed",
    "resultConfigMapNamespace": "openshift-file-integrity"
  }
]
```

The **Failed** condition points to a config map that gives more details about what exactly failed and why:

```
$ oc describe cm aide-ds-worker-fileintegrity-ip-10-0-130-192.ec2.internal-failed
```

Example output

```
Name:      aide-ds-worker-fileintegrity-ip-10-0-130-192.ec2.internal-failed
Namespace: openshift-file-integrity
Labels:    file-integrity.openshift.io/node=ip-10-0-130-192.ec2.internal
           file-integrity.openshift.io/owner=worker-fileintegrity
           file-integrity.openshift.io/result-log=
Annotations: file-integrity.openshift.io/files-added: 0
             file-integrity.openshift.io/files-changed: 1
             file-integrity.openshift.io/files-removed: 0

Data

integritylog:
-----
AIDE 0.15.1 found differences between database and filesystem!!
Start timestamp: 2020-09-15 12:58:15
```


Summary:

Total number of files: 31553

Added files: 0

Removed files: 0

Changed files: 1

Changed files:

changed: /hostroot/etc/resolv.conf

Detailed information about changes:

File: /hostroot/etc/resolv.conf

SHA512 : sTQYpB/AL7FeoGtu/1g7opv6C+KT1CBJ , qAeM+a8yTgHPnIHMaRIS+so61EN8VOpg

Events: <none>

Due to the config map data size limit, AIDE logs over 1 MB are added to the failure config map as a base64-encoded gzip archive. Use the following command to extract the log:

```
$ oc get cm <failure-cm-name> -o json | jq -r '.data.integritylog' | base64 -d | gunzip
```

**NOTE**

Compressed logs are indicated by the presence of a **file-integrity.openshift.io/compressed** annotation key in the config map.

6.6.6. Understanding events

Transitions in the status of the **FileIntegrity** and **FileIntegrityNodeStatus** objects are logged by *events*. The creation time of the event reflects the latest transition, such as **Initializing** to **Active**, and not necessarily the latest scan result. However, the newest event always reflects the most recent status.

```
$ oc get events --field-selector reason=FileIntegrityStatus
```

Example output

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
97s	Normal	FileIntegrityStatus	fileintegrity/example-fileintegrity	Pending
67s	Normal	FileIntegrityStatus	fileintegrity/example-fileintegrity	Initializing
37s	Normal	FileIntegrityStatus	fileintegrity/example-fileintegrity	Active

When a node scan fails, an event is created with the **add/changed/removed** and config map information.

```
$ oc get events --field-selector reason=NodeIntegrityStatus
```

Example output

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-134-173.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-168-238.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-169-175.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-152-92.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-158-144.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-131-30.ec2.internal
87m	Warning	NodeIntegrityStatus	fileintegrity/example-fileintegrity	node ip-10-0-152-92.ec2.internal has changed! a:1,c:1,r:0 \ log:openshift-file-integrity/aide-ds-example-fileintegrity-ip-10-0-152-92.ec2.internal-failed

Changes to the number of added, changed, or removed files results in a new event, even if the status of the node has not transitioned.

```
$ oc get events --field-selector reason=NodeIntegrityStatus
```

Example output

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-134-173.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-168-238.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-169-175.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-152-92.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-158-144.ec2.internal
114m	Normal	NodeIntegrityStatus	fileintegrity/example-fileintegrity	no changes to node ip-10-0-131-30.ec2.internal
87m	Warning	NodeIntegrityStatus	fileintegrity/example-fileintegrity	node ip-10-0-152-92.ec2.internal has changed! a:1,c:1,r:0 \ log:openshift-file-integrity/aide-ds-example-fileintegrity-ip-10-0-152-92.ec2.internal-failed
40m	Warning	NodeIntegrityStatus	fileintegrity/example-fileintegrity	node ip-10-0-152-92.ec2.internal has changed! a:3,c:1,r:0 \ log:openshift-file-integrity/aide-ds-example-fileintegrity-ip-10-0-152-92.ec2.internal-failed

6.7. CONFIGURING THE CUSTOM FILE INTEGRITY OPERATOR

You can configure the Custom File Integrity Operator to meet your cluster requirements.

6.7.1. Viewing FileIntegrity object attributes

As with any Kubernetes custom resources (CRs), you can run **oc explain fileintegrity**, and then examine the individual attributes.

Procedure

- View the **FileIntegrity** spec attributes by running the following command:

```
$ oc explain fileintegrity.spec
```

- View the **FileIntegrity** config attributes by running the following command:

```
$ oc explain fileintegrity.spec.config
```

6.7.2. Important attributes

The following **spec** and **spec.config** attributes are important when configuring a **FileIntegrity** CR.

Table 6.1. Important **spec** and **spec.config** attributes

Attribute	Description
spec.nodeSelector	Specifies a map of key-value pairs that labels for a node must match for a cluster to schedule Advanced Intrusion Detection Environment (AIDE) pods on that node. Typically, you can configure only a single key-value pair. For example, node-role.kubernetes.io/worker: "" schedules AIDE on all compute nodes, while node.openshift.io/os_id: "rhel" schedules AIDE on all RHEL nodes.
spec.debug	A boolean attribute. If set to true , the daemon running in the AIDE daemon set pods would output extra information.
spec.tolerations	Specify tolerations to schedule on nodes with custom taints. When not specified, a default toleration is applied, which allows tolerations to run on control plane nodes.
spec.config.gracePeriod	The number of seconds to pause in between AIDE integrity checks. Frequent AIDE checks on a node can be resource intensive, so it can be useful to specify a longer interval. Defaults to 900 , or 15 minutes.
maxBackups	The maximum number of AIDE database and log backups leftover from the re-init process to keep on a node. Older backups beyond this number are automatically pruned by the daemon.

Attribute	Description
spec.config.name	Name of a configMap that contains custom AIDE configuration. If omitted, a default configuration is created.
spec.config.namespace	Namespace of a configMap that contains custom AIDE configuration. If unset, the FIO generates a default configuration suitable for RHCOS systems.
spec.config.key	Key that contains actual AIDE configuration in a config map specified by name and namespace . The default value is aide.conf .
spec.config.initialDelay	The number of seconds to wait before starting the first AIDE integrity check. Default is set to 0. This attribute is optional.

6.7.3. Examine the default configuration

The default File Integrity Operator configuration is stored in a config map with the same name as the **FileIntegrity** CR.

Procedure

- To examine the default config, run:

```
$ oc describe cm/worker-fileintegrity
```

6.7.4. Understanding the default File Integrity Operator configuration

The default configuration for a **FileIntegrity** instance provides coverage for files under key system directories and excludes others.

Below is an excerpt from the **aide.conf** key of the config map:

```
@@define DBDIR /hostroot/etc/kubernetes
@@define LOGDIR /hostroot/etc/kubernetes
database=file:@@{DBDIR}/aide.db.gz
database_out=file:@@{DBDIR}/aide.db.gz
gzip_dbout=yes
verbose=5
report_url=file:@@{LOGDIR}/aide.log
report_url=stdout
PERMS = p+u+g+acl+selinux+xattrs
CONTENT_EX = sha512+ftype+p+u+g+n+acl+selinux+xattrs

/hostroot/boot/  CONTENT_EX
/hostroot/root/\.* PERMS
/hostroot/root/  CONTENT_EX
```

The default configuration for a **FileIntegrity** instance provides coverage for files under the following directories:

- **/root**
- **/boot**
- **/usr**
- **/etc**

The following directories are not covered:

- **/var**
- **/opt**
- Some OpenShift Container Platform-specific excludes under **/etc/**

6.7.5. Supplying a custom AIDE configuration

Any entries that configure AIDE internal behavior such as **DBDIR**, **LOGDIR**, **database**, and **database_out** are overwritten by the Operator. The Operator adds a prefix to **/hostroot/** before all paths to be watched for integrity changes. As a result, you can reuse existing AIDE configs that might not be tailored for a containerized environment and that start from the root directory.



NOTE

/hostroot is the directory where the pods running AIDE mount the host file system. Changing the configuration triggers a reinitializing of the database.

6.7.6. Defining a custom File Integrity Operator configuration

This example focuses on defining a custom configuration for a scanner that runs on the control plane nodes based on the default configuration provided for the **worker-fileintegrity** CR. This workflow might be useful if you are planning to deploy a custom software running as a daemon set and storing its data under **/opt/mydaemon** on the control plane nodes.

Procedure

1. Make a copy of the default configuration.
2. Edit the default configuration with the files that must be watched or excluded.
3. Store the edited contents in a new config map.
4. Point the **FileIntegrity** object to the new config map through the attributes in **spec.config**.
5. Extract the default configuration:

```
$ oc extract cm/worker-fileintegrity --keys=aide.conf
```

This creates a file named **aide.conf** that you can edit. To illustrate how the Operator post-processes the paths, this example adds an exclude directory without the prefix:

```
$ vim aide.conf
```

Example output

```
/hostroot/etc/kubernetes/static-pod-resources
!/hostroot/etc/kubernetes/aide.*
!/hostroot/etc/kubernetes/manifests
!/hostroot/etc/docker/certs.d
!/hostroot/etc/selinux/targeted
!/hostroot/etc/openswitch/conf.db
```

Exclude a path specific to control plane nodes:

```
!/opt/mydaemon/
```

Store the other content in **/etc**:

```
/hostroot/etc/ CONTENT_EX
```

6. Create a config map based on this file:

```
$ oc create cm master-aide-conf --from-file=aide.conf
```

7. Define a **FileIntegrity** CR manifest that references the config map:

```
apiVersion: fileintegrity.openshift.io/v1alpha1
kind: FileIntegrity
metadata:
  name: master-fileintegrity
  namespace: openshift-file-integrity
spec:
  nodeSelector:
    node-role.kubernetes.io/master: ""
  config:
    name: master-aide-conf
    namespace: openshift-file-integrity
```

The Operator processes the provided config map file and stores the result in a config map with the same name as the **FileIntegrity** object:

```
$ oc describe cm/master-fileintegrity | grep /opt/mydaemon
```

Example output

```
!/hostroot/opt/mydaemon
```

6.7.7. Changing the custom File Integrity configuration

To change the File Integrity configuration, never change the generated config map. Instead, change the config map that is linked to the **FileIntegrity** object through the **spec.name**, **namespace**, and **key** attributes.

Procedure

- Update the config map referenced by the **spec.config** attributes of the **FileIntegrity** object.

6.8. PERFORMING ADVANCED CUSTOM FILE INTEGRITY OPERATOR TASKS

This document describes advanced tasks for the Custom File Integrity Operator.

6.8.1. Reinitializing the database

If the File Integrity Operator detects a change that was planned, it might be required to reinitialize the database.

Procedure

- Annotate the **FileIntegrity** custom resource (CR) with **file-integrity.openshift.io/re-init**:

```
$ oc annotate fileintegrities/worker-fileintegrity file-integrity.openshift.io/re-init=
```

The old database and log files are backed up and a new database is initialized. The old database and logs are retained on the nodes under **/etc/kubernetes**, as seen in the following output from a pod spawned using **oc debug**:

Example output

```
ls -lR /host/etc/kubernetes/aide.*
-rw-----. 1 root root 1839782 Sep 17 15:08 /host/etc/kubernetes/aide.db.gz
-rw-----. 1 root root 1839783 Sep 17 14:30 /host/etc/kubernetes/aide.db.gz.backup-
20200917T15_07_38
-rw-----. 1 root root 73728 Sep 17 15:07 /host/etc/kubernetes/aide.db.gz.backup-
20200917T15_07_55
-rw-r--r--. 1 root root 0 Sep 17 15:08 /host/etc/kubernetes/aide.log
-rw-----. 1 root root 613 Sep 17 15:07 /host/etc/kubernetes/aide.log.backup-
20200917T15_07_38
-rw-r--r--. 1 root root 0 Sep 17 15:07 /host/etc/kubernetes/aide.log.backup-
20200917T15_07_55
```

To provide some permanence of record, the resulting config maps are not owned by the **FileIntegrity** object, so manual cleanup is necessary. As a result, any previous integrity failures would still be visible in the **FileIntegrityNodeStatus** object.

6.8.2. Machine config integration

In OpenShift Container Platform 4.21, the cluster node configuration is delivered through **MachineConfig** objects. You can assume that the changes to files that are caused by a **MachineConfig** object are expected and should not cause the file integrity scan to fail.

To suppress changes to files caused by **MachineConfig** object updates, the File Integrity Operator watches the node objects; when a node is being updated, the AIDE scans are suspended for the duration of the update. When the update finishes, the database is reinitialized and the scans resume.

This pause and resume logic only applies to updates through the **MachineConfig** API, as they are reflected in the node object annotations.

6.8.3. Exploring the daemon sets

Each **FileIntegrity** object represents a scan on several nodes. The scan itself is performed by pods managed by a daemon set. The config maps created by the AIDE daemon are not retained and are deleted after the File Integrity Operator processes them. However, on failure and error, the contents of these config maps are copied to the config map that the **FileIntegrityNodeStatus** object points to.

Procedure

1. To find the daemon set that represents a **FileIntegrity** object, run:

```
$ oc -n openshift-file-integrity get ds/aide-worker-fileintegrity
```

2. To list the pods in that daemon set, run:

```
$ oc -n openshift-file-integrity get pods -lapp=aide-worker-fileintegrity
```

3. To view logs of a single AIDE pod, call **oc logs** on one of the pods:

```
$ oc -n openshift-file-integrity logs pod/aide-worker-fileintegrity-mr8x6
```

Example output

```
Starting the AIDE runner daemon
initializing AIDE db
initialization finished
running aide check
...
```

6.9. TROUBLESHOOTING THE FILE INTEGRITY OPERATOR

Use the following information to troubleshoot common issues with the File Integrity Operator.

6.9.1. General troubleshooting

Issue

You want to generally troubleshoot issues with the File Integrity Operator.

Resolution

Enable the debug flag in the **FileIntegrity** object. The **debug** flag increases the verbosity of the daemons that run in the **DaemonSet** pods and run the AIDE checks.

6.9.2. Checking the AIDE configuration

Issue

You want to check the AIDE configuration.

Resolution

The AIDE configuration is stored in a config map with the same name as the **FileIntegrity** object. All AIDE configuration config maps are labeled with **file-integrity.openshift.io/aide-conf**.

6.9.3. Determining the FileIntegrity object phase

Issue

You want to determine if the **FileIntegrity** object exists and see its current status.

Resolution

To see the current status of the **FileIntegrity** object, run:

```
$ oc get fileintegrities/worker-fileintegrity -o jsonpath="{ .status }"
```

After the **FileIntegrity** object and the backing daemon set are created, the status should switch to **Active**. If it does not, check the Operator pod logs.

6.9.4. Determining that the daemon set pods are running on the expected nodes

Issue

You want to confirm that the daemon set exists and that its pods are running on the nodes you expect them to run on.

Resolution

Run:

```
$ oc -n openshift-file-integrity get pods -lapp=aide-worker-fileintegrity
```



NOTE

Adding **-owide** includes the IP address of the node that the pod is running on.

To check the logs of the daemon pods, run **oc logs**.

Check the return value of the AIDE command to see if the check passed or failed.

6.10. UNINSTALLING THE FILE INTEGRITY OPERATOR

You can remove the File Integrity Operator from your cluster by using the OpenShift Container Platform web console.

6.10.1. Uninstall the File Integrity Operator using the web console

To remove the File Integrity Operator, you must first delete the **FileIntegrity** objects in all namespaces. After the objects are removed, you can then remove the Operator and its namespace.

Prerequisites

- You have access to an OpenShift Container Platform cluster that uses an account with **cluster-admin** permissions.

- The File Integrity Operator is installed.

Procedure

1. Navigate to the **Ecosystem** → **Installed Operators** → **File Integrity Operator** page.
2. From the **File Integrity** tab, ensure the **Show operands in: All namespaces** default option is selected to list all **FileIntegrity** objects in all namespaces.

3. Click the Options menu  for a **FileIntegrity** object.

4. Select **Delete FileIntegrity**.

5. Repeat the previous two steps until no **FileIntegrity** objects remain.

6. Go to the **Administration** → **Ecosystem** → **Installed Operators** page.

7. Click the Options menu  on the **File Integrity Operator** entry.

8. Select **Uninstall Operator**.

9. Go to the **Home** → **Projects** page.

10. Search for **openshift-file-integrity**.

11. Click the Options menu  for the **openshift-file-integrity** project entry.

12. Select **Delete Project**. A **Delete Project** dialog box opens on the web console.

Verification

- Type **openshift-file-integrity** in the **Delete Project** dialog box.
- Click the **Delete** button.

CHAPTER 7. SECURITY PROFILES OPERATOR

7.1. SECURITY PROFILES OPERATOR OVERVIEW

With the OpenShift Container Platform Security Profiles Operator (SPO), you can define seccomp and SELinux profiles as custom resources and keep them synchronized across every node in a namespace.

The SPO distributes seccomp and SELinux profile custom resources to each node and keeps them up to date when profiles change. You can also bind policies to pods and record workloads. See [Additional resources](#) for advanced tasks such as enabling the log enricher, configuring webhooks and metrics, or restricting profiles to a single namespace, and for advanced audit logging that correlates cluster users with actions during **oc exec**, **oc rsh**, and **oc debug** sessions.

7.1.1. Additional resources

- [Security Profiles Operator release notes](#)
- [Security Profiles Operator support](#)
- [Understanding the Security Profiles Operator](#)
- [Enabling the Security Profiles Operator](#)
- [Managing seccomp profiles](#)
- [Managing SELinux profiles](#)
- [Advanced Security Profiles Operator tasks](#)
- [Advanced Audit Logging Framework](#)
- [Troubleshooting the Security Profiles Operator](#)
- [Uninstalling the Security Profiles Operator](#)
- [seccomp](#)

7.2. RELEASE NOTES FOR THE SECURITY PROFILES OPERATOR

The Security Profiles Operator provides a way to define secure computing (seccomp) and SELinux profiles as custom resources, synchronizing profiles to every node in a given namespace.

These release notes track the development of the Security Profiles Operator in OpenShift Container Platform.

7.2.1. Release notes for Security Profiles Operator 0.10.0

Release notes for Security Profiles Operator 0.10.0.

The following Red Hat Security Advisory (RHSA) is available for the Security Profiles Operator 0.10.0: [RHSA-2026:2852 - OpenShift Security Profiles Operator update](#)

7.2.1.1. Bug fixes

- In some instances, using Security Profiles Operator (SPO) 0.9.0 with OpenShift Container Platform version 4.20 and above caused SPO to create the **profilerecording** resource but the workload would fail. Failure of the workload prevented the creation of the needed container for running the Operator. With the 0.10.0 release of SPO, the **profilerecording** resource is reliably created, therefore the needed container for running the Operator is reliably created ([CMP-3537](#)).
- For version 0.9.0 of Security Profiles Operator (SPO), the **spod** pods would fail to run with the error message **fsmount:fscontext:proc/: could not get mount id: operation not permitted**. With the release of version 0.10.0, the **spod** pods run reliably. [CMP-4007](#).
- In releases of SPO 0.9.0 and earlier, there was a bug in syntax of the **selinux** usage. With this release of SPO, the change is from **<policyName>_process** to **<policyName>.process**. The new syntax omits the **_**. Examples in the documentation now show this updated usage. [CMP-4104](#)

7.2.1.2. New features and enhancements

- With the release of SPO v0.10.0, the Operator now supports Red Hat Enterprise Linux CoreOS (RHCOS) 10 containers. [CMP-4033](#)
- In this release of the Security Profiles Operator, the Advanced Audit Logging Framework is available as a General Availability (GA) feature. The Advanced Audit Logging Framework uses the Audit JSON Log Enricher to capture and log terminal-based command activity in Red Hat Enterprise Linux CoreOS (RHCOS) containers, including **oc rsh**, **oc exec**, and **oc debug** commands. For more details, see [Advanced Audit Logging Framework](#).

7.2.2. Release notes for Security Profiles Operator 0.9.0

Release notes for Security Profiles Operator 0.9.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.9.0: [RHBA-2025:15655 - OpenShift Security Profiles Operator update](#)

This update manages security profiles as cluster-wide resources rather than namespace resources. To update Security Profiles Operator to a version later than 0.8.6 requires manual migration. For migration instructions, see [Security Profiles Operator 0.9.0 Update Migration Guide](#).

7.2.2.1. Bug fixes

- Before this update, the **spod** pods could fail to start and enter into a **CrashLoopBackOff** state due to an error in parsing the semanage configuration file. A change to the RHEL 9 image naming convention beginning in OpenShift Container Platform 4.19 causes this issue. ([OCPBUGS-55829](#))
- Before this update, the Security Profiles Operator would fail to apply a **RawSelinuxProfile** to newly added nodes due to a reconciler type mismatch error. With this update, the Operator now correctly handles **RawSelinuxProfile** objects and applies policies to all nodes as expected. ([OCPBUGS-33718](#))

7.2.3. Release notes for Security Profiles Operator 0.8.6

Release notes for Security Profiles Operator 0.8.6.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.8.6:

- [RHBA-2024:10380 - OpenShift Security Profiles Operator update](#)

This update includes upgraded dependencies in underlying base images.

7.2.4. Release notes for Security Profiles Operator 0.8.5

Release notes for Security Profiles Operator 0.8.5.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.8.5:

- [RHBA-2024:5016 - OpenShift Security Profiles Operator bug fix update](#)

7.2.4.1. Bug fixes

- When attempting to install the Security Profiles Operator from the web console, the option to enable Operator-recommended cluster monitoring was unavailable for the namespace. With this update, you can now enable Operator-recommended cluster monitoring in the namespace. ([OCPBUGS-37794](#))
- Before this update, the Security Profiles Operator would intermittently be not visible in the OperatorHub, which caused limited access to install the Operator through the web console. With this update, the Security Profiles Operator is present in the OperatorHub.

7.2.5. Release notes for Security Profiles Operator 0.8.4

Release notes for Security Profiles Operator 0.8.4.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.8.4:

- [RHBA-2024:4781 - OpenShift Security Profiles Operator bug fix update](#)

This update addresses CVEs in underlying dependencies.

7.2.5.1. New features and enhancements

- You can now specify a default security profile in the **image** attribute of a **ProfileBinding** object by setting a wildcard. For more information, see [Binding workloads to profiles with ProfileBinding objects \(SELinux\)](#) and [Binding workloads to profiles with ProfileBinding objects \(Seccomp\)](#).

7.2.6. Release notes for Security Profiles Operator 0.8.2

Release notes for Security Profiles Operator 0.8.2.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.8.2:

- [RHBA-2023:5958 - OpenShift Security Profiles Operator bug fix update](#)

7.2.6.1. Bug fixes

- Before this update, **SELinuxProfile** objects did not inherit custom attributes from the same namespace. With this update, **SELinuxProfile** object attributes inherit from the same namespace as expected. ([OCPBUGS-17164](#))

- Before this update, **RawSELinuxProfile** objects would hang during the creation process and would not reach an **Installed** state. With this update, the operator creates **RawSELinuxProfile** objects successfully. ([OCPBUGS-19744](#))
- Before this update, patching the **enableLogEnricher** to **true** would cause the **seccompProfile log-enricher-trace** pods to remain in a **Pending** state. With this update, **log-enricher-trace** pods reach an **Installed** state as expected. ([OCPBUGS-22182](#))
- Before this update, the Security Profiles Operator generated high cardinality metrics, causing Prometheus pods using high amounts of memory. With this update, the following metrics will no longer apply in the Security Profiles Operator namespace:
 - **rest_client_request_duration_seconds**
 - **rest_client_request_size_bytes**
 - **rest_client_response_size_bytes**
([OCPBUGS-22406](#))

7.2.7. Release notes for Security Profiles Operator 0.8.0

Release notes for Security Profiles Operator 0.8.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.8.0:

- [RHBA-2023:4689 - OpenShift Security Profiles Operator bug fix update](#)

7.2.7.1. Bug fixes

- Before this update, while trying to install Security Profiles Operator in a disconnected cluster, the secure hash algorithms (SHAs) provided were wrong due to an SHA relabeling issue. With this update, the secure hash algorithms work consistently with disconnected environments. ([OCPBUGS-14404](#))

7.2.8. Release notes for Security Profiles Operator 0.7.1

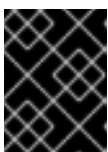
Release notes for Security Profiles Operator 0.7.1.

The following Red Hat Security Advisory (RHSA) is available for the Security Profiles Operator 0.7.1:

- [RHSA-2023:2029 - OpenShift Security Profiles Operator bug fix update](#)

7.2.8.1. New features and enhancements

- Security Profiles Operator (SPO) now automatically selects the appropriate **selinuxd** image for RHEL 8- and 9-based Red Hat Enterprise Linux CoreOS (RHCOS) systems.



IMPORTANT

Users that mirror images for disconnected environments must mirror both **selinuxd** images provided by the Security Profiles Operator.

- You can now enable memory optimization inside of an **spod** daemon. For more information, see [Enabling memory optimization in the spod daemon](#).

**NOTE**

SPO memory optimization is not enabled by default.

- The daemon resource requirements are now configurable. For more information, see [Customizing daemon resource requirements](#).
- The priority class name is now configurable in the **spod** configuration. For more information, see [Setting a custom priority class name for the spod daemon pod](#).

7.2.8.2. Deprecated and removed features

- The default **nginx-1.19.1** seccomp profile is now removed from the Security Profiles Operator deployment.

7.2.8.3. Bug fixes

- Before this update, a Security Profiles Operator (SPO) SELinux policy did not inherit low-level policy definitions from the container template. If you selected another template, such as **net_container**, the policy would not work because it required low-level policy definitions that only existed in the container template. This issue occurred when the SPO SELinux policy attempted to translate SELinux policies from the SPO custom format to the Common Intermediate Language (CIL) format. With this update, the container template appends to any SELinux policies that require translation from SPO to CIL. Additionally, the SPO SELinux policy can inherit low-level policy definitions from any supported policy template. ([OCPBUGS-12879](#))

7.2.8.4. Known issue

- When uninstalling the Security Profiles Operator, the **MutatingWebhookConfiguration** object is not deleted and must be manually removed. As a workaround, delete the **MutatingWebhookConfiguration** object after uninstalling the Security Profiles Operator. For these steps, see [Uninstalling the Security Profiles Operator](#). ([OCPBUGS-4687](#))

7.2.9. Release notes for Security Profiles Operator 0.5.2

Release notes for Security Profiles Operator 0.5.2.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.5.2:

- [RHBA-2023:0788 - OpenShift Security Profiles Operator bug fix update](#)

This update addresses a CVE in an underlying dependency.

7.2.9.1. Known issue

- When uninstalling the Security Profiles Operator, the **MutatingWebhookConfiguration** object is not deleted and must be manually removed. As a workaround, delete the **MutatingWebhookConfiguration** object after uninstalling the Security Profiles Operator. For these steps, see [Uninstalling the Security Profiles Operator](#). ([OCPBUGS-4687](#))

7.2.10. Release notes for Security Profiles Operator 0.5.0

Release notes for Security Profiles Operator 0.5.0.

The following Red Hat Bug Fix Advisory (RHBA) is available for the Security Profiles Operator 0.5.0:

- [RHBA-2022:8762 - OpenShift Security Profiles Operator bug fix update](#)

7.2.10.1. Known issue

- When uninstalling the Security Profiles Operator, the **MutatingWebhookConfiguration** object is not deleted and must be manually removed. As a workaround, delete the **MutatingWebhookConfiguration** object after uninstalling the Security Profiles Operator. For these steps, see [Uninstalling the Security Profiles Operator](#). (OCPBUGS-4687)

7.2.11. Additional resources

- [Security Profiles Operator Overview \(Kubernetes documentation\)](#)
- [Kubernetes seccomp tutorial](#)

7.3. SECURITY PROFILES OPERATOR SUPPORT

The Security Profiles Operator is a "Rolling Stream" Operator, meaning updates are available asynchronously of OpenShift Container Platform releases.

7.3.1. Getting support

Red Hat offers several support channels to help you troubleshoot issues and get the most from OpenShift Container Platform.

From the Red Hat Customer Portal, you can:

- Search or browse through the Red Hat Knowledgebase of articles and solutions about Red Hat products.
- Submit a support case to Red Hat Support.
- Access other product documentation.

To identify issues with your cluster, you can use Red Hat Lightspeed in [OpenShift Cluster Manager](#). Red Hat Lightspeed provides details about issues and, if available, information about how to solve a problem.

To suggest improvements or report errors, give specific details such as the section name and OpenShift Container Platform version.

7.3.2. Additional resources

- [OpenShift Operator Life Cycles on the Red Hat Customer Portal](#)

7.4. UNDERSTANDING THE SECURITY PROFILES OPERATOR

OpenShift Container Platform administrators can use the Security Profiles Operator to define increased security measures in clusters.

**IMPORTANT**

The Security Profiles Operator supports only Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes. Red Hat Enterprise Linux (RHEL) nodes are not supported.

7.4.1. About security profiles

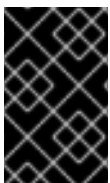
Security profiles limit what containers can do on a node, so you can reduce the attack surface of workloads in your cluster.

Seccomp security profiles list the syscalls a process can make. Permissions are broader than SELinux, enabling users to restrict operations system-wide, such as **write**.

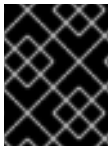
SELinux security profiles provide a label-based system that restricts the access and usage of processes, applications, or files in a system. All files in an environment have labels that define permissions. SELinux profiles can define access within a given structure, such as directories.

7.5. ENABLING THE SECURITY PROFILES OPERATOR

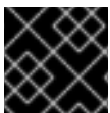
Before you can use the Security Profiles Operator, you must ensure the Operator is deployed in the cluster.

**IMPORTANT**

All cluster nodes must have the same release version in order for this Operator to function properly. As an example, for nodes running RHCOS, all nodes must have the same RHCOS version.

**IMPORTANT**

The Security Profiles Operator supports only Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes. Red Hat Enterprise Linux (RHEL) nodes are not supported.

**IMPORTANT**

The Security Profiles Operator supports **x86_64** and **ppc64le** architecture.

7.5.1. Installing the Security Profiles Operator

You can use the OpenShift Container Platform web console to install the Security Profiles Operator. This installs the Security Profiles Operator into the **openshift-security-profiles** namespace by default. You can also verify correct installation by using the OpenShift Container Platform web console.

Prerequisites

- You must have access to the web console as a user with **cluster-admin** privileges.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Software Catalog**.
2. Search for the Security Profiles Operator, then click **Install**.

3. Keep the default selection of **Installation mode** and **namespace** to ensure that the Operator will be installed to the **openshift-security-profiles** namespace.
4. Click **Install**.

Verification

To confirm that the installation is successful:

1. Navigate to the **Ecosystem → Installed Operators** page.
2. Check that the Security Profiles Operator is installed in the **openshift-security-profiles** namespace and its status is **Succeeded**.

If the Operator is not installed successfully:

1. Navigate to the **Ecosystem → Installed Operators** page and inspect the **Status** column for any errors or failures.
2. Navigate to the **Workloads → Pods** page and check the logs in any pods in the **openshift-security-profiles** project that are reporting issues.

7.5.2. Installing the Security Profiles Operator using the CLI

You can install the OpenShift Container Platform Security Profiles Operator by using the command line interface.

Prerequisites

- You must have **cluster-admin** privileges.

Procedure

1. Define a **Namespace** object:

Example namespace-object.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-security-profiles
labels:
  openshift.io/cluster-monitoring: "true"
```

2. Create the **Namespace** object:

```
$ oc create -f namespace-object.yaml
```

3. Define an **OperatorGroup** object:

Example operator-group-object.yaml

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
```

```

metadata:
  name: security-profiles-operator
  namespace: openshift-security-profiles

```

4. Create the **OperatorGroup** object:

```
$ oc create -f operator-group-object.yaml
```

5. Define a **Subscription** object:

Example subscription-object.yaml

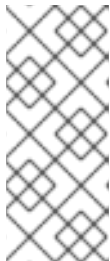
```

apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: security-profiles-operator-sub
  namespace: openshift-security-profiles
spec:
  channel: release-alpha-rhel-8
  installPlanApproval: Automatic
  name: security-profiles-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace

```

6. Create the **Subscription** object:

```
$ oc create -f subscription-object.yaml
```



NOTE

If you are setting the global scheduler feature and enable **defaultNodeSelector**, you must create the namespace manually and update the annotations of the **openshift-security-profiles** namespace, or the namespace where the Security Profiles Operator was installed, with **openshift.io/node-selector: ""**. This removes the default node selector and prevents deployment failures.

Verification

1. Verify the installation succeeded by inspecting the following CSV file:

```
$ oc get csv -n openshift-security-profiles
```

2. Verify that the Security Profiles Operator is operational by running the following command:

```
$ oc get deploy -n openshift-security-profiles
```

7.5.3. Configuring logging verbosity

The Security Profiles Operator supports the default logging verbosity of **0** and an enhanced verbosity of **1**.

Procedure

- To enable enhanced logging verbosity, patch the **spod** configuration and adjust the value by running the following command:

```
$ oc -n openshift-security-profiles patch spod \
  spod --type=merge -p '{"spec":{"verbosity":1}}'
```

Example output

```
securityprofilesoperatordaemon.security-profiles-operator.x-k8s.io/spod patched
```

7.6. MANAGING SECCOMP PROFILES

Create and manage seccomp profiles and bind them to workloads.



IMPORTANT

The Security Profiles Operator supports only Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes. Red Hat Enterprise Linux (RHEL) nodes are not supported.

7.6.1. Creating seccomp profiles

Use the **SeccompProfile** object to create seccomp profiles.

SeccompProfile objects can restrict syscalls within a container, limiting the access of your application.

Procedure

- Create a project by running the following command:

```
$ oc new-project my-namespace
```

- Create the **SeccompProfile** object:

```
apiVersion: security-profiles-operator.x-k8s.io/v1beta1
kind: SeccompProfile
metadata:
  name: profile1
spec:
  defaultAction: SCMP_ACT_LOG
```

The seccomp profile will be saved in **/var/lib/kubelet/seccomp/operator/<namespace>/<name>.json**.

An **init** container creates the root directory of the Security Profiles Operator to run the Operator without **root** group or user ID privileges. A symbolic link is created from the rootless profile storage **/var/lib/openshift-security-profiles** to the default **seccomp** root path inside of the kubelet root **/var/lib/kubelet/seccomp/operator**.

7.6.2. Apply seccomp profiles to a pod

To enforce a recorded or custom seccomp profile on a workload, create a pod that references the profile in its security context.

Procedure

1. Create a pod object that defines a **securityContext**:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  securityContext:
    runAsNonRoot: true
  seccompProfile:
    type: Localhost
    localhostProfile: operator/profile1.json
  containers:
  - name: test-container
    image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
    securityContext:
      allowPrivilegeEscalation: false
    capabilities:
      drop: [ALL]
```

2. View the profile path of the **seccompProfile.localhostProfile** attribute by running the following command:

```
$ oc get seccompprofile profile1 --output wide
```

Example output

```
NAME      STATUS  AGE  SECCOMPPROFILE.LOCALHOSTPROFILE
profile1  Installed 14s  operator/profile1.json
```

3. View the path to the localhost profile by running the following command:

```
$ oc get sp profile1 --output=jsonpath='{.status.localhostProfile}'
```

Example output

```
operator/profile1.json
```

4. Apply the **localhostProfile** output to the patch file:

```
spec:
  template:
    spec:
      securityContext:
        seccompProfile:
          type: Localhost
          localhostProfile: operator/profile1.json
```

5. Apply the profile to any other workload, such as a **Deployment** object, by running the following command:

```
$ oc -n my-namespace patch deployment myapp --patch-file patch.yaml --type=merge
```

Example output

```
deployment.apps/myapp patched
```

Verification

- Confirm the profile was applied correctly by running the following command:

```
$ oc -n my-namespace get deployment myapp --  
output=jsonpath='{.spec.template.spec.securityContext}' | jq .
```

Example output

```
{  
  "seccompProfile": {  
    "localhostProfile": "operator/profile1.json",  
    "type": "localhost"  
  }  
}
```

7.6.2.1. Binding workloads to profiles with ProfileBindings

You can use the **ProfileBinding** resource to bind a security profile to the **SecurityContext** of a container.

Procedure

1. To bind a pod that uses a **quay.io/security-profiles-operator/test-nginx-unprivileged:1.21** image to the example **SeccompProfile** profile, create a **ProfileBinding** object in the same namespace with the pod and the **SeccompProfile** objects:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha1  
kind: ProfileBinding  
metadata:  
  namespace: my-namespace  
  name: nginx-binding  
spec:  
  profileRef:  
    kind: SeccompProfile  
    name: profile  
  image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
```

where:

spec.profileRef.kind

Specifies the kind of the profile.

spec.profileRef.name

Specifies the name of the profile.

spec.image

Allows you to enable a default security profile by using a wildcard in the image attribute:

image: "*"



IMPORTANT

Using the **image: "*" wildcard attribute binds all new pods with a default security profile in a given namespace.**

2. Label the namespace with **enable-binding=true** by running the following command:

```
$ oc label ns my-namespace spo.x-k8s.io/enable-binding=true
```

3. Define a pod named **test-pod.yaml**:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - name: test-container
    image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
```

4. Create the pod:

```
$ oc create -f test-pod.yaml
```



NOTE

If the pod already exists, you must re-create the pod for the binding to work properly.

Verification

- Confirm the pod inherits the **ProfileBinding** by running the following command:

```
$ oc get pod test-pod -o jsonpath='{.spec.containers[*].securityContext.seccompProfile}'
```

Example output

```
{"localhostProfile":"operator/profile.json","type":"Localhost"}
```

7.6.3. Record profiles from workloads

The Security Profiles Operator can record system calls with **ProfileRecording** objects to create baseline profiles for applications.

When using the log enricher for recording seccomp profiles, verify the log enricher feature is enabled. See *Additional resources* for more information.



NOTE

A container with **privileged: true** security context restraints prevents log-based recording. Privileged containers are not subject to seccomp policies, and log-based recording makes use of a special seccomp profile to record events.

Procedure

1. Create a project by running the following command:

```
$ oc new-project my-namespace
```

2. Label the namespace with **enable-recording=true** by running the following command:

```
$ oc label ns my-namespace spo.x-k8s.io/enable-recording=true
```

3. Create a **ProfileRecording** object containing a **recorder: logs** variable:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha1
kind: ProfileRecording
metadata:
  namespace: my-namespace
  name: test-recording
spec:
  kind: SeccompProfile
  recorder: logs
  podSelector:
    matchLabels:
      app: my-app
```

4. Create a workload to record:

```
apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-pod
  labels:
    app: my-app
spec:
  securityContext:
    runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
  containers:
    - name: nginx
      image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
      ports:
        - containerPort: 8080
    - name: redis
      securityContext:
        allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
```



```
image: quay.io/security-profiles-operator/redis:6.2.1
securityContext:
  allowPrivilegeEscalation: false
capabilities:
  drop: [ALL]
```

- Confirm the pod is in a **Running** state by entering the following command:

```
$ oc -n my-namespace get pods
```

Example output

```
NAME    READY  STATUS   RESTARTS  AGE
my-pod  2/2    Running  0         18s
```

- Confirm the enricher indicates that it receives audit logs for those containers:

```
$ oc -n openshift-security-profiles logs --since=1m --selector name=spod -c log-enricher
```

Example output

```
I0523 14:19:08.747313 430694 enricher.go:445] log-enricher "msg"="audit"
"container"="redis" "executable"="/usr/local/bin/redis-server" "namespace"="my-namespace"
"node"="xiyuan-23-5g2q9-worker-eastus2-6rpgf" "pid"=656802 "pod"="my-pod" "syscallID"=0
"syscallName"="read" "timestamp"="1684851548.745:207179" "type"="seccomp"
```

Verification

- Remove the pod:

```
$ oc -n my-namespace delete pod my-pod
```

- Confirm the Security Profiles Operator reconciles the two seccomp profiles:

```
$ oc get seccompprofiles -lspo.x-k8s.io/recording-id=test-recording
```

Example output for seccomp profile

```
NAME                STATUS   AGE
test-recording-nginx Installed 2m48s
test-recording-redis Installed 2m48s
```

7.6.3.1. Merge per-container profile instances

To reuse one recorded profile when deploying applications with a **ReplicaSet** or **Deployment**, configure the Security Profiles Operator to merge per-container profile instances into a single profile instead of keeping a separate profile for each container.

Procedure

- Edit a **ProfileRecording** object to include a **mergeStrategy: containers** variable:

```

apiVersion: security-profiles-operator.x-k8s.io/v1alpha1
kind: ProfileRecording
metadata:
  # The name of the Recording is the same as the resulting SeccompProfile CRD
  # after reconciliation.
  name: test-recording
  namespace: my-namespace
spec:
  kind: SeccompProfile
  recorder: logs
  mergeStrategy: containers
  podSelector:
    matchLabels:
      app: sp-record

```

2. Label the namespace by running the following command:

```

$ oc label ns my-namespace security.openshift.io/scc.podSecurityLabelSync=false pod-
security.kubernetes.io/enforce=privileged pod-security.kubernetes.io/audit=privileged pod-
security.kubernetes.io/warn=privileged --overwrite=true

```

3. Create the workload with the following YAML:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  namespace: my-namespace
spec:
  replicas: 3
  selector:
    matchLabels:
      app: sp-record
  template:
    metadata:
      labels:
        app: sp-record
    spec:
      serviceAccountName: spo-record-sa
      containers:
        - name: nginx-record
          image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
          ports:
            - containerPort: 8080

```

4. To record the individual profiles, delete the deployment by running the following command:

```

$ oc delete deployment nginx-deploy -n my-namespace

```

5. To merge the profiles, delete the profile recording by running the following command:

```

$ oc delete profilerecording test-recording -n my-namespace

```

6. To start the merge operation and generate the results profile, run the following command:

```
$ oc get seccompprofiles -lspo.x-k8s.io/recording-id=test-recording -n my-namespace
```

Example output for seccomp profile

```
NAME                      STATUS    AGE
test-recording-nginx-record Installed  55s
```

- To view the permissions used by any of the containers, run the following command:

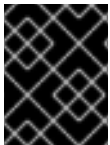
```
$ oc get seccompprofiles test-recording-nginx-record -o yaml
```

7.6.4. Additional resources

- [Managing security context constraints](#)
- [Managing SCCs in OpenShift](#)
- [Using the log enricher](#)
- [About security profiles](#)

7.7. MANAGING SELINUX PROFILES

To control what namespaced workloads can access on RHCOS nodes, use the Security Profiles Operator to create SELinux profiles, bind them to pods, and record policies from running applications.



IMPORTANT

The Security Profiles Operator supports only Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes. Red Hat Enterprise Linux (RHEL) nodes are not supported.

7.7.1. Creating SELinux profiles

Use the **SelinuxProfile** object to create SELinux profiles.

The **SelinuxProfile** object has several features that allow for better security hardening and readability:

- Restricts the profiles to inherit from to the current namespace or a system-wide profile. Because there are typically many profiles installed on the system, but only a subset should be used by cluster workloads, the inheritable system profiles are listed in the **spod** instance in **spec.selinuxOptions.allowedSystemProfiles**.
- Performs basic validation of the permissions, classes and labels.
- Adds a new keyword **@self** that describes the process using the policy. This allows reusing a policy between workloads and namespaces easily, as the usage of the policy is based on the name and namespace.
- Adds features for better security hardening and readability compared to writing a profile directly in the SELinux CIL language.

Procedure

1. Create a project by running the following command:

```
$ oc new-project nginx-deploy
```

2. Create a policy that can be used with a non-privileged workload by creating the following **SelinuxProfile** object:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha2
kind: SelinuxProfile
metadata:
  name: nginx-secure
spec:
  allow:
    '@self':
      tcp_socket:
        - listen
      http_cache_port_t:
        tcp_socket:
          - name_bind
      node_t:
        tcp_socket:
          - node_bind
  inherit:
    - kind: System
    name: container
```

3. Wait for **selinuxd** to install the policy by running the following command:

```
$ oc wait --for=condition=ready selinuxprofile nginx-secure
```

Example output

```
selinuxprofile.security-profiles-operator.x-k8s.io/nginx-secure condition met
```

The policies are placed into an **emptyDir** in the container owned by the Security Profiles Operator. The policies are saved in Common Intermediate Language (CIL) format in **/etc/selinux.d/<name>_<namespace>.cil**.

4. Access the pod by running the following command:

```
$ oc -n openshift-security-profiles rsh -c selinuxd ds/spod
```

Verification

1. View the file contents with **cat** by running the following command:

```
$ cat /etc/selinux.d/nginx-secure.cil
```

Example output

```
(block nginx-secure
(blockinherit container)
(allow process nginx-secure.process ( tcp_socket ( listen )))
```

```
(allow process http_cache_port_t ( tcp_socket ( name_bind )))
(allow process node_t ( tcp_socket ( node_bind )))
)
```

2. Verify that a policy has been installed by running the following command:

```
$ semodule -l | grep nginx-secure
```

Example output

```
nginx-secure
```

7.7.2. Apply SELinux profiles to a pod

To enforce a recorded or custom SELinux profile on a workload, create a pod that references the profile in its security context.

For SELinux profiles, the namespace must be labeled to allow [privileged](#) workloads.

Procedure

1. Apply the **scc.podSecurityLabelSync=false** label to the **nginx-deploy** namespace by running the following command:

```
$ oc label ns nginx-deploy security.openshift.io/scc.podSecurityLabelSync=false
```

2. Apply the **privileged** label to the **nginx-deploy** namespace by running the following command:

```
$ oc label ns nginx-deploy --overwrite=true pod-security.kubernetes.io/enforce=privileged
```

3. Obtain the SELinux profile usage string by running the following command:

```
$ oc get selinuxprofile.security-profiles-operator.x-k8s.io/nginx-secure -
  ojsonpath='{.status.usage}'
```

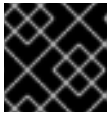
Example output

```
nginx-secure.process
```

4. Apply the output string in the workload manifest in the **.spec.containers[].securityContext.seLinuxOptions** attribute:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-secure
  namespace: nginx-deploy
spec:
  securityContext:
    runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
```

```
containers:
  - image: nginxinc/nginx-unprivileged:1.21
    name: nginx
    securityContext:
      allowPrivilegeEscalation: false
    capabilities:
      drop: [ALL]
    seLinuxOptions:
      # NOTE: This uses an appropriate SELinux type
      type: nginx-secure.process
```



IMPORTANT

The SELinux **type** must exist before creating the workload.

7.7.2.1. Apply SELinux log policies

To log policy violations or AVC denials, set the **SELinuxProfile** profile to **permissive**.



IMPORTANT

This procedure defines logging policies. It does not set enforcement policies.

Procedure

- Add **permissive: true** to an **SELinuxProfile**:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha2
kind: SelinuxProfile
metadata:
  name: nginx-secure
spec:
  permissive: true
```

7.7.2.2. Binding workloads to profiles with ProfileBindings

You can use the **ProfileBinding** resource to bind a security profile to the **SecurityContext** of a container.

Procedure

1. To bind a pod that uses a **quay.io/security-profiles-operator/test-nginx-unprivileged:1.21** image to the example **SelinuxProfile** profile, create a **ProfileBinding** object in the same namespace with the pod and the **SelinuxProfile** objects:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha1
kind: ProfileBinding
metadata:
  namespace: my-namespace
  name: nginx-binding
spec:
  profileRef:
```

```
kind: SelinuxProfile
name: profile
image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
```

where:

spec.profileRef.kind

Specifies the kind of the profile.

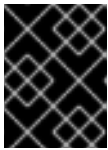
spec.profileRef.name

Specifies the name of the profile.

spec.image

Allows you to enable a default security profile by using a wildcard in the image attribute:

image: "*"



IMPORTANT

Using the **image: "*"** wildcard attribute binds all new pods with a default security profile in a given namespace.

2. Label the namespace with **enable-binding=true** by running the following command:

```
$ oc label ns my-namespace spo.x-k8s.io/enable-binding=true
```

3. Define a pod named **test-pod.yaml**:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
spec:
  containers:
  - name: test-container
    image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
```

4. Create the pod:

```
$ oc create -f test-pod.yaml
```



NOTE

If the pod already exists, you must re-create the pod for the binding to work properly.

Verification

- Confirm the pod inherits the **ProfileBinding** by running the following command:

```
$ oc get pod test-pod -o jsonpath='{.spec.containers[*].securityContext.seLinuxOptions.type}'
```

Example output

■

```
profile.process
```

7.7.2.3. Replicate controllers and SecurityContextConstraints

Deploy SELinux policies for replicating controllers such as deployments or daemon sets so the pods those controllers create can use custom SELinux policies.

Pods that the controllers create do not run with the identity of the user who creates the workload. Unless you select a **ServiceAccount**, the pods might use a restricted **SecurityContextConstraints** (SCC) object that does not allow custom security policies.

Procedure

1. Create a project by running the following command:

```
$ oc new-project nginx-secure
```

2. Create the following **RoleBinding** object to allow SELinux policies to be used in the **nginx-secure** namespace:

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: spo-nginx
  namespace: nginx-secure
subjects:
- kind: ServiceAccount
  name: spo-deploy-test
roleRef:
  kind: Role
  name: spo-nginx
  apiGroup: rbac.authorization.k8s.io
```

3. Create the **Role** object:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  creationTimestamp: null
  name: spo-nginx
  namespace: nginx-secure
rules:
- apiGroups:
  - security.openshift.io
  resources:
  - securitycontextconstraints
  resourceNames:
  - privileged
  verbs:
  - use
```

4. Create the **ServiceAccount** object:

```
apiVersion: v1
```



```

kind: ServiceAccount
metadata:
  creationTimestamp: null
  name: spo-deploy-test
  namespace: nginx-secure

```

5. Create the **Deployment** object:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: selinux-test
  namespace: nginx-secure
  metadata:
    labels:
      app: selinux-test
spec:
  replicas: 3
  selector:
    matchLabels:
      app: selinux-test
  template:
    metadata:
      labels:
        app: selinux-test
    spec:
      serviceAccountName: spo-deploy-test
      securityContext:
        seLinuxOptions:
          type: nginx-secure.process
      containers:
        - name: nginx-unpriv
          image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
          ports:
            - containerPort: 8080

```

The **spec.template.spec.securityContext.seLinuxOptions.type** must exist before the Deployment is created.



NOTE

The SELinux type is not specified in the workload and is handled by the SCC. When the pods are created by the deployment and the **ReplicaSet**, the pods will run with the appropriate profile.

Ensure that your SCC is usable by only the correct service account. Refer to *Additional resources* for more information.

7.7.3. Record profiles from workloads

The Security Profiles Operator can record system calls with **ProfileRecording** objects to create baseline profiles for applications.

When using the log enricher for recording SELinux profiles, verify the log enricher feature is enabled. See *Additional resources* for more information.



NOTE

A container with **privileged: true** security context restraints prevents log-based recording. Privileged containers are not subject to SELinux policies, and log-based recording makes use of a special SELinux profile to record events.

Procedure

1. Create a project by running the following command:

```
$ oc new-project my-namespace
```

2. Label the namespace with **enable-recording=true** by running the following command:

```
$ oc label ns my-namespace spo.x-k8s.io/enable-recording=true
```

3. Create a **ProfileRecording** object containing a **recorder: logs** variable:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha1
kind: ProfileRecording
metadata:
  namespace: my-namespace
  name: test-recording
spec:
  kind: SelinuxProfile
  recorder: logs
  podSelector:
    matchLabels:
      app: my-app
```

4. Create a workload to record:

```
apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-pod
  labels:
    app: my-app
spec:
  securityContext:
    runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
  containers:
    - name: nginx
      image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
      ports:
        - containerPort: 8080
      securityContext:
        allowPrivilegeEscalation: false
```

```

    capabilities:
      drop: [ALL]
- name: redis
  image: quay.io/security-profiles-operator/redis:6.2.1
  securityContext:
    allowPrivilegeEscalation: false
  capabilities:
    drop: [ALL]

```

5. Confirm the pod is in a **Running** state by entering the following command:

```
$ oc -n my-namespace get pods
```

Example output

```

NAME    READY  STATUS   RESTARTS  AGE
my-pod  2/2    Running  0          18s

```

6. Confirm the enricher indicates that it receives audit logs for those containers:

```
$ oc -n openshift-security-profiles logs --since=1m --selector name=spod -c log-enricher
```

Example output

```

I0517 13:55:36.383187 348295 enricher.go:376] log-enricher "msg"="audit"
"container"="redis" "namespace"="my-namespace" "node"="ip-10-0-189-53.us-east-
2.compute.internal" "perm"="name_bind" "pod"="my-pod" "profile"="test-
recording_redis_6kmb_1684331729"
"scontext"="system_u:system_r:selinuxrecording.process:s0:c4,c27" "tclass"="tcp_socket"
"tcontext"="system_u:object_r:redis_port_t:s0" "timestamp"="1684331735.105:273965"
"type"="selinux"

```

Verification

1. Remove the pod:

```
$ oc -n my-namespace delete pod my-pod
```

2. Confirm the Security Profiles Operator reconciles the two SELinux profiles:

```
$ oc get selinuxprofiles -lspo.x-k8s.io/recording-id=test-recording
```

Example output for SELinux profile

```

NAME                USAGE                                STATE
test-recording-nginx test-recording-nginx.process  Installed
test-recording-redis test-recording-redis.process   Installed

```

7.7.3.1. Merge per-container profile instances

To reuse one recorded profile when deploying applications with a **ReplicaSet** or **Deployment**, configure the Security Profiles Operator to merge per-container profile instances into a single profile instead of keeping a separate profile for each container.

Procedure

1. Edit a **ProfileRecording** object to include a **mergeStrategy: containers** variable:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha1
kind: ProfileRecording
metadata:
  # The name of the Recording is the same as the resulting SelinuxProfile CRD
  # after reconciliation.
  name: test-recording
  namespace: my-namespace
spec:
  kind: SelinuxProfile
  recorder: logs
  mergeStrategy: containers
  podSelector:
    matchLabels:
      app: sp-record
```

2. Label the namespace by running the following command:

```
$ oc label ns my-namespace security.openshift.io/scc.podSecurityLabelSync=false pod-
security.kubernetes.io/enforce=privileged pod-security.kubernetes.io/audit=privileged pod-
security.kubernetes.io/warn=privileged --overwrite=true
```

3. Create the workload with the following YAML:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  namespace: my-namespace
spec:
  replicas: 3
  selector:
    matchLabels:
      app: sp-record
  template:
    metadata:
      labels:
        app: sp-record
    spec:
      serviceAccountName: spo-record-sa
      containers:
        - name: nginx-record
          image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
          ports:
            - containerPort: 8080
```

4. To record the individual profiles, delete the deployment by running the following command:

■

```
$ oc delete deployment nginx-deploy -n my-namespace
```

- To merge the profiles, delete the profile recording by running the following command:

```
$ oc delete profilerecording test-recording -n my-namespace
```

- To start the merge operation and generate the results profile, run the following command:

```
$ oc get selinuxprofiles -lspo.x-k8s.io/recording-id=test-recording -n my-namespace
```

Example output for SELinux profile

NAME	USAGE	STATE
test-recording-nginx-record	test-recording-nginx-record.process	Installed

- To view the permissions used by any of the containers, run the following command:

```
$ oc get selinuxprofiles test-recording-nginx-record -o yaml
```

7.7.3.2. About seLinuxContext: RunAsAny

To record SELinux policies, a webhook injects a permissive SELinux type so the pod logs AVC denials while recording.

The SELinux type makes the pod run in **permissive** mode, logging all the AVC denials into **audit.log**. By default, a workload is not allowed to run with a custom SELinux policy, but uses an automatically generated type.

To record a workload, the workload must use a service account that has permissions to use an SCC that allows the webhook to inject the permissive SELinux type. The **privileged** SCC contains **seLinuxContext: RunAsAny**.

In addition, the namespace must be labeled with **pod-security.kubernetes.io/enforce: privileged** if your cluster enables Pod Security Admission, because only the **privileged** Pod Security Standard allows using a custom SELinux policy.

7.7.4. Additional resources

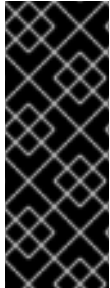
- [Managing security context constraints](#)
- [Managing SCCs in OpenShift](#)
- [About security profiles](#)
- [Use the log enricher](#)
- [Pod Security Admission](#)
- [Pod Security Standard](#)

7.8. ADVANCED SECURITY PROFILES OPERATOR TASKS

You can use advanced Security Profiles Operator tasks to enable metrics, configure webhooks, or restrict syscalls.

7.8.1. Restrict the allowed syscalls in seccomp profiles

The Security Profiles Operator does not restrict **syscalls** in **seccomp** profiles by default. You can define the list of allowed **syscalls** in the **spod** configuration.



IMPORTANT

The Operator will install only the **seccomp** profiles, which have a subset of **syscalls** defined into the allowed list. All profiles not complying with this ruleset are rejected.

When the list of allowed **syscalls** is modified in the **spod** configuration, the Operator will identify the already installed profiles which are noncompliant and remove them automatically.

Procedure

- To define the list of **allowedSyscalls**, adjust the **spec** parameter by running the following command:

```
$ oc -n openshift-security-profiles patch spod spod --type merge \
-p '{"spec":{"allowedSyscalls": ["exit", "exit_group", "futex", "nanosleep"]}]'
```

7.8.2. Base syscalls for a container runtime

You can use the **baseProfileName** attribute to establish the minimum required **syscalls** for a given runtime to start a container.

Procedure

- Edit the **SeccompProfile** kind object and add **baseProfileName: runc-v1.0.0** to the **spec** field:

```
apiVersion: security-profiles-operator.x-k8s.io/v1beta1
kind: SeccompProfile
metadata:
  name: example-name
spec:
  defaultAction: SCMP_ACT_ERRNO
  baseProfileName: runc-v1.0.0
  syscalls:
    - action: SCMP_ACT_ALLOW
      names:
        - exit_group
```

7.8.3. Enabling memory optimization in the spod daemon

The controller running inside of **spod** daemon process watches all pods available in the cluster when profile recording is enabled. This can lead to very high memory usage in large clusters, resulting in the **spod** daemon running out of memory or crashing.

To prevent crashes, the **spod** daemon can be configured to only load the pods labeled for profile recording into the cache memory.



NOTE

SPO memory optimization is not enabled by default.

Procedure

1. Enable memory optimization by running the following command:

```
$ oc -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"enableMemoryOptimization":true}}'
```

2. To record a security profile for a pod, the pod must be labeled with **spo.x-k8s.io/enable-recording: "true"**:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-recording-pod
  labels:
    spo.x-k8s.io/enable-recording: "true"
# ...
```

7.8.4. Customizing daemon resource requirements

The default resource requirements of the daemon container can be adjusted by using the field **daemonResourceRequirements** from the **spod** configuration.

Procedure

- To specify the memory and cpu requests and limits of the daemon container, run the following command:

```
$ oc -n openshift-security-profiles patch spod spod --type merge -p \
{"spec":{"daemonResourceRequirements": { \
  "requests": {"memory": "256Mi", "cpu": "250m"}, \
  "limits": {"memory": "512Mi", "cpu": "500m"}}}}
```

7.8.5. Setting a custom priority class name for the spod daemon pod

The default priority class name of the **spod** daemon pod is set to **system-node-critical**. A custom priority class name can be configured in the **spod** configuration by setting a value in the **priorityClassName** field.

Procedure

- Configure the priority class name by running the following command:

```
$ oc -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"priorityClassName":"my-priority-class"}}'
```

Example output

```
securityprofilesoperatordaemon.openshift-security-profiles.x-k8s.io/spod patched
```

7.8.6. Using metrics

The **openshift-security-profiles** namespace provides metrics endpoints, which are secured by the **kube-rbac-proxy** container. All metrics are exposed by the **metrics** service within the **openshift-security-profiles** namespace.

The Security Profiles Operator includes a cluster role and corresponding binding **spo-metrics-client** to retrieve the metrics from within the cluster. There are two metrics paths available:

- **metrics.openshift-security-profiles/metrics**: for controller runtime metrics
- **metrics.openshift-security-profiles/metrics-spod**: for the Operator daemon metrics

Procedure

1. To view the status of the metrics service, run the following command:

```
$ oc get svc/metrics -n openshift-security-profiles
```

Example output

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
metrics	ClusterIP	10.0.0.228	<none>	443/TCP	43s

2. To retrieve the metrics, query the service endpoint using the default **ServiceAccount** token in the **openshift-security-profiles** namespace by running the following command:

```
$ oc run --rm -i --restart=Never --image=registry.fedoraproject.org/fedora-minimal:latest \
-n openshift-security-profiles metrics-test -- bash -c \
'curl -ks -H "Authorization: Bearer $(cat \
/var/run/secrets/kubernetes.io/serviceaccount/token)" https://metrics.openshift-security-
profiles/metrics-spod'
```

Example output

```
# HELP security_profiles_operator_seccomp_profile_total Counter about seccomp profile
operations.
# TYPE security_profiles_operator_seccomp_profile_total counter
security_profiles_operator_seccomp_profile_total{operation="delete"} 1
security_profiles_operator_seccomp_profile_total{operation="update"} 2
```

3. To retrieve metrics from a different namespace, link the **ServiceAccount** to the **spo-metrics-client ClusterRoleBinding** by running the following command:

```
$ oc get clusterrolebinding spo-metrics-client -o wide
```

Example output

```
■
```


NAME	ROLE	AGE	USERS	GROUPS	SERVICEACCOUNTS
spo-metrics-client profiles/default	ClusterRole/spo-metrics-client	35m			openshift-security-

7.8.6.1. controller-runtime metrics

The controller-runtime **metrics** and the DaemonSet endpoint **metrics-spod** provide a set of default metrics. Additional metrics are provided by the daemon, which are always prefixed with **security_profiles_operator_**.

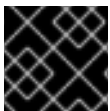
Table 7.1. Available controller-runtime metrics

Metric key	Possible labels	Type	Purpose
seccomp_profile_total	operation= {delete,update}	Counter	Amount of seccomp profile operations.
seccomp_profile_audit_total	node, namespace, pod, container, executable, syscall	Counter	Amount of seccomp profile audit operations. Requires the log enricher to be enabled.
seccomp_profile_bpf_total	node, mount_namespace, profile	Counter	Amount of seccomp profile bpf operations. Requires the bpf recorder to be enabled.
seccomp_profile_error_total	reason= {SeccompNotSupportedOnNode,InvalidSeccompProfile,CannotSaveSeccompProfile,CannotRemoveSeccompProfile,CannotUpdateSeccompProfile,CannotUpdateNodeStatus}	Counter	Amount of seccomp profile errors.
selinux_profile_total	operation= {delete,update}	Counter	Amount of SELinux profile operations.
selinux_profile_audit_total	node, namespace, pod, container, executable, scontext,tcontext	Counter	Amount of SELinux profile audit operations. Requires the log enricher to be enabled.

Metric key	Possible labels	Type	Purpose
selinux_profile_error_total	reason= {CannotSaveSelinuxPolicy,CannotUpdatePolicyStatus,CannotRemoveSelinuxPolicy,CannotContactSelinuxd,CannotWritePolicyFile,CannotGetPolicyStatus}	Counter	Amount of SELinux profile errors.

7.8.7. Use the log enricher

The Security Profiles Operator contains a log enrichment feature, which is disabled by default. The log enricher container runs with **privileged** permissions in the host PID namespace (**hostPID**) so it can read audit logs from the local node.



IMPORTANT

The log enricher must have permissions to read the host processes.

Procedure

1. Patch the **spod** configuration to enable the log enricher by running the following command:

```
$ oc -n openshift-security-profiles patch spod spod \
  --type=merge -p '{"spec":{"enableLogEnricher":true}}'
```

Example output

```
securityprofilesoperatordaemon.security-profiles-operator.x-k8s.io/spod patched
```



NOTE

The Security Profiles Operator will re-deploy the **spod** daemon set automatically.

2. View the audit logs by running the following command:

```
$ oc -n openshift-security-profiles logs -f ds/spod log-enricher
```

Example output

```
I0623 12:51:04.257814 1854764 deleg.go:130] setup "msg"="starting component: log-
enricher" "buildDate"="1980-01-01T00:00:00Z" "compiler"="gc" "gitCommit"="unknown"
"gitTreeState"="clean" "goVersion"="go1.16.2" "platform"="linux/amd64" "version"="0.4.0-
dev"
I0623 12:51:04.257890 1854764 enricher.go:44] log-enricher "msg"="Starting log-enricher on
```

```
node: 127.0.0.1"
I0623 12:51:04.257898 1854764 enricher.go:46] log-enricher "msg"="Connecting to local
GRPC server"
I0623 12:51:04.258061 1854764 enricher.go:69] log-enricher "msg"="Reading from file
/var/log/audit/audit.log"
2021/06/23 12:51:04 Seaked /var/log/audit/audit.log - &{Offset:0 Whence:2}
```

7.8.7.1. Using the log enricher to trace an application

You can use the Security Profiles Operator log enricher to trace an application.

Procedure

1. To trace an application, create a **SeccompProfile** logging profile:

```
apiVersion: security-profiles-operator.x-k8s.io/v1beta1
kind: SeccompProfile
metadata:
  name: log
spec:
  defaultAction: SCMP_ACT_LOG
```

2. Create a pod object to use the profile:

```
apiVersion: v1
kind: Pod
metadata:
  name: log-pod
  namespace: default
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: Localhost
      localhostProfile: operator/log.json
  containers:
  - name: log-container
    image: quay.io/security-profiles-operator/test-nginx-unprivileged:1.21
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: [ALL]
```

3. Examine the log enricher output by running the following command:

```
$ oc -n openshift-security-profiles logs -f ds/spod log-enricher
```

Example output

```
...
I0623 12:59:11.479869 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/" "namespace"="default" "node"="127.0.0.1"
"pid"=1905792 "pod"="log-pod" "syscallID"=3 "syscallName"="close"
"timestamp"="1624453150.205:1061" "type"="seccomp"
```

```

10623 12:59:11.487323 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/" "namespace"="default" "node"="127.0.0.1"
"pid"=1905792 "pod"="log-pod" "syscallID"=157 "syscallName"="prctl"
"timestamp"="1624453150.205:1062" "type"="seccomp"
10623 12:59:11.492157 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/" "namespace"="default" "node"="127.0.0.1"
"pid"=1905792 "pod"="log-pod" "syscallID"=157 "syscallName"="prctl"
"timestamp"="1624453150.205:1063" "type"="seccomp"
...
10623 12:59:20.258523 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=12 "syscallName"="brk"
"timestamp"="1624453150.235:2873" "type"="seccomp"
10623 12:59:20.263349 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=21 "syscallName"="access"
"timestamp"="1624453150.235:2874" "type"="seccomp"
10623 12:59:20.354091 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=257 "syscallName"="openat"
"timestamp"="1624453150.235:2875" "type"="seccomp"
10623 12:59:20.358844 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=5 "syscallName"="fstat"
"timestamp"="1624453150.235:2876" "type"="seccomp"
10623 12:59:20.363510 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=9 "syscallName"="mmap"
"timestamp"="1624453150.235:2877" "type"="seccomp"
10623 12:59:20.454127 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=3 "syscallName"="close"
"timestamp"="1624453150.235:2878" "type"="seccomp"
10623 12:59:20.458654 1854764 enricher.go:111] log-enricher "msg"="audit"
"container"="log-container" "executable"="/usr/sbin/nginx" "namespace"="default"
"node"="127.0.0.1" "pid"=1905792 "pod"="log-pod" "syscallID"=257 "syscallName"="openat"
"timestamp"="1624453150.235:2879" "type"="seccomp"
...

```

7.8.8. Configure webhooks

Configure webhooks for profile binding and recording so you can limit them to selected namespaces or objects, or allow requests to continue if a webhook fails. Profile binding and recording object configurations are **MutatingWebhookConfiguration** CRs, managed by the Security Profiles Operator.

To change the webhook configuration, edit the **webhookOptions** field in the **spod** custom resource. You can modify the **failurePolicy**, **namespaceSelector**, and **objectSelector** variables to set the webhooks to soft-fail or to restrict them to a subset of namespaces. If a webhook fails, other namespaces or resources are not affected.

Procedure

1. Set the **recording.spo.io** webhook configuration to record only pods labeled with **spo-record=true** by creating the following patch file:

```
spec:
  webhookOptions:
    - name: recording.spo.io
  objectSelector:
    matchExpressions:
      - key: spo-record
        operator: In
    values:
      - "true"
```

2. Patch the **spod/spod** instance by running the following command:

```
$ oc -n openshift-security-profiles patch spod \
  spod -p $(cat /tmp/spod-wh.patch) --type=merge
```

3. To view the resulting **MutatingWebhookConfiguration** object, run the following command:

```
$ oc get MutatingWebhookConfiguration \
  spo-mutating-webhook-configuration -oyaml
```

7.9. ADVANCED AUDIT LOGGING FRAMEWORK

With the Advanced Audit Logging Framework in the OpenShift Container Platform Security Profiles Operator (SPO), you can correlate cluster users with actions during **oc exec**, **oc rsh**, and **oc debug** sessions.

The Advanced Audit Logging Framework in SPO 0.10.0 logs activity from an Red Hat Enterprise Linux CoreOS (RHCOS) container back to the hosting cluster and produces detailed logs in a JSON Lines format.

7.9.1. Benefits of the Advanced Audit Logging Framework

The **kubectl exec**, **oc exec**, **oc rsh** and **oc debug** commands do not pass user authentication details into the exec session on the container, making it hard to correlate Kubernetes user actions caused by actions on the host. The audit logger in SPO addresses this with mutating webhooks that inject the request UID from the Kubernetes API server as an environment variable into the session. Every request to the API server including the request to start a new exec session has a request UID. This request UID is then logged by the Advanced Audit Logging Framework. The request ID is used to correlate the activity with the API server audit logs, providing an audit trail within the node.

With the addition of the Advanced Audit Logging Framework, SPO now has two use cases:

- Pod auditing
- Node auditing

The use of privileged **seccompProfile** configuration is required only for the case of node auditing.

**NOTE**

The Security Profiles Operator supports only Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes appropriate to the version of OpenShift Container Platform in use.

Red Hat Enterprise Linux (RHEL) nodes are not supported.

7.9.2. Performance considerations

It is important to consider the performance cost of using seccomp profiles for extensive logging. SPO is designed to minimize this impact by primarily logging only process creations and handling them asynchronously. This approach helps prevent logging from becoming a bottleneck on your nodes.

The Advanced Audit Logging feature uses eBPF as a supplemental data source. While it is possible for eBPF to be used as a primary data source for this type of logging, that functionality is not currently a configurable feature within the Operator. For most use cases, the default asynchronous, process-creation-focused logging approach provides an excellent balance between security visibility and cluster performance.

7.9.3. Prerequisites for the Advanced Audit Logging Framework

Before enabling the Advanced Audit Logging Framework, ensure the following requirements are met.

Security Profiles Operator version 0.10.0 or later is installed. The Advanced Audit Logging Framework requires Security Profiles Operator version 0.10.0 or later.

For node debugging sessions:

- To audit **oc debug** node sessions, CRI-O version 1.33 or later is required. It is available in OpenShift Container Platform 4.20 or later.
- The `--privileged-seccomp-profile` flag must be configured in CRI-O to apply **seccompProfiles** to privileged containers.
- The supported Linux used with the Advanced Audit Logging Framework is Red Hat Enterprise Linux CoreOS (RHCOS) running in a container in OpenShift Container Platform 4.20 or later.

If you are using the CRI-O runtime, you must configure it to allow **seccompProfile** to be applied to privileged containers. Add the following flag to your CRI-O runtime configuration: **`--privileged-seccomp-profile=/var/lib/kubelet/seccomp/operator/profile1.json`**. This is explained in more detail in the Advanced Audit Logging installation and enablement steps. The **`--privileged-seccomp-profile`** flag is available starting with OpenShift Container Platform 4.20 and later.

If you are using any version of SPO before 0.9.0, you must perform a migration procedure to install versions 0.9.0 or 0.10.0. The migration procedure converts SPO to operate on cluster-scoped resources.

First-time installation of SPO version 0.10.0 does not require migration. Also, if you are currently on SPO 0.9.0, you do not require migration and can directly upgrade to SPO 0.10.0.



IMPORTANT

Do not attempt to upgrade directly from SPO versions before 0.9.0 to either 0.9.0 or 0.10.0 if you are currently running SPO. You must perform the migration procedure to convert SPO for operation at the cluster level.

This change allows Advanced Audit Logging of events inside the worker node.

This capability is not provided for control nodes since SPO does not operate on **etcd** nodes.

7.9.4. The Audit JSON log enricher

The SPO Advanced Audit Logging Framework is enabled by the Audit JSON log enricher. Similar to the log enricher feature, the Audit JSON log enricher watches the **auditd** (**/var/log/audit/audit.log**) or the **syslog** (**/var/log/syslog**) daemons and generates a new audit log in JSON lines format.

Each JSON line includes the following:

- Timestamp: When the activity happened, shown in a standard ISO format
- Executable Name: The name of the program that was run (**bash**, **ls**).
- Linux Command Line Arguments (**cmdline**): The extra instructions given when the program was started (**ls -l /home**).
- User and Group IDs (**UID/GID**): The identification numbers of the system user who ran the program.
- System Calls (**syscalls**): A list of system calls (**syscalls**) that the process made

This log format and configuration is similar to how Kubernetes records audit logs. This is useful for:

- Seeing what users and automated processes are doing inside a pod.
- Tracking when someone uses commands such as **kubectl exec** to enter a running container and run commands or scripts.
- Monitoring activities in debug containers where users might run various tools.

7.9.5. Kubernetes API log compared to Advanced Audit Logging output

To understand the value of Advanced Audit Logging, compare the Kubernetes API Server Audit Log to the Advanced Audit Logging output:

Kubernetes API Server Audit Log

```
{
  "kind": "Event",
  "apiVersion": "audit.k8s.io/v1",
  "level": "Metadata",
  "auditID": "4d434cd4-xxxx-xxxx-xxxx-2d9aa46292ce",
  "stage": "ResponseComplete",
  "requestURI": "/api/v1/namespaces/test-namespace/pods/test-pod/exec?command=sh&command=-c&command=touch+/tmp/testfile.txt&container=nginx",
  "verb": "create",
```

```

"user": {
  "username": "kube:admin",
  "groups": ["system:cluster-admins", "system:authenticated"]
},
"sourceIPs": ["xxx.xxx.xxx.xxx"],
"userAgent": "oc/4.19.0 (linux/amd64)",
"objectRef": {
  "resource": "pods",
  "namespace": "test-namespace",
  "name": "test-pod",
  "subresource": "exec"
},
"responseStatus": {
  "code": 101
},
"requestReceivedTimestamp": "2026-02-16T14:01:06.056518Z",
"annotations": {
  "authorization.k8s.io/decision": "allow",
  "authorization.k8s.io/reason": "RBAC: allowed by ClusterRoleBinding...",
  "execmetadata.spo.io/SPO_EXEC_REQUEST_UID": "aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9"
}
}

```

The correlation key is the **SPO_EXEC_REQUEST_UID** on the last line in the above file.

Advanced Audit Logging Output

```

{
  "auditID": "d586679d-xxxx-xxxx-xxxx-9dc8ab273065",
  "cmdLine": "touch /tmp/testfile.txt ", // Linux command with arguments
  "executable": "/bin/dash",
  "gid": 0,
  "node": {
    "name": "worker-1"
  },
  "pid": 144968,
  "requestUID": "aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9", // Correlation key
  "resource": {
    "container": "nginx",
    "namespace": "test-namespace",
    "pod": "test-pod"
  },
  "syscalls": ["execve"],
  "timestamp": "2026-02-16T14:01:07.000Z",
  "uid": 0,
  "version": "spo/v1_alpha"
}

```

7.9.6. Enabling Advanced Audit Logging

To enable Advanced Audit Logging, configure the Audit JSON log enricher and specify a set of filters to only log user activity.

Procedure

1. Enable the JSON enricher by running the following command:

```
# kubectl -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"enableJsonEnricher":true}}'
```

Monitor the SPOD pods for correct restart and wait for all SPOD pods to show **Running** before proceeding.

2. Check the pods for application of the change with the following command:

```
$ oc get pods -n openshift-security-profiles -l name=spod -w
```

Wait until all SPOD pods show **Running** before proceeding.



NOTE

Each configuration change triggers a restart of the SPOD pods. After applying a patch, wait for all SPOD pods to return to the **Running** state before continuing.

The Audit JSON log enricher uses eBPF as a supplemental data source. While processing **auditd** logs from **/var/log/audit/audit.log**, the enricher attempts to fetch ephemeral data from **/proc/<pid>** directories. Due to a race condition, these files might be deleted before they can be read. To ensure data completeness, the enricher falls back to fetching the necessary information from eBPF whenever it is not found in **/proc/<pid>**.

7.9.7. Audit JSON Log Enricher configuration

Configure the Audit JSON Log Enricher to set the audit log interval, destination, and file path so Advanced Audit Logging records are written on a schedule and to a location you can collect and review.



NOTE

Each configuration change triggers a restart of the SPOD pods. After applying a patch, wait for all SPOD pods to return to the **Running** state before continuing.

Setting the audit log interval determines how often audit logs are created using the **auditLogIntervalSeconds** option.

Procedure

1. Configure the audit log interval to 30 seconds by using the following command:

```
# kubectl -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"enableJsonEnricher":true,"verbosity":0,"jsonEnricherOptions":
{"auditLogIntervalSeconds":30}}'
```

Wait until all SPOD pods show **Running** before proceeding. By default, audit logs go to your standard output in JSON lines format. You can send them to a file instead.

Configure the Security Profiles Operator to store the log file on the node. Update the **security-profiles-operator-profile configmap** with two keys. This example YAML uses both keys to set up a host path volume at **/tmp/logs**.

2. Create a file such as **patch-volume-source.json** that contains the following content:

```
{
  "data": {
    "json-enricher-log-volume-mount-path": "/tmp/logs",
    "json-enricher-log-volume-source.json": "{\"hostPath\": {\"path\": \"/tmp/logs\", \"type\": \"DirectoryOrCreate\"}}"
  }
}
```

- **json-enricher-log-volume-source.json**: Defines the type of volume (for example, a host path and empty directory) where logs are stored. This value must be a JSON string that represents a **corev1.VolumeSource** object.
- **json-enricher-log-volume-mount-path**: Specifies the directory path where the log file is generated.

3. Verify the file contents by running the following command:

```
$ cat patch-volume-source.json
```

4. Update the **security-profiles-operator-profile configmap** by using the following command:

```
# kubectl patch configmap security-profiles-operator-profile -n openshift-security-profiles --
patch-file patch-volume-source.json
```

Wait until all SPOD pods show **Running** before proceeding.

5. Set the audit log file path by configuring the JSON log enricher with the full path to your audit log file, including the file name, by running the following command:

```
# kubectl -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"enableJsonEnricher":true,"verbosity":0,"jsonEnricherOptions":
{"auditLogPath":"/tmp/logs/audit1.log"}}}'
```

Wait until all SPOD pods show **Running** before proceeding.

7.9.8. Audit log file fine-tuning and rotation

For audit logging to a file, you can manage file size and how long each file is kept. These options are similar to [Kubernetes API server log settings](#).

Procedure

1. You configure these by patching the JSON log enricher options:

```
# kubectl -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"enableJsonEnricher":true,"verbosity":0,"jsonEnricherOptions":
{"auditLogPath":"/tmp/logs/audit1.log","auditLogMaxSize":500,"auditLogMaxBackups":2,"auditLogMaxAge":10}}}'
```

Wait until all SPOD pods show **Running** before proceeding.

- **auditLogMaxSize:** The maximum size (in megabytes) a log file can reach before it's rotated (a new file is started).
 - **auditLogMaxBackups:** The maximum number of older, rotated log files to keep. Set to 0 for no limit.
 - **auditLogMaxAge:** The maximum number of days to keep old log files.
2. Increase the logging level for the JSON log enricher container to help with debugging. A value of **0** sets minimal logs. A value of **1** sets more detailed logs. You can choose either of these two levels and enable either level with the following command:

```
# kubectl -n openshift-security-profiles patch spod spod --type=merge -p '{"spec":
{"enableJsonEnricher":true, "verbosity": 1}}'
```

Wait until all SPOD pods show **Running** before proceeding.

7.9.9. Advanced audit logs for a specific pod

To log activity for a single pod, create a **SeccompProfile** that logs specific syscalls such as **execve**, and create a **ProfileBinding** that applies the profile to pods in a target namespace. The **SeccompProfile** applies cluster-wide. The **ProfileBinding** applies to workloads in that namespace.

Starting with OpenShift Container Platform 4.20 and CRI-O 1.33, you can apply a **SeccompProfile** to privileged containers. Add the **--privileged-seccomp-profile** flag to the CRI-O runtime configuration so that privileged debugging pods are also covered by the profile.

Bind the profile to a namespace to apply it to workloads. New pods in that namespace then receive the profile automatically.

Procedure

1. Create a file such as **profile1.yaml** with the following content:

```
apiVersion: security-profiles-operator.x-k8s.io/v1beta1
kind: SeccompProfile
metadata:
  name: profile1
  namespace: openshift-security-profiles
spec:
  defaultAction: SCMP_ACT_ALLOW
  syscalls:
    - action: SCMP_ACT_LOG
      names:
        - execve
        - clone
        - getpid
```

This profile allows all normal actions (**defaultAction: SCMP_ACT_ALLOW**). It specifically tells the system to log when a process tries to run a new program (**execve**), create a new process (**clone**), or get its own process ID (**getpid**). These actions often indicate user interaction within a pod.

2. Apply this **SeccompProfile** to your cluster by running the following command:

■

```
# kubectl apply -f profile1.yaml
```

**NOTE**

The Security Profiles Operator must use the privileged **SeccompProfile**.

3. Create a file named **image_sec_comp.yaml** that contains the following YAML:

```
apiVersion: security-profiles-operator.x-k8s.io/v1alpha1
kind: ProfileBinding
metadata:
  namespace: default
  name: all-pod-binding
spec:
  profileRef:
    kind: SeccompProfile
    name: profile1
  image: ""
```

4. Apply the binding by running the following command:

```
# kubectl apply -f image_sec_comp.yaml
```

5. Label the namespace to activate the binding by running the following command:

```
# kubectl label ns default spo.x-k8s.io/enable-binding=true
```

6. Create a file such as **my-pod.yaml** that contains the following pod definition:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  labels:
    app: my-app
spec:
  securityContext:
    seccompProfile:
      type: Localhost
      localhostProfile: operator/profile1.json
  containers:
    - name: nginx
      image: quay.io/security-profiles-operator/test-nginx:1.19.1
```

- **type: Localhost** means you are using a profile that you defined in the cluster.
- **localhostProfile: operator/profile1.json** tells the pod to use the **profile1** profile that you created. The **operator/** path is where the Security Profiles Operator stores these profiles.

7. Apply the pod definition by running the following command:

```
# kubectl apply -f my-pod.yaml
```

8. Open a shell in the pod by running the following command:

```
# kubectl exec -it my-pod -- /bin/sh
```

9. Create an empty file in the pod by running the following command:

```
# touch /tmp/audittest/demo-file
```

10. Stream the advanced audit log by running the following command:

```
# kubectl -n openshift-security-profiles logs --since=1m --selector name=spod -c json-enricher --max-log-requests 6 -f
```

11. Identify the node where the pod runs by running the following command:

```
# kubectl get pod my-pod -o wide
```

The audit log file specified in the **auditLogPath** field is written to the file system on the node where the pod is running. To inspect the audit logs, access the node and open the file at the configured path, such as **/tmp/logs/audit1.log**.

12. Access the node by running the following command:

```
$ sudo ssh core@<node_name>
```

13. View the audit log by running the following command:

```
$ cat /tmp/logs/audit1.log
```

Example output

```
{
  "auditID": "a1b2c3d4-e5f6-7890-abcd-111111111111",
  "cmdLine": "mkdir /tmp/audittest ",
  "executable": "/bin/bash",
  "gid": 0,
  "node": {"name": "worker-1"},
  "pid": 27184,
  "requestUID": "f011c4a3-b20e-44ed-bb91-23e03ae31b3e",
  "resource": {
    "container": "nginx",
    "namespace": "default",
    "pod": "my-pod"
  },
  "syscalls": ["getpid", "execve"],
  "timestamp": "2026-02-16T06:34:53.000Z",
  "uid": 0,
  "version": "spo/v1_alpha"
}
{
  "auditID": "a1b2c3d4-e5f6-7890-abcd-222222222222",
  "cmdLine": "touch /tmp/audittest/demo-file ",
  "executable": "/bin/bash",
```

```

"gid": 0,
"node": {"name": "worker-1"},
"pid": 27274,
"requestUID": "f011c4a3-b20e-44ed-bb91-23e03ae31b3e",
"resource": {
  "container": "nginx",
  "namespace": "default",
  "pod": "my-pod"
},
"syscalls": ["getpid", "execve"],
"timestamp": "2026-02-16T06:35:02.000Z",
"uid": 0,
"version": "spo/v1_alpha"
}

```

7.9.10. Monitor the audit logs

Monitor advanced audit logs from the **json-enricher** container, by streaming pod logs or by reading the audit log file on the node, so you can verify that Advanced Audit Logging is capturing session activity.

The audit log file is specified in the **auditLogPath** field and is written to the file system on the node where the pod is running. To inspect the audit logs, access the node and open the file at the configured path, such as **/tmp/logs/audit1.log**.

Procedure

1. Stream the advanced audit log by using the following command:

```
# kubectl -n openshift-security-profiles logs --since=1m --selector name=spod -c json-enricher --max-log-requests 6 -f
```

2. Identify the node on which the pod is scheduled by using the following command:

```
# kubectl get pod my-pod -o wide
```

3. Access the node by using the following command:

```
$ sudo ssh core@<node_name>
```

4. View the audit log by using the following command:

```
$ cat /tmp/logs/audit1.log
```

Example output

```

{
  "auditID": "a1b2c3d4-e5f6-7890-abcd-111111111111",
  "cmdLine": "mkdir /tmp/audittest ",
  "executable": "/bin/bash",
  "gid": 0,
  "node": {"name": "worker-1"},
  "pid": 27184,
  "requestUID": "f011c4a3-b20e-44ed-bb91-23e03ae31b3e",

```

```

"resource": {
  "container": "nginx",
  "namespace": "default",
  "pod": "my-pod"
},
"syscalls": ["getpid", "execve"],
"timestamp": "2026-02-16T06:34:53.000Z",
"uid": 0,
"version": "spo/v1_alpha"
}
{
  "auditID": "a1b2c3d4-e5f6-7890-abcd-222222222222",
  "cmdLine": "touch /tmp/audittest/demo-file ",
  "executable": "/bin/bash",
  "gid": 0,
  "node": {"name": "worker-1"},
  "pid": 27274,
  "requestUID": "f011c4a3-b20e-44ed-bb91-23e03ae31b3e",
  "resource": {
    "container": "nginx",
    "namespace": "default",
    "pod": "my-pod"
  },
  "syscalls": ["getpid", "execve"],
  "timestamp": "2026-02-16T06:35:02.000Z",
  "uid": 0,
  "version": "spo/v1_alpha"
}

```

7.9.11. Audit node debugging sessions

Enable privileged **seccomp** profiles in CRI-O so Advanced Audit Logging can record activity from **kubectrl debug** and **oc debug** node sessions.

If you are using the CRI-O runtime, you must configure it to allow **seccomp** profiles on privileged containers by adding the **--privileged-seccomp-profile=/var/lib/kubelet/seccomp/operator/profile1.json** flag to your CRI-O runtime configuration.



NOTE

The **--privileged-seccomp-profile** flag is available starting with OpenShift Container Platform 4.20 or later and CRI-O version 1.33 or later.

Procedure

1. SSH into the target node using the following command:

```
# ssh core@<node_ip_address>
```

2. Check to see if the files are there:

```
# ls /var/lib/kubelet/seccomp/operator/
```

Example output

```
# kubelet-config.json profile1.json
```

3. Stop the **kubelet** with the following commands:

```
# systemctl stop kubelet
```

4. Stop CRI-O with the command:

```
# systemctl stop crio
```

5. Set the CRI-O options with the following command:

```
# echo "CRIO_CONFIG_OPTIONS --privileged-seccomp-  
profile=/var/lib/kubelet/seccomp/operator/profile1.json" > /etc/sysconfig/crio
```

6. Now restart the kubelet with this command:

```
# systemctl start kubelet
```

7. Restart CRI-O with this command:

```
# systemctl start crio
```

8. To audit kubectl debug sessions, run the following command:

```
# kubectl debug node/<node_name> -it --image=ubuntu -- bash
```

Create the file on the container for the debugging information.

9. **chroot** to the host with this command:

```
# chroot /host
```

10. Go into the **/tmp** directory with this command:

```
# cd /tmp
```

11. Create an empty file named **demonodedebug** with this command:

```
# touch demonodedebug
```

12. Exit the node with the command:

```
# exit
```

13. To monitor the logs, SSH to a node with this command:

```
$ sudo ssh core@<node_name>
```

14. View the audit log using this command:


```
$ cat /tmp/logs/audit1.log
```

Example output

```
{
  "auditID": "edce381a-998d-4f3f-99d8-d0c4d0c8a613",
  "cmdLine": "touch demonodedebug",
  "executable": "/usr/bin/bash",
  "gid": 0,
  "node": {"name": "worker-1"},
  "pid": 99086,
  "resource": {
    "container": "container-00",
    "namespace": "openshift-security-profiles",
    "pod": "worker-1-debug"
  },
  "syscalls": ["execve", "getpid"],
  "timestamp": "2026-02-12T13:50:11.000Z",
  "uid": 0,
  "version": "spo/v1_alpha"
}
```

7.9.12. Correlate with Kubernetes audit logs

Use the **requestUID** from the Security Profiles Operator (SPO) log to find the corresponding API server log entry, confirming who initiated the session.

Procedure

1. Start the pod by running the following command:

```
$ oc exec my-pod -c nginx -- sh -c "touch /tmp/testfile.txt"
```

2. Identify the node where the pod is running:

```
$ NODE=$(oc get pod my-pod -o jsonpath='{.spec.nodeName}')
```

3. Access the node and check the JSON enriched audit log using the following commands:

```
# oc debug node/$NODE
# chroot /host
# grep "testfile" /tmp/logs/audit1.log | jq .
```

7.9.13. Audit JSON Log Enricher output

For an exec session, the Audit JSON Log Enricher records two entries: the **SPO_EXEC_REQUEST_UID** injection and the command that ran on the pod. Match the shared **UID** value to connect the entries.

1. The first listing is the container runtime wrapper.

```
{
```

```

"auditID": "062e2bd2-xxxx-xxxx-xxxx-57fb39d65a99",
"cmdLine": "env SPO_EXEC_REQUEST_UID=aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9 sh
-c touch /tmp/testfile.txt ",
"executable": "/usr/bin/crun",
"pid": 144966,
"requestUID": "aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9",
"resource": {
  "container": "nginx",
  "namespace": "default",
  "pod": "my-pod"
},
"node": {
  "name": "worker-1"
},
"syscalls": ["execve", "getpid", "clone"],
"timestamp": "2026-02-16T14:01:07.000Z"
}

```

- The second listing is the actual command executed.

```

{
  "auditID": "d586679d-xxxx-xxxx-xxxx-9dc8ab273065",
  "cmdLine": "touch /tmp/testfile.txt ",
  "executable": "/bin/dash",
  "pid": 144968,
  "requestUID": "aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9", // Correlation key
  "resource": {
    "container": "nginx",
    "namespace": "default",
    "pod": "my-pod"
  },
  "node": {
    "name": "worker-1"
  },
  "syscalls": ["execve"],
  "timestamp": "2026-02-16T14:01:07.000Z"
}

```

- You can search the Kubernetes API audit log by using the **requestUID** with the following command:

```
$ oc adm node-logs --role=master --path=kube-apiserver/audit.log | grep request_UID
```

7.9.14. Kubernetes API audit log output

The Kubernetes API audit log output is YAML. It includes the **SPO_EXEC_REQUEST_UID** field that provides the correlation key for searching the Advanced Audit Logging output.

```

{
  "kind": "Event",
  "apiVersion": "audit.k8s.io/v1",
  "level": "Metadata",
  "auditID": "4d434cd4-xxxx-xxxx-xxxx-2d9aa46292ce",
  "stage": "ResponseComplete",

```

```

    "requestURI": "/api/v1/namespaces/test-namespace/pods/test-pod/exec?command=sh&command=-
c&command=touch+/tmp/testfile.txt&container=nginx",
    "verb": "create",
    "user": {
      "username": "kube:admin",
      "groups": ["system:cluster-admins", "system:authenticated"]
    },
    "sourceIPs": ["xxx.xxx.xxx.xxx"],
    "userAgent": "oc/4.19.0 (linux/amd64)",
    "objectRef": {
      "resource": "pods",
      "namespace": "default",
      "name": "my-pod",
      "subresource": "exec"
    },
    "responseStatus": {
      "code": 101
    },
    "requestReceivedTimestamp": "2026-02-16T14:01:06.056518Z",
    "annotations": {
      "authorization.k8s.io/decision": "allow",
      "authorization.k8s.io/reason": "RBAC: allowed by ClusterRoleBinding...",
      "execmetadata.spo.io/SPO_EXEC_REQUEST_UID": "aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9"
    }
  }

```

The final field in this example, **SPO_EXEC_REQUEST_UID** is the correlation key.

7.9.15. Correlation key

You can build a complete audit trail by matching correlation keys across logs. The **requestUID** field in Audit JSON Enricher logs matches the **annotations.execmetadata.spo.io/SPO_EXEC_REQUEST_UID** annotation in the Kubernetes API audit log.

For example, the API audit log can show that **kube:admin** ran a command, and the SPO JSON Enricher log can show the system-level action, such as **touch /tmp/testfile.txt**.

```

aec3e0e1-xxxx-xxxx-xxxx-a7c58241f1a9

```

7.9.16. Correlating with API Server Audit Log

By default, when you use the **kubectl exec** command to access a pod or container, Kubernetes does not pass the user's authentication details into that session's environment. This means the Audit JSON log enricher cannot provide audit information for **exec** commands. The **UID** or **GID** shown, maps to the system user. In most cases this would be the root user.

To address this, the Audit JSON log enricher relies on mutating webhooks (**execmetadata.spo.io** and **nodebuggingpod.spo.io**). The webhook injects the **exec requestUID** as an environment variable into the **exec** session. When the administrator enables audit logging on the API server, the webhooks add the **SPO_EXEC_REQUEST_UID** audit annotation. The API server audit log contains this information. This request ID is also available in the JSON lines produced by the Audit JSON log enricher, specifically within the **requestUID** field.

By default, these webhooks are enabled for all namespaces with the Audit JSON log enricher enabled. To reduce the scope of this webhook you can disable it for certain namespaces.

Procedure

1. Edit the spod security profile by running the following command:

```
$ oc edit spod spod -n openshift-security-profiles
```

2. Add **webhookOptions** to the **spec**. Locate the **spec** section and add the following **webhookOptions** block to instruct the webhook to apply to a specific namespace.

```
spec:
  webhookOptions:
    - name: execmetadata.spo.io # or nodedebuggingpod.spo.io
      namespaceSelector:
        #...add rules
```

After saving your changes, the Operator reconfigures the mutating webhook, allowing request details to be passed into **oc exec** sessions cluster-wide.

7.9.17. Use the mutating webhook

Use the mutating webhook so Advanced Audit Logging can correlate cluster users with actions in **oc exec**, **oc rsh**, and **oc debug** sessions.

The mutating webhook injects the **SPO_EXEC_REQUEST_UID** environment variable into your exec request. If a container already defines a variable with that name, the injected value overrides it for the exec session.

When you use **kubectl debug node/<node_name>**, the **nodedebuggingpod.spo.io** webhook injects **SPO_EXEC_REQUEST_UID** into the debug pod.

7.9.17.1. The debug pod

This webhook primarily identifies kubectl debug pods by the label **app.kubernetes.io/managed-by: kubectl-debug**, which is added by the kubectl client. Because this label might vary across different Kubernetes client implementations, such as how **oc debug** in OpenShift Container Platform uses **debug.openshift.io/managed-by: oc-debug**, you might need to configure additional **webhookOptions** to ensure the webhook catches all relevant debug pods.

For example, to add oc debug pods, use the following **yaml**:

```
# ... (rest of your spod configuration)
spec:
  webhookOptions:
    - name: nodedebuggingpodmetada.spo.io
      objectSelector:
        matchLabels: # Use matchLabels for exact matching
          debug.openshift.io/managed-by: "oc-debug"
# ... (other webhook rule details such as rules, clientConfig, etc.)
```

7.9.18. Disabling Advanced Audit Logging

You can disable advanced audit logging and revert all configurations by deleting the test pod, the **seccompProfile**, the JSON Log Enricher and resetting all **spod** pod options.

Procedure

1. Delete the test pod with the following command:

```
oc delete pod my-pod
```

2. Delete the **seccompProfile** using this command:

```
oc delete seccompprofile profile1 -n openshift-security-profiles
```

3. Disable the JSON Log Enricher and reset all options:

```
oc patch spod spod -n openshift-security-profiles --type merge -p '{ "spec": {
  "enableJsonEnricher": false, "jsonEnricherOptions": { "auditLogPath": "", "auditLogMaxSize":
  0, "auditLogMaxBackups": 0, "auditLogMaxAge": 0, "auditLogIntervalSeconds": 0 } } }'
```

4. Wait for **spod** pods to restart. Run the following command to check:

```
oc get pods -n openshift-security-profiles -l name=spod -w
```

Wait until all **spod** pods show **Running**.

5. Revert the ConfigMap volume patch with the following command:

```
oc patch configmap security-profiles-operator-profile -n openshift-security-profiles --type
merge -p '{"data":{"patch-volume-source.json":""}}'
```

6. Verify that the configuration has been successfully updated:

```
oc get spod spod -n openshift-security-profiles -o jsonpath='{.spec.enableJsonEnricher}'
```

Expected output:

```
false
```

7.9.19. Additional resources

- [About security profiles](#)
- [Installing the Security Profiles Operator](#)
- [Migration procedure](#)
- [Use the log enricher](#)
- [Troubleshooting the Security Profiles Operator](#)
- [Uninstalling SPO](#)

7.10. TROUBLESHOOTING THE SECURITY PROFILES OPERATOR

Troubleshoot the Security Profiles Operator to diagnose a problem or provide information in a bug report.

7.10.1. Inspecting seccomp profiles

Corrupted **seccomp** profiles can disrupt your workloads. Ensure that the user cannot abuse the system by not allowing other workloads to map any part of the path **/var/lib/kubelet/seccomp/operator**.

Procedure

1. Confirm that the profile is reconciled by running the following command:

```
$ oc -n openshift-security-profiles logs openshift-security-profiles-<id>
```

Example output

```
I1019 19:34:14.942464    1 main.go:90] setup "msg"="starting openshift-security-profiles"
"buildDate"="2020-10-19T19:31:24Z" "compiler"="gc"
"gitCommit"="a3ef0e1ea6405092268c18f240b62015c247dd9d" "gitTreeState"="dirty"
"goVersion"="go1.15.1" "platform"="linux/amd64" "version"="0.2.0-dev"
I1019 19:34:15.348389    1 listener.go:44] controller-runtime/metrics "msg"="metrics server
is starting to listen" "addr"=":8080"
I1019 19:34:15.349076    1 main.go:126] setup "msg"="starting manager"
I1019 19:34:15.349449    1 internal.go:391] controller-runtime/manager "msg"="starting
metrics server" "path"="/metrics"
I1019 19:34:15.350201    1 controller.go:142] controller "msg"="Starting EventSource"
"controller"="profile" "reconcilerGroup"="security-profiles-operator.x-k8s.io"
"reconcilerKind"="SeccompProfile" "source"={"Type":{"metadata":
{"creationTimestamp":null},"spec":{"defaultAction":""}}}
I1019 19:34:15.450674    1 controller.go:149] controller "msg"="Starting Controller"
"controller"="profile" "reconcilerGroup"="security-profiles-operator.x-k8s.io"
"reconcilerKind"="SeccompProfile"
I1019 19:34:15.450757    1 controller.go:176] controller "msg"="Starting workers"
"controller"="profile" "reconcilerGroup"="security-profiles-operator.x-k8s.io"
"reconcilerKind"="SeccompProfile" "worker count"=1
I1019 19:34:15.453102    1 profile.go:148] profile "msg"="Reconciled profile from
SeccompProfile" "namespace"="openshift-security-profiles" "profile"="nginx-1.19.1"
"name"="nginx-1.19.1" "resource version"="728"
I1019 19:34:15.453618    1 profile.go:148] profile "msg"="Reconciled profile from
SeccompProfile" "namespace"="openshift-security-profiles" "profile"="openshift-security-
profiles" "name"="openshift-security-profiles" "resource version"="729"
```

2. Confirm that the **seccomp** profiles are saved into the correct path by running the following command:

```
$ oc exec -t -n openshift-security-profiles openshift-security-profiles-<id> \
-- ls /var/lib/kubelet/seccomp/operator/my-namespace/my-workload
```

Example output

```
profile-block.json
profile-complain.json
```

7.11. UNINSTALLING THE SECURITY PROFILES OPERATOR

You can remove the Security Profiles Operator from your cluster by using the OpenShift Container Platform web console.

7.11.1. Uninstall the Security Profiles Operator by using the web console

To remove the Security Profiles Operator, you must first delete the **seccomp** and SELinux profiles. After the profiles are removed, you can then remove the Operator and its namespace by deleting the **openshift-security-profiles** project.

Prerequisites

- You have access to the web console as a user with **cluster-admin** privileges.
- The Security Profiles Operator is installed.

Procedure

1. Navigate to the **Ecosystem → Installed Operators** page.
2. Delete all **seccomp** profiles, SELinux profiles, and webhook configurations.
3. Switch to the **Administration → Ecosystem → Installed Operators** page.

4. Click the Options menu  on the **Security Profiles Operator** entry.

5. Select **Uninstall Operator**.

6. Switch to the **Home → Projects** page.

7. Search for **security profiles**.

8. Click the Options menu  next to the **openshift-security-profiles** project.

9. Select **Delete Project**.

- a. Enter **openshift-security-profiles** in the dialog box.

- b. Click **Delete**.

10. Delete the **MutatingWebhookConfiguration** object by running the following command:

```
$ oc delete MutatingWebhookConfiguration spo-mutating-webhook-configuration
```

CHAPTER 8. NBDE TANG SERVER OPERATOR

8.1. NBDE TANG SERVER OPERATOR OVERVIEW

Network-bound Disk Encryption (NBDE) provides an automated unlocking of LUKS-encrypted volumes using one or more dedicated network-binding servers. The client side of NBDE is called the Clevis decryption policy framework and the server side is represented by Tang.

The NBDE Tang Server Operator allows the automation of deployments of one or several Tang servers in the OpenShift Container Platform (OCP) environment.

8.2. NBDE TANG SERVER OPERATOR RELEASE NOTES

The following release notes track the development of the NBDE Tang Server Operator in OpenShift Container Platform.

[RHEA-2023:7491](#)

The NBDE Tang Server Operator 1.0 has been released in the Red Hat OpenShift Enterprise 4 catalog.

[RHEA-2024:0854](#)

The NBDE Tang Server Operator 1.0.1 has been moved from the **alpha** channel to the **stable** channel.

[RHBA-2024:8681](#)

The 1.0.2 update contains fixes that increase the Container Health Index of containers deployed with the NBDE Tang Server Operator to the highest grade.

[RHEA-2024:10970](#)

The 1.0.3 update contains changes that re-increase the Container Health Index to the highest grade.

[RHBA-2025:0663](#)

The NBDE Tang Server Operator 1.1 includes the **golang** package version 1.23.2 and the **golang.org/x/net/html** package version 0.33.0. The updates improve the Container Health Index.

[RHBA-2025:4453](#)

The 1.1.1 update provides a fix for [CVE-2025-22866](#).

8.3. UNDERSTANDING THE NBDE TANG SERVER OPERATOR

You can use the NBDE Tang Server Operator to automate the deployment of a Tang server in an OpenShift Container Platform cluster that requires Network Bound Disk Encryption (NBDE) internally, leveraging the tools that OpenShift Container Platform provides to achieve this automation.

The NBDE Tang Server Operator simplifies the installation process and uses native features provided by the OpenShift Container Platform environment, such as multi-replica deployment, scaling, traffic load balancing, and so on. The Operator also provides automation of certain operations that are error-prone when you perform them manually, for example:

- server deployment and configuration
- key rotation
- hidden keys deletion

The NBDE Tang Server Operator is implemented using the Operator SDK and allows the deployment of one or more Tang servers in OpenShift through custom resource definitions (CRDs).

8.3.1. Additional resources

- [Tang-Operator: Providing NBDE in OpenShift](#)
- [Tang Server Operator](#)
- [Configuring automated unlocking of encrypted volumes using policy-based decryption](#)

8.4. INSTALLING THE NBDE TANG SERVER OPERATOR

You can install the NBDE Tang Operator either by using the web console or through the **oc** command from CLI.

8.4.1. Installing the NBDE Tang Server Operator using the web console

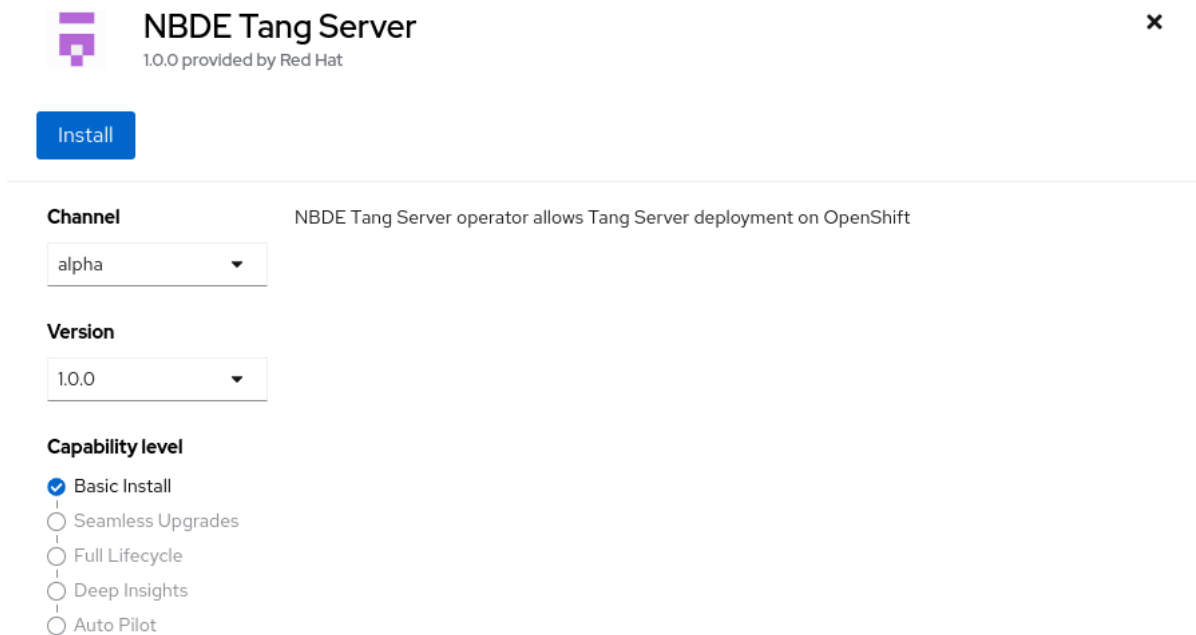
You can install the NBDE Tang Server Operator from the software catalog using the web console.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Software Catalog**.
2. Search for the NBDE Tang Server Operator:



NBDE Tang Server
1.0.0 provided by Red Hat

Install

Channel NBDE Tang Server operator allows Tang Server deployment on OpenShift
alpha

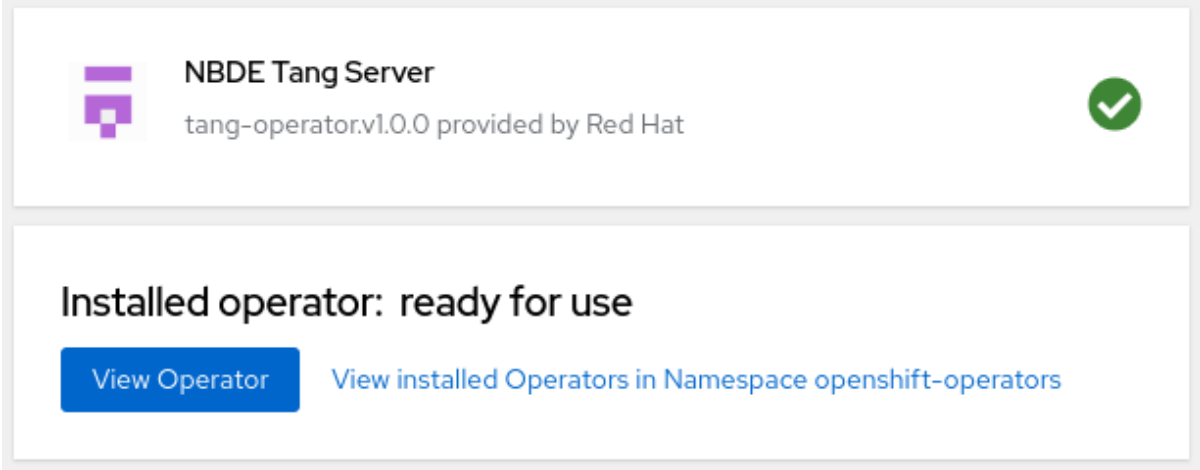
Version
1.0.0

Capability level

- ☒ Basic Install
- ☐ Seamless Upgrades
- ☐ Full Lifecycle
- ☐ Deep Insights
- ☐ Auto Pilot

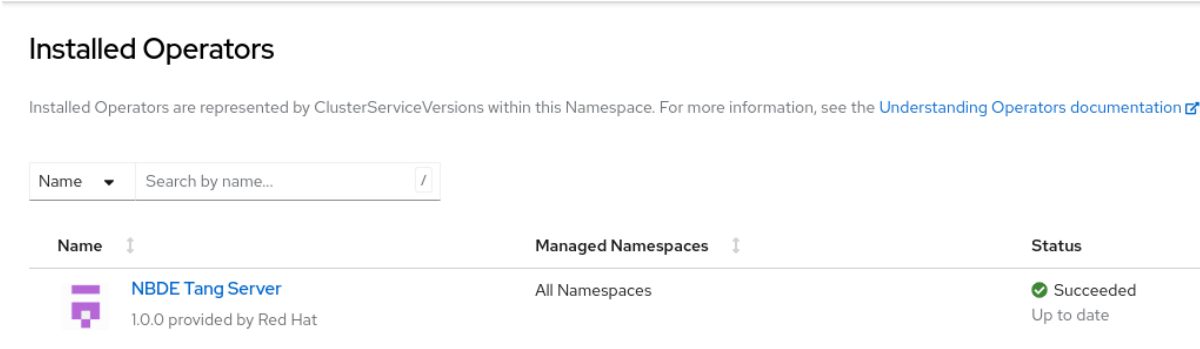
3. Click **Install**.
4. On the **Operator Installation** screen, keep the **Update channel**, **Version**, **Installation mode**, **Installed Namespace**, and **Update approval** fields on the default values.

5. After you confirm the installation options by clicking **Install**, the console displays the installation confirmation.



Verification

1. Navigate to the **Ecosystem → Installed Operators** page.
2. Check that the NBDE Tang Server Operator is installed and its status is **Succeeded**.



8.4.2. Installing the NBDE Tang Server Operator using CLI

You can install the NBDE Tang Server Operator from the software catalog using the CLI.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Use the following command to list available Operators in the software catalog, and limit the output to Tang-related results:

```
$ oc get packagemanifests -n openshift-marketplace | grep tang
```

Example output

```
tang-operator      Red Hat
```

In this case, the corresponding packagemanifest name is **tang-operator**.

2. Create a **Subscription** object YAML file to subscribe a namespace to the NBDE Tang Server Operator, for example, **tang-operator.yaml**:

Example subscription YAML for tang-operator

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: tang-operator
  namespace: openshift-operators
spec:
  channel: stable ❶
  installPlanApproval: Automatic
  name: tang-operator ❷
  source: redhat-operators ❸
  sourceNamespace: openshift-marketplace ❹
```

- ❶ Specify the channel name from where you want to subscribe the Operator.
- ❷ Specify the name of the Operator to subscribe to.
- ❸ Specify the name of the CatalogSource that provides the Operator.
- ❹ The namespace of the CatalogSource. Use **openshift-marketplace** for the default software catalog sources.

3. Apply the **Subscription** to the cluster:

```
$ oc apply -f tang-operator.yaml
```

Verification

- Check that the NBDE Tang Server Operator controller runs in the **openshift-operators** namespace:

```
$ oc -n openshift-operators get pods
```

Example output

```
NAME                                READY STATUS RESTARTS AGE
tang-operator-controller-manager-694b754bd6-4zk7x 2/2 Running 0      12s
```

8.5. CONFIGURING AND MANAGING TANG SERVERS USING THE NBDE TANG SERVER OPERATOR

With the NBDE Tang Server Operator, you can deploy and quickly configure Tang servers. On the deployed Tang servers, you can list existing keys and rotate them.

8.5.1. Deploying a Tang server using the NBDE Tang Server Operator

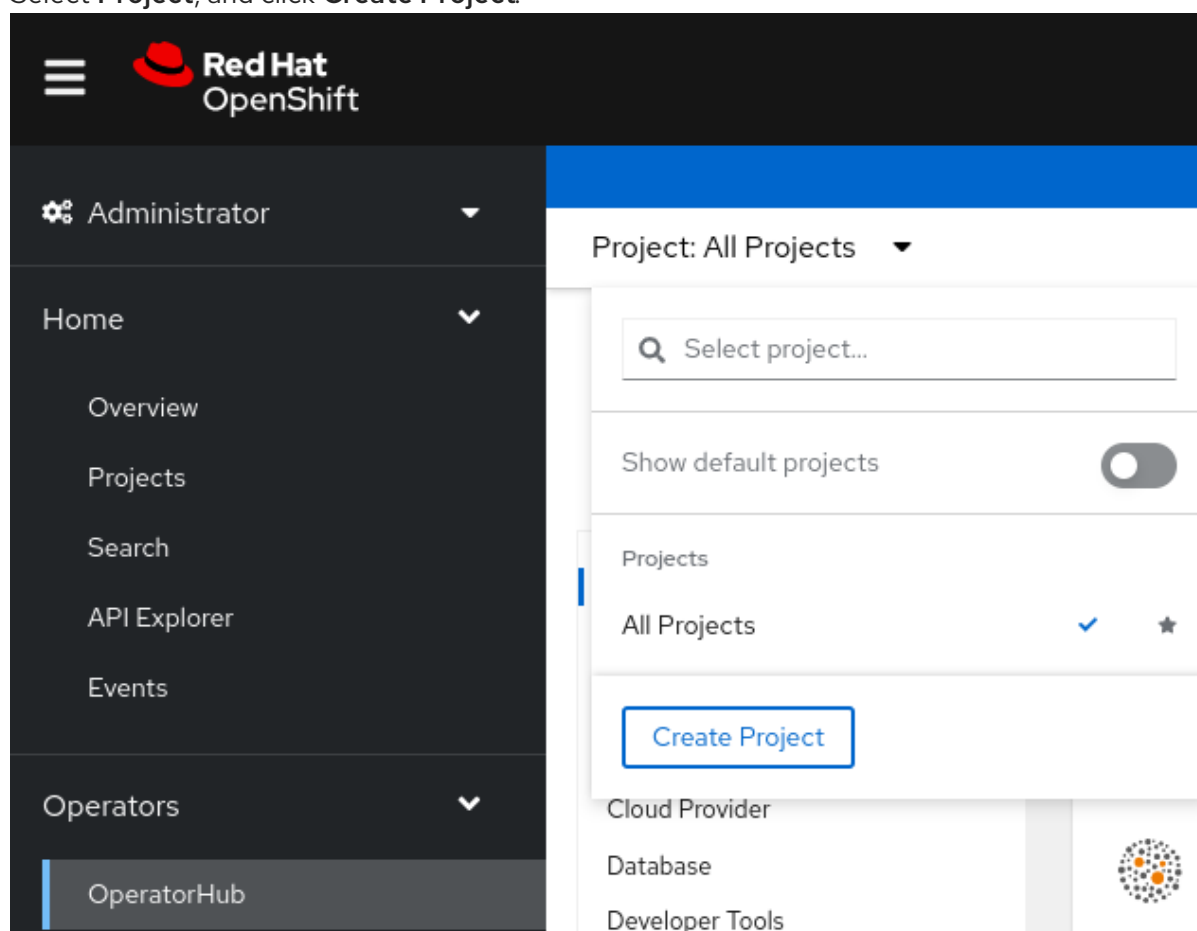
You can deploy and quickly configure one or more Tang servers using the NBDE Tang Server Operator in the web console.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.
- You have installed the NBDE Tang Server Operator on your OCP cluster.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem** → **Software Catalog**.
2. Select **Project**, and click **Create Project**:



3. On the **Create Project** page, fill in the required information, for example:

Create Project

An OpenShift project is an alternative representation of a Kubernetes namespace.

[Learn more about working with projects](#)

Name * ?

nbde

Display name

Network Bound Disk Encryption

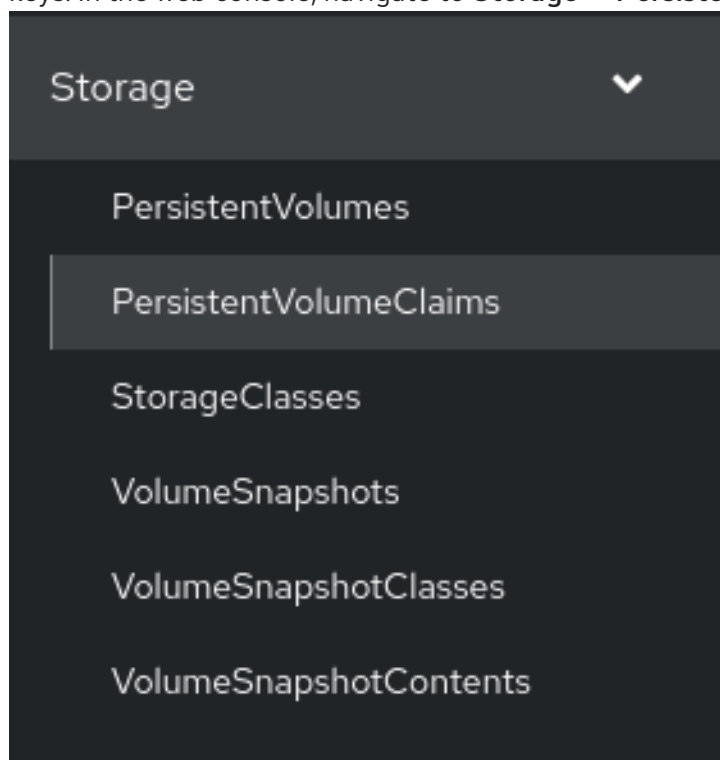
Description

Network Bound Disk Encryption

Cancel

Create

- Click **Create**.
- NBDE Tang Server replicas require a Persistent Volume Claim (PVC) for storing encryption keys. In the web console, navigate to **Storage → PersistentVolumeClaims**:



- On the following **PersistentVolumeClaims** screen, click **Create PersistentVolumeClaim**.
- On the **Create PersistentVolumeClaim** page, select a storage that fits your deployment scenario. Consider how often you want to rotate the encryption keys. Name your PVC and choose the claimed storage capacity, for example:

Project: nbde ▼

Create PersistentVolumeClaim

StorageClass

 standard-csi

StorageClass for the new claim

PersistentVolumeClaim name *

tang-server-pvc

A unique name for the storage claim within the project

Access mode *

☒ Single user (RWO) ☐ Shared access (RWX) ☐ Read only (ROX)

Access mode is set by StorageClass and cannot be changed

Size *

▼

Desired storage capacity

☐ Use label selectors to request storage

PersistentVolume resources that match all label selectors will be considered for binding.

Volume mode *

☒ Filesystem ☐ Block

- Navigate to **Ecosystem → Installed Operators**, and click **NBDE Tang Server**.
- Click **Create instance**.

Project: openshift-operators ▼

[Installed Operators](#) > Operator details



NBDE Tang Server
1.0.0 provided by Red Hat

[Details](#) [YAML](#) [Subscription](#) [Events](#) [Tang Server](#)

Provided APIs

TS Tang Server

TangServer is the Schema for the tangservers API

[+ Create instance](#)

10. On the **Create TangServer** page, choose the name of the Tang Server instance, amount of replicas, and specify the name of the previously created Persistent Volume Claim, for example:

Project: nbde ▾

Create TangServer

Create by completing the form. Default values may be provided by the Operator authors.

Configure via: ☒ Form view ☐ YAML view

Note: Some fields may not be represented in this form view. Please select "YAML view" for full control.

Name *

tangserver

Labels

app=frontend

Amount of replicas to launch *

1

Replicas is the Tang Server amount to bring up

Health Script to execute

/usr/bin/tangd-health-check

HealthScript is the script to run for healthiness/readiness

Hidden Keys contains a list with the keys (with sha1 or sha256) to hide

HiddenKeys

Image of Container to deploy

registry.redhat.io/rhel9/tang

Image is the base container image of the TangServer to use

Key Path

/var/db/tang

KeyPath is field of TangServer. It allows to specify the path where keys will be generated

Refresh Interval to update key status

KeyRefreshInterval

Persistent Volume Claim to attach to (default:tangserver-pvc)

tangserver-pvc

11. After you enter the required values a change settings that differ from the default values in your scenario, click **Create**.

8.5.2. Rotating keys using the NBDE Tang Server Operator

With the NBDE Tang Server Operator, you also can rotate your Tang server keys. The precise interval at which you should rotate them depends on your application, key sizes, and institutional policy.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.
- You deployed a Tang server using the NBDE Tang Server Operator on your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. List the existing keys on your Tang server, for example:

```
$ oc -n nbde describe tangserver
```

Example output

```
...
Status:
  Active Keys:
  File Name:  QS82aXnPKA4XpfHr3umbA0r2iTbRcpWQ0VI2Qdhi6xg
  Generated:  2022-02-08 15:44:17.030090484 +0000
  sha1:       PvYQKtrTuYsMV2AomUeHrUWkCGg
  sha256:     QS82aXnPKA4XpfHr3umbA0r2iTbRcpWQ0VI2Qdhi6xg
...
```

2. Create a YAML file for moving your active keys to hidden keys, for example, **minimal-keyretrieve-rotate-tangserver.yaml**:

Example key-rotation YAML for tang-operator

```
apiVersion: daemons.redhat.com/v1alpha1
kind: TangServer
metadata:
  name: tangserver
  namespace: nbde
  finalizers:
    - finalizer.daemons.tangserver.redhat.com
spec:
  replicas: 1
  hiddenKeys:
    - sha1: "PvYQKtrTuYsMV2AomUeHrUWkCGg" ❶
```

- ❶ Specify the SHA-1 thumbprint of your active key to rotate it.

3. Apply the YAML file:

```
$ oc apply -f minimal-keyretrieve-rotate-tangserver.yaml
```

Verification

1. After a certain amount of time depending on your configuration, check that the previous **activeKey** value is the new **hiddenKey** value and the **activeKey** key file is newly generated, for example:

```
$ oc -n nbde describe tangserver
```

Example output

```
...
Spec:
  Hidden Keys:
    sha1:  PvYQKtrTuYsMV2AomUeHrUWkCGg
  Replicas: 1
Status:
  Active Keys:
    File Name: T-0wx1HusMeWx4WMOk4eK97Q5u4dY5tamdDs7_ughnY.jwk
    Generated: 2023-10-25 15:38:18.134939752 +0000
    sha1:    vVxkNCNq7gygeeA9zrHrbc3_NZ4
    sha256:  T-0wx1HusMeWx4WMOk4eK97Q5u4dY5tamdDs7_ughnY
  Hidden Keys:
    File Name: .QS82aXnPKA4XpfHr3umbA0r2iTbRcpWQ0VI2Qdhi6xg.jwk
    Generated: 2023-10-25 15:37:29.126928965 +0000
    Hidden:    2023-10-25 15:38:13.515467436 +0000
    sha1:      PvYQKtrTuYsMV2AomUeHrUWkCGg
    sha256:    QS82aXnPKA4XpfHr3umbA0r2iTbRcpWQ0VI2Qdhi6xg
...
```

8.5.3. Deleting hidden keys with the NBDE Tang Server Operator

After you rotate your Tang server keys, the previously active keys become hidden and are no longer advertised by the Tang instance. You can use the NBDE Tang Server Operator to remove encryption keys no longer used.

WARNING

Do not remove any hidden keys unless you are sure that all bound Clevis clients already use new keys.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.
- You deployed a Tang server using the NBDE Tang Server Operator on your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. List the existing keys on your Tang server, for example:

```
$ oc -n nbde describe tangserver
```

Example output

```
-
```

```
...
Status:
  Active Keys:
  File Name:  PvYQKtrTuYsMV2AomUeHrUWkCGg.jwk
  Generated:  2022-02-08 15:44:17.030090484 +0000
  sha1:       PvYQKtrTuYsMV2AomUeHrUWkCGg
  sha256:     QS82aXnPKA4XpfHr3umbA0r2iTbRcpWQ0VI2Qdhi6xg
...
```

2. Create a YAML file for removing all hidden keys, for example, **hidden-keys-deletion-tangserver.yaml**:

Example hidden-keys-deletion YAML for tang-operator

```
apiVersion: daemons.redhat.com/v1alpha1
kind: TangServer
metadata:
  name: tangserver
  namespace: nbde
  finalizers:
    - finalizer.daemons.tangserver.redhat.com
spec:
  replicas: 1
  hiddenKeys: [] 1
```

- 1 The empty array as the value of the **hiddenKeys** entry indicates you want to preserve no hidden keys on your Tang server.

3. Apply the YAML file:

```
$ oc apply -f hidden-keys-deletion-tangserver.yaml
```

Verification

1. After a certain amount of time depending on your configuration, check that the previous active key still exists, but no hidden key is available, for example:

```
$ oc -n nbde describe tangserver
```

Example output

```
...
Spec:
  Hidden Keys:
    sha1:  PvYQKtrTuYsMV2AomUeHrUWkCGg
  Replicas: 1
Status:
  Active Keys:
    File Name:  T-0wx1HusMeWx4WMOk4eK97Q5u4dY5tamdDs7_ughnY.jwk
    Generated:  2023-10-25 15:38:18.134939752 +0000
    sha1:       vVxkNCNq7gygeeA9zrHrbc3_NZ4
    sha256:     T-0wx1HusMeWx4WMOk4eK97Q5u4dY5tamdDs7_ughnY
Status:
```

```
Ready:          1
Running:        1
Service External URL: http://35.222.247.84:7500/adv
Tang Server Error: No
Events:
...
```

8.6. IDENTIFYING URL OF A TANG SERVER DEPLOYED WITH THE NBDE TANG SERVER OPERATOR

Before you can configure your Clevis clients to use encryption keys advertised by your Tang servers, you must identify the URLs of the servers.

8.6.1. Identifying URL of the NBDE Tang Server Operator using the web console

You can identify the URLs of Tang servers deployed with the NBDE Tang Server Operator from the software catalog by using the OpenShift Container Platform web console. After you identify the URLs, you use the **clevis luks bind** command on your clients containing LUKS-encrypted volumes that you want to unlock automatically by using keys advertised by the Tang servers. See the [Configuring manual enrollment of LUKS-encrypted volumes](#) section in the RHEL 9 Security hardening document for detailed steps describing the configuration of clients with Clevis.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.
- You deployed a Tang server by using the NBDE Tang Server Operator on your OpenShift cluster.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Ecosystem → Installed Operators → Tang Server**.
2. On the NBDE Tang Server Operator details page, select **Tang Server**.

The screenshot shows the Red Hat OpenShift web console interface. On the left is a dark sidebar with navigation links: Administrator, Home, Operators, OperatorHub, Installed Operators (selected), Workloads, Networking, Storage, and Builds. The main content area has a top bar with 'Project: default' and a breadcrumb 'Installed Operators > Operator details'. Below this, the 'NBDE Tang Server' is listed as '1.0.5 provided by Red Hat'. A tabbed interface shows 'Details', 'YAML', 'Subscription', 'Events', and 'Tang Server' (active). Under the 'TangServers' section, there's a filter 'Show operands in: All namespaces (selected) / Current namespace only' and a search bar. A table lists the TangServer: 'tangserver-mini' with a 'TS' icon and 'Kind' 'TangServer'.

3. The list of Tang servers deployed and available for your cluster appears. Click the name of the Tang server you want to bind with a Clevis client.
4. The web console displays an overview of the selected Tang server. You can find the URL of your Tang server in the **Tang Server External Url** section of the screen:

The screenshot shows the 'Tang Server overview' page. At the top, it says 'Project: nbde'. Below is a horizontal tab bar with 'Details' (active), 'YAML', 'Resources', and 'Events'. The main heading is 'Tang Server overview'. Below this, there are three sections: 'Name' with the value 'tangserver-mini', 'Namespace' with a green 'NS' icon and the value 'nbde', and 'Labels' with the text 'No labels'.

NO labels

Annotations

1 annotation 

Created at

 Oct 30, 2023, 11:05 AM

Owner

No owner

Tang Server External Url

`http://34.28.173.205:7500/adv`

In this example, the URL of the Tang server is **http://34.28.173.205:7500**.

Verification

- You can check that the Tang server is advertising by using **curl**, **wget**, or similar tools, for example:

```
$ curl 2> /dev/null http://34.28.173.205:7500/adv | jq
```

Example output

```
{
  "payload": "eyJrZXlzlj...eSJdfV19",
  "protected": "eyJhbGciOiJFUzUxMlslmN0eSI6Imp3ay1zZXQranNvbiJ9",
  "signature": "AUB0qSFx0FJLeTU...aV_GYWIDx50vCXKNyMMCRx"
}
```

8.6.2. Identifying URL of the NBDE Tang Server Operator using CLI

You can identify the URLs of Tang servers deployed with the NBDE Tang Server Operator from the software catalog by using the CLI. After you identify the URLs, you use the **clevis luks bind** command on your clients containing LUKS-encrypted volumes that you want to unlock automatically by using keys

advertised by the Tang servers. See the [Configuring manual enrollment of LUKS-encrypted volumes](#) section in the RHEL 9 Security hardening document for detailed steps describing the configuration of clients with Clevis.

Prerequisites

- You must have **cluster-admin** privileges on an OpenShift Container Platform cluster.
- You have installed the OpenShift CLI (**oc**).
- You deployed a Tang server by using the NBDE Tang Server Operator on your OpenShift cluster.

Procedure

1. List details about your Tang server, for example:

```
$ oc -n nbde describe tangserver
```

Example output

```
...
Spec:
...
Status:
  Ready:          1
  Running:        1
  Service External URL: http://34.28.173.205:7500/adv
  Tang Server Error: No
Events:
...
```

2. Use the value of the **Service External URL:** item without the **/adv** part. In this example, the URL of the Tang server is **http://34.28.173.205:7500**.

Verification

- You can check that the Tang server is advertising by using **curl**, **wget**, or similar tools, for example:

```
$ curl 2> /dev/null http://34.28.173.205:7500/adv | jq
```

Example output

```
{
  "payload": "eyJrZXlzlj...eSJdfV19",
  "protected": "eyJhbGciOiJIUzUxMiIsImN0eSI6ImUyZm9udGVzZXQranNvbiJ9",
  "signature": "AUB0qSFx0FJLeTU...aV_GYWIDx50vCXKNyMMCRx"
}
```

8.6.3. Additional resources

- [Configuring manual enrollment of LUKS-encrypted volumes](#)

CHAPTER 9. UNDERSTANDING SECRETS MANAGEMENT IN OPENSIFT CONTAINER PLATFORM

Secret management tools can be used to automate the lifecycle of sensitive data, such as passwords, private files, and certificates, by providing a centralized system to control and monitor access. This approach enhances security by limiting the uncontrolled spread of secrets and enables automation for the entire secret lifecycle, including updates, expiration, and removal.

OpenShift Container Platform uses a flexible Operator and plugin design to decouple your workloads from external secret managers, ensuring you are not locked into a single vendor. In this model, the Operator acts as an intermediary, while a vendor-specific plugin manages communication between the cluster and the external storage. This allows applications to access secrets without needing to know the details of where or how they are stored.

9.1. SECRETS MANAGEMENT OPERATORS IN OPENSIFT CONTAINER PLATFORM

OpenShift Container Platform offers a suite of supported Operators designed to secure and automate the management of sensitive data, such as external credentials and digital certificates. Each secrets management Operator provides quick starts and sample YAML manifests to streamline the onboarding process. These tools simplify installation and deployment, and help you build complex custom resources by using pre-defined YAML snippets. The following list details the key Operators available for these tasks:

- **Secrets Store CSI driver:** Enables Kubernetes to connect to external systems, and mount credentials from the external system into an application workload.
- **External Secrets Operator for Red Hat OpenShift** Retrieves credentials stored in external management systems and makes them available within OpenShift Container Platform as standard Kubernetes Secrets.
- **cert-manager Operator for Red Hat OpenShift** Manages the lifecycle of digital certificates that are used by applications running on OpenShift Container Platform by automating the process of issuance and renewal.

9.2. SECRETS MANAGEMENT USE CASES

Using secrets management tools with other Red Hat products can protect sensitive data across your OpenShift Container Platform cluster. You can integrate secrets management Operators with other OpenShift Container Platform components to securely manage, automate, and consume credentials across various infrastructure and application workflows.

9.2.1. External Secrets Operator for Red Hat OpenShift use cases

You can integrate the External Secrets Operator with other OpenShift Container Platform components to securely manage and inject credentials. Learn how to apply External Secrets Operator in real-world deployment strategies, by reviewing the following example.

Securing Red Hat OpenShift GitOps by using External Secrets Operator short-lived tokens

To reduce the security risk of compromised credentials, you can configure the External Secrets Operator to generate short-lived tokens. Red Hat OpenShift GitOps can then use these temporary tokens to securely authenticate when accessing GitHub repositories. You can refer to an example of the integration in the External Secrets Operator and GitOps demonstration.

Additional resources

- [External Secrets Operator and GitOps demonstration](#)
- [Zero trust GitOps: Build a secure, secretless GitOps pipeline](#)

Additional resources

- [Secrets Store Container Storage Interface Driver Operator](#)
- [External Secrets Operator for Red Hat OpenShift](#)
- [cert-manager Operator for Red Hat OpenShift](#)

CHAPTER 10. CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

10.1. CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT OVERVIEW

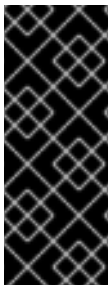
The cert-manager Operator for Red Hat OpenShift is a cluster-wide service that provides application certificate lifecycle management. The cert-manager Operator for Red Hat OpenShift allows you to integrate with external certificate authorities and provides certificate provisioning, renewal, and retirement.

10.1.1. About the cert-manager Operator for Red Hat OpenShift

The **cert-manager** project introduces certificate authorities and certificates as resource types in the Kubernetes API, which makes it possible to provide certificates on-demand to developers working within your cluster. The cert-manager Operator for Red Hat OpenShift provides a supported way to integrate **cert-manager** into your OpenShift Container Platform cluster.

The cert-manager Operator for Red Hat OpenShift provides the following features:

- Support for integrating with external certificate authorities
- Tools to manage certificates
- Ability for developers to self-serve certificates
- Automatic certificate renewal



IMPORTANT

Do not attempt to use both cert-manager Operator for Red Hat OpenShift for OpenShift Container Platform and the community cert-manager Operator at the same time in your cluster.

Also, you should not install cert-manager Operator for Red Hat OpenShift for OpenShift Container Platform in multiple namespaces within a single OpenShift cluster.

10.1.2. cert-manager Operator for Red Hat OpenShift issuer providers

To configure certificate authorities for your cluster, review the issuer providers offered with the cert-manager Operator for Red Hat OpenShift. You can use the following issuer types to automate certificate validation and issuance:

- Automated Certificate Management Environment (ACME)
- Certificate Authority (CA)
- Self-signed
- Vault
- Venafi
- Nokia NetGuard Certificate Manager (NCM)

- Google Cloud Certificate Authority Service (Google CAS)



NOTE

OpenShift Container Platform does not test all factors associated with third-party cert-manager Operator for Red Hat OpenShift provider functionality. For more information about third-party support, see "OpenShift Container Platform third-party support policy" in Additional resources.

10.1.3. Certificate request methods

To obtain certificates for your workloads, choose a request method supported by the cert-manager Operator for Red Hat OpenShift. You can select the approach that fits your operational requirements and automation workflow.

There are two ways to request a certificate using the cert-manager Operator for Red Hat OpenShift:

Using the `cert-manager.io/CertificateRequest` object

With this method a service developer creates a **CertificateRequest** object with a valid **issuerRef** pointing to a configured issuer (configured by a service infrastructure administrator). A service infrastructure administrator then accepts or denies the certificate request. Only accepted certificate requests create a corresponding certificate.

Using the `cert-manager.io/Certificate` object

With this method, a service developer creates a **Certificate** object with a valid **issuerRef** and obtains a certificate from a secret that they pointed to the **Certificate** object.

10.1.4. Supported cert-manager Operator for Red Hat OpenShift versions

To maintain a supported configuration, review the compatibility of the cert-manager Operator for Red Hat OpenShift with different OpenShift Container Platform releases. To find the list of supported versions of the cert-manager Operator for Red Hat OpenShift across different OpenShift Container Platform releases, see the "Platform Agnostic Operators" section in "OpenShift Container Platform update and support policy".

10.1.5. About FIPS compliance for cert-manager Operator for Red Hat OpenShift

Starting with version 1.14.0, cert-manager Operator for Red Hat OpenShift is designed for FIPS compliance. When running on OpenShift Container Platform in FIPS mode, it uses the RHEL cryptographic libraries submitted to NIST for FIPS validation on the x86_64, ppc64le, and s390X architectures. For more information about the NIST validation program, see "Cryptographic module validation program". For the latest NIST status for the individual versions of the RHEL cryptographic libraries submitted for validation, see "Compliance activities and government standards".

To enable FIPS mode, you must install cert-manager Operator for Red Hat OpenShift on an OpenShift Container Platform cluster configured to operate in FIPS mode. For more information, see "Do you need extra security for your cluster?"

10.1.6. Additional resources

- [Cryptographic module validation program](#)
- [cert-manager project documentation](#)

- [OpenShift Container Platform update and support policy](#)
- [Understanding compliance](#)
- [Installing a cluster in FIPS mode](#)
- [Do you need extra security for your cluster?](#)
- [Vault](#)
- [Venafi](#)
- [Nokia NetGuard Certificate Manager](#)
- [Google Cloud Certificate Authority Service](#)
- [OpenShift Container Platform third-party support policy](#)

10.2. CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT RELEASE NOTES

The cert-manager Operator for Red Hat OpenShift is a cluster-wide service that provides application certificate lifecycle management.

These release notes track the development of cert-manager Operator for Red Hat OpenShift.

For more information, see [About the cert-manager Operator for Red Hat OpenShift](#).

10.2.1. cert-manager Operator for Red Hat OpenShift 1.20.0

Review the release notes for the cert-manager Operator for Red Hat OpenShift 1.20.0 to learn what is new and updated with this release.

Issued: 2 July 2026

The following advisories are available for the cert-manager Operator for Red Hat OpenShift for OpenShift Container Platform 1.20.0:

- [RHBA-2026:34294](#)
- [RHBA-2026:34309](#)
- [RHBA-2026:34336](#)
- [RHBA-2026:34714](#)

Version **v1.20.0** of the cert-manager Operator for Red Hat OpenShift is based on the upstream cert-manager version **v1.20.3**. For more information, see the [cert-manager project release notes for v1.20.3](#).

10.2.1.1. New features and enhancements



IMPORTANT

TLS adherence for cert-manager operands is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

TrustManager Technology Preview no longer requires a cluster preview FeatureSet

With this release, the cert-manager Operator for Red Hat OpenShift no longer requires the **featuregates.config.openshift.io/cluster** object to use a preview **FeatureSet**, such as **TechPreviewNoUpgrade**, in order to enable the TrustManager Technology Preview operand. Previously, enabling TrustManager required both of the following conditions to be met:

- The cluster **FeatureSet** must be set to a preview value, such as **TechPreviewNoUpgrade**, **DevPreviewNoUpgrade**, or **CustomNoUpgrade**.
- The Operator subscription must opt in to TrustManager by setting **UNSUPPORTED_ADDON_FEATURES=TrustManager=true**. Customers running clusters with the **Default FeatureSet** were unable to evaluate TrustManager without first switching the cluster to a preview **FeatureSet**, which is a disruptive, cluster-wide change that prevents upgrades.

With this update, the cluster **FeatureSet** requirement is removed. Enabling TrustManager now requires only that the Operator subscription includes **UNSUPPORTED_ADDON_FEATURES=TrustManager=true**. TrustManager remains a Technology Preview feature and is disabled by default.

For more information, see [Enabling the TrustManager Operand](#).

New performance-tuning override arguments for the cert-manager controller

With this release, the cert-manager Operator for Red Hat OpenShift supports configuring performance-tuning parameters for the cert-manager controller by using the **overrideArgs** field of the **CertManager** custom resource (CR). Previously, users had to rely on **spec.unsupportedConfigOverrides** to tune these settings.

You can now set the following arguments under **spec.controllerConfig.overrideArgs**:

- **--concurrent-workers**: The number of concurrent workers for each controller. The default value is **5**.
- **--kube-api-qps**: The maximum number of queries per second sent to the Kubernetes API server. The default value is **20**.
- **--kube-api-burst**: The maximum burst of queries per second sent to the Kubernetes API server. Must be greater than or equal to **--kube-api-qps**. The default value is **50**.
- **--max-concurrent-challenges**: The maximum number of ACME challenges that can be scheduled as processing at the same time. The default value is **60**. The Operator validates that **--kube-api-burst** is greater than or equal to **--kube-api-qps** when both values are set. If this constraint is not met, the Operator sets the **Degraded**

condition on the **CertManager** CR and does not apply the invalid configuration to the controller deployment.

For more information, see [Overridable arguments for the cert-manager components](#).

Cluster TLS security profile applied to cert-manager operands

With this release, the cert-manager Operator for Red Hat OpenShift can read the cluster TLS security profile from the **apiserver.config.openshift.io/cluster** object and automatically apply the corresponding TLS configuration to the cert-manager controller, webhook, and CA injector deployments.

Previously, the TLS configuration for cert-manager operands was not tied to the cluster-wide TLS security profile. Cluster administrators who configured a stricter TLS profile at the cluster level had no automated mechanism to propagate those settings to cert-manager operands, creating a gap in cluster-wide TLS posture enforcement.

With this update, when the **spec.tlsAdherence** field of the **CertManager** custom resource (CR) is set to **StrictAllComponents**, the Operator reads the **spec.tlsSecurityProfile** value from **apiserver.config.openshift.io/cluster** and applies the corresponding TLS arguments to the cert-manager operand deployments. The Operator reconciles the deployments whenever the cluster TLS profile changes.

TLS arguments are applied per operand component as follows:

- **cert-manager-webhook**: serving TLS flags and metrics endpoint TLS flags.
 - **cert-manager** (controller): metrics endpoint TLS flags only.
 - **cert-manager-cainjector**: metrics endpoint TLS flags only.
- TLS profile enforcement is not yet supported for the IstioCSR and TrustManager operands.

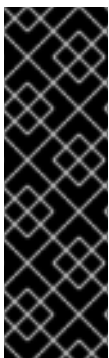
To support this feature, the Operator now requires **get**, **list**, and **watch** permissions on the **apiservers** resource in the **config.openshift.io** API group.

This feature is gated by the **TLSAdherence** feature gate. To use this feature, you must enable the **TechPreviewNoUpgrade** feature set. For more information, see [Understanding feature gates](#).



NOTE

Elliptic curve preferences are not configurable because cert-manager does not yet support specifying curve preferences upstream.



IMPORTANT

{FeatureName} is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

10.2.1.2. Fixed issues

- Before this update, the cert-manager Operator for Red Hat OpenShift installation failed on clusters with the Console capability disabled because the **ConsoleYAMLSample** resources were missing the required capability annotation. With this release, the Operator installs successfully on Console-less clusters. ([OCBUGS-85579](#))

10.2.2. cert-manager Operator for Red Hat OpenShift 1.19.1

Review the release notes for the cert-manager Operator for Red Hat OpenShift 1.19.1 to learn what is new and updated with this release.

Issued: 13 August 2026

The following advisories are available for the cert-manager Operator for Red Hat OpenShift for OpenShift Container Platform 1.19.1:

- [RHSA-2026:54527](#)
- [RHBA-2026:54529](#)
- [RHSA-2026:54531](#)
- [RHBA-2026:54551](#)

Version **v1.19.6** of the cert-manager Operator for Red Hat OpenShift is based on the upstream cert-manager version **v1.19.6**. For more information, see the [cert-manager project release notes for v1.19.6](#).

10.2.2.1. Fixed issues

- Before this update, the cert-manager Operator for Red Hat OpenShift installation failed on clusters without the console capability because the OLM bundle included **ConsoleYAMLSample** and **ConsoleQuickStart** resources that require the **console.openshift.io** APIs. With this release, the Operator creates the console resources at runtime only when the required APIs are available, ensuring successful installation. ([OCBUGS-85579](#))

10.2.2.2. CVEs

- [CVE-2026-33186](#)
- [CVE-2026-46595](#)
- [CVE-2026-39821](#)
- [CVE-2026-39828](#)
- [CVE-2026-39830](#)
- [CVE-2026-42499](#)
- [CVE-2026-25681](#)
- [CVE-2026-39820](#)
- [CVE-2026-46597](#)

- [CVE-2026-27145](#)
- [CVE-2026-42504](#)
- [CVE-2026-27136](#)
- [CVE-2026-42502](#)

10.2.3. cert-manager Operator for Red Hat OpenShift 1.19.0

Review the release notes for the cert-manager Operator for Red Hat OpenShift 1.19.0 to learn what is new and updated with this release.

Issued: 20 April 2026

The following advisories are available for the cert-manager Operator for Red Hat OpenShift for OpenShift Container Platform 1.19.0:

- [RHBA-2026:9064](#)
- [RHBA-2026:9024](#)
- [RHBA-2026:8953](#)
- [RHBA-2026:9025](#)
- [RHBA-2026:8956](#)

Version **v1.19.4** of the cert-manager Operator for Red Hat OpenShift is based on the upstream cert-manager version **v1.19.4**. For more information, see the [cert-manager project release notes for v1.19.4](#).

10.2.3.1. New features and enhancements

Distribution of trust bundles with the trust manager operand (Technology Preview)

In this release, the cert-manager Operator for Red Hat OpenShift adds support for the trust-manager operand as a Technology Preview feature. You can now install the trust-manager operand to automate the secure distribution of trust bundles, such as certificate authority (CA) certificates, to application namespaces across your cluster. For more information, see [Distributing certificates by using trust-manager operand](#).

Support for configuring the certificate request backoff duration

In this release, the cert-manager Operator for Red Hat OpenShift adds support for the **--certificate-request-minimum-backoff-duration** flag. With this flag, you can configure the minimum backoff period for certificate requests by overriding the default configuration. For more information, see [Overridable arguments for the cert-manager components](#).

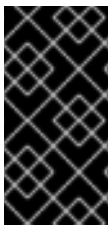
10.2.3.2. Fixed issues

- Before this update, the **ClusterIssuer** form view lacked an option to remove the self-signed field. As a consequence, you could not create issuer types other than self-signed. With this release, the form view sets the certificate authority (CA) as the default issuer type. As a result, you can switch to other issuer types by using the form view. ([OCPBUGS-65620](#))

10.3. INSTALLING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

The cert-manager Operator for Red Hat OpenShift is not installed in OpenShift Container Platform by default. You can install the cert-manager Operator for Red Hat OpenShift by using the web console and command-line interface (CLI).

The cert-manager Operator for Red Hat OpenShift sets the **features.operators.openshift.io/token-auth-aws**, **features.operators.openshift.io/token-auth-azure**, and **features.operators.openshift.io/token-auth-gcp** annotations in the **ClusterServiceVersion** custom resource of the Operator. The OpenShift Container Platform web console requires the credential details when these annotations are set. Currently, the Operator does not use the values collected by the OpenShift web console and you can provide any value when asked for the input. For example, when installing on the managed OpenShift Container Platform cluster, the **identity-provider-arn** is asked and any value can be provided to proceed.



IMPORTANT

The cert-manager Operator for Red Hat OpenShift version 1.15 or later supports the **AllNamespaces**, **SingleNamespace**, and **OwnNamespace** installation modes. Earlier versions, such as 1.14, support only the **SingleNamespace** and **OwnNamespace** installation modes.

10.3.1. Installing the cert-manager Operator for Red Hat OpenShift by using the web console

You can use the web console to install the cert-manager Operator for Red Hat OpenShift.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Navigate to **Ecosystem** → **Software Catalog**.
3. Enter **cert-manager Operator for Red Hat OpenShift** into the filter box.
4. Select the **cert-manager Operator for Red Hat OpenShift**
5. Select the cert-manager Operator for Red Hat OpenShift version from **Version** drop-down list, and click **Install**.



NOTE

See supported cert-manager Operator for Red Hat OpenShift versions in the following "Additional resources" section.

6. On the **Install Operator** page:

- a. Update the **Update channel**, if necessary. The channel defaults to **stable-v1**, which installs the latest stable release of the cert-manager Operator for Red Hat OpenShift.
- b. Choose the **Installed Namespace** for the Operator. The default Operator namespace is **cert-manager-operator**.
If the **cert-manager-operator** namespace does not exist, it is created for you.



NOTE

During the installation, the OpenShift Container Platform web console allows you to select between **AllNamespaces** and **SingleNamespace** installation modes. For installations with cert-manager Operator for Red Hat OpenShift version 1.15.0 or later, it is recommended to choose the **AllNamespaces** installation mode. **SingleNamespace** and **OwnNamespace** support will remain for earlier versions but will be deprecated in future versions.

- c. Select an **Update approval** strategy.
 - The **Automatic** strategy allows Operator Lifecycle Manager (OLM) to automatically update the Operator when a new version is available.
 - The **Manual** strategy requires a user with appropriate credentials to approve the Operator update.
- d. Click **Install**.

Verification

1. Navigate to **Ecosystem → Installed Operators**.
2. Verify that **cert-manager Operator for Red Hat OpenShift** is listed with a **Status** of **Succeeded** in the **cert-manager-operator** namespace.
3. Verify that cert-manager pods are up and running by entering the following command:

```
$ oc get pods -n cert-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
cert-manager-bd7fbb9fc-wvbbt	1/1	Running	0	3m39s
cert-manager-cainjector-56cc5f9868-7g9z7	1/1	Running	0	4m5s
cert-manager-webhook-d4f79d7f7-9dg9w	1/1	Running	0	4m9s

You can use the cert-manager Operator for Red Hat OpenShift only after cert-manager pods are up and running.

10.3.2. Installing the cert-manager Operator for Red Hat OpenShift by using the CLI

You can install the cert-manager Operator for Red Hat OpenShift by using the command-line interface (CLI).

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Create a new project named **cert-manager-operator** by running the following command:

```
$ oc new-project cert-manager-operator
```

2. Create an **OperatorGroup** object:

- a. Create a YAML file, for example, **operatorGroup.yaml**, with the following content:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-cert-manager-operator
  namespace: cert-manager-operator
spec:
  targetNamespaces:
    - "cert-manager-operator"
```

- b. For cert-manager Operator for Red Hat OpenShift v1.15.0 or later, create a YAML file with the following content:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-cert-manager-operator
  namespace: cert-manager-operator
spec:
  targetNamespaces: []
  spec: {}
```



NOTE

Starting from cert-manager Operator for Red Hat OpenShift version 1.15.0, it is recommended to install the Operator using the **AllNamespaces OLM installMode**. Older versions can continue using the **SingleNamespace** or **OwnNamespace OLM installMode**. Support for **SingleNamespace** and **OwnNamespace** will be deprecated in future versions.

- c. Create the **OperatorGroup** object by running the following command:

```
$ oc create -f operatorGroup.yaml
```

3. Create a **Subscription** object:

- a. Create a YAML file, for example, **subscription.yaml**, that defines the **Subscription** object:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: openshift-cert-manager-operator
```

```

namespace: cert-manager-operator
spec:
  channel: stable-v1
  name: openshift-cert-manager-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic

```

- b. Create the **Subscription** object by running the following command:

```
$ oc create -f subscription.yaml
```

Verification

1. Verify that the OLM subscription is created by running the following command:

```
$ oc get subscription -n cert-manager-operator
```

Example output

NAME	PACKAGE	SOURCE	CHANNEL
openshift-cert-manager-operator	openshift-cert-manager-operator	redhat-operators	stable-v1

2. Verify whether the Operator is successfully installed by running the following command:

```
$ oc get csv -n cert-manager-operator
```

Example output

NAME	DISPLAY	VERSION	REPLACES
cert-manager-operator.v1.13.0	cert-manager Operator for Red Hat OpenShift	1.13.0	
cert-manager-operator.v1.12.1	Succeeded		

3. Verify that the status cert-manager Operator for Red Hat OpenShift is **Running** by running the following command:

```
$ oc get pods -n cert-manager-operator
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
cert-manager-operator-controller-manager-695b4d46cb-r4hld	2/2	Running	0	7m4s

4. Verify that the status of cert-manager pods is **Running** by running the following command:

```
$ oc get pods -n cert-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
cert-manager-58b7f649c4-dp6l4	1/1	Running	0	7m1s
cert-manager-cainjector-5565b8f897-gx25h	1/1	Running	0	7m37s
cert-manager-webhook-9bc98cbdd-f972x	1/1	Running	0	7m40s

Additional resources

- [Supported cert-manager Operator for Red Hat OpenShift versions](#)

10.3.3. Understanding update channels of the cert-manager Operator for Red Hat OpenShift

Update channels are the mechanism by which you can declare the version of your cert-manager Operator for Red Hat OpenShift in your cluster. The cert-manager Operator for Red Hat OpenShift offers the following update channels:

- **stable-v1**
- **stable-v1.y**

10.3.3.1. stable-v1 channel

The **stable-v1** channel installs and updates the latest release version of the cert-manager Operator for Red Hat OpenShift. Select the **stable-v1** channel if you want to use the latest stable release of the cert-manager Operator for Red Hat OpenShift.



NOTE

The **stable-v1** channel is the default and suggested channel while installing the cert-manager Operator for Red Hat OpenShift.

The **stable-v1** channel offers the following update approval strategies:

Automatic

If you choose automatic updates for an installed cert-manager Operator for Red Hat OpenShift, a new version of the cert-manager Operator for Red Hat OpenShift is available in the **stable-v1** channel. The Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without human intervention.

Manual

If you select manual updates, when a newer version of the cert-manager Operator for Red Hat OpenShift is available, OLM creates an update request. As a cluster administrator, you must then manually approve that update request to have the cert-manager Operator for Red Hat OpenShift updated to the new version.

10.3.3.2. stable-v1.y channel

The y-stream version of the cert-manager Operator for Red Hat OpenShift installs updates from the **stable-v1.y** channels such as **stable-v1.10**, **stable-v1.11**, and **stable-v1.12**. Select the **stable-v1.y** channel if you want to use the y-stream version and stay updated to the z-stream version of the cert-manager Operator for Red Hat OpenShift.

The **stable-v1.y** channel offers the following update approval strategies:

Automatic

If you choose automatic updates for an installed cert-manager Operator for Red Hat OpenShift, a new z-stream version of the cert-manager Operator for Red Hat OpenShift is available in the **stable-v1.y** channel. OLM automatically upgrades the running instance of your Operator without human intervention.

Manual

If you select manual updates, when a newer version of the cert-manager Operator for Red Hat OpenShift is available, OLM creates an update request. As a cluster administrator, you must then manually approve that update request to have the cert-manager Operator for Red Hat OpenShift updated to the new version of the z-stream releases.

10.3.4. Additional resources

- [Adding Operators to a cluster](#)
- [Updating installed Operators](#)

10.4. CONFIGURING THE EGRESS PROXY FOR THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

If a cluster-wide egress proxy is configured in OpenShift Container Platform, Operator Lifecycle Manager (OLM) automatically configures Operators that it manages with the cluster-wide proxy. OLM automatically updates all of the Operator's deployments with the **HTTP_PROXY**, **HTTPS_PROXY**, **NO_PROXY** environment variables.

You can inject any CA certificates that are required for proxying HTTPS connections into the cert-manager Operator for Red Hat OpenShift.

10.4.1. Injecting a custom CA certificate for the cert-manager Operator for Red Hat OpenShift

If your OpenShift Container Platform cluster has the cluster-wide proxy enabled, you can inject any CA certificates that are required for proxying HTTPS connections into the cert-manager Operator for Red Hat OpenShift.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have enabled the cluster-wide proxy for OpenShift Container Platform.

Procedure

1. Create a config map in the **cert-manager** namespace by running the following command:

```
$ oc create configmap trusted-ca -n cert-manager
```

2. Inject the CA bundle that is trusted by OpenShift Container Platform into the config map by running the following command:

```
$ oc label cm trusted-ca config.openshift.io/inject-trusted-cabundle=true -n cert-manager
```

3. Update the deployment for the cert-manager Operator for Red Hat OpenShift to use the config map by running the following command:

```
$ oc -n cert-manager-operator patch subscription openshift-cert-manager-operator --
type='merge' -p '{"spec":{"config":{"env":
[{"name":"TRUSTED_CA_CONFIGMAP_NAME","value":"trusted-ca"}]}}}'
```

Verification

1. Verify that the deployments have finished rolling out by running the following command:

```
$ oc rollout status deployment/cert-manager-operator-controller-manager -n cert-manager-
operator && \
oc rollout status deployment/cert-manager -n cert-manager && \
oc rollout status deployment/cert-manager-webhook -n cert-manager && \
oc rollout status deployment/cert-manager-cainjector -n cert-manager
```

Example output

```
deployment "cert-manager-operator-controller-manager" successfully rolled out
deployment "cert-manager" successfully rolled out
deployment "cert-manager-webhook" successfully rolled out
deployment "cert-manager-cainjector" successfully rolled out
```

2. Verify that the CA bundle was mounted as a volume by running the following command:

```
$ oc get deployment cert-manager -n cert-manager -o=jsonpath=
{.spec.template.spec.containers[0].volumeMounts}
```

Example output

```
[{"mountPath":"/etc/pki/tls/certs/cert-manager-tls-ca-bundle.crt","name":"trusted-
ca","subPath":"ca-bundle.crt"}]
```

3. Verify that the source of the CA bundle is the **trusted-ca** config map by running the following command:

```
$ oc get deployment cert-manager -n cert-manager -o=jsonpath=
{.spec.template.spec.volumes}
```

Example output

```
[{"configMap":{"defaultMode":420,"name":"trusted-ca"},"name":"trusted-ca"}]
```

10.4.2. Additional resources

- [Configuring proxy support in Operator Lifecycle Manager](#)

10.5. CUSTOMIZING THE CERT-MANAGER OPERATOR BY USING THE CERTMANAGER CUSTOM RESOURCE

You can customize the cert-manager Operator for Red Hat OpenShift after installation to suit your cluster requirements.

- Configure the **CertManager** custom resource (CR) to modify the behavior of cert-manager components, such as the cert-manager controller, CA injector, and webhook.
- Set environment variables for the controller pod.
- Define resource requests and limits to manage CPU and memory usage.
- Configure scheduling rules to control where pods run in your cluster.
- Configure the cluster **APIServer** custom resource (CR) to apply the cluster-wide TLS security profile to cert-manager components.

Example CertManager CR YAML file

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
spec:
  controllerConfig:
    overrideArgs:
      - "--dns01-recursive-nameservers=8.8.8.8:53,1.1.1.1:53"
    overrideEnv:
      - name: HTTP_PROXY
        value: http://proxy.example.com:8080
    overrideResources:
      limits:
        cpu: "200m"
        memory: "512Mi"
      requests:
        cpu: "100m"
        memory: "256Mi"
    overrideScheduling:
      nodeSelector:
        custom: "label"
      tolerations:
        - key: "key1"
          operator: "Equal"
          value: "value1"
          effect: "NoSchedule"
    overrideReplicas: 2
  #...

  webhookConfig:
    overrideArgs:
  #...
    overrideResources:
  #...
    overrideScheduling:
  #...
    overrideReplicas:
  #...
```



```

cainjectorConfig:
  overrideArgs:
#...
  overrideResources:
#...
  overrideScheduling:
#...
  overrideReplicas:
#...

```



WARNING

To override unsupported arguments, you can add **spec.unsupportedConfigOverrides** section in the **CertManager** resource, but using **spec.unsupportedConfigOverrides** is unsupported.

10.5.1. Explanation of fields in the CertManager custom resource

To configure core components of the cert-manager Operator for Red Hat OpenShift, use the CertManager custom resource (CR). You can define settings for the cert-manager controller, such as the `spec.controllerConfig` field, to customize your deployment.

The core components of the cert-manager Operator for Red Hat OpenShift are as follows:

- Cert-manager controller: You can use the **spec.controllerConfig** field to configure the cert-manager controller pod.
- Webhook: You can use the **spec.webhookConfig** field to configure the webhook pod, which handles validation and mutation requests.
- CA injector: You can use the **spec.cainjectorConfig** field to configure the CA injector pod.

Additional resources

- [Deleting a TLS secret automatically upon Certificate removal](#)

10.5.1.1. Common configurable fields in the CertManager CR for the cert-manager components

You can configure common fields in the **spec.controllerConfig**, **spec.webhookConfig**, and **spec.cainjectorConfig** sections in the **CertManager** CR to customize the cert-manager components.

Table 10.1. Common configurable fields in the CertManager CR for the cert-manager components

Field	Type	Description
overrideArgs	string	You can override the supported arguments for the cert-manager components.

Field	Type	Description
overrideEnv	dict	You can override the supported environment variables for the cert-manager controller. This field is only supported for the cert-manager controller component.
overrideReplicas	int	You can configure the replicas for the cert-manager components. The default value is 1 . For production environments, the following replica counts are recommended: • controller: 2 • cainjector: 2 • webhook: At least 3.
overrideResources	object	You can configure the CPU and memory limits for the cert-manager components.
overrideScheduling	object	You can configure the pod scheduling constraints for the cert-manager components.

Additional resources

- [High Availability](#)

10.5.1.2. Overridable arguments for the cert-manager components

You can configure the overridable arguments for the cert-manager components in the **spec.controllerConfig**, **spec.webhookConfig**, and **spec.cainjectorConfig** sections in the **CertManager** CR to customize the cert-manager controller, webhook, and cainjector components.

The following table describes the overridable arguments for the cert-manager components:

Table 10.2. Overridable arguments for the cert-manager components

Argument	Component	Description
----------	-----------	-------------

Argument	Component	Description
--dns01-recursive-nameservers=<server_address>	Controller	Provide a comma-separated list of nameservers to query for the DNS-01 self check. The nameservers can be specified either as <host>:<port>, for example, 1.1.1.1:53, or use DNS over HTTPS (DoH), for example, https://1.1.1.1/dns-query.  NOTE DNS over HTTPS (DoH) is supported starting only from cert-manager Operator for Red Hat OpenShift version 1.13.0 and later.
--dns01-recursive-nameservers-only	Controller	Specify to only use recursive nameservers instead of checking the authoritative nameservers associated with that domain.
--acme-http01-solver-nameservers=<host>:<port>	Controller	Provide a comma-separated list of <host>:<port> nameservers to query for the Automated Certificate Management Environment (ACME) HTTP01 self check. For example, --acme-http01-solver-nameservers=1.1.1.1:53 .
--metrics-listen-address=<host>:<port>	Controller	Specify the host and port for the metrics endpoint. The default value is --metrics-listen-address=0.0.0.0:9402 .
--issuer-ambient-credentials	Controller	You can use this argument to configure an ACME Issuer to solve DNS-01 challenges by using ambient credentials.

Argument	Component	Description
--enable-certificate-owner-ref	Controller	This argument sets the certificate resource as an owner of the secret where the TLS certificate is stored. For more information, see "Deleting a TLS secret automatically upon Certificate removal".
--acme-http01-solver-resource-limits-cpu	Controller	Defines the maximum CPU limit for ACME HTTP-01 solver pods. The default value is 100m .
--acme-http01-solver-resource-limits-memory	Controller	Defines the maximum memory limit for ACME HTTP-01 solver pods. The default value is 64Mi .
--acme-http01-solver-resource-request-cpu	Controller	Defines the minimum CPU request for ACME HTTP-01 solver pods. The default value is 10m .
--acme-http01-solver-resource-request-memory	Controller	Defines the minimum memory request for ACME HTTP-01 solver pods. The default value is 64Mi .
--certificate-request-minimum-backoff-duration	Controller	Specify the minimum backoff duration for certificate requests. The default value is 1h0m0s .
--concurrent-workers	Controller	The number of concurrent workers for each controller. The default value is 5 .
--kube-api-qps	Controller	The maximum number of queries per second sent to the Kubernetes API server. The default value is 20 .
--kube-api-burst	Controller	The maximum burst of queries per second sent to the Kubernetes API server. Must be greater than or equal to --kube-api-qps . The default value is 50 .
--max-concurrent-challenges	Controller	The maximum number of ACME challenges that can run concurrently. The default value is 60 .

Argument	Component	Description
--v=<verbosity_level>	Controller, Webhook, CA injector	Specify the log level verbosity to determine the verbosity of log messages.

10.5.1.3. Overridable environment variables for the cert-manager controller

You can configure the overridable environment variables for the cert-manager controller in the **spec.controllerConfig.overrideEnv** field in the **CertManager** CR to control proxy settings for the cert-manager controller.

The following table describes the overridable environment variables for the cert-manager controller:

Table 10.3. Overridable environment variables for the cert-manager controller

Environment variable	Description
HTTP_PROXY	Proxy server for outgoing HTTP requests.
HTTPS_PROXY	Proxy server for outgoing HTTPS requests.
NO_PROXY	Comma-separated list of hosts that bypass the proxy.

10.5.1.4. Overridable resource parameters for the cert-manager components

You can configure the CPU and memory request and limits for the cert-manager components in the **CertManager** CR to control resource consumption for the controller, webhook, and cainjector pods.

The following table describes the overridable resource parameters for the cert-manager components:

Table 10.4. Overridable resource parameters for the cert-manager components

Field	Description
overrideResources.limits.cpu	Defines the maximum amount of CPU that a component pod can use.
overrideResources.limits.memory	Defines the maximum amount of memory that a component pod can use.
overrideResources.requests.cpu	Defines the minimum amount of CPU requested by the scheduler for a component pod.
overrideResources.requests.memory	Defines the minimum amount of memory requested by the scheduler for a component pod.

10.5.1.5. Overridable scheduling parameters for the cert-manager components

To optimize resource usage or isolate specific workloads, you can control the pod placement of your cert-manager components.

You can easily configure node selectors and tolerations by modifying the **spec.controllerConfig**, **spec.webhookConfig**, and **spec.cainjectorConfig** sections of the **CertManager** custom resource (CR).

The following table describes the pod scheduling parameters for the cert-manager components:

Table 10.5. Overridable scheduling parameters for the cert-manager components

Field	Description
overrideScheduling.nodeSelector	Key and value pairs to constrain pods to specific nodes.
overrideScheduling.tolerations	List of tolerations to schedule pods on tainted nodes.

10.5.2. Customizing cert-manager by overriding environment variables from the cert-manager Operator API

To refine your deployment for specific operational requirements, override supported environment variables for the cert-manager Operator for Red Hat OpenShift. You can customize these variables through the Operator API to apply configurations, such as proxy settings or system-level adjustments, that differ from the default values.

You can override the supported environment variables for the cert-manager Operator for Red Hat OpenShift by adding a **spec.controllerConfig** section in the **CertManager** resource.

Prerequisites

- You have access to the OpenShift Container Platform cluster as a user with the **cluster-admin** role.

Procedure

- Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

- Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideEnv:
      - name: HTTP_PROXY
        value: http://<proxy_url>
```

```
- name: HTTPS_PROXY
  value: https://<proxy_url>
- name: NO_PROXY
  value: <ignore_proxy_domains>
```

where:

HTTP_PROXY

Specifies the proxy server URL.

NO_PROXY

Specifies a comma separated list of domains. These domains are ignored by the proxy server.



NOTE

For more information about the overridable environment variables, see "Overridable environment variables for the cert-manager components" in "Explanation of fields in the CertManager custom resource".

3. Save your changes and quit the text editor to apply your changes.

Verification

1. Verify that the cert-manager controller pod is redeployed by running the following command:

```
$ oc get pods -l app.kubernetes.io/name=cert-manager -n cert-manager
```

Example output

```
NAME                                READY STATUS  RESTARTS  AGE
cert-manager-bd7fbb9fc-wvbbt 1/1    Running    0          39s
```

2. Verify that environment variables are updated for the cert-manager pod by running the following command:

```
$ oc get pod <redeployed_cert-manager_controller_pod> -n cert-manager -o yaml
```

Example output

```
env:
  ...
  - name: HTTP_PROXY
    value: http://<PROXY_URL>
  - name: HTTPS_PROXY
    value: https://<PROXY_URL>
  - name: NO_PROXY
    value: <IGNORE_PROXY_DOMAINS>
```

Additional resources

- [Explanation of fields in the CertManager custom resource](#)

10.5.3. Customizing cert-manager by overriding arguments from the cert-manager Operator API

You can override the supported arguments for the cert-manager Operator for Red Hat OpenShift by adding a **spec.controllerConfig** section in the **CertManager** resource.

Prerequisites

- You have access to the OpenShift Container Platform cluster as a user with the **cluster-admin** role.

Procedure

1. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

2. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideArgs:
      - '--dns01-recursive-nameservers=<server_address>'
      - '--dns01-recursive-nameservers-only'
      - '--acme-http01-solver-nameservers=<host>:<port>'
      - '--v=<verbosity_level>'
      - '--metrics-listen-address=<host>:<port>'
      - '--issuer-ambient-credentials'
      - '--acme-http01-solver-resource-limits-cpu=<quantity>'
      - '--acme-http01-solver-resource-limits-memory=<quantity>'
      - '--acme-http01-solver-resource-request-cpu=<quantity>'
      - '--acme-http01-solver-resource-request-memory=<quantity>'
      - '--certificate-request-minimum-backoff-duration=<duration>'
      - '--concurrent-workers=<quantity>'
      - '--kube-api-qps=<quantity>'
      - '--kube-api-burst=<quantity>'
      - '--max-concurrent-challenges=<quantity>'
    webhookConfig:
      overrideArgs:
        - '--v=<verbosity_level>'
    cainjectorConfig:
      overrideArgs:
        - '--v=<verbosity_level>'
```

For information about the overridable arguments, see "Overridable arguments for the cert-manager components" in "Explanation of fields in the CertManager custom resource".

3. Save your changes and quit the text editor to apply your changes.

Verification

- Verify that arguments are updated for cert-manager pods by running the following command:

```
$ oc get pods -n cert-manager -o yaml
```

Example output

```
...
metadata:
  name: cert-manager-6d4b5d4c97-kldwl
  namespace: cert-manager
...
spec:
  containers:
  - args:
    # ...
    - --acme-http01-solver-nameservers=1.1.1.1:53
    - --concurrent-workers=5
    - --dns01-recursive-nameservers=1.1.1.1:53
    - --dns01-recursive-nameservers-only
    - --kube-api-burst=50
    - --kube-api-qps=20
    - --max-concurrent-challenges=60
    - --metrics-listen-address=0.0.0.0:9042
    - --v=6
...
metadata:
  name: cert-manager-cainjector-866c4fd758-ltxxj
  namespace: cert-manager
...
spec:
  containers:
  - args:
    # ...
    - --v=2
...
metadata:
  name: cert-manager-webhook-6d48f88495-c88gd
  namespace: cert-manager
...
spec:
  containers:
  - args:
    # ...
    - --v=2
```

Additional resources

- [Explanation of fields in the CertManager custom resource](#)

10.5.4. Deleting a TLS secret automatically upon Certificate removal

You can enable the **--enable-certificate-owner-ref** flag for the cert-manager Operator for Red Hat OpenShift by adding a **spec.controllerConfig** section in the **CertManager** resource. The **--enable-**

certificate-owner-ref flag sets the certificate resource as an owner of the secret where the TLS certificate is stored.



WARNING

If you uninstall the cert-manager Operator for Red Hat OpenShift or delete certificate resources from the cluster, the secret is deleted automatically. This might cause network connectivity issues depending upon where the certificate TLS secret is being used.

Prerequisites

- You have access to the OpenShift Container Platform cluster as a user with the **cluster-admin** role.
- You have installed version 1.12.0 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

1. Check that the **Certificate** object and its secret are available by running the following command:

```
$ oc get certificate
```

Example output

NAME	READY	SECRET	AGE
certificate-from-clusterissuer-route53-ambient	True	certificate-from-clusterissuer-route53-ambient	8h

2. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

3. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
# ...
spec:
# ...
  controllerConfig:
    overrideArgs:
      - '--enable-certificate-owner-ref'
```

4. Save your changes and quit the text editor to apply your changes.

Verification

- Verify that the **--enable-certificate-owner-ref** flag is updated for cert-manager controller pod by running the following command:

```
$ oc get pods -l app.kubernetes.io/name=cert-manager -n cert-manager -o yaml
```

Example output

```
# ...
metadata:
  name: cert-manager-6e4b4d7d97-zmdnb
  namespace: cert-manager
# ...
spec:
  containers:
  - args:
    - --enable-certificate-owner-ref
```

10.5.5. Overriding CPU and memory limits for the cert-manager components

To ensure stable resource allocation and operation, configure CPU and memory limits for cert-manager Operator for Red Hat OpenShift components. You can set specific constraints for the cert-manager controller, CA injector, and Webhook to align with your specific cluster requirements.

Prerequisites

- You have access to the OpenShift Container Platform cluster as a user with the **cluster-admin** role.
- You have installed version 1.12.0 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

1. Check that the deployments of the cert-manager controller, CA injector, and Webhook are available by entering the following command:

```
$ oc get deployment -n cert-manager
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
cert-manager	1/1	1	1	53m
cert-manager-cainjector	1/1	1	1	53m
cert-manager-webhook	1/1	1	1	53m

2. Before setting the CPU and memory limit, check the existing configuration for the cert-manager controller, CA injector, and Webhook by entering the following command:

```
$ oc get deployment -n cert-manager -o yaml
```

Example output

■

```
# ...
  metadata:
    name: cert-manager
    namespace: cert-manager
# ...
  spec:
    template:
      spec:
        containers:
        - name: cert-manager-controller
          resources: {}
# ...
  metadata:
    name: cert-manager-cainjector
    namespace: cert-manager
# ...
  spec:
    template:
      spec:
        containers:
        - name: cert-manager-cainjector
          resources: {}
# ...
  metadata:
    name: cert-manager-webhook
    namespace: cert-manager
# ...
  spec:
    template:
      spec:
        containers:
        - name: cert-manager-webhook
          resources: {}
# ...
```

The **spec.resources** field is empty by default. The cert-manager components do not have CPU and memory limits.

3. To configure the CPU and memory limits for the cert-manager controller, CA injector, and Webhook, enter the following command:

```
$ oc patch certmanager.operator cluster --type=merge -p="
spec:
  controllerConfig:
    overrideResources:
      limits:
        cpu: 200m
        memory: 64Mi
      requests:
        cpu: 10m
        memory: 16Mi
  webhookConfig:
    overrideResources:
      limits:
        cpu: 200m
        memory: 64Mi"
```

```

    requests:
      cpu: 10m
      memory: 16Mi
  cainjectorConfig:
    overrideResources:
      limits:
        cpu: 200m
        memory: 64Mi
      requests:
        cpu: 10m
        memory: 16Mi
"

```

For information about the overridable resource parameters, see "Overridable resource parameters for the cert-manager components" in "Explanation of fields in the CertManager custom resource".

Example output

```
certmanager.operator.openshift.io/cluster patched
```

Verification

1. Verify that the CPU and memory limits are updated for the cert-manager components:

```
$ oc get deployment -n cert-manager -o yaml
```

Example output

```

# ...
metadata:
  name: cert-manager
  namespace: cert-manager
# ...
spec:
  template:
    spec:
      containers:
      - name: cert-manager-controller
        resources:
          limits:
            cpu: 200m
            memory: 64Mi
          requests:
            cpu: 10m
            memory: 16Mi
# ...
metadata:
  name: cert-manager-cainjector
  namespace: cert-manager
# ...
spec:
  template:
    spec:
      containers:

```

```

- name: cert-manager-cainjector
  resources:
    limits:
      cpu: 200m
      memory: 64Mi
    requests:
      cpu: 10m
      memory: 16Mi
# ...
  metadata:
    name: cert-manager-webhook
    namespace: cert-manager
# ...
  spec:
    template:
      spec:
        containers:
- name: cert-manager-webhook
  resources:
    limits:
      cpu: 200m
      memory: 64Mi
    requests:
      cpu: 10m
      memory: 16Mi
# ...

```

Additional resources

- [Explanation of fields in the CertManager custom resource](#)

10.5.6. Configuring scheduling overrides for cert-manager components

You can configure the pod scheduling from the cert-manager Operator for Red Hat OpenShift API for the cert-manager Operator for Red Hat OpenShift components, such as the cert-manager controller, CA injector, and Webhook.

Prerequisites

- You have access to the OpenShift Container Platform cluster as a user with the **cluster-admin** role.
- You have installed version 1.15.0 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

- Update the **certmanager.operator** custom resource to configure pod scheduling overrides for the desired components by running the following command. Use the **overrideScheduling** field under the **controllerConfig**, **webhookConfig**, or **cainjectorConfig** sections to define **nodeSelector** and **tolerations** settings.

```

$ oc patch certmanager.operator cluster --type=merge -p="
spec:
  controllerConfig:
    overrideScheduling:

```

```

nodeSelector:
  node-role.kubernetes.io/control-plane: ""
tolerations:
  - key: node-role.kubernetes.io/master
    operator: Exists
    effect: NoSchedule
webhookConfig:
  overrideScheduling:
    nodeSelector:
      node-role.kubernetes.io/control-plane: ""
    tolerations:
      - key: node-role.kubernetes.io/master
        operator: Exists
        effect: NoSchedule
cainjectorConfig:
  overrideScheduling:
    nodeSelector:
      node-role.kubernetes.io/control-plane: ""
    tolerations:
      - key: node-role.kubernetes.io/master
        operator: Exists
        effect: NoSchedule"
"

```

For information about the overridable scheduling parameters, see "Overridable scheduling parameters for the cert-manager components" in "Explanation of fields in the CertManager custom resource".

Verification

1. Verify pod scheduling settings for **cert-manager** pods:
 - a. Check the deployments in the **cert-manager** namespace to confirm they have the correct **nodeSelector** and **tolerations** by running the following command:

```
$ oc get pods -n cert-manager -o wide
```

Example output

```

NAME                                READY STATUS  RESTARTS  AGE  IP            NODE
NOMINATED NODE  READINESS GATES
cert-manager-58d9c69db4-78mzp      1/1   Running  0       10m  10.129.0.36  ip-10-0-1-106.ec2.internal <none> <none>
cert-manager-cainjector-85b6987c66-rhxf7 1/1   Running  0       11m  10.128.0.39  ip-10-0-1-136.ec2.internal <none> <none>
cert-manager-webhook-7f54b4b858-29bsp 1/1   Running  0       11m  10.129.0.35  ip-10-0-1-106.ec2.internal <none> <none>

```

- b. Check the **nodeSelector** and **tolerations** settings applied to deployments by running the following command:

```
$ oc get deployments -n cert-manager -o jsonpath='{range .items[*]}{.metadata.name}
{"\n"}{.spec.template.spec.nodeSelector}{"\n"}{.spec.template.spec.tolerations}{"\n\n"}
{end}'
```

Example output

```
cert-manager
{"kubernetes.io/os":"linux","node-role.kubernetes.io/control-plane":""}
[{"effect":"NoSchedule","key":"node-role.kubernetes.io/master","operator":"Exists"}]

cert-manager-cainjector
{"kubernetes.io/os":"linux","node-role.kubernetes.io/control-plane":""}
[{"effect":"NoSchedule","key":"node-role.kubernetes.io/master","operator":"Exists"}]

cert-manager-webhook
{"kubernetes.io/os":"linux","node-role.kubernetes.io/control-plane":""}
[{"effect":"NoSchedule","key":"node-role.kubernetes.io/master","operator":"Exists"}]
```

2. Verify pod scheduling events in the **cert-manager** namespace by running the following command:

```
$ oc get events -n cert-manager --field-selector reason=Scheduled
```

Additional resources

- [Explanation of fields in the CertManager custom resource](#)

10.5.7. Configuring cluster TLS security profile adherence for cert-manager components

You can configure the cert-manager Operator for Red Hat OpenShift to apply the cluster-wide TLS security profile by setting the TLS adherence policy on the cluster **APIServer** resource. When the adherence policy is set to **StrictAllComponents**, cert-manager components automatically apply the cluster TLS security profile settings.



IMPORTANT

TLS adherence for cert-manager operands is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You installed the cert-manager Operator for Red Hat OpenShift.
- You enabled the **TechPreviewNoUpgrade** feature set. For more information, see "Enabling features using feature gates".

Procedure

1. Edit the cluster **APIServer** custom resource (CR) by running the following command:

```
$ oc edit apiserver cluster
```

2. Add or modify the **tlsAdherence** field in the **spec** section and set it to **StrictAllComponents** by using the following example:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  name: cluster
spec:
  tlsSecurityProfile:
    type: Intermediate
    intermediate: {}
  tlsAdherence: StrictAllComponents
```

where:

tlsSecurityProfile.type

Optional: Specifies the TLS security profile type. Valid values are **Old**, **Intermediate**, **Modern**, or **Custom**. If not specified, the default is **Intermediate**. When specifying a profile type, you must also include the corresponding profile-specific field, for example, **intermediate: {}** for the **Intermediate** profile.

tlsAdherence: StrictAllComponents

Specifies that cluster-wide TLS settings are enforced on all components, including cert-manager.

3. Save the changes and exit the editor.

Verification

1. The cert-manager Operator for Red Hat OpenShift automatically applies the cluster TLS security profile to the cert-manager controller, webhook, and CA injector deployments. Check that the TLS configuration is applied to the cert-manager components. For more information, see "Verifying TLS security profile adherence for cert-manager components".

10.5.8. Verifying TLS security profile adherence for cert-manager components

After configuring the cluster TLS security profile adherence, you can verify that the TLS configuration is applied to the cert-manager controller, webhook, and CA injector deployments.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You installed the cert-manager Operator for Red Hat OpenShift.
- You enabled the **TechPreviewNoUpgrade** feature set. For more information, see "Enabling features using feature gates".

- You configured the cluster TLS security profile adherence for cert-manager components. For more information, see "Configuring cluster TLS security profile adherence for cert-manager components".

Procedure

1. Verify that the cert-manager controller deployment has the TLS configuration applied by running the following command:

```
$ oc get deployment -n cert-manager cert-manager -o yaml | grep -A 15 "args:"
```

Example output

```
args:
- --v=2
- --cluster-resource-namespace=$(POD_NAMESPACE)
- --leader-election-namespace=kube-system
- --acme-http01-solver-image=registry.redhat.io/cert-manager/cert-manager-acmesolver-rhel9@sha256:...
- --max-concurrent-challenges=60
- --metrics-tls-min-version=VersionTLS12
- --metrics-tls-cipher-suites=TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
```

The **--metrics-tls-min-version** flag shows the minimum TLS version configured based on the cluster TLS security profile. The **--metrics-tls-cipher-suites** flag shows TLS cipher suites configured based on the cluster TLS security profile.

2. Verify that the webhook deployment has the TLS configuration applied by running the following command:

```
$ oc get deployment -n cert-manager cert-manager-webhook -o yaml | grep -A 20 "args:"
```

Example output

```
args:
- --v=2
- --dynamic-serving-ca-secret-namespace=$(POD_NAMESPACE)
- --dynamic-serving-ca-secret-name=cert-manager-webhook-ca
- --dynamic-serving-dns-names=cert-manager-webhook
- --tls-min-version=VersionTLS12
- --tls-cipher-suites=TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
- --metrics-tls-min-version=VersionTLS12
- --metrics-tls-cipher-suites=TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
```

The webhook deployment includes serving TLS flags (**--tls-min-version** and **--tls-cipher-suites**) for the webhook HTTPS endpoint, and TLS flags for the metrics endpoint.

3. Verify that the CA injector deployment has the TLS configuration applied by running the following command:

```
$ oc get deployment -n cert-manager cert-manager-cainjector -o yaml | grep -A 10 "args:"
```

Example output

```
args:
- --v=2
- --leader-election-namespace=kube-system
- --metrics-tls-min-version=VersionTLS12
- --metrics-tls-cipher-
suites=TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AE
S_128_GCM_SHA256
```

The CA injector deployment includes metrics endpoint TLS flags.



NOTE

TLS profile enforcement is not available for the IstioCSR and TrustManager operands.

When the cluster TLS security profile is set to **Modern** (TLS 1.3), the cipher suite flags are automatically omitted.

Additional resources

- [Understanding feature gates](#)
- [Understanding TLS security profiles](#)

10.6. AUTHENTICATING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

To enable the operator to manage components on your cloud provider, authenticate the cert-manager Operator for Red Hat OpenShift by configuring cloud credentials. You can grant the Operator access to external services required for certificate issuance, such as DNS providers.

10.6.1. Authenticating on AWS

To securely access AWS resources from your applications, authenticate your workloads on AWS by using the cert-manager Operator for Red Hat OpenShift.

Prerequisites

- You have installed version 1.11.1 or later of the cert-manager Operator for Red Hat OpenShift.
- You have configured the Cloud Credential Operator to operate in *mint* or *passthrough* mode.

Procedure

1. Create a **CredentialsRequest** resource YAML file, for example, **sample-credential-request.yaml**, as follows:

```

apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: cert-manager
  namespace: openshift-cloud-credential-operator
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
      - action:
          - "route53:GetChange"
        effect: Allow
        resource: "arn:aws:route53:::change/*"
      - action:
          - "route53:ChangeResourceRecordSets"
          - "route53:ListResourceRecordSets"
        effect: Allow
        resource: "arn:aws:route53:::hostedzone/*"
      - action:
          - "route53:ListHostedZonesByName"
        effect: Allow
        resource: "*"
  secretRef:
    name: aws-creds
    namespace: cert-manager
  serviceAccountNames:
    - cert-manager

```

2. Create a **CredentialsRequest** resource by running the following command:

```
$ oc create -f sample-credential-request.yaml
```

3. Update the subscription object for cert-manager Operator for Red Hat OpenShift by running the following command:

```
$ oc -n cert-manager-operator patch subscription openshift-cert-manager-operator --
type=merge -p '{"spec":{"config":{"env":
[{"name":"CLOUD_CREDENTIALS_SECRET_NAME","value":"aws-creds"}]}}'
```

Verification

1. Get the name of the redeployed cert-manager controller pod by running the following command:

```
$ oc get pods -l app.kubernetes.io/name=cert-manager -n cert-manager
```

Example output

```

NAME                                READY STATUS  RESTARTS  AGE
cert-manager-bd7fbb9fc-wvbbt 1/1    Running    0         15m39s

```

2. Verify that the cert-manager controller pod is updated with AWS credential volumes that are mounted under the path specified in **mountPath** by running the following command:

```
$ oc get -n cert-manager pod/<cert-manager_controller_pod_name> -o yaml
```

Example output

```
...
spec:
  containers:
    - args:
      ...
      - mountPath: /.aws
        name: cloud-credentials
    ...
  volumes:
    ...
    - name: cloud-credentials
      secret:
        ...
        secretName: aws-creds
```

10.6.2. Authenticating with AWS Security Token Service

To securely access AWS resources from your applications without managing long-lived keys, authenticate your workloads by using the AWS Security Token Service (STS).

Prerequisites

- You have extracted and prepared the **ccocli** binary.
- You have configured an OpenShift Container Platform cluster with AWS STS by using the Cloud Credential Operator in manual mode.

Procedure

1. Create a directory to store a **CredentialsRequest** resource YAML file by running the following command:

```
$ mkdir credentials-request
```

2. Create a **CredentialsRequest** resource YAML file under the **credentials-request** directory, such as, **sample-credential-request.yaml**, by applying the following yaml:

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: cert-manager
  namespace: openshift-cloud-credential-operator
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
```

```

- action:
  - "route53:GetChange"
  effect: Allow
  resource: "arn:aws:route53:::change/*"
- action:
  - "route53:ChangeResourceRecordSets"
  - "route53:ListResourceRecordSets"
  effect: Allow
  resource: "arn:aws:route53:::hostedzone/*"
- action:
  - "route53:ListHostedZonesByName"
  effect: Allow
  resource: "*"
secretRef:
  name: aws-creds
  namespace: cert-manager
serviceAccountNames:
- cert-manager

```

3. Use the **ccoctl** tool to process **CredentialsRequest** objects by running the following command:

```

$ ccoctl aws create-iam-roles \
  --name <user_defined_name> --region=<aws_region> \
  --credentials-requests-dir=<path_to_credrequests_dir> \
  --identity-provider-arn <oidc_provider_arn> --output-dir=<path_to_output_dir>

```

Example output

```

2023/05/15 18:10:34 Role arn:aws:iam::XXXXXXXXXXXX:role/<user_defined_name>-cert-
manager-aws-creds created
2023/05/15 18:10:34 Saved credentials configuration to:
<path_to_output_dir>/manifests/cert-manager-aws-creds-credentials.yaml
2023/05/15 18:10:35 Updated Role policy for Role <user_defined_name>-cert-manager-
aws-creds

```

Copy the **<aws_role_arn>** from the output to use in the next step. For example,
"arn:aws:iam::XXXXXXXXXXXX:role/<user_defined_name>-cert-manager-aws-creds"

4. Add the **eks.amazonaws.com/role-arn=<aws_role_arn>** annotation to the service account by running the following command:

```

$ oc -n cert-manager annotate serviceaccount cert-manager eks.amazonaws.com/role-arn="
<aws_role_arn>"

```

5. To create a new pod, delete the existing cert-manager controller pod by running the following command:

```

$ oc delete pods -l app.kubernetes.io/name=cert-manager -n cert-manager

```

The AWS credentials are applied to a new cert-manager controller pod within a minute.

Verification

1. Get the name of the updated cert-manager controller pod by running the following command:

```
$ oc get pods -l app.kubernetes.io/name=cert-manager -n cert-manager
```

Example output

```
NAME                READY STATUS  RESTARTS  AGE
cert-manager-bd7fbb9fc-wvbbt 1/1   Running  0         39s
```

2. Verify that AWS credentials are updated by running the following command:

```
$ oc set env -n cert-manager po/<cert_manager_controller_pod_name> --list
```

Example output

```
# pods/cert-manager-57f9555c54-vbcpq, container cert-manager-controller
# POD_NAMESPACE from field path metadata.namespace
AWS_ROLE_ARN=XXXXXXXXXXXX
AWS_WEB_IDENTITY_TOKEN_FILE=/var/run/secrets/eks.amazonaws.com/serviceaccount/to
ken
```

Additional resources

- [Configuring the Cloud Credential Operator utility](#)

10.6.3. Authenticating on Google Cloud

To securely access Google Cloud resources, authenticate your workloads on Google Cloud by using the cert-manager Operator for Red Hat OpenShift.

Prerequisites

- You have installed version 1.11.1 or later of the cert-manager Operator for Red Hat OpenShift.
- You have configured the Cloud Credential Operator to operate in *mint* or *passthrough* mode.

Procedure

1. Create a **CredentialsRequest** resource YAML file, such as, **sample-credential-request.yaml** by applying the following yaml:

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: cert-manager
  namespace: openshift-cloud-credential-operator
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: GCPProviderSpec
    predefinedRoles:
      - roles/dns.admin
  secretRef:
    name: gcp-credentials
```

```
namespace: cert-manager
serviceAccountNames:
- cert-manager
```

NOTE

The **dns.admin** role provides admin privileges to the service account for managing Google Cloud DNS resources. To ensure that the cert-manager runs with the service account that has the least privilege, you can create a custom role with the following permissions:

- **dns.resourceRecordSets.***
- **dns.changes.***
- **dns.managedZones.list**

2. Create a **CredentialsRequest** resource by running the following command:

```
$ oc create -f sample-credential-request.yaml
```

3. Update the subscription object for cert-manager Operator for Red Hat OpenShift by running the following command:

```
$ oc -n cert-manager-operator patch subscription openshift-cert-manager-operator --
type=merge -p '{"spec":{"config":{"env":
[{"name":"CLOUD_CREDENTIALS_SECRET_NAME","value":"gcp-credentials"]}}}]'
```

Verification

1. Get the name of the redeployed cert-manager controller pod by running the following command:

```
$ oc get pods -l app.kubernetes.io/name=cert-manager -n cert-manager
```

Example output

```
NAME                                READY STATUS RESTARTS AGE
cert-manager-bd7fbb9fc-wvbbt        1/1   Running 0      15m39s
```

2. Verify that the cert-manager controller pod is updated with Google Cloud credential volumes that are mounted under the path specified in **mountPath** by running the following command:

```
$ oc get -n cert-manager pod/<cert-manager_controller_pod_name> -o yaml
```

Example output

```
spec:
  containers:
  - args:
    ...
    volumeMounts:
```



```

...
- mountPath: /.config/gcloud
  name: cloud-credentials
...
volumes:
...
- name: cloud-credentials
  secret:
    ...
    items:
      - key: service_account.json
        path: application_default_credentials.json
      secretName: gcp-credentials

```

10.6.4. Authenticating with Google Cloud Workload Identity

To securely access Google Cloud resources from your applications without managing long-lived keys, authenticate your workloads by using Google Cloud Workload Identity.

Prerequisites

- You extracted and prepared the **ccctl** binary.
- You have installed version 1.11.1 or later of the cert-manager Operator for Red Hat OpenShift.
- You have configured an OpenShift Container Platform cluster with Google Cloud Workload Identity by using the Cloud Credential Operator in a manual mode.

Procedure

1. Create a directory to store a **CredentialsRequest** resource YAML file by running the following command:

```
$ mkdir credentials-request
```

2. In the **credentials-request** directory, create a YAML file that contains the following **CredentialsRequest** manifest:

```

apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: cert-manager
  namespace: openshift-cloud-credential-operator
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: GCPPProviderSpec
    predefinedRoles:
      - roles/dns.admin
  secretRef:
    name: gcp-credentials
    namespace: cert-manager
  serviceAccountNames:
    - cert-manager

```



NOTE

The **dns.admin** role provides admin privileges to the service account for managing Google Cloud DNS resources. To ensure that the cert-manager runs with the service account that has the least privilege, you can create a custom role with the following permissions:

- **dns.resourceRecordSets.***
- **dns.changes.***
- **dns.managedZones.list**

3. Use the **ccoctl** tool to process **CredentialsRequest** objects by running the following command:

```
$ ccoctl gcp create-service-accounts \
  --name <user_defined_name> --output-dir=<path_to_output_dir> \
  --credentials-requests-dir=<path_to_credrequests_dir> \
  --workload-identity-pool <workload_identity_pool> \
  --workload-identity-provider <workload_identity_provider> \
  --project <gcp_project_id>
```

Example command

```
$ ccoctl gcp create-service-accounts \
  --name abcde-20230525-4bac2781 --output-dir=/home/outputdir \
  --credentials-requests-dir=/home/credentials-requests \
  --workload-identity-pool abcde-20230525-4bac2781 \
  --workload-identity-provider abcde-20230525-4bac2781 \
  --project openshift-gcp-devel
```

4. Apply the secrets generated in the manifests directory of your cluster by running the following command:

```
$ ls <path_to_output_dir>/manifests/*-credentials.yaml | xargs -l{} oc apply -f {}
```

5. Update the subscription object for cert-manager Operator for Red Hat OpenShift by running the following command:

```
$ oc -n cert-manager-operator patch subscription openshift-cert-manager-operator --
type=merge -p '{"spec":{"config":{"env":
[{"name":"CLOUD_CREDENTIALS_SECRET_NAME","value":"gcp-credentials"]}}}]'
```

Verification

1. Get the name of the redeployed cert-manager controller pod by running the following command:

```
$ oc get pods -l app.kubernetes.io/name=cert-manager -n cert-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
cert-manager-bd7fbb9fc-wvbbt	1/1	Running	0	15m39s

2. Verify that the cert-manager controller pod is updated with Google Cloud workload identity credential volumes that are mounted under the path specified in **mountPath** by running the following command:

```
$ oc get -n cert-manager pod/<cert-manager_controller_pod_name> -o yaml
```

Example output

```
spec:
  containers:
  - args:
    ...
    volumeMounts:
    - mountPath: /var/run/secrets/openshift/serviceaccount
      name: bound-sa-token
    ...
    - mountPath: /.config/gcloud
      name: cloud-credentials
    ...
  volumes:
  - name: bound-sa-token
    projected:
    ...
    sources:
    - serviceAccountToken:
        audience: openshift
    ...
    path: token
  - name: cloud-credentials
    secret:
    ...
    items:
    - key: service_account.json
      path: application_default_credentials.json
    secretName: gcp-credentials
```

Additional resources

- [Configuring the Cloud Credential Operator utility](#)
- [Manual mode with short-term credentials for components](#)
- [Default behavior of the Cloud Credential Operator](#)

10.7. CONFIGURING AN ACME ISSUER

The cert-manager Operator for Red Hat OpenShift supports using Automated Certificate Management Environment (ACME) CA servers, such as *Let's Encrypt*, to issue certificates. Explicit credentials are configured by specifying the secret details in the **Issuer** API object. Ambient credentials are extracted from the environment, metadata services, or local files which are not explicitly configured in the **Issuer** API object.

The **Issuer** object is namespace scoped. It can only issue certificates from the same namespace. You can also use the **ClusterIssuer** object to issue certificates across all namespaces in the cluster.

Example YAML file that defines the **ClusterIssuer** object

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: acme-cluster-issuer
spec:
  acme:
    ...
```



NOTE

By default, you can use the **ClusterIssuer** object with ambient credentials. To use the **Issuer** object with ambient credentials, you must enable the **--issuer-ambient-credentials** setting for the cert-manager controller.

10.7.1. About ACME issuers

The ACME issuer type for the cert-manager Operator for Red Hat OpenShift represents an Automated Certificate Management Environment (ACME) certificate authority (CA) server. ACME CA servers rely on a *challenge* to verify that a client owns the domain names that the certificate is being requested for. If the challenge is successful, the cert-manager Operator for Red Hat OpenShift can issue the certificate. If the challenge fails, the cert-manager Operator for Red Hat OpenShift does not issue the certificate.



NOTE

Private DNS zones are not supported with *Let's Encrypt* and internet ACME servers.

10.7.1.1. Supported ACME challenges types

To validate domain ownership with ACME issuers, you can use the challenge types supported by the cert-manager Operator for Red Hat OpenShift.

The cert-manager Operator for Red Hat OpenShift supports the following challenge types for ACME issuers:

HTTP-01

With the HTTP-01 challenge type, you provide a computed key at an HTTP URL endpoint in your domain. If the ACME CA server can get the key from the URL, it can validate you as the owner of the domain.



NOTE

HTTP-01 requires that the Let's Encrypt servers can access the route of the cluster. If an internal or private cluster is behind a proxy, the HTTP-01 validations for certificate issuance fail.

The HTTP-01 challenge is restricted to port 80.

DNS-01

With the DNS-01 challenge type, you provide a computed key at a DNS TXT record. If the ACME CA server can get the key by DNS lookup, it can validate you as the owner of the domain.

10.7.1.2. Supported DNS-01 providers

To configure DNS-01 challenges for ACME issuers, you can validate domain ownership by integrating with supported services, such as Amazon Route 53, Azure DNS, and Google Cloud DNS, or by using Webhooks.

The cert-manager Operator for Red Hat OpenShift supports the following DNS-01 providers for ACME issuers:

- Amazon Route 53
- Azure DNS



NOTE

The cert-manager Operator for Red Hat OpenShift does not support using Microsoft Entra ID pod identities to assign a managed identity to a pod.

- Google Cloud DNS
- Webhook
Red Hat tests and supports DNS providers using an external webhook with cert-manager on OpenShift Container Platform. The following DNS providers are tested and supported with OpenShift Container Platform:

- [cert-manager-webhook-ibmcis](#)



NOTE

Using a DNS provider that is not listed might work with OpenShift Container Platform, but the provider was not tested by Red Hat and therefore is not supported by Red Hat.

10.7.2. Configuring an ACME issuer to solve HTTP-01 challenges

You can use cert-manager Operator for Red Hat OpenShift to set up an ACME issuer to solve HTTP-01 challenges. This procedure uses *Let's Encrypt* as the ACME CA server.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have a service that you want to expose. In this procedure, the service is named **sample-workload**.

Procedure

1. Create an ACME cluster issuer.
 - a. Create a YAML file, **acme-cluster-issuer.yaml**, that defines the **ClusterIssuer** object:

```

apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: <cluster_issuer_name>
spec:
  acme:
    preferredChain: ""
    privateKeySecretRef:
      name: <secret_for_private_key>
    server: <url>
    solvers:
      - http01:
          ingress:
            ingressClassName: <ingress_class_name>

```

where:

<cluster_issuer_name>

Specifies a name for the cluster issuer.

<secret_for_private_key>

Specifies the name of secret to store the ACME account private key in.

<url>

Specifies the URL to access the ACME server's **directory** endpoint. This example uses the *Let's Encrypt* staging environment.

<ingress_class_name>

Specifies the Ingress class, for example, **openshift-default**.

- b. Optional: If you create the object without specifying **ingressClassName**, use the following command to patch the existing ingress:

```

$ oc patch ingress/<ingress-name> --type=merge --patch '{"spec":
{"ingressClassName":"openshift-default"}}' -n <namespace>

```

- c. Create the **ClusterIssuer** object by running the following command:

```

$ oc create -f acme-cluster-issuer.yaml

```

2. Create an Ingress to expose the service of the user workload.

- a. Create a YAML file, for example, **namespace.yaml**, that defines a **Namespace** object:

```

apiVersion: v1
kind: Namespace
metadata:
  name: <ingress_namespace>

```

Replace **<ingress_namespace>** with the namespace for the Ingress.

- b. Create the **Namespace** object by running the following command:

```

$ oc create -f namespace.yaml

```

- c. Create a YAML file, for example, **ingress.yaml**, that defines the **Ingress** object:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: <ingress_name>
  namespace: <ingress_namespace>
  annotations:
    cert-manager.io/cluster-issuer: <cluster_issuer_name>
spec:
  ingressClassName: <ingress_class_name>
  tls:
  - hosts:
    - <tls_hostname>
    secretName: <secret_name>
  rules:
  - host: <hostname>
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: <service_name>
            port:
              number: 80
```

where:

<ingress_name>

Specifies the name of the Ingress.

<ingress_namespace>

Specifies the namespace that you created for the Ingress.

<cluster_issuer_name>

Specifies the cluster issuer that you created.

<ingress_class_name>

Specifies the Ingress class name.

<tls_hostname>

Specifies the Subject Alternative Name (SAN) to be associated with the certificate. This name is used to add DNS names to the certificate.

<secret_name>

Specifies the secret that stores the certificate.

<hostname>

Specifies the host name. You can use the **<host_name>.<cluster_ingress_domain>** syntax to take advantage of the ***.<cluster_ingress_domain>** wildcard DNS record and serving certificate for the cluster. For example, you might use **apps**.

<cluster_base_domain>. Otherwise, you must ensure that a DNS record exists for the chosen hostname.

<service_name>

Specifies the name of the service to expose. This example uses a service named **sample-workload**.

- d. Create the **Ingress** object by running the following command:

```
$ oc create -f ingress.yaml
```

10.7.3. Configuring an ACME issuer by using explicit credentials for AWS Route53

You can use cert-manager Operator for Red Hat OpenShift to set up an Automated Certificate Management Environment (ACME) issuer to solve DNS-01 challenges by using explicit credentials on AWS. This procedure uses *Let's Encrypt* as the ACME certificate authority (CA) server and shows how to solve DNS-01 challenges with Amazon Route 53.

Prerequisites

- You must provide the explicit **accessKeyID** and **secretAccessKey** credentials. For more information, see [Route53](#) in the upstream cert-manager documentation.



NOTE

You can use Amazon Route 53 with explicit credentials in an OpenShift Container Platform cluster that is not running on AWS.

Procedure

- Optional: Override the name server settings for the DNS-01 self check.
This step is required only when the target public-hosted zone overlaps with the cluster's default private-hosted zone.

- a. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

- b. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideArgs:
      - '--dns01-recursive-nameservers-only'
      - '--dns01-recursive-nameservers=1.1.1.1:53'
```

where:

--dns01-recursive-nameservers-only

Specifies recursive name servers instead of checking the authoritative name servers associated with that domain.

--dns01-recursive-nameservers=1.1.1.1:53

Specifies a comma-separated list of **<host>:<port>** names servers to query for the DNS-01 self check. You must use a **1.1.1.1:53** value to avoid the public and private zones overlapping.

c. Save the file to apply the changes.

2. Optional: Create a namespace for the issuer:

```
$ oc new-project <issuer_namespace>
```

3. Create a secret to store your AWS credentials in by running the following command:

```
$ oc create secret -n my-issuer-namespace generic aws-secret \
  --from-literal=awsSecretAccessKey=<aws_secret_access_key>
```

Replace **<aws_secret_access_key>** with your AWS secret access key.

4. Create an issuer:

a. Create a YAML file that defines the **Issuer** object:

Example issuer.yaml file

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: <issuer_name>
  namespace: <issuer_namespace>
spec:
  acme:
    server: <server>
    email: "<email_address>"
    privateKeySecretRef:
      name: <secret_private_key>
    solvers:
      - dns01:
          route53:
            accessKeyID: <aws_key_id>
            hostedZoneID: <hosted_zone_id>
            region: <region_name>
            secretAccessKeySecretRef:
              name: "<aws_secret>"
              key: "<aws_secret_access_key>"
```

where:

<issuer_name>

Specifies a name for the issuer.

<issuer_namespace>

Specifies the namespace that you created for the issuer.

server

Specifies the URL to access the ACME server's **directory** endpoint. This example uses the *Let's Encrypt* staging environment.

<email_address>

Specifies your email address.

<secret_private_key>

Specifies the name of the secret to store the ACME account private key in.

<aws_key_id>

Specifies your AWS key ID.

<hosted_zone_id>

Specifies your hosted zone ID.

<region_name>

Specifies the AWS region name. For example, **us-east-1**.

<aws_secret>

Specifies the name of the secret you created.

<aws_secret_access_key>

Specifies the key in the secret you created that stores your AWS secret access key.

- b. Create the **Issuer** object by running the following command:

```
$ oc create -f issuer.yaml
```

10.7.4. Configuring an ACME issuer by using ambient credentials on AWS

You can use cert-manager Operator for Red Hat OpenShift to set up an ACME issuer to solve DNS-01 challenges by using ambient credentials on AWS. This procedure uses *Let's Encrypt* as the ACME CA server and shows how to solve DNS-01 challenges with Amazon Route 53.

Prerequisites

- If your cluster is configured to use the AWS Security Token Service (STS), you followed the instructions from the *Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift for the AWS Security Token Service cluster* section.
- If your cluster does not use the AWS STS, you followed the instructions from the *Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift on AWS* section.

Procedure

1. Optional: Override the name server settings for the DNS-01 self check.
This step is required only when the target public-hosted zone overlaps with the cluster's default private-hosted zone.
 - a. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

- b. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
```

```
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideArgs:
      - '--dns01-recursive-nameservers-only'
      - '--dns01-recursive-nameservers=1.1.1.1:53'
```

where:

--dns01-recursive-nameservers-only

Specifies recursive name servers instead of checking the authoritative name servers associated with that domain.

--dns01-recursive-nameservers=1.1.1.1:53

Specifies a comma-separated list of **<host>:<port>** name servers to query for the DNS-01 self check. You must use a **1.1.1.1:53** value to avoid the public and private zones overlapping.

- c. Save the file to apply the changes.
2. Optional: Create a namespace for the issuer:

```
$ oc new-project <issuer_namespace>
```

3. Modify the **CertManager** resource to add the **--issuer-ambient-credentials** argument:

```
$ oc patch certmanager/cluster \
  --type=merge \
  -p='{"spec":{"controllerConfig":{"overrideArgs":["--issuer-ambient-credentials"]}}}'
```

4. Create an issuer:
 - a. Create a YAML file that defines the **Issuer** object:

Example issuer.yaml file

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: <issuer_name>
  namespace: <issuer_namespace>
spec:
  acme:
    server: <server>
    email: "<email_address>"
    privateKeySecretRef:
      name: <secret_private_key>
    solvers:
      - dns01:
```

```
route53:
  hostedZoneID: <hosted_zone_id>
  region: us-east-1
```

where:

<issuer_name>

Specifies a name for the issuer.

<issuer_namespace>

Specifies the namespace that you created for the issuer.

<server>

Specifies the URL to access the ACME server's **directory** endpoint. This example uses the *Let's Encrypt* staging environment.

<email_address>

Specifies your email address.

<secret_private_key>

Specifies the name of the secret to store the ACME account private key in.

<hosted_zone_id>

Specifies your hosted zone ID.

- b. Create the **Issuer** object by running the following command:

```
$ oc create -f issuer.yaml
```

10.7.5. Configuring an ACME issuer by using explicit credentials for Google Cloud DNS

You can use the cert-manager Operator for Red Hat OpenShift to set up an ACME issuer to solve DNS-01 challenges by using explicit credentials on Google Cloud. This procedure uses *Let's Encrypt* as the ACME CA server and shows how to solve DNS-01 challenges with Google Cloud DNS.

Prerequisites

- You have set up a Google Cloud service account with a desired role for Google Cloud DNS.



NOTE

You can use Google Cloud DNS with explicit credentials in an OpenShift Container Platform cluster that is not running on Google Cloud.

Procedure

1. Optional: Override the name server settings for the DNS-01 self check.
This step is required only when the target public-hosted zone overlaps with the cluster's default private-hosted zone.
 - a. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

- b. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideArgs:
      - '--dns01-recursive-nameservers-only'
      - '--dns01-recursive-nameservers=1.1.1.1:53'
```

where:

--dns01-recursive-nameservers-only

Specifies recursive name servers instead of checking the authoritative name servers associated with that domain.

--dns01-recursive-nameservers=1.1.1.1:53

Specifies a comma-separated list of **<host>:<port>** name servers to query for the DNS-01 self check. You must use a **1.1.1.1:53** value to avoid the public and private zones overlapping.

- c. Save the file to apply the changes.
2. Optional: Create a namespace for the issuer:

```
$ oc new-project my-issuer-namespace
```

3. Create a secret to store your Google Cloud credentials by running the following command:

```
$ oc create secret generic clouddns-dns01-solver-svc-acct --from-file=service_account.json=
<path/to/gcp_service_account.json> -n my-issuer-namespace
```

4. Create an issuer:

- a. Create a YAML file, for example, **issuer.yaml**, that defines the **Issuer** object:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: <acme_dns01_clouddns_issuer>
  namespace: <issuer_namespace>
spec:
  acme:
    preferredChain: ""
    privateKeySecretRef:
      name: <secret_private_key>
    server: <server>
    solvers:
      - dns01:
          cloudDNS:
            project: <project_id>
```

```

serviceAccountSecretRef:
  name: <secret>
  key: <service_account.json>

```

where:

<acme_dns01_clouddns_issuer>

Specifies a name for the issuer.

<issuer_namespace>

Specifies your issuer namespace.

<secret_private_key>

Specifies the name of the secret to store the ACME account private key in.

<server>

Specifies the URL to access the ACME server's **directory** endpoint. This example uses the *Let's Encrypt* staging environment.

<project_id>

Specifies the name of the Google Cloud project that contains the Cloud DNS zone.

<secret>

Specifies the name of the secret you created.

<service_account.json>

Specifies the key in the secret you created that stores your Google Cloud secret access key.

- b. Create the **Issuer** object by running the following command:

```
$ oc create -f issuer.yaml
```

10.7.6. Configuring an ACME issuer by using ambient credentials on Google Cloud

You can use the cert-manager Operator for Red Hat OpenShift to set up an ACME issuer to solve DNS-01 challenges by using ambient credentials on Google Cloud. This procedure uses *Let's Encrypt* as the ACME CA server and shows how to solve DNS-01 challenges with Google Cloud DNS.

Prerequisites

- If your cluster is configured to use Google Cloud Workload Identity, you followed the instructions from the *Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift with Google Cloud Workload Identity* section.
- If your cluster does not use Google Cloud Workload Identity, you followed the instructions from the *Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift on Google Cloud* section.

Procedure

1. Optional: Override the name server settings for the DNS-01 self check.
This step is required only when the target public-hosted zone overlaps with the cluster's default private-hosted zone.
 - a. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```

- b. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideArgs:
      - '--dns01-recursive-nameservers-only'
      - '--dns01-recursive-nameservers=1.1.1.1:53'
```

where:

--dns01-recursive-nameservers-only

Specifies recursive name servers instead of checking the authoritative name servers associated with that domain.

--dns01-recursive-nameservers=1.1.1.1:53

Specifies a comma-separated list of **<host>:<port>** name servers to query for the DNS-01 self check. You must use a **1.1.1.1:53** value to avoid the public and private zones overlapping.

- c. Save the file to apply the changes.
2. Optional: Create a namespace for the issuer:

```
$ oc new-project <issuer_namespace>
```

3. Modify the **CertManager** resource to add the **--issuer-ambient-credentials** argument:

```
$ oc patch certmanager/cluster \
  --type=merge \
  -p='{ "spec": { "controllerConfig": { "overrideArgs": [ "--issuer-ambient-credentials" ] } } }'
```

4. Create an issuer:

- a. Create a YAML file that defines the **Issuer** object:

Example issuer.yaml file

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: <issuer_name>
  namespace: <issuer_namespace>
spec:
  acme:
    preferredChain: ""
    privateKeySecretRef:
```

```

    name: <secret_private_key>
    server: <server>
    solvers:
    - dns01:
        cloudDNS:
            project: <gcp_project_id>

```

where:

<issuer_name>

Specifies a name for the issuer.

<issuer_namespace>

Specifies a namespace for the issuer.

<secret_private_key>

Specifies the name of the secret to store the ACME account private key in.

<server>

Specifies the URL to access the ACME server's **directory** endpoint. This example uses the *Let's Encrypt* staging environment.

<gcp_project_id>

Specifies the name of the Google Cloud project that contains the Cloud DNS zone.

- b. Create the **Issuer** object by running the following command:

```
$ oc create -f issuer.yaml
```

10.7.7. Configuring an ACME issuer by using explicit credentials for Microsoft Azure DNS

You can use cert-manager Operator for Red Hat OpenShift to set up an ACME issuer to solve DNS-01 challenges by using explicit credentials on Microsoft Azure. This procedure uses *Let's Encrypt* as the ACME CA server and shows how to solve DNS-01 challenges with Azure DNS.

Prerequisites

- You have set up a service principal with desired role for Azure DNS.



NOTE

You can follow this procedure for an OpenShift Container Platform cluster that is not running on Microsoft Azure.

Procedure

1. Optional: Override the nameserver settings for the DNS-01 self check.
This step is required only when the target public-hosted zone overlaps with the cluster's default private-hosted zone.
 - a. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager cluster
```


- b. Add a **spec.controllerConfig** section with the following override arguments:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
  ...
spec:
  ...
  controllerConfig:
    overrideArgs:
      - '--dns01-recursive-nameservers-only'
      - '--dns01-recursive-nameservers=1.1.1.1:53'
```

where:

--dns01-recursive-nameservers-only

Specifies recursive name servers instead of checking the authoritative name servers associated with that domain.

--dns01-recursive-nameservers=1.1.1.1:53

Specifies a comma-separated list of **<host>:<port>** name servers to query for the DNS-01 self check. You must use a **1.1.1.1:53** value to avoid the public and private zones overlapping.

- c. Save the file to apply the changes.
2. Optional: Create a namespace for the issuer:

```
$ oc new-project my-issuer-namespace
```

3. Create a secret to store your Azure credentials in by running the following command:

```
$ oc create secret generic <secret_name> --from-literal=
<azure_secret_access_key_name>=<azure_secret_access_key_value> \
-n my-issuer-namespace
```

- Replace **<secret_name>** with your secret name.
 - Replace **<azure_secret_access_key_name>** with your Azure secret access key name.
 - Replace **<azure_secret_access_key_value>** with your Azure secret key.
4. Create an issuer:
- a. Create a YAML file, for example, **issuer.yaml**, that defines the **Issuer** object:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: <acme-dns01-azuredns-issuer>
  namespace: <issuer_namespace>
spec:
  acme:
    preferredChain: ""
```

```

privateKeySecretRef:
  name: <secret_private_key>
server: <server>
solvers:
- dns01:
  azureDNS:
    clientID: <azure_client_id>
    clientSecretSecretRef:
      name: <secret_name>
      key: <azure_secret_access_key_name>
    subscriptionID: <azure_subscription_id>
    tenantID: <azure_tenant_id>
    resourceGroupName: <azure_dns_zone_resource_group>
    hostedZoneName: <azure_dns_zone>
    environment: AzurePublicCloud

```

where:

<acme-dns01-azuredns-issuer>

Specifies a name for the issuer.

<issuer_namespace>

Specifies your issuer namespace.

<secret_private_key>

Specifies the name of the secret to store the ACME account private key in.

<server>

Specifies the URL to access the ACME server's **directory** endpoint. This example uses the *Let's Encrypt* staging environment.

<azure_client_id>

Specifies your Azure client ID.

<secret_name>

Specifies a name of the client secret.

<azure_secret_access_key_name>

Specifies the client secret key name.

<azure_subscription_id>

Specifies your Azure subscription ID.

<azure_tenant_id>

Specifies your Azure tenant ID.

<azure_dns_zone_resource_group>

Specifies the name of the Azure DNS zone resource group.

<azure_dns_zone>

Specifies the name of Azure DNS zone.

- b. Create the **Issuer** object by running the following command:

```
$ oc create -f issuer.yaml
```

10.7.8. Additional resources

- [Azure DNS](#)
- [Google Cloud DNS](#)
- [Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift for the AWS Security Token Service cluster](#)
- [Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift on AWS](#)
- [Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift with Google Cloud Workload Identity](#)
- [Configuring cloud credentials for the cert-manager Operator for Red Hat OpenShift on Google Cloud](#)
- [HTTP01](#)
- [HTTP-01 challenge](#)
- [DNS01](#)

10.8. CONFIGURING CERTIFICATES WITH AN ISSUER

By using the cert-manager Operator for Red Hat OpenShift, you can manage certificates, handling tasks such as renewal and issuance, for workloads within the cluster, as well as components interacting externally to the cluster.

10.8.1. Creating certificates for user workloads

To secure communications for your applications, create and manage TLS certificates for your workloads by using the cert-manager Operator for Red Hat OpenShift

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed the cert-manager Operator for Red Hat OpenShift.

Procedure

1. Create an issuer. For more information, see "Configuring an issuer" in the "Additional resources" section.
2. Create a certificate:
 - a. Create a YAML file, for example, **certificate.yaml**, that defines the **Certificate** object:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: <tls_cert>
  namespace: <issuer_namespace>
spec:
  isCA: false
  commonName: '<common_name>'
  secretName: <secret_name>
```

```

dnsNames:
- "<domain_name>"
issuerRef:
  name: <issuer_name>
  kind: Issuer

```

where:

<tls_cert>

Specifies a name for the certificate.

<issuer_namespace>

Specifies the namespace of the issuer.

<common_name>

Specifies the common name (CN).

<secret_name>

Specifies the name of the secret to create that contains the certificate.

<domain_name>

Specifies the domain name.

<issuer_name>

Specifies the name of the issuer.

- b. Create the **Certificate** object by running the following command:

```
$ oc create -f certificate.yaml
```

Verification

- Verify that the certificate is created and ready to use by running the following command:

```
$ oc get certificate -w -n <issuer_namespace>
```

Once certificate is in **Ready** status, workloads on your cluster can start using the generated certificate secret.

10.8.2. Creating certificates for the API server

To secure interactions with the cluster control plane, create TLS certificates for the API server by using the cert-manager Operator for Red Hat OpenShift.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed version 1.13.0 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

1. Create an issuer. For more information, see "Configuring an issuer" in the "Additional resources" section.

2. Create a certificate:

- a. Create a YAML file, for example, **certificate.yaml**, that defines the **Certificate** object:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: <tls_cert>
  namespace: openshift-config
spec:
  isCA: false
  commonName: "api.<cluster_base_domain>"
  secretName: <secret_name>
  dnsNames:
  - "api.<cluster_base_domain>"
  issuerRef:
    name: <issuer_name>
    kind: Issuer
```

where:

<tls_cert>

Specifies a name for the certificate.

<cluster_base_domain>

Specifies the common name (CN).

<secret_name>

Specifies the name of the secret to create that contains the certificate.

<issuer_name>

Specifies the name of the issuer.

- b. Create the **Certificate** object by running the following command:

```
$ oc create -f certificate.yaml
```

3. Add the API server named certificate. For more information, see "Adding an API server named certificate" section in the "Additional resources" section.



NOTE

To ensure the certificates are updated, run the **oc login** command again after the certificate is created.

Verification

- Verify that the certificate is created and ready to use by running the following command:

```
$ oc get certificate -w -n openshift-config
```

Once certificate is in **Ready** status, API server on your cluster can start using the generated certificate secret.

10.8.3. Creating certificates for the Ingress Controller

You can create a certificate for the Ingress Controller and then replace bootstrapped default self-signed certificates with cert-manager-managed external certificates.



NOTE

Before using the procedure, ensure you understand the following Ingress Controller behaviors:

- When certificates are renewed or rotated by using the cert-manager Operator, only the contents of the secret, such as the certificate and key, are updated. The secret name remains unchanged. Kubelet automatically propagates these updates to the mounted volume, allowing the router to detect the file changes and hot-reload the new certificate and key. As a result, no rolling update of the router deployment is triggered or required.
- The secret name is referenced in the Ingress Controller configuration. If you want to replace the default ingress certificate or use different secret name in Ingress Controller configuration, you must patch or edit the configuration to apply the change. This operation triggers a rolling update for router pods where new router pods load the new cert/key pair.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed version 1.13.0 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

1. Create an issuer. For more information, see "Configuring an issuer" in the "Additional resources" section.
2. Create a certificate:
 - a. Create a YAML file, for example, **certificate.yaml**, that defines the **Certificate** object:

Example **certificate.yaml** file

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: <tls_cert>
  namespace: openshift-ingress
spec:
  isCA: false
  commonName: "apps.<cluster_base_domain>"
  secretName: <secret_name>
  dnsNames:
  - "apps.<cluster_base_domain>"
  - "*.apps.<cluster_base_domain>"
  issuerRef:
    name: <issuer_name>
    kind: Issuer
```

where:

<tls_cert>

Specifies the name for the certificate.

<cluster_base_domain>

Specifies the common name (CN).

<secret_name>

Specifies the name of the secret to create that contains the certificate.

<cluster_base_domain>

Specifies the DNS name of the ingress.

<issuer_name>

Specifies the name of the issuer.

- b. Create the **Certificate** object by running the following command:

```
$ oc create -f certificate.yaml
```

3. Replace the default ingress certificate. For more information, see "Replacing the default ingress certificate" section in the "Additional resources" section.

Verification

1. Verify that the certificate is created and ready to use by running the following command:

```
$ oc get certificate -n openshift-ingress
```

2. Verify the definition and content of the secret object by running the following command:

```
$ oc get secret <secretName> -n openshift-ingress
```

3. Verify that the default TLS certificate has the correct configuration details for the Ingress Controller by running the following command:

```
$ oc get ingresscontroller default -n openshift-ingress-operator -o yaml | grep -A2 defaultCertificate
```

After the certificate is in **Ready** status, the Ingress Controller on your cluster can start using the generated certificate secret.

Additional resources

- [Red Hat Knowledgebase solution](#)

10.8.4. Additional resources

- [Supported issuer types](#)
- [Configuring an ACME issuer](#)
- [Adding an API server named certificate](#)

- [Replacing the default ingress certificate](#)

10.9. SECURING ROUTES WITH THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

In the OpenShift Container Platform, the route API is extended to provide a configurable option to reference TLS certificates via secrets. With [externally managed certificates](#) enabled, you can minimize errors from manual intervention, streamline the certificate management process, and enable the OpenShift Container Platform router to promptly serve the referenced certificate.

10.9.1. Configuring certificates to secure routes in your cluster

To encrypt traffic between external clients and your applications, configure certificates for routes in your OpenShift Container Platform cluster. You can secure your routes by defining TLS termination types, such as edge, passthrough, or re-encrypt, to match your specific security policies.

Prerequisites

- You have installed version 1.14.0 or later of the cert-manager Operator for Red Hat OpenShift.
- You have **create** permission on the **routes/custom-host** sub-resource, which is used for both creating and updating routes.
- You have a **Service** resource that you want to expose.

Procedure

1. Create an **Issuer** to configure the HTTP-01 solver by running the following command. For other ACME issuer types, see "Configuring ACME an issuer".

```
$ oc create -f - << EOF
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: letsencrypt-acme
  namespace: <namespace>
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-acme-account-key
    solvers:
      - http01:
          ingress:
            ingressClassName: openshift-default
EOF
```

where:

<namespace>

Specifies the namespace where the **Issuer** is located. It must be the same as the route namespace.

2. Create a **Certificate** object for the route by running the following command. The **secretName** specifies the TLS secret that is going to be issued and managed by cert-manager and will also be referenced in your route in the following steps.

```
$ oc create -f - << EOF
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: example-route-cert
  namespace: <namespace>
spec:
  commonName: <common_host>
  dnsNames:
    - <hostname>
  usages:
    - server auth
  issuerRef:
    kind: Issuer
    name: letsencrypt-acme
  secretName: <secret_name>
EOF
```

where:

<namespace>

Specifies the **namespace** where the **Certificate** resource is located. It should be the same as the namespace of your route.

<common_host>

Specifies the common name of your certificate by using the hostname of the route.

<hostname>

Specifies the hostname of your route to the DNS names of your certificate.

<secret_name>

Specifies the name of the secret that contains the certificate.

3. Create a **Role** to provide the router service account permissions to read the referenced secret by using the following command:

```
$ oc create role secret-reader \
  --verb=get,list,watch \
  --resource=secrets \
  --resource-name=<secret_name> \
  --namespace=<namespace>
```

where:

<secret_name>

Specifies the name of the secret that you want to grant access to. It should be consistent with your **secretName** specified in the **Certificate** resource.

<namespace>

Specifies the namespace where both your secret and route are located.

4. Create a **RoleBinding** resource to bind the router service account with the newly created **Role** resource by using the following command:

```
$ oc create rolebinding secret-reader-binding \
  --role=secret-reader \
  --serviceaccount=openshift-ingress:router \
  --namespace=<namespace>
```

where:

<namespace>

Specifies the namespace where both your secret and route are located.

5. Create a route for your service resource, that uses edge TLS termination and a custom hostname, by running the following command. The hostname is used when creating a **Certificate** resource in the next step.

```
$ oc create route edge <route_name> \
  --service=<service_name> \
  --hostname=<hostname> \
  --namespace=<namespace>
```

where:

<route_name>

Specifies the name of your route.

<service_name>

Specifies the service you want to expose.

<hostname>

Specifies the hostname of your route.

<namespace>

Specifies the namespace where your route is located.

6. To reference the secret and use the certificate issued by **cert-manager**, update the **.spec.tls.externalCertificate** field in the route by using the following command:

```
$ oc patch route <route_name> \
  -n <namespace> \
  --type=merge \
  -p '{"spec":{"tls":{"externalCertificate":{"name":"<secret_name>"}}}}'
```

where:

<route_name>

Specifies the name of your route.

<namespace>

Specifies the namespace where both your secret and route are located.

<secret_name>

Specifies the name of the secret that contains the certificate.

Verification

1. Verify that the certificate is created and ready to use by running the following command:

```
$ oc get certificate -n <namespace>
$ oc get secret -n <namespace>
```

where:

<namespace>

Specifies the namespace where both your secret and route are located.

2. Verify that the router is using the referenced external certificate by running the following command. The command should return with the status code **200 OK**.

```
$ curl -IsS https://<hostname>
```

where:

<hostname>

Specifies the hostname of your route.

3. Verify the **subject**, **subjectAltName**, and **issuer** fields of your server certificate are all as expected from the curl verbose outputs by running the following command:

```
$ curl -v https://<hostname>
```

where:

<hostname>

Specifies the hostname of your route.

The certificate from the referenced secret secures the route. The **cert-manager** component issues the certificate and automatically manages the certificate lifecycle.

10.9.2. Additional resources

- [Creating a route with externally managed certificate](#)
- [Configuring an ACME issuer](#)

10.10. INTEGRATING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT WITH ISTIO-CSR

The cert-manager Operator for Red Hat OpenShift provides enhanced support for securing workloads and control plane components in Red Hat OpenShift Service Mesh or Istio. This includes support for certificates enabling mutual TLS (mTLS), which are signed, delivered, and renewed using cert-manager issuers. You can secure Istio workloads and control plane components by using the cert-manager Operator for Red Hat OpenShift managed Istio-CSR agent.

With this Istio-CSR integration, Istio can now obtain certificates from the cert-manager Operator for Red Hat OpenShift, simplifying security and certificate management.

10.10.1. Creating a root CA issuer for the Istio-CSR agent

To enable certificate signing for the Istio-CSR agent, configure a root CA issuer using the cert-manager Operator for Red Hat OpenShift. You can establish a trusted root by using the cert-manager Operator for Red Hat OpenShift to ensure secure communication between workloads.



NOTE

Other supported issuers can be used, except for the ACME issuer, which is not supported. For more information, see "cert-manager Operator for Red Hat OpenShift issuer providers".

Procedure

1. Create a YAML file that defines the **Issuer** and **Certificate** objects by using the following example configuration:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned
  namespace: <istio_project_name>
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: istio-ca
  namespace: <istio_project_name>
spec:
  isCA: true
  duration: 87600h # 10 years
  secretName: istio-ca
  commonName: istio-ca
  privateKey:
    algorithm: ECDSA
    size: 256
  subject:
    organizations:
      - cluster.local
      - cert-manager
  issuerRef:
    name: selfsigned
    kind: Issuer
    group: cert-manager.io
---
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: istio-ca
  namespace: <istio_project_name>
spec:
  ca:
    secretName: istio-ca
```

where:

Issuer

Specifies the **Issuer** or **ClusterIssuer**.

<istio_project_name>

Specifies the name of the Istio project.

Verification

- Verify that the Issuer is created and ready to use by running the following command:

```
$ oc get issuer istio-ca -n <istio_project_name>
```

Example output

```
NAME      READY  AGE
istio-ca  True   3m
```

Additional resources

- [cert-manager Operator for Red Hat OpenShift issuer providers](#)

10.10.2. Creating the IstioCSR custom resource

To secure your communications, install the Istio-CSR agent by creating the **IstioCSR** custom resource through the cert-manager Operator for Red Hat OpenShift.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have enabled the Istio-CSR feature.
- You have created the **Issuer** or **ClusterIssuer** resources required for generating certificates for the Istio-CSR agent.



NOTE

If you are using **Issuer** resource, create the **Issuer** and **Certificate** resources in the Red Hat OpenShift Service Mesh or **Istiod** namespace. Certificate requests are generated in the same namespace, and role-based access control (RBAC) is configured accordingly.

Procedure

1. Create a new project for installing Istio-CSR by running the following command. If you have an existing project for installing Istio-CSR, skip this step.

```
$ oc new-project <istio_csr_project_name>
```

2. Create the **IstioCSR** custom resource to enable Istio-CSR agent managed by the cert-manager Operator for Red Hat OpenShift for processing Istio workload and control plane certificate signing requests.



NOTE

Only one **IstioCSR** custom resource (CR) is supported at a time. If multiple **IstioCSR** CRs are created, only one will be active. Use the **status** sub-resource of **IstioCSR** to check if a resource is unprocessed.

- If multiple **IstioCSR** CRs are created simultaneously, none will be processed.
- If multiple **IstioCSR** CRs are created sequentially, only the first one will be processed.
- To prevent new requests from being rejected, delete any unprocessed **IstioCSR** CRs.
- The Operator does not automatically remove objects created for **IstioCSR**. If an active **IstioCSR** resource is deleted and a new one is created in a different namespace without removing the previous deployments, multiple **istio-csr** deployments may remain active. This behavior is not recommended and is not supported.

- a. Create a YAML file that defines the **IstioCSR** object by using the following example:

```
apiVersion: operator.openshift.io/v1alpha1
kind: IstioCSR
metadata:
  name: default
  namespace: <istio_csr_project_name>
spec:
  istioCSRConfig:
    certManager:
      issuerRef:
        name: istio-ca
        kind: Issuer
        group: cert-manager.io
    istiodTLSConfig:
      trustDomain: cluster.local
  istio:
    namespace: <istio_project_name>
```

where:

name

Specifies the **Issuer** or **ClusterIssuer** name. It should be the same name as the CA issuer defined in the **issuer.yaml** file.

kind

Specifies the **Issuer** or **ClusterIssuer** kind. It should be the same kind as the CA issuer defined in the **issuer.yaml** file.

- b. Create the **IstioCSR** custom resource by running the following command:

—

```
$ oc create -f IstioCSR.yaml
```

Verification

1. Verify that the Istio-CSR deployment is ready by running the following command:

```
$ oc get deployment -n <istio_csr_project_name>
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
cert-manager-istio-csr	1/1	1	1	24s

2. Verify that the Istio-CSR pods are running by running the following command:

```
$ oc get pod -n <istio_csr_project_name>
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
cert-manager-istio-csr-5c979f9b7c-bv57w	1/1	Running	0	45s

- Verify that the Istio-CSR pod is not reporting any errors in the logs by running the following command:

```
$ oc -n <istio_csr_project_name> logs <istio_csr_pod_name>
```

- Verify that the cert-manager Operator for Red Hat OpenShift pod is not reporting any errors by running the following command:

```
$ oc -n cert-manager-operator logs <cert_manager_operator_pod_name>
```

10.10.3. Setting the log level for the istio-csr component

You can set the log level for the istio-csr component to control the verbosity and format of its log messages.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **IstioCSR** custom resource (CR).

Procedure

1. Edit the **IstioCSR** CR by running the following command:

```
$ oc edit istiocsr.operator.openshift.io default -n <istio_csr_project_name> 1
```

Replace **<istio_csr_project_name>** with the namespace where you created the **IstioCSR** CR.

- Configure the log level and format in the **spec.istioCSRConfig** section by using the following example configuration:

```
apiVersion: operator.openshift.io/v1alpha1
kind: IstioCSR
...
spec:
  istioCSRConfig:
    logFormat: text
    logLevel: 2
# ...
```

where:

istioCSRConfig.logFormat

Specifies the log output format. You can set this field to either **text** or **json**.

istioCSRConfig.logLevel

Specifies the log level. Supported values are in the range **1** through **5**, as defined by Kubernetes logging guidelines. The default value is **1**.

- Save and close the editor to apply your changes. After the changes are applied, the cert-manager Operator updates the log configuration for the istio-csr operand.

10.10.4. Configuring the namespace selector for CA bundle distribution

The Istio-CSR agent creates and updates the **istio-ca-root-cert ConfigMap**, which contains the CA bundle. Workloads in the service mesh use this CA bundle to validate connections to the Istio control plane. You can configure a namespace selector to specify the namespaces in which the Istio-CSR agent creates this **ConfigMap**. If you do not configure a selector, the Istio-CSR agent creates the **ConfigMap** in all namespaces.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **IstioCSR** custom resource (CR).

Procedure

- Edit the **IstioCSR** CR by running the following command:

```
oc edit istiocsr.operator.openshift.io default -n <istio_csr_project_name>
```

Replace **<istio_csr_project_name>** with the namespace where you created the **IstioCSR** CR.

- Configure the **spec.istioCSRConfig.istioDataPlaneNamespaceSelector** section to set the namespace selector. See the following example:

```
apiVersion: operator.openshift.io/v1alpha1
kind: IstioCSR
...
spec:
```



```
istioCSRConfig:
  istioDataPlaneNamespaceSelector: maistra.io/member-of=istio-system
# ...
```

The **maistra.io/member-of=istio-system** namespace selector defines the label key and value that identify the namespaces in your service mesh. Use the **<key>=<value>** format.

NOTE

The istio-csr component does not delete or manage **ConfigMap** objects in namespaces that do not match the configured selector. If you create or update the selector after deploying the **IstioCSR** CR, or if you remove a label from a namespace, you must manually delete these **ConfigMap** objects to avoid conflicts.

You can run the following command to list **ConfigMap** objects that are not in namespaces matching the selector. In this example, the selector is **maistra.io/member-of=istio-system**:

```
printf "%-25s %10s\n" "ConfigMap" "Namespace"; \
for ns in $(oc get namespaces -l "maistra.io/member-of!=istio-system" -
o=jsonpath='{.items[*].metadata.name}'); do \
  oc get configmaps -l "istio.io/config=true" -n $ns --no-headers -o
jsonpath='{.items[*].metadata.name}{"\t"}{.items[*].metadata.namespace}{"\n"}'
--ignore-not-found; \
done
```

3. Save and close the editor to apply your changes. After the changes are applied, the cert-manager Operator for Red Hat OpenShift updates the namespace selector configuration for the istio-csr operand.

10.10.5. Configuring the CA certificate for the Istio server

You can configure the **ConfigMap** that contains the CA bundle used by Istio workloads to verify the Istio server certificate. If not configured, the cert-manager Operator for Red Hat OpenShift looks for the CA certificate in the configured issuer and in the Kubernetes Secret that contains the Istio certificates.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **IstioCSR** custom resource (CR).

Procedure

1. Edit the **IstioCSR** CR by running the following command:

```
$ oc edit istiocsrs.operator.openshift.io default -n <istio_csr_project_name>
```

Replace **<istio_csr_project_name>** with the namespace where you created the **IstioCSR** CR.

2. Configure the CA bundle by editing the **spec.istioCSRConfig.certManager** section. See the following example:

```

apiVersion: operator.openshift.io/v1alpha1
kind: IstioCSR
...
spec:
  istioCSRConfig:
    certManager:
      istioCACertificate:
        key: <key_in_the_configmap>
        name: <configmap_name>
        namespace: <configmap_namespace>

```

where:

<key_in_the_configmap>

Specifies the key name in the **ConfigMap** that contains the CA bundle.

<configmap_name>

Specifies the name of the **ConfigMap**. Ensure that the referenced **ConfigMap** and key exist before you update this field.

<configmap_namespace>

Optional. Specifies the namespace where the **ConfigMap** exists. If you do not set this field, the cert-manager Operator for Red Hat OpenShift searches for the **ConfigMap** in the namespace where you have installed the **IstioCSR** CR.



NOTE

Whenever the CA certificate is rotated, you must manually update the **ConfigMap** with the latest certificate.

3. Save and close the editor to apply your changes. After the changes are applied, the cert-manager Operator updates the CA bundle for the **istio-csr** operand.

10.10.6. Uninstalling the Istio-CSR agent managed by cert-manager Operator for Red Hat OpenShift

You can uninstall the Istio-CSR agent managed by the cert-manager Operator for Red Hat OpenShift to remove the agent and its associated resources.

Prerequisites

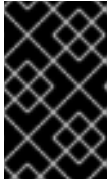
- You have access to the cluster with **cluster-admin** privileges.
- You have enabled the Istio-CSR feature.
- You have created the **IstioCSR** custom resource.

Procedure

1. Remove the **IstioCSR** custom resource by running the following command:

```
$ oc -n <istio_csr_project_name> delete istiocsrs.operator.openshift.io default
```

2. Remove related resources:

**IMPORTANT**

To avoid disrupting any Red Hat OpenShift Service Mesh or Istio components, ensure that no component is referencing the Istio-CSR service or the certificates issued for Istio before removing the following resources.

- a. List the cluster scoped-resources by running the following command and save the names of the listed resources for later reference:

```
$ oc get clusterrolebindings,clusterroles -l "app=cert-manager-istio-csr,app.kubernetes.io/name=cert-manager-istio-csr"
```

- b. List the resources in Istio-csr deployed namespace by running the following command and save the names of the listed resources for later reference:

```
$ oc get certificate,deployments,services,serviceaccounts -l "app=cert-manager-istio-csr,app.kubernetes.io/name=cert-manager-istio-csr" -n <istio_csr_project_name>
```

- c. List the resources in Red Hat OpenShift Service Mesh or Istio deployed namespaces by running the following command and save the names of the listed resources for later reference:

```
$ oc get roles,rolebindings -l "app=cert-manager-istio-csr,app.kubernetes.io/name=cert-manager-istio-csr" -n <istio_csr_project_name>
```

- d. For each resource listed in previous steps, delete the resource by running the following command:

```
$ oc -n <istio_csr_project_name> delete <resource_type>/<resource_name>
```

Repeat this process until all of the related resources have been deleted.

10.11. NETWORK POLICY CONFIGURATION FOR CERT-MANAGER OPERATOR

The cert-manager Operator for Red Hat OpenShift provides predefined **NetworkPolicy** resources to enhance security by controlling the ingress and egress traffic for its components. By default, this feature is disabled to prevent connectivity issues or breaking changes during an upgrade. To use this feature, you must enable it in the **CertManager** custom resource (CR).

After enabling the default policies, you must manually configure additional egress rules to allow outbound traffic. These rules are required for cert-manager Operator for Red Hat OpenShift to communicate with external services beyond the API server and internal DNS.

The examples of services that require custom egress rules include the following:

- ACME servers, for example, Let's Encrypt
- DNS-01 challenge providers, for example, AWS Route53 or Cloudflare
- External CAs, such as HashiCorp Vault

**NOTE**

Network policies are expected to be enabled by default in a future release, which could cause connectivity failures during an upgrade. To prepare for this change, configure the required egress policies.

10.11.1. Default ingress and egress rules

The default network policy applies the following ingress and egress rules to each component.

Component	Ingress ports	Egress ports	Description
cert-manager	9402	6443, 5353	Allows ingress traffic to metrics server and egress traffic to OpenShift API server.
cert-manager-webhook	9402, 10250	6443	Allows ingress traffic to metrics and webhook servers, and egress traffic to OpenShift API server and internal DNS server.
cert-manager-cainjector	9402	6443	Allows ingress traffic to metrics server and egress traffic to OpenShift API server.
istio-csr	6443, 9402	6443	Allows ingress traffic to the gRPC Istio certificate request API, metrics servers and egress traffic to OpenShift API server.

10.11.2. Network policy configuration parameters

You can enable and configure network policies for the cert-manager Operator components by updating the **CertManager** custom resource (CR). The CR includes the following parameters for enabling default network policies and defining custom egress rules.

Field	Type	Description
-------	------	-------------

Field	Type	Description
spec.defaultNetworkPolicy	boolean	Specifies whether to enable the default network policy for the cert-manager Operator components.  IMPORTANT Once you enable default network policies, you cannot disable them. This restriction prevents accidental security degradation. Before enabling this setting, ensure that you plan the network policy requirements.
spec.networkPolicies	object	Defines a list of custom network policy configuration. To apply the configuration, you must set spec.defaultNetworkPolicy to true .
spec.networkPolicies.componentName	string	Specifies the component that this network policy targets. The only valid value is CoreController .
spec.networkPolicies.egress	object	Defines the egress rules for the specified component. Set to {} to allow connections to all external providers.
spec.networkPolicies.egress.ports	object	Defines a list of network ports and protocols for the specified providers.
spec.networkPolicies.name	string	Specifies a unique name for the custom network policy, which is used to generate the NetworkPolicy resource name.

10.11.3. Network policy configuration examples

To control traffic flow and enhance cluster security, enable network policies and custom rules for the cert-manager Operator for Red Hat OpenShift.

To enable network policy and custom rules, see the following example:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
spec:
  defaultNetworkPolicy: "true"
```

To allow egress access to all external issuer providers, see the following example:

```

apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
spec:
  defaultNetworkPolicy: "true"
  networkPolicies:
    - name: allow-egress-to-all
      componentName: CoreController
      egress:
        - {}

```

To allow the cert-manager Operator controller to perform the ACME challenge self-check, see the following example. This process requires connections to the ACME provider, DNS API endpoints, and recursive DNS servers.

```

apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
metadata:
  name: cluster
spec:
  defaultNetworkPolicy: "true"
  networkPolicies:
    - name: allow-egress-to-acme-server
      componentName: CoreController
      egress:
        - ports:
            - port: 80
              protocol: TCP
            - port: 443
              protocol: TCP
    - name: allow-egress-to-dns-service
      componentName: CoreController
      egress:
        - ports:
            - port: 53
              protocol: UDP
            - port: 53
              protocol: TCP

```

10.11.4. Verifying the network policy creation

You can verify that the default and custom **NetworkPolicy** resources are created.

Prerequisites

- You have enabled network policy for cert-manager Operator for Red Hat OpenShift in the **CertManager** custom resource.

Procedure

- Verify the list of **NetworkPolicy** resources in the **cert-manager** namespace by running the following command:

```
$ oc get networkpolicy -n cert-manager
```

Example output

NAME	POD-SELECTOR	AGE
cert-manager-allow-egress-to-api-server	app.kubernetes.io/instance=cert-manager	7s
cert-manager-allow-egress-to-dns	app=cert-manager	6s
cert-manager-allow-ingress-to-metrics	app.kubernetes.io/instance=cert-manager	7s
cert-manager-allow-ingress-to-webhook	app=webhook	6s
cert-manager-deny-all	app.kubernetes.io/instance=cert-manager	8s
cert-manager-user-allow-egress-to-acme-server	app=cert-manager	8s
cert-manager-user-allow-egress-to-dns-service	app=cert-manager	7s

The output lists the default policies and any custom policies that you created.

10.12. DISTRIBUTING CERTIFICATES BY USING TRUST-MANAGER OPERAND

The trust-manager operand simplifies the distribution of certificate authority (CA) certificates across OpenShift Container Platform clusters. As an administrator, you can configure the operand according to the cluster requirements and manage trust bundles efficiently.

IMPORTANT

Distributing certificates by using trust manager is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

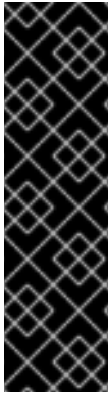
For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

The trust-manager operand provides the following benefits:

- Distribution of CA certificates across your cluster as a Day 2 operation.
- Consolidation of certificates from multiple sources, such as ConfigMaps, Secrets, inline data, and default CAs, into a single trust bundle.
- Automatic updates to target objects whenever the underlying source certificates change.
- Creation of trust bundles as secret objects for applications that explicitly require secrets instead of ConfigMap objects.
- Automatic integration with the default trusted CA bundle of the cluster, requiring no manual configuration.

10.12.1. Installing the trust-manager operand

You can install the trust-manager operand to enable the automated distribution of trust bundles across your cluster namespaces. The trust-manager operand is not installed by default.



IMPORTANT

Distributing certificates by using trust manager is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed cert-manager Operator for Red Hat OpenShift.

Procedure

1. Enable the trust manager add-on feature in the Operator subscription by running the following command:

```
oc -n cert-manager-operator patch subscription cert-manager-operator \
  --type='merge' \
  -p '{"spec":{"config":{"env":
[{"name":"UNSUPPORTED_ADDON_FEATURES","value":"TrustManager=true"}]}}}'
```

2. Create a YAML file, for example, **trust-manager.yaml**, that defines the **TrustManager** custom resource (CR) as shown in the following example:

Example trust-manager.yaml

```
apiVersion: operator.openshift.io/v1alpha1
kind: TrustManager
metadata:
  name: cluster
spec:
  trustManagerConfig:
    logLevel: 2
    logFormat: "text"
    trustNamespace: "cert-manager"
    filterExpiredCertificates: "Enabled"
    secretTargets:
      policy: "Custom"
      authorizedSecrets:
        - "my-trust-bundle"
        - "app-ca-bundle"
    defaultCAPackage:
      policy: "Enabled"
  resources: {}
  affinity: {}
  tolerations: []
  nodeSelector: {}
  controllerConfig:
```



```
labels:
  environment: "production"
  team: "platform"
annotations:
  example.com/managed-by: "cert-manager-operator"
```



NOTE

Because you can create only one instance of **TrustManager** CR per cluster, the **metadata.name** field must be set to **cluster**.

3. Create the **TrustManager** CR by running the following command:

```
$ oc create -f trust-manager.yaml
```

Verification

- Verify that the **trust-manager** operand is running successfully by running the following command:

```
$ oc get TrustManager cluster -o jsonpath='{.status.conditions}' | jq
```

Example output

```
[
  {
    "lastTransitionTime": "2026-03-27T11:54:50Z",
    "message": "",
    "reason": "Ready",
    "status": "False",
    "type": "Degraded"
  },
  {
    "lastTransitionTime": "2026-03-27T11:54:50Z",
    "message": "reconciliation successful",
    "reason": "Ready",
    "status": "True",
    "type": "Ready"
  }
]
```

The **message** field in the output must have the value **reconciliation successful**.

- Verify that the **trust-manager** deployment is running successfully in the **cert-manager** namespace:

```
$ oc get Deployments -l "app.kubernetes.io/name=cert-manager-trust-manager" -n cert-manager
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
trust-manager	1/1	1	1	109s

- Verify that the status of the pod is **Running** by running the following command:

```
$ oc get pods -l "app.kubernetes.io/name=cert-manager-trust-manager" -n cert-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
trust-manager-547bb59b4b-hd6mv	1/1	Running	0	24s

Next Step

- Configuring trust bundle

10.12.2. Configuring trust bundle

After installing the trust-manager operand, you must use the Bundle custom resource (CR) to distribute certificate authority (CA) certificates across your cluster. A trust bundle combines certificate sources and maintains target **ConfigMap** and **Secret** objects across selected namespaces.

If you configure your trust bundle to use the default CAs, you do not need to manually provision the source certificates. The controller reads them from the **cert-manager-operator-trusted-ca-bundle** ConfigMap, which is injected by the Cluster Network Operator (CNO) during the Operator installation.



IMPORTANT

Distributing certificates by using trust manager is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed **trust-manager** operand.

Procedure

1. To inject the trust bundle into a specific namespace, apply the required label by running the following command:

```
$ oc patch namespace <namespace> --type=merge '{"metadata":{"labels":{"trust.cert-manager.io/inject":"true"}}}'
```

The trust-manager operand creates the target bundle in all namespaces that match the label selector defined in your **Bundle** CR.

2. Create a YAML file, for example, **bundle.yaml**, that defines the **Bundle** object as shown in the following example:

```
apiVersion: trust.cert-manager.io/v1alpha1
kind: Bundle
metadata:
  name: example-bundle
spec:
  sources:
    - useDefaultCAs: true
  target:
    configMap:
      key: ca-certificates.crt
    secret:
      key: ca-certificates.crt
    namespaceSelector:
      matchLabels:
        trust.cert-manager.io/inject: "true"
```

For more information on bundle configurations, see [trust-manager usage](#).



NOTE

If your Bundle CR targets a **Secret** object, you must set the **spec.trustManagerConfig.secretTargets.policy** field in your TrustManager CR to **Custom** and add the name of target secret to the **spec.trustManagerConfig.secretTargets.authorizedSecrets** list. If the **spec.trustManagerConfig.secretTargets.policy** field is set to **Disabled**, the Bundle CR fails to create the target secret.

3. Create the **Bundle** custom resource by running the following command:

```
$ oc create -f bundle.yaml
```

Verification

- Verify the status of Bundle CR by running the following command:

```
$ oc get Bundle example-bundle -o jsonpath='{.status.conditions}' | jq
```

In the output, the **reason** must be set to **Synced** and **status** must be set to **True**, as shown in the following example:

```
[
  {
    "lastTransitionTime": "2026-03-27T12:03:42Z",
    "message": "Successfully synced Bundle to namespaces that match this label selector: trust.cert-manager.io/inject=true",
    "observedGeneration": 1,
    "reason": "Synced",
    "status": "True",
```

```
"type": "Synced"
  }
]
```

- Verify the target secret by running the following command:

```
$ oc describe secret example-bundle -n trust-bundle-target
```

Example output

```
Name:      example-bundle
Namespace: trust-bundle-target
Labels:    trust.cert-manager.io/bundle=example-bundle
Annotations: trust.cert-manager.io/hash:
55c00f8109c4c6b1ee4710aa53ad280355973f25444d6bb13a93851af0d8f5d8

Type: Opaque

Data
====
ca-certificates.crt: 219257 bytes
```

- Verify the target ConfigMap by running the following command:

```
$ oc get cm example-bundle -n trust-bundle-target
```

Example output

```
NAME          DATA  AGE
example-bundle 1      4m25s
```

10.12.3. Uninstalling the trust-manager operand

You can uninstall the trust-manager operand by deleting the TrustManager custom resource (CR). Deleting the TrustManager CR stops the operator from reconciling trust-manager resources, but does not automatically remove the trust-manager deployment or its associated resources. You must manually delete these resources after deleting the CR if you need a complete cleanup.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have enabled the trust manager feature.
- You have created the **TrustManager** custom resource.

Procedure

1. Delete any Bundle CRs that you created. Deleting a Bundle CR causes trust-manager to remove the corresponding target ConfigMap and Secret objects from the target namespaces.
 - a. Fetch the list of bundles created by running the following command:

```
$ oc get Bundle
```

- b. Delete each Bundle in the list by running the following command:

```
$ oc delete Bundle <bundle_name>
```

2. Delete the **TrustManager** custom resource by running the following command:

```
$ oc delete TrustManager cluster
```

3. Delete all the labeled resources to complete the cleanup:

- a. Delete the namespace-scoped resources in the **cert-manager** namespace:

```
$ oc delete deployments,services,serviceaccounts,configmaps,certificates,issuers -l
"app.kubernetes.io/name=cert-manager-trust-manager" -n cert-manager
```

- b. Delete the cluster-scoped resources:

```
$ oc delete clusterroles,clusterrolebindings,validatingwebhookconfigurations -l
"app.kubernetes.io/name=cert-manager-trust-manager"
```

- c. If you configured a custom trust namespace, delete the role and role binding resources in that namespace:

```
$ oc delete roles,rolebindings -l "app.kubernetes.io/name=cert-manager-trust-manager" -
n <trust_namespace>
```

10.12.4. Trust manager custom resource fields

You can configure the behavior of the trust-manager operand by modifying the **TrustManager** custom resource (CR).

IMPORTANT

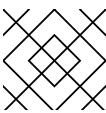
Distributing certificates by using trust manager is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

The following table lists the parameters for configuring trust-manager settings.

Field	Type	Description
spec.controllerConfig.labels	object	Optional. Specifies a list of key-value pairs to apply as labels to all resources created for the trust manager deployment.

Field	Type	Description
spec.controllerConfig.annotations	object	Optional. Specifies a list of key-value pairs to apply as annotations to all resources created for the trust manager deployment.
spec.trustManagerConfig.affinity	object	Optional. Specifies the scheduling constraints for the trust manager pod. For more information, see Assigning Pods to Nodes .
spec.trustManagerConfig.defaultCAPackage	object	Optional. Configures the default CA package for trust manager. When enabled, the Operator uses the OpenShift Container Platform trusted CA bundle injection mechanism.
spec.trustManagerConfig.defaultCAPackage.policy	string	Optional. Specifies whether the default CA package feature is enabled. When set to Enabled, the Operator configures the trusted CA bundle to trust manager. When set to Disabled, no default CA package is configured. The default value is Disabled.  NOTE To enable the useDefaultCAs: true setting in your Bundle CR, you must set the value to Enabled.
spec.trustManagerConfig.filterExpiredCertificates	string	Optional. Specifies whether trust manager filters out expired certificates from trust bundles before distributing them. When set to Enabled , the expired certificates are removed from bundles. When set to Disabled , the expired certificates are included in bundles. The default value is Disabled .
spec.trustManagerConfig.loggingLevel	integer	Optional. Specifies the verbosity of trust manager logging. The minimum value is 1 and the maximum value is 5 . The default value is 1 .
spec.trustManagerConfig.loggingFormat	string	Optional. Specifies the output format for trust manager logging. The supported formats are text and json . The default value is text .
spec.trustManagerConfig.nodeSelector	object	Optional. Specifies the key-value pairs that limit which nodes can host the trust manager pod. You can specify a maximum of 50 node selectors. For more information, see Assigning Pods to Nodes .
spec.trustManagerConfig.resources	object	Optional. Defines the compute resource requirements for the trust manager pod.

Field	Type	Description
spec.trustManagerConfig.secretTargets	object	Optional. Defines the configuration for writing trust bundles to Secrets .
spec.trustManagerConfig.secretTargets.authorizedSecrets	array	Optional. A list of specific secret names that trust manager is authorized to create and update.  NOTE If spec.trustManagerConfig.secretTargets.policy is set to Custom , you must specify a value. If spec.trustManagerConfig.secretTargets.policy is set to Disabled , you must not specify a value.
spec.trustManagerConfig.secretTargets.policy	string	Optional. Specifies whether trust manager can write trust bundles to Secrets . When set to Disabled , trust manager cannot write trust bundles to Secrets . When set to Custom , trust manager is granted permission to create and update only the secrets listed in the authorizedSecrets parameter. The default value is Disabled .
spec.trustManagerConfig.tolerations	array	Optional. Allows the trust manager pod to be scheduled on nodes with specific taints. You can specify a maximum of 50 tolerations.
spec.trustManagerConfig.trustNamespace	string	Optional. Specifies the namespace where trust manager locates CA certificate sources, such as ConfigMaps and Secrets. This namespace must exist before you create the TrustManager custom resource. The default value is cert-manager .  NOTE You cannot change the value once set.

10.13. MONITORING CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

By default, the cert-manager Operator for Red Hat OpenShift exposes metrics for the three core components: controller, cainjector, and webhook. You can configure OpenShift Monitoring to collect these metrics by using the Prometheus Operator format.

10.13.1. Enabling user workload monitoring

To collect metrics from your specific applications, enable monitoring for user-defined projects. You can enable monitoring for user-defined projects by configuring user workload monitoring in the cluster. For more information, see "Setting up metrics collection for user-defined projects".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Create the **cluster-monitoring-config.yaml** YAML file:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

- Apply the **ConfigMap** by running the following command:

```
$ oc apply -f cluster-monitoring-config.yaml
```

Verification

- Verify that the monitoring components for user workloads are running in the **openshift-user-workload-monitoring** namespace by running the following command:

```
$ oc -n openshift-user-workload-monitoring get pod
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
prometheus-operator-6cb6bd9588-dtzxq	2/2	Running	0	50s
prometheus-user-workload-0	6/6	Running	0	48s
prometheus-user-workload-1	6/6	Running	0	48s
thanos-ruler-user-workload-0	4/4	Running	0	42s
thanos-ruler-user-workload-1	4/4	Running	0	42s

The status of the pods such as **prometheus-operator**, **prometheus-user-workload**, and **thanos-ruler-user-workload** must be **Running**.

Additional resources

- [Setting up metrics collection for user-defined projects](#)

10.13.2. Configuring metrics collection for cert-manager Operator for Red Hat OpenShift operands by using a ServiceMonitor

You can configure metrics collection for the cert-manager Operator for Red Hat OpenShift operands by creating a **ServiceMonitor** custom resource (CR).

The cert-manager Operator for Red Hat OpenShift operands expose metrics by default on port **9402** at the **/metrics** service endpoint. The **ServiceMonitor** CR enables Prometheus Operator to collect custom metrics.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the cert-manager Operator for Red Hat OpenShift.
- You have enabled the user workload monitoring.

Procedure

1. Create the **ServiceMonitor** CR:
 - a. Create the YAML file that defines the **ServiceMonitor** CR:

Example servicemonitor-cert-manager.yaml file

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app: cert-manager
    app.kubernetes.io/instance: cert-manager
    app.kubernetes.io/name: cert-manager
  name: cert-manager
  namespace: cert-manager
spec:
  endpoints:
    - honorLabels: false
      interval: 60s
      path: /metrics
      scrapeTimeout: 30s
      targetPort: 9402
  selector:
    matchExpressions:
      - key: app.kubernetes.io/name
        operator: In
        values:
          - cainjector
          - cert-manager
          - webhook
      - key: app.kubernetes.io/instance
        operator: In
        values:
          - cert-manager
      - key: app.kubernetes.io/component
        operator: In
        values:
          - cainjector
          - controller
```

```
- webhook
```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-cert-manager.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance begins metrics collection from the cert-manager Operator for Red Hat OpenShift operands. The collected metrics are labeled with **job="cert-manager"**, **job="cert-manager-cainjector"**, and **job="cert-manager-webhook"**.

Verification

1. In the OpenShift Container Platform web console, navigate to **Observe → Targets**.
2. In the **Label** filter field, enter the following labels to filter the metrics targets for each operand:

```
$ service=cert-manager
```

```
$ service=cert-manager-webhook
```

```
$ service=cert-manager-cainjector
```

3. Confirm that the **Status** column shows **Up** for the **cert-manager**, **cert-manager-webhook**, and **cert-manager-cainjector** entries.

Additional resources

- [Configuring user workload monitoring](#)

10.13.3. Querying metrics for the cert-manager Operator for Red Hat OpenShift operands

As a cluster administrator, or as a user with view access to all namespaces, you can query cert-manager Operator for Red Hat OpenShift operands metrics by using the OpenShift Container Platform web console or the command-line interface (CLI). For more information, see "Accessing metrics".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the cert-manager Operator for Red Hat OpenShift.
- You have enabled monitoring and metrics collection by creating **ServiceMonitor** object.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Observe → Metrics**.
2. In the query field, enter the following PromQL expressions to query the cert-manager Operator for Red Hat OpenShift operands metric for each operand:

```
{job="cert-manager"}
```

```
{job="cert-manager-webhook"}
```

```
{job="cert-manager-cainjector"}
```

Additional resources

- [Accessing metrics as an administrator](#)

10.13.4. Configuring metrics collection for the istio-csr operand

The **istio-csr** operand exposes metrics by default on port **9402** at the **/metrics** service endpoint. You can configure metrics collection for the operand by creating a **ServiceMonitor** custom resource (CR), which enables the Prometheus Operator to collect custom metrics. For more information, see "Configuring user workload monitoring".

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed the cert-manager Operator for Red Hat OpenShift.
- You have enabled user workload monitoring.

Procedure

1. Create the **ServiceMonitor** CR definition file:

Example servicemonitor-istio-csr.yaml file

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app: cert-manager-istio-csr
    app.kubernetes.io/instance: cert-manager-istio-csr
    app.kubernetes.io/name: cert-manager-istio-csr
  name: cert-manager-istio-csr
  namespace: <istio_csr_project_name>
spec:
  endpoints:
    - honorLabels: false
      interval: 60s
      path: /metrics
      scrapeTimeout: 30s
      targetPort: 9402
  namespaceSelector:
    matchNames:
      - <istio_csr_project_name>
  selector:
    matchLabels:
```

```
app: cert-manager-istio-csr
app.kubernetes.io/instance: cert-manager-istio-csr
app.kubernetes.io/name: cert-manager-istio-csr
```

Replace **<istio_csr_project_name>** with the namespace where you created the **IstioCSR** CR.

2. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-istio-csr.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance starts collecting metrics from the istio-csr operand. The collected metrics are labeled with **job="cert-manager-istio-csr"**.

Verification

1. Log in to the OpenShift Container Platform web console.
2. Click **Observe → Targets**.
3. In the **Label filter** field, enter the **service=cert-manager-istio-csr** label to filter the metrics targets.
4. Confirm that the **Status** column shows **Up** for the **cert-manager-istio-csr** target.

Additional resources

- [Configuring user workload monitoring](#)

10.13.5. Querying metrics for the istio-csr operand

Cluster administrators, or users with view access to all namespaces, can query metrics for the istio-csr operand by using the OpenShift Container Platform web console. For more information, see "Accessing metrics".

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed the cert-manager Operator for Red Hat OpenShift.
- You have enabled monitoring and metrics collection by creating the **ServiceMonitor** object for the istio-csr operand.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Click **Observe → Metrics**.
3. In the query field, enter the **{job="cert-manager-istio-csr"}** PromQL expression to query the **istio-csr** operand metrics. The results display metrics collected for the istio-csr operand, which can help you monitor its performance and behavior.

Additional resources

- [Accessing metrics as an administrator](#)

10.14. CONFIGURING LOG LEVELS FOR CERT-MANAGER AND THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

To troubleshoot issues with the cert-manager components and the cert-manager Operator for Red Hat OpenShift, you can configure the log level verbosity.



NOTE

To use different log levels for different cert-manager components, see *Customizing cert-manager Operator API fields*.

10.14.1. Setting a log level for cert-manager

To troubleshoot issues and control log volume, configure the log level for the cert-manager Operator for Red Hat OpenShift. You can set specific verbosity levels to capture the necessary details for debugging or to reduce noise in your cluster logs.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed version 1.11.1 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

1. Edit the **CertManager** resource by running the following command:

```
$ oc edit certmanager.operator cluster
```

2. Set the log level value by editing the **spec.logLevel** section:

```
apiVersion: operator.openshift.io/v1alpha1
kind: CertManager
...
spec:
  logLevel: <log_level>
```

The **CertManager** resource supports the following **logLevel** values:

Normal

Audits logs and records common operations. The default setting. Use this level when there are no issues.

Debug

Provides verbose logs. Use this level to troubleshoot minor issues.

Trace

Provides highly verbose logs. Use this level to troubleshoot major issues.

TraceAll

Provides maximum log detail. Use this level to troubleshoot serious issues.



NOTE

TraceAll generates huge amount of logs. After setting **logLevel** to **TraceAll**, you might experience performance issues.

3. Save your changes and quit the text editor to apply your changes.
After applying the changes, the verbosity level for the cert-manager components controller, CA injector, and webhook is updated.

10.14.2. Setting a log level for the cert-manager Operator for Red Hat OpenShift

To troubleshoot issues and control log volume, set the log level for the cert-manager Operator for Red Hat OpenShift. You can configure the verbosity of the Operator log messages to capture the specific details required for your environment.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed version 1.11.1 or later of the cert-manager Operator for Red Hat OpenShift.

Procedure

- Update the subscription object for cert-manager Operator for Red Hat OpenShift to provide the verbosity level for the operator logs by running the following command:

```
$ oc -n cert-manager-operator patch subscription openshift-cert-manager-operator --
type='merge' -p '{"spec":{"config":{"env":[{"name":"OPERATOR_LOG_LEVEL","value":"v"}]}}}'
```

Replace **v** with the desired log level number. The valid values for **v** can range from **1** to **10**. The default value is **2**.

Verification

1. The cert-manager Operator pod is redeployed. Verify that the log level of the cert-manager Operator for Red Hat OpenShift is updated by running the following command:

```
$ oc set env deploy/cert-manager-operator-controller-manager -n cert-manager-operator --
list | grep -e OPERATOR_LOG_LEVEL -e container
```

Example output

```
# deployments/cert-manager-operator-controller-manager, container kube-rbac-proxy
OPERATOR_LOG_LEVEL=9
# deployments/cert-manager-operator-controller-manager, container cert-manager-operator
OPERATOR_LOG_LEVEL=9
```

2. Verify that the log level of the cert-manager Operator for Red Hat OpenShift is updated by running the **oc logs** command:

■

```
$ oc logs deploy/cert-manager-operator-controller-manager -n cert-manager-operator
```

10.14.3. Additional resources

- [Customizing cert-manager Operator API fields](#)

10.15. UNINSTALLING THE CERT-MANAGER OPERATOR FOR RED HAT OPENSIFT

You can remove the cert-manager Operator for Red Hat OpenShift from OpenShift Container Platform by uninstalling the Operator and removing its related resources.

10.15.1. Uninstalling the cert-manager Operator for Red Hat OpenShift

You can uninstall the cert-manager Operator for Red Hat OpenShift by using the web console.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.
- The cert-manager Operator for Red Hat OpenShift is installed.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Uninstall the cert-manager Operator for Red Hat OpenShift Operator.
 - a. Navigate to **Ecosystem** → **Installed Operators**.



- b. Click the Options menu next to the **cert-manager Operator for Red Hat OpenShift** entry and click **Uninstall Operator**.
- c. In the confirmation dialog, click **Uninstall**.

10.15.2. Removing cert-manager Operator for Red Hat OpenShift resources

After you uninstall the cert-manager Operator for Red Hat OpenShift, you can delete its associated resources from your cluster.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.

2. Remove the deployments of the cert-manager components, such as **cert-manager**, **cainjector**, and **webhook**, present in the **cert-manager** namespace.
 - a. Click the **Project** drop-down menu to see a list of all available projects, and select the **cert-manager** project.
 - b. Navigate to **Workloads → Deployments**.
 - c. Select the deployment that you want to delete.
 - d. Click the **Actions** drop-down menu, and select **Delete Deployment** to see a confirmation dialog box.
 - e. Click **Delete** to delete the deployment.
 - f. Alternatively, delete deployments of the cert-manager components such as **cert-manager**, **cainjector** and **webhook** present in the **cert-manager** namespace by using the command-line interface (CLI).

```
$ oc delete deployment -n cert-manager -l app.kubernetes.io/instance=cert-manager
```

3. Optional: Remove the custom resource definitions (CRDs) that were installed by the cert-manager Operator for Red Hat OpenShift:

- a. Remove the finalizers from the **CertManager** custom resource (CR) by running the following command:

```
$ oc patch certmanagers.operator cluster --type=merge -p='{"metadata": {"finalizers": null}}'
```

- b. Navigate to **Administration → CustomResourceDefinitions**.
- c. Enter **certmanager** in the **Name** field to filter the CRDs.



- d. Click the Options menu next to each of the following CRDs, and select **Delete Custom Resource Definition**:

- **Certificate**
- **CertificateRequest**
- **CertManager (operator.openshift.io)**
- **Challenge**
- **ClusterIssuer**
- **Issuer**
- **Order**

4. Optional: Remove the **cert-manager-operator** namespace.

- a. Navigate to **Administration → Namespaces**.

- b. Click the Options menu  next to the **cert-manager-operator** and select **Delete Namespace**.
- c. In the confirmation dialog, enter **cert-manager-operator** in the field and click **Delete**.

CHAPTER 11. ZERO TRUST WORKLOAD IDENTITY MANAGER

11.1. ZERO TRUST WORKLOAD IDENTITY MANAGER OVERVIEW

The Zero Trust Workload Identity Manager is an OpenShift Container Platform Operator that manages the lifecycle of SPIFFE Runtime Environment (SPIRE) components. It enables workload identity management based on the Secure Production Identity Framework for Everyone (SPIFFE) standard, providing cryptographically verifiable identities (SVIDs) to workloads running in OpenShift Container Platform clusters.

The following are components of the Zero Trust Workload Identity Manager architecture:

11.1.1. SPIFFE

Establish trust between software workloads in distributed systems with Secure Production Identity Framework for Everyone (SPIFFE). SPIFFE assigns unique IDs to workloads, allowing workloads to verify identities and communicate securely. This ensures secure authentication across dynamic environments.

The SPIFFE IDs are contained in the SPIFFE Verifiable Identity Document (SVID). SVIDs are used by workloads to verify their identity to other workloads so that the workloads can communicate with each other. The two main SVID formats are:

- X.509-SVIDs: X.509 certificates where the SPIFFE ID is embedded in the Subject Alternative Name (SAN) field.
- JWT-SVIDs: JSON Web Tokens (JWTs) where the SPIFFE ID is included as the **sub** claim.

For more information, see [SPIFFE Overview](#).

11.1.2. SPIRE Server

The SPIRE Server is the central management component of SPIRE that issues SPIFFE identities and maintains the registration database for a trust domain.

Additional resources

- [About the SPIRE Server](#)

11.1.3. SPIRE Agent

The SPIRE Agent performs workload attestation to ensure that workloads receive a verified identity when requesting authentication through the SPIFFE Workload API. The agent uses configured workload attester plugins to verify these identities.

SPIRE and the SPIRE Agent perform node attestation via node plugins. The plugins are used to verify the identity of the node on which the agent is running. For more information, see [About the SPIRE Agent](#).

11.1.4. Attestation

The attestation process verifies the identity of nodes and workloads before issuing SPIFFE IDs. By comparing attributes against defined selectors, this process ensures that only legitimate entities within the trust domain receive cryptographic credentials.

The two main types of attestation in SPIFFE/SPIRE are:

- Node attestation: verifies the identity of a machine or a node on a system, before a SPIRE Agent running on that node can be trusted to request identities for workloads.
- Workload attestation: verifies the identity of an application or service running on an attested node before the SPIRE Agent on that node can provide it with a SPIFFE ID and SVID.

For more information, see [Attestation](#).

11.2. ZERO TRUST WORKLOAD IDENTITY MANAGER COMPONENTS

Review the components available in the initial release of Zero Trust Workload Identity Manager to understand the architecture. These components provide the foundation for identifying and securing your workloads.

11.2.1. SPIFFE CSI Driver

The SPIFFE Container Storage Interface (CSI) driver helps pods securely obtain their SPIFFE Verifiable Identity Document (SVID) by delivering the Workload API socket. By using Kubernetes ephemeral inline volumes, the driver simplifies how applications request temporary storage for identity management.

When the pod starts, the Kubelet calls the SPIFFE CSI driver to provision and mount a volume into the pod's containers. The SPIFFE CSI driver mounts a directory that contains the SPIFFE Workload API into the pod. Applications in the pod then communicate with the Workload API to obtain their SVIDs. The driver guarantees that each SVID is unique.

11.2.2. SPIRE OpenID Connect Discovery Provider

Use the SPIRE OpenID Connect (OIDC) Discovery Provider to integrate SPIRE workload identities with OIDC-compliant systems. This component exposes endpoints for token verification. It helps ensure compatibility between SPIRE-issued credentials and external APIs requiring standard OIDC tokens.

While SPIRE primarily issues identities for workloads, additional workload-related claims can be embedded into JWT-SVIDs through the configuration of SPIRE, which these claims to be included in the token and verified by OIDC-compliant clients.

11.2.3. SPIRE Controller Manager

Use the SPIRE Controller Manager to automate workload registration with custom resource definitions (CRDs). The manager monitors pods and CRDs to create, update, or delete entries on the SPIRE Server. This process helps ensure that your SPIRE entries accurately reflect your active resources.

The SPIRE Controller Manager is designed to be deployed on the same pod as the SPIRE Server. The manager communicates with the SPIRE Server API using a private UNIX Domain Socket within a shared volume.

11.2.4. SPIRE Server and Agent telemetry

Use the SPIRE Controller Manager to register workloads by using custom resource definitions (CRDs). The manager monitors pods and CRDs for changes and triggers a reconciliation process. This process creates, updates, or deletes SPIRE Server entries to help ensure they match your configuration.

11.2.5. About the Zero Trust Workload Identity Manager workflow

Understand the high-level workflow of Zero Trust Workload Identity Manager to help you manage secure identities. This process relies on SPIRE components and custom resource definitions (CRDs) to validate nodes and workloads.

The following is a high-level workflow of the Zero Trust Workload Identity Manager within the Red Hat OpenShift cluster.

1. The SPIRE, SPIRE Agent, SPIFFE CSI Driver, and the SPIRE OIDC Discovery Provider operands are deployed and managed by Zero Trust Workload Identity Manager via associated customer resource definitions (CRDs).
2. Watches are then registered for relevant Kubernetes resources and the necessary SPIRE CRDs are applied to the cluster.
3. The CR for the ZeroTrustWorkloadIdentityManager resource named **cluster** is deployed and managed by a controller.
4. To deploy the SPIRE Server, SPIRE Agent, SPIFFE CSI Driver, and SPIRE OIDC Discovery Provider, you need to create a custom resource of a each certain type and name it **cluster**. The custom resource types are as follows:
 - SPIRE Server - **SpireServer**
 - SPIRE Agent - **SpireAgent**
 - SPIFFE CSI Driver - **SpiffeCSIDriver**
 - SPIRE OIDC discovery provider - **SpireOIDCDiscoveryProvider**
5. When a node starts, the SPIRE Agent initializes, and connects to the SPIRE Server.
6. The SPIRE Agent begins the node attestation process. The agent collects information on the node's identity such as label name and namespace. The agent securely provides the information it gathered through the attestation to the SPIRE Server.
7. The SPIRE Server then evaluates this information against its configured attestation policies and registration entries. If successful, the server generates an agent SVID and the Trust Bundle (CA Certificate) and securely sends this back to the SPIRE Agent.
8. A workload starts on the node and needs a secure identity. The workload connects to the agent's Workload API and requests a SVID.
9. The SPIRE Agent receives the request and begins a workload attestation to gather information about the workload.
10. After the SPIRE Agent gathers the information, the information is sent to the SPIRE Server and the server checks its configured registration entries.
11. The SPIRE Agent receives the workload SVID and Trust Bundle and passes it on to the workload. The workload can now present their SVIDs to other SPIFFE-aware devices to communicate with them.

Additional resources

- [Registering workloads](#)
- [SPIFFE Concepts](#)

- [SPIRE Use Cases](#)

11.3. ZERO TRUST WORKLOAD IDENTITY MANAGER RELEASE NOTES

The Zero Trust Workload Identity Manager leverages Secure Production Identity Framework for Everyone (SPIFFE) and the SPIFFE Runtime Environment (SPIRE) to provide a comprehensive identity management solution for distributed systems.

These release notes track the development of Zero Trust Workload Identity Manager.

11.3.1. Zero Trust Workload Identity Manager 1.1.0

Issued: 30 Jun 2026

This release adds integration and operational capabilities for workloads that use external certificate authorities, service mesh deployments, or file-based Transport Layer Security (TLS) credentials. The release includes a supported SPIFFE Helper container image, SPIRE UpstreamAuthority plugins for cert-manager and HashiCorp Vault, and Red Hat OpenShift Service Mesh integration with SPIRE for single-cluster and federated multi-cluster mutual Transport Layer Security (mTLS).

The following advisories are available for Zero Trust Workload Identity Manager:

- [RHBA-2026:28869](#)
- [RHBA-2026:28784](#)
- [RHBA-2026:28399](#)
- [RHBA-2026:28392](#)
- [RHBA-2026:28391](#)
- [RHBA-2026:28260](#)
- [RHBA-2026:28200](#)
- [RHBA-2026:28140](#)

Zero Trust Workload Identity Manager supports the following components and versions:

Component	Version
Zero Trust Workload Identity Manager	1.1.0
SPIRE Server	1.14.7
SPIRE Agent	1.14.7
SPIRE Controller Manager	0.6.4
SPIRE OIDC Discovery Provider	1.14.7
SPIFFE CSI Driver	0.2.8

11.3.1.1. New features and enhancements

Supported SPIFFE Helper container image

Zero Trust Workload Identity Manager now provides a supported SPIFFE Helper container image for workloads that cannot use the SPIFFE Workload API directly but can read Transport Layer Security (TLS) credentials from a shared volume. The image is based on upstream SPIFFE Helper. The configuration file format, command-line flags, and Workload API behavior remain compatible.

SPIRE UpstreamAuthority plugins for external certificate authorities

Zero Trust Workload Identity Manager now supports SPIRE Server UpstreamAuthority plugins that obtain intermediate signing certificates from external certificate management systems while preserving Secure Production Identity Framework for Everyone (SPIFFE) identity standards.

- Supported plugins:
 - cert-manager UpstreamAuthority plugin: integrates SPIRE Server with cert-manager Operator for Red Hat OpenShift.
 - Vault UpstreamAuthority plugin: integrates SPIRE Server with the HashiCorp Vault Public Key Infrastructure (PKI) secrets engine.

cert-manager UpstreamAuthority plugin

The cert-manager UpstreamAuthority plugin connects SPIRE Server to cert-manager Operator for Red Hat OpenShift for automated intermediate certificate provisioning. SPIRE Server creates a **CertificateRequest** custom resource, then the configured **Issuer** or **ClusterIssuer** signs the request, and then the SPIRE Server uses the signed intermediate certificate to issue workload identities.

Vault UpstreamAuthority plugin

The Vault UpstreamAuthority plugin connects SPIRE Server to the HashiCorp Vault PKI secrets engine for automated intermediate certificate authority (CA) certificate signing. Use this plugin when PKI is centralized in Vault and SPIRE must obtain intermediate signing certificates through Vault policies and authentication.

Single-cluster Service Mesh integration with SPIRE

Zero Trust Workload Identity Manager now supports single-cluster integration with Red Hat OpenShift Service Mesh. SPIRE replaces Istio's built-in certificate authority (CA) with SPIFFE-compliant identities and short-lived certificates that rotate automatically for workload mutual Transport Layer Security (mTLS).

Multi-cluster Service Mesh integration with SPIRE federation

Zero Trust Workload Identity Manager now supports multi-cluster integration with Red Hat OpenShift Service Mesh through SPIRE federation, enabling cross-cluster mutual Transport Layer Security (mTLS) authentication and zero trust workload identity across separate OpenShift Container Platform clusters.

- Cross-cluster trust requires federation at two layers:
 - SPIRE federation: SPIRE Servers exchange trust bundles through the **https_spiffe** profile.
 - Istio federation: Istio discovers remote endpoints through remote secrets and routes traffic through East-West Gateways.

11.3.1.2. Deprecated features

Custom SCC **spire-spiffe-csi-driver**

Starting in Zero Trust Workload Identity Manager 1.1.0, the SPIFFE CSI Driver no longer uses the custom **SecurityContextConstraints** (SCC) **spire-spiffe-csi-driver**.

Zero Trust Workload Identity Manager now grants the CSI **ServiceAccount** access to the platform **privileged** SCC through a **RoleBinding** namespace.

Zero Trust Workload Identity Manager uses only the existing OpenShift **privileged** SCC. Zero Trust Workload Identity Manager does not create, modify, update, or delete the **privileged** SCC.

Action required after upgrade

Zero Trust Workload Identity Manager does not remove the legacy custom SCC **spire-spiffe-csi-driver**. After you upgrade Zero Trust Workload Identity Manager to 1.1.0 from the **OpenShift OperatorHub** catalog, remove it manually once CSI is healthy on the platform SCC.

For more information, see [Manually delete the custom security context constraints](#).

11.3.1.3. Fixed issues

Managed route TLS secrets no longer require manual RBAC configuration

- Before this update, when you configured a managed route with an **externalSecretRef** TLS certificate on the **SpireOIDCDiscoveryProvider** or **SpireServer** custom resource (CR), Zero Trust Workload Identity Manager did not create the **RoleBinding** that grants the OpenShift Ingress router service account permission to read the referenced Secret. As a consequence, route reconciliation failed with a **ManagedRouteUpdateFailed** condition, and you had to manually create **secret-reader**, **Role**, and **RoleBinding** CRs. With this release, Zero Trust Workload Identity Manager automatically reconciles the required **Role** and **RoleBinding** CRs so that the router service account can access the Secrets referenced by the **externalSecretRef** TLS certificate. Managed routes that use externally provided TLS certificates now reconcile without additional manual RBAC steps.
([SPIRE-164](#))

Pre-existing operand resources are no longer overwritten at installation

- Before this update, when you installed Zero Trust Workload Identity Manager on a cluster that already contained Kubernetes resources with the same names as operand objects, Zero Trust Workload Identity Manager reconciled and overwrote those resources during the initial installation without reporting a conflict. As a consequence, manually created or third-party resources could be modified unexpectedly before you had a chance to resolve naming collisions. With this release, Zero Trust Workload Identity Manager checks the **app.kubernetes.io/managed-by** label before updating any existing resource during installation. If a matching resource is not managed by Zero Trust Workload Identity Manager, Zero Trust Workload Identity Manager sets a **ResourceConflict** status condition and stops reconciliation instead of overwriting the resource.
([SPIRE-340](#))

Unmanaged cluster resources are protected during reconciliation

- Before this update, after Zero Trust Workload Identity Manager was already running, operand controllers could still update Kubernetes resources with matching names even when those resources were not owned by Zero Trust Workload Identity Manager. This could occur when a name collision appeared later or when the **app.kubernetes.io/managed-by: zero-trust-workload-identity-manager** label was removed from a resource that Zero Trust

Workload Identity Manager had previously managed. With this release, each operand controller verifies the **managed-by** label on every reconcile cycle before applying updates. When the label is absent, Zero Trust Workload Identity Manager sets a **ResourceConflict** status condition on the custom resource and skips the update to the conflicting object. Operand updates proceed only for resources that Zero Trust Workload Identity Manager currently manages.

([SPIRE-344](#))

Duplicate health check port names resolved in SPIRE Server StatefulSet

- Before this update, the **spire-server** and **spire-controller-manager** containers in the SPIRE Server **StatefulSet** field both declared a port named **healthz** on different container ports. Kubernetes treated this as a duplicate port name within the pod, issued a warning, and the services or probes that selected the ports by name could target the wrong container. With this release, Zero Trust Workload Identity Manager assigns unique port names, such as **server-healthz** and **ctrlmgr-healthz**, and updates the liveness and readiness probes to reference those names. Health checks and monitoring configurations now resolve to the intended container.

([SPIRE-353](#))

Create-only mode disabled status now updates on the main custom resource

- Before this update, after you disabled create-only mode by setting **CREATE_ONLY_MODE** to **false** in the Operator subscription, the **CreateOnlyMode** condition on the main **ZeroTrustWorkloadIdentityManager** CR could remain **True** with the reason **CreateOnlyModeEnabled**. Because Zero Trust Workload Identity Manager set that condition from operand status instead of from the **CREATE_ONLY_MODE** environment variable in the subscription, the main CR status did not reflect that create-only mode was disabled even though the operand reconciliation had resumed. With this release, Zero Trust Workload Identity Manager sets the **CreateOnlyMode** condition on the main CR directly from the **CREATE_ONLY_MODE** environment variable and updates it to **False** with reason **CreateOnlyModeDisabled** when create-only mode is turned off.

([SPIRE-365](#))

Create-only mode status updates reconcile reliably

- Before this update, Zero Trust Workload Identity Manager controllers wrote the custom resource (CR) status twice during each reconciliation cycle. The CR status was written at the start through the **SetInitialReconciliationStatus** cycle and again at the end through the status manager. Because the informer cache could return a stale **ResourceVersion** after the first write, the deferred status update was rejected with **HTTP 409 Conflict**. Status conditions, including **CreateOnlyMode**, could fail to transition to the expected state even after you changed configuration in the Operator subscription. With this release, controllers apply the status in a single update at the end of reconciliation, and status update retry logic now complies with the **RetryOnConflict** contract. CR status updates, including create-only mode transitions, are now completed.

([SPIRE-506](#))

11.3.2. Zero Trust Workload Identity Manager 1.0.1

Issued: 17 May 2026

This release fixes some Common Vulnerabilities and Exposures (CVEs).

The following advisories are available for the Zero Trust Workload Identity Manager:

- [RHBA-2026:17483](#)
- [RHSA-2026:17463](#)
- [RHSA-2026:17462](#)
- [RHSA-2026:17461](#)
- [RHSA-2026:17460](#)
- [RHSA-2026:17457](#)
- [RHSA-2026:17456](#)

11.3.2.1. CVEs

- [CVE-2026-21441](#)
- [CVE-2025-61726](#)
- [CVE-2025-61729](#)
- [CVE-2025-68121](#)

11.3.3. Zero Trust Workload Identity Manager 1.0.0 (General Availability)

Issued: 12 December 2025

This release introduces capabilities for enterprise readiness, security, and operational flexibility. The release includes SPIRE federation for cross-cluster identity, PostgreSQL support for production persistence, and enhanced security through stricter constraints and API validation.

The following advisories are available for the Zero Trust Workload Identity Manager:

- [RHBA-2025:23438](#)
- [RHBA-2025:23439](#)
- [RHBA-2025:23440](#)
- [RHBA-2025:23441](#)
- [RHBA-2025:23442](#)
- [RHBA-2025:23443](#)
- [RHBA-2025:23446](#)

Zero Trust Workload Identity Manager supports the following components and versions:

Component	Version
SPIRE Server	1.13.3

Component	Version
SPIRE Agent	1.13.3
SPIRE Controller Manager	0.6.3
SPIRE OIDC Discovery Provider	1.13.3
SPIFFE CSI Driver	0.2.8

11.3.3.1. New features and enhancements

SPIRE federation support

The Operator now includes support for SPIRE federation, enabling workloads across distinct trust domains to securely communicate and authenticate with each other.

- Key capabilities:
 - Configuration of bundle endpoints using **https_spiffe** (TLS) or **https_web** (Web PKI) profiles.
 - Automatic certificate management via the ACME protocol. For example, **Let's Encrypt**.
 - Automatic OpenShift Container Platform route creation for federation endpoints.
 - Ability to configure relationships with multiple federated trust domains.
- Customer action required:
 - Review the **federation** configuration within the **SpireServer** custom resource (CR).
 - Ensure proper DNS resolution and network connectivity to federated trust domains.

PostgreSQL database support

SPIRE Server now supports PostgreSQL as an external database backend, accommodating production deployments that necessitate enterprise-grade data persistence and high availability.

- Supported Types: **sqlite3** (default), **postgres**, **mysql**.
- Customer action required:
 - For production, evaluation of migration from SQLite to PostgreSQL is recommended.
 - Creation and configuration of Kubernetes Secrets for database TLS certificates and credentials are required.

Configurable agent socket path and Container Storage Interface (CSI) plugin name

The SPIRE Agent socket path and the SPIFFE CSI Driver plugin name are now configurable, providing operational flexibility for environments with specific directory requirements or co-existence with multiple SPIFFE deployments.

- Key configuration points:

- **SpireAgent.spec.socketPath**
- **SpiffeCSIDriver.spec.agentSocketPath**
- **SpiffeCSIDriver.spec.pluginName**
- Customer action required:
 - Ensure consistency between **socketPath** in the **SpireAgent** CR and **agentSocketPath** in the **SpiffeCSIDriver** CR.

Workload attestors verification API

A new API has been introduced to configure kubelet certificate verification for workload attestation, enhancing security and supporting various OpenShift Container Platform configurations.

- Verification types:
 - **auto** (default): Verification utilizes OpenShift Container Platform defaults (**/etc/kubernetes/kubelet-ca.crt**).
 - **hostCert**: Uses a custom CA certificate path.
 - **skip**: Skips TLS verification (not recommended for production use).

Configurable Certificate Authority and JSON Web Token key types

Administrators can now configure the cryptographic key types used for the SPIRE Server Certificate Authority (CA) and JSON Web Token (JWT) signing, ensuring compliance with organizational security policies.

- Supported Key Types: **rsa-2048** (default), **rsa-4096**, **ec-p256**, **ec-p384**.
- Customer action required:
 - Review organizational security policies to determine required key types.

Custom namespace deployment

- The Operator and all associated operands can now be deployed within a custom namespace, providing flexibility for organizations with specific namespace governance requirements.

Proxy-aware Operator and operands

- The Operator and all managed operands are now proxy-aware and automatically inherit cluster-wide proxy settings when configured.

Enhanced Security Context Constraints

- SPIRE Agent and SPIFFE CSI Driver now run with Security Context Constraints (SCC) that prevent root user execution, though privileged container mode remains enabled for necessary host-level operations.
- The Operator and all operand containers are configured with the **ReadOnlyRootFilesystem** set to **true**.

Enhanced API validation

Comprehensive Common Expression Language (CEL) validation has been integrated into all Custom Resource Definitions (CRDs) to prevent configuration errors during admission control.

- Key validations:
 - All Operator CRDs are enforced as singletons (must be named **cluster**).
 - Immutable Fields: Fields including **trustDomain**, **clusterName**, **bundleConfigMap**, **federation**, **bundleEndpoint** profile, and all **Persistence** settings (**size**, **accessMode**, and **storageClass**) are now immutable after initial creation.
- Customer action required:
 - Review existing CR configurations to ensure compliance with the new validation rules.

Common configuration consolidation

- Standard configuration options (**labels**, **resources**, **affinity**, **tolerations**, **nodeSelector**) are now standardized across all operand CRs via a shared **CommonConfig** structure.

Configuring log level and log format for the operands

This release introduces flexible logging controls to improve observability and debugging across the platform:

- SPIRE Components: Users can now configure the **logLevel** (debug, info, warn, error) and **logFormat** (text, JSON) independently for **SpireServer**, **SpireAgent**, and **SpireOIDCDiscoveryProvider** directly within their CR specifications. The defaults are set to "info" for the **logLevel** and "text" for the **logFormat**.
- Operator: The Operator's log verbosity is now configurable via the **OPERATOR_LOG_LEVEL** environment variable using klog's **textlogger**.

Refactor for create-only mode

By setting the **CREATE_ONLY_MODE** environment variable, users can prevent the Operator from reconciling updates. This allows for manual resource modification without interference. If this mode is disabled, the Operator resumes enforcing the state and overwrites any manual changes.

11.3.3.2. Status and observability improvements

Enhanced status reporting

- The main CR now aggregates status information from all operand CRs.
- New status conditions include Upgradeable (indicating a safe upgrade path) and Progressing (detailing deployment progress).

Operator metrics

- Operator metrics are now exposed and secured with appropriate RBAC configuration.
- Integration is supported with the OpenShift Container Platform monitoring stack.

11.3.3.3. Fixed issues

Enhanced Security Context Constraints for SPIRE Agent

- Before this update, the SPIRE Agent and SPIFFE CSI Driver containers were running as root user, leading to potential security violations. With this release, Security Context Constraints (SCC) have been configured to ensure these components no longer run as root. While privileged container mode is still required for necessary capabilities, this change reduces potential security risks for the user.
([SPIRE-60](#))

SpireServer updates now propagate without Operator restart

- Before this update, the Operator failed to trigger reconciliation after updating the operand CR spec. As a consequence, user updates to **SpireServer** CR resources were not propagated to the **StatefulSet**, causing reconciliation to fail and changes to be ignored, leading to inconsistent resource allocation. With this release, the race condition between the manager and reconciler's cache to trigger reconciliation after CR updates has been fixed. As a result, post installation patch operations on **SpireServer** CRs reliably trigger reconciliation, ensuring updated values are applied to the StatefulSet without manual Operator restart.
([SPIRE-68](#))

Removed unnecessary security context constraint for OpenID Connect discovery provider

- Before this update, the system unnecessarily created a custom security context constraint (SCC) for the OpenID Connect (OIDC) discovery provider, which increased the security footprint and configuration complexity even though the deployment did not require it. With this release, the custom SCC creation logic has been removed, resulting in a configuration where the OIDC discovery provider operates successfully without the extra security constraints.
([SPIRE-190](#))

Fixed ConfigMap Reconciliation for SPIRE Controller Manager

- Before this update, Spire-controller manager ConfigMap reconciliation failed due to an unhandled edge case in the previous implementation. As a consequence, users experienced configuration inconsistencies. With this release, the Spire-controller manager ConfigMap reconciliation issue has been resolved. As a result, end users now experience seamless Spire-controller manager configuration.
([SPIRE-195](#))

OIDC discovery provider now restarts automatically on configuration changes

- Before this update, the SPIRE OIDC discovery provider failed to automatically restart following **configmap** changes, leading to persistent authentication failures. With this release, updates to the CR now trigger an automatic pod restart, ensuring that **configmap** changes are applied immediately.
([SPIRE-225](#))

Corrected update rollback for DaemonSets, Deployments, and StatefulSets

- Before this update, **daemonset**, **deployment**, and **statefulsets** were not properly reverted to their original form in all valid scenarios due to an oversight in the update logic. As a consequence, user data loss or inconsistency occurred in valid scenarios. With this release, the update logic has been corrected, ensuring all valid scenarios revert to their original form.
([SPIRE-248](#))

- Other bug fixes included:
 - Fixed issues related to continuous reconciliation and unnecessary updates.
 - Eliminated requeue logic for user input validation errors.

11.3.4. Zero Trust Workload Identity Manager 0.2.0 (General Availability)

Issued: 09 August 2025

The following advisories are available for the Zero Trust Workload Identity Manager:

- [RHBA-2025:15425](#)
- [RHBA-2025:15426](#)
- [RHBA-2025:15427](#)
- [RHBA-2025:15428](#)

11.3.4.1. New features and enhancements

Support for the managed OIDC Discovery Provider Route

- The Operator exposes the **SPIREOIDCDiscoveryProvider** spec through OpenShift Container Platform Routes under the domain ***.apps.<cluster_domain>** for the selected default installation.
- The **managedRoute** and **externalSecretRef** fields have been added to the **spireOidcDiscoveryProvider** spec.
- The **managedRoute** field is boolean and is set to **true** by default. If set to **false**, the Operator stops managing the route and the existing route will not be deleted automatically. If set back to **true**, the Operator resumes managing the route. If a route does not exist, the Operator creates a new one. If a route already exists, the Operator will override the user configuration if a conflict exists.
- The **externalSecretRef** references an externally managed Secret that has the TLS certificate for the **oidc-discovery-provider** Route host. When provided, this populates the route's **.Spec.TLS.ExternalCertificate** field. For more information, see [Creating a route with externally managed certificate](#)

Enabling the custom Certificate Authority Time-To-Live for the SPIRE bundle

- The following Time-To-Live (TTL) fields have been added to the **SpireServer** custom resource definition (CRD) API for SPIRE Server certificate management:
 - **CAValidity** (default: 24h)
 - **DefaultX509Validity** (default: 1h)
 - **DefaultJWTValidity** (default: 5m)

- The default values can be replaced in the server configuration with user-configurable options that give users the flexibility to customize certificate and SPIFFE Verifiable Identity Document (SVID) lifetimes based on their security requirements.

Enabling Manual User Configurations

- The Operator controller switches to **create-only** mode once the **ztwim.openshift.io/create-only=true** annotation is present on the Operator's APIs. This allows resource creation while skipping the updates. A user can update the resources manually to test their configuration. This annotation supports APIs such as **SpireServer**, **SpireAgents**, **SpiffeCSIDriver**, **SpireOIDCDiscoveryProvider**, and **ZeroTrustWorkloadIdentityManager**.
- When the annotation is applied, all derived resources including resources created and managed by the Operator are created but not updated.
- After the annotation is removed and the pod restarts, the Operator tries to come back to the required state. The annotation is applied only once during start or a restart.

11.3.4.2. Fixed issues

JSON Web Token Issuer field now requires a valid URL

- Before this update, the **JwtIssuer** field for both the **SpireServer** and the **SpireOidcDiscoveryProvider** custom resources did not require the input to be a URL, frequently causing configuration errors. With this release, the validation has been updated, and users must now manually enter a valid issuer URL in the **JwtIssuer** field for both custom resources. As a result, misconfigurations caused by a malformed issuer values are prevented, ensuring a stable and reliable setup.
([SPIRE-117](#))

11.3.5. Zero Trust Workload Identity Manager 0.1.0 (General Availability)

Issued: 16 June 2025

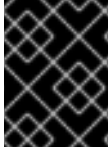
The following advisories are available for the Zero Trust Workload Identity Manager:

- [RHBA-2025:9088](#)
- [RHBA-2025:9085](#)
- [RHBA-2025:9090](#)
- [RHBA-2025:9084](#)
- [RHBA-2025:9089](#)
- [RHBA-2025:9087](#)
- [RHBA-2025:9101](#)
- [RHBA-2025:9104](#)

11.4. INSTALLING THE ZERO TRUST WORKLOAD IDENTITY MANAGER

Install Zero Trust Workload Identity Manager to help ensure secure communication between your workloads. You can install the Zero Trust Workload Identity Manager by using either the web console or CLI.

If you install the Operator into a custom namespace (for example, **my-custom-namespace**), all managed operand resources are deployed within that same namespace. All secrets and ConfigMaps referenced by the Custom Resources (CRs) must also exist in that custom namespace.



IMPORTANT

The Operator installation is not supported in the **openshift-*** namespaces and the **default** namespace.

11.4.1. Installing the Zero Trust Workload Identity Manager by using the web console

Use the Software Catalog in the OpenShift Container Platform web console to install the Zero Trust Workload Identity Manager. This process streamlines deployment and helps ensure the Operator is installed in the correct namespace with the appropriate installation mode.



NOTE

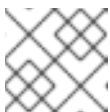
A minimum of 1Gi persistent volume is required to install the SPIRE Server.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Go to **Ecosystem** → **Software Catalog**.
3. Search for **Zero Trust Workload Identity Manager**.
4. On the **Install Operator** page:
 - a. Update the **Update channel**, if necessary. The channel defaults to **stable-v1**, which installs the latest **stable-v1** release of the Zero Trust Workload Identity Manager.
 - b. Choose the **Installed Namespace** for the Operator. The default Operator namespace is **zero-trust-workload-identity-manager**.
If the **zero-trust-workload-identity-manager** namespace does not exist, it is created for you.



NOTE

The Operator and operands are deployed in the same namespace.

- c. Select an **Update Approval** strategy

- The **Automatic strategy** allows Operator Lifecycle Manager (OLM) to automatically update the Operator when a new version is available.
- The **Manual strategy** requires a user with appropriate credentials to approve the Operator update.

5. Click **Install**.

Verification

1. Navigate to **Ecosystem → Installed Operators**.
 - a. Verify that **Zero Trust Workload Identity Manager** is listed with a **Status** of **Succeeded** in the **zero-trust-workload-identity-manager** namespace.
 - b. Verify that Zero Trust Workload Identity Manager controller manager deployment is ready and available by running the following command:

```
$ oc get deployment -l name=zero-trust-workload-identity-manager -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
zero-trust-workload-identity-manager-controller-manager	1/1	1	1	3h36m

2. To check the Operator logs, run the following command:

```
$ oc logs -f deployment/zero-trust-workload-identity-manager-controller-manager -n zero-trust-workload-identity-manager
```

11.4.2. Installing the Zero Trust Workload Identity Manager by using the CLI

Install the Zero Trust Workload Identity Manager by using the command-line interface (CLI) to create the required project, **OperatorGroup**, and **Subscription** objects. You can then deploy the Operator components necessary for managing workload identities on your OpenShift Container Platform cluster.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.



NOTE

A minimum of 1Gi persistent volume is required to install the SPIRE Server.

Procedure

1. Create a new project named **zero-trust-workload-identity-manager** by running the following command:

```
$ oc new-project zero-trust-workload-identity-manager
```

2. Create an **OperatorGroup** object:

- a. Create a YAML file, for example, **operatorGroup.yaml**, with the following content:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-zero-trust-workload-identity-manager
  namespace: zero-trust-workload-identity-manager
spec:
  upgradeStrategy: Default
```

- b. Create the **OperatorGroup** object by running the following command:

```
$ oc create -f operatorGroup.yaml
```

3. Create a **Subscription** object:

- a. Create a YAML file, for example, **subscription.yaml**, that defines the **Subscription** object:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: openshift-zero-trust-workload-identity-manager
  namespace: zero-trust-workload-identity-manager
spec:
  channel: stable-v1
  name: openshift-zero-trust-workload-identity-manager
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
```

- b. Create the **Subscription** object by running the following command:

```
$ oc create -f subscription.yaml
```

Verification

- Verify that the OLM subscription is created by running the following command:

```
$ oc get subscription -n zero-trust-workload-identity-manager
```

Example output

NAME	PACKAGE	SOURCE
CHANNEL		
openshift-zero-trust-workload-identity-manager	zero-trust-workload-identity-manager	
redhat-operators	stable-v1	

- Verify whether the Operator is successfully installed by running the following command:

```
$ oc get csv -n zero-trust-workload-identity-manager
```

Example output

NAME	DISPLAY	VERSION	PHASE
zero-trust-workload-identity-manager.v1.0.0	Zero Trust Workload Identity Manager	1.0.0	Succeeded

- Verify that the Zero Trust Workload Identity Manager controller manager is ready by running the following command:

```
$ oc get deployment -l name=zero-trust-workload-identity-manager -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
zero-trust-workload-identity-manager-controller-manager	1/1	1	1	43m

11.5. DEPLOYING ZERO TRUST WORKLOAD IDENTITY MANAGER OPERANDS

Deploy the Zero Trust Workload Identity Manager operands by creating their custom resources in a specific order. Adhering to the sequence ensures the successful installation of components, such as the Security Production Identity Framework for Everyone (SPIRE) Server, SPIRE Agent, and Secure Production Identity Framework For Everyone (SPIFFE) CSI driver.

You must deploy the operands in the following sequence to ensure successful installation:

- **ZeroTrustWorkloadIdentityManager** CR
- SPIRE Server
- SPIRE Agent
- SPIFFE CSI driver
- SPIRE OIDC discovery provider

11.5.1. About the ZeroTrustWorkloadIdentityManager custom resource

The **ZeroTrustWorkloadIdentityManager** is the primary custom resource that initializes the SPIRE deployments. This primary resource defines the trust domain and cluster name to help ensure secure workload identity management.

Reference the complete YAML specification to correctly structure the **ZeroTrustWorkloadIdentityManager** CR. This example helps you identify required fields and immutable parameters for your configuration.

```
apiVersion: operator.openshift.io/v1alpha1
kind: ZeroTrustWorkloadIdentityManager
metadata:
  name: cluster
labels:
  app.kubernetes.io/name: zero-trust-workload-identity-manager
  app.kubernetes.io/managed-by: zero-trust-workload-identity-manager
spec:
```

```
trustDomain: "example.com"
clusterName: "production-cluster"
bundleConfigMap: "spire-bundle"
```

where:

spec.trustDomain

Specifies the trust domain to be used for the SPIFFE identifiers. Must be a valid SPIFFE trust domain (lowercase alphanumeric, hyphens, and dots). Maximum length is 255 characters. After setting a value for the field, the field is immutable. Red Hat highly recommends to set this value to match your the base application URL (for example, **apps.mycluster.example.com**) of your OpenShift Container Platform cluster. Using a different value might cause automatically generated OpenShift Routes or federation endpoints to be created with incorrect or mismatched hostnames later in the configuration process.

spec.clusterName

Specifies the name that identifies this cluster within the trust domain. Must be a valid DNS-1123 subdomain with a maximum length of 63 characters. Once set, this field is immutable.

spec.bundleConfigMap

Specifies the name of the ConfigMap that stores the SPIRE trust bundle. This ConfigMap contains the root certificates for the trust domain and is created and maintained by the Operator. Must be a valid Kubernetes name with a maximum length of 253 characters. This field is optional (defaults to **spire-bundle**) and once set, is immutable.

11.5.2. Deploying the SPIRE Server

Deploy the SPIRE Server by configuring the **SpireServer** custom resource (CR). This establishes a central authority that manages and issues identities to the workloads in your cluster.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed Zero Trust Workload Identity Manager in the cluster.

Procedure

1. Create the **SpireServer** CR:
 - a. Create a YAML file that defines the **SpireServer** CR, for example, **SpireServer.yaml**:
The following is an example of a **SpireServer.yaml** file.

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  jwtIssuer: "https://oidc-discovery.apps.cluster.example.com"
  caValidity: "24h"
  defaultX509Validity: "1h"
  defaultJWTValidity: "5m"
  jwtKeyType: "rsa-2048"
```

```

caSubject:
  country: "US"
  organization: "Example Corporation"
  commonName: "SPIRE Server CA"
persistence:
  size: "5Gi"
  accessMode: "ReadWriteOnce"
  storageClass: "gp3-csi"
datastore:
  databaseType: "sqlite3"
  connectionString: "/run/spire/data/datastore.sqlite3"
  tlsSecretName: ""
  maxOpenConns: 100
  maxIdleConns: 10
  connMaxLifetime: 0
  disableMigration: "false"

```

where:

metadata.name

Specifies that the value must be **cluster**.

spec.logLevel

Specifies the logging level for the SPIRE Server. The valid options are **debug**, **info**, **warn**, and **error**.

spec.logFormat

Specifies the logging format for the SPIRE Server. The valid options are **text** and **json**.

spec.jwtIssuer

Specifies the JWT issuer URL. Must be a valid HTTPS or HTTP URL with a maximum length of 512 characters.

spec.caValidity

Specifies the validity period (Time to Live (TTL)) for the SPIRE Server's CA certificate. This determines how long the server's root or intermediate certificate is valid. The format is a duration string (for example, **24h**, **168h**).

spec.defaultX509Validity

Specifies the default validity period (TTL) for X.509 SVIDs issued to workloads. This value is used if a specific TTL is not configured for a registration entry.

spec.defaultJWTValidity

Specifies the default validity period (TTL) for JWT SVIDs issued to workloads. This value is used if a specific TTL is not configured for a registration entry.

spec.wtKeyType

Specifies the key type used for JWT signing. The valid options are **rsa-2048**, **rsa-4096**, **ec-p256**, and **ec-p384**. This field is optional.

spec.caSubject.country

Specifies the country for the SPIRE Server certificate authority (CA). Must be an ISO 3166-1 alpha-2 country code (2 characters).

spec.caSubject.organization

Specifies the organization for the SPIRE Server CA. Maximum length is 64 characters.

spec.caSubject.commonName

Specifies the common name for the SPIRE Server CA. Maximum length is 255 characters.

spec.persistence.size

Specifies the size of the persistent volume (for example, **1Gi**, **5Gi**). Once set, this field is immutable.

spec.persistence.accessMode

Specifies the access mode for the persistent volume. The valid options are **ReadWriteOnce**, **ReadWriteOncePod**, and **ReadWriteMany**. Once set, this field is immutable.

spec.persistence.storageClass

Specifies the storage class to be used for the PVC. Once set, this field is immutable.

spec.datastore.databaseType

Specifies the type of database to use for the datastore. The valid options are **sql**, **sqlite3**, **postgres**, **mysql**, **aws_postgresql**, and **aws_mysql**.

spec.datastore.connectionString

Specifies the connection string for the database. For PostgreSQL with SSL, include **sslmode** and certificate paths (for example, **dbname=spire user=spire host=postgres.example.com sslmode=verify-full**).

spec.datastore.tlsSecretName

Specifies the name of a Kubernetes Secret containing TLS certificates for database connections. The Secret will be mounted at **/run/spire/db/certs**. This field is optional.

spec.datastore.maxOpenConns

Specifies the maximum number of open database connections. Must be between 1 and 10000.

spec.datastore.maxIdleConns

Specifies the maximum number of idle database connections in the pool. Must be between 0 and 10000.

spec.datastore.connMaxLifetime

Specifies the maximum lifetime of a database connection in seconds. A value of 0 means connections are not closed due to age.

spec.datastore.disableMigration

Specifies whether to disable automatic database migration. The valid options are **true** and **false**.

- b. Apply the configuration by running the following command:

```
$ oc apply -f SpireServer.yaml
```

Verification

- Verify that the stateful set of SPIRE Server is ready and available by running the following command:

```
$ oc get statefulset -l app.kubernetes.io/name=server -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	AGE
spire-server	1/1	65s

- Verify that the status of the SPIRE Server pod is **Running** by running the following command:

```
$ oc get po -l app.kubernetes.io/name=server -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
spire-server-0	2/2	Running	1 (108s ago)	111s

- Verify that the persistent volume claim (PVC) is bound, by running the following command:

```
$ oc get pvc -l app.kubernetes.io/name=server -n zero-trust-workload-identity-manager
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS
MODES	STORAGECLASS	VOLUMEATTRIBUTECLASS	AGE	
spire-data-spire-server-0	Bound	pvc-27a36535-18a1-4fde-ab6d-e7ee7d3c2744	5Gi	
RW0	gp3-csi	<unset>	22m	

11.5.3. Deploying the SPIRE Agent

Use the **SpireAgent** custom resource to configure the SPIRE Agent **DaemonSet** on your nodes. This defines how the agent verifies workloads and manages identity attestation across your OpenShift Container Platform cluster.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed Zero Trust Workload Identity Manager in the cluster.

Procedure

1. Create the **SpireAgent** CR:
 - a. Create a YAML file that defines the **SpireAgent** CR, for example, **SpireAgent.yaml**:
The following is an example of a **SpireAgent.yaml** file.

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireAgent
metadata:
  name: cluster
spec:
  socketPath: "/run/spire/agent-sockets"
  logLevel: "info"
  logFormat: "text"
  nodeAttestor:
    k8sPSATEnabled: "true"
```

```

workloadAttestors:
  k8sEnabled: "true"
workloadAttestorsVerification:
  type: "auto"
  hostCertBasePath: "/etc/kubernetes"
  hostCertFileName: "kubelet-ca.crt"
disableContainerSelectors: "false"
useNewContainerLocator: "true"

```

where:

metadata.name

Specifies that the value must be **cluster**.

spec.socketPath

Specifies the directory on the host where the SPIRE agent socket is created. This directory is shared with the SPIFFE CSI driver via the **hostPath** volume. Must match the **SpiffeCSIDriver.spec.agentSocketPath** for workloads to access the socket. Must be an absolute path with a maximum length of 256 characters.

spec.logLevel

Specifies the logging level for the SPIRE Server. The valid options are **debug**, **info**, **warn**, and **error**.

spec.logFormat

Specifies the logging format for the SPIRE Server. The valid options are **text** and **json**.

spec.nodeAttestor.k8sPSATEnabled

Specifies whether Kubernetes Projected Service Account Token (PSAT) node attestation is enabled. When enabled, the SPIRE agent uses K8s PSATs to prove its identity to the SPIRE server during node attestation. The valid options are **true** and **false**.

spec.workloadAttestors.k8sEnabled

Specifies whether the Kubernetes workload attestor is enabled. When enabled, the SPIRE agent can verify workload identities using Kubernetes pod information and service account tokens. The valid options are **true** and **false**.

spec.workloadAttestors.workloadAttestorsVerification.type

Specifies the kubelet certificate verification mode. The valid options are **auto**, **hostCert**, and **skip**.

spec.workloadAttestors.workloadAttestorsVerification.hostCertBasePath

Specifies the directory containing the kubelet CA certificate. Required when type is **hostCert**. Optional when type is **auto** (defaults to `/etc/kubernetes` if not specified).

spec.workloadAttestors.workloadAttestorsVerification.hostCertFileName

Specifies the file name for the kubelet's CA certificate. When combined with **hostCertBasePath**, forms the full path. Required when type is **hostCert**. Optional when type is **auto**. Defaults to **kubelet-ca.crt** if not specified.

spec.workloadAttestors.disableContainerSelectors

Specifies whether to disable container selectors in the Kubernetes workload attestor. Set to **true** if using **holdApplicationUntilProxyStarts** in Istio. The valid options are **true** and **false**.

spec.workloadAttestors.useNewContainerLocator

Specifies enabling the new container locator algorithm that has support for cgroups v2. The valid options are **true** and **false**.

- b. Apply the configuration by running the following command:

```
$ oc apply -f SpireAgent.yaml
```

Verification

- Verify that the daemon set of the SPIRE Agent is ready and available by running the following command:

```
$ oc get daemonset -l app.kubernetes.io/name=agent -n zero-trust-workload-identity-manager
```

Example output

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
spire-agent	3	3	3	3	<none>	10m

- Verify that the status of SPIRE Agent pods is **Running** by running the following command:

```
$ oc get po -l app.kubernetes.io/name=agent -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
spire-agent-dp4jb	1/1	Running	0	12m
spire-agent-nvwjm	1/1	Running	0	12m
spire-agent-vtvtk	1/1	Running	0	12m

11.5.4. Deploying the SPIFFE Container Storage Interface driver

Configure the Container Storage Interface (CSI) driver using the **SpiffeCSIDriver** CR. This configuration mounts SPIFFE sockets directly into workload pods, which allows your applications to access the SPIFFE Workload API securely.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed Zero Trust Workload Identity Manager in the cluster.

Procedure

1. Create the **SpiffeCSIDriver** CR:
 - a. Create a YAML file that defines the **SpiffeCSIDriver** CR object, for example, **SpiffeCSIDriver.yaml**:

Example SpiffeCSIDriver.yaml

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpiffeCSIDriver
metadata:
  name: cluster
spec:
  agentSocketPath: "/run/spire/agent-sockets"
  pluginName: "csi.spiffe.io"
```

where:

metadata.name

Specifies that the name must be **cluster**.

spec.agentSocketPath

Specifies the path to the directory containing the SPIRE agent’s Workload API socket. This directory is bind-mounted into workload containers by the CSI driver. The directory is shared between the SPIRE agent and CSI driver via a **hostPath** volume. Must be an absolute path with a maximum length of 256 characters. This value must match **SpireAgent.spec.socketPath** for workloads to access the socket.

spec.pluginName

Specifies the name of the CSI plugin. This sets the CSI driver name that is deployed to the cluster and used in **VolumeMount** configurations. Must match the driver name referenced in the workload pods. Must be a valid domain name format (for example, **csi.spiffe.io**) with a maximum length of 127 characters.

- b. Apply the configuration by running the following command:

```
$ oc apply -f SpiffeCSIDriver.yaml
```

Verification

- Verify that the daemon set of the SPIFFE CSI driver is ready and available by running the following command:

```
$ oc get daemonset -l app.kubernetes.io/name=spiffe-csi-driver -n zero-trust-workload-identity-manager
```

Example output

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
SELECTOR	AGE					
spire-spiffe-csi-driver	3	3	3	3	<none>	114s

- Verify that the status of SPIFFE Container Storage Interface (CSI) Driver pods is **Running** by running the following command:

```
$ oc get po -l app.kubernetes.io/name=spiffe-csi-driver -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

spire-spiffe-csi-driver-gpwc	2/2	Running	0	2m37s
spire-spiffe-csi-driver-rrbrd	2/2	Running	0	2m37s
spire-spiffe-csi-driver-w6s6q	2/2	Running	0	2m37s

11.5.5. Deploying the SPIRE OpenID Connect Discovery Provider

Deploy the SPIRE OpenID Connect (OIDC) Discovery Provider by configuring the **SpireOIDCDiscoveryProvider** CR. This allows you to define the trust domain and JSON web token (JWT) issuer for your cluster.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed Zero Trust Workload Identity Manager in the cluster.

Procedure

1. Create the **SpireOIDCDiscoveryProvider** CR:
 - a. Create a YAML file that defines the **SpireOIDCDiscoveryProvider** CR, for example, **SpireOIDCDiscoveryProvider.yaml**:

Example SpireOIDCDiscoveryProvider YAML

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireOIDCDiscoveryProvider
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  csiDriverName: "csi.spiffe.io"
  jwtIssuer: "https://oidc-discovery.apps.cluster.example.com"
  replicaCount: 1
  managedRoute: "true"
  externalSecretRef: ""
```

where:

metadata.name

Specifies that the value must be **cluster**.

spec.logLevel

Specifies the logging level for the SPIRE Server. The valid options are **debug**, **info**, **warn**, and **error**.

spec.logFormat

Specifies the logging format for the SPIRE Server. The valid options are **text** and **json**.

spec.csiDriverName

Specifies the name of the CSI driver to use for mounting the Workload API socket. This must match the **SpiffeCSIDriver.spec.pluginName** value for the OIDC provider to access SPIFFE identities. Must be a valid DNS subdomain format (for example,

csi.spiffe.io) with a maximum length of 127 characters.

spec.jwtIssuer

Specifies the JWT issuer URL. Must be a valid HTTPS or HTTP URL with a maximum length of 512 characters. This value must match the **SpireServer.spec.jwtIssuer** value.

spec.replicaCount

Specifies the number of replicas for the OIDC Discovery Provider deployment. Must be between 1 and 5.

spec.managedRoute

Specifies whether the Operator automatically creates an OpenShift route for the OIDC Discovery Provider endpoints. Set to **true** to have the Operator automatically create and maintain an OpenShift route for OIDC discovery endpoints (***.apps.**). Set to **false** for administrators to manually configure routes or ingress.

spec.externalSecretRef

Specifies a reference to an externally managed secret that contains the TLS certificate for the OIDC Discovery Provider route host. Must be a valid Kubernetes secret reference name with a maximum length of 253 characters. This field is optional.

- b. Apply the configuration by running the following command:

```
$ oc apply -f SpireOIDCDiscoveryProvider.yaml
```

Verification

1. Verify that the deployment of OIDC Discovery Provider is ready and available by running the following command:

```
$ oc get deployment -l app.kubernetes.io/name=spiffe-oidc-discovery-provider -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
spire-spiffe-oidc-discovery-provider	1/1	1	1	2m58s

2. Verify that the status of OIDC Discovery Provider pods is **Running** by running the following command:

```
$ oc get po -l app.kubernetes.io/name=spiffe-oidc-discovery-provider -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
spire-spiffe-oidc-discovery-provider-64586d599f-lcc94	2/2	Running	0	7m15s

11.5.6. Verify the health of the operands

View the status fields to verify the operational health of managed components. This information helps you confirm that the SPIRE Server, SPIRE Agent, SPIFFE CSI driver, and the SPIRE OIDC discovery provider operands are ready and functioning correctly.

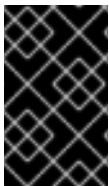
- To verify the operands, run the following command:

```
oc get ZeroTrustWorkloadIdentityManager cluster -o yaml
```

Example output

```
status:
  conditions:
  - lastTransitionTime: "2025-12-16T10:59:06Z"
    message: All components are ready
    reason: Ready
    status: "True"
    type: Ready
  - lastTransitionTime: "2025-12-16T10:59:06Z"
    message: All operand CRs are ready
    reason: Ready
    status: "True"
    type: OperandsAvailable
  operands:
  - kind: SpireServer
    message: Ready
    name: cluster
    ready: "true"
  - kind: SpireAgent
    message: Ready
    name: cluster
    ready: "true"
  - kind: SpiffeCSIDriver
    message: Ready
    name: cluster
    ready: "true"
  - kind: SpireOIDCDiscoveryProvider
    message: Ready
    name: cluster
    ready: "true"
  # ...
```

This status is reflected when all operands are healthy and stable.



IMPORTANT

The Operator adds the owner reference for the **ZeroTrustWorkloadIdentityManager** CR on the other operands' CRs. This causes the operands' resources to be deleted once the **ZeroTrustWorkloadIdentityManager** CRs are deleted.

11.5.7. Manually delete the custom security context constraints

You can delete the **spire-spiffe-csi-driver** custom SCC when the SPIFFE CSI driver is ready and its pods are using the **privileged** security context constraint (SCC).

Prerequisites

- You have upgraded Zero Trust Workload Identity Manager to 1.1.0 from the **OperatorHub** catalog.

Procedure

- Confirm the SPIFFE CSI driver is ready by running the following command:

```
$ oc get spiffecsi driver cluster -o jsonpath='{range .status.conditions[*]}.{type}={.status}}{"\n"}{end}'
```

The expected output includes **DaemonSetAvailable=True** and **Ready=True**.

- Confirm that the CSI **DaemonSet** is available by running the following command:

```
$ oc get ds spire-spiffe-csi-driver -n zero-trust-workload-identity-manager
```

- Confirm the CSI pods are running and are using the **privileged** SCC by running the following commands:

```
$ oc get pods -n zero-trust-workload-identity-manager -l app.kubernetes.io/name=spiffe-csi-driver
```

```
$ oc get pod -n zero-trust-workload-identity-manager -l app.kubernetes.io/name=spiffe-csi-driver \
-o jsonpath='{range .items[*]}.{metadata.name}{"\t"}{.metadata.annotations.openshift.io/scc}{"\n"}{end}'
```

Every pod should show **privileged**.

- After all checks have passed, delete the legacy custom SCC by running the following commands:

```
$ oc get scc spire-spiffe-csi-driver
```

```
$ oc delete scc spire-spiffe-csi-driver
```

If the SCC is already absent, no action is needed.

11.6. CONFIGURING THE EGRESS PROXY FOR THE ZERO TRUST WORKLOAD IDENTITY MANAGER

Operator Lifecycle Manager (OLM) automatically configures managed Operators with proxy settings when you use a cluster-wide egress proxy. To support proxying HTTPS connections, you can inject certificate authority (CA) certificates into the Zero Trust Workload Identity Manager.

11.6.1. Injecting a custom CA certificate for the Zero Trust Workload Identity Manager

Inject certificate authority (CA) certificates into the Zero Trust Workload Identity Manager to support proxying HTTPS connections. This configuration helps ensure that the Identity Manager can communicate securely when you enable a cluster-wide proxy.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have enabled the cluster-wide proxy for OpenShift Container Platform.
- You have installed Zero Trust Workload Identity Manager 1.0.0 or later.
- You have deployed the SPIRE Server, SPIRE Agent, SPIFFE CSI Driver, and the SPIRE OIDC Discovery Provider operands in the cluster.

Procedure

1. Create a config map in the **zero-trust-workload-identity-manager** namespace by running the following command:

```
$ oc create configmap trusted-ca -n zero-trust-workload-identity-manager
```

2. Inject the CA bundle that is trusted by OpenShift Container Platform into the config map by running the following command:

```
$ oc label cm trusted-ca config.openshift.io/inject-trusted-cabundle=true -n zero-trust-workload-identity-manager
```

3. Update the subscription for the Zero Trust Workload Identity Manager to use the config map by running the following command:

```
$ oc -n zero-trust-workload-identity-manager patch subscription openshift-zero-trust-workload-identity-manager --type='merge' -p '{"spec":{"config":{"env":[{"name":"TRUSTED_CA_BUNDLE_CONFIGMAP","value":"trusted-ca"}]}}}'
```

Verification

1. Verify that the operands have finished rolling out by running the following command:

```
$ oc rollout status deployment/zero-trust-workload-identity-manager-controller-manager -n zero-trust-workload-identity-manager && \
$ oc rollout status statefulset/spireserver -n zero-trust-workload-identity-manager && \
$ oc rollout status daemonset/spire-agent -n zero-trust-workload-identity-manager && \
$ oc rollout status deployment/spire-spiFFE-oidc-discovery-provider -n zero-trust-workload-identity-manager
```

Example output

```
deployment "zero-trust-workload-identity-manager-controller-manager" successfully rolled out
statefulset "spire-server" successfully rolled out
daemonset "spire-agent" successfully rolled out
deployment "spire-spiFFE-oidc-discovery-provider" successfully rolled out
```

2. Verify that the CA bundle was mounted as a volume by running the following command:

```
$ oc get deployment zero-trust-workload-identity-manager -n zero-trust-workload-identity-manager -o=jsonpath={.spec.template.spec.containers[0].volumeMounts}
```

```
$ oc get statefulset spire-server -n zero-trust-workload-identity-manager -o jsonpath='{.spec.template.spec.containers[*].volumeMounts[?(@.name=="trusted-ca-bundle")]}'
```

```
$ oc get daemonset spire-agent -n zero-trust-workload-identity-manager -o jsonpath='{.spec.template.spec.containers[*].volumeMounts[?(@.name=="trusted-ca-bundle")]}'
```

```
$ oc get daemonset spire-spiffe-csi-driver -n zero-trust-workload-identity-manager -o jsonpath='{.spec.template.spec.containers[*].volumeMounts[?(@.name=="trusted-ca-bundle")]}'
```

Example output

```
[{"mountPath":"/etc/pki/ca-trust/extracted/pem","name":"trusted-ca-bundle","readOnly":true}]
```

3. Verify that the source of the CA bundle is the **trusted-ca** config map by running the following command:

```
$ oc get deployment zero-trust-workload-identity-manager -n zero-trust-workload-identity-manager -o=jsonpath={.spec.template.spec.volumes}
```

```
$ oc get statefulset spire-server -n zero-trust-workload-identity-manager -o=jsonpath='{.spec.template.spec.volumes}' | jq '.[] | select(.name=="trusted-ca-bundle")'
```

```
$ oc get daemonset spire-agent -n zero-trust-workload-identity-manager -o=jsonpath='{.spec.template.spec.volumes}' | jq '.[] | select(.name=="trusted-ca-bundle")'
```

```
$ oc get deployment spire-spiffe-oidc-discovery-provider -n zero-trust-workload-identity-manager -o=jsonpath='{.spec.template.spec.volumes}' | jq '.[] | select(.name=="trusted-ca-bundle")'
```

Example output

```
{
  "configMap": {
    "defaultMode": 420,
    "items": [
      {
        "key": "ca-bundle.crt",
        "path": "tls-ca-bundle.pem"
      }
    ],
    "name": "trusted-ca"
  }
}
```



```

    },
    "name": "trusted-ca-bundle"
  }

```

11.6.2. Additional resources

- [Configuring proxy support in Operator Lifecycle Manager](#)

11.7. ZERO TRUST WORKLOAD IDENTITY MANAGER OIDC FEDERATION

Ensure that your workloads can receive verifiable JSON Web Tokens (JWT-SVIDs) and allow external systems, such as cloud providers, to retrieve public keys from the discovery endpoint. Configure Zero Trust Workload Identity Manager to act as an OpenID Connect (OIDC) provider through the SPIRE server.

The following providers are verified to work with SPIRE OIDC federation:

- Azure Entra ID
- Vault

11.7.1. About the Entra ID OpenID Connect

Integrate Entra ID OpenID Connect (OIDC) with SPIRE to provide workloads with automatic, short-lived cryptographic identities. This configuration allows you to securely authenticate services without maintaining static secrets.

11.7.1.1. Configuring the external certificate for the managed OIDC discovery provider route

Configure the managed OIDC discovery provider route to use an externally managed TLS certificate. By referencing a TLS secret, you can secure the OIDC endpoint with your own certificate credentials.

Prerequisites

- You have installed Zero Trust Workload Identity Manager 0.2.0 or later.
- You have deployed the SPIRE Server, SPIRE Agent, SPIFFEE CSI Driver, and the SPIRE OIDC Discovery Provider operands in the cluster.
- You have installed the cert-manager Operator for Red Hat OpenShift. For more information, [Installing the cert-manager Operator for Red Hat OpenShift](#).
- You have created a **ClusterIssuer** or **Issuer** configured with a publicly trusted CA service. For example, an Automated Certificate Management Environment (ACME) type **Issuer** with the "Let's Encrypt ACME" service. For more information, see [Configuring an ACME issuer](#)

Procedure

1. Create a **Role** to provide the router service account permissions to read the referenced secret by running the following command:

```

$ oc create role secret-reader \
  --verb=get,list,watch \

```

```
--resource=secrets \
--resource-name=$TLS_SECRET_NAME \
-n zero-trust-workload-identity-manager
```

2. Create a **RoleBinding** resource to bind the router service account with the newly created Role resource by running the following command:

```
$ oc create rolebinding secret-reader-binding \
--role=secret-reader \
--serviceaccount=openshift-ingress:router \
-n zero-trust-workload-identity-manager
```

3. Configure the **SpireOIDCDiscoveryProvider** Custom Resource (CR) object to reference the Secret generated in the earlier step by running the following command:

```
$ oc patch SpireOIDCDiscoveryProvider cluster --type=merge -p='
spec:
  externalSecretRef: ${TLS_SECRET_NAME}
'
```

Verification

1. In the **SpireOIDCDiscoveryProvider** CR, check if the **ManageRouteReady** condition is set to **True** by running the following command:

```
$ oc wait --for=jsonpath='{.status.conditions[?
(@.type=="ManagedRouteReady")].status}'=True SpireOIDCDiscoveryProvider/cluster --
timeout=120s
```

2. Verify that the OIDC endpoint can be accessed securely through HTTPS by running the following command:

```
$ curl https://$JWT_ISSUER_ENDPOINT/.well-known/openid-configuration

{
  "issuer": "https://$JWT_ISSUER_ENDPOINT",
  "jwks_uri": "https://$JWT_ISSUER_ENDPOINT/keys",
  "authorization_endpoint": "",
  "response_types_supported": [
    "id_token"
  ],
  "subject_types_supported": [],
  "id_token_signing_alg_values_supported": [
    "RS256",
    "ES256",
    "ES384"
  ]
}%
```

11.7.1.2. Disabling a managed route

If you want to fully control the behavior of exposing the OIDC Discovery Provider service, you can disable the managed route based on your requirements.

Procedure

- To manually configure the OIDC Discovery Provider, set **managedRoute** to **false** by running the following command:

```
$ oc patch SpireOIDCDiscoveryProvider cluster --type=merge -p='
spec:
  managedRoute: "false"
```

11.7.1.3. Using Entra ID with Microsoft Azure

Configure your Microsoft Azure environment to enable Entra ID integration with Azure. By defining variables and creating a resource group, you establish the infrastructure needed to securely manage workload identities.

Prerequisites

- You have configured the SPIRE OIDC Discovery Provider Route to serve the TLS certificates from a publicly trusted CA.

Procedure

- Log in to Azure by running the following command:

```
$ az login
```

- Configure variables for your Azure subscription and tenant by running the following commands:

```
$ export SUBSCRIPTION_ID=$(az account list --query "[?isDefault].id" -o tsv)
```

```
$ export TENANT_ID=$(az account list --query "[?isDefault].tenantId" -o tsv)
```

```
$ export LOCATION=centralus
```

where:

SUBSCRIPTION_ID

Specifies your unique subscription identifier.

TENANT_ID

Specifies the ID for your Azure Active Directory instance.

LOCATION

The Azure region where your resource is created.

- Define resource variable names by running the following commands:

```
$ export NAME=ztwim
```

```
$ export RESOURCE_GROUP="${NAME}-rg"
```

```
$ export STORAGE_ACCOUNT="${NAME}storage"
```

```
$ export STORAGE_CONTAINER="${NAME}storagecontainer"
```

```
$ export USER_ASSIGNED_IDENTITY_NAME="${NAME}-identity"
```

where:

NAME

Specifies A base name for all resources.

RESOURCE_GROUP

Specifies the name of the resource group.

STORAGE_ACCOUNT

Specifies the name for the storage account.

STORAGE_CONTAINER

Specifies the name for the storage container.

USER_ASSIGNED_IDENTITY_NAME

Specifies the name for a managed identity.

4. Create the resource group by running the following command:

```
$ az group create \
  --name "${RESOURCE_GROUP}" \
  --location "${LOCATION}"
```

11.7.1.4. Configuring Azure blob storage

Create a new Microsoft Azure storage account and container to provide a dedicated location for your content. Configuring this storage ensures that the Zero Trust Workload Identity Manager can successfully store and retrieve blobs for your environment.

Procedure

1. Create a new storage account that is used to store content by running the following command:

```
$ az storage account create \
  --name ${STORAGE_ACCOUNT} \
  --resource-group ${RESOURCE_GROUP} \
  --location ${LOCATION} \
  --encryption-services blob
```

2. Obtain the storage ID for the newly created storage account by running the following command:

```
$ export STORAGE_ACCOUNT_ID=$(az storage account show -n
${STORAGE_ACCOUNT} -g ${RESOURCE_GROUP} --query id --out tsv)
```

3. Create a storage container inside the newly created storage account to provide a location to support the storage of blobs by running the following command:

```
$ az storage container create \
  --account-name ${STORAGE_ACCOUNT} \
  --name ${STORAGE_CONTAINER} \
```

```
--auth-mode login
```

11.7.1.5. Configuring an Azure user managed identity

Create a user-assigned managed identity in Azure to manage access control for your resources. You must also obtain the Client ID to associate roles with the service principal.

Procedure

1. Create a new User Managed Identity and then obtain the Client ID of the related Service Principal associated with the User Managed Identity by running the following command:

```
$ az identity create \
  --name ${USER_ASSIGNED_IDENTITY_NAME} \
  --resource-group ${RESOURCE_GROUP}
```

```
$ export IDENTITY_CLIENT_ID=$(az identity show --resource-group
"${RESOURCE_GROUP}" --name "${USER_ASSIGNED_IDENTITY_NAME}" --query
'clientId' -otsv)
```

2. Retrieve the **CLIENT_ID** of an Azure user-assigned managed identity and save it as an environment variable by running the following command:

```
$ export IDENTITY_CLIENT_ID=$(az identity show --resource-group
"${RESOURCE_GROUP}" --name "${USER_ASSIGNED_IDENTITY_NAME}" --query
'clientId' -otsv)
```

3. Associate a role with the Service Principal associated with the User Managed Identity by running the following command:

```
$ az role assignment create \
  --role "Storage Blob Data Contributor" \
  --assignee "${IDENTITY_CLIENT_ID}" \
  --scope ${STORAGE_ACCOUNT_ID}
```

11.7.1.6. Creating the demonstration application

Create the demonstration application to verify that the entire system functions correctly. This process validates the configuration of your application secrets and namespaces.

Procedure

1. Set the application name and namespace by running the following commands:

```
$ export APP_NAME=workload-app
```

```
$ export APP_NAMESPACE=demo
```

2. Create the namespace by running the following command:

```
$ oc create namespace $APP_NAMESPACE
```

3. Create the application Secret by running the following command:

```
$ oc apply -f - << EOF
apiVersion: v1
kind: Secret
metadata:
  name: $APP_NAME
  namespace: $APP_NAMESPACE
stringData:
  AAD_AUTHORITY: https://login.microsoftonline.com/
  AZURE_AUDIENCE: "api://AzureADTokenExchange"
  AZURE_TENANT_ID: "${TENANT_ID}"
  AZURE_CLIENT_ID: "${IDENTITY_CLIENT_ID}"
  BLOB_STORE_ACCOUNT: "${STORAGE_ACCOUNT}"
  BLOB_STORE_CONTAINER: "${STORAGE_CONTAINER}"
EOF
```

11.7.1.7. Deploying the workload application

Deploy the workload application to your cluster to validate the Zero Trust Workload Identity Manager environment. This application confirms that the SPIFFE Workload API is functioning and can successfully retrieve JWT tokens.

Prerequisites

- The demonstration application has been created and deployed.

Procedure

1. To deploy the application, copy the entire command block provided and paste it directly into your terminal. Press **Enter**.

```
$ oc apply -f - << EOF
apiVersion: v1
kind: ServiceAccount
metadata:
  name: $APP_NAME
  namespace: $APP_NAMESPACE
---
kind: Deployment
apiVersion: apps/v1
metadata:
  name: $APP_NAME
  namespace: $APP_NAMESPACE
spec:
  selector:
    matchLabels:
      app: $APP_NAME
  template:
    metadata:
      labels:
        app: $APP_NAME
        deployment: $APP_NAME
    spec:
      serviceAccountName: $APP_NAME
```

```

containers:
  - name: $APP_NAME
    image: "registry.redhat.io/ubi9/python-311:latest"
    command:
      - /bin/bash
      - "-c"
      - |
        #!/bin/bash
        pip install spiffe azure-cli

        cat << EOF > /opt/app-root/src/get-spiffe-token.py
        #!/opt/app-root/bin/python
        from spiffe import JwtSource
        import argparse
        parser = argparse.ArgumentParser(description='Retrieve SPIFFE Token.')
        parser.add_argument("-a", "--audience", help="The audience to include in the
token", required=True)
        args = parser.parse_args()
        with JwtSource() as source:
            jwt_svid = source.fetch_svid(audience={args.audience})
            print(jwt_svid.token)
        EOF

        chmod +x /opt/app-root/src/get-spiffe-token.py
        while true; do sleep 10; done
    envFrom:
      - secretRef:
          name: $APP_NAME
    env:
      - name: SPIFFE_ENDPOINT_SOCKET
        value: unix:///run/spire/sockets/spire-agent.sock
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop:
          - ALL
      readOnlyRootFilesystem: false
      runAsNonRoot: true
      seccompProfile:
        type: RuntimeDefault
    ports:
      - containerPort: 8080
        protocol: TCP
    volumeMounts:
      - name: spiffe-workload-api
        mountPath: /run/spire/sockets
        readOnly: true
    volumes:
      - name: spiffe-workload-api
        csi:
          driver: csi.spiffe.io
          readOnly: true
EOF

```

Verification

1. Ensure that the **workload-app** pod is running successfully by running the following command:

```
$ oc get pods -n $APP_NAMESPACE
```

Example output

```
NAME                                READY   STATUS    RESTARTS   AGE
workload-app-5f8b9d685b-abcde      1/1     Running   0           60s
```

2. Retrieve the SPIFFE JWT Token (SVID-JWT):

- a. Get the pod name dynamically by running the following command:

```
$ POD_NAME=$(oc get pods -n $APP_NAMESPACE -l app=$APP_NAME -o
jsonpath='{.items[0].metadata.name}')
```

- b. Run the script inside the pod by running the following command:

```
$ oc exec -it $POD_NAME -n $APP_NAMESPACE -- \
/opt/app-root/src/get-spiFFE-token.py -a "api://AzureADTokenExchange"
```

11.7.1.8. Configuring Azure with the SPIFFE identity federation

Configure Microsoft Azure with SPIFFE identity federation to enable password-free, automated authentication for the demonstration application. This federates the User Managed Identity with the SPIFFE identity associated with your workload application.

Procedure

- Federate the identities between the User Managed Identity and the SPIFFE identity associated with the workload application by running the following command:

```
$ az identity federated-credential create \
--name ${NAME} \
--identity-name ${USER_ASSIGNED_IDENTITY_NAME} \
--resource-group ${RESOURCE_GROUP} \
--issuer https://$JWT_ISSUER_ENDPOINT \
--subject spiffe://$APP_DOMAIN/ns/$APP_NAMESPACE/sa/$APP_NAME \
--audience api://AzureADTokenExchange
```

11.7.1.9. Verifying that the application workload can access the content in the Azure Blob Storage

Verify that your application workload can connect to the Azure Blob Storage. By uploading a test file, you validate the authentication token and ensure that the workload has the correct permissions.

Prerequisites

- An Azure Blob Storage has been created.

Procedure

1. Retrieve a JWT token from the SPIFFE Workload API by running the following command:

```
$ oc rsh -n $APP_NAMESPACE deployment/$APP_NAME
```

2. Create and export an environment variable named **TOKEN** by running the following command:

```
$ export TOKEN=$(/opt/app-root/src/get-spiffe-token.py --audience=$AZURE_AUDIENCE)
```

3. Log in to Azure CLI included within the pod by running the following command:

```
$ az login --service-principal \
  -t ${AZURE_TENANT_ID} \
  -u ${AZURE_CLIENT_ID} \
  --federated-token ${TOKEN}
```

4. Create a new file with the application workload pod and upload the file to the Blob Storage by running the following command:

```
$ echo "Hello from OpenShift" > openshift-spire-federated-identities.txt
```

5. Upload a file to the Azure Blob Storage by running the following command:

```
$ az storage blob upload \
  --account-name ${BLOB_STORE_ACCOUNT} \
  --container-name ${BLOB_STORE_CONTAINER} \
  --name openshift-spire-federated-identities.txt \
  --file openshift-spire-federated-identities.txt \
  --auth-mode login
```

Verification

- Confirm the file uploaded successfully by listing the files contained by running the following command:

```
$ az storage blob list \
  --account-name ${BLOB_STORE_ACCOUNT} \
  --container-name ${BLOB_STORE_CONTAINER} \
  --auth-mode login \
  -o table
```

11.7.2. About Vault OpenID Connect

Use Vault OpenID Connect (OIDC) with SPIRE to securely authenticate workloads. Vault uses SPIRE as a trusted OIDC provider to validate workload identities. This configuration enables workloads to receive short-lived tokens to access secrets and perform actions within Vault.

11.7.2.1. Installing Vault

Install HashiCorp Vault to serve as an OpenID Connect (OIDC) provider. This establishes the necessary infrastructure to manage workload identities securely in your Zero Trust Workload Identity Manager environment.

Prerequisites

- Configure a route. For more information, see [Configuring routes](#)
- Helm is installed.
- A command-line JSON processor for easily reading the output from the Vault API.
- A HashiCorp Helm repository is added.

Procedure

1. Create the **vault-helm-value.yaml** file.

```
global:
  enabled: true
  openshift: true
  tlsDisable: true
injector:
  enabled: false
server:
  ui:
    enabled: true
  image:
    repository: docker.io/hashicorp/vault
    tag: "1.19.0"
  dataStorage:
    enabled: true
    size: 1Gi
  standalone:
    enabled: true
  config: |
    listener "tcp" {
      tls_disable = 1
      address = "[::]:8200"
      cluster_address = "[::]:8201"
    }
    storage "file" {
      path = "/vault/data"
    }
  extraEnvironmentVars: {}
```

- The **openshift** field optimizes the deployment for OpenShift-specific security contexts.
 - The **tlsDisable** field disables TLS for Kubernetes objects created by the chart.
 - The **datastorage.enabled** field creates a 1Gi persistent volume to store Vault data.
 - The **standalone.enabled** field deploys a single Vault pod.
 - The **tls_disabled** field tells the Vault server to not use TLS.
2. Run the **helm install** command:

```
$ helm install vault hashicorp/vault \
  --create-namespace -n vault \
  --values ./vault-helm-value.yaml
```

3. Expose the Vault service by running the following command:

```
$ oc expose service vault -n vault
```

4. Set the **VAULT_ADDR** environment variable to retrieve the hostname from the new route and then export it by running the following command:

```
$ export VAULT_ADDR="http://$(oc get route vault -n vault -o jsonpath='{.spec.host}')
```



NOTE

http:// is prepended because TLS is disabled.

Verification

- To ensure your Vault instance is running, run the following command:

```
$ curl -s $VAULT_ADDR/v1/sys/health | jq
```

Example output

```
{
  "initialized": true,
  "sealed": true,
  "standby": true,
  "performance_standby": false,
  "replication_performance_mode": "disabled",
  "replication_dr_mode": "disabled",
  "server_time_utc": 1663786574,
  "version": "1.19.0",
  "cluster_name": "vault-cluster-a1b2c3d4",
  "cluster_id": "5e6f7a8b-9c0d-1e2f-3a4b-5c6d7e8f9a0b"
}
```

11.7.2.2. Initializing and unsealing Vault

To prepare a newly installed Vault server for operation, initialize and unseal it. This process loads the primary encryption key into memory so that Vault can decrypt data and protect other encryption keys.

The steps to initialize a Vault server are:

1. Initialize and unseal Vault
2. Enable the key-value (KV) secrets engine and store a test secret
3. Configure JSON Web Token (JWT) authentication with SPIRE
4. Deploy a demonstration application

5. Authenticate and retrieve the secret

Prerequisites

- Ensure that Vault is running.
- Ensure that Vault is not initialized. You can only initialize a Vault server once.

Procedure

1. Open a remote shell into the **vault** pod by running the following command:

```
$ oc rsh -n vault statefulset/vault
```

2. Initialize Vault to get your unseal key and root token by running the following command:

```
$ vault operator init -key-shares=1 -key-threshold=1 -format=json
```

3. Export the unseal key and root token you received from the earlier command by running the following commands:

```
$ export UNSEAL_KEY=<Your-Unseal-Key>
```

```
$ export ROOT_TOKEN=<Your-Root-Token>
```

4. Unseal Vault using your unseal key by running the following command:

```
$ vault operator unseal -format=json $UNSEAL_KEY
```

5. Exit the pod by entering **exit**.

Verification

- To verify that the Vault pod is ready, run the following command:

```
$ oc get pod -n vault
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
vault-0	1/1	Running	0	65d

11.7.2.3. Enabling the key-value secrets engine and store a test secret

Enable the key-value secrets engine to create a secure, centralized location for managing credentials. You can also store a test secret to verify that the engine is working.

Prerequisites

- Make sure that Vault is initialized and unsealed.

Procedure

1. Open another shell session in the **Vault** pod by running the following command:

```
$ oc rsh -n vault statefulset/vault
```

2. Export your root token again within this new session and log in by running the following command:

```
$ export ROOT_TOKEN=<Your-Root-Token>
```

```
$ vault login "${ROOT_TOKEN}"
```

3. Enable the KV secrets engine at the **secret/** path and create a test secret by running the following commands:

```
$ export NAME=ztwim
```

```
$ vault secrets enable -path=secret kv
```

```
$ vault kv put secret/$NAME version=v0.1.0
```

Verification

- To verify that the secret is stored correctly, run the following command:

```
$ vault kv get secret/$NAME
```

11.7.2.4. Configuring JSON Web Token authentication with SPIRE

To help your applications securely log in to Vault using SPIFFE identities, configure JSON Web Token (JWT) authentication.

Prerequisites

- Make sure that Vault is initialized and unsealed.
- Ensure that a test secret is stored in the key-value secrets engine.

Procedure

1. On your local machine, retrieve the SPIRE Certificate Authority (CA) bundle and save it to a file by running the following command:

```
$ oc get cm -n zero-trust-workload-identity-manager spire-bundle -o jsonpath='{.data.bundle\.crt}' > oidc_provider_ca.pem
```

2. Back in the Vault pod shell, create a temporary file and paste the contents of **oidc_provider_ca.pem** into it by running the following command:

```
$ cat << EOF > /tmp/oidc_provider_ca.pem
```

```
-----BEGIN CERTIFICATE-----  
<Paste-Your-Certificate-Content-Here>  
-----END CERTIFICATE-----  
EOF>
```

3. Set up the necessary environment variables for the JWT configuration by running the following commands:

```
$ export APP_DOMAIN=<Your-App-Domain>
```

```
$ export JWT_ISSUER_ENDPOINT="oidc-discovery.$APP_DOMAIN"
```

```
$ export OIDC_URL="https://$JWT_ISSUER_ENDPOINT"
```

```
$ export OIDC_CA_PEM="$(cat /tmp/oidc_provider_ca.pem)"
```

4. Create a new environment variable by running the following command:

```
$ export ROLE="${NAME}-role"
```

5. Enable the JWT authentication method by running the following command:

```
$ vault auth enable jwt
```

6. Configure your OIDC authentication method by running the following command:

```
$ vault write auth/jwt/config \  
  oidc_discovery_url=$OIDC_URL \  
  oidc_discovery_ca_pem="$OIDC_CA_PEM" \  
  default_role=$ROLE
```

7. Create a policy named **ztwim-policy** by running the following command:

```
$ export POLICY="${NAME}-policy"
```

8. Grant read access to the secret you created earlier by running the following command:

```
$ vault policy write $POLICY -<<EOF  
path "secret/$NAME" {  
  capabilities = ["read"]  
}  
EOF
```

9. Create the following environment variables by running the following commands:

```
$ export APP_NAME=client
```

```
$ export APP_NAMESPACE=demo
```

```
$ export AUDIENCE=$APP_NAME
```

10. Create a JWT role that binds the policy to workload with a specific SPIFFE ID by running the following command:

```
$ vault write auth/jwt/role/$ROLE -<<EOF
{
  "role_type": "jwt",
  "user_claim": "sub",
  "bound_audiences": "$AUDIENCE",
  "bound_claims_type": "glob",
  "bound_claims": {
    "sub": "spiffe://$APP_DOMAIN/ns/$APP_NAMESPACE/sa/$APP_NAME"
  },
  "token_ttl": "24h",
  "token_policies": "$POLICY"
}
EOF
```

11.7.2.5. Deploying a demonstration application

Deploy a demonstration application to create a simple client that uses its SPIFFE identity to authenticate with Vault. By doing this you can verify that the client can successfully authenticate using the configured identity.

Procedure

1. On your local machine, set the environment variables for your application by running the following commands:

```
$ export APP_NAME=client
```

```
$ export APP_NAMESPACE=demo
```

```
$ export AUDIENCE=$APP_NAME
```

2. Apply the Kubernetes manifest to create the namespace, service account, and deployment for the demo app by running the following command. This deployment mounts the SPIFFE CSI driver socket.

```
$ oc apply -f - <<EOF
# ... (paste the full YAML from your provided code here) ...
EOF>>
```

Verification

- Verify that the client deployment is ready by running the following command:

```
$ oc get deploy -n $APP_NAMESPACE
```

Example output

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
frontend-app	2/2	2	2	120d
backend-api	3/3	3	3	120d

11.7.2.6. Authenticating and retrieving the secret

Use the demonstration application to fetch a JWT token from the SPIFFE Workload API. Use the token to authenticate with Vault so that you can securely retrieve the secret and verify the workflow.

Procedure

1. Fetch a JWT-SVID by running the following command inside the running client pod:

```
$ oc -n $APP_NAMESPACE exec -it $(oc get pod -o=jsonpath='{.items[*].metadata.name}' -l
app=$APP_NAME -n $APP_NAMESPACE) \
-- /opt/spire/bin/spire-agent api fetch jwt \
-socketPath /run/spire/sockets/spire-agent.sock \
-audience $AUDIENCE
```

2. Copy the token from the output and export it as an environment variable on your local machine by running the following command:

```
$ export IDENTITY_TOKEN=<Your-JWT-Token>
```

3. Create a new environment variable by running the following command:

```
$ export ROLE="${NAME}-role"
```

4. Use **curl** to send the JWT token to the Vault login endpoint to get a Vault client token by running the following command:

```
$ VAULT_TOKEN=$(curl -s --request POST --data '{ "jwt": ""${IDENTITY_TOKEN}""', "role":
""${ROLE}""}' "${VAULT_ADDR}"/v1/auth/jwt/login | jq -r '.auth.client_token')
```

Verification

- Use the newly acquired Vault token to read the secret from the KV store by running the following command:

```
$ curl -s -H "X-Vault-Token: $VAULT_TOKEN" $VAULT_ADDR/v1/secret/$NAME | jq
```

You should see the contents of the secret ("**version**": "**v0.1.0**") in the output, confirming the entire workflow is successful

11.8. ZERO TRUST WORKLOAD IDENTITY MANAGER SPIRE FEDERATION

Configure SPIRE federation to enable workloads in different trust domains to securely authenticate each other across clusters, cloud providers, and organizational boundaries. By establishing trust relationships between separate SPIRE deployments, you can build a zero-trust architecture that spans multiple environments without compromising security or sharing secrets.

Federation works by securely sharing trust bundles between SPIRE servers through dedicated federation endpoints. Each SPIRE deployment maintains its own trust domain and cryptographic identity, while being able to verify identities from federated trust domains. This approach enables cross-cluster communication, multi-cloud deployments, and secure integration with external partners.

Setting up SPIRE federation involves the following high-level steps:

1. Choose an authentication profile: Select either **https_spiffe** or **https_web**.
2. Configure the bundle endpoints: Each cluster exposes its trust bundle through a federation endpoint secured by the chosen authentication profile.
3. Bootstrap the initial trust: Manually fetch and configure the initial trust bundle from each remote cluster.
4. Establish federation relationships: Create **ClusterFederatedTrustDomain** resources to define which clusters trust each other.
5. Configure automatic synchronization: The SPIRE Controller Manager automatically keeps trust bundles synchronized after initial setup.

11.8.1. Understanding bundle endpoint profiles

The bundle endpoint profile determines how your cluster exposes its trust bundle to other SPIRE deployments and how it authenticates remote clusters accessing the bundle. Choose the profile that best matches your security requirements and infrastructure.

The Zero Trust Workload Identity Manager supports two authentication profiles for federation:

https_spiffe

Uses SPIFFE-based TLS authentication. The SPIRE server presents its own SVID (SPIFFE Verifiable Identity Document) to authenticate itself to remote SPIRE servers. This profile provides strong cryptographic identity verification and is ideal for federation between SPIRE deployments.

https_web

Uses standard Web PKI (X.509 certificates from public or private certificate Authorities). This profile supports both automatic certificate management via ACME (Let's Encrypt) and manual certificate management using tools like cert-manager.

The following table summarizes the key differences between the two profiles:

Criteria	https_spiffe	https_web
Authentication method	SPIFFE SVID (TLS)	X.509 certificate from CA
Certificate management	Automatic (SPIRE-managed)	ACME (automatic) or manual
Trust model	SPIFFE trust domain	Web PKI / CA trust
Best for	Internal SPIRE-to-SPIRE federation	External federation, public endpoints
Security level	Very high (cryptographic identity)	High (CA-based trust)

Criteria	https_spiffe	https_web
Setup complexity	Medium (requires SPIFFE IDs)	Low (ACME) to Medium (manual certs)



IMPORTANT

After enablement, federation cannot be disabled. The bundle endpoint profile is immutable once configured. Changing the profile or disabling federation requires reinstallation of the system. However, peer configurations (**federatesWith**) remain dynamic and can be added or removed at any time. Plan your profile selection carefully based on your long-term federation requirements.

11.8.2. Federation configuration examples

The following examples demonstrate different SPIRE federation configurations. Use these as templates when setting up federation between your clusters.

Example 1: Using ACME for automatic certificate management

The following example shows how to configure federation using Let's Encrypt for automatic certificate provisioning and renewal:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_web
      refreshHint: 300
      httpsWeb:
        acme:
          directoryUrl: https://acme-v02.api.letsencrypt.org/directory
          domainName: federation.apps.cluster1.example.com
          email: admin@example.com
          tosAccepted: "true"
    federatesWith:
      - trustDomain: cluster2.example.com
        bundleEndpointUrl: https://federation.apps.cluster2.example.com
        bundleEndpointProfile: https_web
      - trustDomain: cluster3.example.com
        bundleEndpointUrl: https://federation.apps.cluster3.example.com
        bundleEndpointProfile: https_web
  managedRoute: "true"
```

- The **profile** field uses **https_web** profile for Web PKI certificate-based authentication.
- The **directoryURL** field is used for the production directory. For testing, use staging URL <https://acme-staging-v02.api.letsencrypt.org/directory>

Example 2: Using manual certificate management with cert-manager

The following example shows how to configure federation using externally managed certificates:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_web
      refreshHint: 300
      httpsWeb:
        servingCert:
          fileSyncInterval: 86400
          externalSecretRef: spire-server-federation-tls
  federatesWith:
    - trustDomain: cluster2.example.com
      bundleEndpointUrl: https://federation.apps.cluster2.example.com
      bundleEndpointProfile: https_web
    - trustDomain: cluster3.example.com
      bundleEndpointUrl: https://federation.apps.cluster3.example.com
      bundleEndpointProfile: https_web
  managedRoute: "true"
```

- The **fileSyncInterval** field checks for certificate updates every 24 hours.
- The **externalSecretRef** field is the name of the Kubernetes Secret containing **tls.crt** and **tls.key**

Example 3: Using https_spiffe profile for SPIRE-to-SPIRE federation

The following example shows how to configure federation using SPIFFE-based TLS authentication:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_spiffe
      refreshHint: 300
  federatesWith:
    - trustDomain: cluster2.example.com
      bundleEndpointUrl: https://federation.apps.cluster2.example.com
      bundleEndpointProfile: https_spiffe
      endpointSpiffeId: spiffe://cluster2.example.com/spire/server
    - trustDomain: cluster3.example.com
      bundleEndpointUrl: https://federation.apps.cluster3.example.com
      bundleEndpointProfile: https_spiffe
      endpointSpiffeId: spiffe://cluster3.example.com/spire/server
  managedRoute: "true"
```

- The **profile** field uses **https_spiffe** profile for SPIFFE-based TLS authentication.
- The **endpointSiffeld** field contains the SPIFFE ID of the remote SPIRE server, required for identity validation.

Example 4: Mixed federation with multiple authentication profiles

The following example shows a cluster federating with multiple remote clusters using different authentication profiles:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: internal-cluster.example.com
  federation:
    bundleEndpoint:
      profile: https_spiffe
      refreshHint: 300
    federatesWith:
      # Internal cluster using SPIFFE TLS
      - trustDomain: dev-cluster.example.com
        bundleEndpointUrl: https://federation.apps.dev-cluster.example.com
        bundleEndpointProfile: https_spiffe
        endpointSpiffeld: spiffe://dev-cluster.example.com/spire/server
      # External partner using Web PKI
      - trustDomain: partner.example.com
        bundleEndpointUrl: https://federation.partner.example.com
        bundleEndpointProfile: https_web
      # Another external partner using Web PKI
      - trustDomain: vendor.example.com
        bundleEndpointUrl: https://spire-federation.vendor.example.com
        bundleEndpointProfile: https_web
  managedRoute: "true"
```

- The **profile** field cluster exposes its bundle using **https_spiffe** profile.
- The **bundleEndpointProfile** field cluster exposes its bundle using **https_spiffe** profile.

11.8.3. Configuring SPIRE federation with the https_spiffe profile

The Zero Trust Workload Identity Manager includes SPIRE Federation support, allowing multiple independent SPIRE deployments to establish trust relationships. This procedure demonstrates how to configure federation using the **https_spiffe** profile, which uses SPIFFE-based TLS authentication between SPIRE servers.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have installed the Zero Trust Workload Identity Manager on all clusters that will participate in the federation.
- You have **cluster-admin** privileges on all participating clusters.

- You have network connectivity between the clusters you intend to federate.

Procedure

1. Configure the **SpireServer** custom resource on each cluster to enable federation with the **https_spiffe** profile. The **https_spiffe** profile uses SPIFFE-based TLS authentication, where SPIRE servers authenticate to each other using their own SPIFFE Verifiable Identity Documents (SVIDs).

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_spiffe
      refreshHint: 300
    managedRoute: "true"
```

- The **spec.trustDomain** field sets a unique trust domain for each cluster.
 - The **spec.federation.bundleEndpoint.profile** field uses the **https_spiffe** profile for SPIFFE-based TLS authentication.
 - The **spec.federation.bundleEndpoint.refreshHint** field suggests intervals (in seconds) for remote servers to refresh the trust bundle. Range: 60–3600 seconds.
 - The **spec.federation.managedRoute** field enables automatic route creation by the Operator.
2. Apply the configuration changes by running the following command:

```
$ oc apply -f spire-server.yaml
```

3. Check the status of the SPIRE Server by entering the following command. Wait for the **Ready** status to be returned.

```
$ oc get spireserver cluster -w
```

4. Verify that the federation route has been created:

```
$ oc get route -n zero-trust-workload-identity-manager | grep federation
```

Example output

NAME	HOST/PORT	PATH	SERVICES	PORT
TERMINATION				
spire-server-federation	federation.apps.cluster1.example.com		spire-server	8443
passthrough				

5. Fetch the trust bundle from each remote cluster's federation endpoint:

■

```
$ curl -k https://federation.apps.cluster2.example.com > cluster2-bundle.json
```



NOTE

For **https_spiffe** profile, you might need to use the **-k** flag if the certificate is not trusted by your system's CA bundle:

The response contains the trust bundle in JSON Web Key Set (JWKS) format:

Example trust bundle

```
{
  "keys": [
    {
      "use": "x509-svid",
      "kty": "RSA",
      "n": "...",
      "e": "AQAB",
      "x5c": ["..."]
    }
  ],
  "spiffe_sequence": 1,
  "refresh_hint": 300
}
```

6. Create **ClusterFederatedTrustDomain** resources for each remote trust domain.
 - a. On Cluster 1, create a resource to federate with Cluster 2:

```
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: federation-to-cluster2
spec:
  trustDomain: <CLUSTER2_APPS_DOMAIN>
  bundleEndpointURL: https://federation.<CLUSTER2_APPS_DOMAIN>
  bundleEndpointProfile:
    type: https_spiffe
    endpointSPIFFEID: spiffe://<CLUSTER2_APPS_DOMAIN>/spire/server
  className: zero-trust-workload-identity-manager-spire
  trustDomainBundle: |
    {
      "keys": [
        {
          "use": "x509-svid",
          "kty": "RSA",
          "n": "...",
          "e": "AQAB",
          "x5c": ["..."]
        }
      ],
      "spiffe_sequence": 1
    }
```

where

<CLUSTER2_APPS_DOMAIN>

Specifies the trust domain of the external cluster you are federating with.

spec.bundleEndpointProfile.endpointSPIFFEID

Specifies the SPIFFE ID of the remote SPIRE server. Required for **https_spiffe** profile to validate the remote server's identity.

spec.trustDomainBundle

Specifies the complete trust bundle JSON that you fetched in the previous step.

spec.className

Specifies the name of a class to watch CRs for. Spire-controller-manager watches the resource only if **spec.className** is set to **zero-trust-workload-identity-manager-spire**.

7. Apply the **ClusterFederatedTrustDomain** resource by running the following command:

```
$ oc apply -f clusterfederatedtrustdomain.yaml
```

8. Repeat steps 5-7 on each cluster for every remote cluster it should federate with. For bidirectional federation, each cluster needs a **ClusterFederatedTrustDomain** resource for every other cluster.

9. Update the **SpireServer** resource on each cluster to add the **federatesWith** configuration:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_spiffe
      refreshHint: 300
    federatesWith:
      - trustDomain: cluster2.example.com
        bundleEndpointUrl: https://federation.apps.cluster2.example.com
        bundleEndpointProfile: https_spiffe
        endpointSpiffeld: spiffe://cluster2.example.com/spire/server
      - trustDomain: cluster3.example.com
        bundleEndpointUrl: https://federation.apps.cluster3.example.com
        bundleEndpointProfile: https_spiffe
        endpointSpiffeld: spiffe://cluster3.example.com/spire/server
    managedRoute: "true"
```

- The **spec.federation.federatesWith** field lists all remote trust domains this cluster should federate with.

10. Apply the updated configuration by running the following command:

```
$ oc apply -f spireserver.yaml
```

Verification

1. Verify that the **ClusterFederatedTrustDomain** resources have been created by running the following command:

```
$ oc get clusterfederatedtrustdomains
```

Example output

NAME	TRUST DOMAIN	ENDPOINT URL	AGE
cluster2-federation	cluster2.example.com	https://federation.apps.cluster2.example.com	5m
cluster3-federation	cluster3.example.com	https://federation.apps.cluster3.example.com	5m

2. Check the status of a **ClusterFederatedTrustDomain** to ensure bundle synchronization is working by running the following command:

```
$ oc describe clusterfederatedtrustdomain cluster2-federation
```

Look for successful status conditions indicating that the trust bundle has been synchronized.

3. Verify that the federation endpoint is accessible by running the following command:

```
$ curl https://federation.apps.cluster1.example.com
```

You should receive a JSON response containing the trust bundle.

4. Check the SPIRE Server logs to confirm federation is active by running the following command:

```
$ oc logs -n zero-trust-workload-identity-manager \
statefulset/spire-server -c spire-server --tail=50
```

Look for log messages indicating successful bundle synchronization with federated trust domains.

11.8.4. Using SPIRE federation with Automatic Certificate Management Environment protocol

Using SPIRE federation with Automatic Certificate Management Environment (ACME) protocol provides automatic certificate provisioning from Let's Encrypt. ACME also enables automatic certificate renewal before expiration, eliminating manual certificate management overhead.

Prerequisites

- You have installed the Zero Trust Workload Identity Manager on all clusters that will participate in the federation.
- You have installed the OpenShift CLI (**oc**).
- You have **cluster-admin** privileges on all participating clusters.
- Your federation endpoints must be publicly accessible for Let's Encrypt HTTP-01 challenge validation.

- You have network connectivity between all federated clusters.

Procedure

1. Configure the **SpireServer** custom resource on each cluster to enable federation with ACME certificate management.

Create or update your **SpireServer** resource with the federation configuration:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_web
      refreshHint: 300
    httpsWeb:
      acme:
        directoryUrl: https://acme-v02.api.letsencrypt.org/directory
        domainName: federation.apps.cluster1.example.com
        email: admin@example.com
        tosAccepted: "true"
      managedRoute: "true"
```

- The **spec.trustDomain** field sets a unique trust domain for each cluster (for example, **cluster1.example.com**, **cluster2.example.com**).
 - The **spec.federation.bundleEndpoint.profile** field uses the **https_web** profile for ACME-based certificate management.
 - The **spec.federation.bundleEndpoint.httpsWeb.acme.directoryUrl** field contains the Let's Encrypt production directory URL. For testing, use: <https://acme-staging-v02.api.letsencrypt.org/directory>.
 - The **spec.federation.bundleEndpoint.httpsWeb.acme.domainName** field is the domain name where your federation endpoint is accessible. This automatically sets to **federation.<cluster-apps-domain>** if **managedRoute** is set to "true".
 - The **spec.federation.bundleEndpoint.httpsWeb.acme.email** field is your email address for ACME account registration and certificate expiration notifications.
 - The **spec.federation.bundleEndpoint.httpsWeb.acme.tosAccepted** field accepts the Let's Encrypt Terms of Service.
 - The **spec.federation.managedRoute** field enables an automatic route creation by the operator for the federation bundle endpoint.
2. Apply the configuration to each cluster by running the following command:

```
$ oc apply -f spireserver.yaml
```

3. Check the status of the SPIRE Server by entering the following command. Wait for the **Ready** status to be returned before proceeding to the next step.

```
$ oc get spireserver cluster -w
```

Example output

```
NAME      STATUS AGE
cluster   Ready  5m
```

4. Verify that the federation route has been created by running the following command:

```
$ oc get route -n zero-trust-workload-identity-manager | grep federation
```

Example output

NAME	HOST/PORT	PATH	SERVICES	PORT
spire-server-federation-8443	federation.apps.cluster1.example.com		spire-server	
	passthrough			

5. On each cluster, fetch the trust bundle from the federation endpoint by running the following command:

```
$ curl https://federation.apps.cluster1.example.com > cluster1-bundle.json
```

The response contains the trust bundle in JSON Web Key Set (JWKS) format:

Example trust bundle

```
{
  "keys": [
    {
      "use": "x509-svid",
      "kty": "RSA",
      "n": "...",
      "e": "AQAB",
      "x5c": ["..."]
    }
  ],
  "spiffe_sequence": 1,
  "refresh_hint": 300
}
```

6. Create **ClusterFederatedTrustDomain** resources to establish federation relationships.
 - a. On Cluster 1, create resources to federate with Cluster 2 and Cluster 3:

```
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: cluster2-federation
spec:
```

```

trustDomain: cluster2.example.com
bundleEndpointURL: https://federation.apps.cluster2.example.com
bundleEndpointProfile:
  type: https_web
className: zero-trust-workload-identity-manager-spire
trustDomainBundle: |
  {
    "keys": [...],
    "spiffe_sequence": 1
  }
---
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: cluster3-federation
spec:
  trustDomain: cluster3.example.com
  bundleEndpointURL: https://federation.apps.cluster3.example.com
  bundleEndpointProfile:
    type: https_web
  className: zero-trust-workload-identity-manager-spire
  trustDomainBundle: |
    {
      "keys": [...],
      "spiffe_sequence": 1
    }

```

- The **spec.trustDomainBundle** field contains the complete trust bundle JSON that you fetched using **curl** in step 5.
- The **spec.className** field contains the name of a class to watch CRs for. Spire-controller-manager watches the resource only if **spec.className** is set to **zero-trust-workload-identity-manager-spire**.

7. Apply the **ClusterFederatedTrustDomain** resources by running the following command:

```
$ oc apply -f cluster-federated-trust-domains.yaml
```

8. Repeat steps 6 and 7 on each cluster to establish bidirectional federation. Each cluster needs **ClusterFederatedTrustDomain** resources for every other cluster it federates with.
9. Update the **SpireServer** resource on each cluster to add the **federatesWith** configuration:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  # ... existing configuration ...
  federation:
    bundleEndpoint:
      # ... existing bundleEndpoint configuration ...
    federatesWith:
      - trustDomain: cluster2.example.com
        bundleEndpointUrl: https://federation.apps.cluster2.example.com

```

```

    bundleEndpointProfile: https_web
  - trustDomain: cluster3.example.com
    bundleEndpointUrl: https://federation.apps.cluster3.example.com
    bundleEndpointProfile: https_web
  managedRoute: "true"

```

- The **spec.federation.federatesWith** field lists all remote trust domains this cluster should federate with.

10. Apply the updated configuration by running the following command:

```
$ oc apply -f spireserver.yaml
```

Verification

1. Verify that the **ClusterFederatedTrustDomain** resources have been created by running the following command:

```
$ oc get clusterfederatedtrustdomains
```

Example output

```

NAME                TRUST DOMAIN      ENDPOINT URL                                     AGE
cluster2-federation cluster2.example.com https://federation.apps.cluster2.example.com 5m
cluster3-federation cluster3.example.com https://federation.apps.cluster3.example.com 5m

```

2. Check the status of a **ClusterFederatedTrustDomain** to ensure bundle synchronization is working by running the following command:

```
$ oc describe clusterfederatedtrustdomain cluster2-federation
```

Look for **Successful** status conditions indicating that the trust bundle has been synchronized.

3. Verify that the federation endpoint is accessible and serving the trust bundle by running the following command:

```
$ curl https://federation.apps.cluster1.example.com
```

You should receive a JSON response containing the trust bundle.

4. Check the SPIRE Server logs to confirm federation is active by running the following command:

```
$ oc logs -n zero-trust-workload-identity-manager deployment/spire-server -c spire-server --tail=50
```

Look for log messages indicating successful bundle synchronization with federated trust domains.

5. Verify that all SPIRE components are running by running the following command:

```
$ oc get pods -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
spire-agent-abcde	1/1	Running	0	10m
spire-server-0	2/2	Running	0	10m

- Optional: Test cross-cluster workload authentication by deploying workloads with SPIFFE identities on different clusters and verifying they can authenticate to each other using the federated trust.

11.8.5. Using SPIRE federation with manual certificate management

You can use SPIRE federation with custom certificate management using cert-manager or other certificate providers. This approach provides flexibility for organizations that require control over certificate issuance, support for internal certificate authorities (CAs), or integration with existing certificate management infrastructure.

Prerequisites

- You have installed the Zero Trust Workload Identity Manager on all clusters that will participate in the federation.
- You have installed the OpenShift CLI (**oc**).
- You have **cluster-admin** privileges on all participating clusters.
- You have installed the cert-manager Operator for Red Hat OpenShift. For more information, see [cert-manager Operator for Red Hat OpenShift](#).
- Your federation endpoints must be publicly accessible for certificate validation.
- You have network connectivity between all federated clusters.

Procedure

- Install the cert-manager Operator on the cluster where you want to use externally managed certificates.

Create a namespace and install the operator:

```
apiVersion: v1
kind: Namespace
metadata:
  name: cert-manager-operator
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-cert-manager-operator
  namespace: cert-manager-operator
spec:
  upgradeStrategy: Default
---
apiVersion: operators.coreos.com/stable-v1
kind: Subscription
metadata:
```

```
name: openshift-cert-manager-operator
namespace: cert-manager-operator
spec:
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  name: openshift-cert-manager-operator
  channel: stable-v1
```

2. Apply the cert-manager installation by running the following command:

```
$ oc apply -f cert-manager-install.yaml
```

3. Check the status of the cert-manager Operator by entering the following command:

```
$ oc get pods -n cert-manager
```

All cert-manager pods should be in **Running** status.

4. Create an Issuer for certificate provisioning.
For Let's Encrypt with HTTP-01 challenge:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: letsencrypt-http01
  namespace: zero-trust-workload-identity-manager
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-account-key
    solvers:
      - http01:
          ingress:
            ingressClassName: openshift-default
```

Alternatively, for an internal CA:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: internal-ca
  namespace: zero-trust-workload-identity-manager
spec:
  ca:
    secretName: internal-ca-key-pair
```

5. Apply the Issuer by running the following command:

```
$ oc apply -f issuer.yaml
```

6. Determine the federation endpoint domain name.
The federation route follows a predictable naming pattern if **managedRoute** is set to **true**. Get your cluster's application domain by running the following command:

```
$ CLUSTER_DOMAIN=$(oc get ingresses.config/cluster -o jsonpath='{.spec.domain}')
$ FEDERATION_DOMAIN="federation.${CLUSTER_DOMAIN}"
$ echo "Federation domain will be: $FEDERATION_DOMAIN"
```

Example output

```
Federation domain will be: federation.apps.cluster1.example.com
```



NOTE

The federation route is created automatically if **managedRoute** is set to **true** when you apply the **SpireServer** configuration in a later step. The route name is **spire-server-federation** and the hostname is **federation.<cluster-apps-domain>**.

7. Create a Certificate resource to request a TLS certificate.
Use the federation domain determined in the previous step:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: spire-server-federation-tls
  namespace: zero-trust-workload-identity-manager
spec:
  secretName: spire-server-federation-tls
  duration: 2160h
  renewBefore: 360h
  commonName: federation.apps.cluster1.example.com
  dnsNames:
    - federation.apps.cluster1.example.com
  usages:
    - server auth
    - digital signature
    - key encipherment
  issuerRef:
    kind: Issuer
    name: letsencrypt-http01
```

- The **secretName** field must match the **externalSecretRef** value in SpireServer.
- The **duration** field shows how long a certificate is valid. Certificates are valid for 90 days.
- The **renewBefore** field shows how many days a certificate must be renewed before it expires. Renew a certificate 15 days before expiration.
- The **commonName** field must be replaced with your actual federation domain from the previous step.
- The **dnsNames** field must match the **commonName** and the actual route **hostname** that was created.
- The **name** field must reference the Issuer that was created earlier.

8. Apply the Certificate resource by running the following command:

```
$ oc apply -f certificate.yaml
```

9. Monitor the certificate issuance by running the following command:

```
$ oc get certificate spire-server-federation-tls \
  -n zero-trust-workload-identity-manager -w
```

Example output when ready

NAME	READY	SECRET	AGE
spire-server-federation-tls	True	spire-server-federation-tls	2m

10. Create RBAC permissions for the OpenShift Ingress Router to access the certificate secret. Create a Role by running the following command:

```
$ oc create role secret-reader \
  --verb=get,list,watch \
  --resource=secrets \
  --resource-name=spire-server-federation-tls \
  -n zero-trust-workload-identity-manager
```

Create a **RoleBinding** by running the following command:

```
$ oc create rolebinding secret-reader-binding \
  --role=secret-reader \
  --serviceaccount=openshift-ingress:router \
  -n zero-trust-workload-identity-manager
```

11. Configure the **SpireServer** custom resource to use manual certificate management. Now that the certificate is ready, configure the SpireServer to reference it:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster1.example.com
  federation:
    bundleEndpoint:
      profile: https_web
      refreshHint: 300
    httpsWeb:
      servingCert:
        fileSyncInterval: 86400
        externalSecretRef: spire-server-federation-tls
      managedRoute: "true"
```

- The **profile** field must use **https_web** profile for certificate-based authentication.
- The **fileSyncInterval** field checks for certificate updates every 24 hours (86400 seconds). Range: 3600-7776000 seconds.

- The **externalSecretRef** field is the name of the secret containing the TLS certificate and private key. Must match the certificate secret created in the previous steps.

12. Apply the configuration by running the following command:

```
$ oc apply -f spireserver.yaml
```

13. Wait for the SPIRE Server to be ready:

```
$ oc get spireserver cluster -n zero-trust-workload-identity-manager -w
```

Wait until the status shows **Ready**.

14. Verify that the federation route was created by running the following command:

```
$ oc get route spire-server-federation -n zero-trust-workload-identity-manager
```

Example output

NAME	HOST/PORT	PATH	SERVICES	PORT
spire-server-federation	federation.apps.cluster1.example.com		spire-server	8443
reencrypt				

Verify that the route hostname matches the domain name used in your certificate.

15. Verify that the federation endpoint is accessible by running the following command:

```
$ curl https://$(oc get route spire-server-federation \
-n zero-trust-workload-identity-manager \
-o jsonpath='{.spec.host}')
```

You should receive a JSON response containing the trust bundle.

16. Fetch the trust bundle from each federation endpoint that you want to federate with. For each remote cluster, fetch its trust bundle by running the following commands:

```
$ curl https://federation.apps.cluster1.example.com > cluster1-bundle.json
$ curl https://federation.apps.cluster2.example.com > cluster2-bundle.json
```

The trust bundle is in JSON Web Key Set (JWKS) format:

Example trust bundle

```
{
  "keys": [
    {
      "use": "x509-svid",
      "kty": "RSA",
      "n": "xGOzB...",
      "e": "AQAB",
      "x5c": ["MIIC..."]
    }
  ],
}
```

```

    "spiffe_sequence": 1,
    "refresh_hint": 300
  }

```

17. Create **ClusterFederatedTrustDomain** resources for each remote trust domain you want to federate with:

```

apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: cluster1-federation
spec:
  trustDomain: cluster1.example.com
  bundleEndpointURL: https://federation.apps.cluster1.example.com
  bundleEndpointProfile:
    type: https_web
  className: zero-trust-workload-identity-manager-spire
  trustDomainBundle: |
    {
      "keys": [
        {
          "use": "x509-svid",
          "kty": "RSA",
          "n": "xGOzB...",
          "e": "AQAB",
          "x5c": ["MIIC..."]
        }
      ],
      "spiffe_sequence": 1
    }
---
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: cluster2-federation
spec:
  trustDomain: cluster2.example.com
  bundleEndpointURL: https://federation.apps.cluster2.example.com
  bundleEndpointProfile:
    type: https_web
  className: zero-trust-workload-identity-manager-spire
  trustDomainBundle: |
    {
      "keys": [...],
      "spiffe_sequence": 1
    }

```

- The **trustDomainBundle** field contains the complete trust bundle JSON that you fetched in the previous step.
- The **spec.className** field contains the name of a class to watch CRs for. Spire-controller-manager watches the resource only if **spec.className** is set to **zero-trust-workload-identity-manager-spire**.

18. Apply the **ClusterFederatedTrustDomain** resources by running the following command:

```
$ oc apply -f clusterfederatedtrustdomains.yaml
```

19. Update the **SpireServer** resource to add the **federatesWith** configuration:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  trustDomain: cluster3.example.com
  federation:
    bundleEndpoint:
      profile: https_web
      refreshHint: 300
    httpsWeb:
      servingCert:
        fileSyncInterval: 86400
        externalSecretRef: spire-server-federation-tls
  federatesWith:
    - trustDomain: cluster1.example.com
      bundleEndpointUrl: https://federation.apps.cluster1.example.com
      bundleEndpointProfile: https_web

    - trustDomain: cluster2.example.com
      bundleEndpointUrl: https://federation.apps.cluster2.example.com
      bundleEndpointProfile: https_web
  managedRoute: "true"
```

- The **federatesWith** field lists all remote trust domains this cluster should federate with.

20. Apply the updated configuration by running the following command:

```
$ oc apply -f spireserver.yaml
```

21. Repeat steps 1-15 on each cluster that participates in the federation, ensuring that:

- Each cluster has cert-manager installed and configured
- Each cluster has its own certificate created and ready before applying the **SpireServer** configuration
- Each cluster has the RBAC for the ingress router configured
- Each cluster has **ClusterFederatedTrustDomain** resources for every other cluster it federates with
- Each cluster's **SpireServer** has the complete **federatesWith** list

Verification

1. Verify that the certificate has been issued successfully by running the following command:

```
$ oc get certificate spire-server-federation-tls \
  -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	SECRET	AGE
spire-server-federation-tls	True	spire-server-federation-tls	5m

2. Check the certificate details and expiration by running the following command:

```
$ oc get secret spire-server-federation-tls \
  -n zero-trust-workload-identity-manager \
  -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509 -noout -dates
```

Example output

```
notBefore=Dec 16 10:00:00 2025 GMT
notAfter=Mar 16 10:00:00 2026 GMT
```

3. Verify that the RBAC permissions are configured correctly by running the following command:

```
$ oc get role,rolebinding -n zero-trust-workload-identity-manager \
  | grep secret-reader
```

Example output

```
role.rbac.authorization.k8s.io/secret-reader
rolebinding.rbac.authorization.k8s.io/secret-reader-binding
```

Verify the RoleBinding references the correct ServiceAccount by running the following command:

```
$ oc describe rolebinding secret-reader-binding \
  -n zero-trust-workload-identity-manager
```

Example output

```
Name:      secret-reader-binding
Namespace: zero-trust-workload-identity-manager
Role:
  Kind: Role
  Name: secret-reader
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount router openshift-ingress
```

4. Verify that the **ClusterFederatedTrustDomain** resources have been created by running the following command:

```
$ oc get clusterfederatedtrustdomains
```

Example output

NAME	TRUST DOMAIN	ENDPOINT URL	AGE
cluster1-federation	cluster1.example.com	https://federation.apps.cluster1.example.com	5m
cluster2-federation	cluster2.example.com	https://federation.apps.cluster2.example.com	5m

- Check the status of a **ClusterFederatedTrustDomain** to ensure bundle synchronization is working by running the following command:

```
$ oc describe clusterfederatedtrustdomain cluster1-federation
```

Look for successful status conditions indicating that the trust bundle has been synchronized.

- Verify that the federation endpoint is accessible and using the correct certificate by running the following command:

```
$ curl -v https://$(oc get route spire-server-federation \
  -n zero-trust-workload-identity-manager \
  -o jsonpath='{.spec.host}')
```

In the output, verify that the certificate presented is issued by your configured CA (Let's Encrypt or internal CA).

- Check the SPIRE Server logs to confirm that by running the following command:

- Federation is active with remote trust domains
- Trust bundles are being synchronized
- The bundle endpoint is serving correctly

```
$ oc logs -n zero-trust-workload-identity-manager \
  statefulset/spire-server -c spire-server --tail=100
```

Look for log messages indicating successful federation bundle synchronization.

- Verify that all SPIRE components are running by running the following command:

```
$ oc get pods -n zero-trust-workload-identity-manager
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
spire-agent-abc123	1/1	Running	0	10m
spire-server-0	2/2	Running	0	10m

- Optional: Test cross-cluster workload authentication by deploying workloads with SPIFFE identities on different clusters and verifying they can authenticate to each other using the federated trust.

11.8.6. Federation configuration field reference

This reference provides detailed information about all configuration fields available for SPIRE federation in the **SpireServer** custom resource. Use this reference when customizing your federation setup.

Top-level federation fields

Field	Type	Required	Default	Description
federation.bundleEndpoint	object	Yes	N/A	Configuration for this cluster's federation endpoint that exposes the trust bundle to remote clusters.
federation.federatesWith	array	No	<code>[]</code>	List of remote trust domains to federate with.
federation.managedRoute	string	No	"true"	Enable or disable automatic OpenShift Route creation. Set to "true" for operator-managed routes or "false" for manual route management.

bundleEndpoint configuration fields

Field	Type	Required	Default	Description
federation.bundleEndpoint.profile	string (enum)	Yes	https_spiffe	Authentication profile for the bundle endpoint. Valid values: https_spiffe or https_web . This value is immutable after initial configuration.
federation.bundleEndpoint.refreshHint	integer	No	300	Suggested interval (in seconds) for remote servers to refresh the trust bundle. Valid range: 60–3600.
federation.bundleEndpoint.httpsWeb	object	Conditional	N/A	Required when profile is https_web . Contains certificate configuration.

httpsWeb configuration fields

Field	Type	Required	Default	Description
federation.bundleEndpoint.httpsWeb.acme	object	Conditional	N/A	ACME configuration for automatic certificate management. Mutually exclusive with servingCert .

Field	Type	Required	Default	Description
federation.bundleEndpoint.httpsWeb.servingCert	object	Conditional	N/A	Manual certificate configuration. Mutually exclusive with acme .

ACME configuration fields

Field	Type	Required	Default	Description
federation.bundleEndpoint.httpsWeb.acme.directoryUrl	string	Yes	N/A	ACME directory URL. For Let's Encrypt production: https://acme-v02.api.letsencrypt.org/directory . For staging: https://acme-staging-v02.api.letsencrypt.org/directory
federation.bundleEndpoint.httpsWeb.acme.domainName	string	Yes	N/A	Fully qualified domain name for the certificate. Typically the federation endpoint hostname.
federation.bundleEndpoint.httpsWeb.acme.email	string	Yes	N/A	Email address for ACME account registration and certificate expiration notifications.
federation.bundleEndpoint.httpsWeb.acme.tosAccepted	string	No	"false"	Accept the ACME provider's Terms of Service. Must be "true" to obtain certificates.

servingCert configuration fields

Field	Type	Required	Default	Description
federation.bundleEndpoint.httpsWeb.servingCert.fileSyncInterval	integer	No	86400	Interval (in seconds) to check for certificate updates. Valid range: 3600-7776000 (1 hour to 90 days).
federation.bundleEndpoint.httpsWeb.servingCert.externalSecretRef	string	Yes	N/A	Name of the Kubernetes Secret containing the TLS certificate (tls.crt) and private key (tls.key) for the federation route.

federatesWith configuration fields

Field	Type	Required	Default	Description
federation.federatesWith[].trustDomain	string	Yes	N/A	Trust domain name of the remote SPIRE deployment (for example, cluster2.example.com).
federation.federatesWith[].bundleEndpointUrl	string	Yes	N/A	HTTPS URL of the remote federation endpoint (for example, https://federation.apps.cluster2.example.com).
federation.federatesWith[].bundleEndpointProfile	string (enum)	Yes	N/A	Authentication profile of the remote endpoint. Valid values: https_spiffe or https_web .
federation.federatesWith[].endpointSpiffeId	string	Conditional	N/A	SPIFFE ID of the remote SPIRE server (for example, spiffe://cluster2.example.com/spire/server). Required when bundleEndpointProfile is https_spiffe .

Field validation rules

The following validation rules are enforced by the operator:

- Profile immutability: The **bundleEndpoint.profile** field cannot be changed after initial configuration. Changing it requires deleting and recreating the **SpireServer** resource (re-installation of the system).
- Mutual exclusivity: Within **httpsWeb**, only one of **acme** or **servingCert** can be specified.
- Conditional requirements: When **profile** is **https_web**, the **httpsWeb** object must be present with either **acme** or **servingCert** configured.

- SPIFFE ID requirement: When **bundleEndpointProfile** is **https_spiffe** in the **federatesWith** list, the **endpointSpiffeld** field is required.
- Array limits: The **federatesWith** array supports a maximum of 50 entries.
- Numeric ranges:
 - **refreshHint**: 60–3600 seconds
 - **fileSyncInterval**: 3600–7776000 seconds

11.9. USING THE SPIFFE HELPER CONTAINER IMAGE

SPIFFE Helper writes Transport Layer Security (TLS) certificates to disk for applications that cannot use the SPIFFE Workload API. Use it to eliminate manual certificate management and reduce the risk of expired certificates.

11.9.1. SPIFFE Helper container image

Use SPIFFE Helper with the Zero Trust Workload Identity Manager when your application cannot call the workload API directly but can read TLS certificates from a shared volume.

SPIFFE Helper is a utility that connects to the SPIFFE Workload API, fetches identity credentials, writes them to files on disk, and optionally notifies a workload when X.509 material is renewed.

The Zero Trust Workload Identity Manager provides a supported SPIFFE Helper container image based on the upstream SPIFFE Helper. The configuration file format, command-line flags, and workload API behavior are compatible with upstream.

SPIFFE Helper performs the following tasks:

1. Connects to the SPIFFE Workload API, usually through the SPIFFE Runtime Environment Agent socket exposed by the SPIFFE CSI driver.
2. Fetches X.509 SPIFFE Verifiable Identity Document (SVID)s, JSON Web Token (JWT) SVIDs, and JWT bundles from SPIFFE Runtime Environment.
3. Writes credentials to files under a configured directory such as **cert_dir**.
4. Optionally notifies a workload when X.509 authentication credentials are renewed in daemon mode.

11.9.2. SPIFFE Helper modes

SPIFFE Helper fetches credentials from the SPIFFE Workload API and writes them to disk for workloads that cannot call the API directly. On OpenShift Container Platform, a pod typically uses non-daemon mode in an init container for initial certificates and daemon mode in a sidecar to rotate them before expiring.

SPIFFE Helper runs in one of two modes:

- **Non-daemon mode** (**-daemon-mode=false** or **daemon_mode = false**): Fetches credentials once, writes files, and exits. Use this mode in an init container to bootstrap TLS before the main application starts. In this mode, **cmd** and **renew_signal** are ignored.

- **Daemon mode** (default): Stays running until the pod is terminated or an unrecoverable error occurs. Watches the Workload API and renews credentials before they expire. Supports **cmd**, **pid_file_name**, **renew_signal**, and optional HTTP health checks. SPIFFE Helper does not background itself like a traditional Unix daemon.

On OpenShift Container Platform, a typical pod uses both modes:

- An init container runs SPIFFE Helper in non-daemon mode to populate a shared **emptyDir** volume with initial TLS material.
- A sidecar container runs SPIFFE Helper in daemon mode to rotate certificates before they expire.
- The main application container mounts the shared certificate directory and uses the files for TLS.

The SPIFFE workload API is exposed through the SPIFFE container storage interface (CSI), which mounts the Workload API socket into the pod at a path such as **/spiffe-workload-api/spire-agent.sock**.

The following deployment excerpt shows the init container and sidecar pattern on OpenShift Container Platform:

```
initContainers:
- name: spiffe-helper-init
  image: registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>
  args:
  - '-config'
  - '/etc/spiffe-helper/helper.conf'
  - '-daemon-mode=false'
  volumeMounts:
  - name: spiffe-workload-api
    readOnly: true
    mountPath: /spiffe-workload-api
  - name: postgresql-certs
    mountPath: /opt/postgresql-certs
  - name: spiffe-helper
    mountPath: /etc/spiffe-helper
containers:
- name: spiffe-helper
  image: registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>
  args:
  - '-config'
  - '/etc/spiffe-helper/helper.conf'
  volumeMounts:
  - name: spiffe-workload-api
    readOnly: true
    mountPath: /spiffe-workload-api
  - name: postgresql-certs
    mountPath: /opt/postgresql-certs
  - name: spiffe-helper
    mountPath: /etc/spiffe-helper
volumes:
- name: spiffe-workload-api
  csi:
    driver: csi.spiffe.io
    readOnly: true
```

```
- name: postgresql-certs
  emptyDir:
    medium: Memory
- name: spiffe-helper
  configMap:
    name: spiffe-helper
```

Replace **<version>** with the tag that matches your Zero Trust Workload Identity Manager installation.

11.9.3. SPIFFE Helper credential types

SPIFFE Helper supports X.509 SPIFFE Verifiable Identity Document (SVID), JSON Web Token (JWT) bundle, and JWT SVID outputs configured in the SPIFFE Helper configuration file and written under **cert_dir** for workloads that cannot use the workload API directly.

At least one complete set must be specified:

- **X.509 SVID** – Requires **svid_file_name**, **svid_key_file_name**, and **svid_bundle_file_name**. SPIFFE Helper writes Privacy-Enhanced Mail (PEM) certificate, PEM private key, and PEM trust bundle files under **cert_dir**.
- **JWT bundle** – Requires **jwt_bundle_file_name**. SPIFFE Helper writes a JSON bundle file.
- **JWT SVIDs** – Requires one or more **jwt_svids** blocks with **jwt_audience**, **jwt_svid_file_name**, and related settings. SPIFFE Helper writes base64-encoded token files.

The **cert_dir** directory must exist before SPIFFE Helper starts.

The following **helper.conf** excerpt configures X.509 SVID output for a PostgreSQL server:

```
agent_address = "/spiffe-workload-api/spire-agent.sock"
cert_dir = "/opt/postgresql-certs"
svid_file_name = "svid.pem"
svid_key_file_name = "svid.key"
svid_bundle_file_name = "svid_bundle.pem"
```

The following **helper.conf** excerpt configures JWT SVID output:

```
agent_address = "/spiffe-workload-api/spire-agent.sock"
cert_dir = "/opt/jwt-certs"
jwt_svids = [{
  jwt_audience = "your-audience"
  jwt_svid_file_name = "jwt_svid.token"
}]
```

For JWT bundle output, set **jwt_bundle_file_name** instead of the X.509 or JWT SVID file names.

11.9.4. Deploying PostgreSQL with SPIFFE Helper

Deploy a PostgreSQL server and client that use SPIFFE Helper to fetch X.509 SPIFFE Verifiable Identity Document (SVID) from the SPIFFE Workload API and write them to disk for mutual Transport Layer Security (mTLS).

The manifests in this procedure use the Zero Trust Workload Identity Manager SPIFFE Helper image.

When certificates renew, the server must reload PostgreSQL TLS configuration. Containers in the same pod use separate process namespaces, so the SPIFFE Helper sidecar cannot signal the PostgreSQL container directly. Instead, SPIFFE Helper uses the **managed-child** pattern. This pattern in daemon mode it runs **cmd (/usr/bin/psql)** with **cmd_args** to execute **SELECT pg_reload_conf()**; as a child process inside the helper container whenever X.509 material is updated.

The stock SPIFFE Helper image does not include the **psql** client required for that reload command. Build a custom image that extends the Zero Trust Workload Identity Manager image and adds the PostgreSQL client package.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed the Zero Trust Workload Identity Manager.
- The **ZeroTrustWorkloadIdentityManager** custom resource reports **Ready**.

Procedure

1. Create a **Containerfile** with the following contents:

```
FROM registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>

USER 0

RUN dnf install -y postgresql && \
    dnf clean all

USER 65534
```

Replace **<version>** with the tag that matches your Zero Trust Workload Identity Manager installation.

2. Build and push a custom SPIFFE Helper image for the PostgreSQL server by running the following command.

```
$ podman build -f Containerfile \
    -t <registry>/spiffe-helper-postgresql:latest .

$ podman push <registry>/spiffe-helper-postgresql:latest
```

Replace **<registry>** with your container registry.

3. Create the PostgreSQL server resources by using the following example:

```
$ oc apply -f - << 'EOF'
---
kind: Namespace
apiVersion: v1
metadata:
  name: postgresql-spiffe
---
kind: ServiceAccount
```

```

apiVersion: v1
metadata:
  name: postgresql-spiffe
  namespace: postgresql-spiffe
---
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: postgresql-spiffe
spec:
  className: zero-trust-workload-identity-manager-spire
  dnsNameTemplates:
    - postgresql-spiffe.postgresql-spiffe.svc
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: postgresql-spiffe
  podSelector:
    matchLabels:
      app: postgresql-spiffe
  spiffeIDTemplate: 'spiffe://{{ .TrustDomain }}/ns/{{ .PodMeta.Namespace }}/sa/{{
.PodSpec.ServiceAccountName }}'
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: postgres-config
  namespace: postgresql-spiffe
data:
  pg_hba.conf: |
    # TYPE      DATABASE      USER      ADDRESS      METHOD
    local      all             all                                     trust
    host       postgres      postgres   127.0.0.1/32   trust
    hostnossl  postgres      postgres   127.0.0.1/32   trust
    hostnossl  all           all        0.0.0.0/0      reject
    hostssl    all           all        0.0.0.0/0      cert
  postgresql.conf: |
    listen_addresses '*'
    ssl = on
    ssl_cert_file = '/opt/postgresql-certs/svid.pem'
    ssl_key_file = '/opt/postgresql-certs/svid.key'
    ssl_ca_file = '/opt/postgresql-certs/svid_bundle.pem'
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: postgresql-init-db
  namespace: postgresql-spiffe
data:
  initdb.sh: |
    # Copy configuration files
    cp /opt/pg_hba/pg_hba.conf /var/lib/pgsql/data/userdata
    # Create postgresql resources
    psql -U postgres <<!!EOF
      CREATE DATABASE $SPIFFE_DATABASE;
      CREATE USER $SPIFFE_USER WITH encrypted password '$SPIFFE_PASSWORD';
      GRANT ALL privileges ON database $SPIFFE_DATABASE TO $SPIFFE_USER;

```

```

\c $SPIFFE_DATABASE;
CREATE TABLE test_table (
  id bigserial primary key,
  name VARCHAR(255) NOT NULL,
  text VARCHAR(255) NOT NULL
);
GRANT ALL privileges ON table test_table TO $SPIFFE_USER;
GRANT ALL privileges ON sequence test_table_id_seq TO $SPIFFE_USER;
!!EOF
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: spiffe-helper
  namespace: postgresql-spiffe
data:
  helper.conf: |
    agent_address = "/spiffe-workload-api/spire-agent.sock"
    cmd = "/usr/bin/psql"
    cmd_args = "-U postgres -h 127.0.0.1 -c \"SELECT pg_reload_conf();\""
    cert_dir = "/opt/postgresql-certs"
    renew_signal = ""
    svid_file_name = "svid.pem"
    svid_key_file_name = "svid.key"
    svid_bundle_file_name = "svid_bundle.pem"
---
kind: Service
apiVersion: v1
metadata:
  name: postgresql-spiffe
  namespace: postgresql-spiffe
spec:
  ports:
    - protocol: TCP
      port: 5432
      targetPort: 5432
  type: ClusterIP
  selector:
    app: postgresql-spiffe
---
kind: Deployment
apiVersion: apps/v1
metadata:
  name: postgresql-spiffe
  namespace: postgresql-spiffe
  labels:
    app: postgresql-spiffe
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgresql-spiffe
  template:
    metadata:
      labels:
        app: postgresql-spiffe

```

```

spec:
  serviceAccountName: postgresql-spiffe
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  initContainers:
    - name: spiffe-helper-init
      image: <registry>/spiffe-helper-postgresql:latest
      args:
        - '-config'
        - '/etc/spiffe-helper/helper.conf'
        - '-daemon-mode=false'
      volumeMounts:
        - name: spiffe-workload-api
          readOnly: true
          mountPath: /spiffe-workload-api
        - name: postgresql-certs
          mountPath: /opt/postgresql-certs
        - name: spiffe-helper
          mountPath: /etc/spiffe-helper
      securityContext:
        capabilities:
          drop:
            - ALL
  containers:
    - name: postgresql-spiffe
      image: registry.redhat.io/rhel9/postgresql-16:latest
      env:
        - name: POSTGRESQL_USER
          value: user
        - name: POSTGRESQL_PASSWORD
          value: wearenotusingthissoldontworryaboutit
        - name: SPIFFE_USER
          value: postgresql_spiffe
        - name: SPIFFE_PASSWORD
          value: somealsonotusedpassword
        - name: SPIFFE_DATABASE
          value: testdb
        - name: POSTGRESQL_DATABASE
          value: sampled
      ports:
        - name: postgresql
          containerPort: 5432
          protocol: TCP
      volumeMounts:
        - name: postgresql-certs
          mountPath: /opt/postgresql-certs
        - name: pg-hba
          mountPath: /opt/pg_hba
        - name: postgresql-init-db
          mountPath: /opt/app-root/src/postgresql-init
        - name: postgres-config
          mountPath: /opt/app-root/src/postgresql-cfg
      securityContext:
        capabilities:

```

```

      drop:
        - ALL
- name: spiffe-helper
  image: <registry>/spiffe-helper-postgresql:latest
  args:
    - '-config'
    - /etc/spiffe-helper/helper.conf
  volumeMounts:
    - name: spiffe-workload-api
      readOnly: true
      mountPath: /spiffe-workload-api
    - name: postgresql-certs
      mountPath: /opt/postgresql-certs
    - name: spiffe-helper
      mountPath: /etc/spiffe-helper
  securityContext:
    capabilities:
      drop:
        - ALL
  volumes:
    - name: spiffe-workload-api
      csi:
        driver: csi.spiffe.io
        readOnly: true
    - name: postgresql-certs
      emptyDir:
        medium: Memory
    - name: spiffe-helper
      configMap:
        name: spiffe-helper
    - name: postgresql-init-db
      configMap:
        name: postgresql-init-db
    - name: pg-hba
      configMap:
        name: postgres-config
        items:
          - key: pg_hba.conf
            path: pg_hba.conf
    - name: postgres-config
      configMap:
        name: postgres-config
        items:
          - key: postgresql.conf
            path: postgresql.conf
EOF

```

Replace both **<registry>** occurrences in the deployment with your container registry.

4. Confirm that the SPIFFE Helper configuration matches the PostgreSQL TLS settings by running the following command:

```

$ oc get configmap spiffe-helper -n postgresql-spiffe \
  -o jsonpath='{.data.helper.conf}'

```


The **cert_dir** and **svid_*** file names must match the **ssl_*_file** paths in **postgresql.conf**. The server **helper.conf** must include settings similar to the following example:

```
agent_address = "/spiffe-workload-api/spire-agent.sock"
cmd = "/usr/bin/psql"
cmd_args = "-U postgres -h 127.0.0.1 -c \"SELECT pg_reload_conf();\""
cert_dir = "/opt/postgresql-certs"
renew_signal = ""
svid_file_name = "svid.pem"
svid_key_file_name = "svid.key"
svid_bundle_file_name = "svid_bundle.pem"
```

5. Create the PostgreSQL client by using the following example. The client uses the stock SPIFFE Helper image from the Zero Trust Workload Identity Manager. Only the server deployment requires the custom image with the **psql** client.

```
$ oc apply -f - << 'EOF'
---
kind: Namespace
apiVersion: v1
metadata:
  name: postgresql-spiffe-client
---
kind: ServiceAccount
apiVersion: v1
metadata:
  name: postgresql-spiffe-client
  namespace: postgresql-spiffe-client
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: spiffe-helper
  namespace: postgresql-spiffe-client
data:
  helper.conf: |
    agent_address = "/spiffe-workload-api/spire-agent.sock"
    cmd = ""
    cmd_args = ""
    cert_dir = "/opt/postgresql-certs"
    renew_signal = ""
    svid_file_name = "svid.pem"
    svid_key_file_name = "svid.key"
    svid_bundle_file_name = "svid_bundle.pem"
---
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: postgresql-spiffe-client
spec:
  className: zero-trust-workload-identity-manager-spire
  dnsNameTemplates:
    - postgresql-spiffe
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: postgresql-spiffe-client
```

```

podSelector:
  matchLabels:
    app: postgresql-spiffe-client
  spiffeIDTemplate: 'spiffe://{{ .TrustDomain }}/ns/{{ .PodMeta.Namespace }}/sa/{{
.PodSpec.ServiceAccountName }}'
---
kind: Deployment
apiVersion: apps/v1
metadata:
  name: postgresql-spiffe-client
  namespace: postgresql-spiffe-client
  labels:
    app: postgresql-spiffe-client
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgresql-spiffe-client
  template:
    metadata:
      labels:
        app: postgresql-spiffe-client
    spec:
      serviceAccountName: postgresql-spiffe-client
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: postgresql-spiffe-client
          image: registry.redhat.io/rhel9/postgresql-16:latest
          command:
            - /bin/bash
            - '-c'
            - sleep infinity
          volumeMounts:
            - name: postgresql-certs
              mountPath: /opt/postgresql-certs
          securityContext:
            capabilities:
              drop:
                - ALL
        - name: spiffe-helper
          image: registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:
          args:
            - '-config'
            - /etc/spiffe-helper/helper.conf
          volumeMounts:
            - name: spiffe-workload-api
              readOnly: true
              mountPath: /spiffe-workload-api
            - name: postgresql-certs
              mountPath: /opt/postgresql-certs
            - name: spiffe-helper
              mountPath: /etc/spiffe-helper
          securityContext:

```

```

        capabilities:
          drop:
            - ALL
      volumes:
        - name: spiffe-workload-api
          csi:
            driver: csi.spiffe.io
            readOnly: true
        - name: postgresql-certs
          emptyDir:
            medium: Memory
        - name: spiffe-helper
          configMap:
            name: spiffe-helper
EOF

```

Replace **<version>** with the tag that matches your Zero Trust Workload Identity Manager installation.

6. Confirm that pods are running in both namespaces by running the following commands:

```
$ oc get pods -n postgresql-spiffe
```

```
$ oc get pods -n postgresql-spiffe-client
```

Verification

1. Inspect the server certificate by running the following command:

```
$ oc rsh -n postgresql-spiffe -c postgresql-spiffe \
  deployment/postgresql-spiffe \
  cat /opt/postgresql-certs/svid.pem | openssl x509 -noout -text
```

The server certificate should include the service hostname **postgresql-spiffe.postgresql-spiffe.svc**.

2. Inspect the client certificate by running the following command:

```
$ oc rsh -n postgresql-spiffe-client -c postgresql-spiffe-client \
  deployment/postgresql-spiffe-client \
  cat /opt/postgresql-certs/svid.pem | openssl x509 -noout -text
```

The certificate common name (CN) field should be **postgresql_spiffe**.

3. Connect to PostgreSQL from the client pod by running the following commands:

```
$ oc rsh -n postgresql-spiffe-client -c postgresql-spiffe-client \
  deployment/postgresql-spiffe-client
```

```
$ psql "host=postgresql-spiffe.postgresql-spiffe.svc port=5432 \
  user=postgresql_spiffe dbname=testdb sslmode=verify-full \
  sslcert=/opt/postgresql-certs/svid.pem \
  sslkey=/opt/postgresql-certs/svid.key \
  sslrootcert=/opt/postgresql-certs/svid_bundle.pem"
```

■

If the deployment succeeded, **psql** connects to the database.

11.9.5. SPIFFE Helper image reference

Reference information for the SPIFFE Helper container image included with the Zero Trust Workload Identity Manager, including image location, command-line flags, configuration options, and volume requirements.

11.9.5.1. Image location

The Zero Trust Workload Identity Manager provides a SPIFFE Helper image at the following location:

```
registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>
```

Replace **<version>** with the tag that matches your Zero Trust Workload Identity Manager installation. Some workloads require additional tools in the helper image.

11.9.5.2. Command-line flags

Flag	Description
-config <path>	Path to the SPIFFE Helper configuration file. Required.
-daemon-mode <boolean>	Controls operating mode. true (default) runs continuously and renews credentials. false fetches once and exits.
-version	Prints version information and exits.

Examples of command-line flags

```
$ spiffe-helper -config /etc/spiffe-helper/helper.conf
$ spiffe-helper -config /etc/spiffe-helper/helper.conf -daemon-mode=false
```

11.9.5.3. Configuration file options

SPIFFE Helper reads a configuration file. The following table describes the most common options for X.509 workloads on OpenShift Container Platform.

Option	Description
agent_address	Path to the SPIFFE Runtime Environment Agent Workload API socket. On Linux, SPIFFE Helper connects with unix://<path> . With the SPIFFE CSI driver, use /spiffe-workload-api/spire-agent.sock . You can also set the SPIFFE_ENDPOINT_SOCKET environment variable.
cert_dir	Directory where SPIFFE Helper writes credential files. The directory must exist before SPIFFE Helper starts.

Option	Description
daemon_mode	When true (default), runs continuously. When false , fetches once and exits. Can also be controlled with the -daemon-mode flag.
svid_file_name , svid_key_file_name , svid_bundle_file_name	File names for the X.509 SVID certificate, private key, and trust bundle. All three are required to enable X.509 output.
jwt_bundle_file_name	File name for a JWT bundle JSON file.
jwt_svids	Block defining JWT SVID audiences and output file names.
cmd	Executable path for the managed-child pattern. On the first successful X.509 write, SPIFFE Helper starts this process. Ignored in non-daemon mode.
cmd_args	Space-separated arguments for cmd . Use quoted strings for arguments that contain spaces. Not parsed by a shell unless you start a shell explicitly, for example /bin/sh -c "..." .
pid_file_name	Path to a file containing one integer PID for the external-process pattern. Requires renew_signal . Invalid in non-daemon mode.
renew_signal	POSIX signal name sent on renewal, for example SIGUSR1 or SIGHUP . Required when pid_file_name is set. Optional with cmd ; when empty and cmd is set, later renewals do not signal the child.
health_checks	Optional HTTP liveness and readiness endpoints. Available in daemon mode only.

11.9.5.4. Volume and mount requirements

When deploying SPIFFE Helper on OpenShift Container Platform, mount the following resources:

Volume	Mount path	Access
SPIFFE Workload API (CSI)	/spiffe-workload-api (or path matching agent_address)	Read-only
SPIFFE Helper configuration (ConfigMap)	Path passed to -config , for example /etc/spiffe-helper/helper.conf	Read-only

Volume	Mount path	Access
Certificate directory (emptyDir or persistent volume)	Path matching cert_dir , for example /opt/postgresql-certs or /certs	Read/write for the helper; read-only for the application container when possible

CSI volume example

```

apiVersion: apps/v1
metadata:
  name: postgresql-spiffe-client
  namespace: postgresql-spiffe-client
  labels:
    app: postgresql-spiffe-client
# ...
volumes:
- name: spiffe-workload-api
  csi:
    driver: csi.spiffe.io
    readOnly: true

```

Init and sidecar container example

```

apiVersion: apps/v1
metadata:
  name: postgresql-spiffe
  namespace: postgresql-spiffe
  labels:
    app: postgresql-spiffe
# ...
initContainers:
- name: spiffe-helper-init
  image: registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>
  args:
    - '-config'
    - '/etc/spiffe-helper/helper.conf'
    - '-daemon-mode=false'
  volumeMounts:
    - name: spiffe-workload-api
      readOnly: true
      mountPath: /spiffe-workload-api
    - name: postgresql-certs
      mountPath: /opt/postgresql-certs
    - name: spiffe-helper
      mountPath: /etc/spiffe-helper
containers:
- name: spiffe-helper
  image: registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>
  args:
    - '-config'
    - '/etc/spiffe-helper/helper.conf'

```

```

volumeMounts:
- name: spiffe-workload-api
  readOnly: true
  mountPath: /spiffe-workload-api
- name: postgresql-certs
  mountPath: /opt/postgresql-certs
- name: spiffe-helper
  mountPath: /etc/spiffe-helper

```

SPIFFE Helper configuration example

```

agent_address = "/spiffe-workload-api/spire-agent.sock"
cert_dir = "/opt/postgresql-certs"
svid_file_name = "svid.pem"
svid_key_file_name = "svid.key"
svid_bundle_file_name = "svid_bundle.pem"
cmd = "/usr/bin/psql"
cmd_args = "-U postgres -h 127.0.0.1 -c \"SELECT pg_reload_conf();\""
renew_signal = ""

```

Replace **<version>** with the tag that matches your Zero Trust Workload Identity Manager installation.

11.9.5.5. Quick reference

Question	Answer
What happens if both cmd and pid_file_name are set?	SPIFFE Helper writes files, then runs managed-child logic, then PID-file logic. There is no priority between them.
Is renew_signal required for pid_file_name ?	Yes. Validation fails if pid_file_name is set without renew_signal .
Does SPIFFE Helper watch files for the application?	No. The application watches disk, polls, reads at connection time, or handles signals.
What happens in non-daemon mode?	SPIFFE Helper fetches once, writes files, and exits. No watching, cmd , or signals.
What triggers workload notification?	Only X.509 updates in daemon mode. JWT-only refreshes write files only.
Can SPIFFE Helper signal another container in the same pod?	Not by default. Containers use separate process namespaces unless shareProcessNamespace: true is set.

11.9.6. Troubleshooting SPIFFE Helper deployments

Resolve common issues when deploying the SPIFFE Helper image with Zero Trust Workload Identity Manager, and use the diagnostic **oc** commands to verify readiness, sidecars, and on-disk SVID certificates.

11.9.6.1. Common issues

Certificate output directory is empty

SPIFFE Helper cannot reach the Workload API, cannot write to **cert_dir**, or the workload is not registered with SPIFFE Runtime Environment. Confirm the **ZeroTrustWorkloadIdentityManager** custom resource reports **Ready**, the CSI volume is mounted at the path in **agent_address**, and a matching **ClusterSPIFFEID** exists for the pod namespace and labels. Verify **cert_dir** exists before SPIFFE Helper starts; an **emptyDir** volume satisfies this requirement.

Init container exits but TLS files are missing

The init container must run with **-daemon-mode=false**. Check init container logs. If SPIFFE Helper reports configuration warnings about ignored **cmd** or **renew_signal**, that is expected in non-daemon mode. Ensure the certificate volume is shared with the main application container.

Sidecar runs but certificates are not renewed

Confirm the sidecar container does not set **-daemon-mode=false**. Check sidecar logs for Workload API or write errors. If writes fail, SPIFFE Helper logs the error and does not run **cmd** or PID-file notification logic.

Application does not pick up renewed certificates

SPIFFE Helper overwrites files in **cert_dir**; it does not notify your application unless you configure **cmd**, **pid_file_name**, or the application watches or polls the directory. For sidecar deployments with empty **cmd**, the application must reload TLS from disk. For PostgreSQL, configure the managed-child pattern with **psql** and **pg_reload_conf()** in the server sidecar.

cmd or renew_signal appears to have no effect

In non-daemon mode, SPIFFE Helper ignores **cmd** and **renew_signal**. In daemon mode, **cmd** and PID-file logic run only after a successful X.509 write. JWT bundle or JWT SVID updates do not trigger them. If **renew_signal** is empty while **cmd** is set, the child starts on first write but later renewals send no signal.

Cannot signal the application container from the helper sidecar

Containers in the same pod use separate process namespaces by default. **pid_file_name** and **cmd** cannot signal another container unless you set **shareProcessNamespace: true** or run the reload command as a child of SPIFFE Helper. The PostgreSQL example uses **cmd** to run **psql** locally instead of signaling the database container.

Configuration validation fails for pid_file_name

renew_signal is required when **pid_file_name** is set. **pid_file_name** is not valid in non-daemon mode.

Workload API socket is not accessible

Verify the CSI volume is mounted and the socket path matches **agent_address**, typically **/spiffe-workload-api/spire-agent.sock**. Confirm the Zero Trust Workload Identity Manager operands are running and the pod service account matches the **ClusterSPIFFEID** selectors.

Permission denied when reading certificate files

Check file ownership and permissions on the shared volume. Consider setting **fsGroup** in the pod **securityContext** so the application container can read files written by SPIFFE Helper. Mount the certificate directory read-only in the application container when possible.

Image pull errors for the SPIFFE Helper image

Confirm your cluster can pull **registry.redhat.io/zero-trust-workload-identity-manager/spiffe-helper-rhel9:<version>**. If you use a custom image, verify the registry credentials and image reference in the deployment manifest.

PostgreSQL mTLS connection fails after deployment

Confirm SPIFFE Helper wrote files to **/opt/postgresql-certs**, the client certificate CN matches the PostgreSQL user, and **pg_hba.conf** requires certificate authentication. Compare server and client SVIDs with **openssl x509 -noout -text**.

Use the following **oc** commands when investigating SPIFFE Helper deployments and workloads such as PostgreSQL.

11.9.6.2. Verify Zero Trust Workload Identity Manager readiness

```
$ oc get ZeroTrustWorkloadIdentityManager cluster \
  -o jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

```
$ oc get clusterspiffeid
```

11.9.6.3. Inspect SPIFFE Helper pods

```
$ oc get pods -n postgresql-spiffe
```

```
$ oc get pods -n postgresql-spiffe-client
```

11.9.6.4. Review SPIFFE Helper logs

```
$ oc logs -n postgresql-spiffe deployment/postgresql-spiffe -c spiffe-helper-init
```

```
$ oc logs -n postgresql-spiffe deployment/postgresql-spiffe -c spiffe-helper
```

```
$ oc logs -n postgresql-spiffe-client deployment/postgresql-spiffe-client -c spiffe-helper
```

11.9.6.5. Verify on-disk SVID certificates

```
$ oc rsh -n postgresql-spiffe deployment/postgresql-spiffe -c postgresql-spiffe \
  -- ls -la /opt/postgresql-certs
```

```
$ oc rsh -n postgresql-spiffe deployment/postgresql-spiffe -c postgresql-spiffe -- \
  cat /opt/postgresql-certs/svid.pem | openssl x509 -noout -dates -subject
```

11.10. INTEGRATING RED HAT OPENSIFT SERVICE MESH WITH ZERO TRUST WORKLOAD IDENTITY MANAGER IN A SINGLE-CLUSTER

Deploy and configure SPIFFE Runtime Environment as the certificate authority (CA) for Red Hat OpenShift Service Mesh workloads, replacing the Istio built-in CA with SPIFFE-compliant identities and automatically rotated short-lived certificates.

11.10.1. SPIRE integration with Red Hat OpenShift Service Mesh

Red Hat OpenShift Service Mesh integrates with Zero Trust Workload Identity Manager so Envoy sidecars obtain mTLS certificates from Secure Production Identity Framework for Everyone (SPIFFE) instead of Istio's built-in CA, enabling cryptographically verified workload identities.

SPIRE provides cryptographic workload identities based on the Secure Production Identity Framework for Everyone (SPIFFE) standard. This integration enables a zero-trust security model where workload identities are cryptographically verified rather than relying on network-based authentication.

11.10.1.1. Component overview

The following table summarizes the main components in a single-cluster SPIFFE and Red Hat OpenShift Service Mesh integration and what each one does.

Component	Purpose
Zero Trust Workload Identity Manager	Manages SPIRE deployment on OpenShift Container Platform
SPIRE Server	Certificate Authority; issues SVIDs
SPIRE Agent	Runs on each node; provides SDS API to workloads
SPIFFE CSI Driver	Mounts SPIRE socket into pods
ClusterSPIFFEID	Registers which pods get which identities
Red Hat OpenShift Service Mesh Operator	Manages Istio deployment
Istiod	Istio control plane
Envoy Sidecar	Proxy in each pod; uses SPIRE for certificates

11.10.2. SPIRE integration architecture components

Learn about the key components in the SPIRE integration architecture and how they work together to enable zero-trust workload identity and automated certificate management for secure mTLS connections in Red Hat OpenShift Service Mesh.

Zero Trust Workload Identity Manager

Manages the SPIRE deployment lifecycle on OpenShift Container Platform, including custom resources for SPIRE Server, SPIRE Agent, and related components.

SPIRE Server

Acts as the certificate authority that issues SPIFFE Verifiable Identity Documents (SVIDs) to authenticated workloads.

SPIRE Agent

Runs as a DaemonSet on each cluster node, providing the Envoy Secret Discovery Service (SDS) API to workloads on that node.

SPIFFE CSI Driver

Mounts the SPIRE Agent UNIX domain socket into pods, enabling secure communication between Envoy sidecars and the SPIRE Agent.

Red Hat OpenShift Service Mesh

Manages the Istio deployment through the **servicemeshoperator3** Operator.

Istiod

The Istio control plane that configures Envoy proxies but delegates certificate issuance to SPIRE.

Envoy sidecar

The proxy injected into each workload pod that uses SPIRE-issued certificates for mTLS connections.

11.10.3. Deploying SPIRE operands for Red Hat OpenShift Service Mesh integration

Deploy SPIRE operands by creating the **ZeroTrustWorkloadIdentityManager** custom resource (CR) and related SPIRE operand CRs together. A running SPIRE deployment is required before you configure Red Hat OpenShift Service Mesh to use SPIRE-issued certificates for workload mTLS.

Prerequisites

- You have installed Zero Trust Workload Identity Manager.
- The OpenShift CLI (**oc**) is configured with access to the cluster.
- You have permissions to create custom resources in the **zero-trust-workload-identity-manager** namespace.

Procedure

1. Set the environment variables by running the following commands:

```
$ export TRUST_DOMAIN=ocp.one
$ export ZTWIM_NS=zero-trust-workload-identity-manager
$ export JWT_ISSUER="https://oidc-discovery.$(oc get ingresses.config/cluster -o jsonpath={.spec.domain})"
```

2. Deploy all SPIRE operand CRs, including the **ZeroTrustWorkloadIdentityManager** CR:

- a. Create the **ZeroTrustWorkloadIdentityManager** CR:

```
$ oc apply -f - <<EOF
apiVersion: operator.openshift.io/v1alpha1
kind: ZeroTrustWorkloadIdentityManager
metadata:
  name: cluster
labels:
  app.kubernetes.io/name: zero-trust-workload-identity-manager
  app.kubernetes.io/managed-by: zero-trust-workload-identity-manager
spec:
  trustDomain: ${TRUST_DOMAIN}
  clusterName: ""
  bundleConfigMap: "spire-bundle"
EOF
```

- b. Create the **SpireServer** CR:

```
$ cat <<EOF | oc apply -f -
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
```

```

metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  jwtIssuer: $JWT_ISSUER
  caValidity: "24h"
  defaultX509Validity: "1h"
  defaultJWTValidity: "5m"
  caKeytype: "rsa-2048"
  jwtKeyType: "rsa-2048"
  keyManager: ""
  caSubject:
    country: "US"
    organization: "RH"
    commonName: "SPIRE Server CA"
  persistence:
    size: "5Gi"
    accessMode: "ReadWriteOnce"
  datastore:
    databaseType: "sqlite3"
    connectionString: "/run/spire/data/datastore.sqlite3"
    tlsSecretName: ""
    maxOpenConns: 100
    maxIdleConns: 10
    connMaxLifetime: 0
    disableMigration: "false"
EOF

```

- c. Wait for the SPIRE Server to become ready by running the following commands:

```
$ until oc get statefulset/spire-server -n "${ZTWIM_NS}" &> /dev/null; do sleep 3; done
```

```
$ kubectl rollout status statefulset/spire-server -n "${ZTWIM_NS}" --timeout=300s
```

- d. Create the **SpireAgent** CR:

```

$ cat <<EOF | oc apply -f -
apiVersion: operator.openshift.io/v1alpha1
kind: SpireAgent
metadata:
  name: cluster
spec:
  socketPath: "/run/spire/agent-sockets"
  logLevel: "info"
  logFormat: "text"
  nodeAttestor:
    k8sPSATEnabled: "true"
  workloadAttestors:
    k8sEnabled: "true"
  workloadAttestorsVerification:
    type: "auto"
    hostCertBasePath: "/etc/kubernetes"
    hostCertFileName: "kubelet-ca.crt"

```

```

    disableContainerSelectors: "false"
    useNewContainerLocator: "true"
EOF

```

- e. Wait for the SPIRE Agent to become ready by running the following commands:

```
$ until oc get daemonset/spire-agent -n "${ZTWIM_NS}" &> /dev/null; do sleep 3; done
```

```
$ kubectl rollout status daemonset/spire-agent -n "${ZTWIM_NS}" --timeout=300s
```

- f. Deploy the **SpiffeCSIDriver** CR:

```

$ cat <<EOF | oc apply -f -
apiVersion: operator.openshift.io/v1alpha1
kind: SpiffeCSIDriver
metadata:
  name: cluster
spec:
  agentSocketPath: '/run/spire/agent-sockets'
  pluginName: "csi.spiffe.io"
EOF

```

- g. Wait for the SPIFFE CSI Driver to become ready by running the following commands:

```
$ until oc get daemonset/spire-spiffe-csi-driver -n "${ZTWIM_NS}" &> /dev/null; do sleep 3; done
```

```
$ kubectl rollout status daemonset/spire-spiffe-csi-driver -n "${ZTWIM_NS}" --
timeout=300s
```

- h. Deploy the **SpireOIDCDiscoveryProvider** CR:

```
$ export OIDC_DISCOVERY_CONFIG_MAP=spire-spiffe-oidc-discovery-provider
```

```

$ cat <<EOF | oc apply -f -
apiVersion: operator.openshift.io/v1alpha1
kind: SpireOIDCDiscoveryProvider
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  csiDriverName: "csi.spiffe.io"
  jwtIssuer: $JWT_ISSUER
  replicaCount: 1
  managedRoute: "true"
EOF

```

- i. Wait for the OIDC Discovery Provider to be created by running the following commands:

```
$ until oc get deployment spire-spiffe-oidc-discovery-provider -n "${ZTWIM_NS}" &>
/dev/null; do sleep 3; done
```

```
$ oc wait --for=condition=Available deployment/spire-spiffe-oidc-discovery-provider -n
"${ZTWIM_NS}" --timeout=300s
```

Verification

1. Verify that Zero Trust Workload Identity Manager is installed:
 - a. Deploy the client workload and try to fetch a workload SVID:

```
$ cat <<EOF | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ztwim-client
  namespace: default
  labels:
    app: ztwim-client
spec:
  selector:
    matchLabels:
      app: ztwim-client
  template:
    metadata:
      labels:
        app: ztwim-client
    spec:
      containers:
        - name: client
          image: ghcr.io/spiffe/spire-agent:1.5.1
          command: ["/opt/spiffe/bin/spire-agent"]
          args: [ "api", "watch", "-socketPath", "/run/spiffe/sockets/spire-agent.sock" ]
          volumeMounts:
            - mountPath: /run/spiffe/sockets
              name: spiffe-workload-api
              readOnly: true
      volumes:
        - name: spiffe-workload-api
          csi:
            driver: csi.spiffe.io
            readOnly: true
EOF
```

- b. Wait for the client deployment to become ready by running the following command:

```
$ until oc get deployment ztwim-client -n default &> /dev/null; do sleep 3; done
```

```
$ oc wait --for=condition=Available deployment/ztwim-client -n default --timeout=300s
```

```
$ sleep 5
```

2. Verify that the x509 SVID is available by running the following command:

```
$ oc exec -it \
"${oc get \
```

```

pods -o=jsonpath='{.items[0].metadata.name}' \
-l app=ztwim-client \
-n default \
)" -n default -- \
/opt/spire/bin/spire-agent \
api fetch -socketPath /run/spire/sockets/spire-agent.sock

```

The expected output is an SVID like the following example:

```
Received 1 svid after 29.636075ms
```

```

SPIFFE ID: spiffe://ocp.one/ns/default/sa/default
SVID Valid After: 2025-10-21 14:04:03 +0000 UTC
SVID Valid Until: 2025-10-21 15:04:13 +0000 UTC
CA #1 Valid After: 2025-10-21 07:38:03 +0000 UTC
CA #1 Valid Until: 2025-10-22 07:38:13 +0000 UTC

```

3. Verify that the JSON Web Token (JWT) SVID is available by running the following command:

```

$ oc exec -it \
"$(oc get \
  pods -o=jsonpath='{.items[0].metadata.name}' \
  -l app=ztwim-client \
  -n default \
)" -n default -- \
/opt/spire/bin/spire-agent \
api fetch jwt -audience=sample-aud -socketPath /run/spire/sockets/spire-agent.sock

```

The expected output is a JWT SVID like the following example:

```

token(spiffe://ocp.one/ns/default/sa/default):
eyJhbGciOiJSUzI1NiIsImtpZCI6Ij....lslm
bundle(spiffe://ocp.one):
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "6k9PfhrAdfajT6jvLvR6bdomFvQxMeGf",
      "n": "wEYTV0ri4OOcdgEVgzN0...KhUEGf0NKxnuaeGQ",
      "e": "AQAB"
    }
  ]
}

```

4. Remove the client workload by running the following command:

```
$ oc delete deployment ztwim-client -n default
```

11.10.4. Deploying Red Hat OpenShift Service Mesh for SPIRE integration

Deploy Red Hat OpenShift Service Mesh by creating the **IstioCNI** and **Istio** CRs with SPIRE integration settings so Envoy sidecars obtain SPIRE-issued certificates for workload mTLS after the SPIRE stack is running.

Prerequisites

- You have installed Zero Trust Workload Identity Manager.
- The OpenShift CLI (**oc**) is configured with access to the cluster.
- You have permissions to create namespaces and custom resources in the **istio-cni** and **istio-system** namespaces.
- You have permissions to read secrets in the **zero-trust-workload-identity-manager** namespace.

Procedure

1. Set the Istio environment variables by running the following commands:

```
$ export ZTWIM_NS=zero-trust-workload-identity-manager
$ export TRUST_DOMAIN=ocp.one
$ export JWT_ISSUER="https://oidc-discovery.${oc get ingresses.config/cluster -o jsonpath={.spec.domain}}"
$ export OSSM_NS=istio-system
$ export OSSM_CNI=istio-cni
$ export VERIFY_NS=verify-ossm-ztwim
$ export EXTRA_ROOT_CA="$(oc get secret oidc-serving-cert \
    -n ${ZTWIM_NS} -o json | \
    jq -r '.data."tls.crt"' | \
    base64 -d | \
    sed 's/^/ /')
```

2. Create the **IstioCNI** CR to deploy Istio CNI by running the following commands:

```
$ oc new-project "${OSSM_CNI}" 2>/dev/null || oc project "${OSSM_CNI}"
```

```
$ oc apply -f - <<EOF
apiVersion: sailoperator.io/v1
kind: IstioCNI
metadata:
  name: default
spec:
  version: <version>
  namespace: ${OSSM_CNI}
EOF
```

where:

spec.version

Replace **<version>** with the Istio version supported by your Red Hat OpenShift Service Mesh Operator. You can find supported versions by running **oc get IstioCNI -o jsonpath='{.items[*].spec.version}'** after the Operator is installed.

3. Wait for Istio CNI to become ready by running the following commands:

```
$ until oc get daemonset/istio-cni-node -n "${OSSM_CNI}" &> /dev/null; do sleep 3; done
```



```
$ kubectl rollout status daemonset/istio-cni-node -n "${OSSM_CNI}" --timeout=300s
```

The **until** loop waits for the Red Hat OpenShift Service Mesh Operator to create the **istio-cni-node** DaemonSet. The **oc rollout status** command waits for the DaemonSet pods to become ready.

4. Install the Istio CR with SPIRE integration by running the following commands:

```
$ oc new-project "${OSSM_NS}" 2>/dev/null
```

```
$ cat <<EOF | oc apply -f -
apiVersion: sailoperator.io/v1
kind: Istio
metadata:
  name: default
spec:
  namespace: istio-system
  updateStrategy:
    type: InPlace
  values:
    pilot:
      jwksResolverExtraRootCA: |
        ${EXTRA_ROOT_CA}
    env:
      PILOT_JWT_ENABLE_REMOTE_JWKS: "true"
  meshConfig:
    trustDomain: $TRUST_DOMAIN
  defaultConfig:
    proxyMetadata:
      WORKLOAD_IDENTITY_SOCKET_FILE: "spire-agent.sock"
  sidecarInjectorWebhook:
  templates:
    spire: |
      spec:
        initContainers:
          - name: istio-proxy
            volumeMounts:
              - name: workload-socket
                mountPath: /run/secrets/workload-spiffe-uds
                readOnly: true
        volumes:
          - name: workload-socket
            csi:
              driver: "csi.spiffe.io"
              readOnly: true
    spireGateway: |
      spec:
        containers:
          - name: istio-proxy
            volumeMounts:
              - name: workload-socket
                mountPath: /run/secrets/workload-spiffe-uds
                readOnly: true
        volumes:
          - name: workload-socket
```

```

csi:
  driver: "csi.spiffe.io"
  readOnly: true
EOF

```

5. Wait for all of the resources to become ready by running the following commands:

```
$ until oc get deployment istiod -n "${OSSM_NS}" &> /dev/null; do sleep 3; done
```

```
$ oc wait --for=condition=Available deployment/istiod -n "${OSSM_NS}" --timeout=300s
```

Verification

1. Verify that Istio is integrated with SPIRE:
 - a. Create a test workload with the **spire** injection template by running the following commands:

```
$ oc new-project "${VERIFY_NS}" 2>/dev/null
```

- b. Enable the sidecar injection by running the following command:

```
$ oc label namespace "${VERIFY_NS}" istio-injection=enabled
```

- c. Create the **httpbin** workload:

```

$ cat <<EOF | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: ${VERIFY_NS}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      annotations:
        inject.istio.io/templates: "sidecar,spire"
        spiffe.io/audience: "test-audience"
      labels:
        app: httpbin
        version: v1
    spec:
      containers:
        - image: docker.io/mccutchen/go-httpbin:v2.15.0
          imagePullPolicy: IfNotPresent
          name: httpbin
          ports:
            - containerPort: 8080
EOF

```

- d. Wait for all of the resources to become ready by running the following commands:

```
$ until oc get deployment httpbin -n "${VERIFY_NS}" &> /dev/null; do sleep 3; done

$ oc wait --for=condition=Available deployment/httpbin -n "${VERIFY_NS}" --
timeout=300s
```

- e. Verify the SPIRE workload identity by running the following command:

```
$ HTTPBIN_POD=$(oc get pod -l app=httpbin -n "${VERIFY_NS}" -o jsonpath="
{.items[0].metadata.name}")

$ istioctl proxy-config secret "$HTTPBIN_POD" \
-n "${VERIFY_NS}" -o json \
| jq -r '.dynamicActiveSecrets[0].secret.tlsCertificate.certificateChain.inlineBytes' \
| base64 --decode > chain.pem

openssl x509 -in chain.pem -text | grep SPIRE
```

Example output

```
Issuer: C=US, O=RH, CN=<APP_DOMAIN>/serialNumber=...
Subject: C=US, O=SPIRE
```

If you see **SPIRE** in both **Issuer** and **Subject**, the integration is working. Envoy is getting its certificates from SPIRE, not from Istio's built-in CA.

- f. Remove the namespace by running the following command:

```
$ oc delete namespace "${VERIFY_NS}"
```

11.10.5. Additional resources

- [About OpenShift Service Mesh](#)
- [Installing OpenShift Service Mesh](#)

11.11. INTEGRATING SPIRE FEDERATION WITH MULTI-CLUSTER RED HAT OPENSIFT SERVICE MESH

Configure SPIFFE Runtime Environment (SPIRE) federation across multiple OpenShift Container Platform clusters to enable cross-cluster mutual TLS (mTLS) authentication and zero trust workload identity in a multi-cluster service mesh deployment.

11.11.1. Multi-cluster SPIFFE Runtime Environment integration with Red Hat OpenShift Service Mesh

Understand how SPIFFE Runtime Environment (SPIRE) federation integrates with multi-cluster Red Hat OpenShift Service Mesh. Cross-cluster mutual TLS (mTLS) lets workloads on separate clusters authenticate each other under a unified zero-trust identity framework.

Multi-cluster SPIRE integration extends single-cluster SPIRE capabilities to enable workloads in

different clusters to authenticate each other using Secure Production Identity Framework for Everyone (SPIFFE) identities. This eliminates the need for separate certificate authorities per cluster and enables true cross-cluster zero-trust architecture.

11.11.1.1. What gets federated

Federation happens at two layers, and both are required:

Layer	What	How
SPIRE Federation	Trust bundles	SPIRE Servers exchange bundles via https_spiffe profile
Istio Federation	Service discovery and routing	Istiod discovers remote endpoints via remote secrets, routes traffic through east-west gateways

11.11.2. Preparing the environment for multi-cluster SPIFFE Runtime Environment federation

Export kubeconfig paths, trust domains, federation endpoints, and JWT issuer URLs for Cluster A and Cluster B before you deploy federated SPIFFE Runtime Environment (SPIRE) operands on both clusters.

Prerequisites

- You have two OpenShift Container Platform clusters (4.x) with network connectivity between them.
- You have installed Zero Trust Workload Identity Manager on both clusters.
- The OpenShift CLI (**oc**) is configured with access to both clusters.
- You have installed the **istioctl** CLI tool.
- You have installed **helm**. This is used for gateway deployment.
- You have Istio version 1.29.2 or later.

Procedure

1. Export the namespace variables by running the following commands:

```
$ export ZTWIM_NS=zero-trust-workload-identity-manager
```

```
$ export OSSM_NS=istio-system
```

```
$ export OSSM_CNI=istio-cni
```

2. Export the kubeconfig file paths for Cluster A and Cluster B by running the following commands:

```
$ export CLUSTER_A_KUBECONFIG="/path/to/cluster-a/kubeconfig"
```

```
$ export CLUSTER_B_KUBECONFIG="/path/to/cluster-b/kubeconfig"
```

3. Set the base domain environment variables for Cluster A and Cluster B by running the following commands:

```
$ export CLUSTER_A_BASE_DOMAIN=$(oc get ingresses.config/cluster \
-o jsonpath='{.spec.domain}' --kubeconfig "${CLUSTER_A_KUBECONFIG}")
```

```
$ export CLUSTER_B_BASE_DOMAIN=$(oc get ingresses.config/cluster \
-o jsonpath='{.spec.domain}' --kubeconfig "${CLUSTER_B_KUBECONFIG}")
```

4. Export the trust domain environment variables from the base domain of each cluster by running the following commands:

```
$ export CLUSTER_A_TRUST_DOMAIN="${CLUSTER_A_BASE_DOMAIN}"
```

```
$ export CLUSTER_B_TRUST_DOMAIN="${CLUSTER_B_BASE_DOMAIN}"
```

5. Define the cluster and network environment variables by running the following commands:

```
$ export CLUSTER_A=cluster-a
```

```
$ export CLUSTER_B=cluster-b
```

```
$ export NETWORK_A=network-a
```

```
$ export NETWORK_B=network-b
```

6. Export the federation endpoint URLs for Cluster A and Cluster B by running the following commands:

```
$ export FEDERATION_ENDPOINT_A="https://federation.${CLUSTER_A_BASE_DOMAIN}"
```

```
$ export FEDERATION_ENDPOINT_B="https://federation.${CLUSTER_B_BASE_DOMAIN}"
```

7. Set the JWT issuer environment variables for Cluster A and Cluster B by running the following commands:

```
$ export JWT_ISSUER_A="https://oidc-discovery.${CLUSTER_A_BASE_DOMAIN}"
```

```
$ export JWT_ISSUER_B="https://oidc-discovery.${CLUSTER_B_BASE_DOMAIN}"
```

11.11.3. Deploying SPIFFE Runtime Environment with federation on both clusters

Deploy SPIFFE Runtime Environment (SPIRE) operand custom resources (CRs) with federation enabled on Cluster A and Cluster B, wait for operands to become ready, and verify SDS configuration.

Prerequisites

- You have completed preparing the environment for multi-cluster SPIRE federation. For more information, see "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" procedure are set.

Procedure

1. Deploy SPIRE with federation enabled on Cluster A:
 - a. Create a YAML file that defines the **ZeroTrustWorkloadIdentityManager** CR on Cluster A:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ZeroTrustWorkloadIdentityManager
metadata:
  name: cluster
spec:
  trustDomain: ${CLUSTER_A_TRUST_DOMAIN}
  clusterName: ${CLUSTER_A}
  bundleConfigMap: "spire-bundle"
```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

- c. Create a YAML file that defines the **SpireServer** CR on Cluster A:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  jwtIssuer: $JWT_ISSUER_A
  caValidity: "24h"
  defaultX509Validity: "1h"
  defaultJWTValidity: "5m"
  caKeytype: "rsa-2048"
  jwtKeyType: "rsa-2048"
  keyManager: ""
  caSubject:
    country: "US"
    organization: "RH"
    commonName: "SPIRE Server CA"
  persistence:
    size: "5Gi"
    accessMode: "ReadWriteOnce"
  datastore:
```

```

databaseType: "sqlite3"
connectionString: "/run/spire/data/datastore.sqlite3"
tlsSecretName: ""
maxOpenConns: 100
maxIdleConns: 10
connMaxLifetime: 0
disableMigration: "false"
federation:
  bundleEndpoint:
    profile: "https_spiffe"

```

- d. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

- e. Create a YAML file that defines the **SpireAgent** CR on Cluster A:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpireAgent
metadata:
  name: cluster
spec:
  socketPath: "/run/spire/agent-sockets"
  logLevel: "info"
  logFormat: "text"
  nodeAttestor:
    k8sPSATEnabled: "true"
  workloadAttestors:
    k8sEnabled: "true"
  workloadAttestorsVerification:
    type: "auto"
    hostCertBasePath: "/etc/kubernetes"
    hostCertFileName: "kubelet-ca.crt"
  useNewContainerLocator: "true"

```

- f. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

- g. Create a YAML file that defines the **SpiffeCSIDriver** CR on Cluster A:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpiffeCSIDriver
metadata:
  name: cluster
spec:
  agentSocketPath: "/run/spire/agent-sockets"
  pluginName: csi.spiffe.io

```

- h. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

- i. Create a YAML file that defines the **SpireOIDCDiscoveryProvider** CR on Cluster A:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpireOIDCDiscoveryProvider
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  csiDriverName: "csi.spiffe.io"
  jwtIssuer: $JWT_ISSUER_A
  replicaCount: 1
  managedRoute: "true"

```

- j. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

2. Deploy SPIRE with federation enabled on Cluster B:

- a. Create a YAML file that defines the **ZeroTrustWorkloadIdentityManager** CR on Cluster B:

```

apiVersion: operator.openshift.io/v1alpha1
kind: ZeroTrustWorkloadIdentityManager
metadata:
  name: cluster
spec:
  trustDomain: ${CLUSTER_B_TRUST_DOMAIN}
  clusterName: ${CLUSTER_B}
  bundleConfigMap: "spire-bundle"

```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

- c. Create a YAML file that defines the **SpireServer** CR on Cluster B:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  jwtIssuer: $JWT_ISSUER_B
  caValidity: "24h"
  defaultX509Validity: "1h"
  defaultJWTValidity: "5m"
  caKeytype: "rsa-2048"
  jwtKeyType: "rsa-2048"
  keyManager: ""
  caSubject:
    country: "US"
    organization: "RH"
    commonName: "SPIRE Server CA"
  persistence:

```



```

    size: "5Gi"
    accessMode: "ReadWriteOnce"
  datastore:
    databaseType: "sqlite3"
    connectionString: "/run/spire/data/datastore.sqlite3"
    tlsSecretName: ""
    maxOpenConns: 100
    maxIdleConns: 10
    connMaxLifetime: 0
    disableMigration: "false"
  federation:
    bundleEndpoint:
      profile: "https_spiffe"

```

- d. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

- e. Create a YAML file that defines the **SpireAgent** CR on Cluster B:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpireAgent
metadata:
  name: cluster
spec:
  socketPath: "/run/spire/agent-sockets"
  logLevel: "info"
  logFormat: "text"
  nodeAttestor:
    k8sPSATEnabled: "true"
  workloadAttestors:
    k8sEnabled: "true"
  workloadAttestorsVerification:
    type: "auto"
    hostCertBasePath: "/etc/kubernetes"
    hostCertFileName: "kubelet-ca.crt"
    useNewContainerLocator: "true"

```

- f. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

- g. Create a YAML file that defines the **SpiffeCSIDriver** CR on Cluster B:

```

apiVersion: operator.openshift.io/v1alpha1
kind: SpiffeCSIDriver
metadata:
  name: cluster
spec:
  agentSocketPath: "/run/spire/agent-sockets"
  pluginName: csi.spiffe.io

```

- h. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

- i. Create a YAML file that defines the **SpireOIDCDiscoveryProvider** CR on Cluster B:

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireOIDCDiscoveryProvider
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  csiDriverName: "csi.spiffe.io"
  jwtIssuer: $JWT_ISSUER_B
  replicaCount: 1
  managedRoute: "true"
```

- j. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

3. Wait for the **spire-server** StatefulSet to become ready on Cluster A by running the following command:

```
$ oc rollout status statefulset/spire-server --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n
${ZTWIM_NS} --timeout=300s
```

4. Wait for the **spire-agent** DaemonSet to become ready on Cluster A by running the following command:

```
$ oc rollout status daemonset/spire-agent --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n
${ZTWIM_NS} --timeout=300s
```

5. Wait for the **spire-spiffe-csi-driver** DaemonSet to become ready on Cluster A by running the following command:

```
$ oc rollout status daemonset/spire-spiffe-csi-driver --
kubeconfig="${CLUSTER_A_KUBECONFIG}" -n ${ZTWIM_NS} --timeout=300s
```

6. Wait for the **spire-spiffe-oidc-discovery-provider** deployment to become available on Cluster A by running the following command:

```
$ oc wait --for=condition=Available deployment/spire-spiffe-oidc-discovery-provider \
--kubeconfig="${CLUSTER_A_KUBECONFIG}" -n ${ZTWIM_NS} --timeout=300s
```

7. Wait for the **spire-server** StatefulSet to become ready on Cluster B by running the following command:

```
$ oc rollout status statefulset/spire-server --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n
${ZTWIM_NS} --timeout=300s
```

8. Wait for the **spire-agent** DaemonSet to become ready on Cluster B by running the following command:

```
$ oc rollout status daemonset/spire-agent --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n
${ZTWIM_NS} --timeout=300s
```

- Wait for the **spire-spiffe-csi-driver** DaemonSet to become ready on Cluster B by running the following command:

```
$ oc rollout status daemonset/spire-spiffe-csi-driver --
kubeconfig="${CLUSTER_B_KUBECONFIG}" -n ${ZTWIM_NS} --timeout=300s
```

- Wait for the **spire-spiffe-oidc-discovery-provider** deployment to become available on Cluster B by running the following command:

```
$ oc wait --for=condition=Available deployment/spire-spiffe-oidc-discovery-provider \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" -n ${ZTWIM_NS} --timeout=300s
```

Verification

- Verify that the SDS configuration is available on Cluster A by running the following command:

```
$ oc get cm spire-agent --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n "${ZTWIM_NS}" \
-o jsonpath='{.data.agent\.conf}' | grep -A5 "'sds'"
```

- Verify that the SDS configuration is available on Cluster B by running the following command:

```
$ oc get cm spire-agent --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n "${ZTWIM_NS}" \
-o jsonpath='{.data.agent\.conf}' | grep -A5 "'sds'"
```

Example output

```
"sds": {
  "default_all_bundles_name": "ROOTCA",
  "default_bundle_name": "null"
},
```

11.11.4. Additional resources

- [Installing the Zero Trust Workload Identity Manager](#)
- [Zero Trust Workload Identity Manager SPIRE federation](#)

11.11.5. Configuring Red Hat OpenShift Service Mesh for multi-cluster SPIFFE Runtime Environment integration

Configure Red Hat OpenShift Service Mesh on each cluster with federation settings, east-west gateways, and remote secrets to enable cross-cluster service communication by using SPIRE-issued certificates.

Prerequisites

- You deployed SPIFFE Runtime Environment (SPIRE) with federation for multi-cluster integration. For more information, see "Deploying SPIFFE Runtime Environment with federation on both clusters".

Procedure

1. Verify that the federation routes are created on Cluster A by running the following command:

```
$ oc get route -n ${ZTWIM_NS} --kubeconfig="${CLUSTER_A_KUBECONFIG}" | grep
federation
```

2. Verify that the federation routes are created on Cluster B by running the following command:

```
$ oc get route -n ${ZTWIM_NS} --kubeconfig="${CLUSTER_B_KUBECONFIG}" | grep
federation
```

3. On Cluster A, create a **ClusterFederatedTrustDomain** object pointing to Cluster B by running the following command:

- a. Create a YAML file that defines the **ClusterFederatedTrustDomain** CR on Cluster A:

```
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: federation-to-cluster-b
spec:
  trustDomain: ${CLUSTER_B_TRUST_DOMAIN}
  bundleEndpointURL: ${FEDERATION_ENDPOINT_B}
  bundleEndpointProfile:
    type: https_spiffe
    endpointSPIFFEID: spiffe://${CLUSTER_B_TRUST_DOMAIN}/spire/server
```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

4. On Cluster B, create a **ClusterFederatedTrustDomain** object pointing to Cluster A by running the following command:

- a. Create a YAML file that defines the **ClusterFederatedTrustDomain** CR on Cluster B:

```
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterFederatedTrustDomain
metadata:
  name: federation-to-cluster-a
spec:
  trustDomain: ${CLUSTER_A_TRUST_DOMAIN}
  bundleEndpointURL: ${FEDERATION_ENDPOINT_A}
  bundleEndpointProfile:
    type: https_spiffe
    endpointSPIFFEID: spiffe://${CLUSTER_A_TRUST_DOMAIN}/spire/server
```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

Verification

1. Verify that the SPIRE Server on Cluster A has the trust bundle from Cluster B by running the following command:

```
$ oc exec --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n ${ZTWIM_NS} spire-server-0 -c
spire-server -- \
    spire-server bundle list -socketPath /tmp/spire-server/private/api.sock -format spiffe 2>&1 |
head -5
```

The output must show public keys for **`\${CLUSTER\_B\_TRUST\_DOMAIN}`**.

Example output

```
{
  "trust_domains": {
    "${CLUSTER_B_TRUST_DOMAIN}": {
      "keys": [
```

2. Verify that the SPIRE Server on Cluster B has the trust bundle from Cluster A by running the following command:

```
$ oc exec --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n ${ZTWIM_NS} spire-server-0 -c
spire-server -- \
    spire-server bundle list -socketPath /tmp/spire-server/private/api.sock -format spiffe 2>&1 |
head -5
```

The output must show public keys for **`\${CLUSTER\_A\_TRUST\_DOMAIN}`**.

Example output

```
{
  "trust_domains": {
    "${CLUSTER_A_TRUST_DOMAIN}": {
      "keys": [
```

3. Verify that the federation endpoint on Cluster A is reachable by running the following command:

```
$ curl -sk "${FEDERATION_ENDPOINT_A}" | python3 -c "import sys,json; print(f'Keys:
{len(json.load(sys.stdin).get(\"keys\",[]))}')"

```

The output must show at least one **x509-svid** key and one **jwt-svid** key.

Example output

```
Keys: 2
```

4. Verify that the federation endpoint on Cluster B is reachable by running the following command:

```
$ curl -sk "${FEDERATION_ENDPOINT_B}" | python3 -c "import sys,json; print(f'Keys:
{len(json.load(sys.stdin).get(\"keys\",[]))}')"

```

The output must show at least one **x509-svid** key and one **jwt-svid** key.

Example output

```
Keys: 2
```

11.11.6. Deploying the Red Hat OpenShift Service Mesh CNI on both clusters

Deploy the **IstioCNI** CR and federated **ClusterSPIFFEID** resources on Cluster A and Cluster B. This configures Red Hat OpenShift Service Mesh CNI networking and federated SPIFFE trust for cross-cluster mesh workloads.

Prerequisites

- You have configured Red Hat OpenShift Service Mesh for multi-cluster integration. For more information, see "Configuring Red Hat OpenShift Service Mesh for multi-cluster SPIFFE Runtime Environment integration".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" and "Deploying SPIFFE Runtime Environment with federation on both clusters" procedures are set.
- You have installed Red Hat OpenShift Service Mesh 2.6.11 on both clusters.

Procedure

1. Create the Red Hat OpenShift Service Mesh CNI namespace on Cluster A by running the following command:

```
$ oc new-project "${OSSM_CNI}" --kubeconfig="${CLUSTER_A_KUBECONFIG}"
2>/dev/null || true
```

2. Deploy the **IstioCNI** CR on Cluster A by running the following command:

- a. Create a YAML file that defines the **IstioCNI** CR on Cluster A:

```
apiVersion: sailoperator.io/v1
kind: IstioCNI
metadata:
  name: default
spec:
  namespace: ${OSSM_CNI}
```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

3. Wait for the **istio-cni-node** DaemonSet to be created on Cluster A by running the following command:

```
$ until oc get daemonset/istio-cni-node --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n
"${OSSM_CNI}" &> /dev/null; do
  sleep 3
done
```

4. Wait for the **IstioCNI** DaemonSet to become ready on Cluster A by running the following command:

```
$ oc rollout status daemonset/istio-cni-node --kubeconfig="${CLUSTER_A_KUBECONFIG}"
-n "${OSSM_CNI}" --timeout=300s
```

5. Create the Red Hat OpenShift Service Mesh CNI namespace on Cluster B by running the following command:

```
$ oc new-project "${OSSM_CNI}" --kubeconfig="${CLUSTER_B_KUBECONFIG}"
2>/dev/null || true
```

6. Deploy the **IstioCNI** CR on Cluster B by running the following command:

- a. Create a YAML file that defines the **IstioCNI** CR on Cluster B:

```
apiVersion: sailoperator.io/v1
kind: IstioCNI
metadata:
  name: default
spec:
  namespace: ${OSSM_CNI}
```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

7. Wait for the **istio-cni-node** DaemonSet to be created on Cluster B by running the following command:

```
$ until oc get daemonset/istio-cni-node --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n
"${OSSM_CNI}" &> /dev/null; do
  sleep 3
done
```

8. Wait for the **IstioCNI** DaemonSet to become ready on Cluster B by running the following command:

```
$ oc rollout status daemonset/istio-cni-node --kubeconfig="${CLUSTER_B_KUBECONFIG}"
-n "${OSSM_CNI}" --timeout=300s
```

9. Create the federated **ClusterSPIFFEID** resources on Cluster A by running the following command:

- a. Create a YAML file that defines the federated **ClusterSPIFFEID** resources on Cluster A:

```
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: sample-federation
spec:
  className: zero-trust-workload-identity-manager-spire
  spiffeIDTemplate: "spiffe://{{ .TrustDomain }}/ns/{{ .PodMeta.Namespace }}/sa/{{
.PodSpec.ServiceAccountName }}"
```

```

namespaceSelector:
  matchLabels:
    kubernetes.io/metadata.name: sample
federatesWith:
  - "${CLUSTER_B_TRUST_DOMAIN}"
---
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: istio-system-federation
spec:
  className: zero-trust-workload-identity-manager-spire
  spiffeIDTemplate: "spiffe://{{ .TrustDomain }}/ns/{{ .PodMeta.Namespace }}/sa/{{
.PodSpec.ServiceAccountName }}"
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: istio-system
  federatesWith:
    - "${CLUSTER_B_TRUST_DOMAIN}"

```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

10. Create federated **ClusterSPIFFEID** resources on Cluster B by running the following command:

- a. Create a YAML file that defines the federated **ClusterSPIFFEID** resources on Cluster B:

```

apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: sample-federation
spec:
  className: zero-trust-workload-identity-manager-spire
  spiffeIDTemplate: "spiffe://{{ .TrustDomain }}/ns/{{ .PodMeta.Namespace }}/sa/{{
.PodSpec.ServiceAccountName }}"
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: sample
  federatesWith:
    - "${CLUSTER_A_TRUST_DOMAIN}"
---
apiVersion: spire.spiffe.io/v1alpha1
kind: ClusterSPIFFEID
metadata:
  name: istio-system-federation
spec:
  className: zero-trust-workload-identity-manager-spire
  spiffeIDTemplate: "spiffe://{{ .TrustDomain }}/ns/{{ .PodMeta.Namespace }}/sa/{{
.PodSpec.ServiceAccountName }}"
  namespaceSelector:
    matchLabels:
      kubernetes.io/metadata.name: istio-system
  federatesWith:
    - "${CLUSTER_A_TRUST_DOMAIN}"

```


- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```



IMPORTANT

Do not patch the default **ClusterSPIFFEID** (**zero-trust-workload-identity-manager-spire-default**). The Zero Trust Workload Identity Manager reconciles and reverts manual changes. Instead, create custom **ClusterSPIFFEID** resources for the specific namespaces.

11.11.7. Deploying the Istio custom resource with the federation configuration

Deploy the Istio custom resource on Cluster A and Cluster B with SPIFFE Runtime Environment (SPIRE) federation and multi-cluster Red Hat OpenShift Service Mesh settings. This configures Istiod to obtain workload certificates from SPIRE and to trust SPIFFE bundles from both clusters for cross-cluster mTLS.

The Istio CR must include the following fields and values:

- A **meshConfig.trustDomain** value that matches the SPIRE trust domain.
- A **meshConfig.caCertificates** value with bundle URLs for both clusters. This handles cross-trust-domain validation.
- A **WORKLOAD_IDENTITY_SOCKET_FILE** value for SPIRE SDS integration.
- A **jwtResolverExtraRootCA** value for OIDC validation.
- A multi-cluster configuration that includes **meshID**, **clusterName**, and **network**.
- A SPIRE injection template configuration.



NOTE

Do not use **meshConfig.trustDomainAliases**. Use **meshConfig.caCertificates** with **spiffeBundleUrl** instead.

Prerequisites

- You have completed deploying the Istio Container Network Interface (CNI) on both clusters. For more information, see "Deploying Red Hat OpenShift Service Mesh CNI on both clusters".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" and "Deploying SPIFFE Runtime Environment with federation on both clusters" procedures are set.

Procedure

1. Extract the OpenID Connect (OIDC) certificate on Cluster A by running the following command:

```
$ export EXTRA_ROOT_CA_A="$(oc get secret oidc-serving-cert \
--kubeconfig="${CLUSTER_A_KUBECONFIG}" -n ${ZTWIM_NS} -o json | \
jq -r '.data."tls.crt"' | base64 -d | sed 's/^/ /')"
```

2. Extract the OpenID Connect (OIDC) certificate on Cluster B by running the following command:

```
$ export EXTRA_ROOT_CA_B="$(oc get secret oidc-serving-cert \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" -n ${ZTWIM_NS} -o json | \
jq -r '.data."tls.crt"' | base64 -d | sed 's/^/ /')"
```

3. Get the bundle endpoint URL for Cluster A by running the following command:

```
$ export BUNDLE_URL_A="${FEDERATION_ENDPOINT_A}"
```

4. Get the bundle endpoint URL for Cluster B by running the following command:

```
$ export BUNDLE_URL_B="${FEDERATION_ENDPOINT_B}"
```

5. Create the **Istio** custom resource (CR) on Cluster A by running the following command:

```
$ oc new-project "${OSSM_NS}" --kubeconfig="${CLUSTER_A_KUBECONFIG}" 2>/dev/null
|| true
```

6. Apply the **Istio** CR on Cluster A by running the following command:

- a. Create a YAML file that defines the **Istio** CR on Cluster A:

```
apiVersion: sailoperator.io/v1
kind: Istio
metadata:
  name: default
spec:
  namespace: istio-system
  updateStrategy:
    type: InPlace
  values:
    meshConfig:
      trustDomain: ${CLUSTER_A_TRUST_DOMAIN}
      defaultConfig:
        proxyMetadata:
          WORKLOAD_IDENTITY_SOCKET_FILE: "spire-agent.sock"
    caCertificates:
      - spiffeBundleUrl: ${BUNDLE_URL_A}
        trustDomains:
          - ${CLUSTER_A_TRUST_DOMAIN}
      - spiffeBundleUrl: ${BUNDLE_URL_B}
        trustDomains:
          - ${CLUSTER_B_TRUST_DOMAIN}
    global:
      meshID: mesh1
      multiCluster:
        clusterName: ${CLUSTER_A}
      network: ${NETWORK_A}
```

```

pilot:
  jwksResolverExtraRootCA: |
    ${EXTRA_ROOT_CA_A}
  env:
    ENABLE_CA_SERVER: "true"
sidecarInjectorWebhook:
  templates:
    spire: |
      spec:
        initContainers:
          - name: istio-proxy
            volumeMounts:
              - name: workload-socket
                mountPath: /run/secrets/workload-spiffe-uds
                readOnly: true
        volumes:
          - name: workload-socket
            csi:
              driver: "csi.spiffe.io"
              readOnly: true
    spireGw: |
      spec:
        containers:
          - name: istio-proxy
            volumeMounts:
              - name: workload-socket
                mountPath: /run/secrets/workload-spiffe-uds
                readOnly: true
        volumes:
          - name: workload-socket
            csi:
              driver: "csi.spiffe.io"
              readOnly: true

```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

7. Create the **Istio** CR on Cluster B by running the following command:

```
$ oc new-project "${OSSM_NS}" --kubeconfig="${CLUSTER_B_KUBECONFIG}" 2>/dev/null
|| true
```

8. Apply the **Istio** CR on Cluster B by running the following command:

- a. Create a YAML file that defines the **Istio** CR on Cluster B:

```

apiVersion: sailoperator.io/v1
kind: Istio
metadata:
  name: default
spec:
  namespace: istio-system
  updateStrategy:
    type: InPlace

```

```

values:
  meshConfig:
    trustDomain: ${CLUSTER_B_TRUST_DOMAIN}
    defaultConfig:
      proxyMetadata:
        WORKLOAD_IDENTITY_SOCKET_FILE: "spire-agent.sock"
    caCertificates:
      - spiffeBundleUrl: ${BUNDLE_URL_B}
        trustDomains:
          - ${CLUSTER_B_TRUST_DOMAIN}
      - spiffeBundleUrl: ${BUNDLE_URL_A}
        trustDomains:
          - ${CLUSTER_A_TRUST_DOMAIN}
  global:
    meshID: mesh1
    multiCluster:
      clusterName: ${CLUSTER_B}
      network: ${NETWORK_B}
  pilot:
    jwksResolverExtraRootCA: |
      ${EXTRA_ROOT_CA_B}
  env:
    ENABLE_CA_SERVER: "true"
  sidecarInjectorWebhook:
    templates:
      spire: |
        spec:
          initContainers:
            - name: istio-proxy
              volumeMounts:
                - name: workload-socket
                  mountPath: /run/secrets/workload-spiffe-uds
                  readOnly: true
          volumes:
            - name: workload-socket
              csi:
                driver: "csi.spiffe.io"
                readOnly: true
      spireGw: |
        spec:
          containers:
            - name: istio-proxy
              volumeMounts:
                - name: workload-socket
                  mountPath: /run/secrets/workload-spiffe-uds
                  readOnly: true
          volumes:
            - name: workload-socket
              csi:
                driver: "csi.spiffe.io"
                readOnly: true

```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

9. Wait for the **istiod** deployment to be created on Cluster A by running the following command:

```
$ until oc get deployment istiod --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n
"${OSSM_NS}" &> /dev/null; do
  sleep 3
done
```

10. Wait for **Istiod** to become ready on Cluster A by running the following command:

```
$ oc wait --for=condition=Available deployment/istiod \
--kubeconfig="${CLUSTER_A_KUBECONFIG}" -n "${OSSM_NS}" --timeout=300s
```

11. Wait for the **istiod** deployment to be created on Cluster B by running the following command:

```
$ until oc get deployment istiod --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n
"${OSSM_NS}" &> /dev/null; do
  sleep 3
done
```

12. Wait for **Istiod** to become ready on Cluster B by running the following command:

```
$ oc wait --for=condition=Available deployment/istiod \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" -n "${OSSM_NS}" --timeout=300s
```

11.11.8. Verifying SPIRE integration with Istio on each cluster

Verify that Red Hat OpenShift Service Mesh on Cluster A and Cluster B obtains workload certificates from SPIFFE Runtime Environment (SPIRE). This confirms Istio sidecars use SPIRE-issued identities rather than the built-in Istio certificate authority (CA) before you proceed with cross-cluster mesh verification.

Prerequisites

- You have deployed the Istio custom resource (CR) with the federation configuration. For more information, see "Deploying the Istio custom resource with the federation configuration".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" and "Deploying SPIFFE Runtime Environment with federation on both clusters" procedures are set.
- Istiod is running and ready on Cluster A and Cluster B.

Procedure

1. Set the verification namespace variable by running the following command:

```
$ export VERIFY_NS=verify-ossm-ztwim
```

2. Prepare the verification namespace on both clusters by running the following commands:
 - a. Create the verification namespace on Cluster A:

```
$ oc create namespace ${VERIFY_NS} --kubeconfig="${CLUSTER_A_KUBECONFIG}"
2>/dev/null || true
```

- b. Enable Istio injection for the verification namespace on Cluster A:

```
$ oc label namespace ${VERIFY_NS} istio-injection=enabled \
--kubeconfig="${CLUSTER_A_KUBECONFIG}" --overwrite
```

- c. Create the verification namespace on Cluster B:

```
$ oc create namespace ${VERIFY_NS} --kubeconfig="${CLUSTER_B_KUBECONFIG}"
2>/dev/null || true
```

- d. Enable Istio injection for the verification namespace on Cluster B:

```
$ oc label namespace ${VERIFY_NS} istio-injection=enabled \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" --overwrite
```

3. Deploy the **httpbin** workload on Cluster A by running the following command:

- a. Create a YAML file that defines the **httpbin Deployment** on Cluster A:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: ${VERIFY_NS}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      annotations:
        inject.istio.io/templates: "sidecar,spire"
        spiffe.io/audience: "test-audience"
      labels:
        app: httpbin
        version: v1
    spec:
      containers:
        - image: docker.io/mccutchen/go-httpbin:v2.15.0
          imagePullPolicy: IfNotPresent
          name: httpbin
          ports:
            - containerPort: 8080
```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

4. Deploy the **httpbin** workload on Cluster B by running the following command:

a. Create a YAML file that defines the **httpbin Deployment** on Cluster B:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: ${VERIFY_NS}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      annotations:
        inject.istio.io/templates: "sidecar,spire"
        spiffe.io/audience: "test-audience"
      labels:
        app: httpbin
        version: v1
    spec:
      containers:
        - image: docker.io/mccutchen/go-httpbin:v2.15.0
          imagePullPolicy: IfNotPresent
          name: httpbin
          ports:
            - containerPort: 8080
```

b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

5. Wait for the **httpbin** deployment to become available on Cluster A by running the following command:

```
$ oc rollout status deployment/httpbin \
  -n "${VERIFY_NS}" --kubeconfig="${CLUSTER_A_KUBECONFIG}" --timeout=300s
```

6. Wait for the **httpbin** deployment to become available on Cluster B by running the following command:

```
$ oc rollout status deployment/httpbin \
  -n "${VERIFY_NS}" --kubeconfig="${CLUSTER_B_KUBECONFIG}" --timeout=300s
```

7. Verify the Envoy sidecar certificate on Cluster A by running the following commands:

a. Get the **httpbin** pod name on Cluster A:

```
$ HTTPBIN_POD=$(oc get pod -l app=httpbin -n "${VERIFY_NS}" \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}" -o jsonpath="
  {.items[0].metadata.name})
```

- b. Export the Envoy sidecar certificate chain for the **httpbin** pod on Cluster A:

```
$ istioctl --kubeconfig="${CLUSTER_A_KUBECONFIG}" proxy-config secret
"${HTTPBIN_POD}" \
-n "${VERIFY_NS}" -o json \
| jq -r '.dynamicActiveSecrets[0].secret.tlsCertificate.certificateChain.inlineBytes' \
| base64 --decode > chain-a.pem
```

- c. Confirm the certificate was issued by SPIRE on Cluster A:

```
$ openssl x509 -in chain-a.pem -text | grep SPIRE
```

8. Verify the Envoy sidecar certificate on Cluster B by running the following commands:

- a. Get the **httpbin** pod name on Cluster B:

```
$ HTTPBIN_POD=$(oc get pod -l app=httpbin -n "${VERIFY_NS}" \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" -o jsonpath="
{.items[0].metadata.name}")
```

- b. Export the Envoy sidecar certificate chain for the **httpbin** pod on Cluster B:

```
$ istioctl --kubeconfig="${CLUSTER_B_KUBECONFIG}" proxy-config secret
"${HTTPBIN_POD}" \
-n "${VERIFY_NS}" -o json \
| jq -r '.dynamicActiveSecrets[0].secret.tlsCertificate.certificateChain.inlineBytes' \
| base64 --decode > chain-b.pem
```

- c. Confirm the certificate was issued by SPIRE on Cluster B:

```
$ openssl x509 -in chain-b.pem -text | grep SPIRE
```

Example output

```
Issuer: C=US, O=RH, CN=<APP_DOMAIN>/serialNumber=...
Subject: C=US, O=SPIRE
```

If you see **SPIRE** in both **Issuer** and **Subject** on each cluster, Red Hat OpenShift Service Mesh is obtaining workload certificates from SPIRE rather than the Istio built-in CA.

9. Remove the verification namespace from both clusters by running the following commands:

- a. Remove the verification namespace from Cluster A:

```
$ oc delete namespace ${VERIFY_NS} --kubeconfig="${CLUSTER_A_KUBECONFIG}"
--ignore-not-found
```

- b. Remove the verification namespace from Cluster B:

```
$ oc delete namespace ${VERIFY_NS} --kubeconfig="${CLUSTER_B_KUBECONFIG}"
--ignore-not-found
```


11.11.9. Verifying workload mTLS with SPIRE-issued identities on each cluster

Deploy **httpbin** and **curl** test workloads with SPIFFE Runtime Environment (SPIRE) sidecar injection on both clusters, enable **STRICT** mTLS with **ISTIO_MUTUAL**, and verify HTTP connectivity on each cluster. This confirms workloads use SPIRE-issued certificates under **STRICT** mTLS.

Prerequisites

- You have verified that SPIRE is integrated with Istio on each cluster. For more information, see "Verifying SPIRE integration with Istio on each cluster".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" and "Deploying SPIFFE Runtime Environment with federation on both clusters" procedures are set.
- Istiod is running and ready on both clusters.

Procedure

1. Set the test environment variables by running the following commands:

- a. Set the test namespace environment variable:

```
$ export TPJ=test-ossm-with-ztwim
```

- b. Set the SPIFFE audience environment variable:

```
$ export SPIFFE_AUDIENCE="sky-computing-demo"
```

2. Prepare the test namespace on both clusters by running the following commands:

- a. Create the test namespace on Cluster A:

```
$ oc create namespace ${TPJ} --kubeconfig="${CLUSTER_A_KUBECONFIG}"
2>/dev/null || true
```

- b. Enable Istio injection for the test namespace on Cluster A:

```
$ oc label namespace ${TPJ} istio-injection=enabled \
--kubeconfig="${CLUSTER_A_KUBECONFIG}" --overwrite
```

- c. Create the test namespace on Cluster B:

```
$ oc create namespace ${TPJ} --kubeconfig="${CLUSTER_B_KUBECONFIG}"
2>/dev/null || true
```

- d. Enable Istio injection for the test namespace on Cluster B:

```
$ oc label namespace ${TPJ} istio-injection=enabled \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" --overwrite
```

3. Create the **httpbin** server on Cluster A by running the following command:

- a. Create a YAML file that defines the **httpbin ServiceAccount**, **Service**, and **Deployment** on Cluster A:

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
  namespace: ${TPJ}
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: ${TPJ}
labels:
  app: httpbin
  service: httpbin
spec:
  ports:
    - name: http-ex-spiffe
      port: 443
      targetPort: 8080
    - name: http
      port: 80
      targetPort: 8080
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: ${TPJ}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      annotations:
        inject.istio.io/templates: "sidecar,spire"
        spiffe.io/audience: "${SPIFFE_AUDIENCE}"
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
        - image: docker.io/mccutchen/go-httpbin:v2.15.0
          imagePullPolicy: IfNotPresent
          name: httpbin
          ports:
            - containerPort: 8080

```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

4. Create the **httpbin** server on Cluster B by running the following command:

- a. Create a YAML file that defines the **httpbin ServiceAccount**, **Service**, and **Deployment** on Cluster B:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
  namespace: ${TPJ}
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: ${TPJ}
labels:
  app: httpbin
  service: httpbin
spec:
  ports:
    - name: http-ex-spiffe
      port: 443
      targetPort: 8080
    - name: http
      port: 80
      targetPort: 8080
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: ${TPJ}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      annotations:
        inject.istio.io/templates: "sidecar,spire"
        spiffe.io/audience: "${SPIFFE_AUDIENCE}"
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
```

```
- image: docker.io/mccutchen/go-httpbin:v2.15.0
  imagePullPolicy: IfNotPresent
  name: httpbin
  ports:
  - containerPort: 8080
```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

5. Wait for the **httpbin** deployment to become available on both clusters by running the following commands:

- a. Wait for the **httpbin** deployment on Cluster A:

```
$ oc rollout status deployment/httpbin \
-n "${TPJ}" --kubeconfig="${CLUSTER_A_KUBECONFIG}" --timeout=300s
```

- b. Wait for the **httpbin** deployment on Cluster B:

```
$ oc rollout status deployment/httpbin \
-n "${TPJ}" --kubeconfig="${CLUSTER_B_KUBECONFIG}" --timeout=300s
```

6. Create the **curl** client on Cluster A by running the following command:

- a. Create a YAML file that defines the **curl ServiceAccount**, **Service**, and **Deployment** on Cluster A:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: curl
  namespace: ${TPJ}
---
apiVersion: v1
kind: Service
metadata:
  name: curl
  namespace: ${TPJ}
labels:
  app: curl
  service: curl
spec:
  ports:
  - port: 80
    name: http
  selector:
    app: curl
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: curl
  namespace: ${TPJ}
spec:
```

```

replicas: 1
selector:
  matchLabels:
    app: curl
template:
  metadata:
    annotations:
      inject.istio.io/templates: "sidecar,spire"
      spiffe.io/audience: "${SPIFFE_AUDIENCE}"
    labels:
      app: curl
  spec:
    terminationGracePeriodSeconds: 0
    serviceAccountName: curl
    containers:
      - name: curl
        image: curlimages/curl:8.16.0
        command:
          - /bin/sh
          - -c
          - sleep inf
        imagePullPolicy: IfNotPresent

```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

7. Create the **curl** client on Cluster B by running the following command:

- a. Create a YAML file that defines the **curl ServiceAccount**, **Service**, and **Deployment** on Cluster B:

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: curl
  namespace: ${TPJ}
---
apiVersion: v1
kind: Service
metadata:
  name: curl
  namespace: ${TPJ}
labels:
  app: curl
  service: curl
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: curl
---
apiVersion: apps/v1
kind: Deployment
metadata:

```

```

name: curl
namespace: ${TPJ}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: curl
  template:
    metadata:
      annotations:
        inject.istio.io/templates: "sidecar,spire"
        spiffe.io/audience: "${SPIFFE_AUDIENCE}"
      labels:
        app: curl
    spec:
      terminationGracePeriodSeconds: 0
      serviceAccountName: curl
      containers:
        - name: curl
          image: curlimages/curl:8.16.0
          command:
            - /bin/sh
            - -c
            - sleep inf
          imagePullPolicy: IfNotPresent

```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

8. Wait for the **curl** deployment to become available on both clusters by running the following commands:

- a. Wait for the **curl** deployment on Cluster A:

```
$ oc rollout status deployment/curl \
-n "${TPJ}" --kubeconfig="${CLUSTER_A_KUBECONFIG}" --timeout=300s
```

- b. Wait for the **curl** deployment on Cluster B:

```
$ oc rollout status deployment/curl \
-n "${TPJ}" --kubeconfig="${CLUSTER_B_KUBECONFIG}" --timeout=300s
```

9. Verify that the **curl** client can reach **httpbin** on both clusters before enabling **STRICT** mTLS by running the following commands:

- a. Verify connectivity on Cluster A:

```
$ oc exec deploy/curl -n "${TPJ}" --kubeconfig="${CLUSTER_A_KUBECONFIG}" -it -- \
curl -s -o /dev/null -w "%{http_code}" http://httpbin
```

- b. Verify connectivity on Cluster B:

```
$ oc exec deploy/curl -n "${TPJ}" --kubeconfig="${CLUSTER_B_KUBECONFIG}" -it -- \
  curl -s -o /dev/null -w "%{http_code}" http://httpbin
```

Example output

```
200
```

You must receive an HTTP **200** status code on each cluster.

10. Enable **STRICT** mTLS between the services on Cluster A by running the following command:

- a. Create a YAML file that defines the **PeerAuthentication** and **DestinationRule** resources on Cluster A:

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ${TPJ}
spec:
  mtls:
    mode: STRICT
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: curl
  namespace: ${TPJ}
spec:
  host: curl
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin
  namespace: ${TPJ}
spec:
  host: httpbin
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

11. Enable **STRICT** mTLS between the services on Cluster B by running the following command:

- a. Create a YAML file that defines the **PeerAuthentication** and **DestinationRule** resources on Cluster B:

```

apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ${TPJ}
spec:
  mtls:
    mode: STRICT
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: curl
  namespace: ${TPJ}
spec:
  host: curl
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin
  namespace: ${TPJ}
spec:
  host: httpbin
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL

```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

12. Verify that the **curl** client can reach **httpbin** on both clusters with **STRICT** mTLS enabled by running the following commands:

- a. Verify connectivity on Cluster A:

```
$ oc exec deploy/curl -n "${TPJ}" --kubeconfig="${CLUSTER_A_KUBECONFIG}" -it -- \
curl -s -o /dev/null -w "%{http_code}" http://httpbin
```

- b. Verify connectivity on Cluster B:

```
$ oc exec deploy/curl -n "${TPJ}" --kubeconfig="${CLUSTER_B_KUBECONFIG}" -it -- \
curl -s -o /dev/null -w "%{http_code}" http://httpbin
```

Example output

```
200
```

If you receive an HTTP **200** status code on each cluster, Red Hat OpenShift Service Mesh workloads are communicating under **STRICT** mTLS using SPIRE-issued identities.

13. Remove the test namespace from both clusters by running the following commands:

a. Remove the test namespace from Cluster A:

```
$ oc delete namespace ${TPJ} --kubeconfig="${CLUSTER_A_KUBECONFIG}" --ignore-not-found
```

b. Remove the test namespace from Cluster B:

```
$ oc delete namespace ${TPJ} --kubeconfig="${CLUSTER_B_KUBECONFIG}" --ignore-not-found
```

11.11.10. Deploying east-west gateways

Deploy SPIRE-enabled east-west gateways on both clusters using Helm. Red Hat OpenShift Service Mesh uses east-west gateways to connect cluster networks and enable secure cross-cluster communication in a multi-cluster mesh.

Prerequisites

- You deployed the Istio custom resource with the federation configuration. For more information, see "Deploying the Istio custom resource with the federation configuration".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" and "Deploying SPIFFE Runtime Environment with federation on both clusters" procedures are set.
- Federated **ClusterSPIFFEID** resources exist on both clusters.

Procedure

1. Add the Istio Helm repository by running the following command:

```
$ helm repo add istio https://istio-release.storage.googleapis.com/charts
```

2. Update the Istio Helm repository by running the following command:

```
$ helm repo update
```

3. Grant security context constraints (SCC) permissions on Cluster A by running the following command:

```
$ oc adm policy add-scc-to-user anyuid \
  -z istio-eastwestgateway -n istio-system --kubeconfig="${CLUSTER_A_KUBECONFIG}"
```

4. Grant security context constraints (SCC) permissions on Cluster B by running the following command:

```
$ oc adm policy add-scc-to-user anyuid \
  -z istio-eastwestgateway -n istio-system --kubeconfig="${CLUSTER_B_KUBECONFIG}"
```

5. Install the Istio gateway on Cluster A by running the following command:

```
$ helm upgrade --install istio-eastwestgateway istio/gateway \
  -n istio-system \
  --set-json 'podAnnotations={"inject.istio.io/templates":"gateway,spireGw"}' \
  --set name=istio-eastwestgateway \
  --set networkGateway="${NETWORK_A}" \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}"
```

6. Install the Istio gateway on Cluster B by running the following command:

```
$ helm upgrade --install istio-eastwestgateway istio/gateway \
  -n istio-system \
  --set-json 'podAnnotations={"inject.istio.io/templates":"gateway,spireGw"}' \
  --set name=istio-eastwestgateway \
  --set networkGateway="${NETWORK_B}" \
  --kubeconfig="${CLUSTER_B_KUBECONFIG}"
```

7. Wait for the east-west gateway to become available on Cluster A by running the following command:

```
$ oc wait --for=condition=Available deployment/istio-eastwestgateway \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n istio-system --timeout=300s
```

8. Wait for the east-west gateway to become available on Cluster B by running the following command:

```
$ oc wait --for=condition=Available deployment/istio-eastwestgateway \
  --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n istio-system --timeout=300s
```

9. Create the cross-network **Gateway** custom resource (CR) on Cluster A by running the following command:

- a. Create a YAML file that defines the **Gateway** CR on Cluster A:

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: cross-network-gateway
  namespace: istio-system
spec:
  selector:
    istio: eastwestgateway
  servers:
    - port:
        number: 15443
        name: tls
        protocol: TLS
      tls:
        mode: AUTO_PASSTHROUGH
      hosts:
        - "*.local"
```

- b. Apply the YAML file on Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f <filename>
```

10. Create the cross-network **Gateway** CR on Cluster B by running the following command:

- a. Create a YAML file that defines the **Gateway** CR on Cluster B:

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: cross-network-gateway
  namespace: istio-system
spec:
  selector:
    istio: eastwestgateway
  servers:
  - port:
      number: 15443
      name: tls
      protocol: TLS
    tls:
      mode: AUTO_PASSTHROUGH
    hosts:
      - "*.local"
```

- b. Apply the YAML file on Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f <filename>
```

The **Gateway** CRs configure the east-west gateway deployment to accept cross-cluster TLS traffic on port 15443 using **AUTO_PASSTHROUGH** mode. This preserves SPIRE-issued certificates for end-to-end mTLS.

Verification

1. Verify that the cross-network **Gateway** exists on Cluster A by running the following command:

```
$ oc get gateway cross-network-gateway -n istio-system \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}" \
  -o jsonpath='{.spec.servers[0].tls.mode}'
```

Example output

```
AUTO_PASSTHROUGH
```

2. Verify that the cross-network **Gateway** exists on Cluster B by running the following command:

```
$ oc get gateway cross-network-gateway -n istio-system \
  --kubeconfig="${CLUSTER_B_KUBECONFIG}" \
  -o jsonpath='{.spec.servers[0].tls.mode}'
```

Example output

```
AUTO_PASSTHROUGH
```

11.11.11. Exchanging remote secrets

Create remote secrets on both clusters so **Istiod** can discover services in the peer cluster and route cross-cluster traffic through the east-west gateways.

Prerequisites

- You have deployed the east-west gateway, including the cross-network **Gateway** CR on both clusters. For more information, see "Deploying east-west gateways".
- The environment variables from the "Preparing the environment for multi-cluster SPIFFE Runtime Environment federation" and "Deploying SPIFFE Runtime Environment with federation on both clusters" procedures are set.
- The **istioctl** CLI is available and configured for both clusters.

Procedure

1. Create an Istio remote secret on Cluster A by running the following command:

```
$ istioctl create-remote-secret \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}" \
  --name="${CLUSTER_A}" \
  --istioNamespace=istio-system | \
  oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -f -
```

2. Create an Istio remote secret on Cluster B by running the following command:

```
$ istioctl create-remote-secret \
  --kubeconfig="${CLUSTER_B_KUBECONFIG}" \
  --name="${CLUSTER_B}" \
  --istioNamespace=istio-system | \
  oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -f -
```

3. Verify that the remote cluster is synced on Cluster A by running the following command:

```
$ istioctl remote-clusters --kubeconfig="${CLUSTER_A_KUBECONFIG}"
```

The output must show **\${CLUSTER_B}** with status **syncd**.

Example output

```
NAME      STATUS  SECRET
cluster-b syncd   istio-remote-secret-cluster-b
```

4. Verify that the remote cluster is synced on Cluster B by running the following command:

```
$ istioctl remote-clusters --kubeconfig="${CLUSTER_B_KUBECONFIG}"
```

The output must show **\${CLUSTER_A}** with status **syncd**.

Example output

NAME	STATUS	SECRET
cluster-a	synced	istio-remote-secret-cluster-a

11.11.12. Verifying cross-cluster service communication

Verify cross-cluster service communication between Red Hat OpenShift Service Mesh clusters using sample workloads. This confirms SPIRE-issued identities and federated mesh routing enable end-to-end cross-cluster communication.

Prerequisites

- You have deployed east-west gateways and created the cross-network **Gateway** CR on both clusters.
- You have exchanged remote secrets between clusters.

Procedure

1. Set the sample namespace environment variable by running the following command:

```
$ export SAMPLE_NS=sample
```

2. Create the **sample** namespace on Cluster A by running the following command:

```
$ oc create namespace ${SAMPLE_NS} --kubeconfig="${CLUSTER_A_KUBECONFIG}"
2>/dev/null || true
```

3. Enable Istio injection for the **sample** namespace on Cluster A by running the following command:

```
$ oc label namespace ${SAMPLE_NS} istio-injection=enabled \
--kubeconfig="${CLUSTER_A_KUBECONFIG}" --overwrite
```

4. Create the **sample** namespace on Cluster B by running the following command:

```
$ oc create namespace ${SAMPLE_NS} --kubeconfig="${CLUSTER_B_KUBECONFIG}"
2>/dev/null || true
```

5. Enable Istio injection for the **sample** namespace on Cluster B by running the following command:

```
$ oc label namespace ${SAMPLE_NS} istio-injection=enabled \
--kubeconfig="${CLUSTER_B_KUBECONFIG}" --overwrite
```

6. Install the Istio **HelloWorld Service** in Cluster B by running the following command:

- a. Create a YAML file that defines the **HelloWorld Service** in Cluster B:

```
apiVersion: v1
kind: Service
metadata:
  name: helloworld
```

```

labels:
  app: helloworld
  service: helloworld
spec:
  ports:
  - port: 5000
    name: http
  selector:
    app: helloworld

```

- b. Apply the YAML file in Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n ${SAMPLE_NS} -f
<filename>
```

7. Install the **helloworld-v1 Deployment** in Cluster B by running the following command:

- a. Create a YAML file that defines the **helloworld-v1 Deployment** in Cluster B:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloworld-v1
  labels:
    app: helloworld
    version: v1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helloworld
      version: v1
  template:
    metadata:
      labels:
        app: helloworld
        version: v1
    spec:
      containers:
      - name: helloworld
        image: registry.istio.io/release/examples-helloworld-v1:1.0
        resources:
          requests:
            cpu: "100m"
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 5000

```

- b. Apply the YAML file in Cluster B by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_B_KUBECONFIG}" -n ${SAMPLE_NS} -f
<filename>
```

8. Install the Istio **HelloWorld Service** in Cluster A by running the following command:

- a. Create a YAML file that defines the **HelloWorld Service** in Cluster A:

```
apiVersion: v1
kind: Service
metadata:
  name: helloworld
  labels:
    app: helloworld
    service: helloworld
spec:
  ports:
    - port: 5000
      name: http
  selector:
    app: helloworld
```

- b. Apply the YAML file in Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n ${SAMPLE_NS} -f
<filename>
```

9. Install the **sleep** client in Cluster A by running the following command:

- a. Create a YAML file that defines the **sleep ServiceAccount**, **Service**, and **Deployment** in Cluster A:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sleep
---
apiVersion: v1
kind: Service
metadata:
  name: sleep
  labels:
    app: sleep
    service: sleep
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: sleep
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sleep
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sleep
  template:
```

```

metadata:
  labels:
    app: sleep
spec:
  terminationGracePeriodSeconds: 0
  serviceAccountName: sleep
  containers:
  - name: sleep
    image: docker.io/curlimages/curl:8.16.0
    command: ["/bin/sleep", "infinity"]
    imagePullPolicy: IfNotPresent
    volumeMounts:
    - mountPath: /etc/sleep/tls
      name: secret-volume
  volumes:
  - name: secret-volume
    secret:
      secretName: sleep-secret
      optional: true

```

- b. Apply the YAML file in Cluster A by running the following command:

```
$ oc apply --kubeconfig="${CLUSTER_A_KUBECONFIG}" -n ${SAMPLE_NS} -f
<filename>
```

10. Add the SPIRE injection template to the **sleep** application in Cluster A by running the following command:

```
$ oc patch deploy sleep \
  -n ${SAMPLE_NS} \
  --type='merge' \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}" \
  -p '{"spec": {"template": {"metadata": {"annotations": {"inject.istio.io/templates":
"sidecar,spire"}}}}}'
```

11. Add the SPIRE injection template to the **HelloWorld** application in Cluster B by running the following command:

```
$ oc patch deploy helloworld-v1 \
  -n ${SAMPLE_NS} \
  --type='merge' \
  --kubeconfig="${CLUSTER_B_KUBECONFIG}" \
  -p '{"spec": {"template": {"metadata": {"annotations": {"inject.istio.io/templates":
"sidecar,spire"}}}}}'
```

12. Wait for the **sleep** deployment to become available on Cluster A by running the following command:

```
$ oc rollout status deploy/sleep --kubeconfig "${CLUSTER_A_KUBECONFIG}" -n
${SAMPLE_NS} --timeout=300s
```

13. Wait for the **helloworld-v1** deployment to become available on Cluster B by running the following command:


```
$ oc rollout status deploy/helloworld-v1 --kubeconfig "${CLUSTER_B_KUBECONFIG}" -n
${SAMPLE_NS} --timeout=300s
```

14. Verify that the **sleep** pod uses a SPIRE-issued identity by running the following command:

```
$ oc exec deploy/sleep -n ${SAMPLE_NS} --kubeconfig="${CLUSTER_A_KUBECONFIG}" -
c istio-proxy -- \
  curl -s localhost:15000/certs | jq -r '.certificates[0].cert_chain[0].subject_alt_names[0].uri'
```

Example output

```
spiffe://${CLUSTER_A_TRUST_DOMAIN}/ns/sample/sa/sleep
```

15. Verify that the **sleep** pod on Cluster A can reach the **helloworld.sample** service by running the following command:

```
$ oc exec deploy/sleep \
  -n ${SAMPLE_NS} \
  --kubeconfig="${CLUSTER_A_KUBECONFIG}" \
  -- curl -sS helloworld.sample:5000/hello
```

Example output

```
Hello version: v1, instance: helloworld-v1-5859666d7-pcb8v
```

11.11.13. Additional resources

- [Multi-cluster configuration overview](#)

11.12. ENABLING CREATE-ONLY MODE FOR THE ZERO TRUST WORKLOAD IDENTITY MANAGER

To pause Operator reconciliation, enable **create-only** mode by setting an environment variable in the subscription object. By setting this value, you can perform manual configurations or debug the operator without the controller overwriting your changes.

The following scenarios are examples of when the **create-only** mode might be of use:

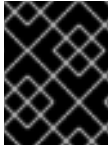
Manual Customization Required: You need to customize operator-managed resources (ConfigMaps, Deployments, DaemonSets, etc.) with specific configurations that differ from the operator's defaults

Day 2 Operations: After initial deployment, you want to prevent the operator from overwriting their manual changes during subsequent reconciliation cycles

Configuration Drift Prevention: You want to maintain control over certain resource configurations while still benefiting from the operator's lifecycle management

11.12.1. Pausing Operator reconciliation

Pause reconciliation of the operands by enabling **create-only** mode. This setting prevents the Operator from automatically reverting your manual changes to the desired state. You can enable this mode by updating the Operator's subscription object.

**IMPORTANT**

When **create-only** mode is disabled, the Operator overwrites the resources if any conflicts exist.

Prerequisites

- You have installed Zero Trust Workload Identity Manager on your machine.
- You have installed the SPIRE Servers, Agents, SPIFFE Container Storage Interface (CSI), and an OpenID Connect (OIDC) Discovery Provider and are in running status.

Procedure

- To pause reconciling the operands resources managed by the Operator, add the environment variable **CREATE_ONLY_MODE: true** in the subscription object by running the following command:

```
$ oc -n $OPERATOR_NAMESPACE patch subscription openshift-zero-trust-workload-identity-manager --type='merge' -p '{"spec":{"config":{"env":[{"name":"CREATE_ONLY_MODE","value":"true"}]}}}'
```

Verification

- Check the status of the **SpireServer** resource to confirm that the **create-only** mode is active. The **status** must be **true** and the **reason** must be **CreateOnlyModeEnabled**.

```
$ oc get SpireServer cluster -o yaml
```

The following is an example that confirms that the 'create-only' mode is active.

```
status:
conditions:
- lastTransitionTime: "2025-12-23T11:36:58Z"
  message: All components are ready
  reason: Ready
  status: "True"
  type: Ready
- lastTransitionTime: "2025-12-23T11:36:58Z"
  message: All operand CRs are ready
  reason: Ready
  status: "True"
  type: OperandsAvailable
- lastTransitionTime: "2025-12-23T11:36:58Z"
  message: create-only mode enabled
  reason: CreateOnlyModeEnabled
  status: "True"
  type: CreateOnlyMode
```

**IMPORTANT**

The Operator updates the upgradeable condition to **false** in the **operatorCondition** resource. You might not be able to upgrade the Operator when in **create-only** mode.

11.12.2. Resuming Operator reconciliation

To resume Operator reconciliation after manual configuration or debugging, disable the **create-only** mode. This allows the controller to resume managing resources and applying the desired state. You can disable this mode by setting the environment variable in the subscription object.

Prerequisites

- You have enabled **create-only** mode on the Zero Trust Workload Identity Manager.
- You have completed your manual configuration or debugging tasks.

Procedure

- To restart reconciling the Operator-managed resources, add the environment variable **CREATE_ONLY_MODE: false** in the subscription object by running the following command:

```
$ oc -n $OPERATOR_NAMESPACE patch subscription openshift-zero-trust-workload-identity-manager --type='merge' -p '{"spec":{"config":{"env":[{"name":"CREATE_ONLY_MODE","value":"false"}]}}}'
```

Verification

- Check the status of the **SpireServer** resource to confirm that **create-only** mode is disabled by running the following command:

```
$ oc get SpireServer cluster -o yaml
```

Example output

```
status:
conditions:
- lastTransitionTime: "2025-12-23T11:40:00Z"
  message: create-only mode disabled
  reason: CreateOnlyModeDisabled
  status: "False"
  type: CreateOnlyMode
```

11.13. SPIRE UPSTREAMAUTHORITY PLUGINS FOR ZERO TRUST WORKLOAD IDENTITY MANAGER

To integrate SPIFFE Runtime Environment (SPIRE) with your existing certificate management infrastructure and keep Secure Production Identity Framework for Everyone (SPIFFE) (SPIFFE) identity standards, configure SPIRE Server with UpstreamAuthority plugins. These plugins obtain intermediate signing certificates from external certificate authorities.

You can configure SPIRE Server to use one of the following UpstreamAuthority plugins:

cert-manager UpstreamAuthority plugin

Integrates SPIRE with cert-manager Operator for Red Hat OpenShift running in Kubernetes or OpenShift Container Platform clusters. The cert-manager Operator for Red Hat OpenShift instance can use various issuer types to provide signing certificates for SPIRE intermediate CAs.

Vault UpstreamAuthority plugin

Integrates SPIRE with the HashiCorp Vault Public Key Infrastructure (PKI) secrets engine. This plugin supports many Vault authentication methods and enables SPIRE to use Vault's security features for certificate management.

Choose the plugin that matches your certificate management infrastructure. You can configure only one UpstreamAuthority plugin at a time. The **SpireServer** CR rejects configurations that specify both **certManager** and **vault** simultaneously.

11.13.1. About the cert-manager upstream authority plugin

The cert-manager Operator for Red Hat OpenShift upstream authority plugin connects SPIRE Server to cert-manager Operator for Red Hat OpenShift for automated intermediate certificate provisioning.

When you configure this plugin, SPIRE Server creates a **CertificateRequest** resource in the cluster. The configured Issuer or ClusterIssuer signs the request and the certificate. The SPIRE Server then uses the signed intermediate certificate to issue workload identities.

11.13.1.1. How the cert-manager plugin works

1. SPIRE Server generates a certificate signing request for an intermediate signing certificate.
2. The plugin creates a **CertificateRequest** in the configured namespace.
3. The **CertificateRequest** references the configured Issuer or ClusterIssuer.
4. cert-manager Operator for Red Hat OpenShift signs the request.
5. SPIRE Server retrieves the signed certificate and CA bundle from the **CertificateRequest**.

11.13.1.2. Requirements

cert-manager Operator for Red Hat OpenShift

cert-manager Operator for Red Hat OpenShift must be installed and running in the cluster.

Issuer

You must configure an **Issuer** or **ClusterIssuer** that can sign intermediate CA certificates.

Permissions

On OpenShift Container Platform, Zero Trust Workload Identity Manager grants the SPIRE Server **ServiceAccount** permission to manage **CertificateRequest** resources when **spec.upstreamAuthority.certManager** is configured. Prepare the Issuer and namespace before you configure the **SpireServer** CR.

Supported issuers

The **Issuer** must support signing certificate requests for intermediate CAs.

11.13.2. Preparing cert-manager for SPIRE Server

Install cert-manager Operator for Red Hat OpenShift and create an **Issuer** or **ClusterIssuer** that can sign SPIRE intermediate certificates. After you complete this procedure, configure **spec.upstreamAuthority.certManager** on the **SpireServer** CR.

Prerequisites

- You are logged in to the cluster with the **cluster-admin** role.
- You have installed Zero Trust Workload Identity Manager or plan to install it after cert-manager Operator for Red Hat OpenShift is ready.

Procedure

1. Create a self-signed bootstrap **ClusterIssuer**:

- a. Save the following manifest as **selfsigned-bootstrap.yaml**:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned-bootstrap
spec:
  selfSigned: {}
```

- b. Apply the **ClusterIssuer** by running the following command:

```
$ oc apply -f selfsigned-bootstrap.yaml
```

2. Use the bootstrap **ClusterIssuer** to issue a root CA certificate into a Secret:

- a. Save the following manifest as **spire-root-ca.yaml**:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: spire-root-ca
  namespace: cert-manager
spec:
  isCA: true
  secretName: spire-root-ca-secret
  issuerRef:
    name: selfsigned-bootstrap
    kind: ClusterIssuer
  commonName: "SPIRE Root CA"
  duration: 87600h
```

- b. Apply the **Certificate** by running the following command:

```
$ oc apply -f spire-root-ca.yaml
```

3. Create a CA **Issuer** that references the root CA Secret:

- a. Save the following manifest as **spire-ca-issuer.yaml**:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: spire-ca
  namespace: cert-manager
```

```
spec:
  ca:
    secretName: spire-root-ca-secret
```

- b. Apply the **Issuer** by running the following command:

```
$ oc apply -f spire-ca-issuer.yaml
```

Verification

- Confirm that the bootstrap **ClusterIssuer**, root CA **Certificate**, and signing **Issuer** are ready by running the following commands:

```
$ oc get clusterissuer selfsigned-bootstrap
```

```
$ oc get certificate spire-root-ca -n cert-manager
```

```
$ oc get issuer spire-ca -n cert-manager
```

Additional resources

- [cert-manager Operator for Red Hat OpenShift](#)

11.13.3. Configuring the cert-manager upstream authority plugin

Configure SPIRE Server to obtain intermediate signing certificates from cert-manager Operator for Red Hat OpenShift by setting **spec.upstreamAuthority.certManager** on the **SpireServer** custom resource. Zero Trust Workload Identity Manager generates SPIRE Server configuration and reconciles the SPIRE Server `StatefulSet`.

Prerequisites

- You have installed Zero Trust Workload Identity Manager and deployed a **SpireServer** CR.
- You have completed preparing cert-manager Operator for Red Hat OpenShift for SPIRE Server use, including creating an **Issuer** or **ClusterIssuer**.

Procedure

- Export the current **SpireServer** CR to a file by running the following command:

```
$ oc get spireserver cluster -o yaml > SpireServer-cert-manager.yaml
```

- In the **SpireServer-cert-manager.yaml**, add the **upstreamAuthority** section under **spec::**

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
```

```

jwtIssuer: "https://oidc-discovery.apps.cluster.example.com"
caValidity: "24h"
defaultX509Validity: "1h"
defaultJWTValidity: "5m"
jwtKeyType: "rsa-2048"
caSubject:
  country: "US"
  organization: "Example Corporation"
  commonName: "SPIRE Server CA"
persistence:
  size: "5Gi"
  accessMode: "ReadWriteOnce"
  storageClass: "gp3-csi"
datastore:
  databaseType: "sqlite3"
  connectionString: "/run/spire/data/datastore.sqlite3"
  tlsSecretName: ""
  maxOpenConns: 100
  maxIdleConns: 10
  connMaxLifetime: 0
  disableMigration: "false"
upstreamAuthority:
  certManager:
    namespace: cert-manager
    issuerName: spire-ca
    issuerKind: Issuer
    issuerGroup: cert-manager.io

```

where:

spec.upstreamAuthority.certManager.namespace

Specifies the namespace where SPIRE Server creates **CertificateRequest** resources. For a namespace-scoped **Issuer**, this must match the **Issuer** namespace. For a **ClusterIssuer**, any namespace is valid.

spec.upstreamAuthority.certManager.issuerName

Specifies the name of the **Issuer** or **ClusterIssuer**.

spec.upstreamAuthority.certManager.issuerKind

Specifies the **Issuer** or **ClusterIssuer**. The default is **Issuer**.

spec.upstreamAuthority.certManager.issuerGroup

Specifies the API group of the issuer. The default is **cert-manager.io**.



NOTE

On OpenShift Container Platform, SPIRE Server uses its in-cluster **ServiceAccount**. Zero Trust Workload Identity Manager grants **CertificateRequest** permissions when **spec.upstreamAuthority.certManager** is configured.

3. Apply the updated CR by running the following command:

```
$ oc apply -f SpireServer-cert-manager.yaml
```

- Wait for Zero Trust Workload Identity Manager to reconcile the SPIRE Server by running the following command:

```
$ oc rollout status statefulset/spire-server -n zero-trust-workload-identity-manager
```

Verification

- Verify that SPIRE Server is healthy and that logs show the **UpstreamAuthority** plugin loaded by running the following commands:

```
$ oc exec -n zero-trust-workload-identity-manager statefulset/spire-server -c spire-server -- \
/opt/spire/bin/spire-server healthcheck
```

```
$ oc logs statefulset/spire-server -n zero-trust-workload-identity-manager -c spire-server --tail=50
```

- Confirm that a cert-manager-signed intermediate certificate is present by running the following command:

```
$ oc exec -n zero-trust-workload-identity-manager statefulset/spire-server -c spire-server -- \
/opt/spire/bin/spire-server bundle show
```

11.13.4. cert-manager upstream authority plugin reference

This reference describes **spec.upstreamAuthority.certManager** fields on the **SpireServer** custom resource and how Zero Trust Workload Identity Manager uses them to request intermediate certificates from cert-manager Operator for Red Hat OpenShift. Use it when you configure or troubleshoot the cert-manager Operator for Red Hat OpenShift UpstreamAuthority plugin and need field descriptions or defaults.

11.13.4.1. SpireServer CR fields

Configure cert-manager Operator for Red Hat OpenShift upstream authority under **spec.upstreamAuthority.certManager**. Zero Trust Workload Identity Manager generates the SPIRE Server configuration from these fields.

Field	Description
namespace	Required. Namespace where SPIRE Server creates CertificateRequest resources.
issuerName	Required. Name of the Issuer or ClusterIssuer.
issuerKind	Optional. Issuer or ClusterIssuer . The default is Issuer .
issuerGroup	Optional. API group of the issuer. The default is cert-manager.io .



NOTE

On OpenShift Container Platform, SPIRE Server uses its in-cluster **ServiceAccount**. Zero Trust Workload Identity Manager grants permissions to create, get, list, and delete **CertificateRequest** resources when **spec.upstreamAuthority.certManager** is configured.

11.13.5. Troubleshooting the cert-manager Operator for Red Hat OpenShift upstream authority plugin

Resolve the most common cert-manager Operator for Red Hat OpenShift upstream authority failures on OpenShift Container Platform.

11.13.5.1. Quick reference

Symptom or error	Likely cause	Section
certificaterequests... is forbidden	Missing permissions or wrong namespace	Section 11.13.5.2, "Permission and issuer errors"
issuer ... not found	Wrong issuerName , issuerKind , or issuer namespace	Section 11.13.5.2, "Permission and issuer errors"
Issuer not READY	Issuer misconfiguration	Section 11.13.5.2, "Permission and issuer errors"
CertificateRequest not approved or denied	Approval policy or approver configuration	Section 11.13.5.3, "SPIRE Server errors"
UpstreamAuthority fails to load in SPIRE Server logs	Invalid or incomplete spec.upstreamAuthority.certManager	Section 11.13.5.3, "SPIRE Server errors"

11.13.5.2. Permission and issuer errors

Permission errors

- Confirm **spec.upstreamAuthority.certManager.namespace** matches the namespace where SPIRE Server creates **CertificateRequest** resources.
- After you configure **spec.upstreamAuthority.certManager**, verify that Zero Trust Workload Identity Manager updated the SPIRE Server role-based access control (RBAC) and the SPIRE Server pod restarted.

Issuer errors

- Verify **issuerName** and **issuerKind** on the **SpireServer** CR match an existing Issuer or ClusterIssuer.
- For a namespace-scoped **Issuer**, the Issuer must exist in the namespace referenced by **CertificateRequest** resources or in the namespace where the issuer is defined, depending on your issuer configuration.
- Check that the Issuer reports **READY=True**:

```
$ oc get issuer,clusterissuer -A
$ oc describe issuer spire-ca -n cert-manager
```

11.13.5.3. SPIRE Server errors

SPIRE Server errors

- Confirm that **namespace**, **issuerName**, and **issuerKind** are set under **spec.upstreamAuthority.certManager**.
- Review SPIRE Server logs by running the following command:

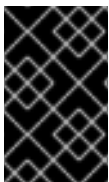
```
$ oc logs statefulset/spire-server -n zero-trust-workload-identity-manager -c spire-server --tail=100
```

11.13.6. About the SPIRE Vault UpstreamAuthority plugin

The Vault UpstreamAuthority plugin connects SPIRE Server to the HashiCorp Vault PKI secrets engine for automated intermediate CA certificate signing. Use this plugin when you centralize PKI in Vault and want SPIRE to obtain intermediate signing certificates through Vault policies and authentication method.

The SPIRE Vault UpstreamAuthority plugin provides the following features:

- Integration with HashiCorp Vault PKI secrets engine as an upstream certificate authority
- Support for **Kubernetes auth** Vault authentication methods
- Automatic signing of SPIRE intermediate CA certificates
- Vault Enterprise namespace support
- Secure certificate management using Vault's security features



IMPORTANT

The Vault UpstreamAuthority plugin does not support the **PublishJWTKey** remote procedure call (RPC) and is not appropriate for use in nested SPIRE topologies where JSON Web Token SPIFFE Verifiable Identity Document (SVID) (JWT-SVIDs) are used.

11.13.6.1. Supported authentication method

The Vault UpstreamAuthority plugin supports only the following authentication methods for connecting to Vault:

Kubernetes authentication

Uses Kubernetes service account tokens to authenticate to Vault.

11.13.6.2. Prerequisites and requirements

Before configuring the Vault UpstreamAuthority plugin, ensure the following requirements are met:

Vault PKI secrets engine

A running HashiCorp Vault instance with the PKI secrets engine enabled at a configured mount point (default: **pki**). The PKI secrets engine must have a root CA certificate configured for signing intermediate certificates.

Vault policy

A Vault policy that grants the **update** capability for the **pki/root/sign-intermediate** endpoint, attached to the credentials SPIRE Server uses.

Authentication credentials

Valid credentials for one of the supported authentication methods, associated with the Vault signing policy.

TTL configuration

SPIRE Server **ca_ttl** must not exceed the Vault PKI secrets engine maximum lease TTL.

Network connectivity

SPIRE Server must reach the Vault server over the network and trust the Vault TLS certificate when TLS is enabled.

Kubernetes RBAC (when using Kubernetes authentication)

A **ServiceAccount** for SPIRE Server bound to the Vault Kubernetes auth role. The SPIRE Server **ServiceAccount** requires no additional RBAC. However, the Vault **ServiceAccount** must have **system:auth-delegator** permissions to validate projected tokens via the Kubernetes **TokenReview** API.

11.13.7. Configuring the SPIRE Vault UpstreamAuthority plugin

Configure SPIRE Server to obtain intermediate signing certificates from HashiCorp Vault by setting **spec.upstreamAuthority.vault** on the **SpireServer** custom resource (CR). Zero Trust Workload Identity Manager generates the SPIRE Server configuration, mounts Vault credentials into the SPIRE Server pod, and reconciles the SPIRE Server **StatefulSet**.

On OpenShift Container Platform, the **SpireServer** CR supports Vault authentication only through the Kubernetes auth method. Zero Trust Workload Identity Manager mounts a projected SPIRE Server **ServiceAccount** token and, when configured, a Vault CA certificate Secret at fixed paths inside the pod.

Prerequisites

- You have installed Zero Trust Workload Identity Manager and deployed a **SpireServer** CR.
- You have a running HashiCorp Vault instance with the PKI secrets engine enabled at your mount point (default: **pki**) and a root CA configured to sign intermediate certificates.
- You have a Vault policy that grants the **update** capability on **<pki_mount>/root/sign-intermediate**.
- You have configured the Vault Kubernetes authentication method and created a Vault role bound to the SPIRE Server **ServiceAccount**. For example, **spire-server** in the **zero-trust-workload-identity-manager** namespace.

- The value of **spec.caValidity** on the **SpireServer** CR is less than or equal to the maximum lease Time to Live (TTL) configured on the Vault PKI secrets engine.
- If Vault uses a private or custom TLS certificate authority, you have a PEM-encoded CA certificate available to store in a Kubernetes **Secret**.

Procedure

1. Export the current **SpireServer** CR to a file by running the following command:

```
$ oc get spireserver cluster -o yaml > SpireServer-vault.yaml
```

2. In **SpireServer-vault.yaml**, add the **upstreamAuthority** section from the following example under **spec::**

```
apiVersion: operator.openshift.io/v1alpha1
kind: SpireServer
metadata:
  name: cluster
spec:
  logLevel: "info"
  logFormat: "text"
  jwtIssuer: "https://oidc-discovery.apps.cluster.example.com"
  caValidity: "24h"
  defaultX509Validity: "1h"
  defaultJWTValidity: "5m"
  jwtKeyType: "rsa-2048"
  caSubject:
    country: "US"
    organization: "Example Corporation"
    commonName: "SPIRE Server CA"
  persistence:
    size: "5Gi"
    accessMode: "ReadWriteOnce"
    storageClass: "gp3-csi"
  datastore:
    databaseType: "sqlite3"
    connectionString: "/run/spire/data/datastore.sqlite3"
    tlsSecretName: ""
    maxOpenConns: 100
    maxIdleConns: 10
    connMaxLifetime: 0
    disableMigration: "false"
  upstreamAuthority:
    vault:
      vaultAddr: "https://vault.example.com:8200"
      pkiMountPoint: "pki"
      caCertSecretRef:
        name: vault-ca-cert
        key: ca.crt
      k8sAuth:
        k8sAuthMountPoint: "kubernetes"
        k8sAuthRoleName: "spire-server"
        audience: "vault"
      vaultNamespace: "vault-namespace" # optional
```



NOTE

For in-cluster Vault over HTTP, set **vaultAddr** to the in-cluster service URL, such as <http://vault.vault.svc:8200>, and omit **caCertSecretRef**.

Include **caCertSecretRef** only when Vault TLS is signed by a custom CA. Omit it when Vault uses a public CA.

where:

spec.caValidity

Specifies the SPIRE Server CA validity. Must be less than or equal to the Vault PKI **max_lease_ttl**. Zero Trust Workload Identity Manager maps this value to SPIRE **ca_ttl**.

spec.upstreamAuthority.vault.vaultAddr

Specifies the URL of the Vault server.

spec.upstreamAuthority.vault.pkiMountPoint

Specifies the Vault PKI secrets engine mount path. Default: **pki**.

spec.upstreamAuthority.vault.caCertSecretRef

Optional. Specifies the **Secret** reference when Vault TLS is signed by a custom CA. Zero Trust Workload Identity Manager mounts the Secret at **/run/spire/upstream-ca/ca.crt**.

spec.upstreamAuthority.vault.k8sAuth.k8sAuthRoleName

Specifies the Vault Kubernetes auth role name.

spec.upstreamAuthority.vault.k8sAuth.k8sAuthMountPoint

Specifies the Vault Kubernetes auth mount path. The default is **kubernetes**.

spec.upstreamAuthority.vault.k8sAuth.audience

Specifies the projected **ServiceAccount** token audience. This must match the Vault role. The default is **vault**.

spec.upstreamAuthority.vault.vaultNamespace

Optional. Specifies the Vault namespace name.

3. Apply the updated CR by running the following command:

```
$ oc apply -f SpireServer-vault.yaml
```

4. Wait for Zero Trust Workload Identity Manager to reconcile the SPIRE Server by running the following command:

```
$ oc rollout status statefulset/spire-server -n zero-trust-workload-identity-manager
```

Verification

1. Verify that SPIRE Server is healthy and that logs show the UpstreamAuthority plugin loaded by running the following commands:

```
$ oc exec -n zero-trust-workload-identity-manager statefulset/spire-server -c spire-server -- \
/opt/spire/bin/spire-server healthcheck
$ oc logs statefulset/spire-server -n zero-trust-workload-identity-manager -c spire-server --
tail=50
```

Example output

```
Server is healthy.
time="2026-04-07T10:15:30Z" level=info msg="Upstream authority loaded"
subsystem_name=ca
time="2026-04-07T10:15:31Z" level=info msg="Server CA activated"
certificate_fingerprint="ABC123..."
```

2. Confirm that a Vault-signed intermediate certificate is present by running the following command:

```
$ oc exec -n zero-trust-workload-identity-manager statefulset/spire-server -c spire-server -- \
/opt/spire/bin/spire-server bundle show
```

11.13.8. SPIRE Vault UpstreamAuthority plugin reference

Reference for **spec.upstreamAuthority.vault** on the **SpireServer** CR and the Vault signing policy SPIRE Server requires.

11.13.8.1. SpireServer CR fields

Configure Vault upstream authority under **spec.upstreamAuthority.vault**. Zero Trust Workload Identity Manager generates SPIRE Server configuration from these fields.

Field	Description
vaultAddr	Required. Vault server URL. Use HTTPS for external endpoints; HTTP is permitted for in-cluster services.
pkiMountPoint	PKI secrets engine mount path. Default: pki .
caCertSecretRef	Optional Secret reference in the zero-trust-workload-identity-manager namespace. Use when Vault TLS is signed by a custom CA. Zero Trust Workload Identity Manager mounts the key at /run/spire/upstream-ca/ca.crt .
insecureSkipVerify	Optional. Accepts any Vault server certificate when true . Default: false . Do not enable insecureSkipVerify in production. This setting disables TLS certificate verification and exposes the connection to man-in-the-middle attacks. Troubleshooting only.
vaultNamespace	Optional. Vault Enterprise namespace.
k8sAuth.k8sAuthMountPoint	Vault Kubernetes auth mount path. Default: kubernetes .
k8sAuth.k8sAuthRoleName	Required. Vault role bound to the spire-server ServiceAccount.

Field	Description
k8sAuth.audience	Token audience for the projected ServiceAccount token. Default: vault . Must match the bound_audiences configured on the Vault role.



NOTE

On OpenShift Container Platform, the **SpireServer** CR supports Vault Kubernetes authentication only. Zero Trust Workload Identity Manager mounts a projected **ServiceAccount** token at **/var/run/secrets/tokens/vault**.

Ensure that **spec.caValidity** does not exceed the Vault PKI **max_lease_ttl**.

11.13.8.2. Required Vault policy

The Vault Kubernetes auth role must include a policy with **update** on the intermediate signing path:

```
path "pki/root/sign-intermediate" {
  capabilities = ["update"]
}
```

Replace **pki** with your PKI mount point when different.

11.13.9. Troubleshooting SPIRE Vault UpstreamAuthority plugin

Resolve the most common SPIRE Vault upstream authority failures on OpenShift Container Platform.

11.13.9.1. Quick reference

Symptom or error	Likely cause	Section
Connection refused, timeout, or unreachable Vault	Wrong vaultAddr or network path from the SPIRE Server pod	Section 11.13.9.2, "Connection and TLS errors"
x509: certificate signed by unknown authority	Missing or invalid caCertSecretRef Secret	Section 11.13.9.2, "Connection and TLS errors"
Vault 403 or Kubernetes auth failure	Vault role, ServiceAccount binding, or signing policy misconfiguration	Section 11.13.9.3, "Authentication and signing errors"

Symptom or error	Likely cause	Section
requested TTL ... exceeds max_lease_ttl	caValidity exceeds the Vault PKI limit	Section 11.13.9.4, "TTL mismatch errors"

11.13.9.2. Connection and TLS errors

Connection failures

- Verify **spec.upstreamAuthority.vault.vaultAddr** on the **SpireServer** CR.
- Confirm the SPIRE Server pod can reach Vault from the **zero-trust-workload-identity-manager** namespace.
- For in-cluster Vault, use the Kubernetes service URL, such as <http://vault.vault.svc:8200>.

TLS failures

- When Vault uses a custom CA, set **caCertSecretRef** to a Secret in the **zero-trust-workload-identity-manager** namespace with a PEM-encoded CA certificate.
- Confirm the **Secret** name and key match **caCertSecretRef**.
- Omit **caCertSecretRef** only for a public CA or in-cluster HTTP.
- Do not use **insecureSkipVerify: true** in production.

11.13.9.3. Authentication and signing errors

SPIRE Server uses Vault Kubernetes authentication with the **spire-server** ServiceAccount.

Check the following:

- **k8sAuth.k8sAuthRoleName** on the **SpireServer** CR matches the Vault Kubernetes auth role.
- The Vault role binds to ServiceAccount **spire-server** in namespace **zero-trust-workload-identity-manager**.
- The role policy grants **update** on **<pki_mount>/root/sign-intermediate**.
- **pkiMountPoint** matches the PKI secrets engine mount in Vault.
- **k8sAuth.audience** matches the Vault role when you changed the default from **vault**.

Review SPIRE Server logs:

```
$ oc logs statefulset/spire-server -n zero-trust-workload-identity-manager -c spire-server --tail=100
```

11.13.9.4. TTL mismatch errors

spec.caValidity on the **SpireServer** CR must be less than or equal to the Vault PKI **max_lease_ttl**.

- Reduce **caValidity** on the **SpireServer** CR, or increase the Vault PKI limit with **vault secrets tune -max-lease-ttl=...**

For Vault Enterprise namespace errors, set **spec.upstreamAuthority.vault.vaultNamespace** to the correct namespace path.

11.14. MONITORING ZERO TRUST WORKLOAD IDENTITY MANAGERGIT

Track the performance of the Zero Trust Workload Identity Manager by collecting metrics. Configure monitoring to collect metrics from the Security Production Identity Framework for Everyone (SPIRE) Server and SPIRE Agent components.

11.14.1. Enabling user workload monitoring

Enable user workload monitoring to track metrics for your user-defined projects. Configuring this feature allows you to observe application performance and helps you maintain the health of your services.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** cluster role.

Procedure

1. Create the **cluster-monitoring-config.yaml** file to define and configure the **ConfigMap**:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

2. Apply the **ConfigMap** by running the following command:

```
$ oc apply -f cluster-monitoring-config.yaml
```

Verification

- Verify that the monitoring components for user workloads are running in the **openshift-user-workload-monitoring** namespace:

```
$ oc -n openshift-user-workload-monitoring get pod
```

Example output

```
NAME                                READY STATUS RESTARTS AGE
prometheus-operator-6cb6bd9588-dtzxq 2/2   Running 0      50s
prometheus-user-workload-0           6/6   Running 0      48s
```

prometheus-user-workload-1	6/6	Running	0	48s
thanos-ruler-user-workload-0	4/4	Running	0	42s
thanos-ruler-user-workload-1	4/4	Running	0	42s

The status of the pods such as **prometheus-operator**, **prometheus-user-workload**, and **thanos-ruler-user-workload** must be **Running**.

Additional resources

- [Setting up metrics collection for user-defined projects](#)

11.14.2. Configuring metrics collection for SPIRE Server by using a ServiceMonitor

To collect custom metrics from the SPIRE Server, create a ServiceMonitor custom resource (CR). This configuration enables the Prometheus Operator to scrape metrics from the default endpoint, which helps you monitor your SPIRE deployment.

The SPIRE Server operand exposes metrics by default on port **9402** at the **/metrics** endpoint. You can configure metrics collection for the SPIRE Server by creating a **ServiceMonitor** custom resource (CR) that enables the Prometheus Operator to collect custom metrics.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** cluster role.
- You have installed the Zero Trust Workload Identity Manager.
- You have deployed the SPIRE Server operand in the cluster.
- You have enabled the user workload monitoring.

Procedure

1. Create the **ServiceMonitor** CR:
 - a. Create the YAML file that defines the **ServiceMonitor** CR:

Example servicemonitor-spire-server file

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app.kubernetes.io/name: server
    app.kubernetes.io/instance: spire
  name: spire-server-metrics
  namespace: zero-trust-workload-identity-manager
spec:
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
  selector:
    matchLabels:
      app.kubernetes.io/name: server
```

```

    app.kubernetes.io/instance: spire
  namespaceSelector:
    matchNames:
      - zero-trust-workload-identity-manager

```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc create -f servicemonitor-spire-server.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance begins metrics collection from the SPIRE Server. The collected metrics are labeled with **job="spire-server"**.

Verification

1. In the OpenShift Container Platform web console, navigate to **Observe → Targets**.
2. In the **Label** filter field, enter the following label to filter the metrics targets:

```
$ service=zero-trust-workload-identity-manager-metrics-service
```

3. Confirm that the **Status** column shows **Up** for the **spire-server-metrics** entry.

Additional resources

- [Configuring user workload monitoring](#)

11.14.3. Configuring metrics collection for SPIRE Agent by using a Service Monitor

Configure metrics collection for the SPIRE Agent by creating a **ServiceMonitor** custom resource (CR). This enables the Prometheus Operator to collect custom metrics that the SPIRE Agent exposes on the default port.

The SPIRE Agent operand exposes metrics by default on port **9402** at the **/metrics** endpoint. You can configure metrics collection for the SPIRE Agent by creating a **ServiceMonitor** custom resource (CR), which enables the Prometheus Operator to collect custom metrics.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** cluster role.
- You have installed the Zero Trust Workload Identity Manager.
- You have deployed the SPIRE Agent operand in the cluster.
- You have enabled the user workload monitoring.

Procedure

1. Create the **ServiceMonitor** CR:
 - a. Create the YAML file that defines the **ServiceMonitor** CR:

Example servicemonitor-spire-agent.yaml file

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app.kubernetes.io/name: agent
    app.kubernetes.io/instance: spire
  name: spire-agent-metrics
  namespace: zero-trust-workload-identity-manager
spec:
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
  selector:
    matchLabels:
      app.kubernetes.io/name: agent
      app.kubernetes.io/instance: spire
    namespaceSelector:
      matchNames:
        - zero-trust-workload-identity-manager

```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc create -f servicemonitor-spire-agent.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance begins metrics collection from the SPIRE Agent. The collected metrics are labeled with **job="spire-agent"**.

Verification

1. In the OpenShift Container Platform web console, navigate to **Observe → Targets**.
2. In the **Label** filter field, enter the following label to filter the metrics targets:

```
$ service=spire-agent
```

3. Confirm that the **Status** column shows **Up** for the **spire-agent-metrics** entry.

11.14.4. Configuring metrics collection for the Operator by using a ServiceMonitor

The Zero Trust Workload Identity Manager exposes metrics by default on port 8443 at the **/metrics** service endpoint. You can configure metrics collection for the Operator by creating a **ServiceMonitor** custom resource (CR) that enables the Prometheus Operator to collect custom metrics. For more information, see "Configuring user workload monitoring".

The SPIRE Server operand exposes metrics by default on port **9402** at the **/metrics** endpoint. You can configure metrics collection for the SPIRE Server by creating a **ServiceMonitor** custom resource (CR) that enables the Prometheus Operator to collect custom metrics.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** cluster role.

- You have installed the Zero Trust Workload Identity Manager.
- You have enabled the user workload monitoring.

Procedure

1. Configure the Operator to use HTTP or HTTPS protocols for the metrics server.
 - a. Update the subscription object for Zero Trust Workload Identity Manager to configure the HTTP protocol by running the following command:

```
$ oc -n zero-trust-workload-identity-manager patch subscription zero-trust-workload-identity-manager-subscription --type='merge' -p '{"spec":{"config":{"env":[{"name":"METRICS_BIND_ADDRESS","value":":8080"}, {"name":"METRICS_SECURE", "value": "false"}]}}}'
```

- b. Verify the Zero Trust Workload Identity Manager pod is redeployed and that the configured values for **METRICS_BIND_ADDRESS** and **METRICS_SECURE** is updated by running the following command:

```
$ oc set env --list deployment/zero-trust-workload-identity-manager-controller-manager -n zero-trust-workload-identity-manager | grep -e METRICS_BIND_ADDRESS -e METRICS_SECURE -e container
```

Example output

```
deployments/zero-trust-workload-identity-manager-controller-manager, container manager
METRICS_BIND_ADDRESS=:8080
METRICS_SECURE=false
```

2. Create the **Secret** resource with **kubernetes.io/service-account.name** annotation to inject the token required for authenticating with the metrics server.
 - a. Create the **secret-zero-trust-workload-identity-manager.yaml** YAML file:

```
apiVersion: v1
kind: Secret
metadata:
  labels:
    name: zero-trust-workload-identity-manager
    name: zero-trust-workload-identity-manager-metrics-auth
  namespace: zero-trust-workload-identity-manager
  annotations:
    kubernetes.io/service-account.name: zero-trust-workload-identity-manager-controller-manager
  type: kubernetes.io/service-account-token
```

- b. Create the **Secret** resource by running the following command:

```
$ oc apply -f secret-zero-trust-workload-identity-manager.yaml
```

3. Create the **ClusterRoleBinding** resource required for granting permissions to access the metrics.

- a. Create the **clusterrolebinding-zero-trust-workload-identity-manager.yaml** YAML file:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  labels:
    name: zero-trust-workload-identity-manager
    name: zero-trust-workload-identity-manager-allow-metrics-access
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: zero-trust-workload-identity-manager-metrics-reader
subjects:
- kind: ServiceAccount
  name: zero-trust-workload-identity-manager-controller-manager
  namespace: zero-trust-workload-identity-manager
```

- b. Create the **ClusterRoleBinding** resource by running the following command:

```
$ oc apply -f clusterrolebinding-zero-trust-workload-identity-manager.yaml
```

4. Create the following **ServiceMonitor** CR if the metrics server is configured to use **http**.

- a. Create the **servicemonitor-zero-trust-workload-identity-manager-http.yaml** YAML file:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    name: zero-trust-workload-identity-manager
    name: zero-trust-workload-identity-manager-metrics-monitor
    namespace: zero-trust-workload-identity-manager
spec:
  endpoints:
    - authorization:
        credentials:
          name: zero-trust-workload-identity-manager-metrics-auth
          key: token
        type: Bearer
      interval: 60s
      path: /metrics
      port: metrics-http
      scheme: http
      scrapeTimeout: 30s
    namespaceSelector:
      matchNames:
        - zero-trust-workload-identity-manager
    selector:
      matchLabels:
        name: zero-trust-workload-identity-manager
```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-zero-trust-workload-identity-manager-http.yaml
```

5. Create the following **ServiceMonitor** CR if the metrics server is configured to use **https**.

a. Create the **servicemonitor-zero-trust-workload-identity-manager-https.yaml** YAML file:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    name: zero-trust-workload-identity-manager
    name: zero-trust-workload-identity-manager-metrics-monitor
    namespace: zero-trust-workload-identity-manager
spec:
  endpoints:
    - authorization:
        credentials:
          name: zero-trust-workload-identity-manager-metrics-auth
          key: token
        type: Bearer
      interval: 60s
      path: /metrics
      port: metrics-https
      scheme: https
      scrapeTimeout: 30s
      tlsConfig:
        ca:
          configMap:
            name: openshift-service-ca.crt
            key: service-ca.crt
          serverName: zero-trust-workload-identity-manager-metrics-service.zero-trust-
workload-identity-manager.svc.cluster.local
      namespaceSelector:
        matchNames:
          - zero-trust-workload-identity-manager
      selector:
        matchLabels:
          name: zero-trust-workload-identity-manager
```

b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-zero-trust-workload-identity-manager-https.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance begins metrics collection from the SPIRE Server. The collected metrics are labeled with **job="zero-trust-workload-identity-manager-metrics-service"**.

Verification

1. In the OpenShift Container Platform web console, navigate to **Observe → Targets**.
2. In the **Label** filter field, enter the following label to filter the metrics targets:

```
$ service=zero-trust-workload-identity-manager-metrics-service
```

3. Confirm that the **Status** column shows **Up** for the **zero-trust-workload-identity-manager** entry.

Additional resources

- [Configuring user workload monitoring](#)

11.14.5. Querying metrics for the Zero Trust Workload Identity Manager

Query SPIRE Agent and SPIRE Server metrics using the OpenShift Container Platform web console or the command line. This helps you monitor the performance of SPIRE components that match specific job labels.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the Zero Trust Workload Identity Manager.
- You have deployed the SPIRE Server and SPIRE Agent operands in the cluster.
- You have enabled monitoring and metrics collection by creating **ServiceMonitor** objects.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Observe → Metrics**.
2. In the query field, enter the following PromQL expression to query SPIRE Server metrics:

```
{job="spire-server"}
```

3. In the query field, enter the following PromQL expression to query SPIRE Agent metrics.

```
{job="spire-agent"}
```

Additional resources

- [Accessing metrics](#)

11.14.6. Zero Trust Workload Identity Manager monitoring available metrics

Monitor the health and performance of Zero Trust Workload Identity Manager components by reviewing exposed metrics. This reference describes controller, certificate, and runtime metrics that help you maintain system health and troubleshoot errors.

The Zero Trust Workload Identity Manager exposes the following metrics:

Controller runtime metrics

- **controller_runtime_active_workers**: Number of currently used workers per controller
- **controller_runtime_max_concurrent_reconciles**: Maximum number of concurrent reconciles per controller
- **controller_runtime_reconcile_errors_total**: Total number of reconciliation errors per controller

- **controller_runtime_reconcile_time_seconds**: Length of time per reconciliation per controller
- **controller_runtime_reconcile_total**: Total number of reconciliations per controller

Certificate watcher metrics

- **certwatcher_read_certificate_errors_total**: Total number of certificate read errors
- **certwatcher_read_certificate_total**: Total number of certificates read

Go runtime metrics

Standard Go runtime metrics including:

- **go_gc_duration_seconds**: Garbage collection duration
- **go_goroutines**: Number of goroutines
- **go_memstats_***: Memory statistics
- **process_***: Process statistics

Custom Operator metrics

The operator also exposes custom metrics related to:

- SPIRE Server status and health
- SPIRE Agent deployment status
- SPIFFE CSI Driver status
- OIDC Discovery Provider status
- Workload identity management operations

11.15. UNINSTALLING THE ZERO TRUST WORKLOAD IDENTITY MANAGER

To remove the Zero Trust Workload Identity Manager from OpenShift Container Platform, uninstall the Operator and delete its related resources. This process removes the component from your cluster.

11.15.1. Uninstalling the Zero Trust Workload Identity Manager

To remove the Zero Trust Workload Identity Manager from your cluster, uninstall the Operator using the web console. This helps you clean up resources and delete the service from your environment.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.
- The Zero Trust Workload Identity Manager is installed.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Uninstall the Zero Trust Workload Identity Manager.
 - a. Go to **Ecosystem → Installed Operators**.
 - b. Click the **Options** menu next to the **Zero Trust Workload Identity Manager** entry, and then click **Uninstall Operator**.
 - c. In the confirmation dialog, click **Uninstall**.

Verification

- Verify that the Zero Trust Workload Identity Manager Operator is uninstalled.

```
$ oc get csv -n openshift-zero-trust-workload-identity
```

Example output

```
No resources found in openshift-zero-trust-workload-identity namespace.
```

11.15.2. Uninstalling Zero Trust Workload Identity Manager resources by using the CLI

Remove Zero Trust Workload Identity Manager resources from your cluster using the CLI. This deletes the remaining operands and definitions to help ensure a clean environment after you uninstall the product.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Uninstall the operands by running each of the following commands:
 - a. Delete the **SpireOIDCDiscoveryProvider** cluster by running the following command:

```
$ oc delete SpireOIDCDiscoveryProvider cluster
```

- b. Delete the **SpiffeCSIDriver** cluster by running the following command:

```
$ oc delete SpiffeCSIDriver cluster -l
```

- c. Delete the **SpireAgent** cluster by running the following command:

```
$ oc delete SpireAgent cluster
```

- d. Delete the **SpireServer** cluster by running the following command:

```
$ oc delete SpireServer cluster
```

- e. Delete the **ZeroTrustWorkloadIdentityManager** cluster by running the following command:

```
$ oc delete ZeroTrustWorkloadIdentityManager cluster
```

- f. Delete the persistent volume claim (PVC) by running the following command:

```
$ oc delete pvc -l=app.kubernetes.io/name=spire-server
```

- g. Delete the service by running the following command:

```
$ oc delete service -l=app.kubernetes.io/name=zero-trust-workload-identity-manager -n zero-trust-workload-identity-manager
```

- h. Delete the namespace by running the following command:

```
$ oc delete ns zero-trust-workload-identity-manager
```

- i. Delete the cluster role by running the following command:

```
$ oc delete clusterrole -l=app.kubernetes.io/name=zero-trust-workload-identity-manager
```

- j. Delete the admission webhook configuration by running the following command:

```
$ oc delete validatingwebhookconfigurations -l=app.kubernetes.io/name=zero-trust-workload-identity-manager
```

2. Delete the custom resource definitions (CRDs) by running each of the following commands:

- a. Delete the SPIRE Server CRD by running the following command:

```
$ oc delete crd spireservers.operator.openshift.io
```

- b. Delete the SPIRE Agent CRD by running the following command:

```
$ oc delete crd spireagents.operator.openshift.io
```

- c. Delete the SPIFFEE CSI Drivers CRD by running the following command:

```
$ oc delete crd spiffeidsdrivers.operator.openshift.io
```

- d. Delete the SPIRE OIDC Discovery Provider CRD by running the following command:

```
$ oc delete crd spireoidcdiscoveryproviders.operator.openshift.io
```

- e. Delete the SPIRE and SPIFFE cluster federated trust domains CRD by running the following command:

```
$ oc delete crd clusterfederatedtrustdomains.spire.spiffe.io
```

- f. Delete the cluster SPIFFE IDs CRD by running the following command:

```
$ oc delete crd clusteridspiffeoperatoroperator.openshift.io
```

```
$ oc delete crd clusterspiffeids.spire.spiffe.io
```

- g. Delete the SPIRE and SPIFFE cluster static entries CRD by running the following command:

```
$ oc delete crd clusterstaticentries.spire.spiffe.io
```

- h. Delete the Zero Trust Workload Identity Manager CRD by running the following command:

```
$ oc delete crd zerotrustworkloadidentitymanagers.operator.openshift.io
```

Verification

To verify that the resources have been deleted, replace each **oc delete** command with **oc get**, and then run the command. If no resources are returned, the deletion was successful.

CHAPTER 12. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

12.1. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT OVERVIEW

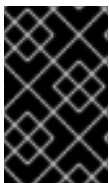
The External Secrets Operator for Red Hat OpenShift operates as a cluster-wide service to deploy and manage the **external-secrets** application. The **external-secrets** application integrates with external secrets management systems and performs secret fetching, refreshing, and provisioning within the cluster.

12.1.1. About the External Secrets Operator for Red Hat OpenShift

Use the External Secrets Operator for Red Hat OpenShift to integrate the **external-secrets** application with the OpenShift Container Platform cluster. The **external-secrets** application fetches secrets stored in external providers such as AWS Secrets Manager, HashiCorp Vault, Google Secret Manager, Azure Key Vault, IBM Cloud Secrets Manager, and AWS Systems Manager Parameter Store, and integrates them with Kubernetes in a secure manner.

Using the External Secrets Operator ensures the following:

- Decouples applications from the secret-lifecycle management.
- Centralizes secret storage to support compliance requirements.
- Enables secure and automated secret rotation.
- Supports multi-cloud secret sourcing with fine-grained access control.
- Centralizes and audits access control.



IMPORTANT

Do not attempt to use more than one External Secrets Operator in your cluster. If you have a community External Secrets Operator installed in your cluster, you must uninstall it before installing the External Secrets Operator for Red Hat OpenShift.

For more information about the **external-secrets** application, see "external-secrets application" in Additional resources.

Use the External Secrets Operator to authenticate with the external secrets store, retrieve secrets, and inject the retrieved secrets into a native Kubernetes secret. This method removes the need for applications to directly access or manage external secrets.

12.1.2. External secrets providers for the External Secrets Operator for Red Hat OpenShift

The External Secrets Operator for Red Hat OpenShift is tested with the following external secrets provider types:

- AWS Secrets Manager
- HashiCorp Vault

- Google Secret Manager
- Azure Key Vault
- IBM Cloud Secrets Manager

**NOTE**

Red Hat does not test all factors associated with third-party secrets store provider functionality. For more information about third-party support, see "Red Hat third-party support policy" in Additional resources.

12.1.3. About FIPS compliance for External Secrets Operator for Red Hat OpenShift

The External Secrets Operator for Red Hat OpenShift supports FIPS compliance. When running on OpenShift Container Platform in FIPS mode, External Secrets Operator uses the RHEL cryptographic libraries submitted to NIST for FIPS validation on the x86_64, ppc64le, and s390X architectures. For more information about the NIST validation program, see "Cryptographic module validation program" in Additional resources. For more information about the latest NIST status for the individual versions of the RHEL cryptographic libraries submitted for validation, see "Compliance activities and government standards" in Additional resources.

To enable FIPS mode, install the External Secrets Operator on an OpenShift Container Platform cluster that runs in FIPS mode. For more information, see "Do you need extra security for your cluster?".

12.1.4. Security considerations

When using the External Secrets Operator for Red Hat OpenShift, there are some security concerns you should consider:

- The **external-secrets** operand fetches the secrets from the configured external providers and stores it in a Kubernetes native **Secrets** resource. This results in a secret zero problem. It is recommended to secure the secret objects using additional encryption. For more information, see [Data encryption options](#).
- When configuring **SecretStore** and **ClusterSecretStore** resources, consider using short-term credential-based authorization. This approach enhances security by limiting the window of opportunity for unauthorized access, even if credentials are compromised.
- To enhance the security of the External Secrets Operator for Red Hat OpenShift, it is crucial to implement role-based access controls (RBACs). These RBACs should define and limit access to the custom resources provided by the External Secrets Operator.

12.1.5. Additional resources

- [external-secrets application](#)
- [Understanding compliance](#)
- [Installing a cluster in FIPS mode](#)
- [Do you need extra security for your cluster?](#)
- [Security considerations](#)

- [Security Best Practices](#)
- [AWS Secrets Manager](#)
- [HashiCorp Vault](#)
- [Google Secret Manager](#)
- [Azure Key Vault](#)
- [IBM Cloud Secrets Manager](#)
- [AWS Systems Manager Parameter Store](#)
- [Cryptographic module validation program](#)
- [Compliance activities and government standards](#)
- [Red Hat third-party support policy](#)

12.2. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT RELEASE NOTES

The External Secrets Operator for Red Hat OpenShift is a cluster-wide service that provides lifecycle management for secrets fetched from external secret management systems.

These release notes track the development of External Secrets Operator.

For more information, see [External Secrets Operator overview](#).

12.2.1. Release notes for External Secrets Operator for Red Hat OpenShift 1.2.0

External Secrets Operator for Red Hat OpenShift version 1.2.0 is based on the upstream external-secrets project, version v2.5.0.

Issued: 2026-07-09

The following advisories are available for the External Secrets Operator for Red Hat OpenShift 1.2.0:

- [RHBA-2026:36640](#)
- [RHBA-2026:36647](#)
- [RHBA-2026:36650](#)
- [RHBA-2026:36676](#)

12.2.1.1. New features and enhancements

Support for user-provided trusted CA bundles on the External Secrets Operator core controller

With this release, you can configure the External Secrets Operator for Red Hat OpenShift to trust custom Certificate Authority (CA) certificates when the **external-secrets** core controller makes outbound TLS connections to external secret management systems, such as HashiCorp Vault or Amazon Web Services (AWS) Secrets Manager.

To use this feature, create a **ConfigMap** object in the operand namespace containing one or more PEM-encoded CA certificates, and reference it in the **ExternalSecretsConfig** custom resource under the **spec.controllerConfig.trustedCABundle** field. The Operator validates the bundle on every reconcile and mounts it as a volume only on the core controller deployment. The webhook and cert-controller deployments are not affected.

If the referenced **ConfigMap** object is missing or contains invalid data, the Operator sets the **ExternalSecretsConfig** custom resource (CR) status to **Degraded** and emits a warning event describing the problem. The Operator recovers automatically when the **ConfigMap** object is created or corrected, without requiring a spec change.

If the proxy is configured and the **ConfigMap** carries the Cluster Network Operator (CNO) **inject-trusted-cabundle** label, the user bundle mount is skipped because the proxy TLS connections already use the OpenShift Container Platform trusted CA bundle injected by the CNO.

For more information, see [Configuring a trusted CA bundle for the External Secrets Operator for Red Hat OpenShift](#).

Optional feature configuration is available for External Secrets Operator deployments

With this release, the **ExternalSecretsManager** CR supports a **spec.features** field for toggling optional capabilities across operator-managed deployments. Each entry is identified by name and can be individually set to **Enabled** or **Disabled**.

The first supported feature is **UnsafeAllowGenericTargets**. When enabled, the Operator passes the **--unsafe-allow-generic-targets** flag to the **external-secrets** core controller, allowing **ExternalSecret** resources to sync secrets into Kubernetes resources other than **Secret** objects.



IMPORTANT

UnsafeAllowGenericTargets is a pre-release feature in the upstream **external-secrets** project. The **UnsafeAllowGenericTargets** feature has the following limitations:

- Only namespaced resources can be targeted, and only by an **ExternalSecret** CR in the same namespace as the target resource.
- Performance is approximately 20% slower than standard **Secret** synchronization.
- Custom resources are not encrypted at rest by Kubernetes. Use this feature only when the target resource does not contain sensitive credentials, or when encryption is provided by other means.

Enabling this feature also requires that the **external-secret** service account has the appropriate role-based access control (RBAC) permissions to create and update the target resource types. Without these permissions, secret synchronization for affected **ExternalSecret** CR resources fails.

Improved status reporting for invalid user configuration

With this release, the External Secrets Operator for Red Hat OpenShift distinguishes between Operator-level failures and user configuration errors during reconciliation. When an invalid or incomplete user configuration is detected, such as a missing cert-manager issuer reference or an incomplete Bitwarden TLS setup, the Operator immediately sets the **ExternalSecretsConfig** CR status to **Degraded=True** and **Ready=False** without entering an exponential backoff retry loop.

The Operator recovers automatically when the configuration is corrected. If a referenced object does not yet exist, the Operator requeues periodically until the object is created.

Automatic proxy egress NetworkPolicy management

With this release, the External Secrets Operator for Red Hat OpenShift automatically creates, updates, and deletes a proxy egress **NetworkPolicy** named **eso-sys-allow-proxy-egress**, for all **external-secrets** pods when a cluster proxy is configured. The policy allows outbound traffic from operand pods to the configured proxy server port. You can control whether the Operator manages this policy by setting the **networkPolicyProvisioning** field on the proxy configuration to **Managed** or **Unmanaged**. When set to **Unmanaged**, no proxy egress policy is created or deleted by the Operator.

Standardized NetworkPolicy naming

With this release, Operator-managed **NetworkPolicies** now use an **eso-sys-** prefix such as **eso-sys-deny-all-traffic**, **eso-sys-allow-to-dns**, and so on. User-configured **NetworkPolicies** defined in the **spec.controllerConfig.networkPolicies** field now use an **eso-user-** prefix when the Kubernetes object is created. This makes it easier to distinguish Operator-managed policies from user-defined ones.

As a result of the **eso-user-** prefix, the maximum length for the name field in **spec.controllerConfig.networkPolicies** entries is reduced from 253 to 243 characters.

Automatic migration of legacy NetworkPolicy names

With this release, when upgrading from a version that used unprefixed **NetworkPolicy** names, the Operator automatically detects and deletes the legacy unprefixed **NetworkPolicies** during the first reconciliation after upgrade. This migration runs once per cluster and is gated by the **externalsecretsconfig.operator.openshift.io/skip-np-cleanup-check** annotation so that subsequent reconciliations do not repeat the cleanup scan.

12.2.1.2. Fixed issues

- Before this release, if the **app=external-secrets** managed label was externally removed from a resource that the External Secrets Operator for Red Hat OpenShift owns, the resource fell out of the label-filtered informer cache. Subsequent reconciliation attempts to create the resource received an **AlreadyExists** error, causing the controller to enter a permanent error loop. With this release, the controller detects this cache-miss condition and restores the managed labels and annotations directly on the API server by using an uncached client, without interrupting the operand. ([ESO-237](#))

12.2.2. Release notes for External Secrets Operator for Red Hat OpenShift 1.1.1

External Secrets Operator for Red Hat OpenShift 1.1.1 is based on the upstream external-secrets version 0.20.4.

Issued: 13 August 2026

This release fixes some Common Vulnerabilities and Exposures (CVEs) and provides related Red Hat advisories.

The following advisories are available for the External Secrets Operator for Red Hat OpenShift:

- [RHBA-2026:54480](#)
- [RHSA-2026:54525](#)

- [RHBA-2026:54526](#)
- [RHBA-2026:54537](#)

12.2.2.1. CVEs

- [CVE-2026-44740](#)
- [CVE-2026-27145](#)
- [CVE-2026-39835](#)
- [CVE-2026-56852](#)

12.2.2.2. New features and enhancements

Operand container arguments can be overridden by using the Operator Subscription

With this release, you can override container arguments for the **external-secrets** operand components by setting environment variables in the **spec.config.env** field of the External Secrets Operator Subscription. The supported variables are **OPERAND_EXTERNAL_SECRETS_ARGS**, **OPERAND_WEBHOOK_ARGS**, **OPERAND_CERT_CONTROLLER_ARGS**, and **OPERAND_BITWARDEN_SDK_SERVER_ARGS**. Use comma-separated **--key=value** flags. Commas inside values such as **--tls-ciphers** are supported.

For more information, see [Customizing the External Secrets Operator for Red Hat OpenShift](#) .

12.2.3. Release notes for External Secrets Operator for Red Hat OpenShift 1.1.0

External Secrets Operator for Red Hat OpenShift version 1.1.0 is based on the upstream external-secrets project, version v0.20.4.

Issued: 2026-03-17

The following advisories are available for the External Secrets Operator for Red Hat OpenShift 1.1.0:

- [RHBA-2026:5554](#)
- [RHBA-2026:5555](#)
- [RHBA-2026:5558](#)
- [RHBA-2026:5589](#)

12.2.3.1. New features and enhancements

Customization feature is now available for External Secrets Operator components

With this release, the Operator API, **externalsecretsconfig.operator.openshift.io** allows users to customize various aspects of the **external-secrets** controllers. The new API allows users to add custom annotations and environment variables, and allows configuring revision history limits for the **external-secrets** deployments.

For more information, see [Customizing the External Secrets Operator for Red Hat OpenShift](#) .

12.2.4. Release notes for External Secrets Operator for Red Hat OpenShift 1.0.1

External Secrets Operator for Red Hat OpenShift 1.0.1 is based on the upstream external-secrets version 0.19.2.

Issued: 16 July 2026

This release fixes some Common Vulnerabilities and Exposures (CVEs) and provides related Red Hat advisories.

The following advisories are available for the External Secrets Operator for Red Hat OpenShift:

- [RHSA-2026:40924](#)
- [RHBA-2026:40923](#)
- [RHBA-2026:40925](#)
- [RHBA-2026:41014](#)

12.2.4.1. CVEs

- [CVE-2025-61726](#)
- [CVE-2025-68121](#)

12.2.5. Release notes for External Secrets Operator for Red Hat OpenShift 1.0.0 (General Availability)

External Secrets Operator for Red Hat OpenShift version 1.0.0 is based on the upstream external-secrets project, version v0.19.0.

Issued: 2025-11-03

The following advisories are available for the External Secrets Operator for Red Hat OpenShift 1.0.0:

- [RHBA-2025:19416](#)
- [RHBA-2025:19417](#)
- [RHBA-2025:19418](#)
- [RHBA-2025:19463](#)

12.2.5.1. Fixed issues

- Before this release, many of the APIs listed in the console for the External Secrets Operator for Red Hat OpenShift were missing descriptions. With this release, the API descriptions have been added. ([OCPBUGS-61081](#))

12.2.5.2. New features and enhancements

Renaming and improvements on the Operator API

With this release, the Operator API, **externalsecrets.operator.openshift.io** has been renamed to **externalsecretsconfigs.operator.openshift.io** to avoid confusion with the external-secrets provided

API that has the same name, but a different purpose. The external-secrets provided API has also been restructured and new features are added.

For more information, see [External Secrets Operator for Red Hat OpenShift APIs](#).

Support to collect metrics of External Secrets Operator

With this release, the External Secrets Operator for Red Hat OpenShift supports collecting metrics for both the Operator and operands. This is optional and must be enabled.

For more information, see [Monitoring the External Secrets Operator for Red Hat OpenShift](#).

Support to configure proxy for External Secrets Operator

With this release, the External Secrets Operator for Red Hat OpenShift supports configuring proxy for both the Operator and operand.

For more information, see [About the egress proxy for the External Secrets Operator for Red Hat OpenShift](#).

Root filesystem is read-only for External Secrets Operator for Red Hat OpenShift containers

With this release, to improve security, the External Secrets Operator for Red Hat OpenShift and all its operands have the **readOnlyRootFilesystem** security context set to true by default. This enhancement hardens the containers and prevents a potential attacker from modifying the contents of the container's root file system.

Network policy hardening is now available for External Secrets Operator components

With this release, External Secrets Operator for Red Hat OpenShift includes pre-defined **NetworkPolicy** resources designed for enhanced security by governing ingress and egress traffic for operand components. These policies cover essential internal traffic, such as ingress to the metrics and webhook servers, and egress to the OpenShift API server and DNS server. Note that deployment of the **NetworkPolicy** is enabled by default and egress allow policies must be explicitly defined in the **ExternalSecretsConfig** custom resource for the **external-secrets** component to fetch secrets from external providers.

For more information, see [Configuring network policy for the operand](#).

12.2.6. Release notes for External Secrets Operator for Red Hat OpenShift 0.1.0 (Technology Preview)

Version **0.1.0** of the External Secrets Operator for Red Hat OpenShift is based on the upstream external-secrets version **0.14.3**.

Issued: 2025-06-26

The following advisories are available for the External Secrets Operator for Red Hat OpenShift 0.1.0:

- [RHBA-2025:9747](#)
- [RHBA-2025:9746](#)
- [RHBA-2025:9757](#)
- [RHBA-2025:9763](#)

12.2.6.1. New features and enhancements

- This is the initial, Technology Preview release of the External Secrets Operator for Red Hat OpenShift.

12.3. INSTALLING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

The External Secrets Operator for Red Hat OpenShift is not installed on the OpenShift Container Platform by default. Install the External Secrets Operator by using either the web console or the command-line interface (CLI).

12.3.1. Limitations of External Secrets Operator for Red Hat OpenShift

There are specific operational constraints to consider when deploying or removing the External Secrets Operator for Red Hat OpenShift, that might require manual intervention or strict dependency ordering.

The following are the limitations of External Secrets Operator for Red Hat OpenShift during the installation and uninstallation of the **external-secrets** application.

- Uninstalling the External Secrets Operator for Red Hat OpenShift does not delete the resources created for **external-secrets** application. you must clean up the resources manually.
- When you add **cert-manager** Operator configurations in **externalsecrets.operator.openshift.io** object after creation, delete the **external-secrets-cert-controller** deployment resource manually to prevent degradation of the **external-secrets** application.
- Enable the **BitwardenSecretManagerProvider** field in **externalsecrets.operator.openshift.io** object only when installed on OpenShift Cluster running on x86_64 and arm64 architectures .
- Ensure **cert-manager** Operator is installed and operational before deploying the External Secrets Operator for Red Hat OpenShift for seamless functioning. If you install the **cert-manager** Operator later, manually restart the **external-secrets-operator** pod to apply cert-manager configurations in **externalsecrets.operator.openshift.io** object.

12.3.2. Installing the External Secrets Operator for Red Hat OpenShift by using the web console

You can install the External Secrets Operator for Red Hat OpenShift by using the OpenShift Container Platform web console. You can select the desired update channel and approval strategy, and deploy the Operator into the recommended namespace without manually defining YAML resources.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Navigate to **Ecosystem** → **Software Catalog**.

3. Enter **External Secrets Operator** in the search box.
4. Select the **External Secrets Operator for Red Hat OpenShift** from the generated list and click **Install**.
5. On the **Install Operator** page:
 - a. Update the **Update channel**, if necessary. The channel defaults to **stable-v1**, which installs the latest stable release of the External Secrets Operator.
 - b. Select the version from **Version** drop-down list.
 - c. Choose the **Installed Namespace** for the Operator.
 - To use the default Operator namespace, select the **Operator recommended Namespace** option.
 - To use the namespace that you created, select the **Select a Namespace** option, and then select the namespace from the drop-down list.
 - If the default **external-secrets-operator** namespace does not exist, it is created for you by the Operator Lifecycle Manager (OLM).
 - d. Select an **Update approval** strategy.
 - The **Automatic** strategy enables OLM to automatically update the Operator when a new version is available.
 - The **Manual** strategy requires a user with appropriate credentials to approve the Operator update.
 - e. Click **Install**.

Verification

1. Navigate to **Ecosystem → Installed Operators**.
2. Verify that **External Secrets Operator** is listed with a **Status** of **Succeeded** in the **external-secrets-operator** namespace.

12.3.3. Installing the External Secrets Operator for Red Hat OpenShift by using the CLI

You can install the External Secrets Operator for Red Hat OpenShift by manually configuring the Operator Lifecycle Manager (OLM) resources using the OpenShift CLI. You can create a dedicated namespace, define the Operator's scope, and install the Operator from the catalog.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Create a new project named **external-secrets-operator** by running the following command:

```
$ oc new-project external-secrets-operator
```

2. Create an **OperatorGroup** object by defining a YAML file with the following content:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-external-secrets-operator
  namespace: external-secrets-operator
spec:
  targetNamespaces: []
```

3. Create the **OperatorGroup** object by running the following command:

```
$ oc create -f operatorGroup.yaml
```

4. Create a **Subscription** object by defining a YAML file with the following content:
The following is an example of a **subscription.yaml** file.

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: openshift-external-secrets-operator
  namespace: external-secrets-operator
spec:
  channel: stable-v1
  name: openshift-external-secrets-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
  startingCSV: external-secrets-operator.v1.0.0
```

5. Create the **Subscription** object by running the following command:

```
$ oc create -f subscription.yaml
```

Verification

1. Verify that the OLM subscription is created by running the following command:

```
$ oc get subscription -n external-secrets-operator
```

The following is example output verifying the OLM subscription is created.

NAME	PACKAGE	SOURCE	CHANNEL
openshift-external-secrets-operator	openshift-external-secrets-operator	redhat-operators	stable-v1

2. Verify whether the Operator is successfully installed by running the following command:

```
$ oc get csv -n external-secrets-operator
```

The following is example output verifying that the Operator is installed.

NAME	DISPLAY	VERSION	REPLACES
PHASE			
external-secrets-operator.v1.0.0	External Secrets Operator for Red Hat OpenShift	1.0.0	
Succeeded			

- Verify that the status of the External Secrets Operator is **Running** by entering the following command:

```
$ oc get pods -n external-secrets-operator
```

The following is example output verifying the External Secrets Operator is **Running**.

NAME	READY	STATUS	RESTARTS	AGE
external-secrets-operator-controller-manager-5699f4bc54-kbsmn	1/1	Running	0	25h

12.3.4. Additional resources

- [Adding Operators to a cluster](#)

12.3.5. Installing the External Secrets operand by using the CLI

To install the External Secrets operand, create an instance of the **ExternalSecrets** custom resource by using the command-line interface (CLI) which deploys necessary operand components such as the core controller, webhook, and certificate controller into the **external-secrets** namespace.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.

Procedure

- Create an **externalsecretsconfig.openshift.operator.io** object by defining a YAML file with the following content:

Example externalsecretsconfig.yaml file

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  labels:
    app: external-secrets-operator
    app.kubernetes.io/name: cluster
  name: cluster
spec:
  controllerConfig:
    networkPolicies:
      - componentName: ExternalSecretsCoreController
        egress:
          - {}
        name: allow-external-secrets-egress
```


For more information on spec configuration, see "External Secrets Operator for Red Hat OpenShift APIs".

2. Create the **externalsecretsconfigs.openshift.operator.io** object by running the following command:

```
$ oc create -f externalsecretsconfig.yaml
```

Verification

1. Verify that the **external-secrets** pods are running by entering the following command:

```
$ oc get pods -n external-secrets
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
external-secrets-75d47cb9c8-6p4n2	1/1	Running	0	4h5m
external-secrets-cert-controller-676444b897-qb6ft	1/1	Running	0	4h5m
external-secrets-webhook-b566658ff-7m4d5	1/1	Running	0	4h5m

2. Verify that the **external-secrets-operator** deployment object reports a successful status by running the following command:

```
$ oc get externalsecretsconfig.operator.openshift.io cluster -n external-secrets-operator -o jsonpath='{.status.conditions}' | jq .
```

Example output

```
[
  {
    "lastTransitionTime": "2025-06-17T14:57:04Z",
    "message": "",
    "observedGeneration": 2,
    "reason": "Ready",
    "status": "False",
    "type": "Degraded"
  },
  {
    "lastTransitionTime": "2025-11-27T05:58:38Z",
    "message": "reconciliation successful",
    "observedGeneration": 2,
    "reason": "Ready",
    "status": "True",
    "type": "Ready"
  }
]
```

Next step

- Configure the network policies of the operand as described in "Configuring network policy for the operand".

12.3.6. Understanding update channels of the External Secrets Operator for Red Hat OpenShift

Control the version of the External Secrets Operator for Red Hat OpenShift in your cluster by selecting an update channel. By using this mechanism, you can declare a specific version track, ensuring your environment receives only the updates you require for stability.

The External Secrets Operator for Red Hat OpenShift offers the following update channels:

- **stable-v1**
- **stable-v1.y**

12.3.6.1. About the External Secrets Operator for Red Hat OpenShift stable-v1 channel

Select the **stable-v1** channel to install and update the latest release of the External Secrets Operator for Red Hat OpenShift. By selecting this channel, you can use the most recent stable release for your Operator.



NOTE

The **stable-v1** channel is the default and suggested channel while installing the External Secrets Operator for Red Hat OpenShift.

The **stable-v1** channel offers the following update approval strategies:

Automatic

If you choose automatic updates for an installed External Secrets Operator for Red Hat OpenShift, a new version of the External Secrets Operator for Red Hat OpenShift is available in the **stable-v1** channel. The Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without human intervention.

Manual

If you select manual updates, when a newer version of the External Secrets Operator for Red Hat OpenShift is available, OLM creates an update request. As a cluster administrator, you must then manually approve that update request to have the cert-manager Operator for Red Hat OpenShift updated to the new version.

12.3.6.2. About the External Secrets Operator for Red Hat OpenShift stable-v1.y channel

Select the **stable-v1** channel to install and update the latest release of the External Secrets Operator for Red Hat OpenShift. By selecting this channel, you can use the latest stable release and allows you to choose between automatic and manual updates.

The y-stream version of the External Secrets Operator for Red Hat OpenShift installs updates from the **stable-v1.y** channels such as **stable-v1.0**, **stable-v1.1**, and **stable-v1.2**. Select the **stable-v1.y** channel if you want to use the y-stream version and stay updated to the z-stream version of the External Secrets Operator for Red Hat OpenShift.

The **stable-v1.y** channel offers the following update approval strategies:

Automatic

If you choose automatic updates for an installed External Secrets Operator for Red Hat OpenShift, a new z-stream version of the External Secrets Operator for Red Hat OpenShift is available in the

stable-v1.y channel. OLM automatically upgrades the running instance of your Operator without human intervention.

Manual

If you select manual updates, when a newer version of the External Secrets Operator for Red Hat OpenShift is available, OLM creates an update request. As a cluster administrator, you must then manually approve that update request to have the External Secrets Operator for Red Hat OpenShift updated to the new version of the z-stream releases.

12.3.7. Additional resources

- [Adding Operators to a cluster](#)
- [Updating installed Operators](#)

12.4. CONFIGURING NETWORK POLICY FOR THE OPERAND

The External Secrets Operator for Red Hat OpenShift for OpenShift Container Platform includes pre-defined **NetworkPolicies** for security that rejects all egress traffic and allows traffic towards services that are required for the operand functionality. You must configure additional custom policies to allow the **external-secrets** controller to egress traffic towards external providers. These configurable policies are set through the **ExternalSecretsConfig** custom resource to establish the egress allow policy.

12.4.1. Adding a custom network policy to allow egress to all external providers

You must configure custom policies through the **ExternalSecretsConfig** custom resource to allow all egress to all external providers.

Prerequisites

- An **ExternalSecretsConfig** must be predefined.
- You must be able to define specific egress rules, including destination ports and protocols.

Procedure

1. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Set the policy by editing the **networkPolicies** section:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  name: cluster
spec:
  controllerConfig:
    networkPolicies:
      - name: allow-external-secrets-egress
        componentName: CoreController
        egress: # Allow all egress traffic
```

12.4.2. Adding a custom network policy to allow egress to a specific provider

You must configure custom policies through the **ExternalSecretsConfig** custom resource to allow all egress to a specific provider.

Prerequisites

- An **ExternalSecretsConfig** must be predefined.
- You must be able to define specific egress rules, including destination ports and protocols

Procedure

1. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Set the policy by editing the **networkPolicies** section. The following example shows how to allow egress to Amazon Web Services (AWS) endpoints.

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  name: cluster
spec:
  controllerConfig:
    networkPolicies:
      - componentName: ExternalSecretsCoreController
        egress:
          # Allow egress to Kubernetes API server, AWS endpoints, and DNS
          - ports:
              - port: 443 # HTTPS (AWS Secrets Manager)
              protocol: TCP
            - name: allow-external-secrets-egress
```

where:

componentName

Specifies the name for the core controller which is **ExternalSecretsCoreController**. Egress rules must specify the required ports, such as Transmission Control Protocol (TCP) port 443, for services such as the AWS Secrets Manager.

12.4.3. Default ingress and egress rules

The ingress and egress rules are necessary to build a secure setup where every component acts with the least amount of privilege necessary. These rules protect your cluster by strictly blocking unnecessary traffic and only allowing the outbound connections needed to fetch secrets. They also permit the specific inbound connections required to validate webhooks and observe system performance.

The following table summarizes the specific ports and protocols used by each component.

Component	Ingress ports	Egress ports	Description
external-secrets	8080	6443	Allows retrieving metrics and interacting with the API server
external-secrets-webhook	8080/10250	6443	Allows retrieving metrics, handling webhook requests, and interacting with the API server
external-secrets-cert-controller	8080	6443	Allows retrieving metrics and interacting with the API server
external-secrets-bitwarden-server	9998	6443	Handles Bitwarden server connections and interacts with the API server
external-secrets-allow-dns		5353	Enables DNS lookups to find external secret providers.

12.5. ABOUT THE EGRESS PROXY FOR THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

If a cluster-wide egress proxy is configured in OpenShift Container Platform, the Operator Lifecycle Manager (OLM) automatically configures Operators that it manages with the cluster-wide proxy. OLM automatically updates all of the Operator deployments with the **HTTP_PROXY**, **HTTPS_PROXY**, and **NO_PROXY** environment variables.

12.5.1. Configuring the egress proxy for the External Secrets Operator for Red Hat OpenShift

The egress proxy can be configured in the **ExternalSecretsConfig** or the **ExternalSecretsManager** custom resource (CR). The Operator and the operand make use of the OpenShift Container Platform supported certificate authority (CA) bundle for the proxy validations.

The Operator can automatically create and manage a **NetworkPolicy** such as **eso-sys-allow-proxy-egress**, that allows all **external-secrets** pods to reach the proxy server. You control this behavior by using the **networkPolicyProvisioning** field. The field can be set in either the **ExternalSecretsConfig** CR or the **ExternalSecretsManager** CR, and can be configured independently of proxy URL fields. For example, when the proxy is provided by Operator Lifecycle Manager (OLM) environment variables at the cluster level, you can set only **networkPolicyProvisioning** in either CR without specifying any proxy URLs.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have created the **ExternalSecretsConfig** custom CR.

Procedure

1. To set the proxy in the **ExternalSecretsConfig** resource, perform the following steps:

- a. Edit the **ExternalSecretsConfig** resource by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

- b. Edit the **spec.appConfig.proxy** section to set the proxy values as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
...
spec:
  appConfig:
    proxy:
      httpProxy: <http_proxy>
      httpsProxy: <https_proxy>
      noProxy: <no_proxy>
      networkPolicyProvisioning: Managed
```

where:

spec.appConfig.proxy.httpProxy

Specifies the proxy URL for the HTTP requests.

spec.appConfig.proxy.httpsProxy

Specifies the proxy URL for the HTTPS requests.

spec.appConfig.proxy.noProxy

Specifies a comma-separated list of hostnames, CIDRs, IPs or a combination of these, for which the proxy should not be used.

spec.appConfig.proxy.networkPolicyProvisioning

Specifies whether the Operator automatically creates and manages the **eso-sys-allow-proxy-egress NetworkPolicy**. Accepted values are **Managed**, which is the default, and **Unmanaged**. When set to **Managed**, the Operator creates the policy based on the proxy URL port and deletes it when the proxy is removed. When set to **Unmanaged**, the Operator does not create or delete the policy and you are responsible for managing proxy egress traffic.

2. To set the proxy in the **ExternalSecretsManager** CR, perform the following steps:

- a. Edit the **ExternalSecretsManager** CR by running the following command:

```
$ oc edit externalsecretsmanagers.operator.openshift.io cluster
```

- b. Edit the **spec.globalConfig.proxy** section to set the proxy values as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsManager
```

```
...
spec:
  globalConfig:
    proxy:
      httpProxy: <http_proxy>
      httpsProxy: <https_proxy>
      noProxy: <no_proxy>
      networkPolicyProvisioning: Managed
```

where:

spec.appConfig.proxy.httpProxy

Specifies the proxy URL for the HTTP requests.

spec.appConfig.proxy.httpsProxy

Specifies the proxy URL for the HTTPS requests.

spec.appConfig.proxy.noProxy

Specifies a comma-separated list of hostnames, CIDRs, IPs or a combination of these, for which the proxy should not be used.

spec.appConfig.proxy.networkPolicyProvisioning

Specifies whether the Operator automatically creates and manages the **eso-sys-allow-proxy-egress NetworkPolicy**. The values are `Managed`, which is the default, and `Unmanaged`.



NOTE

When **networkPolicyProvisioning** is set in both the **ExternalSecretsConfig** CR and the **ExternalSecretsManager** CR, the value in the **ExternalSecretsConfig** CR takes precedence.

3. If the proxy is configured at the cluster level through OLM environment variables and you only want to control **NetworkPolicy** provisioning without specifying proxy URLs in a CR, set only the **networkPolicyProvisioning** field in either CR as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
...
spec:
  applicationConfig:
    proxy:
      networkPolicyProvisioning: Unmanaged
```

Verification

1. Verify that the proxy egress **NetworkPolicy** was created by running the following command:

```
$ oc get networkpolicy eso-sys-allow-proxy-egress -n external-secrets -o yaml
```

The policy should show an egress rule allowing transmission control protocol (TCP) traffic on the port derived from the configured proxy URL.

2. Verify that the proxy configuration is applied to the **external-secrets** deployment by running the following command:

```
$ oc set env deployment/external-secrets -n external-secrets --list | grep -i proxy
```

12.5.2. Additional resources

- [Configuring proxy support in Operator Lifecycle Manager](#)

12.6. MONITORING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

By default, the External Secrets Operator for Red Hat OpenShift exposes metrics for the Operator and the operands. You can configure OpenShift Monitoring to collect these metrics by using the Prometheus Operator format.

12.6.1. Enabling user workload monitoring

By default, the OpenShift Container Platform monitoring stack does not scrape metrics from user-installed applications like the External Secrets Operator. Enabling user workload monitoring is necessary to collect critical operational data, such as synchronization status, API error rates, and controller performance. This helps you to configure custom alerts for secret sync failures and create dashboards to monitor the overall health of your secret management system. You can enable monitoring for user-defined projects by configuring user workload monitoring in the cluster. For more information, see "Setting up metrics collection for user-defined projects".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Create the **cluster-monitoring-config.yaml** YAML file:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

2. Apply the **ConfigMap** by running the following command:

```
$ oc apply -f cluster-monitoring-config.yaml
```

Verification

- Verify that the monitoring components for user workloads are running in the **openshift-user-workload-monitoring** namespace by running the following command:


```
$ oc -n openshift-user-workload-monitoring get pod
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
prometheus-operator-5f79cff9c9-67pjb	2/2	Running	0	25h
prometheus-user-workload-0	6/6	Running	0	25h
thanos-ruler-user-workload-0	4/4	Running	0	25h

The status of the pods such as **prometheus-operator**, **prometheus-user-workload**, and **thanos-ruler-user-workload** must be **Running**.

Additional resources

- [Setting up metrics collection for user-defined projects](#)

12.6.2. Configuring metrics collection for External Secrets Operator for Red Hat OpenShift by using a ServiceMonitor

The External Secrets Operator for Red Hat OpenShift exposes metrics by default on port **8443** at the **/metrics** service endpoint. You can configure metrics collection for the Operator by creating a **ServiceMonitor** custom resource (CR) that enables the Prometheus Operator to collect custom metrics. For more information, see "Configuring user workload monitoring".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the External Secrets Operator for Red Hat OpenShift.
- You have enabled the user workload monitoring.

Procedure

1. Configure the Operator to use **HTTP** for the metrics server. **HTTPS** is enabled by default.
 - a. Update the subscription object for External Secrets Operator for Red Hat OpenShift to configure the **HTTP** protocol by running the following command:

```
$ oc -n external-secrets-operator patch subscription openshift-external-secrets-operator -
-type='merge' -p '{"spec":{"config":{"env":
[{"name":"METRICS_BIND_ADDRESS","value":":8080"}, {"name":
"METRICS_SECURE","value": "false"}]}}}'
```

- b. To verify that the External Secrets Operator pod is redeployed and that the configured values for **METRICS_BIND_ADDRESS** and **METRICS_SECURE** are updated, run the following command:

```
$ oc set env --list deployment/external-secrets-operator-controller-manager -n external-
secrets-operator | grep -e METRICS_BIND_ADDRESS -e METRICS_SECURE -e
container
```

The following example shows that the **METRICS_BIND_ADDRESS** and **METRICS_SECURE** have been updated:

```
# deployments/external-secrets-operator-controller-manager, container manager
METRICS_BIND_ADDRESS=:8080
METRICS_SECURE=false
```

2. Create the **Secret** resource with the **kubernetes.io/service-account.name** annotation to inject the token required for authenticating with the metrics server.

- a. Create the **secret-external-secrets-operator.yaml** YAML file:

```
apiVersion: v1
kind: Secret
metadata:
  labels:
    app: external-secrets-operator
  name: external-secrets-operator-metrics-auth
  namespace: external-secrets-operator
  annotations:
    kubernetes.io/service-account.name: external-secrets-operator-controller-manager
type: kubernetes.io/service-account-token
```

- b. Create the **Secret** resource by running the following command:

```
$ oc apply -f secret-external-secrets-operator.yaml
```

3. Create the **ClusterRoleBinding** resource required for granting permissions to access metrics:

- a. Create the **clusterrolebinding-external-secrets.yaml** YAML file:

The following example shows a **clusterrolebinding-external-secrets.yaml** file.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  labels:
    app: external-secrets-operator
  name: external-secrets-allow-metrics-access
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: external-secrets-operator-metrics-reader
subjects:
  - kind: ServiceAccount
    name: external-secrets-operator-controller-manager
    namespace: external-secrets-operator
```

- b. Create the **ClusterRoleBinding** custom resource by running the following command:

```
$ oc apply -f clusterrolebinding-external-secrets.yaml
```

4. Create the **ServiceMonitor** CR if using the default **HTTPS**:

- a. Create the **servicemonitor-external-secrets-operator-https.yaml** YAML file:

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app: external-secrets-operator
    name: external-secrets-operator-metrics-monitor
    namespace: external-secrets-operator
spec:
  endpoints:
    - authorization:
        credentials:
          name: external-secrets-operator-metrics-auth
          key: token
        type: Bearer
      interval: 60s
      path: /metrics
      port: metrics-https
      scheme: https
      scrapeTimeout: 30s
      tlsConfig:
        ca:
          configMap:
            name: openshift-service-ca.crt
            key: service-ca.crt
          serverName: external-secrets-operator-controller-manager-metrics-service.external-secrets-operator.svc.cluster.local
        namespaceSelector:
          matchNames:
            - external-secrets-operator
        selector:
          matchLabels:
            app: external-secrets-operator
            svc: external-secrets-operator-controller-manager-metrics-service

```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-external-secrets-operator-https.yaml
```

5. Create the **ServiceMonitor** CR if configured to use **HTTP**:

- a. Create the **servicemonitor-external-secrets-operator-http.yaml** YAML file:

```

apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app: external-secrets-operator
    name: external-secrets-operator-metrics-monitor
    namespace: external-secrets-operator
spec:
  endpoints:
    - authorization:
        credentials:
          name: external-secrets-operator-metrics-auth
          key: token

```

```

    type: Bearer
    interval: 60s
    path: /metrics
    port: metrics-http
    scheme: http
    scrapeTimeout: 30s
    namespaceSelector:
      matchNames:
        - external-secrets-operator
    selector:
      matchLabels:
        app: external-secrets-operator
        svc: external-secrets-operator-controller-manager-metrics-service

```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-external-secrets-operator-http.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance begins metrics collection from the Operator. The collected metrics are labeled with **job="external-secrets-operator-controller-manager-metrics-service"**.

Verification

1. In the OpenShift Container Platform web console, navigate to **Observe → Targets**.
2. In the Label filter field, enter the following labels to filter the metrics targets for each operand:

```
$ service=external-secrets-operator-controller-manager-metrics-service
```

3. Confirm that the **Status** column shows **Up** for the **external-secrets-operator**.

Additional resources

- [Configurable monitoring components](#)

12.6.3. Querying metrics for the External Secrets Operator for Red Hat OpenShift

As a cluster administrator, or as a user with view access to all namespaces, you can query the Operator metrics by using the OpenShift Container Platform web console or the command-line interface (CLI). For more information, see "Accessing metrics".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the External Secrets Operator for Red Hat OpenShift.
- You have enabled monitoring and metrics collection by creating a **ServiceMonitor** object.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Observe → Metrics**.

2. In the query field, enter the following PromQL expressions to query the External Secrets Operator for Red Hat OpenShift metric:

```
{job="external-secrets-operator-controller-manager-metrics-service"}
```

Additional resources

- [Accessing metrics](#)

12.6.4. Configuring metrics collection for External Secrets Operator for Red Hat OpenShift operands by using a ServiceMonitor

The External Secrets Operator for Red Hat OpenShift operands exposes metrics by default on port **8080** at the **/metrics** service endpoint for all three components (**external-secrets**, **external-secrets-cert-controll**, and **external-secrets-webhook**). You can configure metrics collection for the external-secrets operands by creating a **ServiceMonitor** custom resource (CR) that enables the Prometheus Operator to collect custom metrics. For more information, see "Configuring user workload monitoring".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the External Secrets Operator for Red Hat OpenShift.
- You have enabled the user workload monitoring.

Procedure

1. Create the **ClusterRoleBinding** resource required for granting permissions to access metrics:
 - a. Create the **clusterrolebinding-external-secrets.yaml** YAML file:
The following example shows a **clusterrolebinding-external-secrets.yaml** file.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  labels:
    app: external-secrets
  name: external-secrets-allow-metrics-access
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: external-secrets-operator-metrics-reader
subjects:
- kind: ServiceAccount
  name: external-secrets
  namespace: external-secrets
- kind: ServiceAccount
  name: external-secrets-cert-controller
  namespace: external-secrets
- kind: ServiceAccount
  name: external-secrets-webhook
  namespace: external-secrets
```

- b. Create the **ClusterRoleBinding** custom resource by running the following command:

```
$ oc apply -f clusterrolebinding-external-secrets.yaml
```

2. Create the **ServiceMonitor** CR:

- a. Create the **servicemonitor-external-secrets.yaml** YAML file:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app: external-secrets
  name: external-secrets-metrics-monitor
  namespace: external-secrets
spec:
  endpoints:
    - interval: 60s
      path: /metrics
      port: metrics
      scheme: http
      scrapeTimeout: 30s
  namespaceSelector:
    matchNames:
      - external-secrets
  selector:
    matchExpressions:
      - key: app.kubernetes.io/name
        operator: In
        values:
          - external-secrets
          - external-secrets-cert-controller
          - external-secrets-webhook
      - key: app.kubernetes.io/instance
        operator: In
        values:
          - external-secrets
      - key: app.kubernetes.io/managed-by
        operator: In
        values:
          - external-secrets-operator
```

- b. Create the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f servicemonitor-external-secrets.yaml
```

After the **ServiceMonitor** CR is created, the user workload Prometheus instance begins metrics collection from the External Secrets Operator for Red Hat OpenShift operands. The collected metrics are labeled with **job="external-secrets"**, **job="external-secrets-cainjector"**, and **job="external-secrets-webhook"**.

Verification

1. In the OpenShift Container Platform web console, navigate to **Observe → Targets**.

2. In the Label filter field, enter the following labels to filter the metrics targets for each operand:

```
$ service=external-secrets
```

```
$ service=external-secrets-cert-controller-metrics
```

```
$ service=external-secrets-webhook
```

3. Confirm that the **Status** column shows **Up** for the **external-secrets**, **external-secrets-cert-controller** and **external-secrets-webhook**.

Additional resources

- [Configuring user workload monitoring](#)

12.6.5. Querying metrics for the external-secrets operand

As a cluster administrator, or as a user with view access to all namespaces, you can query **external-secrets** operand metrics by using the OpenShift Container Platform web console or the command-line interface (CLI). For more information, see "Accessing metrics".

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the External Secrets Operator for Red Hat OpenShift.
- You have enabled monitoring and metrics collection by creating a **ServiceMonitor** object.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Observe → Metrics**.
2. In the query field, enter the following PromQL expressions to query the External Secrets Operator for Red Hat OpenShift operands metric for each operand:

```
{job="external-secrets"}
```

```
{job="external-secrets-webhook"}
```

```
{job="external-secrets-cert-controller-metrics"}
```

Additional resources

- [Accessing metrics](#)

12.7. CUSTOMIZING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

You can customize the behavior of the External Secrets Operator for Red Hat OpenShift operand components by configuring custom annotations, deployment lifecycle settings, and environment variables through the **ExternalSecretsConfig** custom resource (CR).

These configurations provide administrators with fine-grained control over the external-secrets deployment.

You can customize the External Secrets Operator for Red Hat OpenShift operand by using the **ExternalSecretsConfig** custom resource (CR). The CR supports a set of deployment and runtime options, such as custom annotations, revision history limits, environment variables, resource limits, tolerations, and proxy settings—so you can control how the operand is deployed and run without editing the operand resources directly.

All supported options are defined in the **ExternalSecretsConfig** CR (for example under the **spec.controllerConfig** for controller-related settings). The Operator reconciles the operand from this CR. Changes made directly to operand resources are overwritten. Use the **ExternalSecretsConfig** CR as the only supported way to customize the operand.

For the complete list of fields and allowed values, see the **ExternalSecretsConfig** API reference in the External Secrets Operator for Red Hat OpenShift documentation.

Additional resources

- [External Secrets Operator for Red Hat OpenShift APIs](#)

12.7.1. Setting a log level for the External Secrets Operator for Red Hat OpenShift

You can configure the log verbosity for the lifecycle manager. You must adjust this setting to troubleshoot issues related to the installation, upgrade, or configuration of the operator itself, rather than secret synchronization.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.

Procedure

- Update the subscription object for the External Secrets Operator for Red Hat OpenShift to provide the verbosity level for the operator logs by running the following command:

```
$ oc -n <external_secrets_operator_namespace> patch subscription openshift-external-secrets-operator --type='merge' -p '{"spec":{"config":{"env":[{"name":"OPERATOR_LOG_LEVEL","value":"<log_level>"}]}}}'
```

where:

external_secrets_operator_namespace

Specifies the namespace where the Operator is installed.

log_level

Specifies the level of log detail. Values range from 1-5. The default is 2.

Verification

1. The External Secrets Operator pod is redeployed. Verify that the log level of the External Secrets Operator for Red Hat OpenShift is updated by running the following command:

```
$ oc set env deploy/external-secrets-operator-controller-manager -n external-secrets-operator --list | grep -e OPERATOR_LOG_LEVEL -e container
```

The following example verifies that the log level of the External Secrets Operator for Red Hat OpenShift is updated.

```
# deployments/external-secrets-operator-controller-manager, container manager
OPERATOR_LOG_LEVEL=2
```

2. Verify that the log level of the External Secrets Operator for Red Hat OpenShift is updated by running the **oc logs** command:

```
$ oc logs -n external-secrets-operator -f deployments/external-secrets-operator-controller-manager -c manager
```

12.7.2. Setting a log level for the External Secrets Operator for Red Hat OpenShift operand

You can troubleshoot common issues, such as secret synchronization failures, provider authentication errors, or data formatting problems, by configuring the log verbosity for the core controller.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.

Procedure

1. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Set the log level value by editing the **spec.appConfig.logLevel** section:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
...
spec:
  appConfig:
    logLevel: <log_level>
```

where:

log_level

Supports the value range of 1-5. The log level gets mapped to the following operand support levels:

- 1 - warnings
- 2 - error logs
- 3 - info logs
- 4 and 5 - debug logs

3. Save your changes and exit the editor.

12.7.3. Configuring cert-manager for the external-secrets certificate requirements

You can optionally configure cert-manager to manage certificates for the External Secrets Operator for Red Hat OpenShift webhook and plugins. If you do not use cert-manager, the Operator automatically generates webhook certificates, but you must manually configure certificates for any plugins.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.
- You have installed the cert-manager Operator for Red Hat OpenShift. For more information, see "Installing the cert-manager Operator for Red Hat OpenShift"

Procedure

1. Edit the **ExternalSecretsConfig** custom resource by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Configure **cert-manager** by editing the **spec.controllerConfig.certProvider.certManager** section as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
...
spec:
  controllerConfig:
    certProvider:
      certManager:
        injectAnnotations: "true"
        issuerRef:
          name: <issuer_name>
          kind: <issuer_kind>
          group: <issuer_group>
        mode: Enabled
```

where:

injectAnnotation

Must be set to **true** when enabled.

name

Specifies the name of the issuer object referenced in **ExternalSecretsConfig**.

kind

Specifies the API issuer. Can be set to either **Issuer** or **ClusterIssuer**.

group

Specifies the API issuer group. The group name must be **cert-manager.io**.

mode

Must be set to **Enabled**. This is an immutable field and cannot be modified once it is configured.

3. Save your changes.
4. After you update the **cert-manager** configurations in the **externalsecretsconfig.operator.openshift.io** object, you must manually delete **external-secrets-cert-controller** deployment by running the following command. This prevents performance degradation of the **external-secrets** application.

```
$ oc delete deployments.apps external-secrets-cert-controller -n external-secrets
```

5. Optionally, you can delete other resources created for the **cert-controller** by running the following commands:

```
$ oc delete clusterrolebindings.rbac.authorization.k8s.io external-secrets-cert-controller
```

```
$ oc delete clusterroles.rbac.authorization.k8s.io external-secrets-cert-controller
```

```
$ oc delete serviceaccounts external-secrets-cert-controller -n external-secrets
```

```
$ oc delete secrets external-secrets-webhook -n external-secrets
```

Additional resources

- [External Secrets Operator for Red Hat OpenShift APIs](#)
- [cert-manager Operator for Red Hat Openshift](#)
- [Installing the cert-manager-Operator for Red Hat Openshift](#)

12.7.4. Configuring the bitwardenSecretManagerProvider plugin

You must configure the **bitwardenSecretManagerProvider** plugin to enable communication with the Bitwarden API. This configuration enables the Operator to authenticate and fetch secrets for synchronization.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.

Procedure

1. Edit the **ExternalSecretsConfig** custom resource by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Edit the **spec.plugins.bitwardenSecretManagerProvider** section as follows to enable the Bitwarden Secrets Manager:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
...
spec:
  plugins:
    bitwardenSecretManagerProvider:
      mode: Enabled
      secretRef:
        name: <secret_object_name>
```

where:

name

The name of the secret containing the certificate key pair for the plugin. The key name in the secret for the certificate must be **tls.crt**. The key name for the private key must be **tls.key**. The key name for the Certificate Authority (CA) certificate key name must be **ca.crt**. Configuring the secret is optional when the cert-manager certificate provider is configured.

3. Save your changes and exit the editor.
4. If you disable the plugin the following resources must be deleted manually by running the following commands:

```
$ oc delete deployments.apps bitwarden-sdk-server -n external-secrets
```

```
$ oc delete certificates.cert-manager.io bitwarden-tls-certs -n external-secrets
```

```
$ oc delete service bitwarden-sdk-server -n external-secrets
```

```
$ oc delete serviceaccounts bitwarden-sdk-server -n external-secrets
```

12.7.5. Adding custom annotations to external-secrets resources

To customize your resources, you can define up to 20 custom annotations in the custom resource (CR). The Operator merges the annotations with the defaults, prioritizes them, and safely preserves annotations set by external systems.

When an annotation is removed from the CR, the Operator automatically removes it from all managed resources during the next reconciliation. Annotations set by external sources, such as Kubernetes system annotations or annotations added by other controllers, are preserved and are not affected by the Operator.

Annotation keys containing the following reserved domain prefixes are not allowed and are rejected by validation if applied:

- **kubernetes.io/** (including subdomains such as ***.kubernetes.io/**)

- **k8s.io/** (including subdomains such as ***.k8s.io/**)
- **openshift.io/** (including subdomains such as ***.openshift.io/**)
- **cert-manager.io/**

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.

Procedure

1. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Add the **annotations** field under **spec.controllerConfig** as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  name: cluster
spec:
  controllerConfig:
    annotations:
      prometheus.io/scrape: "true"
      example.com/environment: "production"
```

Verification

1. Verify that annotations are applied to the external-secrets deployment by running the following command:

```
$ oc get deployment external-secrets -n external-secrets -o
jsonpath='{.metadata.annotations}' | jq .
```

The output should include the custom annotations you specified.

2. Verify that annotations are applied to the pod template by running the following command:

```
$ oc get deployment external-secrets -n external-secrets -o
jsonpath='{.spec.template.metadata.annotations}' | jq .
```

The output should include the custom annotations you specified.

3. Verify that annotations are applied to other managed resources such as Services by running the following command:

```
$ oc get service external-secrets-webhook -n external-secrets -o
jsonpath='{.metadata.annotations}' | jq .
```

The output should include the custom annotations you specified.

12.7.6. Configuring the revisionHistoryLimit for external-secrets components

Configure the number of old **ReplicaSet** objects retained for rollback by setting the **revisionHistoryLimit** parameter for **external-secrets** components.

The following components can be configured:

Component name	Description
ExternalSecretsCoreController	The main external-secrets controller.
Webhook	The external-secrets webhook server.
CertController	The certificate controller for webhook TLS.
BitwardenSDKServer	The Bitwarden SDK server plugin.

Each component can only have one configuration entry. A maximum of 4 component configuration entries are allowed, one per component.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.

Procedure

1. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Add the **componentConfigs** field under **spec.controllerConfig** as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  name: cluster
spec:
  controllerConfig:
    componentConfigs:
      - componentName: ExternalSecretsCoreController
        deploymentConfigs:
          revisionHistoryLimit: 5
      - componentName: Webhook
        deploymentConfigs:
          revisionHistoryLimit: 3
```

where

spec.controllerConfig.componentConfigs.componentName.deploymentConfigs.revisionHistoryLimit

Specifies the number of old **ReplicaSet** objects to retain for rollback. The value must be at least 1 to ensure rollback capability. The maximum value is 50. If not specified, the default is 10.

Verification

- Verify that the **revisionHistoryLimit** parameter is applied to the deployment by running the following command:

```
$ oc get deployment external-secrets -n external-secrets -o
jsonpath='{.spec.revisionHistoryLimit}'
```

The output should display the value you configured.

12.7.7. Setting custom environment variables for external-secrets components

To configure component behavior at runtime or integrate with external services, set custom environment variables for individual **external-secrets** components.

Custom environment variables are merged with the default environment variables set by the Operator. User-specified variables take precedence in case of conflicts with the Operator defaults. A maximum of 50 custom environment variables can be specified per component.

The environment variable names starting with the following prefixes are reserved:

- **HOSTNAME**
- **KUBERNETES_**
- **EXTERNAL_SECRETS_**

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have created the **ExternalSecretsConfig** custom resource.

Procedure

1. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

2. Add the **overrideEnv** field under the desired component in the **spec.controllerConfig.componentConfigs** stanza as follows:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  name: cluster
spec:
  controllerConfig:
```

```

componentConfigs:
  - componentName: ExternalSecretsCoreController
    overrideEnv:
      - name: Example
        value: "4"

```

where

spec.controllerConfig.componentConfigs.overrideEnv.name

Specifies the name of the environment variable. Environment variable names starting with **HOSTNAME**, **KUBERNETES_**, or **EXTERNAL_SECRETS_** are reserved and are not allowed.

spec.controllerConfig.componentConfigs.overrideEnv.value

Specifies the value of the environment variable.

Verification

- Verify that the environment variable is set on the deployment by running the following command:

```
$ oc set env deployment/external-secrets -n external-secrets --list
```

The output should include the custom environment variable you specified.

12.7.8. Enabling optional features for External Secrets Operator for Red Hat OpenShift

The External Secrets Operator for Red Hat OpenShift supports optional capabilities that can be enabled cluster-wide through the **ExternalSecretsManager** custom resource (CR). Features are disabled by default and must be explicitly enabled.

You can enable or disable a feature at any time. The Operator reconciles the core controller deployment when the feature state changes, without requiring a restart or reinstallation.



WARNING

UnsafeAllowGenericTargets is a pre-release feature. It is not recommended for production use. Enabling this feature allows **ExternalSecret** resources to write secret data to arbitrary Kubernetes resource types beyond Secret objects. This might cause data managed by other controllers to be overwritten and can expose sensitive values through non-secret resources. This feature provides no additional access control beyond standard Kubernetes role-based access control (RBAC).

When enabled, **ExternalSecret** resources can target arbitrary Kubernetes resource types as their sync destination, instead of being limited to **Secret** objects.

The Operator passes the **--unsafe-allow-generic-targets=true** flag to the core **external-secrets** controller. The webhook and cert-controller are not affected.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed the External Secrets Operator for Red Hat OpenShift and created the **ExternalSecretsConfig** CR.

Procedure

1. Edit the **ExternalSecretsManager** CR by running the following command:

```
$ oc edit externalsecretsmanagers.operator.openshift.io cluster
```

2. Add the **features** field under **spec** and set the desired feature mode:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsManager
metadata:
  name: cluster
spec:
  features:
    - name: UnsafeAllowGenericTargets
      mode: Enabled
```

To disable the feature, set **mode: Disabled** or remove the entry from the features list.

Verification

1. Verify that the feature flag is passed to the core controller by running the following command:

```
$ oc get deployment external-secrets \
  -n external-secrets \
  -o jsonpath='{.spec.template.spec.containers[0].args}' | jq .
```

Example output

```
[
  "--concurrent=1",
  "--metrics-addr=:8080",
  "--loglevel=warn",
  "--zap-time-encoding=epoch",
  "--enable-leader-election=true",
  "--enable-push-secret-reconciler=true",
  "--enable-cluster-store-reconciler=true",
  "--enable-cluster-external-secret-reconciler=true",
  "--unsafe-allow-generic-targets=true"
]
```

When the feature is enabled, the output includes **--unsafe-allow-generic-targets=true**. When disabled or not configured, the flag is absent.

2. Verify that the **ExternalSecretsManager** CR reflects the configured feature by running the following command:

```
$ oc get externalsecretsmanagers.operator.openshift.io cluster -o jsonpath='{.spec.features}' | jq .
```

Example output

```
[
  {
    "mode": "Enabled",
    "name": "UnsafeAllowGenericTargets"
  }
]
```

12.7.9. Mounting a custom trusted certificate authority bundle for external-secrets

You can configure the External Secrets Operator for Red Hat OpenShift to trust a custom certificate authority (CA) bundle when the **external-secrets** core controller communicates with external secret backends over transport layer socket (TLS). This is required when your organization uses a private CA or a self-signed certificate that is not included in the default system truststore.

To enable mounting a custom trusted CA, you reference a **ConfigMap** that contains the Privacy Enhanced Mail (PEM)-encoded CA certificates in the **spec.controllerConfig.trustedCABundle** field of the **ExternalSecretsConfig** custom resource (CR). The Operator mounts the bundle into the core controller pod and configures the TLS library to use it alongside the default system trust stores.

The External Secrets Operator for Red Hat OpenShift applies the following rules to the CA bundle **ConfigMap**:

- The **ConfigMap** must reside in the **external-secrets** namespace and must contain only PEM-encoded X.509 CA certificates. Leaf certificates and private key PEM blocks are rejected.
- If the **ConfigMap** key contains an invalid bundle, the **ExternalSecretsConfig** CR enters a **Degraded** state. The Operator automatically recovers and mounts the bundle when the **ConfigMap** is corrected, without requiring manual intervention.
- If the referenced **ConfigMap** does not exist, the Operator removes any previously mounted CA bundle from the core controller deployment and sets the **ExternalSecretsConfig** CR to a **Degraded** state until the **ConfigMap** is created.
- The CA bundle is mounted only on the core **external-secrets** controller container. The webhook and cert-controller containers are not affected.
- If the **ConfigMap** has the **config.openshift.io/inject-trusted-cabundle: "true"** label and a cluster proxy is configured, the Operator skips the user-defined mount. The cluster-wide CA bundle injected by the Cluster Network Operator (CNO) is already available to the controller through the proxy CA bundle mechanism.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have installed the External Secrets Operator for Red Hat OpenShift and created the **ExternalSecretsConfig** CR.

- A **ConfigMap** containing PEM-encoded X.509 CA certificates exists in the **external-secrets** namespace.

Procedure

1. Create the **ConfigMap** containing your CA bundle by running the following command:

```
$ oc create configmap user-ca-bundle \
  --from-file=ca-bundle.crt=/path/to/ca.pem \
  -n external-secrets
```

2. Edit the **ExternalSecretsConfig** CR by running the following command:

```
$ oc edit externalsecretsconfigs.operator.openshift.io cluster
```

3. Add the **trustedCABundle** field under **spec.controllerConfig**:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  name: cluster
spec:
  controllerConfig:
    trustedCABundle:
      name: user-ca-bundle
      key: ca-bundle.crt
```

where:

spec.controllerConfig.trustedCABundle.name

Specifies the name of the **ConfigMap** in the **external-secrets** namespace that contains the CA certificate bundle.

spec.controllerConfig.trustedCABundle.key

Optional. Specifies the key within the **ConfigMap** that holds the PEM-encoded CA bundle. The default is **ca-bundle.crt**.

Verification

1. Verify that the CA bundle volume is mounted on the core controller deployment by running the following command:

```
$ oc get deployment external-secrets \
  -n external-secrets \
  -o jsonpath='{.spec.template.spec.volumes}' | jq '[] | select(.name=="user-ca-bundle")'
```

Example output

```
{
  "configMap": {
    "defaultMode": 420,
    "items": [
      {
        "key": "ca-bundle.crt",
```

```

    "path": "ca-bundle.crt"
  },
],
"name": "trusted-ca-bundle-for-es"
},
"name": "user-ca-bundle"
}

```

2. Verify that the **SSL_CERT_DIR** is set on the core controller container by running the following command:

```

$ oc set env deployment/external-secrets \
  -n external-secrets \
  --list | grep SSL_CERT_DIR

```

Example output

```

SSL_CERT_DIR=/etc/pki/tls/user-certs:/etc/pki/tls/certs:/etc/ssl/certs

```

3. Verify that the **ExternalSecretsConfig** CR is not in a **Degraded** state by running the following command:

```

$ oc get externalsecretsconfigs.operator.openshift.io cluster \
  -o jsonpath='{.status.conditions[?(@.type=="Degraded")]}' | jq .

```

Example output

```

{
  "lastTransitionTime": "2026-06-22T10:29:11Z",
  "message": "",
  "observedGeneration": 5,
  "reason": "Ready",
  "status": "False",
  "type": "Degraded"
}

```

The **Degraded** condition should show **"status": "False"**. If the condition is **True**, review the message field for the specific validation error and correct the referenced **ConfigMap**.

12.7.10. Overriding operand container arguments for the External Secrets Operator for Red Hat OpenShift

You can override container arguments for the **external-secrets** operand deployments by setting environment variables on the External Secrets Operator for Red Hat OpenShift subscription. Use this method when you need to pass additional or replacement **--key=value** flags to operand containers.



NOTE

This is a temporary feature available only in the External Secrets Operator for Red Hat OpenShift 1.1 and 1.2 z-streams. External Secrets Operator for Red Hat OpenShift 1.3.0 adds support in the **ExternalSecretsConfig** API to configure the same behavior. Consider migrating to the **ExternalSecretsConfig** API when upgrading to External Secrets Operator 1.3.0. This subscription-based method is scheduled to be deprecated and removed in External Secrets Operator for Red Hat OpenShift 1.4.0.

Prerequisites

- Your installed External Secrets Operator for Red Hat OpenShift version is 1.1 or 1.2.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Update the Subscription for the External Secrets Operator to set the operand argument overrides by running the following command:

```
$ oc -n <external_secrets_operator_namespace> patch subscription openshift-external-secrets-operator --type='merge' -p '{"spec":{"config":{"env":[{"name":"OPERAND_EXTERNAL_SECRETS_ARGS","value":"<controller_args>"}, {"name":"OPERAND_WEBHOOK_ARGS","value":"<webhook_args>"}, {"name":"OPERAND_CERT_CONTROLLER_ARGS","value":"<cert_controller_args>"}, {"name":"OPERAND_BITWARDEN_SDK_SERVER_ARGS","value":"<bitwarden_args>"}]}}}'
```

where:

<external_secrets_operator_namespace>

Specifies the namespace where the Operator is installed.

<controller_args>

Specifies a comma-separated list of **--key** or **--key=value** flags for the **external-secrets** core controller. For example, **--concurrent=2,--loglevel=debug**.

<webhook_args>

Specifies a comma-separated list of flags for the webhook. Commas inside a flag value are preserved when the next flag begins with **--**. For example, **--tls-ciphers=TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256,TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256,--loglevel=debug**.

<cert_controller_args>

Specifies a comma-separated list of flags for the **cert-controller**. For example, **--crd-request-interval=10m,--loglevel=debug**.

<bitwarden_args>

Specifies a comma-separated list of flags for the **bitwarden-sdk-server**. For example, **--key-file=/certs/key.pem,--cert-file=/certs/cert.pem**.

Set only the environment variables you need. Omit any unused entries from the **env** list.

Verification

1. Verify that the environment variables are set on the Operator by running the following command:

```
$ oc set env deploy/external-secrets-operator-controller-manager -n
<external_secrets_operator_namespace> --list | grep -e OPERAND_ -e container
```

Example output

```
$ deployments/external-secrets-operator-controller-manager, container manager
OPERAND_EXTERNAL_SECRETS_ARGS=--concurrent=2,--loglevel=debug
OPERAND_WEBHOOK_ARGS=--tls-
ciphers=TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256,TLS_ECDHE_RSA_
WITH_CHACHA20_POLY1305_SHA256,--loglevel=debug
```

2. Verify that the operand **Deployment** container arguments were updated by running the following command:

```
$ oc get deploy/external-secrets-webhook -n <operand_namespace> -o
jsonpath='{.spec.template.spec.containers[0].args}'
```

Example output

```
$ ["webhook","--dns-name=external-secrets-webhook.external-secrets.svc","--port=10250","--
cert-dir=/tmp/certs","--check-interval=15m0s","--metrics-addr=:8080","--healthz-
addr=:8081","--loglevel=debug","--zap-time-encoding=epoch","--tls-
ciphers=TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256,TLS_ECDHE_RSA_
WITH_CHACHA20_POLY1305_SHA256"]
```

12.8. UNINSTALLING THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

You can remove the External Secrets Operator for Red Hat OpenShift from OpenShift Container Platform by uninstalling the Operator and removing its related resources.

12.8.1. Uninstalling the External Secrets Operator for Red Hat OpenShift using the web console

You can uninstall the External Secrets Operator for Red Hat OpenShift from your cluster using the OpenShift Container Platform web console. Uninstalling the Operator does not automatically delete the **ExternalSecrets** custom resources or the running **external-secrets** application workload. These resources remain in the cluster to prevent accidental data loss and must be removed manually if they are no longer needed.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.
- The External Secrets Operator is installed.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Uninstall the External Secrets Operator for Red Hat OpenShift using the following steps:
 - a. Navigate to **Ecosystem → Installed Operators**.
 - b. Click the Options menu  next to the **External Secrets Operator for Red Hat OpenShift** entry and click **Uninstall Operator**.
 - c. In the confirmation dialog, click **Uninstall**.

12.8.2. Removing External Secrets Operator for Red Hat OpenShift resources by using the web console

After you have uninstalled the External Secrets Operator for Red Hat OpenShift, you can optionally eliminate its associated resources from your cluster.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Remove the deployments of the **external-secrets** application components in the **external-secrets** namespace:
 - a. Click the **Project** drop-down menu to see a list of all available projects, and select the **external-secrets** project.
 - b. Navigate to **Workloads → Deployments**.
 - c. Select the deployment that you want to delete.
 - d. Click the **Actions** drop-down menu, and select **Delete Deployment** to see a confirmation dialog box.
 - e. Click **Delete** to delete the deployment.
3. Remove the custom resource definitions (CRDs) that were installed by the External Secrets Operator using the following steps:
 - a. Navigate to **Administration → CustomResourceDefinitions**.
 - b. Choose **external-secrets.io/component: controller** from the suggestions in the **Label** field to filter the CRDs.
 - c. Click the Options menu  next to each of the following CRDs, and select **Delete Custom Resource Definition**:

- ACRAccessToken
- ClusterExternalSecret
- ClusterGenerator
- ClusterPushSecret
- ClusterSecretStore
- ECRAuthorizationToken
- ExternalSecret
- GCRAccessToken
- GeneratorState
- GithubAccessToken
- Grafana
- MFA
- Password
- PushSecret
- QuayAccessToken
- SecretStore
- SSHKey
- STSSessionToken
- UUID
- VaultDynamicSecret
- Webhook

4. Remove the **external-secrets-operator** namespace using the following steps:

a. Navigate to **Administration** → **Namespaces**.

b. Click the Options menu  next to the **External Secrets Operator** and select **Delete Namespace**.

c. In the confirmation dialog, enter **external-secrets-operator** in the field and click **Delete**.

12.8.3. Removing External Secrets Operator for Red Hat OpenShift resources by using the CLI

After you have uninstalled the External Secrets Operator for Red Hat OpenShift, you can optionally eliminate its associated resources from your cluster by using the command-line interface (CLI).

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.

Procedure

- Delete the deployments of the **external-secrets** application components in the **external-secrets** namespace by running the following command:

```
$ oc delete deployment -n external-secrets -l app=external-secrets
```

- Delete the custom resource definitions (CRDs) that were installed by the External Secrets Operator by running the following command:

```
$ oc delete customresourcedefinitions.apiextensions.k8s.io -l external-secrets.io/component=controller
```

- Delete the **external-secrets-operator** namespace by running the following command:

```
$ oc delete project external-secrets-operator
```

12.9. EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT APIS

External Secrets Operator for Red Hat OpenShift uses the following two APIs to configure the **external-secrets** application deployment.

Group	Version	Kind
operator.openshift.io	v1alpha1	externalsecretsConfig
operator.openshift.io	v1alpha1	externalsecretsmanager

The following list contains the External Secrets Operator for Red Hat OpenShift APIs:

- ExternalSecretsConfig
- ExternalSecretsManager

12.9.1. applicationConfig

The **applicationConfig** object customizes the runtime behavior and deployment constraints of the operand. Use this section to control observability, define the operational scope, and configure webhook specifics. Additionally, you can tailor the deployment to your infrastructure requirements.

Field	Type	Description	Default	Validation
-------	------	-------------	---------	------------

Field	Type	Description	Default	Validation
logLevel	<i>integer</i>	logLevel supports a range of values as defined in the kubernetes logging guidelines .	1	The maximum range value is 5 The minimum range value is 1 Optional
operatingName space	<i>string</i>	operatingName space restricts the external-secrets operand operations to the provided namespace. Enabling this field disables ClusterSecretStore and ClusterExternalSecret .		The maximum length is 63 The minimum length is 1 Optional
webhookConfig	<i>object</i>	webhookConfig configures webhook specifics of the external-secrets operand.		
resources	ResourceRequirements	resources defines the resource requirements. You cannot change the value of this field after setting it initially. For more information, see https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/		Optional

Field	Type	Description	Default	Validation
affinity	<i>Affinity</i>	affinity sets the scheduling affinity rules. For more information, see https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/		Optional
tolerations	<i>Toleration array</i>	tolerations sets the pod tolerations. For more information, see https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/		The maximum number of items is 50 The minimum number of items is 0 Optional
nodeSelector	<i>object (keys:string, values:string)</i>	nodeSelector defines the scheduling criteria by using node labels. For more information, see https://kubernetes.io/docs/concepts/configuration/assign-pod-node/		The maximum number of properties is 50 The minimum number of properties is 0 Optional
proxy	<i>object (keys:string, values:string)</i>	proxy sets the proxy configurations available in operand containers managed by the Operator as environment variables.		Optional

12.9.2. bitwardenSecretManagerProvider

To enable the Bitwarden secrets manager provider and set up the additional service required to connect to the Bitwarden server, you can configure the **bitwardenSecretManagerProvider** field.

Field	Type	Description	Default	Validation
mode	<i>string</i>	mode field enables the bitwardenSecretManagerProvider provider state, which can be set to Enabled or Disabled . If set to Enabled , the Operator ensures the plugin is deployed and synchronized. If set to Disabled , the Bitwarden provider plugin reconciliation is disabled. The plugin and resources remain in their current state, and are not managed by the Operator.	Disabled	enum: [Enabled Disabled] Optional
secretRef	<i>SecretReference</i>	SecretRef specifies the Kubernetes secret that contains the TLS key pair for the Bitwarden server. If this reference is not provided and the certManagerConfig field is configured, the issuer defined in certManagerConfig generates the required certificate. The secret must use tls.crt for certificate, tls.key for the private key, and ca.crt for CA certificate.		Optional

12.9.3. certManagerConfig

You can integrate the External Secrets Operator for Red Hat OpenShift with cert-manager to secure internal webhooks. Use these settings to replace the default internal certificate management with cert-manager, specify custom issuers, and define certificate lifecycle and renewal policies.

Field	Type	Description	Default	Validation
mode	<i>string</i>	mode specifies whether to use cert-manager for certificate management instead of the built-in cert-controller which can be indicated by setting either Enabled or Disabled . If set to Enabled , uses cert-manager for obtaining the certificates for the webhook server and other components. If set to Disabled , uses the cert-controller for obtaining the certificates for the webhook server. Disabled is the default behavior.		enum: [Enabled Disabled]

Field	Type	Description	Default	Validation
injectAnnotations	<i>string</i>	injectAnnotations adds the cert-manager.io/inject-ca-from annotation to the webhooks and custom resource definitions (CRDs) to automatically configure the webhook with the cert-manager Operator certificate authority (CA). This requires CA Injector to be enabled in cert-manager Operator. Set this field to true or false . When set, this field cannot be changed.	false	enum: [true false]
issuerRef	<i>ObjectReference</i>	issuerRef contains details of the referenced object used for obtaining certificates. The object must exist in the external-secrets namespace unless a cluster-scoped cert-manager Operator issuer is used.		
certificateDuration	<i>Duration</i>	certificateDuration sets the validity period of the webhook certificate.	8760h	

Field	Type	Description	Default	Validation
certificateRenewBefore	<i>Duration</i>	certificateRenewBefore sets the ahead time to renew the webhook certificate before expiry.	30m	

12.9.4. certProvidersConfig

The **certProvidersConfig** defines the configuration for the certificate providers used to manage TLS certificates for webhook and plugins.

Field	Type	Description	Default	Validation
certManager	<i>object</i>	certManager defines the configuration for cert-manager provider specifics.		

12.9.5. commonConfigs

The **commonConfigs** specifies the common configurations available for all operands managed by the Operator.

Field	Type	Description	Default	Validation
logLevel	<i>integer</i>	logLevel supports the value range as defined in the <i>Time</i> .	1	The maximum number of log levels is 5. The minimum number of log levels is 1.
resources	<i>ResourceRequirements</i> .	resources defines the resource requirements. This cannot be updated. See Resource Management for Pods and Containers .		

Field	Type	Description	Default	Validation
affinity	<i>affinity</i> .	affinity is used for setting scheduling affinity rules. See Assigning Pods to Nodes .		
tolerations	<i>toleration array</i>	tolerations sets the pod tolerations.		The maximum number of items is 50. The minimum number of items is 0.
nodeSelector	<i>object (keys:string, values:string)</i>	nodeSelector defines the scheduling criteria using node labels.		The maximum number of properties is 50. The minimum number of properties is 0.
proxy	<i>proxyConfig</i>	proxy sets the proxy configurations which are made available in operand containers managed by the Operator as environment variables.		

12.9.6. componentConfig

The **componentConfig** field defines configuration overrides for a specific **external-secrets** component.

Field	Type	Description	Default	Validation
-------	------	-------------	---------	------------

Field	Type	Description	Default	Validation
componentName	<i>string</i>	componentName identifies which external-secrets component this configuration applies to. Valid values are ExternalSecretsCoreController , Webhook , CertController , and BitwardenSDKServer .		Enum: [ExternalSecretsCoreController , Webhook , CertController , BitwardenSDKServer] Required
deploymentConfigs	<i>object</i>	deploymentConfigs specifies overrides for the Kubernetes Deployment resource of this component.		
overrideEnv	EnvVar <i>array</i>	overrideEnv specifies custom environment variables for this component's container. These are merged with operator-managed environment variables, with user-defined values taking precedence. Environment variable names starting with HOSTNAME , KUBERNETES_ or EXTERNAL_SECRETS_ are reserved and are not allowed.		The maximum number of items is 50.

12.9.7. componentName

The **componentName** field represents the different external-secrets components that can have network policies applied.

Field	Type	Description
ExternalSecretsCoreController	<i>object</i>	ExternalSecretsCoreController represents the external-secret component.
BitwardenSDKServer	<i>object</i>	BitwardenSDKServer represents the bitwarden-sdk-server component.
Webhook	<i>object</i>	Webhook represents the external-secrets webhook component.
CertController	<i>object</i>	CertController represents the cert-controller component.

12.9.8. condition

The **condition** object reports the current health and operational state of the External Secrets Operator for Red Hat OpenShift deployment. It provides a standardized status check by detailing the specific type of condition, its current status, and a message to verify deployment success or troubleshooting errors.

Field	Type	Description
type	<i>string</i>	type contains the condition of the deployment.
status	<i>ConditionStatus</i>	status contains the status of the condition of the deployment
message	<i>string</i>	message provides details on the state of the deployment

12.9.9. conditionalStatus

The **conditionalStatus** field holds information about the current state of the **external-secrets** deployment.

Field	Type	Description
-------	------	-------------

Field	Type	Description
conditions	<i>array</i>	conditions contains information on the current state of the deployment.

12.9.10. configMapKeyReference

The **configMapKeyReference** specifies a specific key in a ConfigMap.

Field	Type	Description	Default	Validation
name	<i>string</i>	name specifies the name of the ConfigMap resource being referred to.		The maximum length of the name is 253 characters. The minimum length of the name is 1 character.
key	<i>string</i>	key specifies the specific key to be used in the ConfigMap. When omitted, defaults to ca-bundle.crt .	ca-bundle.crt	The maximum length of the key is 253 characters. The minimum length of the key is 1 character. The pattern is: ^[_a-zA-Z0-9]+\$

12.9.11. controllerConfig

The **controllerConfig** specifies the configurations used by the controller when installing the **external-secrets** operand and the plugins.

Field	Type	Description	Default	Validation
certProvider	<i>string</i>	certProvider defines the configuration for the certificate providers used to manage TLS certificates for webhook and plugins.		

Field	Type	Description	Default	Validation
labels	<i>object (keys:string, values:string)</i>	labels field applies labels to all resources created for the external-secrets operand deployment.		The maximum number of properties is 20. The minimum number of properties is 0.
annotations	<i>object (keys:string, values:string)</i>	annotations add custom annotations to all the resources created for the external-secrets deployment. The annotations are merged with any default annotations set by the Operator. User-specified annotations take precedence over defaults in case of conflicts. Annotation keys containing the reserved domains kubernetes.io/, openshift.io/, k8s.io/, or cert-manager.io/ (including subdomains like *.kubernetes.io/) are not allowed.		The maximum number of annotations is 20. The minimum number of annotations is 0.

Field	Type	Description	Default	Validation
networkPolicies	<i>networkPolicy array</i>	networkPolicies specifies the list of network policy configurations to be applied to the external-secrets pods. Each entry allows specifying a name for the generated NetworkPolicy object, along with its full Kubernetes NetworkPolicy definition. The Operator prepends eso-user- to the provided name when creating the Kubernetes object. If this field is not provided, external-secrets components are isolated with deny-all network policies, which prevents proper operation.		The maximum number of items is 50. The minimum number of items is 0.
componentConfigs	<i>ComponentConfig array</i>	componentConfigs allows specifying deployment-level configuration overrides for individual external-secrets components. This field enables fine-grained control over deployment settings for each component independently. Each component can have only one configuration entry.		The maximum number of items is 4. The minimum number of items is 0.

Field	Type	Description	Default	Validation
trustedCABundle ConfigMapKeyReference	<i>object</i>	trustedCABundle references a ConfigMap containing PEM-encoded CA certificates for the external-secrets core controller to trust when making outbound TLS connections. If specified, this bundle is used for all outbound TLS traffic, including connections to external secret management systems and configured proxies. The ConfigMap must exist in the external-secrets Operand namespace and must not carry the CNO inject-trusted-cabundle label when proxy is configured. When omitted, external providers use standard system certificates. When proxy is configured, proxy TLS connections use the operator-managed OpenShift Container Platform trusted CA bundle injected by the Cluster Network Operator.		

12.9.12. controllerStatus

The **controllerStatus** field tracks the health and synchronization state of the individual controllers managed by the Operator. It identifies each controller by name, details its current operational conditions, and verifies that the controller is processing the latest configuration version.

Field	Type	Description	Default	Validation
name	<i>string</i>	name specifies the name of the controller for which the observed condition is recorded.		
conditions	<i>array</i>	conditions contains information about the current state of the External Secrets Operator controllers.		
observedGeneration	<i>integer</i>	observedGeneration represents the .metadata.generation on the observed resource.		The minimum number of observed resources is 0.

12.9.13. deploymentConfig

The **deploymentConfig** field defines configuration overrides for a Kubernetes Deployment resource.

Field	Type	Description	Default	Validation
revisionHistoryLimit	<i>integer</i>	revisionHistoryLimit specifies the number of old ReplicaSets to retain for rollback purposes. This allows rolling back to previous deployment versions using the command oc rollout undo . Must be at least 1 to ensure rollback capability.	10	The maximum value is 50. The minimum value is 1.

12.9.14. externalSecretsConfig

The **externalSecretsConfig** object defines the configuration and information for the managed **external-secrets** operand deployment. Set the name to **cluster** as **externalSecretsConfig** object allows only one instance per cluster.

Creating an **externalSecretsConfig** object triggers the deployment of the **external-secrets** operand and maintains the desired state.

Field	Type	Description
apiVersion	<i>string</i>	The apiVersion specifies the version of the schema in use, which is operator.openshift.io/v1alpha1 .
kind	<i>string</i>	kind specifies the type of the object, which is externalSecrets for this object.
metadata	<i>ObjectMeta</i>	Refer to Kubernetes API documentation for details about the metadata fields.
spec	<i>object</i>	spec contains the specifications of the desired behavior of the externalSecrets object.
status	<i>object</i>	status displays the most recently observed status of the externalSecrets object.

12.9.15. externalSecretsConfigList

The **externalSecretsConfigList** object fetches the list of **externalSecretsConfig** objects.

Field	Type	Description
apiVersion	<i>string</i>	The apiVersion specifies the version of the schema in use, which is operator.openshift.io/v1alpha1 .
kind	<i>string</i>	kind specifies the type of the object, which is externalSecretsList for this API.

Field	Type	Description
metadata	ListMeta	Refer to Kubernetes API documentation for details about the metadata fields.
items	<i>array</i>	Items contains a list of externalSecrets objects.

12.9.16. externalSecretsConfigSpec

The **externalSecretsConfigSpec** field defines the desired behavior of the **externalSecrets** object.

Field	Type	Description
appConfig	<i>object</i>	appConfig configures the behavior of the external-secrets operand.
plugins	<i>object</i>	plugins configures the optional provider plugins.
controllerConfig	<i>object</i>	controllerConfig configures the controller to set up defaults that enable external-secrets operand.

12.9.17. externalSecretsConfigStatus

The **externalSecretsConfigStatus** field shows the most recently observed status of the **externalSecretsConfig** Object.

Field	Type	Description
conditions	Condition <i>array</i>	conditions contains information about the current state of deployment.
externalSecretsImage	<i>string</i>	externalSecretsImage specifies the image name and tag used for deploy external-secrets operand.
bitwardenSDKServerImage	<i>string</i>	bitwardenSDKServerImage specifies the name of the image and tag used for deploying the bitwarden-sdk-server .

12.9.18. externalSecretsManager

The **externalSecretsManager** object defines the configuration and information of deployments managed by the External Secrets Operator. Set the name to **cluster** as this allows only one instance of **externalSecretsManager** per cluster. You can configure global options by using **externalSecretsManager**. This serves as a centralized configuration for managing multiple controllers of the Operator. The Operator automatically creates the **externalSecretsManager** object during installation.

Field	Type	Description
apiVersion	<i>string</i>	The apiVersion specifies the version of the schema in use, which is operator.openshift.io/v1alpha1 .
kind	<i>string</i>	kind specifies the type of the object, which is externalSecretsManager for this Object.
metadata	<i>ObjectMeta</i>	Refer to Kubernetes API documentation for details about the metadata fields.
spec	<i>object</i>	spec contains specifications of the desired behavior.
status	<i>object</i>	status displays the most recently observed state of the controllers in the External Secrets Operator.

12.9.19. externalSecretsManagerList

The **externalSecretsManagerList** object fetches the list of **externalSecretsManager** objects.

Field	Type	Description	Default	Validation
apiVersion	<i>string</i>	The apiVersion specifies the version of the schema in use, which is operator.openshift.io/v1alpha1 .		

Field	Type	Description	Default	Validation
kind	<i>string</i>	kind specifies the type of the object, which is externalSecrets ManagerList for this API.		
metadata	ListMeta	Refer to Kubernetes API documentation for details about the metadata fields.		
items	<i>array</i>			

12.9.20. externalSecretsManagerSpec

The **externalSecretsManagerSpec** field defines the desired behavior of the **externalSecretsManager** object.

Field	type	Description	Default	Validation
globalConfig	<i>object</i>	globalConfig configures the behavior of deployments that External Secrets Operator manages.		Optional

12.9.21. externalSecretsManagerStatus

The **externalSecretsManagerStatus** field shows the most recently observed status of the **externalSecretsManager** object.

Field	Type	Description	Default	Validation
controllerStatuses	<i>array</i>	controllerStatuses holds the observed conditions of the controllers used by the Operator.		

Field	Type	Description	Default	Validation
lastTransitionTime	<i>Time</i>	lastTransitionTime records the most recent time the status of the condition changed.		Format: date-time Type: string

12.9.22. Feature

The **Feature** field configures an optional capability that is applied by the **external-secrets-operator** across its managed deployments.

Field	Type	Description	Default	Validation
name	FeatureName <i>string</i>	name identifies the optional feature to configure. Currently, the only supported value is UnsafeAllowGenericTargets .		Enum: [UnsafeAllowGenericTargets]
mode	mode <i>string</i>	mode mode controls whether the feature is active. When set to Enabled , the Operator applies the configuration associated with the named feature to the relevant managed deployments. For UnsafeAllowGenericTargets , this passes the --unsafe-allow-generic-targets flag to the external-secrets core controller, allowing ExternalSecret resources to target Kubernetes resources other	Disabled	Enum:[Enabled Disabled]

Field	Type	than Secrets. For example, Description	Default	Validation
		ConfigMaps or custom resources.  W A R N I N G G e n e r i c t a r g e t s r e q u i r e a d d i t i o n a l R B A C p e r m i s s i o n s o		

Field	Type	Description	Default	Validation
		not affected by the operator and; enabling this feature without the appropriate permissions		

Field	Type	Description	Default	Validation
		Switch will cause user record reconciliation failures.		

12.9.23. featureName

The **featureName** field identifies an optional feature that can be configured on the **ExternalSecretsManager** and applied by the **external-secrets-operator**.

Field	Type	Description
UnsafeAllowGenericTargets	<i>object</i>	UnsafeAllowGenericTargets configures the external-secrets core controller to run with the --unsafe-allow-generic-targets startup flag, which allows ExternalSecret resources to sync data into Kubernetes resources other than Secrets .

12.9.24. globalConfig

The **globalConfig** field defines the baseline behavior and deployment parameters for the External Secrets Operator for Red Hat OpenShift. Use this section to apply labels to all managed resources and configure the logging verbosity. It also provides infrastructure-level controls to govern where and how the Operator is scheduled, alongside proxy settings for network compatibility.

Field	Type	Description	Default	Validation
logLevel	<i>integer</i>	logLevel supports a range of values as defined in the kubernetes logging guidelines .	1	The maximum range value is 5 The minimum range value is 1
resources	<i>ResourceRequirements</i>	resources defines the resource requirements. You cannot change the value of this field after setting it initially. For more information, see https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/		
affinity	<i>Affinity</i>	affinity sets the scheduling affinity rules. For more information, see https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/		
tolerations	<i>Toleration array</i>	tolerations sets the pod tolerations. For more information, see https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/		The maximum number of items is 50 The minimum number of items is 0

Field	Type	Description	Default	Validation
nodeSelector	<i>object (keys:string, values:string)</i>	nodeSelector defines the scheduling criteria by using the node labels. For more information, see https://kubernetes.io/docs/concepts/configuration/assign-pod-node/		The maximum number of properties is 50 The minimum number of properties is 0
proxy	<i>object</i>	proxy sets the proxy configurations available in the operand containers managed by the Operator as environment variables.		
labels	<i>object (keys:string, values:string)</i>	labels applies to all resources created by the Operator. This field can have a maximum of 20 entries		The maximum number of properties is 20 The minimum number of properties is 0

12.9.25. managementState

The **managementState** field controls whether the Operator manages the resource lifecycle.

Field	Type	Description
Managed	<i>string</i>	ManagementStateManaged indicates the Operator is responsible for the resource lifecycle.
Unmanaged	<i>string</i>	ManagementStateUnmanaged indicates the user is responsible for the resource lifecycle.

12.9.26. mode

The **mode** field indicates the operational state of the optional features.

Field	Type	Description
Enabled	<i>string</i>	Enabled indicates the optional configuration is enabled.
Disabled	<i>string</i>	Disabled indicates the optional configuration is disabled.

12.9.27. networkPolicy

The **networkPolicy** field represents a custom network policy configuration for operator-managed components. The field includes a name for identification and the network policy rules to be enforced.

Field	Type	Description	Default	Validation
name	<i>string</i>	name is the logical identifier for this network policy entry. The Operator prepends eso-user- to this value when creating the Kubernetes NetworkPolicy object, for example allow-egress becomes eso-user-allow-egress . The maximum length is 243 to accommodate the prefix within the 253-character Kubernetes name limit.		The maximum length is 243 characters. The minimum length is 1 character.
componentName	<i>string</i>	componentName specifies which external-secrets component this network policy applies to.		Enum: [ExternalSecretsCoreController BitwardenSDIServer]

Field	Type	Description	Default	Validation
egress NetworkPolicy egressRule	<i>array</i>	egress is a list of egress rules to be applied to the selected pods. Outgoing traffic is allowed if there are no NetworkPolicies selecting the pod, and cluster policy otherwise allows the traffic, or if the traffic matches at least one egress rule across all the NetworkPolicy objects whose podSelector matches the pod. If this field is empty, then this NetworkPolicy limits all outgoing traffic and serves solely to ensure that the pods it selects are isolated by default. The Operator automatically handles ingress rules based on the current running ports.		

12.9.28. objectReference

The **ObjectReference** object acts as a pointer to a specific Kubernetes resource. It uniquely identifies the target by requiring its name, and optionally, helps scope the reference to a specific resource type and API group.

Field	Type	Description	Default	Validation
-------	------	-------------	---------	------------

Field	Type	Description	Default	Validation
name	<i>string</i>	name specifies the name of the resource being referred to.		The maximum length is 253 characters. The minimum length is 1 character. Required
kind	<i>string</i>	kind specifies the kind of the resource being referred to.		The maximum length is 253 characters. The minimum length is 1 character. Optional
group	<i>string</i>	group specifies the group of the resource being referred to.		The maximum length is 253 characters. The minimum length is 1 character. Optional

12.9.29. pluginsConfig

The **pluginsConfig** configures the optional plugins.

Field	Type	Description	Default	Validation
bitwardenSecretManagerProvider	<i>object</i>	bitwardenSecretManagerProvider enables the bitwarden-secrets-manager provider plugin for connecting with the 'bitwarden-secrets-manager'.		Optional

12.9.30. proxyConfig

The **proxyConfig** object defines the network proxy settings that the Operator injects into managed containers as environment variables. Use this configuration to ensure proper connectivity in restricted network environments, or to bypass the proxy and connect directly.

Field	Type	Description	Default	Validation
httpProxy	<i>string</i>	The httpProxy field contains the URL of the proxy for HTTP requests. This field can have a maximum of 2048 characters.		The maximum length is 2048 characters. The minimum length is 0 characters.
httpsProxy	<i>string</i>	The httpsProxy field contains the URL of the proxy for HTTPS requests. This field can have a maximum of 2048 characters.		The maximum length is 2048 characters. The minimum length is 0 characters.
noProxy	<i>string</i>	The noProxy field is a comma-separated list of hostnames, classless inter-domain routings (CIDRs), and IP addresses or a combination of the three for which the proxy should not be used. This field can have a maximum of 4096 characters.		The maximum length is 4096 characters. The minimum length is 0 characters.

Field	Type	Description	Default	Validation
networkPolicyProvisioningManagementState	<i>string</i>	The networkPolicyProvisioning field defines the management strategy for the proxy egress rule. When set to Managed , the Operator automatically provisions and maintains a NetworkPolicy allowing traffic to the configured proxy. If no proxy is configured, a NetworkPolicy is not created regardless of this setting.	Managed	Enum:[Managed Unmanaged]

12.9.31. secretReference

The **secretReference** field refers to a secret with the given name in the same namespace where it used.

Field	Type	Description	Default	Validation
name	<i>string</i>	name specifies the name of the secret resource being referred to.		The maximum length is 253. The minimum length is 1.

12.9.32. webhookConfig

The **webhookConfig** field configures the specifics of the **external-secrets** application webhook.

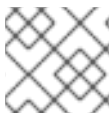
Field	Type	Description	Default	Validation
certificateCheckInterval	<i>Duration</i>	certificateCheckInterval configures the polling interval to check certificate validity.	5m	Optional

12.10. MIGRATING FROM THE COMMUNITY EXTERNAL SECRETS OPERATOR TO THE EXTERNAL SECRETS OPERATOR FOR RED HAT OPENSIFT

You can migrate from the community version of the External Secrets Operator. Migrating to External Secrets Operator for Red Hat OpenShift provides you with an officially supported product giving you access to enterprise-grade support. It also provides you with seamless integration from installation to upgrades.

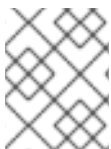
The following migration versions have been fully tested.

Upstream version	Installation method	Downstream version
0.11.0	OLM	v1.0.0 GA
0.19.0	Helm	v1.0.0 GA



NOTE

The migration does not support rollbacks.



NOTE

External Secrets Operator for Red Hat OpenShift is based on the upstream version 0.19.0. Do not try to migrate from a higher version of the External Secrets Operator.

12.10.1. Deleting the community External Secrets Operator

Delete the configuration resource for the community Operator so that the legacy application is fully removed. This action prevents conflicts before installing the External Secrets Operator for Red Hat OpenShift.

Prerequisites

- You must be logged in as a user with the **cluster-admin** role.
- You must have the **oc** command-line tool installed and configured.

Procedure

1. Find your community Operator's **namespace** by running the following command:

```
$ oc get operatorconfigs.operator.external-secrets.io -A
```

The following is an example of finding the **namespace**:

```
NAMESPACE    NAME    AGE
external-secrets  cluster  9m18s
```

2. Delete the **operatorconfig** custom resource (CR) by running the following command:

■

```
$ oc delete operatorconfig <config_name> -n <operator_namespace>
```

Verification

1. To verify that the **operatorconfig** CR is deleted, run the following command:

```
$ oc get operatorconfig -n <operator_namespace>
```

The command must return **no resource found**.

2. To verify that the old webhooks are deleted, run the following commands:

```
$ oc get validatingwebhookconfigurations | grep external-secrets
```

```
$ oc get mutatingwebhookconfigurations | grep external-secrets
```

The commands must return no results.

12.10.2. Uninstalling the community External Secrets Operator

Uninstall the community External Secrets Operator to prevent conflicts or accidental recreation after you migrate to External Secrets Operator for Red Hat OpenShift.

You must uninstall the community External Secrets Operator to prevent it from being recreated or conflicting with the new one. The steps to uninstall are different based on how the community External Secrets Operator was installed but the prerequisites are the same for each.

12.10.2.1. Uninstalling a helm installed community External Secrets Operator

Remove the community External Secrets Operator that was installed using Helm. This helps you free up resources and maintain a clean environment for your cluster.

Prerequisites

- You must be logged in as a user with the **cluster-admin** role.
- You must have deleted the **operatorconfig** custom resource (CR).

Procedure

1. Install the External Secrets Operator for Red Hat OpenShift. The **external-secrets-operator** namespace must be null.
2. Delete the External Secrets Operator by running the following command:

```
$ oc helm delete <release_name> -n <operator_namespace>
```



NOTE

Using **helm delete** might delete all Custom Resource Definitions (CRDs) and CRs. It is recommended to install the downstream Operator first if the namespace **external-secrets-operator** is empty.

12.10.2.2. Uninstalling an Operator Lifecycle Manager installed community External Secrets Operator

Remove the community External Secrets Operator that was installed by an Operator Lifecycle Manager (OLM) subscription. This helps you free up resources and maintain a clean environment for your cluster.

Prerequisites

- You must be logged in as a user with the **cluster-admin** role.
- You must have deleted the **operatorconfig** CR.

Procedure

1. Find the subscription name by running the following command:

```
$ oc get subscription -n <operator_namespace> | grep external-secrets
```

2. Delete the subscription by running the following command:

```
$ oc delete subscription <subscription_name> -n <operator_namespace>
```

3. Delete the **ClusterServiceVersion** by running the following command:

```
$ oc delete csv <csv_name> -n <operator_namespace>
```

12.10.2.3. Uninstalling a raw manifest installed community External Secrets Operator

Remove the community External Secrets Operator that was installed by raw manifests. This helps you free up resources and maintain a clean environment for your cluster.

Prerequisites

- You must be logged in as a user with the **cluster-admin** role.
- You must have deleted the **operatorconfig** CR.

Procedure

- To remove the community External Secrets Operator that was installed by raw manifests, run the following command:

```
$ oc delete -f /path/to/your/old/manifests.yaml -n <operator_namespace>
```

12.10.3. Installing the External Secrets Operator for Red Hat OpenShift

Install the External Secrets Operator for Red Hat OpenShift after cleaning up the community version. This establishes the officially supported service for managing secrets in your cluster.

12.10.4. Creating the ExternalSecretsConfig Operator

Create the **ExternalSecretsConfig** resource to install and configure the core **external-secrets** component. This setup helps ensure that features like Bitwarden and cert-manager support are correctly enabled.

Prerequisites

- External Secrets Operator for Red Hat OpenShift is installed.
- cert-manager Operator for Red Hat OpenShift is installed.
- You have access to the cluster with **cluster-admin** privileges.

Procedure

1. Create an **externalsecretsconfig** file by defining a YAML file with the following content:

```
apiVersion: operator.openshift.io/v1alpha1
kind: ExternalSecretsConfig
metadata:
  labels:
    app.kubernetes.io/name: cluster
  name: cluster
spec:
  appConfig:
    logLevel: 1
  controllerConfig:
    networkPolicies:
      - componentName: ExternalSecretsCoreController
        egress:
          - {}
        name: allow-external-secrets-egress
    plugins: {}
```

2. Create the **ExternalSecretsConfig** object by running the following command:

```
$ oc create -f externalsecretsconfig.yaml
```

Verification

Verify that all custom resources (CRs) are present and that the APIs are using **v1** instead of **v1beta1**. There CRs are retained and automatically converted by the new Operator.

1. To verify that the **external-secrets** pods are in a **running** state, run the following command:

```
$ oc get pods -n external-secret
```

The following is example output that the **external-secrets** pods are in a **running** state.

NAME	READY	STATUS	RESTARTS	AGE
bitwarden-sdk-server-5b4cf48766-w7zp7	1/1	Running	0	5m
external-secrets-5854b85dd5-m6zf9	1/1	Running	0	5m
external-secrets-webhook-5cb85b8fdb-6jtbq	1/1	Running	0	5m

2. To verify that the **SecretStore** CR is present, run the following command:

```
$ oc get secretstores.external-secrets.io -A
```

The following is example output from validating that the **SecretStore** is present:

NAMESPACE	NAME	AGE	STATUS	CAPABILITIES
READY				
external-secrets-1	gcp-store	18min	Valid	ReadWrite True
external-secrets-2	aws-secretstore	11min	Valid	ReadWrite True
external-secrets	bitwarden-secretsmanager	20min	Valid	Readwrite True

- To verify that the **ExternalSecret** CR is present, run the following command:

```
$ oc get externalsecrets.external-secrets.io -A
```

The following is example output from validating that the **SecretStore** is present:

NAMESPACE	NAME	STORE	REFRESH INTERVAL
STATUS	READY		
external-secrets-1	gcp-externalsecret	gcp-store	1hr SecretSynced
True			
external-secrets-2	aws-external-secret	aws-secret-store	1hr
SecretSynced	True		
external-secrets	bitwarden	bitwarden-secretsmanager	1hr
SecretSynced	True		

- To verify that the **SecretStore** is **apiVersion: external-secrets.io/v1**, run the following command:

```
$ oc get secretstores.external-secrets.io -n external-secrets-1 gcp-store -o yaml
```

The following is example output that the **SecretStore** is **apiVersion: external-secrets.io/v1**.

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  creationTimestamp: "2025-10-27T11:38:19Z"
  generation: 1
  name: gcp-store
  namespace: external-secrets-1
  resourceVersion: "104519"
  uid: 7bccb0cc-2557-4f4a-9caa-1577f0108f4b
spec:
  .
  .
  .
status:
  capabilities: ReadWrite
  conditions:
  - lastTransitionTime: "2025-10-27T11:38:19Z"
    message: store validated
    reason: Valid
    status: "True"
    type: Ready
```

5. To verify that the **ExternalSecret** is **apiVersion: external-secrets.io/v1**, run the following command:

```
$ oc get externalsecrets.external-secrets.io -n external-secrets-1 gcp-externalsecret -o yaml
```

The following is example output that the **ExternalSecret** is **apiVersion: external-secrets.io/v1**.

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  creationTimestamp: "2025-10-27T11:39:03Z"
  generation: 1
  name: gcp-externalsecret
  namespace: external-secrets-1
  resourceVersion: "104532"
  uid: 93a3295a-a3ad-4304-90e1-1328d951e5fb
spec:
  .
  .
  .
status:
  binding:
    name: k8s-secret-gcp
  conditions:
    - lastTransitionTime: "2025-10-27T11:39:03Z"
      message: secret synced
      reason: SecretSynced
      status: "True"
      type: Ready
  refreshTime: "2025-10-27T12:13:15Z"
  syncedResourceVersion: 1-
f47fe3c0b255b6dd8047cdffa772587bb829efe7a1cb70febeda2eb2
```

CHAPTER 13. VIEWING AUDIT LOGS

OpenShift Container Platform auditing provides a security-relevant chronological set of records documenting the sequence of activities that have affected the system by individual users, administrators, or other components of the system.

13.1. ABOUT THE API AUDIT LOG

Audit works at the API server level, logging all requests coming to the server. Each audit log contains the following information:

Table 13.1. Audit log fields

Field	Description
level	The audit level at which the event was generated.
auditID	A unique audit ID, generated for each request.
stage	The stage of the request handling when this event instance was generated.
requestURI	The request URI as sent by the client to a server.
verb	The Kubernetes verb associated with the request. For non-resource requests, this is the lowercase HTTP method.
user	The authenticated user information.
impersonatedUser	Optional. The impersonated user information, if the request is impersonating another user.
sourceIPs	Optional. The source IPs, from where the request originated and any intermediate proxies.
userAgent	Optional. The user agent string reported by the client. Note that the user agent is provided by the client, and must not be trusted.
objectRef	Optional. The object reference this request is targeted at. This does not apply for List -type requests, or non-resource requests.
responseStatus	Optional. The response status, populated even when the ResponseObject is not a Status type. For successful responses, this will only include the code. For non-status type error responses, this will be auto-populated with the error message.

Field	Description
requestObject	Optional. The API object from the request, in JSON format. The RequestObject is recorded as is in the request (possibly re-encoded as JSON), prior to version conversion, defaulting, admission or merging. It is an external versioned object type, and might not be a valid object on its own. This is omitted for non-resource requests and is only logged at request level and higher.
responseObject	Optional. The API object returned in the response, in JSON format. The ResponseObject is recorded after conversion to the external type, and serialized as JSON. This is omitted for non-resource requests and is only logged at response level.
requestReceivedTimestamp	The time that the request reached the API server.
stageTimestamp	The time that the request reached the current audit stage.
annotations	Optional. An unstructured key value map stored with an audit event that may be set by plugins invoked in the request serving chain, including authentication, authorization and admission plugins. Note that these annotations are for the audit event, and do not correspond to the metadata.annotations of the submitted object. Keys should uniquely identify the informing component to avoid name collisions, for example podsecuritypolicy.admission.k8s.io/policy . Values should be short. Annotations are included in the metadata level.

Example output for the Kubernetes API server:

```
{
  "kind": "Event",
  "apiVersion": "audit.k8s.io/v1",
  "level": "Metadata",
  "auditID": "ad209ce1-fec7-4130-8192-c4cc63f1d8cd",
  "stage": "ResponseComplete",
  "requestURI": "/api/v1/namespaces/openshift-kube-controller-manager/configmaps/cert-recovery-controller-lock?timeout=35s",
  "verb": "update",
  "user": {
    "username": "system:serviceaccount:openshift-kube-controller-manager:localhost-recovery-client",
    "uid": "dd4997e3-d565-4e37-80f8-7fc122ccd785",
    "groups": [
      "system:serviceaccounts",
      "system:serviceaccounts:openshift-kube-controller-manager",
      "system:authenticated"
    ],
    "sourceIPs": [
      "::1"
    ],
    "userAgent": "cluster-kube-controller-manager-operator/v0.0.0 (linux/amd64) kubernetes/$Format",
    "objectRef": {
      "resource": "configmaps",
      "namespace": "openshift-kube-controller-manager",
      "name": "cert-recovery-controller-lock",
      "uid": "5c57190b-6993-425d-8101-8337e48c7548",
      "apiVersion": "v1",
      "resourceVersion": "574307"
    },
    "responseStatus": {
      "metadata": {
        "code": 200
      },
      "requestReceivedTimestamp": "2020-04-02T08:27:20.200962Z",
      "stageTimestamp": "2020-04-02T08:27:20.206710Z",
      "annotations": {
        "authorization.k8s.io/decision": "allow",
        "authorization.k8s.io/reason": "RBAC: allowed by ClusterRoleBinding 'system:openshift:operator:kube-controller-manager-recovery' of ClusterRole 'cluster-admin' to ServiceAccount 'localhost-recovery-client/openshift-kube-controller-manager'"
      }
    }
  }
}
```

13.2. VIEWING THE AUDIT LOGS

You can view the logs for the OpenShift API server, Kubernetes API server, OpenShift OAuth API server, and OpenShift OAuth server for each control plane node.

Procedure

- View the OpenShift API server audit logs:
 - a. List the OpenShift API server audit logs that are available for each control plane node:

```
$ oc adm node-logs --role=master --path=openshift-apiserver/
```

Example output

```
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit-2021-03-09T00-12-19.834.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit-2021-03-09T00-11-49.835.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit-2021-03-09T00-13-00.128.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit.log
```

- b. View a specific OpenShift API server audit log by providing the node name and the log name:

```
$ oc adm node-logs <node_name> --path=openshift-apiserver/<log_name>
```

For example:

```
$ oc adm node-logs ci-ln-m0wpfjb-f76d1-vnb5x-master-0 --path=openshift-apiserver/audit-2021-03-09T00-12-19.834.log
```

Example output

```
{"kind":"Event","apiVersion":"audit.k8s.io/v1","level":"Metadata","auditID":"381acf6d-5f30-4c7d-8175-c9c317ae5893","stage":"ResponseComplete","requestURI":"/metrics","verb":"get","user":{"username":"system:serviceaccount:openshift-monitoring:prometheus-k8s","uid":"825b60a0-3976-4861-a342-3b2b561e8f82","groups":["system:serviceaccounts","system:serviceaccounts:openshift-monitoring","system:authenticated"]},"sourceIPs":["10.129.2.6"],"userAgent":"Prometheus/2.23.0","responseStatus":{"metadata":{"code":200},"requestReceivedTimestamp":"2021-03-08T18:02:04.086545Z","stageTimestamp":"2021-03-08T18:02:04.107102Z","annotations":{"authorization.k8s.io/decision":"allow","authorization.k8s.io/reason":"RBAC: allowed by ClusterRoleBinding \"/>
```

- View the Kubernetes API server audit logs:
 - a. List the Kubernetes API server audit logs that are available for each control plane node:

```
$ oc adm node-logs --role=master --path=kube-apiserver/
```

Example output

```
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit-2021-03-09T14-07-27.129.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit-2021-03-09T19-24-22.620.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit-2021-03-09T18-37-07.511.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit.log
```

- b. View a specific Kubernetes API server audit log by providing the node name and the log name:

```
$ oc adm node-logs <node_name> --path=kube-apiserver/<log_name>
```

For example:

```
$ oc adm node-logs ci-ln-m0wpfjb-f76d1-vnb5x-master-0 --path=kube-apiserver/audit-2021-03-09T14-07-27.129.log
```

Example output

```
{"kind":"Event","apiVersion":"audit.k8s.io/v1","level":"Metadata","auditID":"cfce8a0b-b5f5-4365-8c9f-79c1227d10f9","stage":"ResponseComplete","requestURI":"/api/v1/namespaces/openshift-kube-scheduler/serviceaccounts/openshift-kube-scheduler-sa","verb":"get","user":{"username":"system:serviceaccount:openshift-kube-scheduler-operator:openshift-kube-scheduler-operator","uid":"2574b041-f3c8-44e6-a057-baef7aa81516","groups":["system:serviceaccounts","system:serviceaccounts:openshift-kube-scheduler-operator","system:authenticated"]},"sourceIPs":["10.128.0.8"],"userAgent":"cluster-kube-scheduler-operator/v0.0.0 (linux/amd64) kubernetes/$Format","objectRef":{"resource":"serviceaccounts","namespace":"openshift-kube-scheduler","name":"openshift-kube-scheduler-sa","apiVersion":"v1"},"responseStatus":{"metadata":{"code":200},"requestReceivedTimestamp":"2021-03-08T18:06:42.512619Z","stageTimestamp":"2021-03-08T18:06:42.516145Z"},"annotations":{"authentication.k8s.io/legacy-token":"system:serviceaccount:openshift-kube-scheduler-operator:openshift-kube-scheduler-operator","authorization.k8s.io/decision":"allow","authorization.k8s.io/reason":"RBAC: allowed by ClusterRoleBinding \"system:openshift:operator:cluster-kube-scheduler-operator\" of ClusterRole \"cluster-admin\" to ServiceAccount \"openshift-kube-scheduler-operator/openshift-kube-scheduler-operator\"\"}}
```

- View the OpenShift OAuth API server audit logs:
 - a. List the OpenShift OAuth API server audit logs that are available for each control plane node:

```
$ oc adm node-logs --role=master --path=oauth-apiserver/
```

Example output

```
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit-2021-03-09T13-06-26.128.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit-2021-03-09T18-23-21.619.log
```



```
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit-2021-03-09T17-36-06.510.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit.log
```

- b. View a specific OpenShift OAuth API server audit log by providing the node name and the log name:

```
$ oc adm node-logs <node_name> --path=oauth-apiserver/<log_name>
```

For example:

```
$ oc adm node-logs ci-ln-m0wpfjb-f76d1-vnb5x-master-0 --path=oauth-apiserver/audit-2021-03-09T13-06-26.128.log
```

Example output

```
{"kind":"Event","apiVersion":"audit.k8s.io/v1","level":"Metadata","auditID":"dd4c44e2-3ea1-4830-9ab7-c91a5f1388d6","stage":"ResponseComplete","requestURI":"/apis/user.openshift.io/v1/users/~","verb":"get","user":{"username":"system:serviceaccount:openshift-monitoring:prometheus-k8s","groups":["system:serviceaccounts","system:serviceaccounts:openshift-monitoring","system:authenticated"]},"sourceIPs":["10.0.32.4","10.128.0.1"],"userAgent":"dockerregistry/v0.0.0 (linux/amd64) kubernetes/$Format","objectRef":{"resource":"users","name":"~","apiGroup":"user.openshift.io","apiVersion":"v1"},"responseStatus":{"metadata":{"code":200},"requestReceivedTimestamp":"2021-03-08T17:47:43.653187Z","stageTimestamp":"2021-03-08T17:47:43.660187Z"},"annotations":{"authorization.k8s.io/decision":"allow","authorization.k8s.io/reason":"RBAC: allowed by ClusterRoleBinding \"basic-users\" of ClusterRole \"basic-user\" to Group \"system:authenticated\"\"}}
```

- View the OpenShift OAuth server audit logs:
 - a. List the OpenShift OAuth server audit logs that are available for each control plane node:

```
$ oc adm node-logs --role=master --path=oauth-server/
```

Example output

```
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit-2022-05-11T18-57-32.395.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-0 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit-2022-05-11T19-07-07.021.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-1 audit.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit-2022-05-11T19-06-51.844.log
ci-ln-m0wpfjb-f76d1-vnb5x-master-2 audit.log
```

- b. View a specific OpenShift OAuth server audit log by providing the node name and the log name:

```
$ oc adm node-logs <node_name> --path=oauth-server/<log_name>
```

For example:

```
$ oc adm node-logs ci-ln-m0wpfjb-f76d1-vnb5x-master-0 --path=oauth-server/audit-2022-05-11T18-57-32.395.log
```

Example output

```
{
  "kind": "Event",
  "apiVersion": "audit.k8s.io/v1",
  "level": "Metadata",
  "auditID": "13c20345-f33b-4b7d-b3b6-e7793f805621",
  "stage": "ResponseComplete",
  "requestURI": "/login",
  "verb": "post",
  "user": {
    "username": "system:anonymous",
    "groups": ["system:unauthenticated"],
    "sourceIPs": ["10.128.2.6"],
    "userAgent": "Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0",
    "responseStatus": {
      "metadata": {
        "code": 302
      },
      "requestReceivedTimestamp": "2022-05-11T17:31:16.280155Z",
      "stageTimestamp": "2022-05-11T17:31:16.297083Z",
      "annotations": {
        "authentication.openshift.io/decision": "error",
        "authentication.openshift.io/username": "kubeadmin",
        "authorization.k8s.io/decision": "allow",
        "authorization.k8s.io/reason": ""
      }
    }
  }
}
```

The possible values for the **authentication.openshift.io/decision** annotation are **allow**, **deny**, or **error**.

13.3. FILTERING AUDIT LOGS

You can use **jq** or another JSON parsing tool to filter the API server audit logs.



NOTE

The amount of information logged to the API server audit logs is controlled by the audit log policy that is set.

The following procedure provides examples of using **jq** to filter audit logs on control plane node **node-1.example.com**. See the [jq Manual](#) for detailed information on using **jq**.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed **jq**.

Procedure

- Filter OpenShift API server audit logs by user:

```
$ oc adm node-logs node-1.example.com \
  --path=openshift-apiserver/audit.log \
  | jq 'select(.user.username == "myusername")'
```

- Filter OpenShift API server audit logs by user agent:

```
$ oc adm node-logs node-1.example.com \
  --path=openshift-apiserver/audit.log \
```

```
| jq 'select(.userAgent == "cluster-version-operator/v0.0.0 (linux/amd64)
kubernetes/$Format")'
```

- Filter Kubernetes API server audit logs by a certain API version and only output the user agent:

```
$ oc adm node-logs node-1.example.com \
--path=kube-apiserver/audit.log \
| jq 'select(.requestURI | startswith("/apis/apiextensions.k8s.io/v1beta1")) | .userAgent'
```

- Filter OpenShift OAuth API server audit logs by excluding a verb:

```
$ oc adm node-logs node-1.example.com \
--path=oauth-apiserver/audit.log \
| jq 'select(.verb != "get")'
```

- Filter OpenShift OAuth server audit logs by events that identified a username and failed with an error:

```
$ oc adm node-logs node-1.example.com \
--path=oauth-server/audit.log \
| jq 'select(.annotations["authentication.openshift.io/username"] != null and
.annotations["authentication.openshift.io/decision"] == "error")'
```

13.4. GATHERING AUDIT LOGS

You can use the `must-gather` tool to collect the audit logs for debugging your cluster, which you can review or send to Red Hat Support.

Procedure

1. Run the **`oc adm must-gather`** command with **`-- /usr/bin/gather_audit_logs`**:

```
$ oc adm must-gather -- /usr/bin/gather_audit_logs
```

2. Create a compressed file from the **`must-gather`** directory that was just created in your working directory. For example, on a computer that uses a Linux operating system, run the following command:

```
$ tar cvaf must-gather.tar.gz must-gather.local.472290403699006248
```

Replace **`must-gather.local.472290403699006248`** with the actual directory name.

3. Attach the compressed file to your support case on the [the Customer Support page](#) of the Red Hat Customer Portal.

13.5. ADDITIONAL RESOURCES

- [Must-gather tool](#)
- [API audit log event structure](#)
- [Configuring the audit log policy](#)

CHAPTER 14. CONFIGURING THE AUDIT LOG POLICY

You can control the amount of information that is logged to the API server audit logs by choosing the audit log policy profile to use.

14.1. ABOUT AUDIT LOG POLICY PROFILES

To monitor activity and maintain compliance, you can apply audit log profiles that define the level of detail recorded for API server requests. While more comprehensive profiles provide request bodies for troubleshooting, they also increase resource overhead on the host system.

Audit log profiles define how to log requests that come to the OpenShift API server, Kubernetes API server, OpenShift OAuth API server, and OpenShift OAuth server.

OpenShift Container Platform provides the following predefined audit policy profiles:

Profile	Description
Default	Logs only metadata for read and write requests; does not log request bodies except for OAuth access token requests. This is the default policy.
WriteRequestBodies	In addition to logging metadata for all requests, logs request bodies for every write request to the API servers (create, update, patch, delete, deletecollection). This profile has more resource overhead than the Default profile. ^[1]
AllRequestBodies	In addition to logging metadata for all requests, logs request bodies for every read and write request to the API servers (get, list, create, update, patch). This profile has the most resource overhead. ^[1]
None	No requests are logged, including OAuth access token requests and OAuth authorize token requests. Custom rules are ignored when this profile is set.  WARNING Do not disable audit logging by using the None profile unless you are fully aware of the risks of not logging data that can be beneficial when troubleshooting issues. If you disable audit logging and a support situation arises, you might need to enable audit logging and reproduce the issue to troubleshoot properly.

1. Sensitive resources, such as **Secret**, **Route**, and **OAuthClient** objects, are only logged at the metadata level. OpenShift OAuth server events are only logged at the metadata level.

By default, OpenShift Container Platform uses the **Default** audit log profile. You can use another audit policy profile that also logs request bodies, but be aware of the increased resource usage such as CPU, memory, and I/O.

14.2. CONFIGURING THE AUDIT LOG POLICY

You can configure the audit log policy to use when logging requests that come to the API servers.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Edit the **APIServer** resource:

```
$ oc edit apiserver cluster
```

2. Update the **spec.audit.profile** field:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  ...
spec:
  audit:
    profile: WriteRequestBodies
```

where:

profile

Set to **Default**, **WriteRequestBodies**, **AllRequestBodies**, or **None**. The default profile is **Default**.



WARNING

It is not recommended to disable audit logging by using the **None** profile unless you are fully aware of the risks of not logging data that can be beneficial when troubleshooting issues. If you disable audit logging and a support situation arises, you might need to enable audit logging and reproduce the issue in order to troubleshoot properly.

3. Save the file to apply the changes.

Verification

- Verify that a new revision of the Kubernetes API server pods is rolled out. It can take several minutes for all nodes to update to the new revision.

```
$ oc get kubeapiserver -o=jsonpath='{range .items[0].status.conditions[?(@.type=="NodeInstallerProgressing")]}{.reason}{"\n"}{.message}{"\n"}'
```

Review the **NodeInstallerProgressing** status condition for the Kubernetes API server to verify that all nodes are at the latest revision. The output shows **AllNodesAtLatestRevision** upon successful update:

```
AllNodesAtLatestRevision
3 nodes are at revision 12
```

In this example, the latest revision number is **12**.

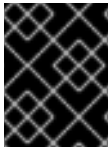
If the output shows a message similar to one of the following messages, the update is still in progress. Wait a few minutes and try again.

- **3 nodes are at revision 11; 0 nodes have achieved new revision 12**
- **2 nodes are at revision 11; 1 nodes are at revision 12**

14.3. CONFIGURING THE AUDIT LOG POLICY WITH CUSTOM RULES

You can configure an audit log policy that defines custom rules. You can specify multiple groups and define which profile to use for that group.

These custom rules take precedence over the top-level profile field. The custom rules are evaluated from top to bottom, and the first that matches is applied.



IMPORTANT

If you set the top-level profile field to **None**, an API server, such as the Kubernetes API server, ignores custom rules and disables audit logging.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Edit the **APIServer** resource:

```
$ oc edit apiserver cluster
```

2. Add the **spec.audit.customRules** field:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
...
spec:
  audit:
```

```

customRules:
- group: system:authenticated:oauth
  profile: WriteRequestBodies
- group: system:authenticated
  profile: AllRequestBodies
profile: Default

```

where:

customRules

Add one or more groups and specify the profile to use for that group. These custom rules take precedence over the top-level profile field. The custom rules are evaluated from top to bottom, and the first that matches is applied.

profile

Set to **Default**, **WriteRequestBodies**, or **AllRequestBodies**. If you do not set this top-level profile field, it defaults to the **Default** profile.

3. Save the file to apply the changes.

Verification

- Verify that a new revision of the Kubernetes API server pods is rolled out. It can take several minutes for all nodes to update to the new revision.

```

$ oc get kubeapiserver -o=jsonpath='{range .items[0].status.conditions[?
(@.type=="NodeInstallerProgressing")]}{.reason}"\\n"}{.message}"\\n"}'

```

Review the **NodeInstallerProgressing** status condition for the Kubernetes API server to verify that all nodes are at the latest revision. The output shows **AllNodesAtLatestRevision** upon successful update:

```

AllNodesAtLatestRevision
3 nodes are at revision 12

```

In this example, the latest revision number is **12**.

If the output shows a message similar to one of the following messages, the update is still in progress. Wait a few minutes and try again.

- **3 nodes are at revision 11; 0 nodes have achieved new revision 12**
- **2 nodes are at revision 11; 1 nodes are at revision 12**

14.4. DISABLING AUDIT LOGGING

You can disable audit logging for OpenShift Container Platform. When you disable audit logging, even OAuth access token requests and OAuth authorize token requests are not logged.

**WARNING**

It is not recommended to disable audit logging by using the **None** profile unless you are fully aware of the risks of not logging data that can be beneficial when troubleshooting issues. If you disable audit logging and a support situation arises, you might need to enable audit logging and reproduce the issue in order to troubleshoot properly.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Edit the **APIServer** resource:

```
$ oc edit apiserver cluster
```

2. Set the **spec.audit.profile** field to **None**:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  ...
spec:
  audit:
    profile: None
```

**NOTE**

You can also disable audit logging only for specific groups by specifying custom rules in the **spec.audit.customRules** field.

3. Save the file to apply the changes.

Verification

- Verify that a new revision of the Kubernetes API server pods is rolled out. It can take several minutes for all nodes to update to the new revision.

```
$ oc get kubeapiserver -o=jsonpath='{range .items[0].status.conditions[?(@.type=="NodeInstallerProgressing")]}{.reason}{"\n"}{.message}{"\n"}'
```

Review the **NodeInstallerProgressing** status condition for the Kubernetes API server to verify that all nodes are at the latest revision. The output shows **AllNodesAtLatestRevision** upon successful update:


```
AllNodesAtLatestRevision  
3 nodes are at revision 12
```

In this example, the latest revision number is **12**.

If the output shows a message similar to one of the following messages, the update is still in progress. Wait a few minutes and try again.

- **3 nodes are at revision 11; 0 nodes have achieved new revision 12**
- **2 nodes are at revision 11; 1 nodes are at revision 12**

CHAPTER 15. CONFIGURING TLS SECURITY PROFILES

To enforce secure cryptographic libraries for the OpenShift Container Platform components, cluster administrators can configure TLS security profiles to control cipher usage when the client connects to the Ingress Controller, the control plane, or the kubelet.

The control plane includes the following components:

- Kubernetes API server
- Kubernetes controller manager
- Kubernetes scheduler
- OpenShift API server
- OpenShift OAuth API server
- OpenShift OAuth server
- etcd
- Machine Config Operator
- Machine Config Server.

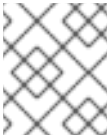
15.1. UNDERSTANDING TLS SECURITY PROFILES

You can use a TLS (Transport Layer Security) security profile, as described in this section, to define which TLS ciphers are required by various OpenShift Container Platform components.

The OpenShift Container Platform TLS security profiles are based on [Mozilla recommended configurations](#).

You can specify one of the following TLS security profiles for each component:

Table 15.1. TLS security profiles

Profile	Description
Old	This profile is intended for use with legacy clients or libraries. The profile is based on the Old backward compatibility recommended configuration. The Old profile requires a minimum TLS version of 1.0. NOTE For the Ingress Controller, the minimum TLS version is converted from 1.0 to 1.1.

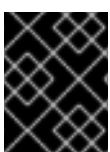
Profile	Description
Intermediate	This profile is the default TLS security profile for the Ingress Controller, kubelet, and control plane. The profile is based on the Intermediate compatibility recommended configuration. The Intermediate profile requires a minimum TLS version of 1.2.  NOTE This profile is the recommended configuration for the majority of clients.
Modern	This profile is intended for use with modern clients that have no need for backwards compatibility. This profile is based on the Modern compatibility recommended configuration. The Modern profile requires a minimum TLS version of 1.3.
Custom	This profile allows you to define the TLS version and ciphers to use.  WARNING Use caution when using a Custom profile, because invalid configurations can cause problems.

**NOTE**

When using one of the predefined profile types, the effective profile configuration is subject to change between releases. For example, given a specification to use the Intermediate profile deployed on release X.Y.Z, an upgrade to release X.Y.Z+1 might cause a new profile configuration to be applied, resulting in a rollout.

15.2. VIEWING TLS SECURITY PROFILE DETAILS

To check the minimum TLS version and ciphers that a security profile applies in OpenShift Container Platform, you can inspect the profile configuration for the Ingress Controller, control plane, or kubelet. Use the **oc explain** command to display settings for a predefined or custom profile.

**IMPORTANT**

The effective configuration of minimum TLS version and list of ciphers for a profile might differ between components.

Procedure

- View details for a specific TLS security profile:

```
$ oc explain <component>.spec.tlsSecurityProfile.<profile>
```

- For **<component>**, specify **ingresscontroller**, **apiserver**, or **kubeletconfig**. For **<profile>**, specify **old**, **intermediate**, or **custom**.
For example, to check the ciphers included for the **intermediate** profile for the control plane:

```
$ oc explain apiserver.spec.tlsSecurityProfile.intermediate
```

Example output

```
KIND:   APIServer
VERSION: config.openshift.io/v1

DESCRIPTION:
  intermediate is a TLS security profile based on:

  https://wiki.mozilla.org/Security/Server_Side_TLS#Intermediate_compatibility_.28recommended_.29
  and looks like this (yaml):
  ciphers: - TLS_AES_128_GCM_SHA256 - TLS_AES_256_GCM_SHA384 -
  TLS_CHACHA20_POLY1305_SHA256 - ECDHE-ECDSA-AES128-GCM-SHA256 -
  ECDHE-RSA-AES128-GCM-SHA256 - ECDHE-ECDSA-AES256-GCM-SHA384 -
  ECDHE-RSA-AES256-GCM-SHA384 - ECDHE-ECDSA-CHACHA20-POLY1305 -
  ECDHE-RSA-CHACHA20-POLY1305 - DHE-RSA-AES128-GCM-SHA256 -
  DHE-RSA-AES256-GCM-SHA384 minTLSVersion: TLSv1.2
```

- View all details for the **tlsSecurityProfile** field of a component:

```
$ oc explain <component>.spec.tlsSecurityProfile
```

- For **<component>**, specify **ingresscontroller**, **apiserver**, or **kubeletconfig**.
For example, to check all details for the **tlsSecurityProfile** field for the Ingress Controller:

```
$ oc explain ingresscontroller.spec.tlsSecurityProfile
```

Example output

```
KIND:   IngressController
VERSION: operator.openshift.io/v1

RESOURCE: tlsSecurityProfile <Object>

DESCRIPTION:
  ...

FIELDS:
  custom <>
    custom is a user-defined TLS security profile. Be extremely careful using a
    custom profile as invalid configurations can be catastrophic. An example
    custom profile looks like this:
    ciphers: - ECDHE-ECDSA-CHACHA20-POLY1305 - ECDHE-RSA-CHACHA20-
    POLY1305 -
    ECDHE-RSA-AES128-GCM-SHA256 - ECDHE-ECDSA-AES128-GCM-SHA256
```

```

minTLSVersion:
  TLSv1.1

intermediate <>
  intermediate is a TLS security profile based on:

https://wiki.mozilla.org/Security/Server_Side_TLS#Intermediate_compatibility_.28recommended.29
  and looks like this (yaml):
  (A list of ciphers and the minimum version for the intermediate profile opens here.)

modern <>
  modern is a TLS security profile based on:
  https://wiki.mozilla.org/Security/Server_Side_TLS#Modern_compatibility and
  looks like this (yaml):
  (A list of ciphers and the minimum version for the modern profile opens here.)
  NOTE: Currently unsupported.

old <>
  old is a TLS security profile based on:
  https://wiki.mozilla.org/Security/Server_Side_TLS#Old_backward_compatibility
  and looks like this (yaml):
  (A list of ciphers and the minimum version for the old profile opens here.)

type <string>
  ...

```

15.3. CONFIGURING THE TLS SECURITY PROFILE FOR THE INGRESS CONTROLLER

To configure a TLS security profile for an Ingress Controller, edit the **IngressController** custom resource (CR) to specify a predefined or custom TLS security profile.

If a TLS security profile is not configured, the default value is based on the TLS security profile set for the API server, as shown in the following example:

```

apiVersion: operator.openshift.io/v1
kind: IngressController
...
spec:
  tlsSecurityProfile:
    old: {}
    type: Old

```

The TLS security profile defines the minimum TLS version and the TLS ciphers for TLS connections for Ingress Controllers.

You can see the ciphers and the minimum TLS version of the configured TLS security profile in the **IngressController** custom resource (CR) under **Status.Tls Profile** and the configured TLS security profile under **Spec.Tls Security Profile**. For the **Custom** TLS security profile, the specific ciphers and minimum TLS version are listed under both parameters.

**NOTE**

The HAProxy Ingress Controller image supports TLS **1.3** and the **Modern** profile.

The Ingress Operator also converts the TLS **1.0** of an **Old** or **Custom** profile to **1.1**.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Edit the **IngressController** CR in the **openshift-ingress-operator** project to configure the TLS security profile:

```
$ oc edit IngressController default -n openshift-ingress-operator.
```

- Add the **spec.tlsSecurityProfile** field:

Sample IngressController CR for a Custom profile

```
apiVersion: operator.openshift.io/v1
kind: IngressController
...
spec:
  tlsSecurityProfile:
    type: Custom
    custom:
      ciphers:
        - ECDHE-ECDSA-CHACHA20-POLY1305
        - ECDHE-RSA-CHACHA20-POLY1305
        - ECDHE-RSA-AES128-GCM-SHA256
        - ECDHE-ECDSA-AES128-GCM-SHA256
      minTLSVersion: VersionTLS11
    ...
```

- Specify the value for the **spec.tlsSecurityProfile** parameter. The TLS security profile types are **Old**, **Intermediate**, or **Custom**. The default type is **Intermediate**.
 - Specify the appropriate field for the selected **spec.tlsSecurityProfile.type**. The fields are **old: {}**, **intermediate: {}**, **modern: {}**, or **custom: {}**.
 - For the **custom** type, specify a list of TLS ciphers and the minimum accepted TLS version.
- Save the file to apply the changes.

Verification

- Verify that the profile is set in the **IngressController** CR:

```
$ oc describe IngressController default -n openshift-ingress-operator
```

Example output

```
-
```

```

Name:      default
Namespace: openshift-ingress-operator
Labels:    <none>
Annotations: <none>
API Version: operator.openshift.io/v1
Kind:      IngressController
...
Spec:
...
  Tls Security Profile:
    Custom:
      Ciphers:
        ECDHE-ECDSA-CHACHA20-POLY1305
        ECDHE-RSA-CHACHA20-POLY1305
        ECDHE-RSA-AES128-GCM-SHA256
        ECDHE-ECDSA-AES128-GCM-SHA256
      Min TLS Version: VersionTLS11
    Type:      Custom
  ...

```

15.4. CONFIGURING THE TLS SECURITY PROFILE FOR THE CONTROL PLANE

To configure a TLS security profile for the control plane, edit the **APIServer** custom resource (CR) to specify a predefined or custom TLS security profile.

Setting the TLS security profile in the **APIServer** CR propagates the setting to the following control plane components:

- Kubernetes API server
- Kubernetes controller manager
- Kubernetes scheduler
- OpenShift API server
- OpenShift OAuth API server
- OpenShift OAuth server
- etcd
- Machine Config Operator
- Machine Config Server



NOTE

The default TLS security profile for the Ingress Controller is based on the TLS security profile set for the API server.

If a TLS security profile is not configured, the default TLS security profile is **Intermediate**.

The following YAML is a sample **APIServer** CR that configures the **Old** TLS security profile.

```
apiVersion: config.openshift.io/v1
kind: APIServer
...
spec:
  tlsSecurityProfile:
    old: {}
    type: Old
  ...
```

The TLS security profile defines the minimum TLS version and the TLS ciphers required to communicate with the control plane components.

You can see the configured TLS security profile in the **APIServer** custom resource (CR) under **Spec.Tls Security Profile**. For the **Custom** TLS security profile, the specific ciphers and minimum TLS version are listed.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Edit the default **APIServer** CR to configure the TLS security profile:

```
$ oc edit APIServer cluster
```

2. Add the **spec.tlsSecurityProfile** field:

Sample APIServer CR for a Custom profile

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  name: cluster
spec:
  tlsSecurityProfile:
    type: Custom
    custom:
      ciphers:
        - ECDHE-ECDSA-CHACHA20-POLY1305
        - ECDHE-RSA-CHACHA20-POLY1305
        - ECDHE-RSA-AES128-GCM-SHA256
        - ECDHE-ECDSA-AES128-GCM-SHA256
      minTLSVersion: VersionTLS11
```

- Specify the value for the **spec.tlsSecurityProfile.type** parameter. The TLS security profile types are **Old**, **Intermediate**, or **Custom**. The default type is **Intermediate**.
- Specify the appropriate field for the selected **spec.tlsSecurityProfile**. The fields are **old: {}**, **intermediate: {}**, **modern: {}**, or **custom:**.
- For the **custom** type, specify a list of TLS ciphers and the minimum accepted TLS version.

3. Save the file to apply the changes.

Verification

1. Verify that the TLS security profile is set in the **APIServer** CR:

```
$ oc describe apiserver cluster
```

Example output

```
Name:      cluster
Namespace:
...
API Version: config.openshift.io/v1
Kind:      APIServer
...
Spec:
  Audit:
    Profile: Default
  Tls Security Profile:
    Custom:
      Ciphers:
        ECDHE-ECDSA-CHACHA20-POLY1305
        ECDHE-RSA-CHACHA20-POLY1305
        ECDHE-RSA-AES128-GCM-SHA256
        ECDHE-ECDSA-AES128-GCM-SHA256
      Min TLS Version: VersionTLS11
    Type:      Custom
  ...
```

2. Verify that the TLS security profile is set in the **etcd** CR:

```
$ oc describe etcd cluster
```

Example output

```
Name:      cluster
Namespace:
...
API Version: operator.openshift.io/v1
Kind:      Etcd
...
Spec:
  Log Level:      Normal
  Management State: Managed
  Observed Config:
    Serving Info:
      Cipher Suites:
        TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
        TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
        TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
        TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
        TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
```

```
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256
Min TLS Version:      VersionTLS12
...
```

3. Verify that the TLS security profile is set in the Machine Config Server pod:

```
$ oc logs machine-config-server-5msdv -n openshift-machine-config-operator
```

Example output

```
# ...
I0905 13:48:36.968688    1 start.go:51] Launching server with tls min version:
VersionTLS12 & cipher suites [TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256
TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256]
# ...
```

15.5. CONFIGURING THE TLS SECURITY PROFILE FOR THE KUBELET

To configure TLS ciphers and minimum versions for the kubelet HTTP server in OpenShift Container Platform, apply a predefined or custom TLS security profile through a **KubeletConfig** custom resource (CR). Without a custom profile, the kubelet defaults to the **Intermediate** profile.

- The kubelet uses its HTTP/GRPC server to communicate with the Kubernetes API server, which sends commands to pods, gathers logs, and run exec commands on pods through the kubelet.

Sample KubeletConfig CR that configures the Old TLS security profile on worker nodes

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
# ...
spec:
  tlsSecurityProfile:
    old: {}
    type: Old
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ""
# ...
```

You can see the ciphers and the minimum TLS version of the configured TLS security profile in the **kubelet.conf** file on a configured node.

Prerequisites

- You are logged in to OpenShift Container Platform as a user with the **cluster-admin** role.

Procedure

1. Create a **KubeletConfig** CR to configure the TLS security profile:

Sample KubeletConfig CR for a Custom profile

```

apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: set-kubelet-tls-security-profile
spec:
  tlsSecurityProfile:
    type: Custom
    custom:
      ciphers:
        - ECDHE-ECDSA-CHACHA20-POLY1305
        - ECDHE-RSA-CHACHA20-POLY1305
        - ECDHE-RSA-AES128-GCM-SHA256
        - ECDHE-ECDSA-AES128-GCM-SHA256
      minTLSVersion: VersionTLS11
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ""
#...
```

where:

spec.tlsSecurityProfile.type

Specifies the TLS security profile type (**Old**, **Intermediate**, or **Custom**). The default is **Intermediate**.

spec.tlsSecurityProfile.type.custom

Specifies the appropriate field for the selected type:

- **old:** {}
- **intermediate:** {}
- **modern:** {}
- **custom:**

spec.tlsSecurityProfile.type.custom

For the **custom** type, specifies a list of TLS ciphers and the minimum accepted TLS version.

spec.machineConfigPoolSelector.matchLabels.custom

Specifies the machine config pool label for the nodes you want to apply the TLS security profile. This parameter is optional.

2. Create the **KubeletConfig** object:

```
$ oc create -f <filename>
```

Depending on the number of worker nodes in the cluster, wait for the configured nodes to be rebooted one by one.

Verification

To verify that the profile is set, perform the following steps after the nodes are in the **Ready** state:

1. Start a debug session for a configured node:

```
$ oc debug node/<node_name>
```

2. Set **/host** as the root directory within the debug shell:

```
sh-4.4# chroot /host
```

3. View the **kubelet.conf** file:

```
sh-4.4# cat /etc/kubernetes/kubelet.conf
```

Example output

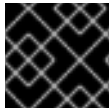
```
"kind": "KubeletConfiguration",
"apiVersion": "kubelet.config.k8s.io/v1beta1",
#...
"tlsCipherSuites": [
  "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256",
  "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",
  "TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384",
  "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384",
  "TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256",
  "TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256"
],
"tlsMinVersion": "VersionTLS12",
#...
```

CHAPTER 16. CONFIGURING SECCOMP PROFILES

An OpenShift Container Platform container or a pod runs a single application that performs one or more well-defined tasks. The application usually requires only a small subset of the underlying operating system kernel APIs. Secure computing mode, seccomp, is a Linux kernel feature that can be used to limit the process running in a container to only using a subset of the available system calls.

The **restricted-v2** SCC applies to all newly created pods in 4.21. The default seccomp profile **runtime/default** is applied to these pods.

Seccomp profiles are stored as JSON files on the disk.



IMPORTANT

Seccomp profiles cannot be applied to privileged containers.

16.1. VERIFYING THE DEFAULT SECCOMP PROFILE APPLIED TO A POD

OpenShift Container Platform ships with a default seccomp profile that is referenced as **runtime/default**. In 4.21, newly created pods have the Security Context Constraint (SCC) set to **restricted-v2** and the default seccomp profile applies to the pod.

Procedure

1. You can verify the Security Context Constraint (SCC) and the default seccomp profile set on a pod by running the following commands:
 - a. Verify what pods are running in the namespace:

```
$ oc get pods -n <namespace>
```

For example, to verify what pods are running in the **workshop** namespace run the following:

```
$ oc get pods -n workshop
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
parksmap-1-4xkwf	1/1	Running	0	2m17s
parksmap-1-deploy	0/1	Completed	0	2m22s

- b. Inspect the pods:

```
$ oc get pod parksmap-1-4xkwf -n workshop -o yaml
```

Example output

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
```

```

k8s.v1.cni.cncf.io/network-status: |-
  [{
    "name": "ovn-kubernetes",
    "interface": "eth0",
    "ips": [
      "10.131.0.18"
    ],
    "default": true,
    "dns": {}
  }]
k8s.v1.cni.cncf.io/network-status: |-
  [{
    "name": "ovn-kubernetes",
    "interface": "eth0",
    "ips": [
      "10.131.0.18"
    ],
    "default": true,
    "dns": {}
  }]
openshift.io/deployment-config.latest-version: "1"
openshift.io/deployment-config.name: parksmapi
openshift.io/deployment.name: parksmapi-1
openshift.io/generated-by: OpenShiftWebConsole
openshift.io/scc: restricted-v2 ❶
seccomp.security.alpha.kubernetes.io/pod: runtime/default ❷

```

- ❶ ❶ The **restricted-v2** SCC is added by default if your workload does not have access to a different SCC.
- ❷ Newly created pods in 4.21 will have the seccomp profile configured to **runtime/default** as mandated by the SCC.

16.1.1. Upgraded cluster

In clusters upgraded to 4.21 all authenticated users have access to the **restricted** and **restricted-v2** SCC.

A workload admitted by the SCC **restricted** for example, on a OpenShift Container Platform v4.10 cluster when upgraded may get admitted by **restricted-v2**. This is because **restricted-v2** is the more restrictive SCC between **restricted** and **restricted-v2**.



NOTE

The workload must be able to run with **restricted-v2**.

Conversely with a workload that requires **privilegeEscalation: true** this workload will continue to have the **restricted** SCC available for any authenticated user. This is because **restricted-v2** does not allow **privilegeEscalation**.

16.1.2. Newly installed cluster

For newly installed OpenShift Container Platform 4.11 or later clusters, the **restricted-v2** replaces the

restricted SCC as an SCC that is available to be used by any authenticated user. A workload with **privilegeEscalation: true**, is not admitted into the cluster since **restricted-v2** is the only SCC available for authenticated users by default.

The feature **privilegeEscalation** is allowed by **restricted** but not by **restricted-v2**. More features are denied by **restricted-v2** than were allowed by **restricted** SCC.

A workload with **privilegeEscalation: true** may be admitted into a newly installed OpenShift Container Platform 4.11 or later cluster. To give access to the **restricted** SCC to the ServiceAccount running the workload (or any other SCC that can admit this workload) using a RoleBinding run the following command:

```
$ oc -n <workload-namespace> adm policy add-scc-to-user <scc-name> -z <serviceaccount_name>
```

In OpenShift Container Platform 4.21 the ability to add the pod annotations **seccomp.security.alpha.kubernetes.io/pod: runtime/default** and **container.seccomp.security.alpha.kubernetes.io/<container_name>: runtime/default** is deprecated.

16.2. CONFIGURING A CUSTOM SECCOMP PROFILE

You can configure a custom seccomp profile, which allows you to update the filters based on the application requirements. This allows cluster administrators to have greater control over the security of workloads running in OpenShift Container Platform.

Seccomp security profiles list the system calls (syscalls) a process can make. Permissions are broader than SELinux, which restrict operations, such as **write**, system-wide.

16.2.1. Creating seccomp profiles

You can use the **MachineConfig** object to create profiles.

Seccomp can restrict system calls (syscalls) within a container, limiting the access of your application.

Prerequisites

- You have cluster admin permissions.
- You have created a custom security context constraints (SCC). For more information, see *Additional resources*.

Procedure

- Create the **MachineConfig** object:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: custom-seccomp
spec:
  config:
    ignition:
      version: 3.2.0
```

```

storage:
  files:
    - contents:
        source: data:text/plain;charset=utf-8;base64,<hash>
        filesystem: root
        mode: 0644
        path: /var/lib/kubelet/seccomp/seccomp-nostat.json

```

16.2.2. Setting up the custom seccomp profile

Prerequisite

- You have cluster administrator permissions.
- You have created a custom security context constraints (SCC). For more information, see "Additional resources".
- You have created a custom seccomp profile.

Procedure

1. Upload your custom seccomp profile to **/var/lib/kubelet/seccomp/<custom-name>.json** by using the Machine Config. See "Additional resources" for detailed steps.
2. Update the custom SCC by providing reference to the created custom seccomp profile:

```

seccompProfiles:
- localhost/<custom-name>.json 1

```

- 1 Provide the name of your custom seccomp profile.

16.2.3. Applying the custom seccomp profile to the workload

Prerequisite

- The cluster administrator has set up the custom seccomp profile. For more details, see "Setting up the custom seccomp profile".

Procedure

- Apply the seccomp profile to the workload by setting the **securityContext.seccompProfile.type** field as following:

Example

```

spec:
  securityContext:
    seccompProfile:
      type: Localhost
      localhostProfile: <custom-name>.json 1

```

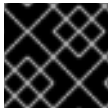
- 1 Provide the name of your custom seccomp profile.

Alternatively, you can use the pod annotations **seccomp.security.alpha.kubernetes.io/pod:localhost/<custom-name>.json**. However, this method is deprecated in OpenShift Container Platform 4.21.

During deployment, the admission controller validates the following:

- The annotations against the current SCCs allowed by the user role.
- The SCC, which includes the seccomp profile, is allowed for the pod.

If the SCC is allowed for the pod, the kubelet runs the pod with the specified seccomp profile.



IMPORTANT

Ensure that the seccomp profile is deployed to all worker nodes.



NOTE

The custom SCC must have the appropriate priority to be automatically assigned to the pod or meet other conditions required by the pod, such as allowing CAP_NET_ADMIN.

16.3. ADDITIONAL RESOURCES

- [Managing security context constraints](#)
- [Machine Config Overview](#)

CHAPTER 17. ALLOWING JAVASCRIPT-BASED ACCESS TO THE API SERVER FROM ADDITIONAL HOSTS

By default, the cluster restricts API server requests to the web console for security. Because the default configuration only permits the web console, you must update the API Server configuration of the cluster to approve additional hostnames for API and OAuth access.

17.1. ALLOWING JAVASCRIPT-BASED ACCESS TO THE API SERVER FROM ADDITIONAL HOSTS

If you need to access the API server or OAuth server from a JavaScript application by using a different hostname, you can configure additional hostnames to allow.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Edit the **APIServer** resource:

```
$ oc edit apiserver.config.openshift.io cluster
```

2. Add the **additionalCORSAAllowedOrigins** field under the **spec** section and specify one or more additional hostnames:

```
apiVersion: config.openshift.io/v1
kind: APIServer
metadata:
  annotations:
    release.openshift.io/create-only: "true"
    creationTimestamp: "2019-07-11T17:35:37Z"
    generation: 1
    name: cluster
    resourceVersion: "907"
    selfLink: /apis/config.openshift.io/v1/apiservers/cluster
    uid: 4b45a8dd-a402-11e9-91ec-0219944e0696
spec:
  additionalCORSAAllowedOrigins:
    - (?i)//my\.subdomain\.domain\.com(:|\z)
```

where:

additionalCORSAAllowedOrigins

The hostname is specified as a [Golang regular expression](#) that matches against CORS headers from HTTP requests against the API server and OAuth server.



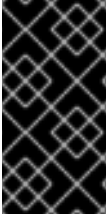
NOTE

This example uses the following syntax:

- The **(?i)** makes it case-insensitive.
- The **//** pins to the beginning of the domain and matches the double slash following **http:** or **https:**.
- The **\.** escapes dots in the domain name.
- The **(:|\z)** matches the end of the domain name **(\z)** or a port separator **(:)**.

3. Save the file to apply the changes.

CHAPTER 18. SCANNING PODS FOR VULNERABILITIES



IMPORTANT

The Red Hat Quay Container Security Operator has been deprecated and is planned for removal in a future release of OpenShift Container Platform. The official replacement product of the Red Hat Quay Container Security Operator is Red Hat Advanced Cluster Security for Kubernetes.

Using the Red Hat Quay Container Security Operator, you can access vulnerability scan results from the OpenShift Container Platform web console for container images used in active pods on the cluster.

The Red Hat Quay Container Security Operator:

- Watches containers associated with pods on all or specified namespaces
- Queries the container registry where the containers came from for vulnerability information, provided an image's registry is running image scanning (such as [Quay.io](#) or a [Red Hat Quay](#) registry with Clair scanning)
- Exposes vulnerabilities via the **ImageManifestVuln** object in the Kubernetes API

Using the instructions here, the Red Hat Quay Container Security Operator is installed in the **openshift-operators** namespace, so it is available to all namespaces on your OpenShift Container Platform cluster.

18.1. INSTALLING THE RED HAT QUAY CONTAINER SECURITY OPERATOR

You can install the Red Hat Quay Container Security Operator from the OpenShift Container Platform web console OperatorHub, or by using the CLI.

Prerequisites

- You have installed the **oc** CLI.
- You have access to the web console as a user with **cluster-admin** privileges.
- You have containers that come from a Red Hat Quay or Quay.io registry running on your cluster.

Procedure

1. You can install the Red Hat Quay Container Security Operator by using the OpenShift Container Platform web console:
 - a. On the web console, navigate to **Ecosystem → Software Catalog** and select **Security**.
 - b. Select the **Red Hat Quay Container Security Operator** Operator, and then select **Install**.
 - c. On the **Red Hat Quay Container Security Operator** page, select **Install**. **Update channel**, **Installation mode**, and **Update approval** are selected automatically. The **Installed Namespace** field defaults to **openshift-operators**. You can adjust these settings as needed.

- d. Select **Install**. The **Red Hat Quay Container Security Operator** is displayed after a few moments on the **Installed Operators** page.
 - e. Optional: You can add custom certificates to the Red Hat Quay Container Security Operator. For example, create a certificate named **quay.crt** in the current directory. Then, run the following command to add the custom certificate to the Red Hat Quay Container Security Operator:


```
$ oc create secret generic container-security-operator-extra-certs --from-file=quay.crt -n openshift-operators
```
 - f. Optional: If you added a custom certificate, restart the Red Hat Quay Container Security Operator pod for the new certificates to take effect.
2. Alternatively, you can install the Red Hat Quay Container Security Operator by using the CLI:
 - a. Retrieve the latest version of the Container Security Operator and its channel by entering the following command:

```
$ oc get packagemanifests container-security-operator \
-o jsonpath='{range .status.channels[*]}{@.currentCSV} {@.name}{"\n"}{end}' \
| awk '{print "STARTING_CSV=" $1 " CHANNEL=" $2 }' \
| sort -Vr \
| head -1
```

Example output

```
STARTING_CSV=container-security-operator.v3.8.9 CHANNEL=stable-3.8
```

- b. Using the output from the previous command, create a **Subscription** custom resource for the Red Hat Quay Container Security Operator and save it as **container-security-operator.yaml**. For example:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: container-security-operator
  namespace: openshift-operators
spec:
  channel: ${CHANNEL}
  installPlanApproval: Automatic
  name: container-security-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  startingCSV: ${STARTING_CSV}
```

where:

spec.channel

Specifies the values you obtained in the previous step for the **spec.channel**.

spec.startingCSV

Specifies the value you obtained in the previous step for the **spec.startingCSV** parameter.

- c. Enter the following command to apply the configuration:

```
$ oc apply -f container-security-operator.yaml
```

Example output

```
subscription.operators.coreos.com/container-security-operator created
```

18.2. USING THE RED HAT QUAY CONTAINER SECURITY OPERATOR

You can use the Red Hat Quay Container Security Operator to access vulnerability scan results from the OpenShift Container Platform web console.

The following procedure shows you how to use the Red Hat Quay Container Security Operator.

Prerequisites

- You have installed the Red Hat Quay Container Security Operator.

Procedure

- On the OpenShift Container Platform web console, navigate to **Home → Overview**. Under the **Status** section, **Image Vulnerabilities** provides the number of vulnerabilities found.
- Click **Image Vulnerabilities** to reveal the **Image Vulnerabilities breakdown** tab, which details the severity of the vulnerabilities, whether the vulnerabilities can be fixed, and the total number of vulnerabilities.
- You can address detected vulnerabilities in one of two ways:
 - Select a link under the **Vulnerabilities** section. This takes you to the container registry that the container came from, where you can see information about the vulnerability.
 - Select the **namespace** link. This takes you to the **Image Manifest Vulnerabilities** page, where you can see the name of the selected image and all of the namespaces where that image is running.
- After you have learned what images are vulnerable, how to fix those vulnerabilities, and the namespaces that the images are being run in, you can improve security by performing the following actions:
 - Alert anyone in your organization who is running the image and request that they correct the vulnerability.
 - Stop the images from running by deleting the deployment or other object that started the pod that the image is in.



NOTE

If you delete the pod, it might take several minutes for the vulnerability information to reset on the dashboard.

18.3. QUERYING IMAGE VULNERABILITIES FROM THE CLI

You can display information about vulnerabilities detected by the Red Hat Quay Container Security Operator by using the **oc** command.

Prerequisites

- You have installed the Red Hat Quay Container Security Operator on your OpenShift Container Platform instance.

Procedure

1. Enter the following command to query for detected container image vulnerabilities:

```
$ oc get vuln --all-namespaces
```

Example output

```
NAMESPACE  NAME                AGE
default    sha256.ca90...      6m56s
skynet     sha256.ca90...      9m37s
```

2. To display details for a particular vulnerability, append the vulnerability name and its namespace to the **oc describe** command. The following example shows an active container whose image includes an RPM package with a vulnerability:

```
$ oc describe vuln --namespace mynamespace sha256.ac50e3752...
```

Example output

```
Name:      sha256.ac50e3752...
Namespace: quay-enterprise
...
Spec:
  Features:
    Name:      nss-util
    Namespace Name: centos:7
    Version:    3.44.0-3.el7
    Versionformat: rpm
    Vulnerabilities:
      Description: Network Security Services (NSS) is a set of libraries...
```

18.4. UNINSTALLING THE RED HAT QUAY CONTAINER SECURITY OPERATOR

To uninstall the Container Security Operator, you must uninstall the Operator and delete the **imagemanifestvulns.secscan.quay.redhat.com** custom resource definition (CRD).

Procedure

1. On the OpenShift Container Platform web console, click **Ecosystem → Installed Operators**.

2. Click the Options menu  of the Container Security Operator.

3. Click **Uninstall Operator**.

4. Confirm your decision by clicking **Uninstall** in the popup window.

5. Use the CLI to delete the **imagemanifestvulns.secscan.quay.redhat.com** CRD.

- a. Remove the **imagemanifestvulns.secscan.quay.redhat.com** custom resource definition by entering the following command:

```
$ oc delete customresourcedefinition imagemanifestvulns.secscan.quay.redhat.com
```

Example output

```
customresourcedefinition.apiextensions.k8s.io  
"imagemanifestvulns.secscan.quay.redhat.com" deleted
```

18.5. ADDITIONAL RESOURCES

- [Quay.io container registry](#)
- [Red Hat Quay](#)

CHAPTER 19. NETWORK-BOUND DISK ENCRYPTION (NBDE)

19.1. ABOUT DISK ENCRYPTION TECHNOLOGY

Network-Bound Disk Encryption (NBDE) allows you to encrypt root volumes of hard drives on physical and virtual machines without having to manually enter a password when restarting machines.

19.1.1. Disk encryption technology comparison

To understand the merits of Network-Bound Disk Encryption (NBDE) for securing data at rest on edge servers, compare key escrow and TPM disk encryption without Clevis to NBDE on systems running Red Hat Enterprise Linux (RHEL).

The following table presents some tradeoffs to consider around the threat model and the complexity of each encryption solution.

Scenario	Key escrow	TPM disk encryption (without Clevis)	NBDE
Protects against single-disk theft	X	X	X
Protects against entire-server theft	X		X
Systems can reboot independently from the network		X	
No periodic rekeying		X	
Key is never transmitted over a network		X	X
Supported by OpenShift		X	X

19.1.1.1. Key escrow

Key escrow is the traditional system for storing cryptographic keys. The key server on the network stores the encryption key for a node with an encrypted boot disk and returns it when queried. The complexities around key management, transport encryption, and authentication do not make this a reasonable choice for boot disk encryption.

Although available in Red Hat Enterprise Linux (RHEL), key escrow-based disk encryption setup and management is a manual process and not suited to OpenShift Container Platform automation operations, including automated addition of nodes, and currently not supported by OpenShift Container Platform.

19.1.1.2. TPM encryption

Trusted Platform Module (TPM) disk encryption is best suited for data centers or installations in remote

protected locations. Full disk encryption utilities such as dm-crypt and BitLocker encrypt disks with a TPM bind key, and then store the TPM bind key in the TPM, which is attached to the motherboard of the node. The main benefit of this method is that there is no external dependency, and the node is able to decrypt its own disks at boot time without any external interaction.

TPM disk encryption protects against decryption of data if the disk is stolen from the node and analyzed externally. However, for insecure locations this may not be sufficient. For example, if an attacker steals the entire node, the attacker can intercept the data when powering on the node, because the node decrypts its own disks. This applies to nodes with physical TPM2 chips as well as virtual machines with Virtual Trusted Platform Module (VTPM) access.

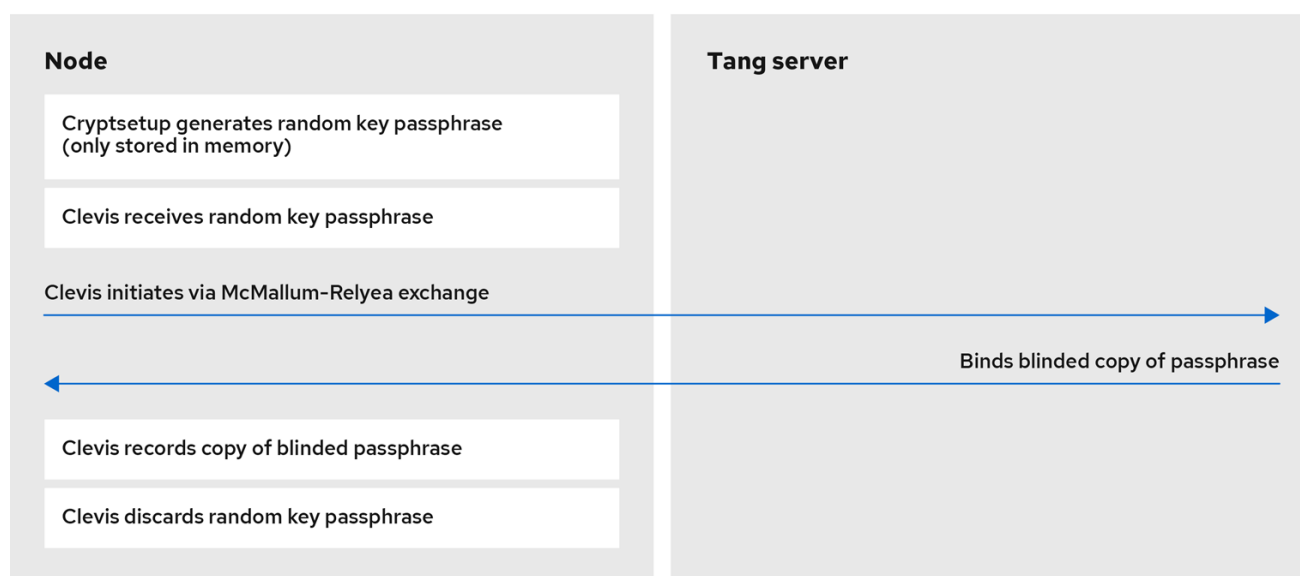
19.1.1.3. Network-Bound Disk Encryption (NBDE)

Network-Bound Disk Encryption (NBDE) effectively ties the encryption key to an external server or set of servers in a secure and anonymous way across the network. This is not a key escrow, in that the nodes do not store the encryption key or transfer it over the network, but otherwise behaves in a similar fashion.

Clevis and Tang are generic client and server components that provide network-bound encryption. Red Hat Enterprise Linux CoreOS (RHCOS) uses these components in conjunction with Linux Unified Key Setup-on-disk-format (LUKS) to encrypt and decrypt root and non-root storage volumes to accomplish Network-Bound Disk Encryption.

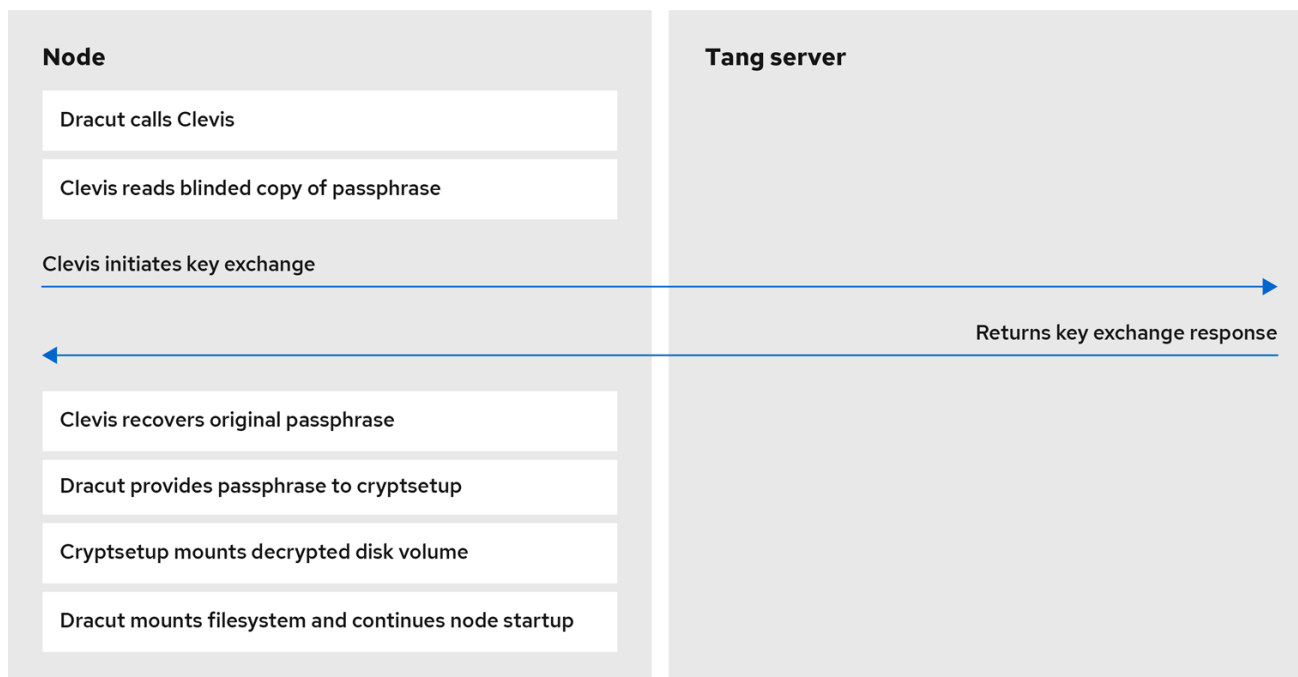
When a node starts, it attempts to contact a predefined set of Tang servers by performing a cryptographic handshake. If it can reach the required number of Tang servers, the node can construct its disk decryption key and unlock the disks to continue booting. If the node cannot access a Tang server due to a network outage or server unavailability, the node cannot boot and continues retrying indefinitely until the Tang servers become available again. Because the key is effectively tied to the node's presence in a network, an attacker attempting to gain access to the data at rest would need to obtain both the disks on the node, and network access to the Tang server as well.

The following figure illustrates the deployment model for NBDE.



179_OpenShift_0821

The following figure illustrates NBDE behavior during a reboot.



179_OpenShift_0821

19.1.1.4. Secret sharing encryption

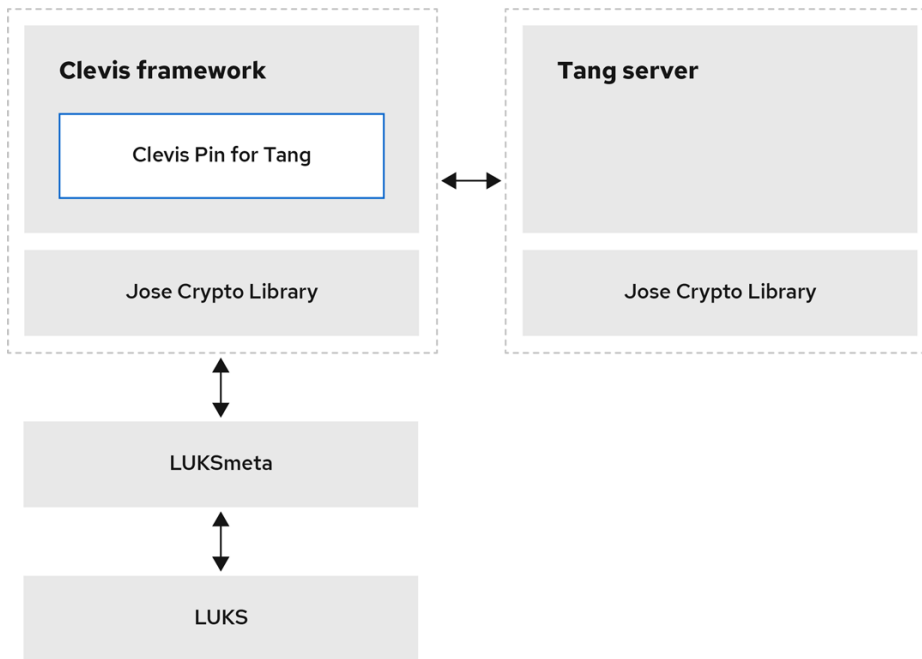
Shamir's secret sharing (sss) is a cryptographic algorithm to securely divide up, distribute, and re-assemble keys. Using this algorithm, OpenShift Container Platform can support more complicated mixtures of key protection.

When you configure a cluster node to use multiple Tang servers, OpenShift Container Platform uses sss to set up a decryption policy that will succeed if at least one of the specified servers is available. You can create layers for additional security. For example, you can define a policy where OpenShift Container Platform requires both the TPM and one of the given list of Tang servers to decrypt the disk.

19.1.2. Tang server disk encryption

The following components and technologies implement Network-Bound Disk Encryption (NBDE).

Figure 19.1. NBDE scheme when using a LUKS1-encrypted volume. The luksmeta package is not used for LUKS2 volumes.



179_OpenShift_0821

Tang is a server for binding data to network presence. It makes a node containing the data available when the node is bound to a certain secure network. Tang is stateless and does not require Transport Layer Security (TLS) or authentication. Unlike escrow-based solutions, where the key server stores all encryption keys and has knowledge of every encryption key, Tang never interacts with any node keys, so it never gains any identifying information from the node.

Clevis is a pluggable framework for automated decryption that provides automated unlocking of Linux Unified Key Setup-on-disk-format (LUKS) volumes. The Clevis package runs on the node and provides the client side of the feature.

A *Clevis pin* is a plugin into the Clevis framework. There are three pin types:

TPM2

Binds the disk encryption to the TPM2.

Tang

Binds the disk encryption to a Tang server to enable NBDE.

Shamir's secret sharing (sss)

Allows more complex combinations of other pins. It allows more nuanced policies such as the following:

- Must be able to reach one of these three Tang servers
- Must be able to reach three of these five Tang servers
- Must be able to reach the TPM2 AND at least one of these three Tang servers

19.1.3. Tang server location planning

When planning your Tang server environment, consider the physical and network locations of the Tang servers.

Physical location

The geographic location of the Tang servers is relatively unimportant, as long as they are suitably secured from unauthorized access or theft and offer the required availability and accessibility to run a critical service.

Nodes with Clevis clients do not require local Tang servers as long as the Tang servers are available at all times. Disaster recovery requires both redundant power and redundant network connectivity to Tang servers regardless of their location.

Network location

Any node with network access to the Tang servers can decrypt their own disk partitions, or any other disks encrypted by the same Tang servers.

Select network locations for the Tang servers that ensure the presence or absence of network connectivity from a given host allows for permission to decrypt. For example, firewall protections might be in place to prohibit access from any type of guest or public network, or any network jack located in an unsecured area of the building.

Additionally, maintain network segregation between production and development networks. This assists in defining appropriate network locations and adds an additional layer of security.

Do not deploy Tang servers on the same resource, for example, the same **rolebindings.rbac.authorization.k8s.io** cluster, that they are responsible for unlocking. However, a cluster of Tang servers and other security resources can be a useful configuration to enable support of multiple additional clusters and cluster resources.

19.1.4. Tang server sizing requirements

The requirements around availability, network, and physical location drive the decision of how many Tang servers to use, rather than any concern over server capacity.

Tang servers do not maintain the state of data encrypted using Tang resources. Tang servers are either fully independent or share only their key material, which enables them to scale well.

There are two ways Tang servers handle key material:

- Multiple Tang servers share key material:
 - You must load balance Tang servers sharing keys behind the same URL. The configuration can be as simple as round-robin DNS, or you can use physical load balancers.
 - You can scale from a single Tang server to multiple Tang servers. Scaling Tang servers does not require rekeying or client reconfiguration on the node when the Tang servers share key material and the same URL.
 - Client node setup and key rotation only requires one Tang server.
- Multiple Tang servers generate their own key material:
 - You can configure multiple Tang servers at installation time.
 - You can scale an individual Tang server behind a load balancer.
 - All Tang servers must be available during client node setup or key rotation.

- When a client node boots using the default configuration, the Clevis client contacts all Tang servers. Only n Tang servers must be online to proceed with decryption. The default value for n is 1.
- Red Hat does not support postinstallation configuration that changes the behavior of the Tang servers.

19.1.5. Logging considerations

Centralized logging of Tang traffic is advantageous because it might allow you to detect such things as unexpected decryption requests. For example:

- A node requesting decryption of a passphrase that does not correspond to its boot sequence
- A node requesting decryption outside of a known maintenance activity, such as cycling keys

19.2. TANG SERVER INSTALLATION CONSIDERATIONS

Network-Bound Disk Encryption (NBDE) must be enabled when a cluster node is installed. However, you can change the disk encryption policy at any time after it was initialized at installation.

19.2.1. Installation scenarios

Consider the following recommendations when planning Tang server installations:

- Small environments can use a single set of key material, even when using multiple Tang servers:
 - Key rotations are easier.
 - Tang servers can scale easily to permit high availability.
- Large environments can benefit from multiple sets of key material:
 - Physically diverse installations do not require the copying and synchronizing of key material between geographic regions.
 - Key rotations are more complex in large environments.
 - Node installation and rekeying require network connectivity to all Tang servers.
 - A small increase in network traffic can occur due to a booting node querying all Tang servers during decryption. Note that while only one Clevis client query must succeed, Clevis queries all Tang servers.
- Further complexity:
 - Additional manual reconfiguration can permit the Shamir's secret sharing (sss) of **any N of M servers online** in order to decrypt the disk partition. Decrypting disks in this scenario requires multiple sets of key material, and manual management of Tang servers and nodes with Clevis clients after the initial installation.
- High level recommendations:
 - For a single RAN deployment, a limited set of Tang servers can run in the corresponding domain controller (DC).

- For multiple RAN deployments, you must decide whether to run Tang servers in each corresponding DC or whether a global Tang environment better suits the other needs and requirements of the system.

19.2.2. Installing a Tang server

To deploy one or more Tang servers, you can choose from the following options depending on your scenario:

1. [Deploying a Tang server using the NBDE Tang Server Operator](#)
2. [Deploying a Tang server with SELinux in enforcing mode on RHEL systems](#)
3. [Configuring a Tang server in the RHEL web console](#)
4. [Deploying Tang as a container](#)
5. [Using the nbde_server System Role for setting up multiple Tang servers](#)

19.2.2.1. Compute requirements

The computational requirements for the Tang server are very low. Any typical server grade configuration that you would use to deploy a server into production can provision sufficient compute capacity.

High availability considerations are solely for availability and not additional compute power to satisfy client demands.

19.2.2.2. Automatic start at boot

Due to the sensitive nature of the key material the Tang server uses, you should keep in mind that the overhead of manual intervention during the Tang server's boot sequence can be beneficial.

By default, if a Tang server starts and does not have key material present in the expected local volume, it will create fresh material and serve it. You can avoid this default behavior by either starting with pre-existing key material or aborting the startup and waiting for manual intervention.

19.2.2.3. HTTP versus HTTPS

Traffic to the Tang server can be encrypted (HTTPS) or plain text (HTTP). There are no significant security advantages of encrypting this traffic, and leaving it decrypted removes any complexity or failure conditions related to Transport Layer Security (TLS) certificate checking in the node running a Clevis client.

While it is possible to perform passive monitoring of unencrypted traffic between the node's Clevis client and the Tang server, the ability to use this traffic to determine the key material is at best a future theoretical concern. Any such traffic analysis would require large quantities of captured data. Key rotation would immediately invalidate it. Finally, any threat actor able to perform passive monitoring has already obtained the necessary network access to perform manual connections to the Tang server and can perform the simpler manual decryption of captured Clevis headers.

However, because other network policies in place at the installation site might require traffic encryption regardless of application, consider leaving this decision to the cluster administrator.

Additional resources

- [Configuring automated unlocking of encrypted volumes using policy-based decryption](#)

- [Official Tang server container](#)
- [Encrypting and mirroring disks during installation](#)

19.3. TANG SERVER ENCRYPTION KEY MANAGEMENT

The cryptographic mechanism to recreate the encryption key is based on the *blinded key* stored on the node and the private key of the involved Tang servers.



NOTE

To protect against the possibility of an attacker who has obtained both the Tang server private key and the node's encrypted disk, periodic rekeying is advisable.

You must perform the rekeying operation for every node before you can delete the old key from the Tang server.

The following sections provide procedures for rekeying and deleting old keys.

19.3.1. Backing up keys for a Tang server

The Tang server uses **/usr/libexec/tangd-keygen** to generate new keys and stores them in the **/var/db/tang** directory by default. To recover the Tang server in the event of a failure, back up this directory. The keys are sensitive and because they are able to perform the boot disk decryption of all hosts that have used them, the keys must be protected accordingly.

Procedure

- Copy the backup key from the **/var/db/tang** directory to the temp directory from which you can restore the key.

19.3.2. Recovering keys for a Tang server

You can recover the keys for a Tang server by accessing the keys from a backup.

Procedure

- Restore the key from your backup folder to the **/var/db/tang/** directory.
When the Tang server starts up, it advertises and uses these restored keys.

19.3.3. Rekeying Tang servers

This procedure uses a set of three Tang servers, each with unique keys, as an example.

Using redundant Tang servers reduces the chances of nodes failing to boot automatically.

Rekeying a Tang server, and all associated NBDE-encrypted nodes, is a three-step procedure.

Prerequisites

- A working Network-Bound Disk Encryption (NBDE) installation on one or more nodes.

Procedure

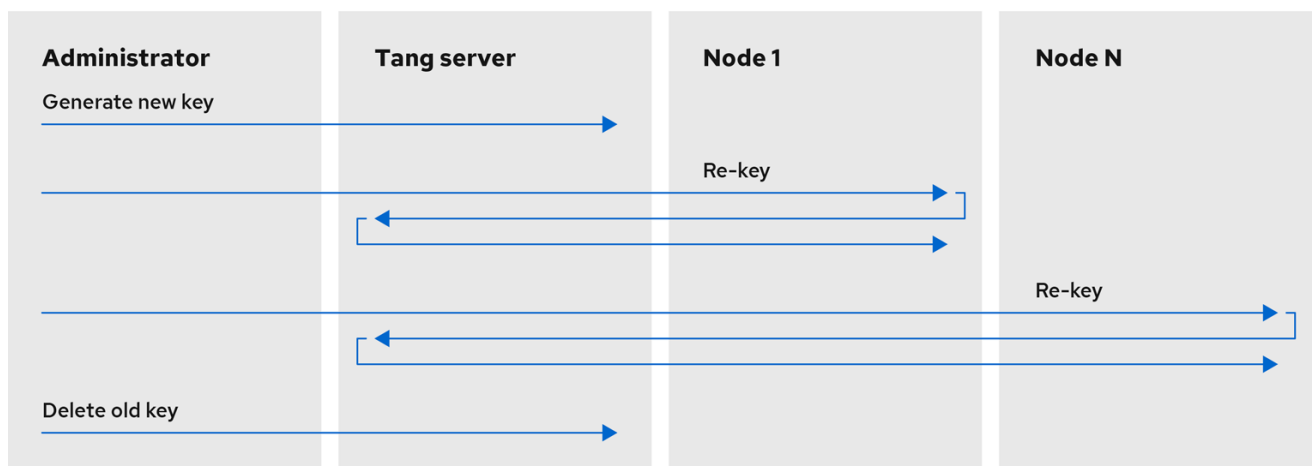
1. Generate a new Tang server key.
2. Rekey all NBDE-encrypted nodes so they use the new key.
3. Delete the old Tang server key.



NOTE

Deleting the old key before all NBDE-encrypted nodes have completed their rekeying causes those nodes to become overly dependent on any other configured Tang servers.

Figure 19.2. Example workflow for rekeying a Tang server



179_OpenShift_0821

19.3.3.1. Generating a new Tang server key

Prerequisites

- A root shell on the Linux machine running the Tang server.
- To facilitate verification of the Tang server key rotation, encrypt a small test file with the old key:

```
# echo plaintext | clevis encrypt tang '{"url":"http://localhost:7500"}' -y >/tmp/encrypted.oldkey
```

- Verify that the encryption succeeded and the file can be decrypted to produce the same string **plaintext**:

```
# clevis decrypt </tmp/encrypted.oldkey
```

Procedure

1. Locate and access the directory that stores the Tang server key. This is usually the **/var/db/tang** directory. Check the currently advertised key thumbprint:

```
# tang-show-keys 7500
```

Example output

```
36AHjNH3NZDSnIONLz1-V4ie6t8
```

2. Enter the Tang server key directory:

```
# cd /var/db/tang/
```

3. List the current Tang server keys:

```
# ls -A1
```

Example output

```
36AHjNH3NZDSnIONLz1-V4ie6t8.jwk  
gJZiNPMLRBnyo_ZKfK4_5SrnhYo.jwk
```

During normal Tang server operations, there are two **.jwk** files in this directory: one for signing and verification, and another for key derivation.

4. Disable advertisement of the old keys:

```
# for key in *.jwk; do \  
  mv -- "$key" ".$key"; \  
done
```

New clients setting up Network-Bound Disk Encryption (NBDE) or requesting keys will no longer see the old keys. Existing clients can still access and use the old keys until they are deleted. The Tang server reads but does not advertise keys stored in UNIX hidden files, which start with the **.** character.

5. Generate a new key:

```
# /usr/libexec/tangd-keygen /var/db/tang
```

6. List the current Tang server keys to verify the old keys are no longer advertised, as they are now hidden files, and new keys are present:

```
# ls -A1
```

Example output

```
.36AHjNH3NZDSnIONLz1-V4ie6t8.jwk  
.gJZiNPMLRBnyo_ZKfK4_5SrnhYo.jwk  
Bp8XjITceWSN_7XFfW7WfJDTomE.jwk  
WOjQYkyK7DxY_T5pMncMO5w0f6E.jwk
```

Tang automatically advertises the new keys.

**NOTE**

More recent Tang server installations include a helper `/usr/libexec/tangd-rotate-keys` directory that takes care of disabling advertisement and generating the new keys simultaneously.

7. If you are running multiple Tang servers behind a load balancer that share the same key material, ensure the changes made here are properly synchronized across the entire set of servers before proceeding.

Verification

1. Verify that the Tang server is advertising the new key, and not advertising the old key:

```
# tang-show-keys 7500
```

Example output

```
WOjQYkyK7DxY_T5pMncMO5w0f6E
```

2. Verify that the old key, while not advertised, is still available to decryption requests:

```
# clevis decrypt </tmp/encrypted.oldkey
```

19.3.3.2. Rekeying all NBDE nodes

You can rekey all of the nodes on a remote cluster by using a **DaemonSet** object without incurring any downtime to the remote cluster.

**NOTE**

If a node loses power during the rekeying, it is possible that it might become unbootable, and must be redeployed via Red Hat Advanced Cluster Management (RHACM) or a GitOps pipeline.

Prerequisites

- **cluster-admin** access to all clusters with Network-Bound Disk Encryption (NBDE) nodes.
- All Tang servers must be accessible to every NBDE node undergoing rekeying, even if the keys of a Tang server have not changed.
- Obtain the Tang server URL and key thumbprint for every Tang server.

Procedure

1. Create a **DaemonSet** object based on the following template. This template sets up three redundant Tang servers, but can be easily adapted to other situations. Change the Tang server URLs and thumbprints in the **NEW_TANG_PIN** environment to suit your environment:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
```

```

name: tang-rekey
namespace: openshift-machine-config-operator
spec:
  selector:
    matchLabels:
      name: tang-rekey
  template:
    metadata:
      labels:
        name: tang-rekey
    spec:
      containers:
        - name: tang-rekey
          image: registry.access.redhat.com/ubi9/ubi-minimal:latest
          imagePullPolicy: IfNotPresent
          command:
            - "/sbin/chroot"
            - "/host"
            - "/bin/bash"
            - "-ec"
          args:
            - |
              rm -f /tmp/rekey-complete || true
              echo "Current tang pin:"
              clevis-luks-list -d $ROOT_DEV -s 1
              echo "Applying new tang pin: $NEW_TANG_PIN"
              clevis-luks-edit -f -d $ROOT_DEV -s 1 -c "$NEW_TANG_PIN"
              echo "Pin applied successfully"
              touch /tmp/rekey-complete
              sleep infinity
      readinessProbe:
        exec:
          command:
            - cat
            - /host/tmp/rekey-complete
        initialDelaySeconds: 30
        periodSeconds: 10
      env:
        - name: ROOT_DEV
          value: /dev/disk/by-partlabel/root
        - name: NEW_TANG_PIN
          value: >-
            {"t":1,"pins":{"tang":[
              {"url":"http://tangserver01:7500","thp":"WOjQYkyK7DxY_T5pMncMO5w0f6E"},
              {"url":"http://tangserver02:7500","thp":"I5Ynh2JefoAO3tNH9TgI4oblaXI"},
              {"url":"http://tangserver03:7500","thp":"38qWZVeDKzCPG9pHLqKzs6k1ons"}
            ]}}
      volumeMounts:
        - name: hostroot
          mountPath: /host
      securityContext:
        privileged: true
      volumes:
        - name: hostroot
          hostPath:
            path: /

```

```
nodeSelector:
  kubernetes.io/os: linux
priorityClassName: system-node-critical
restartPolicy: Always
serviceAccount: machine-config-daemon
serviceAccountName: machine-config-daemon
```

In this case, even though you are rekeying **tangserver01**, you must specify not only the new thumbprint for **tangserver01**, but also the current thumbprints for all other Tang servers. Failure to specify all thumbprints for a rekeying operation opens up the opportunity for a man-in-the-middle attack.

- To distribute the daemon set to every cluster that must be rekeyed, run the following command:

```
$ oc apply -f tang-rekey.yaml
```

However, to run at scale, wrap the daemon set in an ACM policy. This ACM configuration must contain one policy to deploy the daemon set, a second policy to check that all the daemon set pods are READY, and a placement rule to apply it to the appropriate set of clusters.



NOTE

After validating that the daemon set has successfully rekeyed all servers, delete the daemon set. If you do not delete the daemon set, it must be deleted before the next rekeying operation.

Verification

After you distribute the daemon set, monitor the daemon sets to ensure that the rekeying has completed successfully. The script in the example daemon set terminates with an error if the rekeying failed, and remains in the **CURRENT** state if successful. There is also a readiness probe that marks the pod as **READY** when the rekeying has completed successfully.

- This is an example of the output listing for the daemon set before the rekeying has completed:

```
$ oc get -n openshift-machine-config-operator ds tang-rekey
```

Example output

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
SELECTOR	AGE					
tang-rekey	1	1	0	1	0	kubernetes.io/os=linux 11s

- This is an example of the output listing for the daemon set after the rekeying has completed successfully:

```
$ oc get -n openshift-machine-config-operator ds tang-rekey
```

Example output

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE
SELECTOR	AGE					
tang-rekey	1	1	1	1	1	kubernetes.io/os=linux 13h



Rekeying usually takes a few minutes to complete.



NOTE

If you use ACM policies to distribute the daemon sets to multiple clusters, you must include a compliance policy that checks every daemon set's **READY** count is equal to the **DESIRED** count. In this way, compliance to such a policy demonstrates that all daemon set pods are **READY** and the rekeying has completed successfully. You could also use an ACM search to query all of the daemon sets' states.

19.3.3.3. Troubleshooting temporary rekeying errors for Tang servers

To determine if the error condition from rekeying the Tang servers is temporary, perform the following procedure. Temporary error conditions might include:

- Temporary network outages
- Tang server maintenance

Generally, when these types of temporary error conditions occur, you can wait until the daemon set succeeds in resolving the error or you can delete the daemon set and not try again until the temporary error condition has been resolved.

Procedure

1. Restart the pod that performs the rekeying operation using the normal Kubernetes pod restart policy.
2. If any of the associated Tang servers are unavailable, try rekeying until all the servers are back online.

19.3.3.4. Troubleshooting permanent rekeying errors for Tang servers

If, after rekeying the Tang servers, the **READY** count does not equal the **DESIRED** count after an extended period of time, it might indicate a permanent failure condition. In this case, the following conditions might apply:

- A typographical error in the Tang server URL or thumbprint in the **NEW_TANG_PIN** definition.
- The Tang server is decommissioned or the keys are permanently lost.

Prerequisites

- The commands shown in this procedure can be run on the Tang server or on any Linux system that has network access to the Tang server.

Procedure

1. Validate the Tang server configuration by performing a simple encrypt and decrypt operation on each Tang server's configuration as defined in the daemon set.
This is an example of an encryption and decryption attempt with a bad thumbprint:

```
$ echo "okay" | clevis encrypt tang \
'{"url":"http://tangserver02:7500","thp":"badthumbprint"}' | \
clevis decrypt
```

Example output

```
Unable to fetch advertisement: 'http://tangserver02:7500/adv/badthumbprint'!
```

This is an example of an encryption and decryption attempt with a good thumbprint:

```
$ echo "okay" | clevis encrypt tang \
'{"url":"http://tangserver03:7500","thp":"goodthumbprint"}' | \
clevis decrypt
```

Example output

```
okay
```

2. After you identify the root cause, remedy the underlying situation:
 - a. Delete the non-working daemon set.
 - b. Edit the daemon set definition to fix the underlying issue. This might include any of the following actions:
 - Edit a Tang server entry to correct the URL and thumbprint.
 - Remove a Tang server that is no longer in service.
 - Add a new Tang server that is a replacement for a decommissioned server.
3. Distribute the updated daemon set again.



NOTE

When replacing, removing, or adding a Tang server from a configuration, the rekeying operation will succeed as long as at least one original server is still functional, including the server currently being rekeyed. If none of the original Tang servers are functional or can be recovered, recovery of the system is impossible and you must redeploy the affected nodes.

Verification

Check the logs from each pod in the daemon set to determine whether the rekeying completed successfully. If the rekeying is not successful, the logs might indicate the failure condition.

1. Locate the name of the container that was created by the daemon set:

```
$ oc get pods -A | grep tang-rekey
```

Example output

```
openshift-machine-config-operator tang-rekey-7ks6h 1/1 Running 20 (8m39s ago) 89m
```

2. Print the logs from the container. The following log is from a completed successful rekeying operation:

```
$ oc logs tang-rekey-7ks6h
```

Example output

```
Current tang pin:
1: sss '{"t":1,"pins":{"tang":[{"url":"http://10.46.55.192:7500"}, {"url":"http://10.46.55.192:7501"}, {"url":"http://10.46.55.192:7502"}]}}'
Applying new tang pin: {"t":1,"pins":{"tang":[{"url":"http://tangserver01:7500","thp":"WOjQYkyK7DxY_T5pMncMO5w0f6E"}, {"url":"http://tangserver02:7500","thp":"l5Ynh2JefoAO3tNH9Tgl4oblaXI"}, {"url":"http://tangserver03:7500","thp":"38qWZVeDKzCPG9pHLqKzs6k1ons"}]}}
Updating binding...
Binding edited successfully
Pin applied successfully
```

19.3.4. Deleting old Tang server keys

Prerequisites

- A root shell on the Linux machine running the Tang server.

Procedure

1. Locate and access the directory where the Tang server key is stored. This is usually the **/var/db/tang** directory:

```
# cd /var/db/tang/
```

2. List the current Tang server keys, showing the advertised and unadvertised keys:

```
# ls -A1
```

Example output

```
.36AHjNH3NZDSnlONLz1-V4ie6t8.jwk
.gJZiNPMLRBnyo_ZKfK4_5SrnhYo.jwk
Bp8XjITceWSN_7XFfW7WfJDTomE.jwk
WOjQYkyK7DxY_T5pMncMO5w0f6E.jwk
```

3. Delete the old keys:

```
# rm *.jwk
```

4. List the current Tang server keys to verify the unadvertised keys are no longer present:

```
# ls -A1
```


Example output

```
Bp8XjITceWSN_7XFfW7WfJDTomE.jwk
WOjQYkyK7DxY_T5pMncMO5w0f6E.jwk
```

Verification

At this point, the server still advertises the new keys, but an attempt to decrypt based on the old key will fail.

1. Query the Tang server for the current advertised key thumbprints:

```
# tang-show-keys 7500
```

Example output

```
WOjQYkyK7DxY_T5pMncMO5w0f6E
```

2. Decrypt the test file created earlier to verify decryption against the old keys fails:

```
# clevis decrypt </tmp/encryptValidation
```

Example output

```
Error communicating with the server!
```

If you are running multiple Tang servers behind a load balancer that share the same key material, ensure the changes made are properly synchronized across the entire set of servers before proceeding.

19.4. DISASTER RECOVERY CONSIDERATIONS

This section describes several potential disaster situations and the procedures to respond to each of them. Additional situations will be added here as they are discovered or presumed likely to be possible.

19.4.1. Loss of a client machine

The loss of a cluster node that uses the Tang server to decrypt its disk partition is *not* a disaster. Whether the machine was stolen, suffered hardware failure, or another loss scenario is not important: the disks are encrypted and considered unrecoverable.

However, in the event of theft, a precautionary rotation of the Tang server's keys and rekeying of all remaining nodes would be prudent to ensure the disks remain unrecoverable even in the event the thieves subsequently gain access to the Tang servers.

To recover from this situation, either reinstall or replace the node.

19.4.2. Planning for a loss of client network connectivity

The loss of network connectivity to an individual node will cause it to become unable to boot in an unattended fashion.

If you are planning work that might cause a loss of network connectivity, you can reveal the passphrase for an onsite technician to use manually, and then rotate the keys afterwards to invalidate it:

Procedure

1. Before the network becomes unavailable, show the password used in the first slot **-s 1** of device **/dev/vda2** with this command:

```
$ sudo clevis luks pass -d /dev/vda2 -s 1
```

2. Invalidate that value and regenerate a new random boot-time passphrase with this command:

```
$ sudo clevis luks regen -d /dev/vda2 -s 1
```

19.4.3. Unexpected loss of network connectivity

If the network disruption is unexpected and a node reboots, consider the following scenarios:

- If any nodes are still online, ensure that they do not reboot until network connectivity is restored. This is not applicable for single-node clusters.
- The node will remain offline until such time that either network connectivity is restored, or a pre-established passphrase is entered manually at the console. In exceptional circumstances, network administrators might be able to reconfigure network segments to reestablish access, but this is counter to the intent of NBDE, which is that lack of network access means lack of ability to boot.
- The lack of network access at the node can reasonably be expected to impact that node's ability to function as well as its ability to boot. Even if the node were to boot via manual intervention, the lack of network access would make it effectively useless.

19.4.4. Recovering network connectivity manually

A somewhat complex and manually intensive process is also available to the onsite technician for network recovery.

Procedure

1. The onsite technician extracts the Clevis header from the hard disks. Depending on BIOS lockdown, this might involve removing the disks and installing them in a lab machine.
2. The onsite technician transmits the Clevis headers to a colleague with legitimate access to the Tang network who then performs the decryption.
3. Due to the necessity of limited access to the Tang network, the technician should not be able to access that network via VPN or other remote connectivity. Similarly, the technician cannot patch the remote server through to this network in order to decrypt the disks automatically.
4. The technician reinstalls the disk and manually enters the plain text passphrase provided by their colleague.
5. The machine successfully starts even without direct access to the Tang servers. Note that the transmission of the key material from the install site to another site with network access must be done carefully.
6. When network connectivity is restored, the technician rotates the encryption keys.

19.4.5. Emergency recovery of network connectivity

If you are unable to recover network connectivity manually, consider the following steps. Be aware that these steps are discouraged if other methods to recover network connectivity are available.

- This method must only be performed by a highly trusted technician.
- Taking the Tang server's key material to the remote site is considered to be a breach of the key material and all servers must be rekeyed and re-encrypted.
- This method must be used in extreme cases only, or as a proof of concept recovery method to demonstrate its viability.
- Equally extreme, but theoretically possible, is to power the server in question with an Uninterruptible Power Supply (UPS), transport the server to a location with network connectivity to boot and decrypt the disks, and then restore the server at the original location on battery power to continue operation.
- If you want to use a backup manual passphrase, you must create it before the failure situation occurs.
- Just as attack scenarios become more complex with TPM and Tang compared to a stand-alone Tang installation, so emergency disaster recovery processes are also made more complex if leveraging the same method.

19.4.6. Loss of a network segment

The loss of a network segment, making a Tang server temporarily unavailable, has the following consequences:

- OpenShift Container Platform nodes continue to boot as normal, provided other servers are available.
- New nodes cannot establish their encryption keys until the network segment is restored. In this case, ensure connectivity to remote geographic locations for the purposes of high availability and redundancy. This is because when you are installing a new node or rekeying an existing node, all of the Tang servers you are referencing in that operation must be available.

A hybrid model for a vastly diverse network, such as five geographic regions in which each client is connected to the closest three clients is worth investigating.

In this scenario, new clients are able to establish their encryption keys with the subset of servers that are reachable. For example, in the set of **tang1**, **tang2** and **tang3** servers, if **tang2** becomes unreachable clients can still establish their encryption keys with **tang1** and **tang3**, and at a later time re-establish with the full set. This can involve either a manual intervention or a more complex automation to be available.

19.4.7. Loss of a Tang server

The loss of an individual Tang server within a load balanced set of servers with identical key material is completely transparent to the clients.

The temporary failure of all Tang servers associated with the same URL, that is, the entire load balanced set, can be considered the same as the loss of a network segment. Existing clients have the ability to decrypt their disk partitions so long as another preconfigured Tang server is available. New clients cannot enroll until at least one of these servers comes back online.

You can mitigate the physical loss of a Tang server by either reinstalling the server or restoring the server from backups. Ensure that the backup and restore processes of the key material is adequately protected from unauthorized access.

19.4.8. Rekeying compromised key material

If key material is potentially exposed to unauthorized third parties, such as through the physical theft of a Tang server or associated data, immediately rotate the keys.

Procedure

1. Rekey any Tang server holding the affected material.
2. Rekey all clients using the Tang server.
3. Destroy the original key material.
4. Scrutinize any incidents that result in unintended exposure of the master encryption key. If possible, take compromised nodes offline and re-encrypt their disks.

TIP

Reformatting and reinstalling on the same physical hardware, although slow, is easy to automate and test.